

테스트타임 학습과 신경 메모리: Google Titans 계보 6 부작 과 대규모 Scaling 청사진

Seungho (Jimmy) Lee

2026-07

Contents

서문	10
누구를 위한책인가	10
왜 이 여섯편인가	10
어떻게 읽는가	11
이 책의 척추: pair thesis 예고	11
정직성 계약	11
표기와 3 층 구분	12
 제 1 부 — Inference 엔지니어를 위한 배경	13
Part I — inference 엔지니어를 위한배경	14
ch01. Orientation: inference 세계와 learning 세계의 Rosetta Stone	15
1.1 두 세계: weights 가 얼어 있는 세계와 움직이는세계	15
1.2 모든 sequence 모델은 이미 memory 였다	16
1.3 Rosetta-Stone 사전	16
1.4 두 개의 loop: 누가 무엇을학습하는가	18
1.5 여섯 편의지도	18
1.6 Worked micro-example: 2×2 memory 를 손으로 굴러보기	19
1.7 Systems bridge: 첫 back-of-envelope	20
요약	20
자가 점검체크리스트	20
다음 장으로	21
ch02. Training as a system: backprop 과 optimizer 를 (state, update, cost) 객체로 읽는다	22
2.1 파라미터가 움직이는 세계: training loop 의해부	23
2.2 gradient: 스칼라 하나가 모든 parameter 에 내리는지시	23
2.3 backward pass: 반복되는 VJP, 그리고 $dW = \delta x^\top$ 라는 shape	24
2.4 loss surface: 왜 η 하나로는부족한가	25
2.5 optimizer = (state, update, cost) 객체	25
2.5.1 SGD: stateless 한기준점	26
2.5.2 momentum: 첫 번째 state, 그리고 linear recurrence 라는정체	26
2.5.3 weight decay: $(1 - \eta\lambda)$ 라는 retention 의원형	27
2.5.4 Adam 과 AdamW: 좌표별 보폭의표준	27
2.5.5 AdaGrad 와 Shampoo: preconditioning 이라는일반화	28
2.5.6 Muon: update 의 직교화, 그리고 test-time compute 로의예고	28
2.6 batch 축 vs sequence 축 — 이 주제 최대의 혼동지점	29
2.7 Worked micro-example: 2×2 memory 에손으로 돌리는 네 개의 optimizer	30
2.8 Systems bridge: optimizer 한 step 의 비용회계	31
요약	32

자가 점검체크리스트	32
다음 장으로	33
ch03. Online learning: OGD, regret, FTRL	34
3.1 Online protocol: 독자가 이미 돌리고 있는 loop	34
3.2 OGD 와 regret: stream 압축의 품질언어	35
3.3 FTL: 왜 “지금까지의 최적해” 로 점프하면 안되는가	36
3.4 FTRL: regularizer 가 memory 를안정화한다	37
3.5 Mirror descent 와 Bregman divergence	38
3.6 Loss geometry: ℓ_2 , ℓ_1 , ℓ_p , Huber	39
3.7 Systems bridge: 학습된 eviction policy 로서의 retention	39
3.8 Worked micro-example: FTL 의 진동, FTRL 의 안정화, OGD 와의동치	40
요약	41
자가 점검체크리스트	41
다음 장으로	42
ch04. Meta-learning 과 bilevel optimization: inner loop vs outer loop	43
4.1 두 개의 최적화, 두 개의시간축	44
4.2 hypergradient: gradient 를 통과하는 gradient	44
4.3 learning-to-learn 계보: 무엇을 meta-변수로삼는가	45
4.4 이 라인의 bilevel 구조: outer loop 는 정확히 무엇을배우는가	47
4.5 Attention 은 이미 GD 를 하고 있었다: ICL 과 mesa-optimization	48
4.6 Worked micro-example: 손으로 도는 두 개의 loop	48
4.7 Systems bridge: 정적 graph, truncation, 그리고 serving 의분리	49
요약	50
자가 점검체크리스트	50
다음 장으로	51
ch05. Associative memory: Hopfield 에서 delta rule 까지	52
5.1 key $\rightarrow$ value 연상: 독자가 이미 매일 서빙하는연산	52
5.2 outer-product memory: 행렬 하나에 여러 쌍을 겹쳐쓰기	53
5.3 Hopfield network: energy 로 본 memory 와 최초의 capacity 상수	54
5.4 dense/modern Hopfield: energy 를 바꾸면 capacity 가 바뀐다 — Atlas 의 이론 적골격	55
5.5 delta rule: append 에서 error-correcting overwrite 로	58
5.6 attention 은 압축하지 않는 associative memory 다	59
5.7 Worked micro-example: crosstalk 을 손으로 계산하고 delta rule 로고치기	60
5.8 Systems bridge: $d \times d$ state 의 “유효 cache 크기”	61
요약	62
자가 점검체크리스트	63
다음 장으로	63
ch06. Linear attention 과 fast-weight programming: DeltaNet, Gated DeltaNet, Longhorn, RWKV-7	64
6.1 Kernel trick: KV cache 를 $d_v \times d_k$ 행렬로압축한다	64
6.2 Fast-weight programming: 1992 년의 발명, 2021 년의재발견	65
6.3 Gate 의 도입: RetNet 과 GLA — 학습된 eviction	67
6.4 DeltaNet: append 에서 overwrite 로 — write 연산의교정	67
6.5 Longhorn: update rule 을 발명하지 말고, 문제를풀어라	68
6.6 RWKV-7: generalized delta rule — gate 를 채널별벡터로	70
6.7 Expressivity 사이드바: state 의 착각과 음의고유값	72
6.8 Worked micro-example: $d = 2$ 에서 세 가지 write 를 손으로돌린다	73

6.9 Systems bridge: 당신의 roofline 위에올려놓기	74
요약	75
자가 점검체크리스트	76
다음 장으로	76
ch07. SSM 계보: S4 → Mamba → Mamba-2, 그리고 SSD duality	77
7.1 continuous SSM 과 discretization: 모든 gate 의기원	77
7.2 Mamba: selectivity — gate 가 input 의 함수가되다	78
7.3 Mamba-2 와 SSD: 한 recurrence, 두 얼굴, 세 계산경로	80
7.4 계보 정리: 세 세대의 gate, 그리고 이 라인의접속점	83
7.5 Worked micro-example: 한 recurrence 를 세 경로로계산하기	84
7.6 Systems bridge: scan 에서 GEMM 으로 — 9 장리허설	85
요약	86
자가 점검체크리스트	86
다음 장으로	86
ch08. TTT lineage: test-time adaptation 에서 TTT-Linear/TTT-MLP 와 dual form	87
까지	87
8.1 전사: inference 중의 weight update 는 이미 검증된기법이였다	87
8.2 TTT layer: hidden state 가 곧 model 이다	88
8.3 TTT-Linear 는 delta rule 이다	89
8.4 Dual form: per-token GD 를 chunk 단위 GEMM 으로	90
8.5 Outer loop: 이 layer 자체는 누가훈련하는가	91
8.6 스케일 증거와파생	93
8.7 여섯 논문이 TTT 에서 바꾸는 것 — Part II 의지도	93
8.8 Worked micro-example: $d = 2$ 손계산 — chunk 가 함수를바꾼다	94
8.9 Systems bridge: decode 안의 backward pass	95
요약	96
자가 점검체크리스트	97
다음 장으로	97
ch09. Chunkwise-parallel training: 하나의 일반 scheme, 네 개의인스턴스	98
9.1 문제 설정: training 안에 들어온 decode loop	98
9.2 일반 scheme: 경계에서 무엇을얼리는가	99
9.3 인스턴스 1 — GLA / RetNet / Mamba-2: 대각 transition, exact 한 folding	100
9.4 인스턴스 2 — DeltaNet: 비대각 transition 과 WY representation	101
9.5 인스턴스 3 — TTT dual form: anchor 라는 선택, 함수라는대가	102
9.6 인스턴스 4 — Titans: retention folding + momentum scan	102
9.7 Semantic hyperparameter 로서의 C : 두지위의총정리	103
9.8 Systems frontier: 그래도 deep memory 는 느리다 — TNT 의 처방 (골격)	105
9.9 Worked micro-example: 같은 두 token, 두 개의병렬화	105
9.10 Systems bridge: roofline 위의 C	106
요약	107
자가 점검체크리스트	107
다음 장으로	108
ch10. 통합 systems bridge: cost model cheat sheet, roofline vs chunk size, glossary	109
10.1 완성된 Rosetta-Stone 사전	109
10.2 Cost-model cheat sheet	110
10.3 KV cache vs fast weights: memory-budget 산수	112
10.4 Prefill/decode 비대칭과 serving engine: backward pass 가들어온다	112

10.5 FlashAttention tiling vs chunkwise training: roofline 위의 chunk 크기 . . .	113
10.6 Worked micro-example: chunk 하나의 arithmetic intensity 손계산	114
10.7 Glossary: Part II 에 등장하는 training 용어의 systems 한 줄사전	115
요약	117
자가 점검체크리스트	117
다음 장으로	118
ch11. Continual learning, complementary learning systems, distillation, RL-lite	119
11.1 Catastrophic forgetting: 쓰기가 곧 파괴인세계	119
11.2 고전 완화책 3 종: replay, EWC, parameter isolation	120
11.3 Complementary learning systems: 두 학습자의분업	122
11.4 Online vs offline consolidation, 그리고 update-frequency spectrum	123
11.5 Knowledge distillation: soft target 에서 on-policy GKD 까지	124
11.6 RL-lite: REINFORCE, imitation, 그리고 post-training 의모양	124
11.7 Worked micro-example: forgetting, EWC, REINFORCE 를손으로	126
11.8 Systems bridge: wake/sleep 은 serving fleet 의 lifecycle 이다	127
요약	127
자가 점검체크리스트	128
다음 장으로	128
제 2 부 — 여섯 편의 논문	129
Part II — 여섯 편의 정밀독해	130
ch12. Titans: Learning to Memorize at Test Time	131
12.1 Bridge-in: TTT 가 남긴문제	131
12.2 문제의식과 논문의 핵심주장	131
12.3 Core mechanism (통일표기)	132
12.3.1 memory 는 weights 다: 상태와 함수의분리	132
12.3.2 momentary surprise 와 기본 update	132
12.3.3 past surprise: momentum 이 sequence layer 안으로들어오다	133
12.3.4 forgetting: weight decay 가 per-token retention 이되다	133
12.3.5 read: 갱신 없는 forward	134
12.3.6 왜 deep memory 인가	134
12.3.7 persistent memory	134
12.3.8 세 가지 합성: MAC / MAG / MAL, 그리고 LMM 단독	134
12.3.9 특수 사례 회수: 이 update 는 기존 계보 전부를포함한다	136
12.3.10 표기대응표	136
12.4 Outer-loop training vs inner-loop test-time learning	138
12.5 Concept ledger delta	140
12.6 실험과스케일	141
12.7 Systems/serving 함의	143
12.8 한계와 bridge-out	144
ch13. Miras: It' s All Connected — sequence model 설계 공간의통일이론	146
13.1 Bridge-in: Titans 가 남긴 질문 — “왜 하필 그선택인가”	146
13.2 문제의식과 논문의 핵심주장	146
13.3 Core mechanism (통일표기)	147
13.3.1 Associative memory 의 재정식화와 attentional bias	147
13.3.2 두 개의 optimization 관점: FTRL 과 Learning – Retaining	148
13.3.3 Miras 의 네 축	149

13.3.4 기존 모델의 재유도: 전부가 이 공간의점이다	149
13.3.5 새로운 attentional bias: ℓ_p , Huber, value-shift robustness	151
13.3.6 새로운 retention gate: 정규화의동물원	152
13.3.7 출하된 세 모델: Moneta, Yaad, Memora	153
13.3.8 표기 대응표	154
13.4 Outer-loop training vs inner-loop test-time learning	155
13.5 Concept ledger delta	157
13.6 실험과스케일	158
13.7 Systems/serving 함의	159
13.8 한계와 bridge-out	160
ch14. Atlas: Learning to Optimally Memorize the Context at Test Time	162
14.1 Bridge-in: Miras 가 남긴문제	162
14.2 문제의식과 논문의 핵심주장	162
14.3 Core mechanism (통일표기)	163
14.3.1 Capacity 이론: 고정 크기 memory 는 몇 쌍을저장하는가	163
14.3.2 Omega rule: token 이 아니라 context 를 memorize 한다	164
14.3.3 OmegaNet: rank-c 전이로의일반화	166
14.3.4 Atlas: inner loop 의 Muon — 근사 2 차 memory 관리	167
14.3.5 Transformer 일반화 가족: DLA/SWLA → DeepTransformers → Dot	168
14.3.6 Backbone, 표준 memory, Atlas++	169
14.3.7 표기 대응표	169
14.4 Outer-loop training vs inner-loop test-time learning	170
14.5 Concept ledger delta	172
14.6 실험과스케일	174
14.7 Systems/serving 함의	176
14.8 한계와 bridge-out	177
ch15. TNT: Improving Chunkwise Training for Test-Time Memorization	179
15.1 Bridge-in: 전작에서 남은문제	179
15.2 문제의식: 세 개의 challenge	179
15.3 Core mechanism (통일표기)	180
15.3.1 두 연산으로 정식화된 deep memory	180
15.3.2 chunkwise parallel training 복습	182
15.3.3 Stage 1 — hierarchical memory: global + N local	182
15.3.4 Stage 1 — Q-K Projection: read 를 write 의 domain 으로되돌리기	185
15.3.5 Stage 2 — 더 fine 한 해상도로의 fine-tuning	186
15.3.6 각 부품이 존재하는 이유 (한 줄정리)	186
15.3.7 표기 대응표	187
15.4 Outer-loop training vs inner-loop test-time learning	187
15.5 Concept ledger delta	189
15.6 실험과스케일	190
15.7 Systems/serving 함의	192
15.8 한계와 bridge-out	193
ch16. Nested Learning: deep learning 아키텍처라는 착시, 그리고 Hope	195
16.1 Bridge-in: TNT 가 남긴문제	195
16.2 문제의식과 논문의 핵심주장	195
16.3 Core mechanism (통일표기)	196
16.3.1 대수적 엔진: associative memory 정의와 proximal/FTRL 등가	196
16.3.2 훈련은 surprise 에 대한 memory 다: LSS 와 backprop 의자기참조성	197
16.3.3 update frequency, level, 그리고 NSAM	198

16.3.4 level 간 knowledge transfer: 다섯 가지메커니즘	200
16.3.5 optimizer 재해석: momentum, Adam, AdaGrad, Muon	200
16.3.6 프레임의 생성적 사용: 새 learning rule 들 — Delta Momentum, DMGD, DGD, GGD	201
16.3.7 아키텍처 재해석: 카탈로그의재확인	202
16.3.8 Continuum Memory System (CMS)	203
16.3.9 CMS 를 optimizer 에 적용하면: M3	204
16.3.10 self-modifying Titans	204
16.3.11 Hope block	205
16.3.12 표기대응표	206
16.4 Outer-loop training vs inner-loop test-time learning	207
16.4.1 outer loop: 무엇이 meta-learn 되고, gradient 는 어떻게 inner 를관통하 는가	208
16.4.2 inner loop: inference 에서 무엇이 어떤 rule 로움직이는가	209
16.5 Concept ledger delta	209
16.6 실험과스케일	211
16.7 Systems/serving 함의	213
16.8 한계와 bridge-out	214
ch17. Sleep: Learning to Self-Modify and Consolidate Memories	215
17.1 Bridge-in: [NL] 이 남긴 문제 — consolidation 의절반	215
17.2 문제의식과 논문의 핵심주장	216
17.3 Core mechanism (통일표기)	217
17.3.1 기반 구조: CMS 와 식 (M5) — wake 가 돌리는것	217
17.3.2 sleep 은 언제 오는가: chunk 경계스케줄	218
17.3.3 Stage 1a — parameter expansion: 덮어쓰지 말고키워라	218
17.3.4 Stage 1b — Knowledge Seeding: 위로 가는 distillation	219
17.3.5 Stage 1c — Learning to Imitate: 아는 것과 쓰는 것은다르다	220
17.3.6 Stage 2 — Dreaming: 자기 생성 데이터에 의한 self-modification	221
17.3.7 한 번의 sleep step: 전체절차	221
17.3.8 표기 대응표	222
17.4 Outer-loop training vs inner-loop test-time learning —그리고 세 번째 regime	223
17.5 Concept ledger delta	225
17.6 실험과스케일	226
17.7 Systems/serving 함의	228
17.8 한계와 bridge-out: 라인의 완성형과 남은문제	229
제 3 부 — 원저 기여: Scaling 과 대규모 학습·서빙 청사진	231
Part III — 원저 기여	232
ch18. 여섯 편의 수렴과 Part III 의 기여프로그램	233
18.1 Bridge-in: 초대장에서시작한다	233
18.2 완성형, 여섯 편의텍스트만으로	233
18.3 완성형을 엔지니어의 질문으로: D4 pair thesis	234
18.4 이 책이 돌린 실측 프로그램: 8 개 claim	235
18.5 Part III 로드맵	236
18.6 정직성 계약: exploration-grade	237
요약	237
자가 점검체크리스트	238
다음 장으로	238

ch19. Scaling 분석과 memory-centric 모델의 후보 scaling law	239
19.1 Bridge-in: scaling law 는 왜 두 축이모자란가	239
19.2 여섯 편의 공개 scaling 점을 digitize 한다	240
19.3 방법론: tokenizer 이질성과 setting_group — fit 의 경계가왜하나인가	240
19.4 빠진 축 하나: state-bytes	241
19.5 후보 법칙을 실제로 fit 한다 — E4 정량백본	242
19.6 빠진 축 둘: capacity 를 일급 축으로 (그리고 A3 가 그것을어디에놓는가)	244
19.7 후보 scaling law 의형태	246
19.8 정직 평가: A3 falsification 조건 대 실제판정	247
19.9 systems 합의: sizing 숫자가 둘에서셋으로	249
요약	249
자가 점검체크리스트	250
다음 장으로	250
ch20. 하드웨어 병목 분석: decode roofline 과 새 serving primitive	251
20.1 Bridge-in: crossover 에서 roofline 으로	251
20.2 아키텍처 class 별 decode roofline	252
20.2.1 matrix memory: rank-1 write 가 만드는 roofline 바닥	253
20.2.2 deep memory vs matrix: 연산과트래픽이 함께 md^2 로자란다	253
20.2.3 Atlas 의 세 얼굴: matrix·poly·exp 는 pair thesis 를 한 축에담는다	254
20.2.4 Hope 의 6-memory 블록: 한 token 에 여섯 개의 RMW	254
20.2.5 batch 축을 더한 memory forward: per-request fast-weight 의 batched 실행형태	254
20.3 decode 에 들어온 backward pass: 새 serving primitive	256
20.3.1 fused backward-decode kernel: 설계 수준의요건	256
20.4 NS-5 와 deep-memory 의 FLOP density: PIM 경계선을긋는다	257
20.4.1 NS-5 의 FLOP density: $O(\kappa d^3)$ 와 $C=1$ 에서의 traffic 지배	258
20.5 chunk kernel: 같은 알고리즘이 compute-bound 로 넘어가는곳	258
20.5.1 MFU 분해: 천장 (작동점) $\times$ 포획 (구현)	259
20.6 state placement: 상주는 설계 knob 이고, 경계에서 대역폭이꺾인다	260
20.7 종합: kernel 수준에서 본 pair	262
요약	263
자가 점검체크리스트	264
다음 장으로	264
ch21. Transformer/NVIDIA scaling 시대의 교훈: hardware lottery	265
21.1 Bridge-in: 20 장이 측정한 병목의 역사적원인	265
21.2 attention 이 이긴 진짜 이유: GEMM density	265
21.2.1 GEMM density 는 왜 하필 승리 조건이었나 — systolic array 에서 tensor core 까지	266
21.3 이 라인은 matmul 안에 설계되어있다	266
21.4 그런데 FlashAttention-moment 은 아직 오지않았다	268
21.4.1 왜 momentum·deep memory 는 커널화가 어려운가 (기계적해부)	269
21.5 이 라인의 FlashAttention-moment 이 예고하는것	270
21.6 이 독법의 한계 (정직성각주)	271
요약	271
자가 점검체크리스트	272
다음 장으로	272

ch22. Player-strategy 분석: 누가 TTT scaling 에 어떻게참여하는가	274
22.1 Bridge-in: hardware lottery 에서 player 지도로	274
22.2 Google Research 가 자연스러운 저자인이유	274
22.3 Google 이 합리적으로 두는 다음수	277
22.4 NVIDIA 의 위치: Gated DeltaNet 계보와 kernel moat	277
22.5 Open-source kernel 생태계: flash-linear-attention	278
22.6 학계 · 스타트업의진입점	279
22.7 세 player 의 균형과 pair thesis	280
요약	281
자가 점검체크리스트	281
다음 장으로	282
ch23. 대규모 학습 · 서빙 projection	283
23.1 Bridge-in: 완성형을 배포하면 청구서가 두 장나온다	283
23.2 TNT economics 를 7 – 70B 로 외삽 (accelerator 절반)	283
23.3 Hierarchical memory 의 prefill/decode 분리	285
23.4 Per-session weight state: 새로운 cache class 의 sizing 과 batching	286
23.5 KV-manager 의 범주 오류: 왜 새 매니저가필요한가	288
23.6 Frequency-tiered placement: 서빙 메모리 계층 그자체	289
23.7 Sleep 을 fleet-level background job 으로	290
23.8 오늘의 KV-cache 서빙 대비 비용모델	291
23.9 정직성 결산과 A100 runbook 이월	293
ch24. 제안: 알고리즘 레벨과 하드웨어레벨	294
24.1 Bridge-in: 측정에서제안으로	294
24.2 알고리즘 레벨제안	295
24.2.1 이미 확정한 것: 두 microbench 가 고정한국선	295
24.2.2 대형 실험 제안 A: train/serve chunk-consistency	295
24.2.3 대형 실험 제안 B: state-capacity 를 scaling 축으로	295
24.2.4 이 실험들의 vehicle: HOPE/Sleep 재현 runbook	296
24.2.5 알고리즘 제안 J: learned chunk/cadence schedule	297
24.3 하드웨어 레벨 — memory-centric 절반	297
24.3.1 RMW-bandwidth decode 소자	297
24.3.2 frequency-tiered memory placement	300
24.4 하드웨어 레벨 — accelerator 절반 (pair 의 다른쪽)	301
24.5 소프트웨어 제안: per-session-weights serving 스택	303
24.6 배제: FORCED 방향 두가지	304
요약	305
자가 점검체크리스트	305
다음 장으로	306
ch25. 결론과 연구어젠다	307
25.1 Bridge-in: Part III 가 한일	307
25.2 pair thesis 재확인: 측정이 정착시킨 것과 열어 둔것	307
25.3 이 책이 연 것	308
25.4 남긴 것: 연구어젠다	309
25.5 닫는 말	312
요약	312
자가 점검체크리스트	313

후미	314
참고문헌 (Bibliography)	315
A. 주 논문 — Google Titans/neural-memory 6 부작	315
B. 외부 문헌 — 계보별	316
B.1 Linear attention · Fast Weight Programming (FWP)	316
B.2 DeltaNet · Gated DeltaNet · state-tracking	316
B.3 SSM · Mamba 계보	317
B.4 Test-Time Training (TTT) · online-learning 관점의 sequence model	317
B.5 병렬화 · optimizer · in-context learning as gradient descent	318
B.6 Optimizer · 훈련 기초 (Part I 배경)	318
B.7 Online learning · meta-learning · bilevel	319
B.8 Associative memory · Hopfield	319
B.9 Continual learning · distillation · self-improvement	319
B.10 Systems · serving · scaling law	320
B.11 교과서 · 고전참조	321
C. 구현 · 저장소 인용 (implementation citations)	321
D. 미해결 인용 (unresolved)	321
확인 완료 (과거 미해결 → 해소)	321
용어집 색인 (Glossary Index)	322
1. Rosetta 대응 (inference 어휘 ↔ 이 라인의어휘)	322
2. 용어집 (알파벳순)	324
A	324
B	324
C	324
D	325
F	325
H	325
I	325
K	326
L	326
M	326
N	326
O	326
P	327
Q	327
R	327
S	327
T	327
W	328
3. 예약 기호 색인 (STYLE § 1.2 요약)	328
부록 E. 실험환경 · 재현 · 신뢰성증명	330
E.1 무엇을 신뢰하고 무엇을 신뢰하지 말 것인가	330
E.2 실행 환경지문	330
E.3 재현 절차	331
E.4 결정론 · 재현성 증명 (실측)	331
E.5 교차검증 — 신뢰성의 중심 기둥 (3 중 <1% 일치)	332
E.6 명시적으로 주장하지 않는 것 — A100 runbook 이월 (=반증목록)	332

서문

이 책은 여섯 편의 논문 — [Titans], [Miras], [Atlas], [TNT], [NL], [Sleep] — 을 하나의 연속된 연구 프로그램으로 읽고, 그 프로그램이 만들어 낸 배포 형태를 systems 엔지니어의 도구로 측정할 결과다. 논문들은 Google Research 의 Ali Behrouz 등이 2024 년 12 월부터 2026 년 6 월까지 순차 공개한 test-time memorization / neural-memory 계열이며, 저마다 “다음 논문의 abstract 가 이 논문의 future work” 인 방식으로 연결된다. 이 책은 그 연결을 복원하고, 마지막에 그 완성형을 decode roofline·GEMM shape·memory 계층 배치라는 독자의 언어로 계량한다.

누구를 위한 책인가

이 책은 **transformer inference** 를 잘 아는 한 명의 엔지니어를 상정하고 쓰였다. 그 독자는 KV cache, paged attention, FlashAttention tiling, GEMM shape, MFU, roofline, pre-fill/decode 비대칭, continuous batching, scan kernel 을 운영 감각으로 안다. efficient-transformer 와 linear-attention 계열 (DeltaNet, Gated DeltaNet, Mamba-2) 도 최소한 이름과 골격은 안다.

그러나 그 독자는 **training 경험**이 전무하다. backward pass, optimizer state, momentum 과 weight decay 가 무엇을 하는지, loss surface 위에서 gradient step 이 무엇을 움직이는지 — 이 어휘는 독자의 것이 아니다. 이 라인의 정의적 사건이 바로 그 어휘가 **serving 경로 안으로 들어오는 것**이라서, 이 간극이 이 책의 존재 이유다. decode 한 step 이 작은 신경망의 forward·loss·backward·update 가 되는 순간, 추론 엔지니어는 자기가 한 번도 다뤄 본 적 없는 객체 — 움직이는 weights, optimizer trajectory 상태, gradient 가 만드는 트래픽 — 를 serving 시스템 안에서 마주하게 된다. 이 책은 그 마주침을 준비시킨다.

이 독자를 위해 두 가지 원칙을 관철한다. 첫째, 모든 training 개념은 처음부터 (**state, update, cost**) 를 갖는 객체로 제시한다 — optimizer 는 “register file 과 accumulator 를 가진 상태 기계” 로, backprop 은 “surprise 를 저장하는 memory” 로 도입된다. 둘째, 새 개념은 반드시 독자의 inference 어휘에 접속시킨다: memory write 는 KV cache append 로, retention gate 는 학습된 eviction policy 로, chunk 는 (그러나 bit-exact 가 아닌) FlashAttention tile 로 — 이 대응의 성격 (동일/유비/차이가 논점) 까지 매번 밝힌다.

왜 이 여섯편인가

이 여섯 편은 임의로 묶은 reading list 가 아니라 **하나의 설계공간을 한 축씩 열고 닫아 온 프로그램**이다. 먼저 optimizer 를 sequence layer 로 만들고 ([Titans]), 그 수가 여는 설계 공간에 이름을 붙이고 ([Miras]), 그 공간의 각 축을 최적으로 밀어붙이고 ([Atlas]), 훈련 비용을 계산해 지불하고 ([TNT]), 그 수가 보편적임을 — 모델도 optimizer 도 backprop 도 전부 같은 associative memory 임을 — 선언하고 ([NL]), 마지막으로 train/test 경계 자체를 지운다 ([Sleep]). 여섯 편의 open-questions 절이 서로 다른 다섯공백 (스케일, retrieval 격차, serving 경제학, 학습되는 스케줄, 안전한 self-modification) 으로 **수렴한다**는 사실이, 이 라인의 다음 수가 텍스트에 의해 과잉 결정되어 있음을 보여 준다. 그 다음 수 — 세션 상태가 mutable weights 인 continually-learning LLM — 를 읽어 내는 것이 Part II 의 결산이다.

어떻게 읽는가

책은 세 Part 로 나뉜다.

- **Part I (1 – 11 장)** — **inference 엔지니어를 위한 배경**. training 이라는 낯선 세계를 독자의 어휘로 건설한다. optimizer·online learning·meta-learning·associative memory·chunkwise 병렬화가 여기서 (state, update, cost) 객체로 도입된다.
- **Part II (12 – 17 장)** — **여섯 편의 정밀 독해**. 각 논문을 통일표기로 해부하고, 무엇을계승·일반화·폐기했는지 bridge 로 잇는다. 여섯장을 겹치면 하나의 완성형이 떠오른다.
- **Part III (18 – 25 장)** — **원저 기여**. 그 완성형을 척추 명제 (아래) 로정식화하고, 8 개 실측 실험으로 계량하며, scaling·하드웨어·serving·player-strategy·제안으로전개한다.

training 무경험 독자를 위한 우회로. Part I 을 전부 정독하는 것이정도이지만, 이 독자에게 진짜 새로운 장과 이미 아는 장은 다르다. 급행경로는 이렇다: **1·2·8·9 장을 정독하라** — 1 장 (Rosetta 지도), 2 장 (optimizer 를 객체로; 이 독자의 최대 공백), 8 장 (TTT 원형), 9 장 (chunkwise 병렬화; 이 라인 전체의 실행 기반). **3·4 장은 Part II 가 호출할 때 당겨 읽어도된다** (regret/FTRL 는 13 장이, meta-learning/ W_{init} 은 15 장이 호출한다). **5·6·7 장은 이미 linear attention 과 SSM 을 아는 독자라면 훑고** (delta rule 의 crosstalk 과 § 1.6 카탈로그만 확인) 지나가도 좋다. **10·11 장은 참조 장이다** — 10 장은 cost model cheat sheet 라 Part III 에서 되돌아오고, 11 장은 continual learning 으로 17 장 직전에 읽으면 된다. 요컨대 이 독자에게 Part I 의무게중심은 익숙한 memory 계보 (5 – 7 장) 가 아니라 낯선 training 기계 (2 – 4·8 – 9 장) 에 있다.

각 장은 목표 상자 (“이 장을 마친 독자는 ~할 수 있다”) 로열고, Part I 장은 손으로 따라갈 수 치 예제 (worked micro-example) 와 자가점검 체크리스트로 닫는다. 앞선 장을 참조할 때는 “→ 9 장” 처럼 장 번호로가리킨다.

이 책의 척추: pair thesis 예고

Part III 전체는 하나의 명제를 검증한다. 완성형이 만드는 배포 workload 는**질적으로 다른 두 부**하로 갈라지며, 각각의 최적 하드웨어 전략이다르다는 것이다 (정식 진술은 18 장).

- **decode/serving-state 관리 = memory-centric 기획**. decode step 은 token 마다 fast-weight state 전체를읽고·갱신하고·되쓴다 (read-modify-write). 이트래픽은 sequence 마다 unshared 이고 write-heavy 이며 content 로 주소지정되지 않는다 — append-once/read-many 이고 prefix 로 공유가능한 KV cache 와 질적으로 다른 memory 부하다.
- **training/prefill = accelerator 영역**. 같은알고리즘도 chunk 크기 C 를 키우면 compute-bound 로 옮겨 간다. chunk C 는 문자 그대로 roofline 의 x 축이며, 승부는 fused chunk kernel 과 grouped-GEMM 에서 난다.

흥미로운 하드웨어 제안은 둘 중 하나가 아니라 **쌍 (pair)** 이다 — 어느 한쪽만 옹호하는 것은 부하의 절반을 무시하는 것이다. 이 pair thesis(책의 명명 D4) 가 memory-centric 논증을 펴는 유일한 두 지점이며, “이 라인의 훈련이 새 memory 소자를 요구한다” 거나 “일반 PIM 이 답이다” 같은 주장은 논문들의 자체 증거가 반대 방향을 가리키므로 이 책은 펴지않는다.

정직성 계약

이 책은 논문에 불리한 사실의 완곡화·누락을 결함으로취급한다. 여섯 편의 실증 상한은 **1.3B params / 100B tokens** 이며 ([TNT] 는 150M), decode wall-clock 수치는 여섯 편 어디에도없다 — 이 두 사실은 Part II·III 전반에서 반복 명시된다. Atlas 의 Muon 제거가 perplexity 를 오히려 개선한 점, attention 과의 in-context retrieval 격차 (53.55 vs 43.70) 가측정된 채 달하지 않은 점, TNT 가 momentum·gating 을 제거한단순화 Titans 로 검증한 점, Sleep 이 pre-trained

backbone 위의 graft 라라인 최초로 end-to-end meta-learn 되지 않은 점 — 이런 유보는 감추지 않고 해당 장 본문에 담는다.

Part III 의 실측도 같은 계약을 따른다. 이 책이 돌린 8 개 실험은 **exploration-grade** 다 — 순수 analytic cost model(hatir + hat-schema twin) 과 CPU micro-benchmark 에서 나온 값이다. 본문으로 승격되는 것은 **비율**, **crossover** 위치, **tier** 순서, **bound** 분류이지 silicon 정확절대치가 아니다. 여섯 논문이 H100 decode wall-clock 을 하나도 공개하지 않았으므로, $\mu\text{s}/\text{token} \cdot \text{mJ}/\text{token}$ 같은 절대치는 roofline 하한일 뿐이며 **사내 A100 runbook(Part III-a)** 으로 이월한다. novel device twin(scratchpad, PIM) 은 `simulation_ready=False` 로 directional DSE 에만 인용한다. 세 독립 방법이 anchor 설정에서 1% 이내로 일치한다는 교차검증이, 절대치가 아니라 구조 (비율 · 순서 · crossover) 를 본문에 올리는 근거다. 이 실험들의 **정확한 실행 환경 · 재현 절차 · 결정론 증명 · 교차검증 표 · 반증 목록**은 책 끝의 **부록 E**(및 저장소 `experiments/REPRODUCE.md`) 에 못 박아두었다 — “왜 믿을 수 있는가” 의 답이 그곳에 운영 수준으로 적혀 있다.

표기와 3 층 구분

기술 용어는 **영어 원어 (로마자)** 를 그대로 쓴다 — momentum, weight decay, retention gate, surprise, fast weights, chunk, associative memory 는 번역도 음차도 하지 않는다. 조사는 그 용어의 한글 발음을 기준으로 붙인다 (“chunk 가” , “momentum 이”). 동사 · 서술어만 한국어를 허용한다 (“memory 에 쓴다” , “state 를 갱신한다”). 통일 표기의 핵심은 두 loop 의 대문자 구분이다: $W = \text{fast weights}(\text{inner loop 상태})$, $\Theta = \text{slow weights}(\text{outer loop parameter})$. inner learning rate 는 η_t , momentum decay 는 β_t , retention gate 는 $\alpha_t \in [0, 1]$ (남기는 비율)로 예약된다. 전체 규약은 별도 기준 문서 `style/STYLE-NOTATION.md` 에 있다.

서술은 세 층위를 표기로 구분한다. **논문의 주장**은 항상 논문에 귀속하고 위치를 병기한다 (“...이다. [Atlas § 5.2]”). **이 책의 해설** (논문 내용을 독자의 어휘로 재서술)은 필요 시 블록으로 표시한다:

[해설] inference 관점에서 이 식은 KV cache append 를 rank-1 GEMM write 로 바꾼 것이다.

이 책의 평가 (논문과 다른 판단)는 반드시 블록으로 표시한다:

[평가] 이 ablation 은 Muon 축의 가치가 760M 스케일에서 미결임을 뜻한다.

독자가 어디까지가 논문이고 어디부터가 이 책인지 매 문단에서 알 수 있게 하는 것이, 정직성 계약의 표기 차원이다.

저자: 이승호 (Seungho “Jimmy” Lee). 판본: 조립본 (P2 라운드), 2026-07-12. 기준 문서 `style/STYLE-NOTATION.md v1.1` 을 따른다.

제 1 부 — Inference 엔지니어를 위한 배경

Part I — inference 엔지니어를 위한배경

이 라인의 논문들은 하나같이 독자가 이미 안다고 전제하는 어휘 위에 서있다 — backprop, optimizer state, momentum, meta-learning, associative memory, chunkwise 병렬화. 그러나 이 책의 독자는 그 어휘의 절반을 모른다. Part I 은 그 절반을 짓는다. 목표는 training 을 처음부터 가르치는 것이 아니라, transformer inference 를 아는 엔지니어가 **여섯 편의 수식을자기 언어로 읽을 수 있는 최소한의 다리를** 놓는 것이다.

설계 원칙은 둘이다. 첫째, 모든 training 개념은 (state, update, cost) 를 갖는 객체로 도입한다. optimizer 는 알고리즘이 아니라 상태기계이고, momentum 은 gradient 를 저장하는 memory 이며, backprop 은 surprise 를 나르는 통로다 — 이 관점이 Part II 에서 “optimizer 가 곧 sequence layer 다” 라는 주장을 범주 오류가 아니라 자연스러운 문장으로 만든다. 둘째, 각 장은 독자의 production 감각과 만나는 **systems bridge** — FLOP/byte 계산, GEMM shape, roofline 접점 — 를최소 한 번 통과한다.

장들의 연결은 이렇다. **1 장**이 두 세계 (weights 가 얼어 있는 inference 와 움직이는 learning) 를 잇는 Rosetta stone 과 책 전체의 기준수식 (M) 을 세운다. **2 장**은 그 (M) 의 재료 — backward pass 와 optimizer (momentum, Adam, weight decay, Muon 까지) — 를 객체로도입한다 (이 독자의 최대 공백). **3-4 장**은 두 loop 를 형식화한다: 3 장은 online learning·regret·FTRL 로 inner loop 가 무엇을 최소화하는지를, 4 장은 meta-learning·bilevel 로 outer loop 가 무엇을 학습하는지 (W_{init} 과 data-dependent gate) 를 준다. **5-7 장**은 memory 계보다: 5 장 (Hopfield → delta rule, crosstalk 과 capacity), 6 장 (linear attention 과 fast-weight programming; DeltaNet·Gated DeltaNet·Longhorn·RWKV-7), 7 장 (SSM 계보 S4 → Mamba-2 와 SSD duality) — 이미 이 계열을 아는 독자는 훑고 지나가도 된다. **8 장**은 TTT 원형 (test-time adaptation → TTT-Linear/MLP 와 dual form) 으로 Part II 의 직계 조상을세우고, **9 장**은 이 라인 전체의 실행 기반인 chunkwise-parallel training 을 — stale-snapshot 근사와 “chunk 는 함수 자체를 바꾸는 semantic knob” 라는 명제를 — 정식화한다. **10 장**은 흩어진 cost model 을 한장의 cheat sheet (roofline vs chunk size, glossary) 로 묶어 Part III 가되돌아올 참조점을 만들고, **11 장**은 continual learning·complementary learning systems·distillation·RL-lite 로 [Sleep] 직전의 배경을 채운다.

Part I 을 마친 독자는 여섯 편의 어느 수식을 만나도 “이 gate 는 누가학습하는가” , “이 state 는 얼마나 크고 token 당 몇 byte 를 움직이는가” 를물어볼 수 있다. 그 두 질문이 Part II 와 Part III 를 관통한다.

ch01. Orientation: inference 세계와 learning 세계의 Rosetta Stone

이 장의 목표 — 이 장을 마친 독자는 다음을 할 수 있다. 1. KV cache 와 linear-RNN state 를 memory 의 write/read 어휘로재서술하고, softmax attention 을 “압축하지 않는 memory” 라 부를 수 있다. 2. fast/slow weights 와 inner/outer loop 를 구분하고, “이 learning rate 는 누가 학습하는가?” 에 loop 의 이름으로 답할수 있다. 3. Rosetta-Stone 사전 (표 1-1) 의 각 대응이동일/유비/차이가논점 중 무엇인지 판별할 수 있다. 4. 여섯 논문이 기준 수식 (M) 의 어느성분을 바꾸는지 한 장의 지도로 그릴 수 있다.

왜 필요한가 — 여섯 편 전부가 이 장을 전제한다. [Titans] (Titans: Learning to Memorize at Test Time, arXiv:2501.00663) § 2 는 attention 과 현대 linear recurrent 모델 전부를 memory unit 의 write/read 연산으로재서술하고 ([Titans Eq. 6 – 7]), 이 장의 사전은 그것을 독자의 KV cache 어휘에 접속한다. [TNT] (TNT: Improving Chunkwise Training for Test-Time Memorization, arXiv:2511.07343) 의중심 논증은 “chunk 는 FlashAttention tile 과 달리 계산되는 함수 자체를바꾼다” 는 구분 위에 선다. [NL] (Nested Learning: The Illusion of Deep Learning Architecture, arXiv:2512.24695) 과 [Sleep] (Language Models Need Sleep: Learning to Self-Modify and Consolidate Memories, arXiv:2606.03979) 의 “momentum 도 Adam 도 backprop 도 전부 associative memory 다” 라는 주장은 두 loop 의 그림 (§ 1.4) 없이는 범주 오류로 읽힌다.

1.1 두 세계: weights 가 얼어 있는 세계와 움직이는세계

독자의 세계는 하나의 불변식 위에 서 있다: **forward** 는 **weights** 를읽기만 한다. weights 는 여러 request 가 공유하고, 유일한 가변 상태는 append-only KV cache 다 — prefill/decode 비대칭, continuous batching, paged attention 이 전부 이 전제의 파생물이다.

이 라인은 그 불변식의 폐기가 정의적 특징이다: sequence layer 가 decode 한 step 마다 자기 내부의 작은 신경망 weights 를 **gradient step** 으로갱신한다 — token 하나의 처리가 곧 그 네트워크의 forward·loss·backward·update 다. 직계 원형 Sun et al. 2024 (arXiv:2407.04620) 는 이를 “hidden state 가 모델 그 자체이고 update rule 이 self-supervised learning 의 한 step” 이라 요약한다 [2407.04620 abstract· § 1].

움직이는 것은 weights 전부가 아니다. 두 종류의 parameter 가 공존한다.

- **fast weights** (W) — token 마다 갱신되는 작은 네트워크의 weights. W_{init} 에서 출발해 request 가 끝나면 버려지거나세션 상태로 관리된다 (정식 정의·계보 → 6 장).
- **slow weights** (Θ) — 독자가 아는 통상적 parameter(projection, backbone, gate 생성 head). pretraining 후얼어붙고 serving 관점에서 읽기 전용이다.

즉 폐기되는 것은 “모든 weights 가 얼어 있다” 이고 wake/serving 동안 유지되는 것은 “ Θ 는 얼어있다” 이다 (예외: [Sleep] 의 sleep phase 는 이 불변식을 주기적으로 깨고 slow weights 를 offline 으로 갱신·확장한다 → 17 장) — 이대문자 구분 (W vs Θ) 이 이 책의 가장 중요한 표기 결정

이다. 두 parameter 는 두 학습 과정, 곧 token 마다 W 를 갱신하는 **inner loop** 와 pretraining 으로 Θ 를 학습하는 **outer loop** 에속한다 (형식화 → 4 장). 미리 끊어 둘 오해: 이것은 fine-tuning 이 아니다 — label 도 훈련 job 도 없이 layer forward 안의 내장 연산이며, 가장 가까운상은 “**KV cache 에 압축 codec 이 달렸고 그 인코딩 연산이 gradient step**” 이다.

1.2 모든 sequence 모델은 이미 memory 였다

[Titans] § 2 의 재서술이 이 라인의 출발점이다: 임의의 recurrent sequence 모델은 memory unit 의 write/read 연산 쌍이다 [Titans Eq. 6–7].

$$W_t = f(W_{t-1}, x_t) \quad (\text{write}) \quad (1-1)$$

$$y_t = g(W_t, x_t) \quad (\text{read}) \quad (1-2)$$

독자가 아는 두 극단이 양 끝이다. **softmax attention + KV cache** 는 append-only 목록을 attention 으로 읽는 **압축하지 않는** memory 다 — 무손실 · capacity 무한 대신 state 와 token 당 read 가 $O(L)$ 로자란다. 이 책은 이를 “capacity 무한의 non-parametric **associative memory**” (key 로 연관 value 를 회수하는 저장 구조; 고전 정식화 → 5 장, 공식 Definition → 13 장) 라 부른다. **linear attention / linear-RNN state** 는 고정 크기 행렬 $W \in \mathbb{R}^{d_v \times d_k}$ 에 outer product $v_t k_t^\top$ 를 누적하는 **압축하는** memory 다 — 비용이 고정되는 대신 직교하지 않는 key 들이 겹쳐 쓰여 회수가오염된다 (crosstalk; → 5 장, § 1.6 에서 체험).

lossless-growing 이냐 lossy-fixed 나 — 독자가 이미 운영 감각으로 아는 trade 다. 이 라인의 출발 질문은 **write rule f 를 optimizer 의 한 step 으로 만들면 고정 크기 state 의 손실을 얼마나 줄일 수 있는가**이고, 이관점에서 모든 sequence layer 는 네 질문 (state / write / read / retention) 에 대한 답이다 — 이후 모든장이 이 네 칸을 채운다.

1.3 Rosetta-Stone 사전

아래 표가 이 책의 사전이다 — 이후 모든 장이 새 개념 도입 시 재사용한다. 셋째 열이 가장 중요하다: **동일** (같은 대상의 재서술) · **유비** (정확하나 새 요소있음) · **차이가 논점** (직관을 그대로 인식하면 틀림) 을 구분하지 않으면 잘못된 직관을 이식받는다.

표 1-1 — Rosetta-Stone 사전: inference 어휘 ↔ 이 라인의 어휘

inference 세계 (독자의 어휘)	이 라인의 어휘	대응의 성격
KV cache	non-parametric memory state; softmax attention = capacity 무한 (ϕ^*) 의 associative memory	동일 대상의 재서술 — attention 은 압축하지 않는 memory
KV cache append	memory write (outer-product Hebbian write 가최근접 대응; delta rule 은 append 가 아니라 overwrite)	유비 + 차이: cache 는 append-once, fast weights 는 read-modify- write
attention lookup ($q \cdot K$)	memory read $y_t = \mathcal{M}(q_t; W_t)$	동일 추상화 ($k \rightarrow v$ 연상)
linear-RNN state ($d \times d$)	matrix memory = 고정 크기 lossy 압축 memory	동일
prefill	큰 chunk 의 병렬 write (compression) — TNT 에선 global memory 가담당	유비 (정확)

inference 세계 (독자의 어휘)	이 라인의 어휘	대응의 성격
decode	$C = 1$ 의 per-token online write + read	유비 (정확) — 단 backward pass 가 decode 에 들어온다는 점이 신세계
FlashAttention tiling	chunkwise-parallel training 의 chunk	△ 차이가 논점 : tiling 은 bit-exact, chunk 는 함수 자체를 바꾸는 semantic 근사 (M4)
GEMM shape 감각	$\nabla_W \ell$ 의 outer-product 구조: $dW = (\text{오차}) k^\top$ 는 rank- C GEMM	동일 — 훈련 수식을 GEMM shape 로 읽는 것이 이 책의 교수법
scan/prefix-sum kernel	momentum 의 associative scan (S5 식), Π_t 의 누적 합	동일
roofline / arithmetic intensity	chunk 크기 C 가 arithmetic intensity 를 결정; 품질 최적 C (작음) vs MFU 최적 C (큼) 의 긴장	동일 도구, 새 독립변수
batching (shared weights 전제)	per-request fast-weight state 는 shared-weight batching 을 깨뜨린다 → grouped-GEMM decode	△ 차이가 논점
paged KV cache / session cache	per-session weight state — 새로운 cache class (sizing, checkpoint/restore, eviction)	유비 → Part III 의 주제
cache eviction policy	retention gate = 학습된 eviction	유비 (정확)
speculative decoding 의 rollback	memory state snapshot/rollback (optimizer-trajectory 상태 포함)	유비 + 미해결 문제
optimizer state (경험 없음 → 2 장에서 도입)	훈련의 “register file / accumulator” : m_t, h_t	도입용 유비
weight update 없음 (추론 불변식)	폐기되는 불변식 — 이 라인의 정의적 특징	차이 그 자체
sequence 축	훈련의 batch 축 역할을 sequence 축이 대신한다 (inner loop 의 mini-batch = chunk)	△ 최대 혼동 지점 — 2 장 · 9 장에서 반복 강조
distributed training (경험 없음)	context parallelism: reset 이 sequential chain 을 끊어 shard 병렬화 (TNT)	도입용 유비 (data parallelism 과의 대비)
서빙 fleet 의 백그라운드 job	sleep phase = 주기적 offline consolidation/dreaming job (“weights 의 background compaction”)	유비 → 17 장, Part III

표의 **동일 · 유비** 행은 해당 장이 숫자로 회수한다. 지금 붙일 것은 △ **차이가 논점**인 셋뿐이다. (i) FlashAttention tiling 은 bit-exact 재배열이지만 chunk 안 gradient 는 모두 chunk 시작 state 에서 평가되므로 (식 (M4) 의 stale-snapshot 근사) C 가 **계산되는 함수 자체를 바꾼다** — C 가 semantic hyperparameter 라는사실 (명제화 → 9 장) 이 [TNT] 한 편의 존재 이유다. (ii) per-

request fast weights 는 shared-weight batching 의 전제를 깨서 decode 를 grouped-GEMM 으로 만든다 (→ 10 장). (iii) inner loop 에서는 훈련의 batch 축을 **sequence** 축이 대신한다 (inner mini-batch = chunk) — outer 의 example 축인지 inner 의 token 축인지 매번 물어야 한다 (2 장 · 9 장).

1.4 두 개의 loop: 누가 무엇을 학습하는가

training 무경험 독자의 첫 질문은 정해져 있다: “test time 에 학습한다면 그 learning rate 는 누가 정하는가?” 답은 두 loop 의 분업이다. 먼저 기준수식 (master update) 을 미리 본다 — 유도는 2 장 · 5 장 · 12 장이 조립하고 지금은 읽는 법만 익힌다.

$$S_t = \beta_t S_{t-1} - \eta_t \nabla_W \ell(W_{t-1}; k_t, v_t), \quad W_t = \alpha_t W_{t-1} + S_t \quad (\text{M})$$

읽기는 $y_t = \mathcal{M}(q_t; W_t)$. 기호를 독자의 어휘로 옮긴다.

- $\ell(W_{t-1}; k_t, v_t)$ — **inner loss**: 기본형 $\|\mathcal{M}(k_t; W_{t-1}) - v_t\|_2^2$, “이 쌍이 이미 잘 저장되어있는가” 의 per-token 측정기.
- $\nabla_W \ell$ — 그 실패가 가장 커지는 방향; 실제 write 신호는 반대부호인 $-\eta_t \nabla_W \ell$ 이다 (식 (M) 의 $-\eta_t$ 항). [Titans] 는 이 gradient 를 **surprise** 라 부른다 (정의 → 12 장).
- S_t — gradient 누적 buffer, 즉 **momentum** (정의 → 2 장). $S_t = \beta_t S_{t-1} + (\cdot)$ 은 scan kernel 이 처리하는 선형 recurrence 다.
- $\eta_t, \beta_t, \alpha_t$ — inner learning rate, momentum decay, retention gate (→ 13 장). 시간 첨자가 곧 정보다 (상수 아닌 **token** 마다 계산되는 gate); α_t 는 남기는 비율 (1=유지, 0=소거) = § 1.3 의 학습된 eviction.
- $\mathcal{M}(\cdot; W)$ — read 함수. 상태와 함수의 분리가 이 책의 규약이다.

식 (M) 한 줄 요약: **GD + momentum + weight decay** 를 **token 스트림**에서 돌리면 **그 것이 sequence layer** 다. 처음 질문의 답: gate 들은 slow weights 가 만드는 token 의 함수 ($\eta_t = \eta(x_t; \Theta)$) 이고, Θ (gate·projection· W_{init} 포함) 는 outer loop 가 inner loop 전체를 관통해 backpropagate 하며 학습한다 — “무엇을 세게 쓰고 무엇을 잊을지” 의 정책 자체를 배운다. 형식화 (bilevel optimization) 는 4 장, 논문별 분업표는 Part II 각 장의 “outer vs inner” 절이 담당한다.

1.5 여섯 편의지도

여섯 편은 한 연구 라인 (Google, Behrouz 계열) 의 연작이고, 각 편이 (M) 의 어느 성분을 바꾸는지가 전체 지도다. [Miras] (arXiv:2504.13173; 제목은 It’s All Connected: A Journey Through Test-Time Memorization, Attentional Bias, Retention, and Online Optimization 이지만 이 책은 framework 이름 Miras 로 통칭) 와 [Atlas] (Atlas: Learning to Optimally Memorize the Context at Test Time, arXiv:2505.23735) 가 여기서 처음 등장한다.

표 1-2 — 여섯 편의 논문: 식 (M) 기준의 위치와 Part I 열쇠 장

논문	(M) 에서 바꾸는 것	한 줄 요약	열쇠가 되는 Part I 장
[Titans] (2501.00663)	(M) 그 자체의 수립	GD + momentum + weight decay 가 test time 의 deep memory 갱신 규칙이 된다; surprise 가 write 강도를 정한다	2·5·6·8·9 장

논문	(M) 에서 바꾸는 것	한 줄 요약	열쇠가 되는 Part I 장
[Miras] (2504.13173)	ℓ (attentional bias) 과 retention 항	기존 모델전부 (RetNet·Mamba-2·DeltaNet·TTT·Titans 등) 를 “online optimizer 가 곧 sequence layer” 라는 하나의 framework 로재유도	3·5·6 장
[Atlas] (2505.23735)	ℓ 의 범위 (window c) 와 inner optimizer(Muon/ NS_κ), capacity	per-token write 를 sliding-window 목 적합수 (Omega rule) 로 바꾸고, memory capacity 를 feature map 으로 끌어올린다 – test-time memorization 의 정식화	2·3·5·9 장
[TNT] (2511.07343)	훈련 경제학: chunk 크기 C 의 speed – quality 충돌	hierarchical global/local memory + 주기적 state reset + 2-stage training 으로 큰 C 훈련과 작은 C serving 을 양립	8·9·10 장
[NL] (2512.24695)	층위: 2-level $\rightarrow$ K-level, update frequency	모델과 훈련 절차 전체 를 “각자 자기 주파수로 갱신되는 nested associative memory” 의 스펙트럼으로 재구성 (Hope, CMS)	2·4·5·11 장
[Sleep] (2606.03979)	lifecycle: train/test 경계 자체	wake 에 빠른 memory 로 흡수하고, sleep 에 consolidation·dreaming 으로 느린 weights 에 이관하는 주기적 생애 주기	4·11 장

표 1-2 를 세로로 읽으면 계보, 가로로 읽으면 사용법 (각 행의 “열쇠 장” = 그메커니즘을 여는 Part I 배경) 이다. [Titans] 는 자신의 update 가 Gated DeltaNet, Longhorn, TTT layer, RWKV-7 등을 특수 사례로 포함한다고 논하므로 [Titans App. C], 식 (M) 은 이 라인 바깥의 efficient-attention 계보까지 덮는 일반형이다. Part I 는 의존성 순서로 읽는 것이 가장 싸다.

1.6 Worked micro-example: 2×2 memory 를 손으로 굴러보기

숫자로 확인한다. $d_k = d_v = 2$ memory 에 직교하는 두 쌍 $k_A = (1, 0)^\top \rightarrow (4, 0)^\top$, $k_B = (0, 1)^\top \rightarrow (0, 2)^\top$ 를 Hebbian write ($W \leftarrow W + vk^\top$) 로 쌓으면 $Wk_A = v_A$ 는 정확하다. 그러나 직교하지 않는 셋째 쌍 $k_C = (1, 1)^\top \rightarrow v_C = (1, 1)^\top$ 를 누적한 $W' = \begin{pmatrix} 5 & 1 \\ 1 & 3 \end{pmatrix}$ 에서는 $W'k_C = (6, 4)^\top \neq v_C$ 로

방금 쓴 것조차 오염된다 — append-once 직관이 압축 memory 에서 깨지는 crosstalk 이다.

write 를 gradient step 으로 바꾸면 달라진다. inner loss $\ell(W; k_C, v_C) = \|Wk_C - v_C\|_2^2$ 의 gradient $\nabla_W \ell = 2(Wk_C - v_C)k_C^\top$ 는 오차와 key 의 rank-1 outer product 다. A·B 저장 상태 (오차 $(3, 1)^\top$) 에서 $\eta_t = \frac{1}{4}$ 로 한 step 밟으면 $W''k_C = (1, 1)^\top = v_C$ 로 방금 쓴 쌍이 정확히 회수된다 — write rule 을 누적에서 gradient step 으로 바꾼 것만으로 같은 크기 state 의 품질이 달라졌다. 이것이 delta rule (→ 5 장) 이자 (M) 의 최소형 (M1) 이다. 공짜는 아니어서 A 의 회수는 손상되고 (capacity 질문 → 5 장 · 14 장), gate 3 종의 수치 전개는 2 장의 worked example 에서 완성한다.

1.7 Systems bridge: 첫 back-of-envelope

마지막 페이지는 숫자다 — 아래 산수는 예시이며 특정 논문의 수치 주장이 아니다. 전형적 GQA 한 layer (KV head 8, head 차원 128, bf16) 에서 KV cache 는 token 당 $2 \times 8 \times 128 \times 2 \text{ B} = 4 \text{ KiB}$ — 128K context 면 layer 당 512 MiB, 32-layer 면 request 하나가 16 GiB 다. 반면 같은구성의 matrix memory (TTT-Linear 급, head 당 128×128) 는 32 layer 전체에 8 MiB 로 **context 길 이와 무관하게 고정**이다 — 이 라인이 long-context 에서 갖는 구조적 지렛대다.

대신 네 가지를 지불한다: decode 에 들어온 backward·update (token 당 연산 $\approx 3 \times$; backward $\approx$ forward 의 2 배 → 2 장), state 전체의 read-modify-write 트래픽, shared-weight batching 을 깨는 grouped-GEMM decode, chunk 를 키우면 함수가 바뀌어 떨어지는 훈련품질 — [TNT] 는 작은 chunk 의 deep memory 훈련이 peak 대비 5 – 10% 미만 FLOPs utilization 로 흔히 떨어진다고 보고한다 [TNT § 1]. 정밀 계산과 roofline 배치는 10 장 budget 표가 담당한다.

요약

- softmax attention + KV cache 는 압축하지 않는 (capacity 무한, non-parametric) associative memory 이고, linear-RNN 의 $d \times d$ state 는 고정 크기 lossy memory 다 — 모든 sequence 모델은 write (1-1) 과 read (1-2) 의 쌍이다 [Titans Eq. 6 – 7].
- 정의적 특징은 decode 안의 gradient step 이다: “weights 는 읽기 전용” 불변식이 fast weights W 에는 폐기되고 slow weights Θ 에는 유지된다. inner loop 는 W_t 를 갱신하고, outer loop 는 gate·projection· W_{init} 을 포함한 Θ 를 학습한다 — “누가 학습하는가” 의 답은 항상 outer loop 다.
- 여섯 논문은 기준 수식 (M) 의 성분 — ℓ ·retention (Miras), window·optimizer (Atlas), 훈련 경제학 (TNT), 층위 (NL), lifecycle (Sleep) — 을 바꾼 것이다.
- write 는 (오차) $k^\top$ 꼴의 rank-1/rank- C GEMM 이고 write rule 의 선택 (append/Hebbian/delta) 이 곧 memory 품질의 선택이다 (2×2 손계산으로 확인). $\triangle$ 3 대 함정: FlashAttention tiling $\leftrightarrow$ chunk (bit-exact vs semantic), shared-weight batching 의 붕괴, sequence 축이 batch 축을 대신한다는 것.
- 고정 크기 state 의 대가는 decode 내 backward, state RMW 트래픽, grouped-GEMM decode, chunk 크기의 품질-MFU 긴장이다.

자가 점검체크리스트

- ☐ KV cache 와 $d \times d$ matrix state 를 네 설계 질문 (state / write / read / retention) 으로서 술할 수 있다.
- ☐ “inner learning rate 는 누가 학습하는가?” 에 두 loop 의 어휘로 답하고 $\eta_t = \eta(x_t; \Theta)$ 의 의미를 설명할 수 있다.
- ☐ 식 (M) 의 각 기호 ($\ell, \nabla_W \ell, S_t, \eta_t/\beta_t/\alpha_t, \mathcal{M}$) 를 inference 어휘로 옮길 수 있다.
- ☐ § 1.6 의 Hebbian/delta write 를 손으로 재계산하고 delta write 가 k_C 회수를 정확하게 만든 이유를 말할 수 있다.

□ 표 1-1의 임의 행의 대응 성격 (동일/유비/차이 — 예: bit-exact tiling vs semantic chunk)을 판별하고 그 개념을 inference 어휘로 옮길 수 있다.

다음 장으로

이 장은 “write가 gradient step이다”라고 선언만 했다 — gradient가 무엇이고 어떻게 계산되는지는 아직 블랙박스다. 2장은 그 블랙박스를 연다: backward pass를 밑바닥부터 세우고, SGD·momentum·AdamW·Muon을 각각 (state, update, cost)를 갖는 객체로 정의한다. 그 과정에서 이 장의 복선두 개가 회수된다: momentum buffer가 linear-RNN state와 같은 대수를 탄다는 것, 그리고 weight decay의 $(1 - \eta\lambda)$ 가 retention gate와 같은 물건이라는 것. 식 (M)의 부품이 손에 잡히고 나면, Titans의 memory update는 “처음 보는 수식”이 아니라 “아는 부품의 재배선”이 된다.

ch02. Training as a system: backprop과 optimizer를 (state, update, cost) 객체로 읽는다

이 장의 목표 — 이 장을 마치면 독자는 다음을 할 수 있어야 한다. 1. backward pass가 계산하는 두 GEMM의 shape를 쓰고, forward 대비 FLOPs 비율 (약 $2\times$)을 유도할 수 있다. 2. SGD / momentum / AdamW / AdaGrad / Shampoo / Muon 각각을 $\text{update}(\text{state}, \text{gradient}) \rightarrow (\text{state}', \Delta \Theta)$ 서명을 갖는 **stateful 객체**로 기술하고, param 당 state 크기와 step 당 비용을 산정할 수 있다. 3. “momentum은 linear recurrence다”, “weight decay의 감쇠 인자는 retention gate(역사적 별칭 forget gate $\rightarrow$ 13장)의 원형이다”라는 두 identity를 수식으로 진술할 수 있다 — 이 둘을 합치면 그대로 [Titans]의 memory update가 된다. 4. 훈련의 batch 축과 이라인의 sequence 축이 어떻게 역할을 교대하는지 설명할 수 있다.

왜 필요한가 — 6편 전부가 이 장을 전제하며, 특히 다음 지점들이 이 장 없이는 읽히지 않는다. - [Titans] (Titans: Learning to Memorize at Test Time, arXiv:2501.00663) § 3.1: long-term memory의 write 연산이 **문자 그대로 gradient descent + momentum + weight decay 한 step**이다 (원문 Eq. 8, 10, 13 – 14). 세 optimizer 스칼라 (learning rate, momentum decay, weight decay)가 token의 함수인 gate로 승격된 것이 전부다. - [Atlas] (Atlas: Learning to Optimally Memorize the Context at Test Time, arXiv:2505.23735) Eq. 32 – 33: inner optimizer를 GD에서 **Muon**(momentum + Newton–Schulz 직교화)으로 교체한다. Muon이 객체로 잡혀 있지 않으면 이 교체의 의미와 비용을 판정할 수 없다. - [NL] (Nested Learning: The Illusion of Deep Learning Architecture, arXiv:2512.24695) § 4.2 – 4.3, App. B: momentum을 gradient stream을 압축하는 associative memory로, **Adam을 element-wise ℓ_2 objective의 최적 associative memory**로 재구성하고, AdaGrad/Shampoo/Muon을 preconditioned memory로 배열한다. 이 장의 객체 모델이 그 재구성의 원자재다. - [Miras] (It’s All Connected: A Journey Through Test-Time Memorization, Attentional Bias, Retention, and Online Optimization, arXiv:2504.13173; 이 책은 framework 이름 Miras로 통칭한다): sequence model을 (memory architecture, attentional bias, retention gate, memory learning algorithm)의 조합으로 분해한다. 넷째 축 “learning algorithm”의 값들이 바로 이 장의 optimizer 목록이다. - [TNT] (TNT: Improving Chunkwise Training for Test-Time Memorization, arXiv:2511.07343): chunkwise mini-batch GD를 상속하며, “chunk = inner loop의 mini-batch”라는 이 장 § 2.6의 대응 없이는 문제 설정 자체가 성립하지 않는다. - [Sleep] (Language Models Need Sleep: Learning to Self-Modify and Consolidate Memories, arXiv:2606.03979): wake/sleep life-cycle의 consolidation은 낮은 update frequency로 실행되는 parameter update다 — “update가 일어난다는 것”의 비용 구조를 이 장이 확정한다.

독자의 세계에서 weight는 read-only다. serving 엔진은 checkpoint를 메모리에 올리고, 이후 모든 연산은 그 위의 순수 함수다. 이 장은 그 checkpoint를 만들어낸 공정, 즉 **training**

을 하나의 시스템으로 분해한다. 분해의 단위는 이 책 전체가 쓸 단위와 같다: **state** 가 무엇이고, **update** 식이 무엇이며, **cost** 가 얼마인가. 이 세 질문에 답하는 형태로 훈련을 배우면, 이후 장들에서 “optimizer 가 곧 sequence layer 다” 라는 이 라인의 중심 명제가 범주 착오가 아니라 자연스러운 동형사상으로 읽힌다.

2.1 파라미터가 움직이는 세계: training loop 의 해부

훈련의 목적은 단순하다. parameter 집합 Θ — 이 책의 표기로 **slow weights** — 를 조금씩 움직여, 데이터에 대한 **loss** $\mathcal{L}(\Theta)$ 라는 스칼라 하나를 줄이는 것이다. language model 이라면 $\mathcal{L}$ 은 next-token prediction 의 cross-entropy 를 batch 전체에 대해 평균한 값이다. 핵심은 이것이 **스칼라**라는 사실이다: 수십억 개의 parameter 가 만들어내는 모든 행동이 숫자 하나로 요약되고, 훈련은 그 숫자 하나를 내리는 방향으로 전체 parameter 를 갱신하는 반복이다.

한 번의 훈련 step 은 네 단계로 이루어진다.

1. **forward pass**: batch 를 모델에 통과시켜 loss 를 계산한다. 독자가 아는 prefill 과 커널 구성이 같다 — GEMM 과 attention 의 연속이다.
2. **loss 계산**: logits 에서 스칼라 $\mathcal{L}$ 로의 reduction. 비용은 무시 가능하다.
3. **backward pass**: $\mathcal{L}$ 의 모든 parameter 에 대한 미분, 즉 gradient 를 계산한다 (§ 2.3). FLOPs 는 forward 의 약 $2\times$ 다.
4. **optimizer step**: gradient 와 optimizer 의 내부 state 를 읽어 Θ 를 갱신한다 (§ 2.5). AdamW 계열에서는 GEMM 이 없는 순수 elementwise pass 이며 철저히 bandwidth-bound 다 (단 Shampoo·Muon 은 update 에 GEMM 이 들어오는 예외 — § 2.5.5–6).

이 loop 를 독자가 매일 운용하는 serving loop 옆에 놓으면 대응이 즉시 보인다. 표 2-1 은 1 장의 Rosetta-Stone 사전을 training loop 에 특화한 것이다.

표 2-1 — training loop 와 serving loop 의 대응

항목	serving loop (독자의 세계)	training loop (이 장)
weights	read-only (mmap 된 checkpoint)	read-modify-write (매 step 갱신)
반복 단위	request / token	step / batch
지속 상태	KV cache (request 수명)	optimizer state m_t, h_t (훈련 전체 수명)
주 커널	GEMM + attention (forward 만)	forward GEMM + backward GEMM 2 종 + elementwise step
roofline 위치	prefill compute-bound, decode bandwidth-bound	fwd/bwd compute-bound, optimizer step bandwidth-bound
성능 지표	tokens/s, TTFT	tokens/s, MFU

이 표에서 독자에게 낯선 칸은 두 개뿐이다: backward pass 와 optimizer state. 이 장의 나머지는 그 두 칸을 채운다. 그리고 미리 말해 두면, 이 라인의 논문들이 하는 일은 이 표의 왼쪽 열과 오른쪽 열을 **한 프로세스 안에 합치는 것**이다 — decode loop 안에서 작은 네트워크의 weights 가 매 token 갱신된다. “weight update 없음” 이라는 serving 의 불변식은 이 라인에서 폐기되는 불변식이다 (→ 1 장).

2.2 gradient: 스칼라 하나가 모든 parameter 에 내리는지

gradient $\nabla_{\Theta}\mathcal{L}$ 는 Θ 와 같은 shape 의 텐서로, 각 원소는 “그 parameter 를 아주 조금 올리면 loss 가 얼마나 변하는가” 라는 민감도다. 1 차 Taylor 근사로 쓰면

$$\mathcal{L}(\Theta + \Delta\Theta) \approx \mathcal{L}(\Theta) + \langle \nabla_{\Theta} \mathcal{L}, \Delta\Theta \rangle$$

이고, 이동 크기를 고정하면 내적을 가장 빠르게 음수로 만드는 선택은 $\Delta\Theta = -\eta \nabla_{\Theta} \mathcal{L}$ 이다. 여기서 η 는 **learning rate** — 한 step 의 보폭이다 (이 책에서 무첨자 η 는 항상 outer loop 의 상수 learning rate 이고, 시간 첨자가 붙은 η_t 는 token 의 함수인 inner loop 의 gate 다. 첨자의 유무 자체가 정보다). 이것이 **gradient descent** 의 전부다: 민감도 방향의 반대로, 보폭 η 만큼, 반복해서 이동한다.

시스템 관점에서 gradient 의 첫 번째 성질은 shape 다. gradient 는 parameter 와 1:1 대응하는 같은 크기의 텐서이므로, gradient 를 만들고 소비하는 모든 단계의 메모리 트래픽은 최소 “모델 크기 $\times$ 상수 배” 다. 7B 모델이면 gradient 텐서 하나가 이미 수십 GB 의 이동을 뜻한다. 두 번째 성질은 계산 경로다: 원소가 10^9 개를 넘는 텐서의 편미분 전부를, 어떻게 forward 한 번 남짓의 비용으로 얻는가. 그것이 backward pass 다.

2.3 backward pass: 반복되는 VJP, 그리고 $dW = \delta x^\top$ 라는 shape

가장 순진한 방법부터 기각하자. 편미분을 수치적으로 구하려면 parameter 하나를 ϵ 만큼 흔들고 forward 를 다시 돌려야 한다 — parameter 가 N 개면 forward $N + 1$ 번, $N \sim 10^9$ 이므로 논외다. **backpropagation**(reverse-mode automatic differentiation) 은 forward 한 번 + backward 한 번으로 N 개 편미분 전부를 얻는다 (Rumelhart, Hinton & Williams 1986; 교과서 서술은 Goodfellow, Bengio & Courville 2016, Deep Learning, ch. 6–8). 원리는 chain rule 을 출력에서 입력 방향으로, 즉 스칼라 loss 에서 시작해 거꾸로 적용하는 것이다.

backward 의 원자 연산은 **VJP(vector-Jacobian product)** 다. 어떤 layer 가 $y = f(x)$ 를 계산했다면, backward 는 상류에서 내려온 $\partial \mathcal{L} / \partial y$ 를 받아 $\partial \mathcal{L} / \partial x = (\partial y / \partial x)^\top \partial \mathcal{L} / \partial y$ 를 하류로 넘긴다. Jacobian $\partial y / \partial x$ 를 실체화하지 않고 “Jacobian 을 벡터에 곱한 결과” 만 계산한다는 점이 요체다 — 독자에게 익숙한 어휘로는, 큰 중간 행렬을 materialize 하지 않고 fused kernel 한 번으로 통과시키는 것과 같은 감각이다.

구체적으로 L_{layer} 층 MLP 를 보자 ($\ell = 1, \dots, L_{\text{layer}}$). layer ℓ 은 pre-activation $z_\ell = W_\ell x_{\ell-1}$ 과 출력 $x_\ell = \sigma(z_\ell)$ 을 계산한다 (이 장에서 W_ℓ 의 첨자는 layer 인덱스다; 시간 첨자를 단 fast weights W_t 와 혼동하지 말 것 — 여기의 W_ℓ 들은 전부 Θ 의 성분이다). **backprop 오차** $\delta_\ell := \partial \mathcal{L} / \partial z_\ell$ 을 정의하면, chain rule 은 두 줄짜리 recursion 이 된다:

$$\delta_\ell = (W_{\ell+1}^\top \delta_{\ell+1}) \odot \sigma'(z_\ell), \quad \frac{\partial \mathcal{L}}{\partial W_\ell} = \delta_\ell x_{\ell-1}^\top \quad (2-1)$$

첫 식은 오차를 한 층 아래로 전파하고 (오차 전파), 둘째 식은 그 층의 weight gradient 를 만든다 (gradient 생성). 식 (2-1) 의 둘째 식을 뚫어지게 볼 필요가 있다. **weight gradient** 는 “그 층의 입력” 과 “그 층의 오차” 의 **outer product** 다. 훈련이 layer 의 weight 에 가하는 모든 변화는 rank-1 outer product 들의 합이라는 뜻이다.

batch 로 쌓으면 GEMM shape 가 드러난다. 입력을 행으로 쌓은 $X_{\ell-1} \in \mathbb{R}^{B \times d}$, 오차를 행으로 쌓은 $\Delta_\ell \in \mathbb{R}^{B \times d'}$ 에 대해:

- forward: $Z_\ell = X_{\ell-1} W_\ell^\top - (B \times d)(d \times d')$ GEMM 1 개.
- backward 오차 전파: $\partial \mathcal{L} / \partial X_{\ell-1} = \Delta_\ell W_\ell - (B \times d')(d' \times d)$ GEMM 1 개.
- backward gradient 생성: $\partial \mathcal{L} / \partial W_\ell = \Delta_\ell^\top X_{\ell-1} - (d' \times B)(B \times d)$ GEMM 1 개.

forward 가 GEMM 1 개일 때 backward 는 같은 크기의 GEMM 2 개다. 그래서 **backward pass $\approx 2 \times$ forward FLOPs** 라는 훈련 세계의 상수가 나온다 (모델 전체로는 attention 등도 같은 비율을 따르므로 total step $\approx 3 \times$ forward). 독자는 이 세 GEMM 의 shape 를 이미 안다 — 셋째 GEMM $(d' \times B)(B \times d)$ 는 정확히 rank- B 누적, 즉 “outer product B 개를 쌓는” 연산이다.

여기서 이 책 전체를 관통할 다리를 하나 놓는다. $dW = \delta x^\top$ 라는 shape 는, **KV cache append** 를 **outer-product write** $v_t k_t^\top$ 로 바꾼 것과 같은 GEMM family 다. KV cache 는 (k_t, v_t) 쌍을 무손실로 이어 붙이는 memory 이고 ($\rightarrow$ 1 장 Rosetta), 뒤에서 만날 matrix memory 는 같은 쌍

을 $d \times d$ 상태에 rank-1 로 눌러 쓰는 memory 인데 ($\rightarrow$ 5 장, 6 장), 그 write 연산의 GEMM shape 가 gradient 생성의 GEMM shape 와 동일하다. 우연이 아니다 — 5 장에서 보겠지만 delta rule 의 write 는 문자 그대로 ℓ_2 regression loss 의 gradient 이고, 그 loss 의 gradient 가 식 (2-1) 의 특수한 경우이기 때문이다. 훈련 커널과 memory-write 커널이 같은 GEMM 이라는 사실은 이 라인 전체의 하드웨어적 행운이며, 9 장의 chunkwise 병렬화가 성립하는 물질적 기반이다.

backward 에는 FLOPs 외의 비용이 하나 더 있다. 식 (2-1) 을 계산하려면 forward 때의 $x_{\ell-1}$ 과 z_ℓ 이 필요하다 — backward 가 끝날때까지 모든 층의 중간 결과를 붙들고 있어야 한다는 뜻이다. 이 **activation memory** 는 batch 크기 $\times$ sequence 길이 $\times$ hidden 차원 $\times$ 깊이에 비례하며, 큰 모델 훈련에서 weight 보다 먼저 메모리를 바닥낸다. 표준 대응은 **recomputation**(activation/gradient checkpointing) 이다: 일부 층의 activation 을 버리고 backward 중에 forward 를 다시 계산한다 — FLOPs 를 내고 bytes 를 사는 거래로, FlashAttention 이 softmax 중간값을 버리고 재계산하는 것과 정확히 같은 종류의 거래다. 이 개념이 필요한 이유는 후반부에 있다: 이 라인의 layer 는 decode 중에 작은 네트워크의 forward + backward + update 를 수행하므로 ($\rightarrow$ 8 장), “backward 가 무엇을 저장해야 하는가” 가 serving 커널 설계 문제로 넘어온다. 그리고 serving 스택에는 autograd 가 없으므로, 그 gradient 는 식 (2-1) 을 손으로 전개한 fused kernel 로 구현된다 ($\rightarrow$ 10 장).

2.4 loss surface: 왜 η 하나로 부족인가

$\mathcal{L}(\Theta)$ 를 N 차원 지형으로 상상하자. gradient descent 는 이 지형을 국소 경사만 보고 내려간다. 지형은 non-convex 지만, optimizer 설계를 지배하는 것은 전역 구조가 아니라 국소 **curvature** — gradient 가 방향에 따라 얼마나 빨리 변하는가 — 다. 좁고 가파른 골짜기를 생각하면 된다: 골짜기를 가로지르는 방향은 curvature 가 커서 조금만 움직여도 gradient 가 뒤집히고, 골짜기를 따라가는 방향은 curvature 가 작아 한참을 가도 경사가 그대로다. 최적 보폭이 방향마다 다른데 η 는 하나뿐이므로, η 를 가파른 방향에 맞추면 완만한 방향에서 기어가고, 완만한 방향에 맞추면 가파른 방향에서 진동하거나 발산한다. 이것이 ill-conditioning 이며, 이 장 후반의 optimizer 동물원은 전부 이 문제에 대한 서로 다른 응답이다: 방향별 진동을 평균으로 상쇄하고 (momentum), 좌표별로 보폭을 다시 재고 (AdaGrad, Adam), 아예 좌표계를 바꾸고 (Shampoo), update 의 방향 성분만 남긴다 (Muon).

minima 의 모양도 한 단락만큼은 알아야 한다. 훈련이 도달하는 minimum 주변의 지형은 뾰족할 수도 (sharp) 평평할 수도 (flat) 있는데, sharp minimum 은 parameter 의 작은 요동에도 loss 가 크게 변하므로 훈련 데이터와 테스트 데이터의 미세한 분포 차이에 취약하다는 실증 보고가 있다 (Li et al. 2018, arXiv:1712.09913 의 loss landscape 시각화가 표준 참조다). 이 책에서 loss surface 이론은 여기까지만 필요하다 — 수렴 증명은 다루지 않는다. 시스템 독자에게 필요한 요약은 하나다: **step** 의 품질은 **gradient** 만으로 결정되지 않고, **gradient** 를 어떤 상태 (state) 와 어떤 기하 (metric) 로 가공하느냐에 달려 있다. 그 가공기가 optimizer 다.

2.5 optimizer = (state, update, cost) 객체

이제 이 장의 중심 정의다. **optimizer** 는 다음 서명을 갖는 **stateful** 객체다:

$$\text{update} : (\text{state}_t, g_t) \mapsto (\text{state}_{t+1}, \Delta\Theta_t)$$

여기서 $g_t = \nabla_{\Theta} \mathcal{L}$ 은 step t 의 gradient 다. 객체마다 세 속성을 기록한다: **state**(step 사이에 살아남는 텐서가 무엇이고 param 당 몇 byte 인가), **update**(state 와 gradient 에서 $\Delta\Theta$ 를 만드는 식), **cost**(step 당 FLOPs 와 메모리 트래픽). 독자는 attention kernel 을 정확히 이 방식으로 사고한다 — KV cache 가 state, attention 식이 update, roofline 위치가 cost. 같은 틀을 optimizer 에 적용하는 것뿐이다. 표 2-2 가 이 절 전체의 요약이고, 각 소절이한 객체씩 채운다.

표 2-2 — optimizer 객체 요약 (state 크기는 fp32 기준 param 당 byte; Θ 자체는 제외)

객체	state	update 식 (요지)	state 크기	step 비용 특성	이 라인에서의 쓰임
SGD	없음	$\Delta\Theta = -\eta g_t$	0 B	element- wise, bandwidth- bound	delta rule = 1-step SGD (→ 5 장), TTT (→ 8 장)
+ momentum	m_t	$m_t = \beta m_{t-1} + g_t;$ $\Delta\Theta = -\eta m_t$	4 B	element- wise	[Titans] 의 S_t (→ 12 장)
+ weight decay	(없음)	$\Theta \leftarrow (1-\eta\lambda)\Theta + \dots$	0 B	element- wise	retention gate 의 원형 (→ 12·13 장)
AdamW	m_t, h_t	$\Delta\Theta = -\eta \hat{m}_t / (\sqrt{\hat{h}_t} + \epsilon)$	8 B	element- wise	outer loop 의 기본값; [NL] 의 최적 성 정리 (→ 16 장)
AdaGrad	h_t (누적)	$\Delta\Theta = -\eta g_t / (\sqrt{h_t} + \epsilon)$	4 B	element- wise	FTRL 과의 연 결 (→ 3 장), [NL § 4]
Shampoo	L_t, R_t	$\Delta\Theta = -\eta L_t^{-1/4} G_t R_t^{-1/4}$ 행렬당 $\frac{1}{4}(m^2+n^2)$ B		GEMM + 주 기적 행렬 root	precondi- tioning 의 극점, [NL § 4]
Muon	m_t	$\Delta\Theta = -\eta \text{NS}_{\kappa}(m_t)$	4 B	GEMM ~10 - 15 개/행렬	[Atlas] 의 inner optimizer (→ 14 장), [NL] 의 M3

2.5.1 SGD: stateless 한기준점

- **state:** 없다.
- **update:** $\Delta\Theta = -\eta g_t$ (Robbins & Monro 1951 이 확률적 근사로 정식화한 역사적 기준점).
- **cost:** Θ 읽기 + g 읽기 + Θ 쓰기. param 당 FLOP 2 개 남짓의 순수 elementwise pass.

SGD 의 “S(stochastic)” 는 gradient 를 데이터 전체가 아니라 무작위 mini-batch 에서 추정한다는 뜻이다 (§ 2.6). stateless 라는 성질은 사소해보이지만 이 라인에서는 기준선 역할을 한다: TTT-Linear(→ 8 장) 와 delta rule(→ 5 장) 의 inner loop 는 정확히 이 stateless SGD 이고, [Titans] 의 기여는 inner loop 에 state 를 추가한 것으로 요약된다.

2.5.2 momentum: 첫 번째 state, 그리고 linear recurrence 라는정체

- **state:** buffer m_t 하나 (Θ 와 같은 shape).
- **update:**

$$m_t = \beta m_{t-1} + g_t, \quad \Delta\Theta_t = -\eta m_t \quad (2-2)$$

- **cost:** SGD 대비 read-modify-write 대상 텐서가 하나늘어난다 (트래픽 약 2×). FLOPs 는 여전히 param 당 상수 개.

$\beta \in [0, 1)$ 은 momentum decay 상수다 (보통 0.9). 식 (2-2) 를 풀면 $m_t = \sum_{i \leq t} \beta^{t-i} g_i$ — **momentum buffer** 는 과거 **gradient** 들의 **지수가중합**이다. 고전적 해석은 두 가지다: 물리적으로는 공이 관성을 갖고 골짜기를 구르는 것이고 (Polyak 1964; 심층학습 문맥의 재조명은 Sutskever et al. 2013), 통계적으로는 잡음 낀 **gradient** 의 저역통과 필터다 — § 2.4 의 가파른 방향 진동은 부호가 번갈아 나타나므로 합산에서 상쇄되고, 완만한 방향의 일관된 성분은 최대 $1/(1 - \beta)$ 배까지 증폭된다.

이 책이 강조하는 세 번째 해석이 있다. 식 (2-2) 는 **linear recurrence** 다 — 독자가 아는 **linear-RNN state 갱신**, **RetNet** 의 상수 **decay**, **scan kernel** 이 처리하는 점화식과 정확히 같은 대수다. 이 identity 는 장식이 아니라 이 라인의 두 기둥을 떠받친다. 첫째, [Titans] 는 momentum buffer 를 그대로 inner loop 에 이식해 “past surprise” S_t 로 삼는데 (원문 Eq. 10; → 12 장), recurrence 가 linear 이기 때문에 chunk 안에서 associative scan 으로 병렬 계산된다 (→ 9 장) — 독자의 prefix-sum kernel 이여기서 재취업한다. 둘째, [NL § 4.2] 는 식 (2-2) 를 “gradient stream 을 key 없이 압축하는 associative memory” 로 읽고, GD + momentum 전체를 2-level 구조 (안쪽 level 이 gradient 를 momentum memory 로 압축하고, 바깥 level 이 그 state 를 weights 에 적용) 로 재구성한다 — 이것이 **momentum-as-memory** 이며, 이 장은 객체로서의 사실만 확정하고 정리로서의 지위는 16 장이 다룬다. memory 로 읽는 순간 capacity 질문이 성립하는데, [NL § 4.3] 은 $\beta = 0.9$ 일 때 누적 기여의 50% 이상이 최근 gradient 7 개에, 99% 이상이 최근 44 개에 집중됨을 지적한다 — momentum 은 반감기가 짧은 memory 이고, 그 짧음이 continual learning 에서의 forgetting 과 연결된다 (→ 11 장, 16 장).

2.5.3 weight decay: $(1 - \eta\lambda)$ 라는 retention 의원형

- **state**: 없다 (기존 객체에 붙는 modifier 다).
- **update**: 두 형태를 구분해야 한다. (i) **L2 regularization**: loss 에 $\frac{\lambda}{2} \|\Theta\|_2^2$ 를 더한다 — gradient 에 $\lambda\Theta$ 가 섞여 들어가므로, Adam 처럼 gradient 를 재척도하는 optimizer 에서는 decay 강도까지 함께 왜곡된다. (ii) **decoupled weight decay**: gradient 경로와 무관하게 $\Theta \leftarrow (1 - \eta\lambda)\Theta - \eta(\text{optimizer의 update})$ 로 직접 곱해 감쇠시킨다. AdamW 의 W 가 이것이며, 두 형태가 다르다는 지적 자체가 논문 하나다 (Loshchilov & Hutter 2019, arXiv: 1711.05101).
- **cost**: elementwise 곱 하나. 무시 가능.

decoupled 형태에서 update 의 구조가 투명해진다. 매 step Θ 에 $(1 - \eta\lambda)$ 가 곱해지므로, update 기여를 u_i 라 하면

$$\Theta_t = (1 - \eta\lambda)^t \Theta_0 + \sum_{i \leq t} (1 - \eta\lambda)^{t-i} u_i \quad (2-3)$$

— 초기 weights Θ_0 조차 $(1 - \eta\lambda)^t$ 로 지수 감쇠해 사라지므로, **weights 자체가 과거 update 들의 지수가중 memory** 이고, $(1 - \eta\lambda)$ 는 그 **memory** 의 **유지 비율**이다. 식 (2-2) 와 식 (2-3) 을 나란히 놓으면 buffer 와 weights 가 같은 대수의 두 인스턴스임이 보인다. 독자의 어휘로 번역하면 $(1 - \eta\lambda)$ 는 retention gate 의 원형 — 정확히는, cache 에서 오래된 항목을 지수적으로 밀어내는 학습된 eviction 의 가장 원시적 형태다. 이 인자를 상수에서 token 의 함수 α_t 로 승격시킨 것이 식 (M2) 의 retention 항 $\alpha_t W_{t-1}$ 이고, [Titans] 는 이를 memory 의 forgetting mechanism 으로 (→ 12 장), [Miras] 는 retention gate 로 정식화한다 (→ 13 장; 정의 소유권은 그장들에 있다). 이 장에서 확정할 사실은 하나다: **weight decay** 는 이미 언제나 **memory** 관리 연산이었다.

2.5.4 Adam 과 AdamW: 좌표별 보폭의 표준

- **state**: buffer 2 개. 1 차 moment m_t , 2 차 moment h_t (관행상 v_t 로 쓰지만 이 책에서 v 는 value 전용이다).
- **update** (Kingma & Ba 2015, arXiv:1412.6980):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad h_t = \beta_2 h_{t-1} + (1 - \beta_2) g_t \odot g_t, \quad \Delta\Theta_t = -\eta \frac{\hat{m}_t}{\sqrt{\hat{h}_t} + \epsilon} \quad (2-4)$$

($\hat{m}_t = m_t/(1 - \beta_1^t)$, $\hat{h}_t = h_t/(1 - \beta_2^t)$ 는 bias correction — EMA 가 0 으로 초기화된 탓에 초기 step 에서 과소평가되는것을 보정하는 산수이며, 이 책에서 더 깊이 다루지 않는다.) - **cost**: state 텐서 2 개의 read-modify-write. param 당 fp32 8 byte 의 state, step 트래픽은 SGD 의 약 3×. 여전히 GEMM 없는 elementwise pass 다.

m_t 는 식 (2-2) 의 EMA 형태이고, 새 성분은 h_t 다: **좌표별 gradient 제공의 EMA**, 즉 각 좌표의 최근 gradient 크기 추정치다. update 가 $m_t/\sqrt{h_t}$ 이므로 각 좌표의 보폭은 자기 gradient 의 전형적 크기로나뉘 정규화된다 — gradient 가 상시 큰 좌표는 억제되고 상시 작은 좌표는 증폭되어, 모든 좌표가 대략 $\pm\eta$ 스케일로 움직인다. 그래서 Adam 은 “부호에 가까운 (sign-ish) descent + 좌표별 learning rate” 로 요약된다. § 2.4 의 ill-conditioning 에 대한 대각 근사 응답인 셈이다. transformer 훈련에서 Adam 이 사실상 강제인 이유도 여기 있다: embedding row 는 token 빈도에 따라, layer 는 깊이에 따라 gradient 스케일이 수십 배씩 다른데, 단일 η 의 SGD 는 이 이질성을 감당하지 못한다. AdamW = 식 (2-4) + § 2.5.3 의 decoupled decay 이며, 이 조합이 LLM pretraining 의 기본값이다.

이 객체가 이 라인에서 갖는 특별한 지위는 [NL] 이 부여한다: element-wise ℓ_2 regression objective 를 놓으면 그 **최적** associative memory 의 닫힌 형태가 정확히 Adam 의 $m/\sqrt{h}$ 구조로 떨어진다는 재구성이다 [NL App. B]. 즉 Adam 은 “gradient stream 을좌표별로 요약하는 memory” 의 한 최적점이다. 유도는 16 장이 소유한다 —여기서는 state 2 개짜리 객체라는 사실과, 그 두 buffer 가 뒤에서 각각 memory 의 지위를 얻게 된다는 예고만 접수하면 된다.

2.5.5 AdaGrad 와 Shampoo: preconditioning 이라는일반화

Adam 의 $1/\sqrt{h_t}$ 를 일반화하면 optimizer 설계의 남은 절반이보인다. **preconditioning** 은 update 를 $\Delta\Theta = -\eta P^{-1}g_t$ 로 바꾸는 것 — gradient 를 그대로쓰지 않고 행렬 P 가 정의하는 좌표계에서 다시 재는 것이다. P 가 loss 의 curvature 를 닮을수록 step 은 2 차 (Newton) 방법에 가까워진다. Adam 은 P 를 대각으로 제한한 경우다.

AdaGrad(Duchi et al. 2011, JMLR) 는 대각 preconditioner 의원형이다. Adam 의 2 차 moment h_t 가 EMA 인 데 반해 AdaGrad 는 $h_t = h_{t-1} + g_t \odot g_t$ 로 전 이력을 **누적**한다 — 이 decay(EMA) 유무가 preconditioner 를 가르는 핵심 차이다 (더해서 AdaGrad 는 1 차 moment buffer m_t 없이 현재 gradient g_t 를 그대로 분자에쓴다). 갱신은 $\Delta\Theta = -\eta g_t/(\sqrt{h_t} + \epsilon)$ 이다. 누적이므로 보폭은 단조 감소한다 — 유한한 스트림을 한 번 지나가는 online learning 의 이론 (regret 보장) 에서 자연스러운 선택이고, 실제로 AdaGrad 는 FTRL 계열 online 알고리즘과 정확히 접속된다 (→ 3 장). “state 를 decay 없이누적하는가, EMA 로 잇는가” 라는 이 대비를 기억해 두면 3 장의 FTRL vs OGD, 13 장의 retention 논의가 같은 축의 반복임이 보인다.

Shampoo(Gupta et al. 2018, arXiv:1802.09568) 는 대각 제한을 푼다. 행렬 parameter $W \in \mathbb{R}^{m \times n}$ 의 gradient G_t 에 대해좌우 두 통계 $L_t = L_{t-1} + G_t G_t^\top$ ($m \times m$), $R_t = R_{t-1} + G_t^\top G_t$ ($n \times n$) 를 유지하고 $\Delta W = -\eta L_t^{-1/4} G_t R_t^{-1/4}$ 로 갱신한다 — 전체 $mn \times mn$ preconditioner 를 Kronecker 곱 구조로 근사한 것이다. cost 프로파일이 질적으로 다르다: state 가 param 개수가 아니라 행렬 차원의제공 ($m^2 + n^2$) 으로 불고, update 에 GEMM 과 행렬거듭제곱근 (주기적으로만 재계산하는 것이 관행) 이 들어온다. 이 장에서 Shampoo 가 필요한 이유는 실무가 아니라 계보다: [NL § 4] 는 SGD → Adam → AdaGrad → Shampoo → Muon 을 “gradient 를 어떤 memory 로 압축해 어떤좌표계를 학습하는가” 의 한 스펙트럼으로 배열하며, 그 서열의 비대각 지점이 Shampoo 다.

2.5.6 Muon: update 의 직교화, 그리고 test-time compute 로의예고

- **state**: momentum buffer m_t 하나. Adam 의 절반이다.
- **update** (Jordan et al. 2024, blog: kellerjordan.github.io/posts/muon):

$$m_t = \beta m_{t-1} + g_t, \quad \Delta W = -\eta \text{NS}_\kappa(m_t / \|m_t\|_F) \quad (2-5)$$

- **cost:** elementwise 가 아니다. NS_κ 가 parameter shape 의 GEMM 을 반복당 2-3 개, $\kappa = 5$ 기준 행렬당 총 10-15 개 추가한다.

Muon 은 hidden layer 의 2차원 weight 행렬 전용 optimizer 다 (embedding·output head 등은 관행상 AdamW 로 남긴다 — 같은 모델 안에 두 객체가 공존한다). 아이디어는 한 문장이다: momentum 행렬 m_t 를 그대로 쓰지 않고, **가장 가까운 semi-orthogonal 행렬로 사영해서 쓴다**. SVD 로 $m_t = U\Sigma V^\top$ 라 쓰면 그 사영은 $UV^\top$ — 특이값을 전부 1로 갈아 끼운 행렬로, matrix sign 계열 연산이라 **msign** 으로도 불린다. 왜 이것이 좋은가: gradient/momentum 행렬은 소수의 지배적 방향 (큰 특이값) 에 에너지가 몰려 있어, 그대로 적용하면 update 가 사실상 저 rank 가 된다. 직교화는 지배적 방향을 누르고 희귀하지만 유효한 방향을 살려 **모든방향에 고른 크기로** 쓰게 한다 — Adam 이 좌표별로 하던 스케일 평균화를 특이값 스펙트럼에 대해 하는 것이며, spectral norm 기하에서의 steepest descent 이자 2차 정보의 근사로 읽힌다.

SVD 는 비싸므로 실제 구현은 **Newton – Schulz iteration** NS_κ 를 쓴다: 홀수 행렬 다항식 $X \leftarrow aX + bX(X^\top X) + cX(X^\top X)^2$ 을 κ 회 반복하면 X 의 특이벡터는 보존한 채 특이값을 1 근방으로 몰아간다. 고전적 계수는 특이값을 정확히 1로 수렴시키지만, Muon 이 쓰는 $\kappa = 5$ 튜닝된 계수 $(a, b, c) = (3.4445, -4.7750, 2.0315)$ (Jordan et al. 2024) 는 정확한 1 수렴 대신 빠른 근사를 택해 특이값을 대략 1 부근 (정확히 1 이 아니라 얼추 $[0.7, 1.3]$) 에 모은다 — 결과가 이상적 $UV^\top$ 가 아니라 그 근사인 이유다. 시스템 독자에게 이 구현 선택이 핵심이다: “**직교화 = parameter shape 의 GEMM 몇 개**” 이므로, Muon 은 optimizer step 을 bandwidth-bound elementwise pass 에서 tensor core 가 도는 연산으로 바꾸면서도 state 는 buffer 하나로 유지한다. 대규모 LLM pretraining 에서의 실증은 Liu et al. 2025 (arXiv:2502.16982) 가 보고한다.

이 객체가 이 책에 등장하는 진짜 이유는 outer loop 가 아니다. [Atlas] 는 Muon 을 **inner loop 의 optimizer** 로 이식한다: memory 를 GD 대신 식 (2-5) 로 갱신한다 (원문 Eq. 32-33; 통일 표기로는 S_t 에 NS_κ 를 적용해 $W_t = \alpha_t W_{t-1} + \eta_t \text{NS}_\kappa(S_t)$ 꼴 — $\rightarrow$ 14 장). 그 순간 κ 는 decode 중 매 chunk 마다 지불하는 추가 GEMM 개수가 되고, [Atlas §5] 는 이 반복 횟수를 명시적으로 “internal test-time compute parameter” 라고 부른다 — 반복을 늘리면 더 나은 memorization 을 살 수 있다는 것이다.

[NL] 은 한 걸음 더 나가 NS_κ 반복 자체를 “momentum update 안의 내부 최적화 level” 로 읽고 ($\rightarrow$ 16 장), Adam + Muon + 다중 주기 momentum 을 결합한 M3 optimizer 를 제안한다 [NL Alg. 1]. 이 장의 어휘로 요약하면: Muon 은 (state 1 개, GEMM 위주 update, 낮은 state cost / 높은 compute cost) 의 객체이고, 이 라인은 그 객체를 layer 안으로 옮겨 심는다.

2.6 batch 축 vs sequence 축 — 이 주제 최대의 혼동지점

지금까지 gradient 는 “batch 에서 계산된 것” 이었다. **mini-batch** 는 데이터셋에서 독립적으로 뽑은 B 개 샘플이고, batch gradient 는 per-sample gradient 의 평균이다. B 를 키우는 이유는 두 가지다: 통계적으로는 독립 샘플 평균이므로 gradient 추정의 분산이 $1/B$ 로 줄고, 시스템적으로는 §2.3 의 세 GEMM 에서 B 가 클수록 GEMM 이 두꺼워져 MFU 가 오른다 — batch 는 처음부터 품질과 하드웨어 효율을 동시에 사는 knob 이었다. 반대 극단인 $B = 1$ 의 per-sample(online) update 는 같은 대수에 잡음과 skinny GEMM 을 얹은 것이다.

이제 이 책에서 가장 중요한 경고를 볼드로 박는다. **이 라인의 논문들에서, 훈련의 batch 축이 하던 역할을 sequence 축이 대신한다. inner loop 의 “mini-batch” 는 독립 샘플 B 개가 아니라 연속한 token C 개 — 즉 chunk 다.** [Titans] 가 “mini-batch gradient descent” 라고 쓸 때 그 batch 는 서로 이웃한 token 들이고, [TNT] 의 chunk 크기 실험은 전부 의미의 batch 크기 실험이다. 같은 단어가 두 세계에서 다른 축을 가리키므로, 이 대응을 놓치면 논문의 모든 문장이 한 축씩 어긋나게 읽힌다.

역할은 같되 성질이 다르고, 그 차이가 뒤 장들의 논점을 만든다. 첫째, batch 샘플은 독립이지만 sequence token 은 상관되어 있고 인과 순서를 갖는다 — 순서가 있으므로 “chunk 안의 gradient

를 어느 시점의 state 에서 평가하는가” 라는, batch 세계에는 없던 질문이 생긴다. 그 질문의 답 (chunk 시작 state 에 고정하는 stale-snapshot 근사) 이 chunk 크기 C 를 단순한 tiling 파라미터가 아니라 **계산되는 함수 자체를 바꾸는 semantic hyperparameter** 로 만든다 (식 (M4); 명제의 소유는 9 장). FlashAttention 의 tile 크기는 bit-exact 한 성능 knob 이지만 C 는 아니다 — 독자의 tiling 직관을 그대로 이식하면 안 되는 지점이다. 둘째, 그래도 시스템적 역할은 동일하게 작동한다: C 가 클수록 inner loop 의 GEMM 이 두꺼워지고 MFU 가 오른다. 품질이 원하는 C (작게) 와 하드웨어가 원하는 C (크게) 의 긴장이 [TNT] 의 존재 이유다 ($\rightarrow$ 9 장, 15 장). 독자의 어휘로 최종 번역하면: **prefill** 은 큰 **chunk** 의 병렬 **write**, **decode** 는 $C = 1$ 의 **online write** 다 — 이 문장이 성립하도록 만드는 장치 일체가 9 장의 내용이다.

2.7 Worked micro-example: 2×2 memory 에 손으로 돌리는 네 개의 optimizer

이 절에서는 위의 객체들을 전부 한 자리에서, 암산 가능한 숫자로 돌려본다. 대상은 이 라인의 최소 세팅이다: 상태 $W \in \mathbb{R}^{2 \times 2}$ (0 으로 초기화), 저장할 association 은 key $k = (1, 0)^\top$, value $v = (1, 2)^\top$ 하나, inner loss 는 이책의 기본형 $\ell(W; k, v) = \|Wk - v\|_2^2$ 이다. 식 (2-1) 이 곧바로 gradient 를 준다: 오차 $e = Wk - v$ 에 대해 $\delta = 2e$ 이고 gradient 는 outer product

$$\nabla_W \ell = 2 (Wk - v) k^\top.$$

(a) **SGD**, $\eta = 0.25$. step 1: $W_0 k = (0, 0)^\top$ 이므로 $e_1 = (-1, -2)^\top$, $g_1 = 2e_1 k^\top = \begin{bmatrix} -2 & 0 \\ -4 & 0 \end{bmatrix}$.

갱신: $W_1 = W_0 - 0.25 g_1 = \begin{bmatrix} 0.5 & 0 \\ 1 & 0 \end{bmatrix}$. 읽기: $W_1 k = (0.5, 1)^\top$ — 목표의 절반이다. step 2: $e_2 = (-0.5, -1)^\top$, $g_2 = \begin{bmatrix} -1 & 0 \\ -2 & 0 \end{bmatrix}$, $W_2 = \begin{bmatrix} 0.75 & 0 \\ 1.5 & 0 \end{bmatrix}$, 읽기 $(0.75, 1.5)^\top$. 매 step 오차가 정확히 반감한다 ($\eta \cdot 2 k^\top k = 0.5$ 이므로). SGD 는 목표에 기하급수적으로 다가가되 유한 step 에서는 도달하지 못한다. 그리고 둘째열이 끝까지 0 임을 눈여겨보라 — gradient 가 $k^\top$ 방향으로만 뻗는 outer product 이기 때문이며, “write 는 key 가 가리키는 subspace 만 건드린다” 는 5 장 crosstalk 논의의 씨앗이다.

(b) **momentum**, $\beta = 0.5$, $\eta = 0.25$. step 1: $m_1 = g_1$ 이므로 SGD 와 동일하게 $W_1 = \begin{bmatrix} 0.5 & 0 \\ 1 & 0 \end{bmatrix}$. step 2: $m_2 = 0.5 m_1 + g_2 = \begin{bmatrix} -1 & 0 \\ -2 & 0 \end{bmatrix} + \begin{bmatrix} -1 & 0 \\ -2 & 0 \end{bmatrix} = \begin{bmatrix} -2 & 0 \\ -4 & 0 \end{bmatrix}$, $W_2 = W_1 - 0.25 m_2 = \begin{bmatrix} 1 & 0 \\ 2 & 0 \end{bmatrix}$. 읽기: $W_2 k = (1, 2)^\top = v$, **정확히 도달했다**. 과거 gradient 의 잔향 ($0.5 m_1$) 이 현재 step 에 실려 보폭을 늘린 결과다. 이 세팅에서는 운 좋게 정확히 착지했지만 일반적으로는 같은 관성 효과 (overshoot) 도 만든다 — β 와 η 의 균형 문제다. [Titans] 의 S_t 가 하는 일이 대수적으로 정확히 이것이다 (부호관행만 식 (M2) 를 따른다).

(c) **weight decay**, $(1 - \eta\lambda) = 0.9$. (b) 의 W_2 에서 새 token 이 없다고 하자 (gradient 0). decoupled decay 는 그래도 매 step 곱해진다: $W_3 = 0.9 W_2 = \begin{bmatrix} 0.9 & 0 \\ 1.8 & 0 \end{bmatrix}$, 읽기 $(0.9, 1.8)^\top$. 저장된 association 이 접근 없이도 step 마다 10%씩 증발한다 — TTL 이 있는 cache entry 처럼. 이 감쇠 인자를 token 마다 계산되는 α_t 로 바꾸면 식 (M2) 의 retention 향이 된다.

(d) **Adam 의 좌표별 평준화**. (a) 의 g_1 에서 0 이 아닌 두 좌표를 보자: $(1, 1)$ 원소는 -2 , $(2, 1)$ 원소는 -4 로 크기가 2 배 다르다. $t = 1$ 에서 식 (2-4) 를 bias correction 까지 적용하면 (ε 는 무시) $\hat{m}_1 = g_1$, $\hat{h}_1 = g_1 \odot g_1$ 이므로 update 는 $-\eta g_1 / \sqrt{g_1 \odot g_1} = -\eta \text{sign}(g_1)$ — 두 좌표 모두 크기가 정확히 η 다. gradient 크기가 2 배 차이 나도 첫 step 의 보폭은 동일하다: “sign-ish descent” 를 이보다 짧게 보여 주는 계산은 없다. ($t \geq 2$ 부터는 EMA 가 이력을 반영해 순수 sign 에서 벗어난다.)

(e) **Newton – Schulz 의 특이값 평준화**. Muon 의 NS_κ 가 하는 일을 보기 위해, 특이값이 $(1.2, 0.4)$ 인 momentum 행렬을 생각하자 (예: $m = \text{diag}(1.2, 0.4)$; 대각행렬이면 NS 반복이 특

이값 각각에 대한 스칼라 다항식이 된다). 손계산을위해 고전적 3 차 반복 $x \leftarrow 1.5x - 0.5x^3$ 을 쓴다 (실전의 Muon 은 § 2.5.6 의 튜닝된 5 차 다항식을 쓰지만 원리는 같다):

반복	σ_1	σ_2	비율
0	1.2	0.4	$3.0\times$
1	$1.8 - 0.864 = 0.936$	$0.6 - 0.032 = 0.568$	$1.65\times$
2	$1.404 - 0.410 = 0.994$	$0.852 - 0.092 = 0.760$	$1.31\times$

두 번의 반복만에 3 배 차이가 1.3 배로 줄었고, 반복을 계속하면 둘 다 1 로수렴한다. 특이벡터 (방향) 는 건드리지 않고 특이값 (방향별 크기) 만 1 로밀어붙인다 — (d) 의 Adam 이 좌표축에서 하던 평준화를 임의의 직교 방향에대해 하는 것이다. 반복 한 번의 비용이 2×2 에서는 암산이지만 $d \times d$ 에서는 GEMM 2-3 개라는 것, 그것이 Muon cost 모델의 전부다.

다섯 계산의 교혼을 한 줄로 모으면: (a) 가 **delta rule** 이고, (a)+(b)+(c) 가 [Titans] 의 **memory update** 이고, (e) 를 (b) 에 끼우면 [Atlas] 의 **memory update** 다. Part II 에서 만날 update rule 들은 이 절의손계산에 gate 와 chunk 를 입힌 것 이상이 아니다.

2.8 Systems bridge: optimizer 한 step 의 비용회계

이 절은 이 장의 객체들을 독자의 자원 회계 감각에 정착시킨다. 기준 모델로 7B-parameter dense transformer 를 놓자.

state 크기 — optimizer state 는 모델보다 크다. serving 에서 7B 모델은 bf16 으로 약 14 GB 다. mixed-precision 훈련의 표준 레시피에서는 param 당 bf16 weight 2 B + fp32 master weight 4 B + Adam m_t 4 B + h_t 4 B = **14 B/param**, 약 98 GB 가 gradient 와 activation 을 세 기도 전에 상주한다. “훈련은 왜 서빙보다 몇 배의 HBM 을먹는가” 의 답의 절반이 optimizer state 다. 표 2-3 이 객체별 회계다.

표 2-3 — optimizer 객체별 비용 회계 (param 당; state 는 fp32 관행 기준)

객체	state bytes	step 추가 트래픽 (state read+write)	step 연산 성격	roofline 위치
SGD	0	0	elementwise	bandwidth-bound
momentum	4 B	8 B	elementwise	bandwidth-bound
AdamW	8 B	16 B	elementwise	bandwidth-bound
AdaGrad	4 B	8 B	elementwise	bandwidth-bound
Shampoo	행렬당 $4(m^2+n^2)$	통계 갱신 GEMM	GEMM + 주기적 root	혼합
Muon	4 B	8 B	GEMM 10-15 개/행렬	compute 쪽으로 이동

step 의 roofline 위치. AdamW step 은 param 당 read 가 $\text{weight} \cdot \text{gradient} \cdot m \cdot h$, write 가 $\text{weight} \cdot m \cdot h$ — 대략 24-28 B 를움직이며 FLOP 은 십수 개다. arithmetic intensity 가 1 FLOP/byte 언저리인, 독자의 분류로는 decode 와 같은 극단적 bandwidth-bound 커널이다. 훈련 step 전체의 시간 구조가 이제 읽힌다: forward/backward 는 prefill 을 다했고 (두꺼운 GEMM, compute-bound), optimizer step 은 decode 를 다했다 (거대한 state 의 순회, bandwidth-bound). Muon 은 이 그림에서 유일하게 step 을 GEMM 으로 바꾸는객체다 — state 는 Adam 의

절반이면서 연산은 tensor core 로 옮긴다. “state 를 덜 쓰고 compute 를 더 쓴다” 는 이 거래는 14 장에서 inner loop 의 test-time compute 논의로 반복된다.

optimizer state 의 정체 — 훈련의 register file. momentum buffer 와 second moment 는 매 step 읽고 쓰는, 훈련 job 수명 동안 상주하는 accumulator 다. Rosetta 사전의 대응 ($\rightarrow$ 1 장) 을 이 장의 결과로 뒷받침하면: KV cache 가 request 수명의 state 이듯 optimizer state 는 training-run 수명의 state 이고, 차이는 수명과 소유자뿐이다. 이라인이 하는 일은 이 state 의 소유자를 한 번 더 바꾸는 것이다 — momentum buffer S_t 가 **per-session state** 가 되어 decode loop 안으로 들어온다. 그 순간 “optimizer state 회계” 는 훈련 클러스터의 문제가 아니라 serving 의 session cache sizing 문제가 된다 ($\rightarrow$ 10 장, Part III).

두 GEMM 의 동일성 — 이 장에서 가져갈 단 하나의 shape. 식 (2-1) 의 $dW = \delta x^\top$ (batched: $(d' \times B)(B \times d)$) 와, 뒤장들의 memory write $v_t k_t^\top$ (chunked: $(d_v \times C)(C \times d_k)$) 는 같은 GEMM 이다. 훈련의 gradient 생성 커널과 fast-weight memory 의 write 커널이 shape 수준에서 동일하므로, 훈련용으로 존재하는 모든 커널 기술 — tiling, tensor-core 활용, rank-누적 — 이 이 라인 의 inference 에 이식 가능하다. 9 장의 chunkwise 알고리즘들은 이 동일성의 체계적 착취다.

요약

- 훈련은 forward ($\approx$ prefill), backward ($\approx$ forward 의 $2\times$, GEMM 2 종), optimizer step (elementwise, bandwidth-bound) 의 loop 이며, loss 라는 스칼라 하나가 전체 parameter 의 update 를 지휘한다.
- backward pass 는 VJP 의 연쇄이고, weight gradient 는 항상 “층 입력 $\times$ 층오차” 의 outer product $dW = \delta x^\top$ 다 — 이 shape 는 memory write $vk^\top$ 와 동일한 GEMM 이다.
- optimizer 는 $\text{update}(\text{state}, g) \rightarrow (\text{state}', \Delta \Theta)$ 서명의 객체다: SGD 는 stateless, momentum 은 buffer 1 개, AdamW 는 2 개, Shampoo 는 행렬통계, Muon 은 buffer 1 개 + GEMM 연산.
- momentum 식 $m_t = \beta m_{t-1} + g_t$ 는 linear recurrence — linear-RNN state 와 같은 대수 이고, [Titans] 의 S_t 와 [NL] 의 momentum-as-memory 가 이 identity 위에 선다.
- decoupled weight decay 의 $(1 - \eta\lambda)$ 는 상수 retention 인자이며, 이를 token 의 함수로 승격한 것이 식 (M2) 의 retention 항 $\alpha_t W_{t-1}$ 이다.
- Adam 은 좌표별 gradient 스케일로 보폭을 정규화하는 sign-ish descent 이고, [NL App. B] 는 이를 element-wise ℓ_2 objective 의 최적 associative memory 로 재구성한다.
- Muon = momentum + Newton – Schulz 직교화 ($\text{NS}_\kappa, \kappa = 5$): update 의 특이값을 평균화하며, 비용은 행렬당 GEMM 10 – 15 개다. [Atlas] 가 이를 inner optimizer 로 이식한다.
- 이 라인에서 sequence 축이 batch 축의 역할을 맡는다: chunk = inner loop 의 mini-batch. 단 token 은 독립이 아니므로 C 는 semantic hyperparameter 가 된다 ($\rightarrow$ 9 장).

자가 점검체크리스트

- ☐ backward pass 의 GEMM 2 개 (오차 전파, gradient 생성) 의 shape 를 쓰고, 왜 backward 가 forward 의 약 $2\times$ FLOPs 인지 설명할 수 있다.
- ☐ SGD / momentum / AdamW / Muon 의 state 구성과 update 식을 암기가 아니라 재구성으로 쓸 수 있고, param 당 state byte 를 말할 수 있다.
- ☐ momentum recurrence 가 linear-RNN state 갱신과 같은 대수임을 보이고, 그것이 [Titans] 의 S_t 와 어떻게 연결되는지 말할 수 있다.
- ☐ L2 regularization 과 decoupled weight decay 의 차이를 설명하고, $(1 - \eta\lambda)$ 인자가 어떤 gate 의 원형인지 말할 수 있다.
- ☐ § 2.7 의 (d) 와 (e) 를 재현해, Adam 이 좌표별로 $\cdot \text{NS}_\kappa$ 가 특이값별로 각각 무엇을 평균화하는지 (그리고 무엇을 보존하는지) 손계산으로 보일 수 있다.
- ☐ optimizer state 를 inference 어휘로 옮길 수 있다: 어떤 의미에서 “훈련의 register file” 이고, optimizer step 이 roofline 의 어디에 있으며, $dW = \delta x^\top$ 가 어떤 serving 커널과 같

은 shape 인지 말할 수있다.

다음 장으로

이 장의 optimizer 들은 “고정된 데이터셋 위를 여러 epoch 도는” 세계에서 태어났다. 그러나 이 라인의 optimizer 는 다른 세계에서 산다: 데이터가 한번만, 순서대로, 되돌릴 수 없이 흘러가는 token stream 위에서 매 step 갱신해야 한다. 그 세계에서 “잘 배우고 있다” 를 재는 자는 loss 수렴이 아니라 **regret** — 사후 최적의 고정 상태 대비 얼마나 뒤처졌는가 — 이고, update rule 을 설계하는 문법은 FTRL 이다. 3 장은 이 online learning 의 언어를 장착한다. [Miras] 가 sequence model 전체를 그 언어로 다시 쓴 논문이므로, 3 장 없이는 13 장이 읽히지 않는다.

ch03. Online learning: OGD, regret, FTRL

이 장의 목표 — 독자가 이 장을 마치면 다음을 할 수 있어야 한다. (1) token stream 위의 학습을 online protocol 로 정식화하고, 각 단계를 autoregressive decode loop 의 단계에 대응시킬 수 있다. (2) online gradient descent(OGD) 의 update 를 쓰고, 그 품질을 regret 로 정량화하며, “sublinear regret” 가 무엇을 보장하고 무엇을 보장하지 않는지 말할 수 있다. (3) Follow-the-Leader 의 실패를 손계산으로 재현하고, FTRL 의 regularizer 가 왜 memory 를 안정화하는지 설명할 수 있다. (4) FTRL 의 두항 (loss 합 + regularizer) 이 [Miras] 의 attentional bias 와 retention gate 자리 임을 지적하고, mirror descent 가 Memora 형 softmax update 의원형임을 알아볼 수 있다.

왜 필요한가 — [Miras] (It’s All Connected, arXiv:2504.13173; 이 책은 framework 이름 Miras 로 통칭한다) 의 § 3 전체가 이 장의 어휘로 쓰여 있다: fixed-state sequence model 의 gradient-based per-token write 를 online gradient descent 한 step 으로 정식화하고 [Miras Eq. 5], 이를 (FTRL Viewpoint) 와 (Learning-Retaining Viewpoint) 라는 두 online-optimization 렌즈로 해석한다 [Miras § 3.2 – 3.3]. [Atlas] (Atlas: Learning to Optimally Memorize the Context at Test Time, arXiv:2505.23735) 의 문제의식 자체가 기존 recurrent model 의 “online nature” 비판이고, Omega rule(→ 14 장) 은 online objective 를 window objective 로 바꾸는 제안이다 [Atlas § 3.2]. 6 장에서만날 Longhorn 은 SSM update 를 online learning 문제의 닫힌 해로 유도하고 (Liu et al. 2025, arXiv:2407.14207), [Titans] (Titans: Learning to Memorize at Test Time, arXiv:2501.00663) 의 per-token memory update 도 online GD 의 재해석이다 [Miras § 3.1]. [NL] (Nested Learning: The Illusion of Deep Learning Architecture, arXiv:2512.24695) 의 level 별 objective 역시 각 level 이 자기 주기의 stream 위에서 푸는 online 문제다. 요컨대 이 장은 Part II 전체가 닫고 설 “stream 위의 최적화” 언어를 공급한다.

3.1 Online protocol: 독자가 이미 돌리고 있는 loop

2 장은 훈련을 (state, update, cost) 를 갖는 객체로 보는 법을 가르쳤다. 거기서 데이터 소비는 batch training(dataset 전체를 미리 두고 매 step i.i.d. mini-batch 를 뽑아 여러 epoch 재방문) 이었다. 이 장은 그 전제를 버린다.

online learning 은 데이터가 스트림으로 한 번씩만 도착하는 설정에서의 학습이다. 분포 가정도, epoch 도, 재방문도 없다. 형식적으로 **online protocol** 은 매 step $t = 1, \dots, L$ 에서 다음 네 단계를 반복한다:

1. 입력을 받는다 (이 라인에서는 key/value 쌍 (k_t, v_t)).
2. 현재 state W_{t-1} 로 예측한다 (memory 읽기 $\mathcal{M}(k_t; W_{t-1})$).
3. loss $\ell_t(W_{t-1})$ 를 관측한다 (“이 예측이 얼마나 틀렸나”).
4. state 를 갱신한다: $W_{t-1} \rightarrow W_t$.

이 loop 를 독자는 이미 매일 돌리고 있다 — autoregressive decode 가 정확히이 구조다. 차이는 단 하나, 4 단계의 내용물이다: 지금까지 inference 에서 “state 갱신” 은 KV cache append(학습 없는, 계산 없는 write) 였고, 이 라인에서는 그것이 gradient step 이 된다. protocol 의 골격은 바뀌지 않는다.

표 3-1 — online protocol 과 decode loop 의 단계별 대응

online protocol	autoregressive decode (독자의 세계)	TTT-line layer (이 라인)
입력 수신	다음 token 도착	(k_t, v_t) 투영
예측	forward pass, logits	memory 읽기 $y_t = \mathcal{M}(q_t; W_t)$
loss 관측	(없음 — 정답을 모름)	inner loss $\ell(W_{t-1}; k_t, v_t)$ — 정답은 v_t 자신
state 갱신	KV cache append	gradient step $W_{t-1} \rightarrow W_t$

표의 3 행이 가장 낫설 것이다 — inference 에는 “정답” 이 없는데 무슨 loss 를 관측하는가? 이 라인의 inner loss 는 외부 정답이 필요 없는 self-supervised 회귀다: “key k_t 를 넣으면 value v_t 가 나와야한다” 는 조건 자체가 loss 가 된다 — $\ell(W; k_t, v_t) = \|\mathcal{M}(k_t; W) - v_t\|_2^2$. stream 의 각 항목이 자기 label 을 들고 도착하는 셈이다 (기원은 5 장 associative memory).

분포 가정이 없다는 점은 장식이 아니다. online learning 이론은 stream 이적대적 (adversarial) 이어도 깨지지 않는 성능 하한 — worst-case 누적 손해 — 을 추구한다 (서빙을 평균 부하가 아니라 tail latency 로 설계하는 감각이다). 실제 token stream 은 i.i.d. 가 아니므로 (주제 전환, 코드 블록, 두 번 다시 안 나오는 needle), 이 보장은 비정상성 (non-stationarity) 에 대한 보험이 된다.

2 장의 경고를 다시 새긴다: 이 라인에서 훈련의 batch 축 역할은 sequence 축이 대신한다. protocol 의 t 는 mini-batch 가 아니라 token 인덱스이고, inner loop 는 문맥의 token 들을 두 번 다시 지나가지 않는 single-pass 학습이다 — online learning 이 그 자연 언어다.

한 가지 예고. 이 장의 모든 식은 “한 token 에 한 step” 의 순차 이상형이다. state 갱신이 gradient step 이 되는 순간 W_t 가 W_{t-1} 에 의존하는 sequential chain 이 생겨 prefill 식 병렬화가 공짜가 아니게 되고, 실전 구현은 전부 이 이상형의 chunk 단위 근사다 — 그 chain 을 GEMM 으로 푸는 문제는 9 장이 담당한다.

3.2 OGD 와 regret: stream 압축의 품질 언어

online protocol 의 4 단계를 채우는 가장 단순한 방법은 2 장의 SGD 를 그대로 이식하는 것이다. **online gradient descent(OGD)** 는 매 step 현재 항목의 loss 에 대한 gradient 로 한 걸음 내려간다 (Zinkevich 2003, ICML):

$$W_t = W_{t-1} - \eta_t \nabla_W \ell_t(W_{t-1}) \quad (3-1)$$

대수적으로는 2 장의 SGD 와 같은 식이고, 다른 것은 데이터 출처다: mini-batch 를 i.i.d. 로 뽑는 대신 stream 이 주는 순서대로 각 항목을 정확한 한 번 소비한다. $\ell_t(W) := \ell(W; k_t, v_t)$ 로 두면 식 (3-1) 은 표준형 (M1)(delta rule 이자 TTT-Linear 의 write) 과 글자까지 같아진다 — [Miras] 의 출발점이다: [Miras Eq. 5] 는 fixed-state sequence model 의 기본 memory update 를 이 식으로 놓고, [Titans] 의 momentary surprise(→ 12 장) 가 이 online GD gradient 의 재해석 임을 명시한다 [Miras § 3.1]. 즉 **OGD 는 (M1) 의 최적화-이론 쪽 이름이다.**

이 알고리즘이 “잘한다” 는 것을 어떻게 정량화하는가? single-pass stream 에는 held-out set 이 없다. online learning 의 답이 **regret** 다: horizon L 까지의 누적 loss 를, 사후에 (in hindsight) 고를 수 있는 최선의 고정 state 와 비교한 상대 손해로 정의한다.

$$\text{Reg}_L = \sum_{t=1}^L \ell_t(W_t) - \min_{W^*} \sum_{t=1}^L \ell_t(W^*) \quad (3-2)$$

비교 대상 W^* 를 **comparator** 라 부른다: stream 전체를 다보고 단 하나 고를 수 있는 최적의 고정 memory 상태다. 식 (3-2) 는 세겹으로 읽는다. 첫째, regret 는 절대 성능이 아니라 상대 손해다 — stream 자체가 압축 불가능하면 comparator 도 손해를 보고 regret 는 그 차이만 잔다. 둘째, comparator 는 고정이다 — “그때그때 바뀌는 oracle” 이 아니라 “최선의단일 압축 상태” 와 비교한다. 셋째, 목표는 **sublinear regret** ($\text{Reg}_L/L \rightarrow 0$), 즉 step 당 평균 손해가 comparator 대비 0 으로 수렴하는 것 — 이때 “online learner 가 최선의 고정 state 를따라잡는다” 고 말한다.

핵심 정리: ℓ_t 가 convex 이고 gradient norm 이 G , 탐색 영역지름이 D 로 유계이면, iterate 를 지름 D 의 볼록 영역 $\mathcal{W}$ 로 projection 하는 $\eta_t \propto 1/\sqrt{t}$ OGD 는 $\text{Reg}_L = O(GD\sqrt{L})$ 를 달성한다 (Zinkevich 2003; 교과서적정리는 Shalev-Shwartz 2011, Online Learning and Online Convex Optimization; Hazan 2019, arXiv:1909.05207). 지름 D 는이 projection 을 통해서만 bound 에 들어온다 — 무제약 memory-layer 식 (3-1) 은 그 projection 이 없는 형태이므로, 보장이 아니라 knob 의 정체를빌려 오는 것이다. $\sqrt{L}$ 은 sublinear 이라 평균 regret 는 $O(1/\sqrt{L})$ 로 사라진다. 직관만 취하면: $1/\sqrt{t}$ 감쇠는 “초반에 크게 적응, 후반에 안정화” 라는 스케줄이고, 적응 (plasticity) 과 안정 (stability) 의 교환이 $\sqrt{L}$ 에서 균형을이룬다.

step size 스케줄에서 고전 이론과 이 라인의 온도차가 보인다. 고전의 $\eta_t \propto 1/\sqrt{t}$ 는 worst-case regret 를 겨냥한 감쇠스케줄이라, memory layer 에 그대로 이식하면 “문맥이 길수록 새 정보를 덜쓰는” 층 — long context 에서 원하는 행동의 정반대 — 이 된다. 이 라인은스케줄을 폐기한다: $\eta_t = \eta(x_t; \Theta)$ 로 step size 를 token 의함수로 만들고, 그 함수를 slow weights 가 outer loop 에서 학습한다 ($\rightarrow$ 4 장). 그러면 η_t 는 수렴용 감쇠 계수가 아니라 per-token write 강도신호 (“중요한 token 은 세게, 뻥한 token 은 흘려보내라”) 가 된다. regret 이론이 주는 것은 보장이 아니라 knob 의 정체 (step size = write 강도) 이고, 이것이 13 장이 η_t 를 “meta in-context learning rate” 라 부르는이유다.

[해설] inference 어휘로: $d_v \times d_k$ matrix memory 는 KV cache 의 고정 크기 손실 압축이고 ($\rightarrow$ 1 장 Rosetta), regret bound 는 그압축의 품질 보증서다 — 이 $O(d^2)$ state 는 stream 을 다 보고 고른최선의 $O(d^2)$ 압축 상태보다 평균적으로 뒤지지 않는다. 단, 보장의단위가 개별 조희가 아니라 stream 전체 평균이라 특정 needle 하나의 회수는 보장하지 않는다 (§ 3.7 에서 다시).

정직하게 한계도 적는다. 첫째, comparator 가 고정이므로 분포가 도중에바뀌는 stream(문서 경계, 주제 전환) 에서는 “최선의 고정 state” 자체가약한 기준이다 (comparator 이동을 허용하는 dynamic regret 확장은 존재만언급한다). 둘째, 보장은 convexity 를 요구하는데, 이 라인의 실제 memory 는 2-layer MLP(표준 deep memory) 라 inner loss 가 비볼록이므로 $O(\sqrt{L})$ 보장은 성립하지 않는다. 실제로 [Miras] 는 FTRL-mirror descent 라는 online convex optimization 의 기계를설계 언어로 수입하되, 자신의변형 (Moneta/Yaad/Memora) 에 대한 regret bound 는제시하지 않는다 — 정당화는 전적으로 실험이다. 이 장이 가르치는 것은보장이 아니라 **설계 어휘**: 이 라인의 모든 update rule 은 “어떤 online 문제를 어떤 online 알고리즘으로 푸는가” 의 답으로 읽을 수 있다.

3.3 FTL: 왜 “지금까지의 최적해” 로 점프하면 안되는가

OGD 는 gradient 한 걸음이라는 최소한의 갱신이다. 반대편 극단은 매 step, 지금까지 본 모든 loss 의 최적해로 점프하는 것이다. 이를 **Follow-the-Leader(FTL)** 라 부른다:

$$W_t = \arg \min_W \sum_{i=1}^{t-1} \ell_i(W) \quad (3-3)$$

겉보기에 FTL 은 이상적이다 — 매 순간 이력 전체에 최선이다. 문제는 두겹이다. 첫째는 비용: 매 token 마다 과거 전체를 다시 최적화하는 것은 decode 마다 KV cache 전체를 재스캔하는 것과 같은 낭비다. 둘째가치명적이다 — FTL 은 **불안정**하다. loss 가 평평한 (linear) 모양이면마지막 항목 하나가 argmin 을 탐색 영역의 반대편 끝으로 던지고, 적대적 stream 은 이를 이용해 learner

를 매 step 진동시켜 regret 를 $\Theta(L)$ (선형, 즉 학습 실패)로 만든다 — § 3.8 에서 여섯 step 손계산으로 재현한다. 한 문장으로: FTL 의 state 는 마지막 token 에 과민하다.

비용도 GEMM shape로 읽어 둘 가치가 있다: 식 (3-3) 의 argmin 은 이력 전체의 least-squares 해라 매 step Gram 역행렬 적용이 드는, OGD 의 rank-1 write 와자릿수가 다른 per-token 비용이다 (Gram 비정칙 가능성이 다음 절 regularizer 의 복선이고, 그 실제 입주자가 6 장의 Mesa-layer 다).

[해설] 이 라인의 족보에는 FTL 의 “전체 이력 최적해” 를유지하는 극한들이 있다: softmax attention 은 전체 이력의 ℓ_2 regression 을 non-parametric 하게 정확히 푸는 해 (상태 = KV cache 그 자체, § 1.6 카탈로그) 이고, Mesa-layer($\rightarrow$ 6 장) 는 같은 objective 를 Newton 법으로 푼다 — 대가는 자라는 cache 와 step 당 최적화 비용이다. recurrent model 들은반대편 끝 (OGD) 에서 출발해 두 극단 사이를 설계한다 — [Miras] 가 “memory learning algorithm” 을 독립 설계 축으로 선언한 그 스펙트럼이다.

3.4 FTRL: regularizer 가 memory 를안정화한다

FTL 의 진동을 고치는 고전적 처방은 argmin 안에 **regularizer** 를더하는 것이다. **Follow-the-Regularized-Leader(FTRL)** 는 다음을푼다:

$$W_t = \arg \min_W \sum_{i=1}^{t-1} \ell_i(W) + \frac{1}{\eta} R(W) \quad (3-4)$$

$R(W)$ (전형적으로 $\frac{1}{2}\|W\|_2^2$) 는 두 가지 일을 한다: 연속된 해 W_{t-1}, W_t 사이 거리를 η 스케일로 묶어 마지막항목의 지배력을 없애고 (안정화), state 크기를 벌점화해 memory 가 무한정자라는 것을 막는다. 적절한 η 아래 FTRL 은 convex 설정에서 다시 $O(\sqrt{L})$ regret 를 회복한다 (Shalev-Shwartz 2011; McMahan 2011, AISTATS; 현대적 통합 정리는 Orabona 2019, A Modern Introduction to Online Learning, arXiv:1912.13213).

FTRL 이 이 책에서 각별한 이유는 다음 계산에 있다. loss 를 각 step 의 gradient 로 선형화하고 $-\hat{\ell}_i(W) = \langle g_i, W \rangle, g_i = \nabla_W \ell_i(W_{i-1}) - R = \frac{1}{2}\|W\|_2^2$ 를넣으면 argmin 이 닫힌 형태로 풀린다:

$$W_t = \arg \min_W \left\langle \sum_{i=1}^{t-1} g_i, W \right\rangle + \frac{1}{2\eta} \|W\|_2^2 = -\eta \sum_{i=1}^{t-1} g_i \implies W_t = W_{t-1} - \eta g_{t-1} \quad (3-5)$$

즉 **linearized loss + ℓ_2 regularizer 의 FTRL 은 정확히 OGD 다** (제약이 없고 $W_0 = 0$ 일 때). [Miras Eq. 7] 이 명시하는 이동치는 두 방향으로 읽는다. OGD 는 FTRL 의 특수경우이므로 두 슬롯 (loss 모양, regularizer 모양) 을 갈아 끼우면 OGD 를 일반화하는 update rule 의공장이 생기고, 역으로 FTRL 의 state 는 본질적으로 **gradient 누적기**다 — 식 (3-5) 의 중간 형태 $W_t = -\eta \sum g_i$ 가 이를노출한다. 이 둘째 독법은 예고편이다: 13 장 Moneta 는 raw 누적기 A_t 와 정규화된 노출 상태 W_t 를 분리하는 FTRL 형 (dual accumulator) update 를쓴다.

이제 [Miras § 3.2] 의 (FTRL Viewpoint) 를 통일 표기로 쓰면 이 장과의관계가 즉시 보인다:

$$W_t = \arg \min_{W \in \mathcal{W}} \sum_{i=1}^t \hat{\ell}_i(W; k_i, v_i) + \frac{1}{\eta_t} R_t(W)$$

첫 항 — 과거 모든 (key, value) 쌍을 얼마나 잘 기억하는가 — 이 [Miras] 가 attentional bias($\rightarrow$ 13 장) 라 이름 붙이는 자리이고, 둘째 항 — memory 의 크기와 갱신을 다스리는 벌점 — 이 retention gate($\rightarrow$ 13 장) 의자리다. [Miras § 3.3] 은 같은 update 를 다른 렌즈 — 최신 쌍 하나만 fit 하되 이전 state W_{t-1} 근처에 머무는 (Learning-Retaining Viewpoint) — 로도 쓰고, 상수 η · 제약 없는 $\mathcal{W} = \mathbb{R}^d$ · 누적 objective 의 strict convexity 아래 두 렌즈의 동치를 증명한다 [Miras Prop. 3.2, App. B]. 주의하라 — 출하되는 구성 (deep memory, data-dependent gate,

제약된 state) 에서는 세 조건 모두 깨지므로, 이 동치는 설계 동기이지 배포물의보장이 아니다. 두 viewpoint 의 본격적 사용법은 13 장이 소유하고, 이 장은그 둘이 online learning 의 표준 기계 (FTRL, proximal step) 임을 못박는다.

Learning-Retaining 렌즈의 최소형 — proximal step — 도 봐 둘 가치가 있다:

$$W_t = \arg \min_W \ell_t(W) + \frac{1}{2\eta_t} \|W - W_{t-1}\|_2^2$$

여기서 ℓ_t 를 선형화하면 explicit gradient step, 즉 식 (3-1) 이그대로 나온다. 선형화하지 않고 정확히 풀면 **implicit GD**(proximal step) 가 되는데, ℓ_2 regression loss 에서는 이것도 닫힌 해 — 유효 step size 가 $\eta_t/(1 + \eta_t k_t^\top k_t)$ 로 자동 감쇠하는 delta rule 형태 — 를 갖는다. Longhorn(Liu et al. 2025, arXiv:2407.14207, State Space Models are Amortized Online Learners) 이 정확히 이 자리의 모델이고 유도는 6 장에서한다 — “SSM 하나가 online learning 문제의 닫힌 해” 라는 문장이 이 장뒤에는 마케팅이 아니라 명세로 들려야 한다.

마지막으로 반대 방향의 확장. [Atlas § 3.2, Eq. 6] 은 기존 recurrent model 전부를 “현재 token 하나의 loss + retention” 을 푸는 online 문제로요약한 뒤, 이 **online nature**(매 step 현재 항목만 greedy 하게최적화) 를 sub-optimal memorization 의 원인으로 지목한다. Omega rule(→ 14 장) 은 loss 항을 최근 c 개 token 의 window 합으로 바꾼다: $c = 1$ 이면 online(delta rule), c 가 문맥 전체면 global 최적화다 [Atlas § 3.2]. FTRL 의 “모든 과거 항” 과 OGD 의 “현재 항 하나” 사이를 window c 가 매개한다 — online learning 스펙트럼 위에서 여섯 논문이 서로 다른점을 고르는 그림이 이렇게 완성된다.

3.5 Mirror descent 와 Bregman divergence

FTRL 의 regularizer 슬롯에 $\|W\|_2^2$ 를 넣는 것은 state 공간이유클리드라는 암묵적 가정이다. state 에 구조가 있으면 (성분이 전부 양수여야한다든가, 합이 고정이라든가) “가깝다” 의 척도부터 바꾸는 것이 자연스럽다— 그 일반화의 도구가 **Bregman divergence** 다. strictly convex 한함수 F 에 대해

$$D_F(W, W') = F(W) - F(W') - \langle \nabla F(W'), W - W' \rangle \quad (3-6)$$

즉 “ W' 에서의 1 차 근사가 W 에서 실제 F 값을 얼마나밑도는가” 이다. $F = \frac{1}{2} \|\cdot\|_2^2$ 이면 D_F 는 squared Euclidean distance 로 환원되고, $F(W) = \sum_j W_j \log W_j$ (negative entropy, 확률 simplex 위) 이면 D_F 는 KL divergence 가 된다. **mirror descent** 는 § 3.4 의 proximal step 에서 Euclidean 별점을 D_F 로 교체한 것이다:

$$W_t = \arg \min_W \langle g_t, W \rangle + \frac{1}{\eta} D_F(W, W_{t-1})$$

F 가 entropy 이고 state 가 simplex 위이면 이 argmin 은 곱셈형 닫힌 해를낳는다:

$$W_{t,j} = \frac{W_{t-1,j} e^{-\eta g_{t,j}}}{\sum_{j'} W_{t-1,j'} e^{-\eta g_{t,j'}}$$

— exponentiated gradient(= multiplicative weights) update 다 (FTRL 과 mirror descent 의 관계는 McMahan 2011). 성질을 눈여겨보라: state 성분은항상 양수이고 매 step 재정규화되어 총합이 보존되므로, forgetting 이 곱셈감쇠 (decay) 가 아니라 **성분들 사이의 질량 경쟁** 으로 일어난다 —어떤 성분이 커지려면 다른 성분이 줄어야 하고, state 는 문맥이 아무리길어져도 원리적으로 발산할 수 없다.

이 기하가 장식이 아니라 실전 선택지인 이유는 제약에 있다. [Miras § 5.2] 는 수치 불안정 (state 값 폭주) 을 막으려 state 를 스케일된 probability simplex(성분 비음수, 합 고정) 안에 가두는 retention 변형을제안한다. Euclidean 세계에서 이 제약은 매 step 별도 projection 을요구 하지만, entropy 기하에서는 simplex 가 update 의 자연 서식지라 곱셈형 update 와 재정규화가

제약을 부수 비용 없이 유지한다. 어느 기하에서 update 하느냐가 어느 제약을 공짜로 얻느냐를 정한다 — regularizer 선택이 곧 기능 (안정성 보장) 선택이라는 FTRL 슬롯 관점의 사례다.

이 기계의 행선지는 13 장이다. Memora 의 KL-retention update $W_t = \text{Softmax}(\alpha_t \log W_{t-1} - \eta_t \nabla_W \ell)$ 는 위 exponentiated-gradient 식에 retention gate 를 결합한 것이고, [Miras § 5.2] 의 f-divergence retention 일반화도 같은 틀 (D_F 자리에 다른 divergence) 이다. mirror descent 를 여기서 만나 두면 13 장의 “memory update 에 웬 softmax?” 라는 당혹이 “entropy regularizer 구나” 로 바뀐다. systems 점점: 유계 · 정규화된 state 는 발산 걱정 없는 memory 설계의 원리적 근거다 (수치 표현 함의는 13 장 systems 절).

3.6 Loss geometry: ℓ_2 , ℓ_1 , ℓ_p , Huber

FTRL 의 두 슬롯 중 regularizer 는 § 3.4–3.5 가 다뤘다. 남은 슬롯은 loss 의 모양이다. residual $r = \mathcal{M}(k_t; W) - v_t$ 가 어떤 함수로 들어가는가에 따라 같은 OGD 라도 “어떤 token 이 state 를 세게 흔드는가” 가 달라진다 — 1 차원 residual 로 축소해 gradient 크기를 보는 것이 빠르다.

표 3-2 — loss 모양별 gradient 크기 $|d\text{loss}/dr|$ (스칼라 residual r , Huber threshold $\delta = 1$)

loss	gradient 크기 식	$r = 0.1$	$r = 1$	$r = 3$
$\ell_2: r^2$	$2 r $	0.2	2	6
$\ell_1: r $	1	1	1	1
$\ell_3: r ^3$	$3r^2$	0.03	3	27
Huber($\delta=1$)	$ r $ (내부), δ (외부)	0.1	1	1

읽는 법: ℓ_2 는 오차에 비례해 쏜다 — 크게 벗어난 token 하나 (오타, 노이즈, 적대적 스펠) 가 state 를 그 크기만큼 흔든다. ℓ_1 은 방향만 쓰고 크기를 버린다 — “이 key 가 있었다” 만 기록하는 극단으로, [Miras] 는 $p = 1$ 을 value-less associative memory 라 부른다 ($\rightarrow$ 13 장). $p > 2$ 는 반대로 큰 오차를 증폭한다 — surprising token 일수록 더 세게 기록한다. Huber 는 threshold δ 안에서 ℓ_2 , 밖에서 ℓ_1 이라 gradient 크기가 δ 에서 포화한다 — 정확히 **per-token gradient clipping** (rate limiter·saturating counter 의 직관) 이다.

matrix memory 일반형도 적어 둔다. residual $r_t = W_{t-1}k_t - v_t$ 에 대해 ℓ_p bias 의 gradient step 은 $W_t = W_{t-1} - p\eta_t(\text{sign}(r_t) \odot |r_t|^{p-1})k_t^\top$ 풀이다 [Miras Eq. 11] — residual 에 element-wise 비선형 (sign, 성분별 거듭제곱) 을 먹인 뒤에도 여전히 rank-1 outer product 로 쏜다 (loss 모양을 바꿔도 GEMM 골격은 보존된다 — § 3.7). 실전 주의 하나: sign·절댓값의 미분 불가능 점 때문에 이 update 를 **통과하는** outer loop backprop ($\rightarrow$ 4 장) 이 불안정해질 수 있어, [Miras Remark 5] 는 tanh 와 $\sqrt{x^2 + \epsilon}$ 근사로 이를 매끄럽게 만든다 — inner update 의 모양이 outer 훈련의 안정성 제약을 받는, 두 loop 가 서로를 구속하는 첫 사례다.

이 축의 제품이 13 장의 Moneta(ℓ_p bias, 매끄럽게 근사된 sign·절댓값) 와 Yaad (Huber bias, token 마다 학습된 threshold δ_t) 다. 이 장은 모양의 어휘만 공급한다 — 어떤 모양이 이기는가는 [Miras] 실험이 답하고, 수치·ablation 은 13 장이 다룬다.

3.7 Systems bridge: 학습된 eviction policy로서의 retention

이 장의 개념들을 독자의 production 어휘로 되감는다.

state 는 **cache** 이고, **retention** 은 **eviction** 이다. 독자가 아는 KV cache eviction 은 명시적 정책이다 — sliding window 로 자르고 quota 로 총량을 막고 score 로 골라 버린다. fast-weight memory 에는 그런 정책 코드가 없다: § 3.4 의 regularizer $R(\text{retention 항})$ 이 그 역할을 하되, 무엇이 얼마나 오래 살아남는가가 discrete 한 규칙이 아니라 **별점 항선택으로 결정된다** (Rosetta 의 “retention gate = 학습된 eviction” 이 이 뜻이다). 별점의 모양이 eviction 의 성격을 정한다: ℓ_2 는 모든 성분을 조금씩 깎는 곱셈 감쇠 (soft), ℓ_1 은 작은 항목을 정확히 0 으로 자르는 절삭 (hard),

KL 은 질량 경쟁 (§ 3.5) — 세 형태 모두 13 장에서 실제 모델로 만나고, 정책을 사람이 튜닝하는 대신 gate 를 만드는 slow weights 가 outer loop 에서 학습한다 (→ 4 장).

regret 와 recall@position 은 같은 것의 두 투영이다. needle-in-a-haystack recall 곡선은 pointwise 측정 (특정 (key, value) 쌍이 N token 뒤에도 회수되는가) 이고, regret 는 aggregate 보장 (stream 전체누적 loss 가 comparator 대비 유계인가) 이다. 압축 state 가 needle 을 못꺼낸다는 것은 그 쌍의 절대 ℓ_t 가 크다는 뜻이고, comparator 가 그쌍을 회수하는 한 이는 regret 의 양의 국소 성분이 된다 (comparator 도 같은 needle 을 놓치면 국소 기여는 0 이하일 수 있다). 그러나 sublinear regret 는 평균의 보장이지 개별 needle 의 보장이 아니다 — “regret 가 낮는데 recall 이나쁜” 모델은 aggregate 최적화가 pointwise 회수를 함의하지 않는다는 사실의 전시일 뿐 모순이 아니다. retention 품질이 long-context 성능을 지배한다는 [Miras] 의 주장 (→ 13 장) 은 이 구분 위에서 읽어야 한다.

고정 comparator 의 사각지대는 non-stationarity 다. 실서비스문맥은 system prompt, 검색 결과, 대화 이력이 한 stream 에 이어 붙는 “서로다른 문서들의 연결” 이라, 최선의 단일 압축 상태가 모든 구간의 평균이라는 애매한 대상이 되고 고정 comparator 기준의 낮은 regret 조차 위안이 못된다. 실전 응답은 두 갈래다: 경계에서 state 를 통째로 비우는 data-dependent retention gate α_t (→ 12 – 13 장), 그리고 주기적으로 state 를 리셋해 sequential chain 을 끊는 설계 (→ 15 장).

비용의 GEMM shape. matrix memory $W \in \mathbb{R}^{d_v \times d_k}$ 와 ℓ_2 loss 에서 OGD 한 step 은 두 조각이다: 읽기 Wk_t (GEMV), 쓰기 $\nabla_W \ell = (Wk_t - v_t) k_t^\top$ (rank-1 outer product) — 2 장의 $dW = (\text{오차}) k^\top$ 그 shape 다. per-token decode 는 state 전체의 read-modify-write 이라 bandwidth-bound 이고, token C 개를 chunk 로 묶으면 rank- C GEMM 이 되어 tensor core 로 이동한다 — 그 변환이 9 장의 전부다. 변형들의 추가 비용도 shape 로 읽힌다: FTRL 형 dual accumulator 는 상주 state 를 두 배로 만들고, loss·retention 교체는 GEMM 본체를 건드리지 않는 값싼 epilogue 다 — sign·절댓값·threshold 마스크는 elementwise 이고, softmax/simplex 재정규화는 축 방향 reduction 을 하나 더 붙일 뿐 GEMM 골격은 그대로다 — 13 – 14 장의 변형들 이전부 같은 chunkwise 기계 위에서 훈련되는 이유다.

3.8 Worked micro-example: FTL 의 진동, FTRL 의 안정화, OGD 와 의동치

수치로 확인하자. 무대는 가장 작은 memory 다: 스칼라 state $w \in [-1, 1]$ (1×1 matrix memory), loss 는 linear $\ell_t(w) = z_t w$. linear loss 는 장난감이 아니다 — [Miras] 분류에서 Hebbian family (linear attention, RetNet, Mamba-2, GLA; → 6 장, 13 장) 의 attentional bias 가 정확히 linear ($\tilde{\ell}_t = -2\langle Wk_t, v_t \rangle$) 라, $k_t = 1$ 로 두면 이 예제는 그 family 의 1×1 버전이다 ($z_t = -2v_t$).

adversarial stream 을 $z_1 = 0.5$, 이후 $z_t = -1, +1, -1, +1, -1$ (교대) 로 잡고 $L = 6$ 까지 돌린다. FTL 은 식 (3-3) 에 따라 $w_t = \arg \min_{w \in [-1, 1]} (\sum_{i < t} z_i) w$, 즉누적합의 반대쪽 끝점 $-\text{sign}(\sum_{i < t} z_i)$ 을 고른다 (이력이 없는 $t = 1$ 은 $w_1 = 0$).

표 3-3 — FTL 의 여섯 step (누적합은 step 시작 시점 기준)

t	$\sum_{i < t} z_i$	w_t (FTL)	z_t	loss $z_t w_t$
1	0	0	0.5	0
2	0.5	-1	-1	+1
3	-0.5	+1	+1	+1
4	0.5	-1	-1	+1
5	-0.5	+1	+1	+1
6	0.5	-1	-1	+1

누적 loss 는 5 다. comparator 는 $\sum_{t=1}^6 z_t = -0.5$ 이므로 최선의 고정 $w^* = +1$, 그 누적 loss 는 -0.5 다. 따라서 $\text{Reg}_6 = 5 - (-0.5) = 5.5$ — step 당 거의 1 씩 horizon 에 비례해자란다. 메커니즘이 표에 그대로 보인다: 마지막 항목이 누적합의 부호를 뒤집을 때마다 FTL 은 도메인의 반대편

끝으로 점프하고, 적대적 stream 은 그점프 직후의 방향을 정확히 벌준다 — state 가 마지막 token 에 과민하다는 § 3.3 의 진단이 이 진동이다.

이제 FTRL 이다. $R(w) = \frac{1}{2}w^2$, $\eta = 0.5$ 로 식 (3-4) 를 풀면 $w_t = -\eta \sum_{i < t} z_i$ (모두 $[-1, 1]$ 안이므로 제약은 발동하지 않는다). 같은 stream 에서 꺾적은 표 없이도 한 줄로 적힌다: $w_1 = 0$, 이후 누적합이 $+0.5$ 와 -0.5 를 오가므로 w_t 는 ∓ 0.25 로만 진동한다. loss 는 $t = 1$ 에 0, 이후 매 step $+0.25$ — 누적 loss 1.25, $\text{Reg}_6 = 1.25 - (-0.5) = 1.75$. 진동 진폭이 1 에서 0.25 로 줄었고 그것을 정하는 것이 η 다 — $1/\eta$ 가 클수록 state 는 덜 움직이고 덜 배운다. plasticity 와 stability 의 교환이 숫자하나로 손에 잡힌다: [Miras] 가 η_t 를 “meta in-context learning rate” 라 부르며 “크면 더 배우고 더 잊는다” 고 한 것 [Miras § 3.3] 이 이 knob 의 이름이다.

끝으로 식 (3-5) 의 동치를 이 숫자로 계산한다. linear loss 의 gradient 는 $\nabla \ell_t(w) = z_t$ 이므로 OGD 는 $w_t = w_{t-1} - \eta z_{t-1}$ 이다: $w_2 = 0 - 0.5(0.5) = -0.25$, $w_3 = -0.25 - 0.5(-1) = +0.25$, $w_4 = 0.25 - 0.5(1) = -0.25$ — 위 FTRL 꺾적과 자리마다 일치한다. “과거 전체의 regularized argmin” 과 “gradient 한 걸음” 이 같은 꺾적이라는 것, 이것이 이 장의 중심동치다 (제약이 발동하는 설정에서는 FTRL 과 OGD 가 갈라질 수 있고, 그 지도는 McMahan 2011 이다).

Hebbian 독법으로 마무리한다: $z_t = -2v_t$ 로 두면 위 OGD 는 $w_t = w_{t-1} + 2\eta v_{t-1}$, 즉 순수 가산 write — 방금 돌린 여섯 step 이 linear attention 의 1×1 decode 꺾적이었던 셈이다.

요약

- online learning 은 stream 위에서 “수신 → 예측 → loss 관측 → state 갱신” 을 반복하는 protocol 로, autoregressive decode 와 loop 모양이 같다 — 새 요소는 state 갱신이 gradient step 이 된다는 것 하나다.
- OGD $W_t = W_{t-1} - \eta_t \nabla_W \ell_t(W_{t-1})$ 는 표준형 (M1) 의 최적화-이론 이름이고, fixed-state sequence model 의 gradient-based per-token write 는 online GD 한 step 으로 읽힌다 [Miras Eq. 5] — Newton(Mesa)·non-parametric(attention)·implicit(Longhorn) 해는 이 memory-learning-algorithm 축의 다른 점들이다 (§ 3.3).
- regret 는 stream 압축 품질의 언어다: 사후 최선의 고정 comparator W^* 대비 누적 손해. convex·유계 설정에서 OGD 는 $O(GD\sqrt{L})$ sublinear regret 를 달성하되 (Zinkevich 2003), 이는 평균 보장이라 개별 needle 회수를 함의하지 않고 비볼록 deep memory 에는 적용되지 않는다.
- FTL(과거 전체의 argmin 으로 점프) 은 linear loss 에서 마지막 token 에 과민해 진동하며 regret 가 $\Theta(L)$ 이다. FTRL 은 regularizer $\frac{1}{\eta}R(W)$ 로 이를 안정화한다.
- linearized loss + ℓ_2 regularizer 의 FTRL 은 정확히 OGD 이고 (식 (3-5)) FTRL 의 state 는 gradient 누적기다. 이 두 슬롯 (loss 모양, regularizer 모양) 이 [Miras] 에서 attentional bias 와 retention gate(→ 13 장) 가 된다 [Miras § 3.2 – 3.3].
- mirror descent 는 “이전 state 에 가깝게” 의 척도를 Bregman divergence(식 (3-6)) 로 일반화하며, entropy 선택은 곱셈형 (softmax 형) update 를 낳는다 — 13 장 Memora 의 KL-retention 이 이 기계다.
- loss 의 모양은 outlier token 이 state 를 흔드는 정도를 정한다: ℓ_2 는 비례, ℓ_1 은 방향만, $p > 2$ 는 증폭, Huber 는 per-token gradient clipping — Moneta 와 Yaad 의 축이다 (→ 13 장).
- retention gate 는 학습된 cache eviction policy 이고, 이 장의 알고리즘변형은 전부 GEMM 골격을 보존하는 elementwise epilogue 교체다.

자가 점검체크리스트

- online protocol 의 4 단계를 쓰고, 각 단계를 decode loop 의 단계와 대응시켜 설명할 수 있다.
- regret 의 정의 (식 (3-2)) 를 쓰고, comparator 의 역할과 “sublinear regret” 의 의미·한계 (평균 보장, convexity 전제) 를 설명할 수 있다.

- 표 3-3 의 FTL 진동을 스스로 재현하고, 왜 linear loss 에서 FTL 이실패하는지 한 문장으로 말할 수 있다.
- linearized FTRL + ℓ_2 regularizer 에서 OGD 를 유도하고 (식 (3-5)), FTRL 의 loss 항과 regularizer 항이 [Miras] 의 attentional bias / retention gate 자리에 각각 대응함을 설명할 수 있다.
- mirror descent 에서 entropy regularizer 가 곱셈형 update 를 주는 이유와, 그것이 왜 발산 불가능한 state 를 만드는지 설명할 수 있다.
- “retention gate = 학습된 eviction policy” , “regret = aggregate 보장 vs recall@position = pointwise 측정” 을 inference 어휘로 옮길 수 있다.

다음 장으로

이 장은 세 knob — step size η , regularizer R , loss 의 모양 — 을 도입했지만 그것을 누가 정하는가는 열어 두었다. 고전 online learning 에서는 사람이 정한다. 이 라인에서는 $\eta_t \cdot \alpha_t$ 가 token 의 함수로서 slow weights 에서나오고, 초기 상태 W_{init} 과 투영 W_K, W_V, W_Q 까지 전부 pretraining 이 학습한다 — online learner 의 hyperparameter 전체가 또하나의 학습 문제의 변수다. 이 구조 (inner loop 와 outer loop) 를 형식화하는것이 bilevel optimization 이고, “이 gate 는 누가 학습하는가?” 에 체계적으로답하는 것이 4 장의 일이다.

ch04. Meta-learning과 bilevel optimization: inner loop vs outer loop

이 장의 목표 — 이 장을 마치면 다음을 할 수 있어야 한다. 1. sequence model 을 bilevel optimization 문제로 형식화하고, 임의의파라미터가 inner loop 소속인지 outer loop 소속인지를 기호 (W vs Θ) 와 역할만으로 판별할 수 있다. 2. “outer loop 가 inner loop 를 통과해 학습한다” 는 문장을 computational graph 수준에서 설명하고, 1-step 예제의 hypergradient 를 손으로 계산할 수 있다. 3. “이 gate 는 누가 학습하는가?” 라는 질문에 함수와 값을 구분해 정확히 답할 수 있다. 4. MAML 의 initialization-as-meta-variable 아이디어를 이 라인의 W_{init} 과 연결할 수 있다.

왜 필요한가 — 6 편 전부가 이 장의 어휘 위에 서 있다. - [Titans] (Titans: Learning to Memorize at Test Time, arXiv:2501.00663) 는 memory module 을 “meta in-context model” 로 정의하고, inner loss 안의 projection W_K, W_V 를 “hyperparameter” 라고 부른다 [Titans § 3.1]. 이 문장은 bilevel 어휘없이 는 파싱되지 않는다. - [Miras] (It’s All Connected, arXiv:2504.13173 — 실제 제목은 It’s All Connected 이지만 이 책은 framework 이름 Miras 로 통칭한다) 의 네 번째 설계측 “memory learning algorithm(= optimizer)” [Miras § 1] 은 “inner loop 의 optimizer 를 무엇으로 고르는가” 라는 질문 그 자체다. - [Atlas] (Atlas: Learning to Optimally Memorize the Context at Test Time, arXiv:2505.23735) 는 라인의 공식 정의를 명문화한다: “the sequence model is a meta in-context learner with two optimization levels” [Atlas § 2]. Atlas 의 “locally optimal” 주장과 Mesa-layer 대조는 inner 문제를 어디까지 푸는가의 문제다. - [TNT] (TNT: Improving Chunkwise Training for Test-Time Memorization, arXiv:2511.07343) 의 local memory 는 학습되는 공유 초기 상태 W_{init} 으로 주기적으로 reset 된다 [TNT § 4.1.1, Eq. 6] — MAML 의 initialization-as-meta-variable 가 그대로 load-bearing 부품이 된 사례다. - [NL] (Nested Learning: The Illusion of Deep Learning Architecture, arXiv:2512.24695) 은 두 level 을 K 개 level 의 nested optimization 으로 일반화한다 [NL Abstract, § 1]. - [Sleep] (Language Models Need Sleep: Learning to Self-Modify and Consolidate Memories, arXiv:2606.03979) 은 ICL 을 meta-learning process 로 보는 정식화를 출발점으로 삼는다 [Sleep § 1].

2 장에서 optimizer 를 (state, update, cost) 를 갖는 객체로 만들었고, 3 장에서 그 객체를 token stream 위에서 도는 online learner 로 바꿨다. 이장은 마지막 남은 구조적 질문에 답한다: **그 online learner 자체는 누가 만들었는가?** 답은 “또 하나의 optimizer 가, 더 느린 시간축에서” 이며, 이 이중 구조를 정확히 말하는 언어가 bilevel optimization 이다. 1 장에서 비형식적으로 도입한 inner loop / outer loop 구분을 여기서 형식화한다.

4.1 두 개의 최적화, 두 개의 시간축

독자의 세계에는 견고한 불변식이 하나 있다: **serving 중에 weight 는 변하지 않는다**. 엔진 설계 전체 — weight 를 한 번 로드하고, 여러 요청이 shared weights 위에서 batching 되고, KV cache 만 요청마다 자란다 — 가 이 불변식 위에서 있다. 이 라인은 이 불변식을 정확히 절반만 폐기한다 (→ 1 장 Rosetta 사진: “폐기되는 불변식”). weight 의 일부 (W , fast weights → 6 장) 는 decode 중 매 token 갱신되고, 나머지 (Θ , slow weights) 는 여전히 동결된다. 어느 쪽에 속하는지가 이 책의 기호 규약 제 1 조 (W vs Θ) 이고, 그 구분의 수학적 실체가 이 절의 내용이다.

bilevel optimization 은 한 최적화 문제의 제약 조건 안에 또 다른 최적화 문제가 들어 있는 구조를 말한다. 일반형은 다음과 같다. 바깥 문제는 Θ 를 움직여 outer loss $\mathcal{L}$ 을 줄이려 하는데, $\mathcal{L}$ 이 의존하는 W^* 가 그 자체로 안쪽 최적화의 해다:

$$\min_{\Theta} \mathcal{L}(\Theta, W^*(\Theta)) \quad \text{s.t.} \quad W^*(\Theta) \in \arg \min_W \ell(W; \Theta) \quad (4-1)$$

식 (4-1) 이 말하는 것은 의존성의 방향이다: Θ 가 바뀌면 안쪽 문제의 정의 자체가 바뀌고, 따라서 그 해 $W^*(\Theta)$ 가 바뀌고, 그 결과 바깥 loss 가 바뀐다. 안쪽 문제를 **inner loop**(최적화 대상 W , objective ℓ), 바깥 문제를 **outer loop**(최적화 대상 Θ , objective $\mathcal{L}$) 라고 부른다. 소문자 ℓ 과 대문자 $\mathcal{L}$ 의 구분은 이 두 loop 의 구분을 시각화한 것이다 (표기규약 § 0).

그런데 이 라인의 실제 모델은 (4-1) 의 argmin 형태가 아니다. inner 문제는 **끝까지 풀리지 않는다**. token 하나가 도착할 때마다 GD 한 step 만 밟고, 그 중간 상태들의 궤적 자체가 출력을 만든다. 즉 우리가 다루는 것은 **trajectory 형태의 bilevel 문제**다:

$$\min_{\Theta} \mathcal{L}(\Theta) = \sum_{t=1}^L \mathcal{L}_t(y_t), \quad y_t = \mathcal{M}(q_t; W_t(\Theta)) \quad (4-2)$$

$$W_t(\Theta) = W_{t-1}(\Theta) - \eta_t \nabla_W \ell(W_{t-1}(\Theta); k_t, v_t), \quad W_0 = W_{\text{init}} \quad (4-3)$$

식 (4-3) 은 식 (M1) 그대로이되, 모든 재료의 Θ 의존성을 명시한 것이다: $k_t = W_K x_t$, $v_t = W_V x_t$, $q_t = W_Q x_t$ 의 projection 행렬이 Θ 의 일부이고, inner learning rate 는 $\eta_t = \eta(x_t; \Theta)$ 처럼 slow weights 가 만드는 token 의 함수이며, 초기 상태 W_{init} 도 Θ 에 속한다. momentum 과 retention 이 붙은 (M2) 도 구조는 동일하다 — β_t, α_t 의 생산함수가 Θ 에 추가될 뿐이다. 식 (4-2) 의 $\mathcal{L}_t$ 는 next-token prediction loss 다. 표기를 단순화했지만 실제로는 y_t 가 backbone 의 상위 layer 들을 거쳐 token 분포가 되며, 그 경로의 파라미터 전부가 Θ 에 속하므로 $\mathcal{L}_t(y_t)$ 로 축약해도 구조는 같다.

두 loop 는 시간축이 다르다. inner loop 는 **한 시퀀스 안에서** token 마다 돈다 — 3 장에서 본 대로, 이것은 독자가 이미 운영하는 autoregressive decoding loop 와 같은 모양이고, 다만 state 갱신이 gradient step 이라는 점만 다르다. outer loop 는 **pretraining 동안** mini-batch 마다 돌고, serving 이 시작되면 멈춘다. 독자의 어휘로 옮기면: Θ 는 컴파일된 커널에 박힌 상수처럼 배포 시점에 고정되는 것이고, W_t 는 KV cache 처럼 요청과 함께 태어나 요청과 함께 죽는 runtime state 다. “training 이 두 번 있다” 가 아니라, **training(outer) 이 학습해서 내놓는 산출물이 또 하나의 작은 learner(inner)** 라는 것 — 이것이 meta-learning 이라는 이름의 이유다. **meta-learning**(learning-to-learn) 은 학습 절차 자체의 구성요소 (초기값, learning rate, update rule, objective 의 파라미터) 를 더 바깥 최적화의 변수로 삼는 기법의 총칭이다.

4.2 hypergradient: gradient 를 통과하는 gradient

outer loop 도 결국 2 장의 optimizer 객체다: state 는 (Θ, m_t, h_t) (AdamW 의 moment buffer → 2 장), update 는 AdamW step, 필요한 입력은 $\nabla_{\Theta} \mathcal{L}$ 이다. 문제는 이 gradient 의 경로다. Θ 는 식 (4-3) 의 inner update **안에** 들어 있으므로, $\nabla_{\Theta} \mathcal{L}$ 을 얻으려면 inner loop 전체를 미분해야 한다. 이렇게 inner 최적화 절차를 관통해 계산되는 outer 변수의 gradient 를 **hypergradient** 라고 부른다.

가장 정직한 계산법은 **unrolling** 이다: 식 (4-3) 의 update 를 $t = 1$ 부터 L 까지 펼쳐 놓으면 하나의 거대한 computational graph 가된다. inner update 도 미분 가능한 연산의 합성일 뿐이므로 $-\nabla_W \ell$ 자체가 graph 의 한 노드다 — 이 graph 전체에 2 장의 backward pass 를 그대로 적용하면 $\nabla_{\Theta} \mathcal{L}$ 이 나온다. 이것이 “outer loop 가 inner loop 를 통과해 학습한다” 의 문자 그대로 의미이다. 독자에게 익숙한 그림으로 옮기면 unrolled inner loop 는 **같은 연산이 L 번 반복되는, data-dependent 분기가 없는 정적 dataflow graph** 다. 정적이라는 사실이 뒤에서 결정적으로 중요해진다 (§ 4.7).

1 step 만 손으로 미분해 보면 hypergradient 의 구조가 드러난다. inner lr 를 장-국소 기호 η_{in} (학습되는 스칼라 상수; 무침자 η 는 outer lr 로 예약되어 있으므로 구분한다) 으로 두고, $g(W) := \nabla_W \ell(W; k, v)$ 라 하자. 한 step $W_1 = W_0 - \eta_{\text{in}} g(W_0)$ 뒤에 outer loss $\mathcal{L}(W_1)$ 을 평가하면, chain rule 로

$$\frac{\partial \mathcal{L}}{\partial \eta_{\text{in}}} = -g(W_0)^\top \nabla_{W_1} \mathcal{L}, \quad (4-4)$$

$$\frac{\partial \mathcal{L}}{\partial W_0} = \left(I - \eta_{\text{in}} \nabla_W^2 \ell(W_0) \right) \nabla_{W_1} \mathcal{L}. \quad (4-5)$$

식 (4-4) 는 “inner gradient 의 방향이 outer loss 를 줄이는 방향과 얼마나 정렬되어 있는가” 를 재고, 식 (4-5) 에는 inner loss 의 **Hessian** $\nabla_W^2 \ell$ 이 나타난다. inner update 가 이미 gradient 를 포함하므로 그것을 다시 미분하면 gradient 의 gradient, 즉 2 차 미분이 튀어나오는 것이다. L step 을 펼치면 식 (4-5) 의 Jacobian $(I - \eta_{\text{in}} \nabla_W^2 \ell_\tau)$ 들이 곱으로 연쇄된다 — RNN 의 BPTT 에서 transition Jacobian 이 연쇄되는 것과 정확히 같은 구조이고, 같은 병 (긴 연쇄의 소실 · 폭발, 그리고 궤적 전체를 저장해야 하는 activation memory) 을 앓는다. 행렬 W 의 경우 Jacobian 이 고차 tensor 가 되지만 구조는 동일하다.

이 병의 표준 처방이 **truncated unrolling** 이다: graph 를 T step 마다 잘라, 자른 지점 이전으로는 gradient 를 흘리지 않는다. 계산과 메모리를 아끼는 대신 hypergradient 가 편향된다 — 잘린 구간 너머로 전파됐어야 할 신호가 0 으로 처리되기 때문이다. 이 트레이드오프를 기억해두면 9 장이 쉬워진다: chunkwise training 의 stale-snapshot 근사 (식 (M4)) 는 chunk 시작 상태에 gradient 의 평가점을 동결하는 별개의 근사이지만, “계산을 아끼는 대신 근사 오차를 낸다” 는 저울질을 truncation 과 공유한다. 단 기제도 knob 방향도 다르다: truncation 은 gradient 의 backflow 를 자르고 길이가 **길수록** 편향이 줄지만, chunk 는 gradient 의 평가점을 동결하고 C 가 클수록 staleness 가 커져 품질이 떨어진다 (정밀한 구분은 → 9 장).

unrolling 의 대안으로 **implicit differentiation** 이 있다: inner 문제가 argmin 까지 풀린다고 가정하면 (식 (4-1) 의 형태), 최적점의 1 차 조건 $\nabla_W \ell(W^*; \Theta) = 0$ 에 implicit function theorem 을 적용해 궤적을 저장하지 않고도 hypergradient 를 얻는다. 대가는 inner Hessian 이 긴선형계를 푸는 비용이며, hyperparameter optimization 과 meta-learning 의 형식적 통합은 Franceschi et al. 2018 (arXiv:1806.04910) 이 정리했다. 이 라인이 implicit 노선을 쓰지 않는 이유는 이제 자명하다: inner 문제가 애초에 argmin 까지 풀리지 않고, 중간 궤적 $W_1, \dots, W_L$ 하나 하나가 y_t 를 만들기 때문이다. 예외가 궤적 대신 매 step 정확한 해를 쓰는 Mesa-layer 이고 (§ 4.5), 그래서 Atlas 가 이를 대조군으로 세운다.

마지막으로 근사의 계보 하나: 식 (4-5) 의 Hessian 항을 통째로 버리고 $\partial \mathcal{L} / \partial W_0 \approx \nabla_{W_1} \mathcal{L}$ 로 쓰는 것이 first-order MAML (FOMAML) 류의 근사다. 정확도와 비용의 저울질은 학습되는 것이 무엇이냐에 따라 달라지는데, 그 “무엇” 의 목록이 다음 절의 주제다.

4.3 learning-to-learn 계보: 무엇을 meta-변수로 삼는가

meta-learning 의 역사는 “outer loop 에 무엇을 넘길 것인가” 의 역사다. 이 라인이 직접 인용하는 조상만 추리면 세 갈래다.

첫째, **Schmidhuber** 의 자기 수정 계보. Schmidhuber 1987 (diploma thesis) 은 학습 절차 자체를 학습 대상으로 삼는 self-referential learning 을 제안했고, Schmidhuber 1992 (Learning to control fast-weight memories, Neural Computation) 는 느린 network 가 빠른

network 의 weight 를 써넣는 구조 — fast weights(→ 6 장) 의 원형 — 를, Schmidhuber 1993(ICANN) 은 자기 자신의 weight 를 읽고 수정하는 self-referential weight matrix 를 제시했다. 이것은 골동품 인용이 아니다. [NL] 은 backprop 자체가 self-referential process 라는 논증과 self-modifying Titans 의 설계에서 Schmidhuber 1992/1993 을 직접 인용한다 [NL § 4, § 7].

둘째, **learned optimizer**. Andrychowicz et al. 2016 (arXiv:1606.04474) 은 update rule 자체를 학습했다: 작은 recurrent network 가 gradient 를 입력받아 Δw 를 출력하고, 그 network 의 파라미터를 outer loop 가 학습한다. “optimizer 는 import 하는 고정 부품이 아니라 학습 가능한 모듈이다” 라는 관점의 원조이며, 2 장에서 optimizer 를 (state, update, cost) 객체로 세운 것은 정확히 이관점을 미리 깔아 둔 것이다. [Miras] 가 네 번째 설계 축으로 “memory learning algorithm(= optimizer)” 을 놓을 때 [Miras § 1], 이 축의 사상적 기원이 여기서다.

셋째, **MAML**. Finn, Abbeel & Levine 2017 (arXiv:1703.03400) 의 MAML(Model-Agnostic Meta-Learning) 은 meta-변수를 단 하나, **초기값**으로 고른다: 새 task 가 오면 초기 값 W_{init} 에서 GD 몇 step 으로 적응하고, “적응 후 성능” 을 outer loss 로 삼아 W_{init} 자체를 학습한다. hypergradient 는 § 4.2 에서 유도한 그대로이며 — 식 (4-5) 의 Hessian 항을 버린 변형이 FOMAML 이다 — 학습이 끝난 W_{init} 은 “어느 task 로든 몇 step 만에 갈 수 있는 출발점” 이 된다. 같은 계보의 후속으로 Hessian 없이 초기값만 학습하는 first-order 변형 (Reptile), 적응 대상을 소수의 context parameter 로 국한하는 context-parameter 변형 (CAVIA) 이 있다 — [Titans § 3.1] 이 자기 계보로 인용하는 명칭들이다 (→ 12 장). 이 아이디어의 이 라인버전이 **meta-learned initial state** W_{init} 이다: memory 의 초기 상태를 난수가 아니라 outer loop 가 학습한 값으로 두는 것. [Titans] 에서는 암묵적 세부였던 이것이 [TNT] 에서는 구조의 기둥이 된다 — local memory 가 shard 경계마다 “shared, learnable initial state W_{init} ” 으로 reset 되고 [TNT § 4.1.1, Eq. 6], reset 이 이전 shard 의 local memory 정보를 실제로 폐기하면서도 (그렇게 sequential chain 을 끊어 shard 병렬성을 얻는다) 치명적 손실이 되지 않는 것은, 복귀지점 W_{init} 이 meta-learn 된 좋은 공통 출발점이고 잃어버린장거리 문맥은 별도의 global memory 가 보완하기 때문이다 (→ 15 장). 독자의 세계로 옮기면 W_{init} 은 세션 시작마다 복원되는 golden snapshot — 모든 요청이 공유하는 초기 상태 이미지 — 이고, MAML 은 그 이미지를 굶는 절차다.

표 4-1 — learning-to-learn 계보: meta-변수의 선택

계보	outer loop 가 학습하는 것 (Θ 쪽)	inner loop	이 라인에서의 대응
Schmidhuber 1987/1992/1993	자기 수정 규칙, fast-weight 를 쓰는 slow net	fast weights 의 갱신	fast weight programming(→ 6 장), self-modifying Titans(→ 16 장)
Andrychowicz et al. 2016	update rule 자체 (작은 net)	대상 모델의 학습	learned $\eta_t, \beta_t, \alpha_t$ 생산 함수; “optimizer = 모듈” 관점 (→ 2 장)
MAML (Finn et al. 2017)	초기값 W_{init}	task 적응 GD 몇 step	W_{init} (Titans 암묵 [Titans § 3.1] → TNT load-bearing [TNT § 4.1.1])
이 라인 (TTT 계열, → 8 장)	위 전부 + projection + inner objective 의 파라미터	token stream 위 online GD	식 (4-2) – (4-3)

전체 조망은 Hospedales et al. 2021 (arXiv:2004.05439) 의 survey 가 표준 참고문헌이다. 표의 마지막 행이 말하듯, 이 라인은 계보의 어느 한 갈래가 아니라 **세 갈래 전부를 한 layer 안에 합류시킨 것**이다.

4.4 이 라인의 bilevel 구조: outer loop는 정확히 무엇을배우는가

이제 6 편 전체를 여는 열쇠 문장을 말할 수 있다: **inner optimizer의 hyperparameter들이, outer loop가 학습하는 data-dependent 함수가 된다.** 고전 훈련에서 learning rate·momentum 계수·weight decay·초기값은 사람이 고르는 hyperparameter였다. 이 라인에서는 그 각각이 outer loop가 학습하는대상으로 바뀐다: learning rate·momentum·weight decay는 token의 함수 $\eta_t = \eta(x_t; \Theta)$, $\beta_t = \beta(x_t; \Theta)$, $\alpha_t = \alpha(x_t; \Theta)$ 가 되어값이 token마다 바뀌고, 초기값은 고정된 학습 파라미터 $W_{\text{init}} \in \Theta$ 가 된다 (값은 상수이되 난수가 아니라 학습된것). 어느 쪽이든 사람이 아니라 outer loop가 고른다. [Titans]는 gate들이 x_t 의 함수로 계산되는 data-dependent 계수임을 명시하고 [Titans § 3.1], inner loss $\ell(W_{t-1}; k_t, v_t) = \|\mathcal{M}(k_t; W_{t-1}) - v_t\|_2^2$ 의 projection에 대해 “parameters W_K and W_V are hyperparameters”라고 못 박는다 [Titans § 3.1]. [Atlas]는 같은구조를 정의로 승격시킨다: inner loop에서는 memory module의 파라미터만 최적화되며 그때 나머지 전부는 고정된 hyperparameter이고, outer loop에서그 나머지 — projection, MLP 등 — 가 최적화된다 [Atlas § 2].

training 무경험 독자의 1 번 질문 — “test-time learner의 learning rate는누가 학습하는가?” — 에 이제 답한다. 핵심은 **함수와 값의 분리**다. 함수 $\eta(\cdot; \Theta)$ 의 파라미터는 Θ 의 일부로서 **outer loop가 pretraining 중에** 학습한다. 값 $\eta_t = \eta(x_t; \Theta)$ 는 **inner loop가 serving 중에** token마다 평가한다. serving에서 함수는 동결되어 있지만 값은 매 token 다르다 — “학습된 learning rate”라는 말은 언제나 함수에 대한 말이다. gate 생산 함수는 실제로는 작은 head(예: low-rank projection + activation)이므로, decode 경로에 GEMV 몇 개가 추가되는 비용으로 읽으면된다.

표 4-2 — 누가 무엇을 학습하는가 (이 라인의 표준 배치)

구성 요소	기호	소속	움직이는 시점
projection 행렬	W_K, W_V, W_Q	Θ (outer)	pretraining 만; serving에선 동결
gate 생산 함수 (inner lr / momentum / retention)	$\eta(\cdot; \Theta), \beta(\cdot; \Theta), \alpha(\cdot; \Theta)$	Θ (outer)	함수는 pretraining 만; 값 $\eta_t, \beta_t, \alpha_t$ 는 매 token 평가 (pretraining forward·serving 모두)
memory 초기 상태	W_{init}	Θ (outer)	pretraining 만; serving에선 reset 목적지
backbone (attention, MLP, LN, embedding)	Θ	Θ (outer)	pretraining 만
memory 상태	W_t	fast (inner)	매 token (pretraining forward·serving 모두), 식 (M1)/(M2)
inner momentum buffer	S_t	fast (inner)	매 token (pretraining forward·serving 모두) (M2)
outer optimizer state	m_t, h_t	outer의 부속 (→ 2 장)	pretraining 만; 배포물에 포함되지 않음

이 표를 기준으로 6 편의 변주를 미리 읽어 두자. [Miras]는 표의 “inner objective ℓ ”과 retention 항을 설계 축으로 열고 (attentional bias → 13 장), [Atlas]는 inner 문제의 범위를

token 하나에서 sliding window 로 넓히고 inner optimizer 를 Muon 으로 바꾼다 (Omega rule → 14 장). [TNT] 는 outer 훈련의 경제학 — 어떤 chunk 크기로 unrolling 을 자를 것인가 — 을 다루고 (→ 15 장), [NL] 은 표의 행들을 아예 재해석한다: momentum 과 AdaGrad 조차 two-level nested optimization 으로 분해되고 [NL § 1], 모델과 훈련 절차 전체가 각자의 context flow 를 가진 nested, multi-level optimization 문제들의 집합이 되며 [NL Abstract], pre-training 자체가 “context 가 전체 pre-training data 인 in-context learning” 으로 재서술된다 [NL § 1]. 두 loop 는 K 개 level 의 스펙트럼으로 일반화되고 level 마다 update frequency (→ 16 장) 가 붙는다. [Sleep] 은 ICL 을 meta-learning process 로 보는 정식화 [Sleep § 1] 를 전제로, serving 중 동결이라는 Θ 의 지위 자체를 wake/sleep lifecycle (→ 17 장) 로 허문다 — “ Θ 는 serving 중 불변” 이라는 이 절의 규칙이 성립하는 마지막 논문이 TNT 이고, 그 규칙의 해체가 라인의 종착점이라는 것까지 보이면 Part II 의 지도는 완성이다.

[해설] 표 4-2 는 inference 엔지니어에게 배포 체크리스트로 읽힌다: 배포물에 들어가는 것은 Θ 전부 (gate 생산 함수 포함) 와 W_{init} 이고, 요청마다 새로 할당해야 하는 것은 W_t 와 S_t (그리고 (M2) 라면 그것이 전부) 다. outer optimizer state m_t, h_t 는 KV cache 가 아니라 훈련 클러스터에 남는다. per-request 상태가 shared-weight batching 을 깨뜨린다는 함의는 10 장과 Part III 의 주제다.

4.5 Attention 은 이미 GD 를 하고 있었다: ICL 과 mesa-optimization

bilevel 구조가 이 라인의 발명품이 아니라는 정황 증거가 있다: **평범하게 훈련된 transformer 안에서 inner loop 가 저절로 생긴다**는 연구들이다. Akyürek et al. 2023 (arXiv:2211.15661) 은 in-context 로 linear regression 을 푸는 transformer 가 GD 나 ridge regression 같은 표준 학습 알고리즘을 내부적으로 구현할 수 있음을 구성적으로 보였고, von Oswald et al. 2023 (arXiv:2212.07677) 은 linear self-attention layer 의 weight 를 적절히 두면 그 layer 의 forward pass 가 in-context 예제들에 대한 regression loss 의 GD 한 step 과 일치함을 보이고, **합성 linear regression 과제에 훈련된 self-attention-only (linear attention) transformer** 가 실제로 그 construction 과 유사하거나 (다층: GD++류 curvature 보정) 단층에서는 weight 수준까지 일치하는 해에도달한다고 보고한다 [von Oswald et al. 2023 abstract, § 3–4]. 후속작 von Oswald et al. 2023 (arXiv:2309.05858) 은 이를 **mesa-optimization** — forward pass 안에서 내부 목적 함수를 세우고 최적화하는, 훈련이 만들어낸 절차 — 로 명명하고, in-context regression 문제를 매 step 정확히 최적해까지 푸는 **Mesa-layer** 를 제안했다. Atlas 는 Mesa-layer 를 “모든 과거 token 에 대해 memory 를 최적화하지만 훈련이 느린” 극한으로 규정하고 자신의 대조군으로 세운다 [Atlas § 3, 각주 1].

이 결과들이 이 장의 문법에서 하는 말은 정확히 이것이다: softmax attention 은 이미 일종의 inner 문제의 **non-parametric** 해다 (KV cache 가 곧 그 “풀이의 상태” 라는 대응은 1 장 Rosetta 사전의 첫 행이다). 그렇다면 6 편의 라인은 무에서 유를 만드는 것이 아니라, attention 이 **암묵적 · 비모수적 · 얇게** 하던 일을 **명시적 · 모수적 (W_t 로 압축) · 깊게** (deep memory, momentum, 학습된 gate) 만드는 프로젝트다. inner loop 를 명시적으로 쓰는 순간 무엇을 얻는가 — objective 를 갈아끼우고 (13 장), 범위를 넓히고 (14 장), optimizer 를 바꾸고 (14 장), 훈련 경제학을 설계할 (15 장) 자유 — 가 Part II 의 줄거리다.

4.6 Worked micro-example: 손으로 도는 두 개의 loop

스칼라 하나로 bilevel 전체를 돌려 보자. memory 는 1×1 행렬 (스칼라) W , 읽기는 $\mathcal{M}(k; W) = Wk$, inner loss 는 이에제에서만 계수 $\frac{1}{2}$ 을 붙여 $\ell(W; k, v) = \frac{1}{2}(Wk - v)^2$ 로 둔다 (미분을 깔끔하게 하기 위한 장-국소 선택이다). outer 데이터는 다음 시나리오다: pair $(k, v) = (1, 1)$ 을 memory 에 쓴 뒤, query $q = 1$ 로 읽어서 목표값 $v^{\text{out}} = 1$ 을 재현해야 한다. outer loss 는 $\mathcal{L} = \frac{1}{2}(W_1 q - v^{\text{out}})^2$. meta-변수는 두 개다: inner lr η_{in} (초기값 0.5) 와 초기 상태 $W_0 = W_{\text{init}}$ (초기값 0).

inner loop (forward). momentary gradient 는 $g_0 = (W_0 k - v) k = (0 \cdot 1 - 1) \cdot 1 = -1$. 한 step:

$$W_1 = W_0 - \eta_{\text{in}} g_0 = 0 - 0.5 \cdot (-1) = 0.5.$$

읽기: $y = W_1 q = 0.5$. outer loss: $\mathcal{L} = \frac{1}{2}(0.5 - 1)^2 = 0.125$. 절반만 기억된 상태다 $-\eta_{\text{in}} = 0.5$ 는 이 key 를 반만 overwrite 한다.

outer loop (hypergradient). 식 (4-4) 와 (4-5) 에 숫자를 넣는다. 먼저 $\nabla_{W_1} \mathcal{L} = (y - v^{\text{out}}) q = -0.5$.

- $\frac{\partial \mathcal{L}}{\partial \eta_{\text{in}}} = -g_0 \cdot \nabla_{W_1} \mathcal{L} = -(-1)(-0.5) = -0.5$.
- Hessian 은 $\nabla_W^2 \ell = k^2 = 1$ 이므로 $\frac{\partial \mathcal{L}}{\partial W_0} = (1 - \eta_{\text{in}} k^2) \nabla_{W_1} \mathcal{L} = (1 - 0.5) \cdot (-0.5) = -0.25$.

두 번째 식에서 “gradient 를 다시 미분하면 2 차 미분이 나온다” 가 숫자로보인다: 괄호 안의 $1 - \eta_{\text{in}} k^2$ 가 바로 식 (4-5) 의 Jacobian 이다.

outer step. outer lr $\eta = 1$ 의 GD 로 η_{in} 만갱신하면 $\eta_{\text{in}} \leftarrow 0.5 - 1 \cdot (-0.5) = 1.0$. inner loop 를 다시 돌리면 $W_1 = 0 - 1 \cdot (-1) = 1$, $y = 1$, $\mathcal{L} = 0$. outer loop 가 단 한 step 만에 “이 과제에는 완전 overwrite 가 정답” 임을학습했다. 실제로 $\eta_{\text{in}} = 1/k^2$ 는 이 key 에 대한 정확한 overwrite — 5 장에서 delta rule 이라 부르게 될 바로 그 지점 — 이며, 학습된 inner lr 가 delta rule 을 재 발견한 셈이다.

관찰 두 가지. (1) $\eta_{\text{in}} = 1$ 이 된 뒤에는 $\partial W_1 / \partial W_0 = 1 - \eta_{\text{in}} k^2 = 0$: 한 step 에완전히 overwrite 되는 key 에 대해서는 초기값으로 gradient 가 아예 흐르지않는다. W_{init} 이 학습할 거리가 있으려면 inner 가 다 지우지못하는 무언가 — 본 적 없는 key, 부분 갱신, 부족한 step 수 — 가 남아있어야 한다는 것을 이 0 이 말해 준다. (2) 차원을 d 로 올리면 inner update 는 $\Delta W = -\eta_{\text{in}}(Wk - v)k^\top$, 즉 (오차) $k^\top$ 모양의 rank-1 outer product 다 — 1 장 Rosetta 사전의 “KV cache append 를 rank-1 GEMM write 로 읽기” 행이 여기서 그대로재사용된다. 이 예제의 모든 산수는 그 GEMM 의 1×1 특수 경우다.

4.7 Systems bridge: 정적 graph, truncation, 그리고 serving 의분리

이 장의 내용이 독자의 roofline 감각과 만나는 지점을 정리한다.

첫째, unrolled inner loop 는 컴파일 가능한 정적 graph 다. 식 (4-3) 을 L 번 펼친 graph 는 같은 연산 블록의 반복이고 data-dependent 제어 흐름이 없다. 따라서 kernel fusion·재배치·tiling 의 대상이 된다 — FlashAttention 이 attention 의 정적 graph 를 tile 단위로 재조직했듯, 이 graph 를 chunk 단위로 재조직한 것이 9 장의 chunkwise-parallel training 이다. 단 결정적 차이가 있다: FlashAttention tiling 은 bit-exact 지만, chunk 는 gradient anchor 를 chunk 시작 상태에 동결하는 **semantic** 근사로서 계산되는 함수 자체를 바꾼다 (식 (M4), → 9 장). § 4.2 의 truncated unrolling 에서 본 “비용 대 편향” 트레이드오프가, truncation 길이가 말던 “비용 대 정확도” 저울질을 chunk 크기 C 가 다시말는 것이다 — 단 gradient backflow 를 자르는 truncation 과 달리 chunk 는평가점을 동결하는 별개 기제이고, C 가 클수록 (길수록이 아니라) 근사가나빠진다.

둘째, outer 훈련의 비용은 궤적의 저장이다. inner loop 를관통하는 backward pass 는 원칙적으로 $W_1, \dots, W_L$ 전 궤적을 activation memory 에 요구한다 (2 장의 activation·recomputation 논의의 직계). naive 하게는 상태 크기 $|W|$ 의 L 배 — matrix memory $d \times d$ 에 $L=32K$ 면 이미 감당 불가 — 이고, 그래서실전은 chunk 경계 상태만 저장하고 chunk 내부를 재계산하는 쪽으로 간다 (→ 9 장). “훈련이 비싼 이유” 의 절반은 FLOP 이 아니라 이 궤적 저장이라는감각을 여기서 얻어 두면 TNT 의 훈련 경제학 (→ 15 장) 이 바로 읽힌다.

셋째, serving 은 두 loop 중 하나만 가져간다. 배포 후에는 outer loop 가 존재하지 않는다 — ∇_Θ 도, m_t, h_t 도, 궤적 저장도없다. 대신 inner loop 가 decode 안으로 들어온다: 매 token, 작은 memory net 의 forward + backward + update (1 장 Rosetta 사전의 “decode = $C=1$ 의 per-token online write + read” 행). 이때의 backward 는 autograd 가 아니라 손으로 유도된 gradient 의 고정 kernel 이라는 점, 그구체적 GEMM 수와 비용 모델은 8 장과 10 장이 담당한다.

표 4-3 — 두 loop 를 (state, update, cost) 객체로

	inner loop	outer loop
state	$W_t (+ S_t)$ — 요청마다 생성 · 소멸	$\Theta + m_t, h_t$ — fleet 전체에 하나
update	식 (M1)/(M2), token 마다	AdamW step(→ 2 장), mini-batch 마다, hypergradient 필요
cost	decode 경로의 추가 FLOP · state RMW 트래픽	pretraining 클러스터의 캐시 저장 + backward
수명	한 시퀀스/세션	pretraining 기간; serving 에 선동결 (해체는 → 17 장)

요약

- bilevel optimization 은 outer 문제의 제약 안에 inner 최적화가 들어 있는 구조이며, 이 라인의 sequence model 은 inner 문제를 argmin 까지 풀지 않고 GD 캐시 자체를 출력으로 쓰는 trajectory 형태의 bilevel 문제다 (식 (4-2) – (4-3)).
- inner loop 는 fast weights W 를 inner loss ℓ 로, outer loop 는 slow weights Θ 를 outer loss $\mathcal{L}$ 로 최적화한다. 두 loop 는 시간축이 다르다: token 마다 vs mini-batch 마다, serving 중 vs pretraining 중.
- outer loop 는 inner update 의 computational graph 를 unrolling 해 backprop 함으로써 hypergradient $\nabla_{\Theta} \mathcal{L}$ 을 얻고, 그 과정에서 inner loss 의 Hessian 이 나타난다 (식 (4-5)). truncation 은 비용을 깎는 대신 hypergradient 를 편향시킨다. 9 장의 chunk 크기 C 는 계산과 정확도를 저울질한다는 점에서 유사하나, gradient backflow 를 자르는 truncation 과 달리 gradient 평가점을 동결하는 별개 기제이고 C 가 클수록 근사가 나빠진다 (방향이 반대).
- meta-learning 계보의 세 갈래 — Schmidhuber 의 자기 수정, learned optimizer, MAML 의 학습된 초기값 — 가 이 라인에서 각각 self-modifying 구조 (16 장), 학습된 gate 생산 함수, W_{init} (TNT 의 reset 목적지) 으로 합류한다.
- 열쇠 문장: inner optimizer 의 hyperparameter 들은 outer loop 가 학습한다 — $\text{gate}(\eta_t, \beta_t, \alpha_t)$ 는 token 의 data-dependent 함수로 (값이 매 token 바뀐다), projection 과 W_{init} 은 고정된 학습 파라미터로. “누가 학습하는가” 는 항상 함수 (outer, pretraining) 와 값 (inner, serving) 을 나눠 답한다.
- 평범한 transformer 의 ICL 이 이미 암묵적 GD 라는 결과들 (Akyürek; von Oswald; mesa-optimization) 이 이 라인의 정당화 근거이며, 6 편은 그 암묵적 inner loop 를 명시적 · 모수적 · 깊게만드는 프로젝트다.

자가 점검체크리스트

- ☐ 식 (4-2) – (4-3) 을 백지에 쓰고, 각 기호가 W/Θ 어느 쪽 소속인지 표시할 수 있다.
- ☐ 1-step inner update 에 대해 $\partial \mathcal{L} / \partial \eta_{\text{in}}$ 과 $\partial \mathcal{L} / \partial W_0$ 를 유도하고, 후자에 Hessian 이 나타나는 이유를 설명할 수 있다.
- ☐ “이 모델의 learning rate 는 학습된다” 는 문장을 함수/값구분으로 풀어 말하고, serving 중에 무엇이 동결되고 무엇이 token 마다 변하는지 답할 수 있다.
- ☐ MAML 의 meta-변수 선택과 TNT 의 W_{init} reset 이 왜 같은 아이디어인지 설명할 수 있다.
- ☐ truncated unrolling 의 트레이드오프를 말하고, chunk 크기 C 가 왜 그사촌인지 한 문장으로 연결할 수 있다.

- inner/outer loop 를 inference 어휘로 옮길 수 있다: Θ = 컴파일된 커널의 상수 (배포 시 고정), W_t = KV cache 형 runtime state, inner loop = backward 가 들어온 decode loop, outer loop = 그 decode loop 전체를 미분하는 pretraining.

다음 장으로

이 장은 “누가, 언제 최적화하는가” 를 답했지만, inner loss 가 왜 하필 $\|\mathcal{M}(k; W) - v\|_2^2$ 꼴인지 — 즉 무엇을 최적화하는가 — 는 건드리지 않았다. 그 답은 60 년 된 associative memory 이론에 있다: key 에서 value 를 연상해 내는 장치로서의 행렬, outer-product 로 쓰고곱셈으로 읽는 Hopfield – Hebbian 계보, 그리고 그 한계 (crosstalk) 를 고치는 delta rule. 5 장은 이 계보를 복원해 inner loss 의 기본형과 “memory 가 넘친다” 의 수학적 의미 (capacity) 를 확정하고, 그로써 6 장에서 linear attention 을 “결함 있는 associative memory” 로 읽을 준비를 마친다.

ch05. Associative memory: Hopfield에서 delta rule까지

이 장의 목표 — 이 장을 마친 독자는 다음을 할 수 있다. 1. KV cache 와 matrix memory 를 하나의 associative memory 추상화의 양극단 (무손실 non-parametric vs 고정 크기 parametric) 으로 배치하고, 그 trade-off 를 byte 와 FLOP 수치로 말할 수 있다. 2. outer-product write 의 crosstalk 을 $d = 2$ 예제에서 손으로 계산하고, 이것이 [Titans § 2] (Titans: Learning to Memorize at Test Time, arXiv:2501.00663) 가 말하는 “memory overflow” 의미시적 실체임을 설명할 수 있다. 3. delta rule 을 ℓ_2 associative loss 의 1-step gradient descent 로 유도하고, 그것이 식 (M1) 의 linear memory 전개형임을 보일 수 있다. 4. Hopfield → dense Hopfield → softmax attention 으로 이어지는 capacity 개선의 사슬을 서술하고, 그 사슬 위에서 [Atlas § 3.1] (Atlas: Learning to Optimally Memorize the Context at Test Time, arXiv:2505.23735) 의 capacity 정리와 feature map 이 왜 그런 모양인지 예측할 수 있다.

왜 필요한가 — associative memory 는 이 라인이 스스로 선언한 기초추상화다. [Miras Def. 3.1] (It’s All Connected: A Journey Through Test-Time Memorization, Attentional Bias, Retention, and Online Optimization, arXiv:2504.13173 — 이 책은 framework 이름 Miras 로 통칭한다) 과 [NL Def. 1] (Nested Learning: The Illusion of Deep Learning Architecture, arXiv:2512.24695) 은 sequence model 전체를 “objective 아래에서 학습되는 mapping $\mathcal{M} : K \rightarrow V$ ” 로 정의하고, [Titans § 2] 는 linear attention 의 additive write 가 “memory overflow” 를 일으킨다는 비판 — 곧 이 장의 crosstalk 논의 — 위에 momentum 과 gate 를 쌓는다. [Atlas § 3.1] 의 capacity 정리 (Prop 1, Thm 1, Prop 2) 와 polynomial/exponential feature map 은 고전 Hopfield capacity 와 dense Hopfield 의 energy 사슬을 그대로 재사용하며, Atlas 와 TNT 는 related work 에서 자신들의 정식화가 Hopfield 1982 의 associative memory 개념에 “architecturally founded” 되어 있다고 같은문장으로 명시한다 [Atlas App. A; TNT App. A] ([TNT] = TNT: Improving Chunkwise Training for Test-Time Memorization, arXiv:2511.07343). [NL § 1] 은 “back-prop 과 momentum 까지 전부 associative memory” 라는 확장으로 이추상화를 라인의 끝까지 밀어붙인다. 이 장이 없으면 Part II 의 어떤 정의도출발점을 갖지 못한다.

5.1 key → value 연상: 독자가 이미 매일 서빙하는 연산

독자는 associative memory 를 이 장에서 처음 배우는 것이 아니라 이미 운영하고 있다. decode 한 step 에서 attention 이 하는 일 — query q_t 를 cache 에 쌓인 key 들과 대조해서 연관된 value 를 꺼내 오는 것 — 이 정확히 연상 (association) 이다. **associative memory** 는 key 집합 $K \subseteq \mathbb{R}^{d_k}$ 와 value 집합 $V \subseteq \mathbb{R}^{d_v}$ 사이의 mapping 을 저장했다가, query 가 주어지면 연관된 value 를 복원하는 시스템이다. 이 라인은 이 고전적 개념을 그대로 정식화해서 출발점으로 삼는다: [Miras Def. 3.1] 은 associative memory 를 operator $\mathcal{M} : K \rightarrow V$ 로 정의하고 그 mapping 을 학습시키는 objective 를 attentional bias 라고 부르며 (정식화의 소유는 13 장), [NL Def. 1] 도 같은 정의

를 채택한다. 이 장은 그 정의가 딥고 선고전 — 1960 – 80 년대의 correlation matrix memory, Hopfield network, delta rule — 을 다룬다.

associative memory 는 두 축으로 분류된다. 첫째 축은 **무엇을연상하는가**다. key 와 value 가 서로 다른 대상이면 **hetero-associative**(예: 이름 → 얼굴, key 벡터 → value 벡터), 손상된 pattern 으로부터 그 pattern 자신을 복원하면 **auto-associative** 다. KV cache 도, 이 라인의 memory 도 전부 hetero-associative 이고, § 5.3 의 Hopfield network 는 auto-associative 다. 둘째 축은 **어떻게 저장하는가**다. 모든 쌍을 그대로 분리 보관하면 non-parametric(KV cache 가 정확히 이것이다), 고정 크기 parameter 덩어리하나에 겹쳐 쓰면 parametric 이다. 이 장의 주인공은 후자다. 표기는 책표준을 따른다: memory 상태는 W , 읽기는 $y = \mathcal{M}(q; W)$ 이며, 이 장의 memory 는 대부분 linear 여서 $\mathcal{M}(q; W) = Wq$ 다.

이 장의 위치도 분명히 하자. 2 – 4 장은 훈련의 커리큘럼이었다: gradient 와 backward pass, optimizer 라는 stateful 객체, online 프로토콜과 regret, 그리고 두 loop. 이 장은 기억의 커리큘럼의 출발점이다: 무엇을 저장하고, 어디에, 몇 개나 담기고, 넘치면 어떻게 되는가. 두 커리큘럼 은 § 5.5 에서하나로 합류한다 — 좋은 write rule 이 정확히 1-step gradient descent 이기때문이다. 그 합류 지점이 식 (M1) 이고, 이 라인의 여섯 논문은 전부 그 지점위에 서 있다.

systems 관점에서 이 분류를 다시 읽으면, long context 문제는 저장 방식선택의 문제가 된다. non-parametric 저장은 정확하지만 state 가 $O(L)$ 로자란다 — KV cache 의 byte 수를 매일 계산하는 독자에게 새로운 이야기가아니다. parametric 저장은 state 가 $O(1)$ 로 고정되는 대신 어딘가에서반드시 정보를 잃는다. 그래서 이 장의 질문은 이것이다: **고정 크기 W 에 (k, v) 쌍을 몇 개나, 얼마나 정확히 넣을 수 있고, 넘치면 어떤방식으로 망가지는가**. 이 질문의 두 이름이 capacity 와 crosstalk 이다.

한 가지 주의를 미리 두자. Rosetta-Stone 사전 (→ 1 장) 의 대응에는 성격이있고, “동일” 인 것과 “유비” 인 것을 섞으면 잘못된 직관이 이식된다. attention lookup ↔ memory read 는 같은 추상화의 재서술이므로 안심하고기대도 된다. 그러나 KV cache append ↔ memory write 는 유비이고, 그 차이가이 장의 논점 그 자체다: cache 는 append-once 라서 서로 다른 entry 가간섭하지 않지만, 이 장의 write 는 같은 좌표 공간 위에 겹쳐 쓴다 —처음에는 blind 하게 (§ 5.2), 나중에는 read-modify-write 로 (§ 5.5). 간섭이없는 세계에서 온 독자가 이 장에서 배워야 하는 것이 바로 간섭의 물리다.

5.2 outer-product memory: 행렬 하나에 여러 쌍을 겹쳐쓰기

parametric 저장의 가장 오래된 답은 각 쌍을 outer product 로 만들어 전부더하는 것이다 (**correlation matrix memory**, Kohonen 1972, IEEE Trans. Computers; Anderson 1972). m 개의 쌍 (k_i, v_i) 를 저장하면

$$W = \sum_{i=1}^m v_i k_i^\top \in \mathbb{R}^{d_v \times d_k} \quad (5-1)$$

이고, 읽기는 GEMV 한 번이다. key 들이 unit norm 이라고 하고 $q = k_j$ 로읽으면

$$Wk_j = v_j + \sum_{i \neq j} (k_i^\top k_j) v_i. \quad (5-2)$$

첫 항이 원하는 value 이고, 둘째 항이 **crosstalk** 이다: 다른 key 와의내적 크기만큼 다른 value 들이 섞여 들어오는 간섭. key 들이 서로 orthogonal 하면 crosstalk 은 정확히 0 이고 복원은 완벽하다. 그러나 $\mathbb{R}^{d_k}$ 에는 서로 orthogonal 한 벡터가 d_k 개뿐이므로, 이방식의 exact 저장은 $m \leq d_k$ 쌍까지만 — capacity 가 $O(d)$ 라는 첫감각이 여기서 나온다. key 가 orthogonal 하지 않은 일반적인 경우 (random unit key) 에는 쌍별 내적이 대략 $1/\sqrt{d_k}$ 크기이므로, m 개를겹쳐 쓰면 신호 대 잡음 비가 $\sqrt{d_k/m}$ 꼴로 떨어진다. 즉 m 이 d_k 에 접근하기 한참 전부터 모든 read 가 조금씩 오염된다. 숫자로옮기면: $d_k = 128$ 에서 $m = 32$ 쌍이면 $\text{SNR} \approx 2$, $m = 128$ 이면 $\text{SNR} \approx 1$ — 신호와 잡음 이 같은 크기가 된다. retrieval 품질이 요구 수준 아래로 떨어지기 시작하는 m 이 곧 그 memory 의 유효 slot 수다.

[해설] 이 실패 모드는 독자가 아는 cache 의 실패 모드와 다르다. cache miss 나 hash collision 은 특정 entry 가 없거나 틀리는 국소사건이지만, crosstalk 은 **모든 read 가 동시에 점진적으로 오염되는** 전역 사건이다. “cache 가 가득 찼다” 에 해당하는 경계가 명시적으로 없고, 품질이 연속적으로 저하된다. 뒤에서 볼 retention gate(→ 13 장) 는 이 암묵적저하를 명시적 · 학습된 eviction 으로 바꾸려는 시도다.

식 (5-1) 을 online 으로, token 이 하나 올 때마다 갱신하는 형태로 쓰면

$$W_t = W_{t-1} + v_t k_t^\top \quad (5-3)$$

이고, 이것을 **Hebbian write** 라고 부른다 (Hebb 의 “함께 발화하는 뉴런은 함께 배선된다” 는 학습 원리에서 온 이름이다). 두 가지 성질이 결정적이다. 첫째, **blind write** 다: 쓰기 전에 읽지 않는다. 이 key 가 이미 저장되어 있는지 확인하지 않고 무조건 더한다. 같은 key 가 두 번오면 value 가 이중으로 쌓인다. 둘째, local-incremental 이다: 현재 token 의 k_t, v_t 만 필요하고 과거를 다시 볼 필요가 없다 — 그래서 하드웨어 친화적이다.

그리고 이 장의 첫 번째 punchline: 식 (5-3) 은 6 장에서 만날 **linear attention 의 state update 와 문자 그대로 동일하다**. [Titans § 2] 가 linear Transformer 를 비판하는 문장이 정확히 이 지점을 겨냥다: 재귀식이 key-value 를 matrix memory 에 “additively compress” 하는 구조라서, long context 에서는 이 additive 본성이 memory overflow 를 일으켜 성능을 크게 훼손한다는 것이다. Titans 가 말하는 overflow 의 미시적 실체가 식 (5-2) 의 crosstalk 누적이다. 60 년 된 correlation memory 의 결함이 2020 년대 efficient attention 의 결함으로 그대로 재등장한다.

Rosetta-Stone 대응도 여기서 정확해진다. KV cache append 도 write 지만, 각쌍이 분리된 slot 에 저장되므로 간섭이 없다 (무손실 append-once). Hebbian write 는 같은 $d_v \times d_k$ 좌표 공간 위에 겹쳐 쓴다 (간섭 = crosstalk). 그리고 shape 를 보라: $v_t k_t^\top$ 는 rank-1 outer product 로, 2 장의 backward pass 에서 본 $dW = \delta x^\top$ 와 같은 GEMM shape 다. “훈련의 gradient write 와 memory write 는 같은 연산” 이라는 이책의 반복 주제가 여기서 처음 물리적으로 드러난다.

crosstalk 의 크기가 key 들의 기하에 전적으로 달려 있다는 사실은 설계여지를 하나 연다. 고전 문헌에서 key 는 주어진 데이터였다. 그러나 이라인의 모델에서 key 는 학습된 projection 이 만든다: $k_t = W_K x_t$ 이고, W_K 는 slow weights Θ 의 일부다 (→ 4 장). 즉 outer loop 는 pretraining 동안 crosstalk 이 덜 생기도록 key 를 벌려 놓는 방향으로 W_K 를 학습할 수 있다 — inner 문제의 “데이터” 처럼 보이던 것이 사실은 outer loop 가 학습하는 함수의 출력이며, 4 장의 “inner 문제의 hyperparameter 는 outer 에서 학습되는 함수가 된다” 는 명제가 여기서 첫구체적 사례를 얻는다. 다만 outer loop 가 고칠 수 있는 것은 key 의 평균적기하까지다. 고정 크기 행렬에 선형독립 방향이 d_k 개뿐이라는 상한 (§ 5.5) 은 어떤 projection 도 우회하지 못한다.

5.3 Hopfield network: energy 로 본 memory 와 최초의 capacity 상수

Hopfield 1982 (PNAS) 는 auto-associative 문제 — 손상된 pattern 에서 원본을 복원하기 — 를 energy 최소화 로 정식화했다. state 는 이진 벡터 $s \in \{-1, +1\}^d$, 저장할 pattern 은 $x_1, \dots, x_m \in \{-1, +1\}^d$, weight 는 또다시 Hebbian 이다: $W = \sum_i x_i x_i^\top$ (대각 성분은 0 으로 둔다). 여기에 **energy function**

$$E(s) = -\frac{1}{2} s^\top W s$$

를 정의하면, 뉴런을 하나씩 $s_j \leftarrow \text{sign}((Ws)_j)$ 로 갱신하는 dynamics 가 E 를 단조 감소시킨다. 저장된 pattern 들은 (memory 가 제대로 작동하는한) E 의 local minimum, 즉 **attractor** 가 된다. 손상된 입력에서 시작해 energy 의 내리막을 따라가면 그 초기 상태가 속한 attraction basin 의 저장 pattern 으로 수렴한다 — 대개 가장 가까운 pattern 이지만, 보장되는 것은 energy 감소와 어떤 attractor 로의 수렴이지 최근접 선택이 아니다 (spurious attractor 로 갈 수도 있다 → 아래). content-addressable memory 의 원형이다.

주소로 찾는 memory(load/store)와 내용으로 찾는 memory의 구분은 독자에게 이미 익숙하다 — TLB나 cache의 tag match가 후자, 즉 content-addressable이다. Hopfield network는 content-addressable memory를 학습 가능한 신경망으로 구현한 최초의 사례군에 속하고, 이 라인이 “memory”라는 단어를 쓸 때의 의미는 언제나 이쪽이다: 주소가 아니라 query의 내용이 무엇을 꺼낼지 정한다.

systems 독자를 위한 재서술: Hopfield의 read는 lookup한 번이 아니라 **GEMV + sign**을 수렴할 때까지 반복하는 **fixed-point iteration**이다. 읽기가 반복 계산이라는 것, 그리고 “복원 품질”이 energy 지형의 기하에 달려 있다는 것이 이후 이야기의 씨앗이다.

capacity는 어떤가. Hopfield 1982는 pattern 수가 약 $0.15d$ 를 넘으면 회복 오류가 심각해진다고 실험적으로 보고했고, 이후 statistical mechanics 분석은 임계값을 약 $0.138d$ 로 확정했다 (Amit, Gutfreund & Sompolinsky; Phys. Rev. Lett. 1985가 임계비 약 0.14 를 보고, 0.138 은 Ann. Phys. 1987의 정밀화). 흔히 “capacity $\approx 0.14d$ ”로 요약된다. 더 넣으면 우아하게 저하되는 것이 아니라 임계점을 지나며 회복 자체가 붕괴한다.

임계 아래라고 깨끗한 것도 아니다. energy 지형에는 저장한 적 없는 가짜 minimum — 저장 pattern들의 홀수 개 혼합 같은 **spurious attractor** — 가 함께 생기고, 초기 상태가 나쁘면 read는 거기로 수렴한다. 그리고 임계를 넘으면 저장 pattern들이 attractor 자격 자체를 잃는다 (혼합 상태 분석과 포화 붕괴 모두 같은 AGS 분석 계열의 표준 결과다; Amit, Gutfreund & Sompolinsky, Phys. Rev. A, 1985). § 5.2의 crosstalk이 모든 read의 점진적 오염이었다면 Hopfield의 과적재는 절벽이다 — 같은 $O(d)$ 병목이라도, 망가지는 모양은 write rule과 read dynamics에 따라 다르다.

여기서 교훈은 상수 0.14 가 아니라 스케일이다. **parameter**는 d^2 개인데 저장 능력은 $O(d)$ 다. 저장 능력이 parameter 수가 아니라 key 공간의 차원 (rank)에 묶여 있다는 것 — 이 병목의 정체는 § 5.5 말미에서 [Atlas § 3.1 Prop 1]로 정확해지고, Atlas 원문 스스로 자신의 결과를 “Willshaw model과 Hopfield network의 고전적 capacity 결과와 일치하며, capacity가 입력 embedding의 rank에 묶인다”고 자리매김한다 [Atlas App. C, Prop 1 증명].

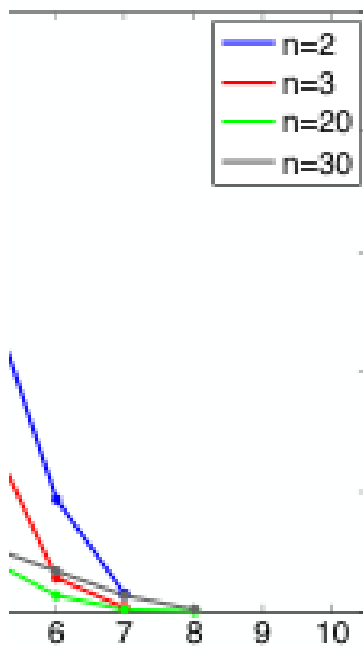
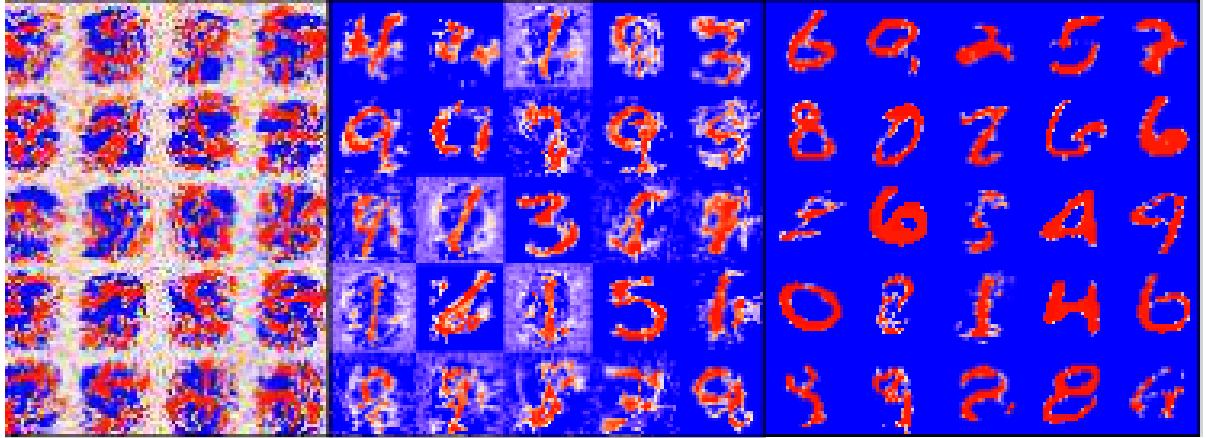
5.4 dense/modern Hopfield: energy를 바꾸면 capacity가 바뀐다 — Atlas의 이론적 골격

고전 Hopfield energy는 더 일반적인 형태 $E(s) = -\sum_{i=1}^m F(x_i^\top s)$ 에서 $F(z) = z^2/2$ 인 특수경우로 볼 수 있다 (전개하면 식 (5-1)의 이차형식이 나온다). Krotov & Hopfield 2016 (arXiv:1606.01164)의 **dense associative memory**는 이 F 를 고차 다항식 $F(z) = z^n$ 으로 바꾸는 한 수로 capacity를 폭증시킨다: 차수 n 의 energy에서 capacity는 d^{n-1} 스케일로 커진다 — 고정 오류율 기준으로 저장 가능 pattern 수가 상수 $\times d^{n-1}$ (상수는 허용 오류율에 따른다)이고, 무오류 기준에서는 로그 인자가 붙는다 [Krotov & Hopfield 2016 Eq. 5–6]. $n=2$ 에서 고전 Hopfield의 $0.14d$ 가 복원된다.

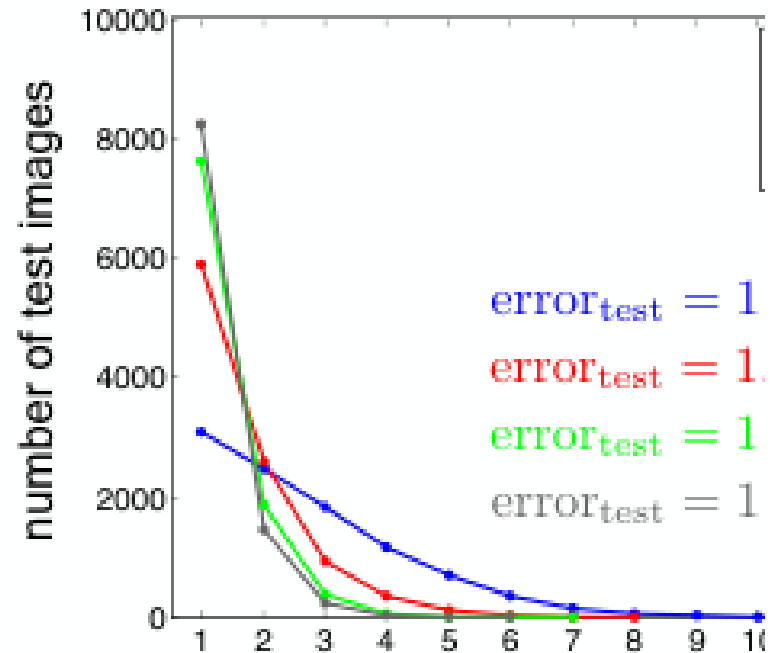
왜 그런지의 직관은 kernel이다. $(x^\top s)^n = \phi_n(x)^\top \phi_n(s)$ — 차수 n monomial feature map의 내적이다. 즉 energy의 차수를 올리는 것은 key들을 더 높은 차원의 공간으로 lift해서 “서로 orthogonal할 자리”를 늘리는 것과 같다. § 5.2에서 capacity를 제한한 것이 “orthogonal한 방향의 개수 $= d_k$ ”였으므로, 공간을 d^n 차원으로 키우면 한계도 따라 올라간다.

이 차수 효과는 저장되는 memory의 성격 변화로도 눈에 보인다 (그림 5-1). 차수 n 이 작으면 ($n=2, 3$) 각 memory는 여러 패턴에 걸친 분산된 feature에 가깝고, n 을 크게 키우면 ($n=20, 30$) 개별 패턴에 날카롭게 정렬된 prototype이 된다. Krotov & Hopfield 2016은 이것을 “feature-to-prototype transition”이라 부르는데, energy 봉우리가 뽕죽해질수록 서로 다른 패턴의 attractor가 더 잘 분리되어 더 많은 패턴을 담을 수 있다는 것 — 즉 이 sharpening이 d^{n-1} 스케일링의 기하학적 실체다.

그림 5-1 — dense associative memory에서 energy 차수 n 을 올리면 저장된 memory의 성격이 바뀐다: 작은 $n(2, 3)$ 에서는 여러 digit에 걸친 분산된 feature, 큰 $n(20, 30)$ 에서는 하나의 패턴에 정렬된 prototype이 된다 (아래 히스토그램은 각 memory가 투표하는 class 수의 분포). energy가 날카로워질수록 attractor가 분리되어 capacity가 커지는 메커니즘의 시각화다. 출처



th $\xi_{\alpha}^{\mu} > 0.99$



number of memories making the

그림 5-1 — dense associative memory 에서 energy 차수 n 을 올리면 저장된 memory 의 성격이 바뀐다: 작은 $n(2, 3)$ 에서는 여러 digit 에 걸친 분산된 feature, 큰 $n(20, 30)$ 에서는 하나의 패턴에 정렬된 prototype 이 된다 (아래 히스토그램은 각 memory 가 투표하는 class 수의 분포). energy 가 날카로워질수록 attractor 가 분리되어 capacity 가 커지는 메커니즘의 시각화다. 출처: Krotov & Hopfield, Dense Associative Memory for Pattern Recognition (arXiv:1606.01164), 원문 Fig.2 — 원저자의 그림 (third-party), 본서의 결과가 아님.

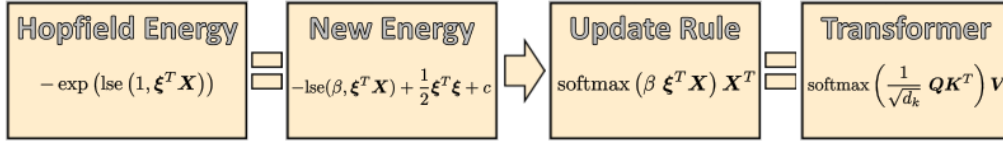


그림 5-2 — modern Hopfield network 의 energy 를 이진에서 연속상태로 일반화하면 (가운데 두 상자: log-sum-exp energy) 그 1-step update rule 이 정확히 transformer 의 softmax attention 이 된다 (오른쪽 두 상자). § 5.4 의 “exponential energy 극한 = softmax attention” 사슬을 한 줄로 보여준다. 출처: Ramsauer et al., Hopfield Networks is All You Need (arXiv: 2008.02217), 원문 Fig.1 —원저자의 그림 (third-party), 본서의 결과가 아님.

처: Krotov & Hopfield, Dense Associative Memory for Pattern Recognition (arXiv:1606.01164), 원문 Fig.2 — 원저자의 그림 (third-party), 본서의 결과가 아님.

이 사슬의 극한이 현대적 결말이다. F 를 exponential 로 보내면 (연속 state + log-sum-exp energy), Ramsauer et al. 2021 (arXiv:2008.02217) 이 보였듯 1-step retrieval update 가 정확히 softmax attention 의 형태가 되고 (pattern 을 선형사상으로 key/value 화하고 softmax 의 역온도를 $1/\sqrt{d_k}$ 로 두는 조건 [Ramsauer et al. 2021 Eq. 10, App. A.4]), capacity 는 d 에 exponential 로 커진다 (저장 가능 pattern 수의 하한이 1 보다 큰 상수의 $(d-1)/4$ 거듭제곱 꼴 [Ramsauer et al. 2021 Thm 3]). 즉 Transformer 의 attention 은 exponential energy 를 갖는 modern Hopfield network 의 retrieval 로 읽을 수 있다.

이 대응은 read 의 비용 구조까지 설명한다. 고전 Hopfield 의 read 는 수렴까지도는 fixed-point iteration 이었다 (§ 5.3). Ramsauer et al. 2021 은 exponential energy 아래에서는 한번의 update 만으로 저장 pattern 근방으로 retrieval 이 사실상 완료된다고 보고한다 (잘 분리된 pattern 에 대해 one update 후 오차가 separation 에 지수적으로 작다 [Ramsauer et al. 2021 Thm 4]) — attention 이 반복 없이 softmax 한 번으로 read 를 끝내는 관행은 이 성질의 번역이다. energy 를 날카롭게 만들수록 attractor 의 basin 이가팔라져서, 반복 read 가 1-step read 로 접힌다.

이 대응 사슬 전체는 한 다이어그램으로 요약된다 (그림 5-2): 고전 Hopfield energy → 연속 상태의 log-sum-exp energy → softmax update rule → transformer attention 의 네 상자가, 왼쪽에서 오른쪽으로 같은 대상을 점점독자에게 익숙한 형태로 재서술한다. Ramsauer et al. 2021 은 오른쪽 두상자의 등식 — continuous modern Hopfield 의 1-step update 가 곧 $\text{softmax}(\beta \xi^T X) X^T$ 이고, 역온도 $\beta = 1/\sqrt{d_k}$ 선형 사상을 두면 정확히 $\text{softmax}(QK^T/\sqrt{d_k})V$ — 을 이 그림의 핵심 주장으로 제시한다.

그림 5-2 — modern Hopfield network 의 energy 를 이진에서 연속 상태로 일반화하면 (가운데 두 상자: log-sum-exp energy) 그 1-step update rule 이 정확히 transformer 의 softmax attention 이 된다 (오른쪽 두 상자). § 5.4 의 “exponential energy 극한 = softmax attention” 사슬을 한 줄로 보여 준다. 출처: Ramsauer et al., Hopfield Networks is All You Need (arXiv: 2008.02217), 원문 Fig.1 —원저자의 그림 (third-party), 본서의 결과가 아님.

사슬을 한 줄로 요약한다: energy 의 차수를 올린다 = key 를 feature space 로 lift 한다 = capacity 가 올라간다; exponential 극한에서 softmax attention 에 도달한다. 이 사슬이 [Atlas § 3.1] 의 이론적 골격 그자체다. Atlas 는 (a) matrix memory + ℓ_2 loss 의 capacity 가 $O(d_k)$ 임을 증명하고 (Prop 1), (b) 차수 p polynomial feature map ϕ_p 로 key 를 lift 하면 $O(d_k^p)$ 로 올라감을 보이고 (Prop 2), (c) exponential feature map ϕ^* 의 극한에서 softmax attention 이 unbounded memory 를 갖는 associative memory 로 나타남을 이용한다 [Atlas § 4.2]. Atlas 원문은 이 kernel 장치가 Krotov & Hopfield 2016 의 방법을 계승한 것임을 명시한다 [Atlas § 3.1]. 이 사슬을 지금 손에 쥐고 있으면 14 장은 corollary 처럼 읽힌다. (capacity 의 formal 정의와 정리의 상세는 14 장 소유다; 이 장은 고전 쪽만 정식으로 다룬다.)

systems 점점 하나: “capacity 를 올린다 = key 를 lift 한다 = key 차원과 그에 붙는 연산이 커진다” 이므로 이것은 공짜가 아니라 state 크기 · FLOP 과의 거래다. ϕ_p 의 lifted 차원은 $D = \Theta(d_k^p)$ 이고, memory state 는 $W \in \mathbb{R}^{d_v \times D}$ 로 함께 커진다. 감각을 위한 숫자하나: $d_k = 128$, 차수 $p = 2$

의 전체 monomial map 이면 $D = \binom{128+2}{2} = 8385$, state 는 32 KiB 에서 약 2 MiB 로 $65 \times$ 커진다 (bf16, $d_v = 128$ 기준). capacity 를 사는 통화가 state byte 라는 것 — 이 거래의 비용 구조 (capacity 의 지불 통화가 state byte·matmul 폭이라는 것) 는 14 장 § 14.7 에서 확정한다 — 단, Atlas 가 구현 차수 p 와 sketch 차원을 공개하지 않아 절대값 계산은 그곳에서도 불가능하다는 것까지가 14 장의 결론이다.

5.5 delta rule: append 에서 error-correcting overwrite 로

Hebbian write 의 결함은 blind 라는 것이었다. 같은 key 가 다시 오면 확인없이 위에 더해 이중 저장이 되고, 비슷한 key 들은 서로를 오염시킨다. 개선의 방향은 write 를 최적화 문제로 바꾸는 것이다. “이 쌍이 지금 얼마나 잘 저장되어 있는가” 를 측정하는 손실

$$\ell(W; k_t, v_t) = \|Wk_t - v_t\|_2^2 \quad (5-4)$$

을 정의하고, 새 쌍이 올 때마다 이 손실을 한 걸음 내려간다. gradient 는 2 장의 chain rule 을 한 번 쓰면 $\nabla_W \ell = 2(Wk_t - v_t)k_t^\top$ — 오차 벡터와 key 의 outer product, 또 rank-1 이다. 계수 2 를 η_t 에 흡수하고 한 step 내려가면

$$W_t = W_{t-1} - \eta_t(W_{t-1}k_t - v_t)k_t^\top = W_{t-1}(I - \eta_t k_t k_t^\top) + \eta_t v_t k_t^\top. \quad (5-5)$$

이것이 **delta rule** 이다 (Widrow & Hoff 1960 의 LMS 알고리즘; 이라인의 논문들은 1988 년 reprint 를 인용한다). 식 (M1) 에 linear memory $\mathcal{M}(q; W) = Wq$ 를 대입한 전개형이며, 6 장에서 만날 DeltaNet 의 update 그 자체다. 이름에 주의: delta rule 은 1960 년의 학습 규칙이고, DeltaNet 은 그것을 sequence layer 로 쓰는 2020 년대의 모델이다.

식 (5-5) 는 세 가지로 읽을 수 있고, 셋 다 이후 장들에서 계속 쓰인다.

1. **read-modify-write.** $W_{t-1}k_t$ 는 이 key 에 지금 저장된 value 의 read 다. delta rule 은 그 read 와 목표 v_t 의 차이, 즉 오차만큼만 고쳐 쓴다. Hebbian write 가 append-only log 라면 delta rule 은 key 로 주소를 찾아가는 in-place update 다. 같은 key 가 두 번 오면 두 번째 write 는 첫 번째를 덮어쓴다 — 이중 저장이 사라진다.
2. **1-step SGD.** 식 (5-5) 는 2 장의 SGD 객체를 손실 (5-4) 에 적용한 것, 그 이상도 이하도 아니다. 독자가 2 장에서 만난 가장 단순한 optimizer 가 사실은 1960 년의 memory write 규칙이었던 것이다. “optimizer 는 associative memory 다” 라는 이 라인의 중심 명제 ([NL]) 가 여기서 처음 실체를 얻는다. 훈련의 커리큘럼 (2 장) 과 기억의 커리큘럼 (이장) 은 의도적으로 이 지점에서 합류한다.
3. **부분 소거 후 재기록.** $W_{t-1}(I - \eta_t k_t k_t^\top)$ 항을 보라. $\eta_t = 1$, $\|k_t\| = 1$ 이면 $I - k_t k_t^\top$ 는 k_t 방향 성분을 정확히 지우는 projection 이다: “이 key 에 대한 옛기억을 지우고 새 value 를 쓴다.” $\eta_t < 1$ 이면 부분만 지운다. η_t 가 write 강도를 조절하는 gate 라는 것 — 이것이 13 장에서 inner learning rate 를 data-dependent gate 로 학습시키는 관점의 예고편이다.

비용의 대조도 명확히 해 두자. Hebbian write 는 outer product 하나 — $d_k d_v$ MAC — 면 끝난다. delta rule 은 쓰기 전에 읽어야 하므로 prediction read $W_{t-1}k_t$ (GEMV) 가 write 경로에 추가된다: FLOP 은 대략 2 배, 그리고 state 전체를 읽는 traffic 이 매 write 마다 발생한다. 더 나은 write rule 은 공짜가 아니라 read 를 지불하고 산다 — 이 패턴은 라인내내 반복된다. momentum 은 buffer S_t 의 저장과 갱신을 (12 장), Omega rule 은 window 안 c 개 token 의 재최적화를 (14 장) 같은 방식으로 지불한다.

delta rule 의 보장은 이렇다. $\eta_t = 1$ 이고 key 가 unit norm 이면 update 직후 $W_t k_t = v_t$ 가 정확히 성립한다 — 방금 쓴 쌍은 crosstalk 없이 복원된다. key 들이 선형독립이고 $m \leq d_k$ 이면 선형계 $WK = V$ 가 consistent 하므로, 같은 쌍들을 반복 제시하며 돌리면 (cycling) delta write 의 잔차가 0 으로 접혀 exact interpolation 해 $W^* = VK^+$ (pseudoinverse) 에 도달한다 (고정 step 으로도 수렴한다). 반대로 $m > d_k$ 라계가 inconsistent 하면 고정 step 의 cyclic LMS 는 least-squares 해로 정확히 수렴하지 않고 잔차 주위를 진동하므로, 일반적 least-squares 수렴에

는 감소하는 step size 가 필요하다. [Atlas § 3.1 Prop 1] 의 증명은 (cyclic online LMS 가 아니라) full-batch gradient descent 가 이 consistent 경우의해로 수렴함을 보인다: exact 저장 조건 $WK = V$ 는 md_v 개의 방정식과 $d_k d_v$ 개의 미지수를 갖는 선형계이므로, 선형독립 key 에 대한가 해 조건은 $m \leq d_k$ 이고, 이때 full-batch gradient descent 는 minimum-norm 해로 수렴한다 [Atlas App. C, Prop 1 증명].

unit norm 이 아닌 일반 key 에서는 exact write 의 조건이 하나 붙는다. update 직후 $W_t k_t = v_t$ 를 원하면, 식 (5-5) 에 대입해 보면 $\eta_t = 1/\|k_t\|_2^2$ 이어야 한다 — norm 이 큰 key 일수록 살살 써야 한다 (adaptive filtering 문헌이 normalized LMS 라는 이름으로 표준화한 선택이다). 한 걸음 더 가서 step size 를 $\eta_t/(1 + \eta_t k_t^\top k_t)$ 로 바꾸면 어떤 $\eta_t > 0$ 에서도 안정한 implicit GD 가 되는데, 이것이 6 장 카탈로그의 Longhorn 이 채택한 update 다. write 강도의 스칼라 하나를 바꾸는 것만으로 모델 하나가 달라져 나온다 — inner optimizer 의 선택이 아키텍처의 축이라는 이 라인의 문법을 미리 보여 주는 사례다.

그리고 냉정한 사실 하나. delta rule 은 Hebbian 의 crosstalk 을 교정하지만, **capacity 상한 자체는 올리지 못한다**. matrix memory + ℓ_2 loss 인 한 상한은 여전히 $O(d_k)$ 다 [Atlas § 3.1 Prop 1]. write rule 의 개선은 상한에 도달하게 해 주는 것이고, 상한을 올리는 것은 다른 축이다: memory 를 deep 하게 만들거나 (2-layer 이상 MLP memory 의 capacity 하한은 $O(d_k d_v)$, [Atlas § 3.1 Thm 1]), key 를 feature map 으로 lift 해야 한다 (ϕ_p 로 $O(d_k^p)$, [Atlas § 3.1 Prop 2]). **write rule 의 축과 capacity 의 축은 독립이다** — 이 구분이 Part II 의 설계 공간을 가른다. Titans 와 Miras 는 주로 write rule 축을 움직이고 (momentum 추가, retention 추가, loss 교체), Atlas 는 두 축을 동시에 움직인다.

5.6 attention 은 압축하지 않는 associative memory 다

이제 사다리의 꼭대기를 명시하자. softmax attention 의 read

$$y_t = \sum_{i \leq t} \text{softmax}_i(q_t^\top k_i / \sqrt{d_k}) v_i$$

는 저장된 모든 쌍을 그대로 보관한 채 (KV cache), read 시점에 kernel 가중평균으로 연상하는 연산이다. parameter 에 겹쳐 쓰지 않으므로 § 5.2 의 compression crosstalk 은 없고, cache 가 자라는 한 저장 용량 제한도 없다. 단 finite-temperature softmax read 는 non-target key 에도 양의 가중치를 주므로 retrieval 은 정확한 lookup 이 아니라 kernel 가중 평균이다 — exact 복원 여부는 pattern separation·온도에 달려 있고 (§ 5.4), “무제한” 인 것은 exact retrieval capacity 가 아니라 원시 KV 저장량이다. 통계학의 언어로는 ℓ_2 regression 의 non-parametric Nadaraya – Watson 해이며 (→ 6 장, 10 장), 이 책의 아키텍처 카탈로그에서 attention 이 “압축하지 않는 극한” 으로 분류되는 근거다. attention 을 associative memory 로 읽는 이 관점의 대표 인용은 Bietti et al. 2023 (arXiv:2306.00802) 이고, Titans·Miras·Atlas 세 편 모두 이 관점을 명시적으로 채택한다 [Titans § 1; Miras § 2; Atlas § 2].

이로써 이 장의 위계가 완성된다:

표 5-1 — 이 장의 세 write 방식과 그 대가.

write 방식	저장	read 오염	exact capacity	read 비용
Hebbian (5-3)	parametric, blind append	crosstalk $\propto k_i^\top k_j$	$\leq d_k$ (orthogonal 일 때만)	$O(d_k d_v)$ GEMV
delta rule (5-5)	parametric, read-modify-write	교정됨 (단일 pass 에서는 recency 편향)	$\leq d_k$ (선형독립이면 도달 가능)	$O(d_k d_v)$ GEMV

write 방식	저장	read 오염	exact capacity	read 비용
softmax attention	non- parametric append	compression crosstalk 없음 (단 softmax 혼 합은 남음)	저장 무제한 (cache 가 자라 는 한); exact retrieval 은 separation 의존	$O(Ld)$ 전체 스캔

이 라인의 존재 이유는 이 표의 가운데 행을 위로 밀어 올리는 것이다: attention 의 연상 품질에 다가가되, state 는 고정 크기로 유지하기. 그수단들이 Part II 의 목차 그 자체다 — write 에 momentum 과 retention 을 더하고 (12 장), inner objective 와 gate 를 다시 고르고 (13 장), memory 를 deep 하게 만들고 key 를 lift 하며 window 단위로 함께 최적화한다 (14 장).

확장의 예고 하나. [NL § 1] 은 이 추상화를 아키텍처 바깥으로 밀어붙인다: 훈련의 backprop 자체가 “각 data sample 을 그 예측의 오차에 mapping 하는 associative memory” 이고 [NL § 3.1], momentum 은 gradient 들을 압축하는 memory 이며, Adam 은 element-wise ℓ_2 objective 에 대한 optimal associative memory 라고 주장한다 [NL § 1]. 그상세는 16 장의 일이다. 여기서 기억할 것은 하나다: 그 모든 주장의 원형이 장의 식 (5-4) – (5-5) 라는 것.

5.7 Worked micro-example: crosstalk 을 손으로 계산하고 delta rule 로 고치기

$d_k = d_v = 2$, 쌍 두 개를 저장한다. key 는 둘 다 unit norm 이고 겹침 (내적) 은 $\mu = k_1^\top k_2 = 0.6$ 이다:

$$k_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, v_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}; \quad k_2 = \begin{pmatrix} 0.6 \\ 0.8 \end{pmatrix}, v_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

(1) **Hebbian**. 식 (5-1) 로 두 쌍을 겹쳐 쓰면

$$W_{\text{heb}} = v_1 k_1^\top + v_2 k_2^\top = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} + \begin{pmatrix} 0 & 0 \\ 0.6 & 0.8 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0.6 & 0.8 \end{pmatrix}.$$

읽어 보면 $W_{\text{heb}} k_1 = (1, 0.6)^\top$ 인데 정답은 $v_1 = (1, 0)^\top$ 이다. 오차 $(0, 0.6)^\top$ 는 식 (5-2) 가 예언한 crosstalk $\mu v_2 = 0.6 v_2$ 그대로다. 대칭적으로 $W_{\text{heb}} k_2 = (0.6, 1)^\top$ 이고 오차는 μv_1 이다. 두 read 모두 오차 norm 이 정확히 $\mu = 0.6 = 60\%$ 의 오염이다.

(2) **delta rule, 첫 pass**. $\eta_t = 1$, $W_0 = 0$ 에서 시작해 식 (5-5) 로 순서대로 쓴다.

- (k_1, v_1) write: read 는 $W_0 k_1 = 0$, 오차는 $-v_1$. 따라서 $W_1 = v_1 k_1^\top$. 빈 memory 에의 첫 write 는 Hebbian 과 동일하다 — 읽을 것이 없기 때문이다.
- (k_2, v_2) write: read 는 $W_1 k_2 = (0.6, 0)^\top = k_2$ 의 자리에 이미 k_1 기억의 그림자가 있다. 오차는 $(0.6, -1)^\top$ (norm ≈ 1.17) 이고,

$$W_2 = W_1 - (0.6, -1)^\top k_2^\top = \begin{pmatrix} 0.64 & -0.48 \\ 0.6 & 0.8 \end{pmatrix}.$$

검산: $W_2 k_2 = (0.64 \cdot 0.6 - 0.48 \cdot 0.8, 0.6 \cdot 0.6 + 0.8 \cdot 0.8)^\top = (0, 1)^\top = v_2$. **방금 쓴 쌍은 정확히 복원된다** — delta rule 의 약속이다. 그런데 $W_2 k_1 = (0.64, 0.6)^\top$ 이고 오차 norm 은 ≈ 0.70 이다. Hebbian 의 0.60 보다 **오히려 나쁘다**. overwrite 는 최신쌍의 정확성을 위해 옛 쌍의 자리를 침범한다.

(2b) **같은 쌍을 두 번 쓰면**. blind write 의 의미를 숫자로 못박자. W_{heb} 에 (k_2, v_2) 를 Hebbian 으로 한 번 더 쓰면 read 가 $(W_{\text{heb}} + v_2 k_2^\top) k_2 = (0.6, 2.0)^\top$ 이 된다 — value 가 이중으로 쌓였다. 반면 W_2 에 (k_2, v_2) 를 delta rule 로 다시 쓰면 오차 $W_2 k_2 - v_2 = 0$ 이라 update 가 정확히 0 이다.

이미 잘저장된 쌍에 대한 delta write 는 no-op 이다 — read-modify-write 의 가치가 이한 줄에 있다.

(3) **cycling**. 두 쌍을 번갈아 다시 쓰면 오차가 기하급수로 줄어든다. 각 write 직전, 그 key 의 read 오차 norm 을 추적하면: $1.17 \rightarrow 0.70 \rightarrow 0.42 \rightarrow 0.25 \rightarrow \dots$ — 매 write 마다 $\times \mu = 0.6$, 한 cycle(두 쌍을 한 번씩 재기록) 마다 $\times \mu^2 = 0.36$ 이다. 수렴 극한은 exact 해

$$W^* = VK^{-1} = \begin{pmatrix} 1 & -0.75 \\ 0 & 1.25 \end{pmatrix}, \quad W^*k_1 = v_1, \quad W^*k_2 = v_2$$

이고, 이것이 가능한 이유는 $m = 2 \leq d_k = 2$ 에 key 가 선형독립이기 때문이다 — [Atlas § 3.1 Prop 1] 의 조건 그대로다. read 오차의 전과정을 표로 정리하면:

표 5-2 — 각 시점에서의 read 오차 norm ($\|\mathcal{M}(k_i; W) - v_i\|_2$).

시점	k_1 오차	k_2 오차
W_{heb} (Hebbian 동시 저장)	0.60	0.60
W_2 (delta 1st pass)	0.70	0.00
W_3 (k_1 재기록)	0.00	0.42
W_4 (k_2 재기록)	0.25	0.00
W^* (극한)	0.00	0.00

세 가지 교훈. 첫째, crosstalk 은 추상적 개념이 아니라 **key 내적 그자체**다 (0.6 이라는 숫자가 두 번 그대로 나타났다). 둘째, delta rule 의 단일 online pass 는 최신 쌍을 정확히 쓰는 대신 옛 기억을 침범한다 — **recency 편향은 overwrite 방식에 내장된 성질**이다. 셋째, cycling 이 가능하면 least-squares 해로 수렴하지만, sequence modeling 에서는 지나간 token 을 다시 쓸 수 없다 (단 한 번의 online pass). 그래서 이 라인은 다른 보완 장치를 쌓는다: 과거의 gradient 방향을 유지하는 momentum([Titans], → 12 장), 무엇을 남길지 학습하는 retention(→ 13 장), 그리고 최근 window 전체를 공동으로 재최적화하는 Omega rule([Atlas], → 14 장). Atlas 가 문제 설정에서 지적하는 비판 — 현재 token 에 대해서만 greedy 하게 최적화하는 online update 는 개별 token 을 memorize 할 뿐 context 전체가 함께 잘 저장되었는지 묻지 않는다 [Atlas § 1] — 이 정확히 이 표의 W_2 행을 겨누다.

참고로 이 예제의 모든 연산은 2×2 GEMV 두 번과 rank-1 outer product 한 번씩이었다. 차원을 d 로 올려도 연산의 종류는 같다 — 그것이다음 절의 비용 계산이다.

5.8 Systems bridge: $d \times d$ state 의 “유효 cache 크기”

이 장의 결과를 독자의 단위계 — byte, FLOP, bandwidth — 로 번역한다. head 하나, $d_k = d_v = 128$, bf16(2 bytes) 기준이다.

표 5-3 — non-parametric vs parametric associative memory, head 하나 기준 ($d_k = d_v = 128$, bf16).

항목	KV cache (softmax attention)	matrix memory $W \in \mathbb{R}^{128 \times 128}$
state 크기	512 B/token, L 에 비례 (64K token 이면 32 MiB)	32 KiB 고정
write	append: 512 B 복사, 간섭 없음	rank-1 read-modify-write: $O(d^2)$ FLOPs + state 전체 왕복
read	전체 스캔: $O(L \cdot d)$ FLOPs·bytes	GEMV: $2d^2 \approx 33$ KFLOPs, 32 KiB read

항목	KV cache (softmax attention)	matrix memory $W \in \mathbb{R}^{128 \times 128}$
exact 저장 한계	cache 가 자라는 한 무제한	$\leq d_k = 128$ 쌍 [Atlas § 3.1 Prop 1]
forgetting	명시적 eviction policy (paged cache 등)	crosstalk 에 의한 암묵적 저하, 또는 학습된 gate ($\rightarrow$ 13 장)

이 표에서 세 가지 계산을 뽑아 두면 Part II 내내 쓴다.

첫째, byte 증가와 capacity 증가는 다르다. 32 KiB 짜리 matrix memory 는 byte 로는 KV cache 64 token 분량이다. 그러나 exact 저장 한계는 $d_k = 128$ 쌍 — byte 증가의 2 배다. 반대로 말하면 겨우 2 배다: $d \times d$ state 는 “ L 을 d 수준으로 압축해 주는 마법” 이 아니라, **잘 관리해야 $O(d)$ 쌍을 담는 유한 cache** 다. context 가 d_k 를 넘는 순간부터는 반드시 lossy 하고, 무엇을 잃을 지는 write rule 과 gate 가 정한다. capacity 이론은 “언제부터 압축이 불가피한가” 를 알려 주는 도구다.

스케일을 실전 크기로 올려 보자. head 8 개 $\times$ layer 48 개면 matrix memory 의 상주 state 는 $32 \text{ KiB} \times 8 \times 48 = 12 \text{ MiB/request}$ 다. 같은 구성의 KV cache 는 token 당 $512 \text{ B} \times 8 \times 48 = 192 \text{ KiB}$ 이므로 64K context 에서 12 GiB — $1024 \times$ 차이다. 이 $1000 \times$ 가 라인 전체의 유혹이고, 이장의 capacity 결과는 그 가격표다: state 를 $1000 \times$ 줄이는 대가로, exact 저장 한계는 context 길이와 무관한 head 당 d_k 개 쌍으로 고정된다.

둘째, decode 의 연산 강도. matrix memory 의 decode 한 step 은 read GEMV($2d^2$ FLOPs) + write 의 prediction GEMV 와 rank-1 교정 (합쳐서 $O(d^2)$) 이고, 그동안 state 32 KiB 를 읽고 다시 쓴다 (RMW). FLOP 과 byte 가 같은 $O(d^2)$ 차수이므로 arithmetic intensity 는 몇 FLOP/byte 수준 — 최신 GPU 의 ridge point 보다 두 자릿수 아래로, 철저히 **bandwidth-bound** 다. per-token 으로는 KV 스캔의 $O(Ld)$ 보다 싸지만, tensor core 를 놀리는 연산이라는 점이 9 장 (chunkwise 병렬화) 의 출발 동기가 된다.

셋째, batching 이 깨진다. 지금까지의 GEMV 들에서 W 는 request 마다 다른 per-sequence state 다. shared weights 를 전제로 여러 request 를 한 GEMM 으로 묶던 독자의 batching 감각 은 여기서 무너진다 — batch 축이 사라진 GEMV 들의 모음이 되고, grouped-GEMM 류의 kernel 이 필요해진다. 이 함의는 10 장에서 cost model 로 정리한다.

마지막으로 shape 관찰 하나. 이 장의 모든 write — Hebbian 의 $v_t k_t^\top$, delta 의 $(W k_t - v_t) k_t^\top$ — 는 rank-1 outer product 였고, 2 장의 backward pass 가 만들던 $dW = \delta x^\top$ 와 동일한 GEMM shape 였다. token 을 C 개 모아 쓰면 rank- C GEMM 이 된다. “memory write 를 GEMM 으로 묶는다” 는 이 한 문장이 9 장 chunkwise training 의 전부다.

요약

- associative memory 는 $\text{key} \rightarrow \text{value}$ mapping 을 저장 · 복원하는 시스템이며, KV cache (무손실 non-parametric) 와 matrix memory (고정 크기 parametric) 는 같은 추상화의 양 극단이다. 이 라인의 공식 정의는 [Miras Def. 3.1]/[NL Def. 1] 이 이 고전 개념을 재정식화한 것이다.
- Hebbian write $W_t = W_{t-1} + v_t k_t^\top$ 는 blind append 이고, read 오염 (crosstalk) 의 크기는 key 간 내적이다. 이것이 [Titans § 2] 가 비판하는 linear attention 의 “memory overflow” 의 실체다.
- Hopfield 1982 는 auto-associative memory 를 energy 최소화로 정식화했고 capacity 는 약 $0.14d$ — parameter d^2 개에 저장 능력은 $O(d)$ 로, 병목은 parameter 수가 아니라 key 차원의 rank 다.
- dense Hopfield 의 사슬 — polynomial energy 는 capacity 를 d^{n-1} 로, exponential energy 는 softmax attention 으로 — 이 [Atlas § 3.1] 의 ϕ_p/ϕ^* 장치의 원형이다.
- delta rule 은 ℓ_2 associative loss 의 1-step gradient descent 이고 (식 (M1) 의 linear 전개), read-modify-write 로 crosstalk 을 교정하지만 matrix memory 의 capacity 상한

$O(d_k)$ 자체는 올리지 못한다 [Atlas § 3.1 Prop 1]. write rule 의 축과 capacity 의 축은 독립이다.

- 단일 online pass 의 delta rule 은 최신 쌍을 정확히 쓰는 대신 옛 기억을 침범한다 (recency 편향). momentum(12 장)·retention(13 장)·window 재최적화 (14 장) 는 이 구조적 성질에 대한 서로 다른 보완이다.
- delta rule 의 exact write 조건은 $\eta_t = 1/\|k_t\|_2^2$ (normalized LMS) 이고, step size 를 $\eta_t/(1 + \eta_t k_t^\top k_t)$ 로 바꾸면 Longhorn 의 implicit GD 가 된다 ($\rightarrow$ 6 장) — inner optimizer 의 선택이 모델을 가른다.
- 이 장의 모든 write 는 rank-1 outer product 로, 2 장의 $dW = \delta x^\top$ 와 같은 GEMM shape 다. decode 한 step 은 GEMV+RMW 로 bandwidth-bound 이며, per-sequence state 는 shared-weight batching 을 깨뜨린다.

자가 점검체크리스트

- ☐ Hebbian write와 delta rule 의 update 식을 쓰고, 각각을 blind append와 read-modify-write 로 설명할 수 있다.
- ☐ 주어진 2-3 개의 (k, v) 쌍에 대해 crosstalk 을 손으로 계산하고, 그 크기가 key 내적임을 보일 수 있다.
- ☐ 식 (5-4) 의 gradient 를 계산해 delta rule (5-5) 를 유도하고, 이것이 식 (M1) 과 같음을 확인할 수 있다.
- ☐ Hopfield $\rightarrow$ dense Hopfield $\rightarrow$ softmax attention 의 capacity 사슬을 설명하고, [Atlas] 의 ϕ_p 와 ϕ^* 가 이 사슬의 어느고리에 해당하는지 말할 수 있다.
- ☐ matrix memory 의 capacity 가 parameter 수 $d_k d_v$ 가 아니라 $O(d_k)$ 인 이유를 [Atlas § 3.1 Prop 1] 의 선형계 논리로 설명할 수 있다.
- ☐ Hebbian write, delta rule, crosstalk, capacity 를 각각 KV cache append, in-place cache update, 검색 오염, 유효 cache 크기라는 inference 어휘로 옮길 수 있다.

다음 장으로

이 장은 write rule 과 capacity 를 “그 자리에 놓인” memory 의 문제로 다뤘다: 쌍이 주어진다면 어떻게 쓰고, 몇 개까지 담기는가. 6 장은 이 memory 를 sequence layer 로 조립한다 — Hebbian write (5-3) 을 그대로 layer 로 만들면 linear attention 이고 (이 대응을 fast weight programming 이라 부른다), delta rule (5-5) 를 넣으면 DeltaNet 이다. 그 순간 새 질문들이 열린다: key-value 를 만드는 projection 과 η_t 같은 gate 는 누가 학습하는가 (outer loop — 4 장의 답을 재사용한다), 그리고 token 마다 순차적인 이 recurrence 를 GPU 에서 어떻게 GEMM 으로 묶는가 (9 장의 주제다). 이 장이 남긴 미해결 질문 — 단 한 번의 online pass 에서 무엇을 남기고 무엇을 덮어쓸 것인가 — 는 3 장의 online learning 언어로 정식화된 뒤, 12 장의 momentum 과 13 장의 retention 이론이 서로 다른 답을 내놓는다.

ch06. Linear attention과 fast-weight programming: DeltaNet, Gated DeltaNet, Longhorn, RWKV-7

이 장의 목표 — 독자가 이 장을 마치면 다음을 할 수 있어야 한다. 1. softmax attention 에서 kernel trick 으로 linear attention 의 재귀형을 유도하고, 그것을 “KV cache 를 고정 크기 행렬로 압축하는 write 연산” 으로 읽을 수 있다. 2. RetNet → GLA → DeltaNet → Gated DeltaNet → Longhorn → RWKV-7 을 별개의 발명품이 아니라 하나의 (state, update, cost) 객체에서 inner objective·optimizer·retention 을 바꿔 긴 변형들로 배치할 수 있다. 3. Longhorn 의 implicit gradient descent 를 직접 유도하고, 왜 step size 에 대해 무조건 안정인지 논증할 수 있다. 4. $d = 2$ 손계산으로 Hebbian write 의 crosstalk 와 delta write 의 overwrite 동작을 재현하고, 각 모델의 per-token decode 비용을 자기 서빙스택의 roofline 위에 올려놓을 수 있다.

왜 필요한가 — 6 편 전부가 이 장의 모델 family 를 직접 전제한다. - [Titans] (Titans: Learning to Memorize at Test Time, arXiv:2501.00663) 의 § 2 는 linear attention 의 재귀형 [Titans Eq. 4–5] 를 출발점으로 놓고, 그 “additive write 의 memory overflow” 비판과 두 가지 개선 방향 — forget 기제 (GLA·Mamba-2 계열) 와 write 개선 (delta rule 계열) — 으로 자기 위치를 정의한다 [Titans § 2]. 12 장의 bridge-in 은 이 서사를 그대로 잇는다. - [Miras] (It’s All Connected, arXiv:2504.13173; 이 책은 framework 이름 Miras 로 통칭) 의 Table 1 은 이 장의 모델 전부를 4 축 (memory 구조, attentional bias, retention, learning algorithm) 으로 분류한다. [Miras Eq. 8] 과 [Miras Eq. 9] 가 이 장의 식 (6-2), (6-3) 이다. 13 장은 이 장의 모델들이 이미 손에 익었다고 가정한다. - [Atlas] (Atlas: Learning to Optimally Memorize the Context at Test Time, arXiv:2505.23735) 의 Omega rule 은 delta rule 의 sliding-window 일반화다. DeltaNet 을 모르면 14 장의 “token 하나 최적화 vs window 최적화” 대비가 공허해진다. - [TNT] (TNT: Improving Chunkwise Training for Test-Time Memorization, arXiv:2511.07343) 는 이 family 를 “linear memory modules” 대조군으로 쓰고, [NL] (Nested Learning, arXiv:2512.24695) 은 linear attention 을 nested optimization 의 상시 2-level 예제로 사용한다.

이 장은 독자의 홈그라운드에서 시작한다. KV cache, GEMV, roofline — 전부 이미 아는 어휘다. 새로 배우는 것은 단 하나의 관점 전환이다: **linear attention 의 state 갱신은 “작은 모델의 weight 를 한 걸음 학습시키는 것” 과 같은 수식이며, 따라서 이 family 의 모든 변형은 optimizer 의 선택 문제로 환원된다.** 이 관점이 서면 Titans 라인 6 편은 “inner optimizer 를 점점 좋은 것으로 갈아 끼우는 연대기” 로 읽힌다.

6.1 Kernel trick: KV cache 를 $d_v \times d_k$ 행렬로 압축한다

softmax attention 의 decode 비용부터 복기한다. token t 에서 attention 은 query q_t 로 지금까지 쌓인 모든 key 를 조회한다:

$$y_t = \sum_{j=1}^t \frac{\exp(q_t^\top k_j)}{\sum_{l=1}^t \exp(q_t^\top k_l)} v_j.$$

독자가 매일 보는 그 비용 구조다: KV cache 는 token 당 $(d_k + d_v)$ 개원소씩 자라고, decode 한 step 은 cache 전체를 한 번 읽는다. 길이 L 에서 read traffic 이 $O(L)$ — decode 가 bandwidth-bound 가 되는 바로그 이유다.

linear attention 은 이 합에서 exp 를 제거하는 데서 출발한다. Katharopoulos et al. 2020 (arXiv:2006.16236) 은 similarity 를 **feature map** ϕ 의 내적 $\phi(q)^\top \phi(k)$ 로 바꾸면 (원문은 $\phi(x)^\top \phi(y) \geq 0$ 을 위해 elu + 1 을 사용) 합의 결합 순서를 재배열할 수 있음을 지적했다 [Titans Eq. 3]:

$$y_t = \frac{\phi(q_t)^\top \sum_{j \leq t} \phi(k_j) v_j^\top}{\phi(q_t)^\top \sum_{l \leq t} \phi(k_l)} \implies \text{누적량 } \sum_{j \leq t} v_j \phi(k_j)^\top \text{만 유지하면 된다.}$$

ϕ 를 항등으로 두면 (이하 이 장의 기본 설정; RetNet 이후의 관행) 누적량은 $d_v \times d_k$ 행렬 하나가 되고, 전체가 재귀형으로 떨어진다 [Titans Eq. 4–5]:

$$W_t = W_{t-1} + v_t k_t^\top, \quad y_t = W_t q_t \quad (6-1)$$

(원문들은 행벡터 관행으로 $M_t = M_{t-1} + K_t^\top V_t$ 로 쓴다. 이책은 열벡터 $\cdot W \in \mathbb{R}^{d_v \times d_k}$ 관행으로 통일한다 — § 표기, 1 장.) 분모의 누적 벡터 $z_t = z_{t-1} + \phi(k_t)$ 도원리상 함께 유지해야 하지만, RetNet 이후의 모델들은 분모 누적을 아예버리고 출력 쪽 normalization 으로 대체한다 — RetNet 은 head 출력에 GroupNorm 을 두고 그 scale-invariance 를 수치 안정화에 사용하며 (Sun et al. 2023, arXiv:2307.08621, § 2.2·§ 3.1), GLA 는 “normalizer 없는” linear attention 이실전에서 잘 동작함을 명시하고 head 별 LayerNorm 을 출력에 둔다 (Yang et al. 2024, arXiv:2312.06635, § 2).

식 (6-1) 을 Rosetta-Stone 사전 ($\rightarrow$ 1 장) 으로 읽자. 이것은 **KV cache 의 손실 압축**이다. softmax attention 은 모든 (k_j, v_j) 쌍을 그대로보관하는 non-parametric memory 이고 — [Miras § 4] 는 이를 “ ℓ_2 regression 의 Nadaraya – Watson 해, retention 없음” 으로 정식화한다 — linear attention 은 같은 스트림을 고정 크기 행렬 W_t 에 겹쳐 쌓는다. write 는 rank-1 outer product $v_t k_t^\top$ 의 accumulate (BLAS 로 말하면 GER), read 는 GEMV 한 번이다. cache 가 자라지 않으므로 decode 의 FLOPs/token 과 bytes/token 이 문맥 길이와 무관한 상수가 된다. 이득의 정체는 산술 강도가 아니라 **traffic 총량의 상한**이다 — 이 구분은 § 6.9 에서 숫자로 확인한다.

단, append 와 write 의 차이에 밑줄을 긋는다. KV cache append 는 append-once 다: 한 번 쓴 항목은 불변이고, 간섭도 없다. 식 (6-1) 의 write 는 **read-modify-write** 다: 모든 쌍이 같은 $d_v \times d_k$ 저장소에 겹쳐 써지고, key 들이 직교하지 않는 한 서로 간섭한다. 5 장에서 본 그대로 — 식 (6-1) 은 correlation matrix memory 의 Hebbian write 이고, 겹쳐 쓰기의 대가는 crosstalk ($\rightarrow$ 5 장) 이다. [Titans § 2] 는 이것을 “additive write 의 memory overflow” 라고 부르며, 이 장의 나머지 전부는 이 한 문제에 대한 답들의 계보다.

6.2 Fast-weight programming: 1992 년의 발명, 2021 년의 재발견

식 (6-1) 의 W_t 에 이름을 붙일 차례다. 이 책 전체에서 가장 중요한 용어구분이 여기서 도입된다.

fast weights 는 sequence 가 흐르는 동안 token 마다 갱신되는 weight — 식 (6-1) 의 W_t — 를 말한다. **slow weights** 는 pretraining 이 끝나면 얼어붙는 보통의 parameter — projection W_K, W_V, W_Q 를 포함한 Θ 전부 — 를 말한다. 표기 규약도 이 구분을 따른다: fast 는 W , slow 는 Θ . 독자의 세계에서 slow weights 는 “모델” 이고 fast weights 는 “요청마다 존재하는 상태” 다. KV cache 가 그랬듯이, W_t 는 session state 이지 model parameter 가 아니다.

fast weight programming (FWP) 은 Schmidhuber 1992 (Learning to control fast-weight memories, Neural Computation 4(1)) 가 제안한 구도다: 한 network (slow net) 가 다

른 network 의 weight(fast weights) 를 입력에 따라 써 넣는다. slow net 의 출력이 데이터가 아니라 “다른 모델의 parameter” 라는 점에서, slow net 은 fast net 을 프로그래밍한다. Schlag, Irie & Schmidhuber 2021 (arXiv:2102.11174) 은 kernel trick 으로 얻은 linear attention 이 정확히 이 1992 년 구도임을 보였다: projection 들이 slow net 이고, outer-product write 가 프로그래밍 명령이며, read W_{tqt} 가 프로그램 실행이다. 같은 그룹의 후속작 Irie et al. 2021 (arXiv:2106.06295) 은 fast net 쪽을 재귀적으로 확장했다. 그리고 같은 2021 년 논문이 Hebbian write 대신 **delta rule**(→ 5 장) 을 **fast-weight update** 로 쓰는 모델 — 오늘날 DeltaNet 이라 불리는 것의 원형 — 을 제안했다 [Miras § 4]. 30 년 묵은 아이디어가 attention 의 어휘로 번역되는 순간 이 이 라인의 역사적 기점이다.

표 6-1 — FWP 계보 timeline. 이 장이 다루는 구간은 굵게.

연도	사건	의미
1960	delta rule (Widrow – Hoff)	error-driven write 의 원형 (→ 5 장)
1972	correlation matrix memory (Kohonen)	outer-product associative memory (→ 5 장)
1992	fast-weight controller (Schmidhuber)	slow net 이 fast weights 를 프로그램
2020	linear attention (arXiv: 2006.16236)	kernel trick, 재귀형 (6-1)
2021	FWP 동치 + delta-rule 변형 (arXiv:2102.11174)	linear attention = FWP; DeltaNet 원형
2023	RetNet (arXiv:2307.08621)	상수 decay gate
2024	GLA (arXiv:2312.06635)	data-dependent diagonal gate + chunkwise kernel delta rule 의 scale-up (WY, → 9 장)
2024	DeltaNet 병렬화 (arXiv: 2406.06484)	state = 작은 MLP 의 weights (→ 8 장)
2024	TTT (arXiv:2407.04620)	online 문제의 closed-form 해 = implicit GD gate + delta 결합
2025	Gated DeltaNet (arXiv: 2412.06464)	generalized delta rule (채널별 gate)
2025	RWKV-7 (arXiv:2503.14456)	momentum + retention + deep memory (→ 12 장)
2025	Titans (arXiv:2501.00663)	

FWP 관점이 주는 실질적 이득은 두 가지다. 첫째, **두 시간 척도의분리**가 명시된다. fast weights 는 inner loop(→ 1 장, 4 장) 에서 token 마다 움직이고, slow weights 는 outer loop 에서 gradient 로 학습된다. “이 write rule 의 η_t 는 누가 정하는가?” 라는 질문의 답이 항상 준비된다: write rule 자체는 inner loop 의 사건이고, write rule 의 계수를 만들어내는 함수 ($\eta_t = \eta(x_t; \Theta)$ 같은 gate head) 는 slow weights 의 일부로 outer loop 가 학습한다. 둘째, 이 장의 모델들을 (state, update, cost) 객체로 다룰 어휘가 생긴다. state 는 $W_t(d_v \times d_k$ 행렬, per head), update 는 이하 각 절의 한 줄 수식, cost 는 GEMV + rank-1 갱신이다. systems 독자에게 fast weights 란 결국 “kernel 이 매 step 갱신하는 레지스터 파일 같은 on-chip 상주 후보 상태” 이며, 실제로 이 family 의 고성능 kernel 들은 W_t 를 SRAM 에 상주시키는 형태로 짜인다 (→ 9 장).

6.3 Gate 의 도입: RetNet 과 GLA — 학습된 eviction

Hebbian write 의 첫 번째 문제는 지우는 수단이 없다는 것이다. 식 (6-1) 은 명시적 감쇠 기제 없이 더하기만 하므로 (부호가 맞는 항끼리 상쇄되지 않는한 state 의 norm 이 자라고), 오래된 쌍이 영원히 남아 crosstalk 를 누적시킨다. cache 어휘로 말하면 **eviction policy 가 없는 cache** 다. 첫 번째 교정은 자명한 방향이다: 매 step 이전 state 를 조금 깎는다.

$$W_t = \alpha_t W_{t-1} + v_t k_t^\top \quad (6-2)$$

여기서 $\alpha_t \in [0, 1]$ 은 retention gate($\rightarrow$ 13 장; 역사적 별칭 “forget gate”) 이고, 이 책의 방향 규약대로 **남기는 비율**이다 ($\alpha_t = 1$ 이면 전부 유지). 식 (6-2) 의 스펙트럼 위에 세모델이 놓인다 [Miras Eq. 8, § 4]:

- **RetNet** (Sun et al. 2023, arXiv:2307.08621): α 가 데이터와 무관한 고정 상수 — head 인덱스로 미리 정한 감쇠율 ($\gamma = 1 - 2^{-5-\text{arange}(h)}$) 풀 이고 학습되지 않는다. head 마다 다른 이 고정 감쇠로 다중 시간 척도를 만든다 [RetNet § 2.2 Eq. 8].
- **GLA** (Yang et al. 2024, arXiv:2312.06635): α_t 가 입력의 함수인 **data-dependent diagonal gate**. key 채널별로 남기는 비율이 달라진다 — 통일 표기로 $W_t = W_{t-1} \text{Diag}(\alpha_t) + v_t k_t^\top$.
- **Mamba-2**: α_t 가 data-dependent **스칼라**. SSM 계보에서 도달한 같은 지점이며, 유도과 duality 는 7 장이 담당한다.

[Miras § 4] 의 재해석이 이 지점에서 처음 빛을 발한다: 식 (6-1) 의 Hebbian write 조차 “1-step gradient descent” 다 — 다만 inner objective 가 ℓ_2 regression 이 아니라 dot-product similarity $\tilde{\ell}_t = -2\langle Wk_t, v_t \rangle$ 일 뿐이다 [Miras Eq. 8]. 이 objective 는 아래로 unbounded 이므로 GD 가 멈출 이유가 없고, 그래서 norm 이 자란다. gate 는 이 발산을 억제하는 regularization — Miras 의언어로 retention — 이며, “왜 gate 가 필요한가” 는 “왜 이 inner objective 에는 자기 제한이 없는가” 의 다른 표현이다.

systems 점점: (6-2) 의 추가 비용은 state 에 대한 elementwise 곱 하나, 즉 $d_k d_v$ FLOPs 와 (fusion 이 없다면) state 한 번의 추가 read-write traffic 이다. decode 에서는 어차피 state RMW 가 지배 항이므로 gate 의 한계비용은 사실상 0 이고, prefill/training 에서는 chunk 경계마다 감쇠 계수를 접어 넣는 형태로 GEMM 화된다 ($\rightarrow$ 9 장). 학습된 eviction 을 공짜로 얻는 셈이다 — 단, eviction 은 **채널 단위**다. 특정 “항목” 을 지목해지울 수는 없다. 그 능력이 다음 절의 주제다.

6.4 DeltaNet: append 에서 overwrite 로 — write 연산의 교정

gate 는 “전체를 잊는” 수단이지 “이 key 의 옛 값을 고쳐 쓰는” 수단이 아니다. key k 가 새 값 v^{new} 로 다시 나타났을 때 Hebbian write 는 $v^{\text{old}} k^\top$ 위에 $v^{\text{new}} k^\top$ 를 그냥 더한다 — 읽으면 두 값의 합이 나온다. 올바른 동작은 **먼저 옛 값을 읽어서 빼고, 새 값을 쓰는 것**이다. 이것이 delta rule($\rightarrow$ 5 장) 이고, 그것을 fast-weight update 로 채택한 모델이 **DeltaNet** 이다 (원형은 Schlag et al. 2021, scale-up 은 Yang et al. 2024, arXiv:2406.06484 [Miras § 4; Titans § 2]).

유도는 식 (M1) 의 linear memory 전개, 그 이상도 이하도 아니다. inner objective 를 $\ell(W; k_t, v_t) = \|Wk_t - v_t\|_2^2$ 로 두면 $\nabla_W \ell = 2(Wk_t - v_t)k_t^\top$ 이고 (계수 2 는 η_t 에 흡수한다), 1-step GD 는

$$W_t = W_{t-1} - \eta_t(W_{t-1}k_t - v_t)k_t^\top = W_{t-1}(I - \eta_t k_t k_t^\top) + \eta_t v_t k_t^\top. \quad (6-3)$$

이 한 줄에 세 개의 얼굴이 있다.

1. **optimizer 의 얼굴**: (6-3) 은 online regression 의 SGD step 이다. 예측 $W_{t-1}k_t$ 를 만들고 (read, GEMV), 오차 $W_{t-1}k_t - v_t$ 를 재고, 오차의 outer product 를 빼는 (write, GER) — 2 장에서 배운 $dW = (\text{오차}) x^\top$ 그 GEMM shape 다. 오차가 0 이면 update 도 0 이다. 즉

delta write 는 **self-limiting** 이다: 같은 쌍이 반복제시되면 저장에 완성되는 순간 write 가 멈춘다. Hebbian write 가 같은쌍을 무한히 덧씌우는 것과 대조된다.

2. **memory 의 얼굴**: $\eta_t = 1, \|k_t\| = 1$ 이면 (6-3) 은 “key k_t 방향의 옛 내용을 정확히 지우고 v_t 를 기록” 하는 exact overwrite 다. append-only cache 가 진짜 read-modify-write 저장소로바뀐다.
3. **선형대수의 얼굴**: transition $I - \eta_t k_t k_t^\top$ 는 **generalized Householder 변환**이다 — k_t 방향의 고유값이 $1 - \eta_t \|k_t\|^2$, 나머지 방향은 1. $\eta_t \|k_t\|^2 \in (0, 1)$ 이면 수축, = 2 면 반사다. gate 계열의 transition(스칼라 · 대각) 과 달리 **비대각**이라는사실이 chunkwise 병렬화의 난이도를 결정적으로 바꾸는데 (누적 곱이 elementwise 로 접히지 않는다), Yang et al. 2024 (arXiv:2406.06484) 의 WY representation 이이를 chunk 당 GEMM 두어 번으로 해결했다. 유도는 9 장의 뭉이다.

실무 구현은 key 를 SiLU 통과 후 ℓ_2 normalize 하고, η_t 를 sigmoid head 의 출력 $\eta_t = \sigma(\cdot) \in (0, 1)$ — 원문의 표현으로 “writing strength” — 인 data-dependent 값으로 만든다 (Yang et al. 2024, arXiv:2406.06484, § 3.1 · § 3.3). ℓ_2 정규화는 Yang et al. 2024 의 선택이다: $\|k_t\|_2 = 1$ 이면 $\eta_t = 1$ 일 때 $I - k_t k_t^\top$ 가 정확한 projection 이 되어 위의 exact-overwrite 해석이 성립한다 (원형인 Schlag et al. 2021, arXiv:2102.11174, § 4.2 는 ℓ_1 계열 sum normalization 을 썼다). 결과적으로 η_t 가 “이 token 을 얼마나 세게쓸 것인가” 의 학습된 per-token write intensity 가 된다.

Gated DeltaNet(이하 GDN; Yang, Kautz & Hatamizadeh 2025, arXiv:2412.06464) 은 두 계보의 합류다: 식 (6-2) 의 retention 과 식 (6-3) 의 targeted overwrite 를 한 update 에 싣는다 [Titans § 2; Miras Eq. 9]:

$$W_t = \alpha_t W_{t-1} (I - \eta_t k_t k_t^\top) + \eta_t v_t k_t^\top. \quad (6-4)$$

([Miras Eq. 9] 는 write 항의 η_t 를 흡수한 표기이고좌 · 우곱은 행벡터 관행의 차이다 — 알고리즘은 동일하다.) 의미는 정확히 “전역 eviction(α_t) + 항목별 overwrite(η_t)” 이며, Miras 의 4 축으로는 ℓ_2 bias + ℓ_2 retention + 1-step GD 의 자리에 놓인다 [Miras Table 1]. 이형태가 이 장 family 의 사실상의 완성형이고, Titans 는 여기서 optimizer 축 (momentum) 과 memory 구조 축 (deep MLP) 을 더 밀고 나간 것이다 (→ 12 장).

6.5 Longhorn: update rule 을 발명하지 말고, 문제를풀어라

지금까지의 모델들은 update rule 을 설계했다. **Longhorn** (Liu et al. 2025, arXiv:2407.14207) 의 제안은방법론 자체의 전환이다: per-token 으로 풀고 싶은 online 최적화 문제를먼저 적고, 그 문제의 **closed-form 해**를 update rule 로 삼는다. 논문의 표현으로 SSM 은 “amortized online learner” 다. 3 장에서 배운 online learning 의 프레임이 여기서 처음으로 모델 유도에 통째로 쓰인다.

논문 자신이 이 전환을 한 판에 담는다 (그림 6-1): sequence mixing layer 를 “history 를 state 로 압축하는 online learner” 로 보고, 일반형 online 목적 (이전 state 에 대한 근접함 + per-token 손실) 의 argmin 을 update 로 삼은뒤, 그 목적을 proximal ℓ_2 로 특수화하면 아래 (6-5) 의 닫힌 해가떨어진다는 그림이다.

그림 6-1 — sequence mixing 을 online learner 로 보는 Longhorn 의 관점. 가운데는 일반형 online 목적함수와 그 argmin update, 오른쪽은 proximal ℓ_2 로 특수화한 Longhorn 목적함수와 닫힌 해 (그림의 state S = 본서의 W_t , target $x = v_t$). 본문 식 (6-5) 의 그림 판. 출처: Liu et al., Longhorn (arXiv:2407.14207), 원문 Fig.2 — 원저자의 그림 (third-party), 본서의 결과가 아님.

문제 설정은 proximal 형태다: 새 쌍은 잘 맞추되, 이전 state 에서 너무멀어지지 말 것.

$$W_t = \arg \min_W \|W - W_{t-1}\|_F^2 + \eta_t \|W k_t - v_t\|_2^2.$$

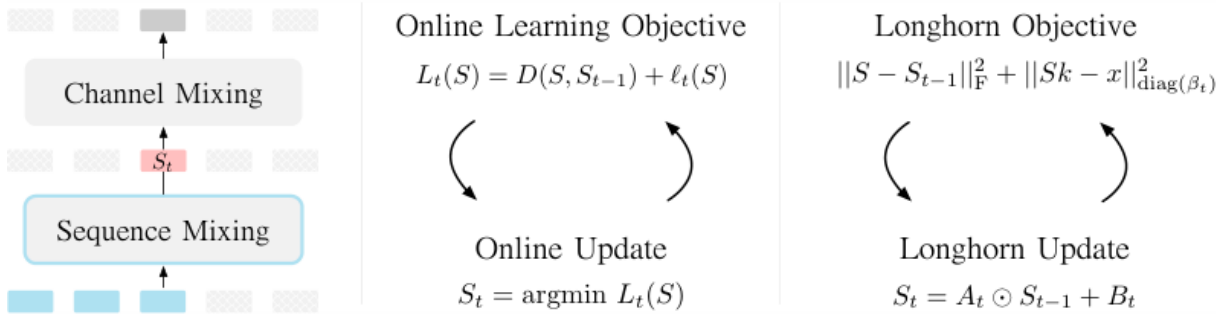


그림 6-1 — sequence mixing 을 online learner 로 보는 Longhorn 의관점. 가운데는 일반형 online 목적함수 $L_t(S) = D(S, S_{t-1}) + \ell_t(S)$ 와 그 argmin update, 오른쪽은 이를 proximal ℓ_2 로 특수화한 Longhorn 목적함수와 그 닫힌 해다 (그림의 state S 가 본서의 W_t , target x 가 v_t). 본문 식 (6-5) 유도의 그림 판. 출처: Liu et al., Longhorn (arXiv:2407.14207), 원문 Fig.2 —원저자의 그림 (third-party), 본서의 결과가 아님.

이차식이므로 미분해서 0 으로 놓으면 $W(I + \eta_t k_t k_t^\top) = W_{t-1} + \eta_t v_t k_t^\top$, Sherman – Morrison 으로 역행렬을 풀면

$$W_t = W_{t-1}(I - \epsilon_t k_t k_t^\top) + \epsilon_t v_t k_t^\top, \quad \epsilon_t = \frac{\eta_t}{1 + \eta_t k_t^\top k_t}. \quad (6-5)$$

표기 주의 두 가지. 첫째, 원문의 objective 는 두 번째 항이 **출력채널별 가중 노름**이다 — 스칼라 η_t 자리에 채널별 벡터 $\eta_t \in \mathbb{R}^{d_v}$ (sigmoid 출력; 원문 기호 β_t) 가왔고, $\eta_{t,i} = 0$ 인 채널의 state 행은 그대로 보존된다. 닫힌 해도그에 맞춰 출력 행별로 $\epsilon_{t,i} = \eta_{t,i} / (1 + \eta_{t,i} k_t^\top k_t)$ 를 갖는다 [Longhorn Eq. 5, Thm 3.1]. 위 (6-5) 는 이 행별 식에서 η_t 를스칼라로 둔 단순화이며, 유도의 본질 (implicit GD, 아래 안정성 논증) 은그대로 보존된다. 둘째, 원문 구현은 parallel scan 을 위해 transition $I - \epsilon_{t,i} k_t k_t^\top$ 를 대각 근사 $\mathbf{1} - \epsilon_{t,i} k_t^{\odot 2}$ 로 치환한다 [Longhorn § 3.2] — 즉출하된 Longhorn kernel 은 delta 계열의 Householder 형 transition 이 아니라 그 **대각 근사판**이다. 근사의 대가로 얻는 것은병렬화 형태다: full transition 은 비대각이라 WY 계열 chunkwise 기법을요구하지만, 대각 transition 은 GLA 류 scan 으로 접한다 (→ 9 장). 이하의분석은 근사 전의 full transition 에 대한 것이다.

형태는 DeltaNet (6-3) 그대로이고, 바뀐 것은 learning rate 의 재정의하나다. 그러나 이 하나가 optimizer 의 **종류**를 바꾼다. (6-3) 은 gradient 를 W_{t-1} 에서 평가하는 explicit GD(forward Euler) 이고, (6-5) 는 gradient 를 도착점 W_t 에서 평가하는 방정식 $W_t = W_{t-1} - \eta_t \nabla_W \ell(W_t; k_t, v_t)$ 을 푼 **implicit gradient descent**(backward Euler, proximal step) 다. Miras Table 1 이 Longhorn 에만 “Implicit GD” 라는 별도의 행을 준 이유다 [Miras Table 1].

implicit 의 값어치는 무조건 안정성이다. k_t 방향의 transition 계수를비교하면:

- explicit (6-3): $1 - \eta_t \|k_t\|^2$. $\eta_t \|k_t\|^2 > 2$ 면절댓값이 1 을 넘어 state 가 폭주한다. 안정성이 step size 제약이라는형태로 사용자에게 전가된다.
- implicit (6-5): $1 - \epsilon_t \|k_t\|^2 = \frac{1}{1 + \eta_t \|k_t\|^2} \in (0, 1)$ —**임의의 $\eta_t > 0$ 에서** 안정하다. $\eta_t \rightarrow \infty$ 극한에서도 발산하지 않고 “이 key 에 대해 v_t 를 정확히 저장하라” 는 hard write 로 수렴할 뿐이다.

training 무경험 독자를 위해 옮기면: explicit GD 의 step size 는 발산이라는절벽이 있는 tuning 대상이고, implicit GD 는 그 절벽을 수식 안에서 제거한것이다. “임의의 $\eta_t > 0$ 에서 안 터진다” 는 것은 closed-form 의 수학적성질이라, gate head 가 어떤 값을 내놓아도 안정성이 보장

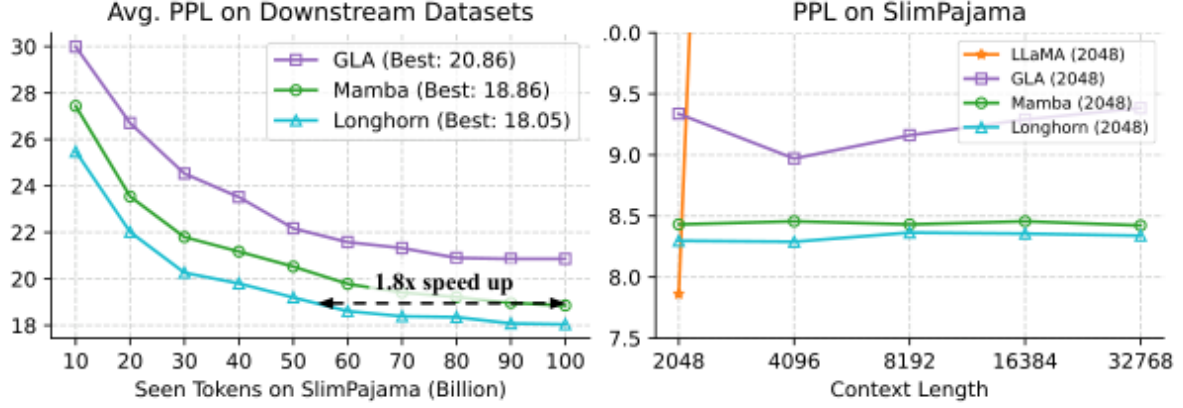


그림 6-2 — Longhorn의 경험적 결과. (왼쪽) SlimPajama 학습 토큰대비 8개 downstream 평균 perplexity — Longhorn이 GLA·Mamba보다 낮고 약 $1.8\times$ sampling efficiency. (오른쪽) 2048 문맥으로 학습한 모델이 최대 $16\times$ 긴 문맥까지 perplexity 열화 없이 외삽. 출처: Liu et al., Longhorn (arXiv:2407.14207), 원문 Fig.1 — 원저자의 그림 (third-party), 본서의 결과가 아님.

되어 step-size 안정성이 outer-loop 학습에서 분리된다 — 실제 Longhorn은 여기에 더해 η_t (원문 β_t) 를 sigmoid 로 $(0, 1)$ 에 가두지만, 안정성 자체는 그 제한과 무관한 closed-form 의 성질이다 [Longhorn § 3.2]. retention gate 는 없다 ($\alpha \equiv 1$): proximal 항 $\|W - W_{t-1}\|_F^2$ 자체가 유일한 (local) retention 이며, 전역 감쇠없이도 위의 수축 계수가 오래된 내용을 서서히 밀어낸다. systems 관점의 비용은 정직하게 0 에 가깝다 — ϵ_t 계산은 내적 하나와 나눗셈 하나이고, kernel 구조는 DeltaNet 과 동일하다. “공짜 안정성” 이라는 점이 이모델의 systems 요약이다.

이 무조건 안정성은 종이 위 성질에 그치지 않는다. Longhorn은 2048 문맥으로 학습한 모델이 최대 $16\times$ 긴 문맥까지 perplexity 열화 없이 외삽하고, 같은 규모의 GLA·Mamba 대비 downstream perplexity 와 sampling efficiency 모두에서 앞선다고 보고한다 (그림 6-2) [Longhorn Fig. 1].

그림 6-2 — Longhorn의 경험적 결과. (왼쪽) 학습 토큰 대비 downstream 평균 perplexity 에서 Longhorn이 GLA·Mamba보다 낮고 약 $1.8\times$ sampling efficiency. (오른쪽) 2048 문맥 학습 → 최대 $16\times$ 긴 문맥 외삽, perplexity 열화 미미. 출처: Liu et al., Longhorn (arXiv:2407.14207), 원문 Fig.1 — 원저자의 그림 (third-party), 본서의 결과가 아님.

6.6 RWKV-7: generalized delta rule — gate 를 채널별 벡터로

RWKV-7 “Goose” (Peng et al. 2025, arXiv:2503.14456) 는 이 장 family 의 현재 시점 최종 일반화다. GDN (6-4) 의 스칼라 손잡이들을 전부 벡터로 승격한다: 스칼라 retention α_t 는 채널별 decay 벡터 $w_t \in (0, 1)^{d_k}$ 로, 스칼라 write intensity η_t 는 채널별 **in-context learning rate** 벡터 $a_t \in [0, 1]^{d_k}$ 로 승격되고, 지우는 key 와 쓰는 key 가 **같은 key 의 서로 다른 채널별 변조**로 분리된다: 제거용 $\hat{k}_t$ 는 k_t 에 학습된 채널별 배율 (원문의 removal-key multiplier) 을 $\odot$ 로 곱한 뒤 head 별 ℓ_2 정규화한 것이고, 기록용 $\tilde{k}_t$ 는 k_t 의 각 채널을 a_t 방향으로 보간한 것이다 — $\tilde{k}_t = k_t \odot \text{lerp}(\mathbf{1}, a_t, \nu)$, ν 는 학습된 보간 계수 (원문 표현 “replacement rate booster”) [RWKV-7 Eq. 6–7, Eq. 15]. 별도의 projection 행렬을 더 두는 것이 아니라 같은 key precursor 를 채널 단위로 두 갈래 성형하는 것이므로, projection 비용은 그대로다. 구조적으로 쓰면

$$W_t = W_{t-1} \left(\text{Diag}(w_t) - \hat{k}_t (a_t \odot \hat{k}_t)^\top \right) + v_t \tilde{k}_t^\top. \quad (6-6)$$

(6-6) 은 원문의 state evolution [RWKV-7 Eq. 17] 을 행벡터 관행에서 이책의 열벡터 관행으로 전치한 것으로, a_t 가 제거 항 내부에 $\odot$ 로 곱해지는 위치까지 원문과 배치가 같다. 제거 key 를 ℓ_2 정규화해두는 이유도 원문이 명시한다: 제거량을 단위 norm 으로 고정해 두면 in-context learning rate a_t 가 “state 에서 얼마나 지우고 얼마나 다시 써 넣는가” 를 다른 항에 오염되지 않고

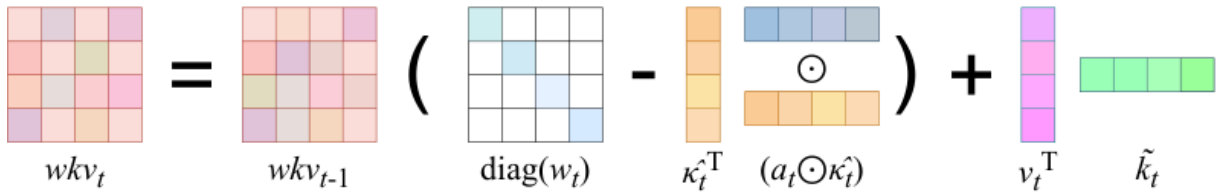


그림 6-3 — RWKV-7의 state 갱신을 head 하나에 대해 시각화한 원문그림 (실제 state는 head 당 64×64 , 그림은 4×4 축소). 대각 retention $\text{Diag}(w_t)$ 에서 제거 key의 rank-1 항을 뺀 “대각 - rank-1” transition에 기록 항 $v_t \tilde{k}_t^\top$ 을 더하는 구조로, 본문 식 (6-6)과 동일하다 (그림의 state wkv_t 가 본서의 W_t , $\hat{k}_t$ 가 제거 key $\hat{k}_t$; 원문은 행벡터 관행이라 전치위치가 반대). 출처: Peng et al., RWKV-7 “Goose” (arXiv:2503.14456), 원문 Fig.2 — 원저자의 그림 (third-party), 본서의 결과가 아님.

단독으로 조절하는 손잡이가 된다 [RWKV-7 Eq. 7 부근]. GDN에서는 지우기 강도와 쓰기 강도가 η_t 하나에 묶여 있었다 — (6-6)은 그 묶음을 채널 단위로 풀어낸 것이다.

원문은 이 갱신을 head 하나에 대한 그림으로 직접 보여준다 (그림 6-3): “대각 - rank-1” transition에 기록 항 $v_t \tilde{k}_t^\top$ 을 더하는 (6-6)의 구조가, 지우는 key $\hat{k}_t$ 와 쓰는 key $\tilde{k}_t$ 가 같은 key precursor의 채널별 두 성형이라는 점과 함께 한눈에 들어온다.

그림 6-3 — RWKV-7의 state 갱신을 head 하나에 대해 시각화한 원문그림 (실제 state는 head 당 64×64 , 그림은 4×4 축소). “대각 - rank-1” transition($\text{Diag}(w_t)$ - 제거 key rank-1)에 기록 항 $v_t \tilde{k}_t^\top$ 을 더하는 구조로 본문 식 (6-6)과 동일 (그림의 wkv_t 는 본서 W_t , $\hat{k}_t = \tilde{k}_t$; 원문은 행벡터 관행). 출처: Peng et al., RWKV-7 “Goose” (arXiv:2503.14456), 원문 Fig.2 — 원저자의 그림 (third-party), 본서의 결과가 아님.

$w_t = \alpha_t \mathbf{1}$, $a_t = \alpha_t \eta_t \mathbf{1}$, $\hat{k}_t = k_t$ 로 두면 transition이 $\alpha_t(I - \eta_t k_t k_t^\top)$ 로 (6-4)의 첫 항과 정확히 일치한다. 단 write 항까지 맞추려면 $\tilde{k}_t = \eta_t k_t$ 여야 하는데 ($v_t \tilde{k}_t^\top = \eta_t v_t k_t^\top$ 가 되도록), RWKV-7의 $\tilde{k}_t = k_t \odot \text{lerp}(\mathbf{1}, a_t, \nu)$ 가 임의의 GDN을 정확히 재현한다는 보장은 원문에 없다. 따라서 (6-6)은 GDN의 transition 구조를 일반화하는 것으로 읽히, 모든 GDN을 부분 경우로 정확히 포함한다는 단정은 유보한다. Miras의 분류로는 delta 계열에서 gate가 채널별 벡터 ($m = d$)인 경우가 정확히 RWKV-7이다 [Miras Eq. 9, § 4]. transition이 “대각 - rank-1”이라는 사실에 주목하라. gate 계열 (순수대각)과 delta 계열 (항등 - rank-1)의 합집합이며, 이 구조 덕에 chunkwise 병렬화는 DeltaNet과 같은 WY 계열 기법으로 처리된다 (→ 9 장).

“이 gate는 누가 학습하는가”라는 § 6.2의 질문을 (6-6)에 적용하면 답은 모두 slow weights로 귀결되나, 두 부류를 구분해야 한다: $w_t \cdot a_t$ 는 작은 head가 token마다 산출하는 data-dependent 값이고 (a_t 도 sigmoid 계열 산출의 $[0, 1]^{d_k}$ 벡터다 [RWKV-7 Eq. 4]), 두 key 변조의 채널 배율 ξ ·보간 계수 ν 는 outer loop가 학습하는 token-independent 고정 파라미터다. 어느 쪽이든 slow weights이고, inner loop에서 움직이는 것은 여전히 W_t 하나뿐이다. 손잡이 수가 늘었을 뿐 두 시간 척도의 구도는 (6-1)에서 한 치도 달라지지 않았다.

정리하면 RWKV-7은 이 장의 gate·delta 계열을 두 축의 격자로 읽게 한다. 한 축은 retention의 해상도 (상수 → data-dependent 스칼라 → 채널별 벡터)이고, 다른 축은 write의 정밀도 (Hebbian additive → delta overwrite → 지우기·쓰기를 분리한 채널별 overwrite)다. RetNet은 retention 축으로만 한 칸, DeltaNet은 write 축으로만 한 칸 움직였고, GDN은 두 축에서 스칼라 한 칸씩 오른 지점이며, RWKV-7은 두 축을 모두 벡터 해상도까지 민 오른쪽 위 모서리다. Longhorn만은 이 격자 밖에 선다 — 좌표를 옮긴 것이 아니라 explicit GD를 implicit GD로 갈아 끼워 step size의 안정성 자체를 다시 정의했기 때문이다 (§ 6.5). Miras가 이 장의 모델을 한 판에 담을 수 있는 것도 격자 위 좌표와 격자 밖 한 점이라는 이 구조 덕이다 [Miras Table 1, § 4]. 이 평면은 여기서 닫힌다 — Titans 이후의 확장 (→ 12 장)은 격자를 더 촘촘히 하는 대신 optimizer 축 (momentum)과 memory 구조 축 (deep MLP)이라는 두 개의 새 차원을 여는 일이다.

[해설] (6-6)을 inference 어휘로 옮기면 이렇다. 채널별 decay w_t 는 cache 항목의

TTL 이 **채널마다 다르게** 설정되는 eviction 이고, 제거용 $\hat{k}_t$ 와 기록용 $\tilde{k}_t$ 의 분리는 같은 cache line 에 대한 invalidate mask 와 write mask 를 따로 가진다는 뜻이며, a_t 는 채널별 write intensity 다. GDN 이 “line 단위 eviction + line 단위 overwrite” 였다면 RWKV-7 은 그 두 연산 모두에 **byte-enable 신호**를 단 것이다 — 제어 자유도는 벡터로 늘었지만, 저장소 ($d_v \times d_k$ 행렬 하나) 와 지배 비용 (state RMW) 은 그대로다.

표현력에 관한 원 논문의 주장도 이 transition 구조에서 나온다: [RWKV-7] 논문은 generalized delta rule 이 (표준 복잡도 가정 하에) transformer 의 TC^0 한계를 넘는 state tracking 을 가능하게 한다고 주장한다 — 정확한 정리 진술과 그 단서는 다음 절에서, 이 주장이 어느 이론지형 위에 놓여 있는지와 함께 본다.

systems 접점: state 는 여전히 head 당 $d_v \times d_k$ 하나이고 decode 의 지배 비용도 그대로다. 추가되는 것은 gate·learning-rate·key 변조를 만들어내는 여러개의 작은 head — RWKV-7 계보의 관행대로 low-rank projection — 이며, 이는 decode 당 skinny GEMM 몇 개, 즉 지배 항 대비 소액이다. **비용은 GDN 급 그대로 두고 update rule 의 자유도만 올린 설계**라는 것이 systems 한 줄 요약이다.

이 장의 모델들을 한 표로 모은다. 각 행이 “무엇을 바꿨는가” 에 답하는지 확인하라 — 전부 식 (M) 의 성분 선택이다.

표 6-2 — 이 장의 모델 카탈로그 (통일 표기; read 는 모두 $y_t = W_t q_t$).

모델	update	inner objective	retention	inner optimizer
linear attention	$W_t = W_{t-1} + v_t k_t^\top$ (6-1)	dot-product	없음	1-step GD (Hebbian)
RetNet	$W_t = \alpha W_{t-1} + v_t k_t^\top$	dot-product	상수 decay	1-step GD
GLA	$W_t = W_{t-1} \text{Diag}(\alpha_t) + v_t k_t^\top$	dot-product	학습된 diagonal	1-step GD
Mamba-2 (→ 7 장)	$W_t = \alpha_t W_{t-1} + v_t k_t^\top$ (6-3)	dot-product	학습된 scalar	1-step GD
DeltaNet	(6-4)	ℓ_2 regression	없음	1-step GD
GDN	(6-4)	ℓ_2 regression	학습된 scalar	1-step GD
Longhorn	(6-5)	ℓ_2 regression	없음 ($\alpha \equiv 1$)	implicit GD
RWKV-7	(6-6)	ℓ_2 regression	채널별 vector	1-step GD (generalized delta)
TTT-Linear (→ 8 장)	(M1)	ℓ_2 regression	없음	1-step GD
Titans-LMM (→ 12 장)	(M2)	ℓ_2 (deep memory)	$\alpha_t +$ momentum	GD + momentum

6.7 Expressivity 사이드바: state 의 착각과 음의 고유값

이 절은 반 페이지짜리 지도다 — Titans·Atlas 가 “deep memory” 를 주장할 때 덮고 서는 이론 지형이 여기 있다.

Merrill et al. 2024 (arXiv:2404.08819) 는 대각 transition 의 SSM 이 (log-precision 가정 하에) transformer 와 같은 회로복잡도 계열 TC^0 에 머문다고 주장한다 — 겉보기에 재귀적 state 가있어도 순차 계산 고유의 문제 (예: S_5 치환 합성 같은 NC^1 -complete state tracking) 를 풀 수 없다는, 논문 제목 그대로 “illusion of state” 다. 반면 Grazzi et al. 2025 (arXiv:2411.12537) 는

DeltaNet 류의 비대각 transition 에서 η_t 의 허용 범위를 $(0, 2)$ 로 넓혀 transition 고유값이 $[-1, 1]$ 전체를 덮게 하면 — 즉 음의 고유값을 허용하면 — parity 같은 state-tracking 문제가 풀리게 됨을 보였다. DeltaProduct (Siems et al. 2025, arXiv:2502.10297) 는 token 당 GD 를 여러 step 밟아 (Householder 곱; [Miras Table 1] 의 multi-step GD 행) transition 의 rank 자체를 올린다.

§ 6.6 이 예고한 RWKV-7 의 표현력 주장은 정확히 이 지형 위에 놓인다. 원논문의 정리는 둘이다. 첫째, **단일 layer** RWKV-7 이 S_5 원소 5 개의 swap tracking — AC^0 환원 하에서 NC^1 -complete 인 문제 — 을 푼다 [RWKV-7 Thm 2, App. D.1]. 둘째, 임의의 정규 언어에 대해 그것을 인식하는 **4-layer** RWKV-7 모델이 존재한다 [RWKV-7 Thm 3, App. D.2]. 두 정리 모두 $TC^0 \neq NC^1$ conjecture 를 전제로 “transformer 가 못 하는 것을 한다” 로 읽힌다. 그리고 단서 하나가 결정적이다: 증명은 (6-6) 의 제거 항에 계수 2 를 둔 변형 $-W_{t-1}(\text{Diag}(w_t) - 2\hat{k}_t(a_t \odot \hat{k}_t)^\top)$ 꼴, 즉 transition 고유값 -1 을 허용하는 판 — 에 대한 것이다 [RWKV-7 App. D.1]. 출하 아키텍처 (계수 1) 가 아니라 음의 고유값을 허용한 변형에 대한 결과이며, 이는 Grazi et al. 의 $\eta_t \in (0, 2)$ 확장과 같은 기제다. “RWKV-7 이 TC^0 를 넘는다” 를 옮길 때는 이 단서 — layer 수, complexity conjecture, 그리고 계수 2 변형 — 를 함께 옮겨야 정직한 문장이 된다.

[해설] 이 지형이 6 편에 주는 함의는 다음과 같다. transition 의 구조 (대각 < 대각+rank-1 < 그 곱) 가 곧 모델이 표현할 수 있는 state 동역학의 계급이고, 이 장의 계보는 그사다리를 한 칸씩 오르는 과정이기도 했다. Titans 라인은 여기서 한 축을 더 꺾는다 — transition 을 더 꾸미는 대신 memory 자체를 nonlinear(deep MLP) 로만들고 read 를 $\mathcal{M}(q; W)$ 로 비선형화하는 방향이다 ($\rightarrow$ 8 장, 12 장). matrix memory 의 read 가 q 에 대해 선형이라는 제약은 어떤 gate 로도 벗겨지지 않기 때문이다.

systems 점점 하나: 표현력 사다리는 공짜가 아니라 **병렬화 예산**과 교환된다. 대각 transition 은 scan 으로, rank-1 은 WY 로 병렬화되지만, rank 가 오르고 비선형이 끼어들수록 chunk 경계의 순차 의존이 두꺼워진다. 사다리의 두 칸이 이 교환의 양극단을 보여준다. Grazi et al. 의 $\eta_t \in (0, 2)$ 확장은 transition 고유값 범위만 $[-1, 1]$ 로 넓힐 뿐 여전히 rank-1 이므로 WY 기법이 그대로 통해 표현력을 거의 공짜로 산다. 반대로 DeltaProduct 는 token 당 GD 를 여러 step 밟아 transition rank 를 올리는 대신, chunk 안에서 Householder 곱이 그만큼 순차로 쌓여 병렬화 GEMM 폭이 두꺼워지는 비용을 치른다 [Siems et al. 2025]. state 동역학의 계급을 한 칸 올릴 때마다 tensor core 를 채우는 형태가 조금씩 나빠진다는 이 교환이 이 라인의 kernel 설계 전체를 관통하며, 그 정식 무대가 9 장이고 극한이 TNT 다 ($\rightarrow$ 15 장).

6.8 Worked micro-example: $d = 2$ 에서 세 가지 write 를 손으로 돌린다

$d_k = d_v = 2$. 저장할 두 쌍은

$$k_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, v_1 = \begin{pmatrix} 2 \\ 0 \end{pmatrix}, \quad k_2 = \begin{pmatrix} 0.6 \\ 0.8 \end{pmatrix}, v_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

두 key 모두 단위 norm 이고 $k_1^\top k_2 = 0.6$ — 일부러 직교시키지 않았다. $W_0 = 0$ 에서 시작한다.

(a) Hebbian (6-1). $W_1 = v_1 k_1^\top = \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix}$, $W_2 = W_1 + v_2 k_2^\top = \begin{pmatrix} 2 & 0 \\ 0.6 & 0.8 \end{pmatrix}$.

읽기: $W_2 k_1 = (2, 0.6)^\top$ — 참값 $v_1 = (2, 0)^\top$ 대비 둘째채널에 0.6 의 crosstalk. 정확히 $v_2(k_2^\top k_1)$, 즉 5 장의 crosstalk 공식 그대로다. $W_2 k_2 = (1.2, 1.0)^\top$ — 이번엔 옛 쌍이새 key 의 읽기를 오염시킨다 ($v_1(k_1^\top k_2) = (1.2, 0)^\top$). 어느 key 도 정확히 회수되지 않는다.

(b) DeltaNet (6-3), $\eta_t = 1$. step 1 은 $W_0 = 0$ 이라 Hebbian 과 동일: $W_1 = \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix}$. step 2 에서 먼저 읽는다: 예측 $W_1 k_2 = (1.2, 0)^\top$, 오차 $e = W_1 k_2 - v_2 = (1.2, -1)^\top$. 오차의 outer product 를 뺀다:

$$W_2 = W_1 - e e k_2^\top = \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix} - \begin{pmatrix} 0.72 & 0.96 \\ -0.6 & -0.8 \end{pmatrix} = \begin{pmatrix} 1.28 & -0.96 \\ 0.6 & 0.8 \end{pmatrix}.$$

읽기: $W_2k_2 = (1.28 \cdot 0.6 - 0.96 \cdot 0.8, 0.36 + 0.64)^\top = (0, 1)^\top = v_2$ — **exact**. 최신 key 는 완벽히 저장된다. 대신 $W_2k_1 = (1.28, 0.6)^\top$: 옛 쌍은 새 key 와 겹치는 성분만큼 수정됐고, 오차 크기는 ≈ 0.94 로 Hebbian 의 0.6 보다 오히려 크다. 이것은 버그가 아니라 semantics 다 — delta rule 이 보장하는 것은 **최신 binding 의 정확성** (마지막으로 쓴 값이 이긴다) 이지 과거의 보존이 아니다. 과거 보존은 retention 축의 몫이고 (α_t , 13 장), 같은 쌍들이 반복 제시되면 (여기처럼 선형독립 · consistent 한 경우) LMS 로서 그 해에 수렴한다 ($\rightarrow$ 5 장; inconsistent 계에서는 감소 step size 가 필요하다). 두 key 가 직교했다면 (a) 와 (b) 모두 오차가 0 이었음을 직접 확인해 보라 — 간섭의 원천은 오직 $k_1^\top k_2 \neq 0$ 이다.

(c) **Longhorn (6-5)**. $\eta_2 = 1$ 이면 $\epsilon_2 = 1/(1+1) = 0.5$: 읽기 $W_2k_2 = 0.5(1.2, 0)^\top + 0.5(0, 1)^\top = (0.6, 0.5)^\top$ — 명목 η 가 같아도 절반만 쓴다. step size 를 키우면: $\eta_2 = 4$ 일때 explicit (6-3) 은 k_2 방향 계수가 $1 - 4 = -3$ 이 되어 읽기가 $(-3.6, 4)^\top$ 으로 폭주하지만, implicit 은 $\epsilon_2 = 4/5 = 0.8$ 로 $(0.24, 0.8)^\top$ — v_2 에 안정적으로 접근한다. $\eta_2 \rightarrow \infty$ 극한에서 $\epsilon_2 \rightarrow 1$, 즉 (b) 의 exact overwrite 로 수렴한다.

(d) **gate 의 시간 상수 감각**. RetNet 처럼 $\alpha = 0.9$ 상수면 100 token 뒤 첫 write 의 잔존 계수는 $0.9^{99} \approx 3 \times 10^{-5}$ — eviction 의 반감기가 $\log 2 / \log(1/0.9) \approx 6.6$ token 이다. gate 값이 곧 cache 항목의 TTL 이라는 대응을 숫자로 확인할 수 있다. 채널별 gate 의 값어치도 같은 산수로 보인다: RWKV-7 류 decay 벡터가 $w = (0.9, 0.999)$ 라면 첫째 채널의 반감기는 6.6 token, 둘째 채널은 $\log 2 / \log(1/0.999) \approx 693$ token — 같은 state 행렬 안에 두 자릿수 차이의 시간 척도가 채널 단위로 공존한다. RetNet 이 head 단위로만 만들 수 있던 다중 시간 척도를 (6-6) 은 채널 단위로, 그것도 token 마다 다시 정해서 만든다.

표 6-3 — micro-example 결과 요약 (읽기 오차의 ℓ_2 norm).

write 방식	$q = k_1$ 오차	$q = k_2$ 오차	성질
Hebbian (6-1)	0.60	1.20	둘 다 부정확, 감쇠 기 제 없이 norm 누적
DeltaNet (6-3), $\eta = 1$	0.94	0 (exact)	최신 binding 우선, self-limiting
Longhorn (6-5), $\eta = 1$	—	0.78	절반 write, 무조건 안정

6.9 Systems bridge: 당신의 roofline 위에 올려놓기

이제 이 family 를 독자의 모국어 — bytes, FLOPs, crossover — 로 결산한다. head 당, bf16(2 bytes), $d_k = d_v = d_h$ 기준이다.

표 6-4 — per-token decode 비용 (per head; L = 현재 문맥 길이, w = window 크기).

구조	상주 state (원소 수)	decode FLOPs/ token	decode traffic/ token	prefill/train 병렬 형태
softmax attention	$L(d_k + d_v)$ — 증가	$\approx 4Ld_h$	KV 전체 read $\approx 2L(d_k + d_v)$ B	attention GEMM (exact tiling)
sliding-window attn	$w(d_k + d_v)$	$\approx 4wd_h$	$\approx 2w(d_k + d_v)$ B	동일, window 마스크
linear attn / RetNet / GLA	$d_k d_v$ — 고정	$\approx 4 - 6 d_k d_v$	state RMW $\approx 4 d_k d_v$ B	chunkwise GEMM + (scan) ($\rightarrow$ 9 장)

구조	상주 state (원소 수)	decode FLOPs/ token	decode traffic/ token	prefill/train 병렬 형태
DeltaNet / GDN / Longhorn	$d_k d_v$ — 고정	$\approx 6 - 8 d_k d_v$	동일	chunkwise GEMM + WY (→ 9 장; 출하 Longhorn kernel 은 대각 근사 scan — § 6.5)
RWKV-7	$d_k d_v$ + gate head 들	상동 + low-rank head	동일	동일계열

세 가지 계산을 직접 해보면 이 표가 몸에 붙는다.

첫째, crossover. state 가 KV cache 보다 작아지는 지점은 $L^* = \frac{d_k d_v}{d_k + d_v}$ 이다. $d_k = d_v = 128$ 이면 $L^* = 64$ token — 고작 64 token 만 넘으면 고정 state 가 이긴다. 단, 독자의 스택이 GQA 로 KV head 를 8:1 로 줄이고 있다면 KV 의 token 당 발자국이 $8\times$ 작아져 crossover 는 512 token 으로 밀린다. “linear attention 은 언제나 메모리이득” 이 아니라 **자기 서빙 구성에 대입해야 하는 산수**라는 뜻이다. 모델 전체 감각: $32 \text{ layer} \times 32 \text{ head} \times d = 128$ 이면 state 는 $32 \cdot 32 \cdot 128 \cdot 128 \cdot 2 \text{ B} = 32 \text{ MiB}$ 로 문맥 길이와 무관한 상수이고, 같은 구성의 MHA KV cache 는 token 당 512 KiB — 4K 문맥에서 이미 2 GiB 다.

둘째, roofline 위치. DeltaNet decode 한 step 은 head 당 FLOPs $\approx 6 d_k d_v$, state RMW traffic $\approx 4 d_k d_v$ B 로 산술강도가 ~ 1.5 FLOP/B 다. softmax attention decode 의 강도도 ~ 1 FLOP/B 수준 — **둘 다 깊은 bandwidth-bound 이고, linear 계열의 승리는 강도 개선이 아니라 traffic 절대량의 상한**이다. tensor core 를 놀리지 않는 형태 (GEMM-rich) 로 바꾸는것은 decode 가 아니라 prefill/training 의 chunkwise 재구성이하는 일이며 (→ 9 장), 이 구분 — decode 는 bandwidth 문제, training 은 utilization 문제 — 이 이 라인의 systems 논의 전체를 가로지른다.

셋째, batching. 식 (6-1)~(6-6) 의 read/write 는 요청마다 다른 W_t 에 대한 GEMV/GER 이므로, weight 가 공유되는 보통의 batched GEMM 과 달리 **state 축이 batch 에 들어온다**. batch B 의 decode 는 B 개의 독립 GEMV — 사실상 batched/grouped GEMV — 가 되고, per-request state $B \cdot 32 \text{ MiB}$ 가 HBM 상주분으로잡힌다. KV cache 관리자가 하던 일 (할당, 상주, 축출, 요청 간 격리) 을 fast-weight state 관리자가 그대로 물려받는 그림이며, 이 문제의식은 TTT 계열에서 backward pass 까지 decode 에 들어오면서 본격화된다 (→ 8 장, 10 장).

요약

- linear attention 은 kernel trick 으로 softmax attention 의 KV cache 를 head 당 $d_v \times d_k$ 고정 행렬 W_t 로 손실 압축한 것이다. write 는 rank-1 outer product, read 는 GEMV 다 [Titans Eq. 3 – 5].
- W_t 는 fast weights — inner loop 에서 token 마다 움직이는 상태 —이고, projection·gate head 등 Θ 는 slow weights 다. linear attention = fast-weight programming 이라는동치 (Schlag et al. 2021) 가 이 라인의 역사적 기점이다.
- Hebbian write 는 dot-product bias 의 1-step GD 라서 자기 제한이 없고 crosstalk 가 누적 된다. 교정 축은 둘이다: retention(RetNet 의 상수 → GLA·Mamba-2 의 data-dependent gate) 과 write 교정 (delta rule) [Titans § 2; Miras § 4].
- DeltaNet 의 update (6-3) 은 ℓ_2 regression 의 1-step GD 이며 transition 은 generalized Householder 다. GDN (6-4) 은 retention 과 overwrite 를 결합한 이 family 의 완성형이다.
- Longhorn (6-5) 은 같은 문제의 closed-form proximal 해 = implicit GD 로, 임의의 $\eta_t > 0$ 에서 무조건 안정이다. update rule 설계가 “online 문제 선택 + optimizer 선택” 으로 대

채될 수 있음을 보인 사례다.

- RWKV-7 (6-6) 은 스칼라 gate 들을 채널별 벡터로 일반화한 generalized delta rule 이다 [RWKV-7 Eq. 17]. 지우는 key 와 쓰는 key 는 별도 projection 이 아니라 같은 key 의 채널별 변조이며 [RWKV-7 Eq. 6-7, Eq. 15], 표 6-2 의 전 모델이 식 (M) 의 성분 선택 하나씩으로 구분된다 [Miras Table 1].
- 원문 이론 · 구현의 단서 두 가지: 출하된 Longhorn kernel 은 delta transition 의 대각 근사판이고 [Longhorn § 3.2], RWKV-7 의 TC^0 초과 정리 (1-layer S_5 tracking, 4-layer 정규 언어) 는 제거항 계수 2 - 음의 고유값 허용 - 변형에 대한 것이다 [RWKV-7 Thm 2-3, App. D].
- decode 에서 이 family 의 이득은 산술 강도가 아니라 traffic 상한이다. $crossover(L^* = d_k d_v / (d_k + d_v))$ 와 per-request state 상주는자기 서빙 구성으로 계산해봐야 하는 산수다.

자가 점검체크리스트

- softmax attention 에서 출발해 식 (6-1) 의 재귀형을 유도하고, 어디서근사가 들어갔는지 (kernel 교체) 지목할 수 있다.
- 표 6-2 의 각 행에 대해 “inner objective 가 무엇이고, optimizer 가 무엇이고, retention 이 무엇인가” 에 즉답할 수 있다.
- $d = 2$ 예제를 백지에서 재계산해 Hebbian 의 crosstalk 와 delta 의 exact overwrite(그리고 옛 key 의 손상) 를 재현할 수 있다.
- Longhorn 의 $\epsilon_t = \eta_t / (1 + \eta_t k_t^T k_t)$ 를 Sherman - Morrison 으로 유도하고, explicit GD 와의 안정성 차이를 transition 고유값으로 설명할 수 있다.
- “이 gate(α_t, η_t, a_t) 는 누가 학습하는가?” 에 fast/slow weights 구분으로 답할 수 있다.
- 이 장의 내용을 inference 어휘로 옮길 수 있다: state = 압축된 KV cache, retention gate = 학습된 eviction, delta write = read-modify-write, decode 비용 = 고정 크기 state 의 RMW traffic.

다음 장으로

이 장은 linear attention 쪽 계보만 따라왔지만, 같은 시기에 전혀 다른출발점 — 연속 시간 state space model — 에서 정확히 같은 지점에 도착한계보가 있다. S4에서 Mamba 로, 그리고 Mamba-2 에 이르러 두 계보가수학적으로 동일함이 증명된다 (state-space duality). 7 장은 그 합류를 다룬다: 왜 Mamba-2 의 gate 가 표 6-2 의 α_t 행에 앉아 있는지, 그리고 scan 기반 계보가 어떻게 GEMM 기반 chunkwise 형태로 수렴하는지. 그다음 8 장에서 비로소 질문이 뒤집힌다 — state 를 행렬이 아니라 **작은모델의 weights** 전체로 만들면 어떻게 되는가?

ch07. SSM 계보: S4 → Mamba → Mamba-2, 그리고 SSD duality

이 장의 목표 — 이 장을 마치면 다음을 할 수 있어야 한다. (1) continuous SSM의 ZOH discretization을 두 줄로 재현하고, step size Δ_t 가 왜 이 라인 전체의 gate의 기원인지 설명한다. (2) Mamba의 selectivity가 왜 convolution 훈련 모드를 제거하고 scan kernel을 강제하는지 설명한다. (3) SSD(state-space duality)에 따라 같은 recurrence를 recurrent form, masked-attention form, chunkwise form 세 경로로 손계산하고 결과가 일치함을 보인다. (4) Mamba-1의 scan과 Mamba-2의 chunkwise GEMM을 roofline 어휘로 비교한다.

왜 필요한가 — [Titans] (Titans: Learning to Memorize at Test Time, arXiv:2501.00663)는 이 장의 gate 계보를 자기 update의 weight decay로 일반화하는 것으로 스스로를 자리매김하고 [Titans § 3.1, App. A.1], momentum recurrence의 병렬화에 S5의 parallel associative scan을 그대로 재사용한다 [Titans § 3.2, Eq. 18]. [Miras] (It's All Connected, arXiv:2504.13173)의 general form $W_t = A_t * W_{t-1} + v_t k_t^\top$ [Miras Eq. 3]과 분류표 [Miras Table 1]가 Mamba-2를 linear attention(→ 6장)과 한 표에 넣을 수 있는 형식적 근거가 바로 이 장의 SSD다. Atlas와 TNT를 포함해 Part II의 실험 절에는 이 계열이 baseline으로 반복 등장한다. 마지막으로 Mamba-2의 chunked block decomposition은 9장에서 배울 chunkwise-parallel training의 첫 리허설이다 — 단, 여기서는 chunk가 아직 “정확한(exact)” 성능 knob이라는 점이 결정적 차이다.

독자는 이 계열을 이미 서빙해 봤을 가능성이 높다. Mamba block의 decode가 왜 빠른지, state가 왜 고정 크기인지는 운영 감각으로 알고 있을 것이다. 이장의 목적은 그 운영 감각 밑에 깔린 유도를 채우고, 6장에서 만든 linear attention 어휘와 이 계열을 **하나의 update** 식으로 접합하는 것이다. 접합이 끝나면 “SSM 이냐 linear attention 이냐”는 질문 자체가 소멸하고, Part II의 논문들이 그러듯 “retention gate가 무엇이고 write rule이 무엇인가”만 남는다.

7.1 continuous SSM과 discretization: 모든 gate의 기원

state space model(SSM)은 연속 시간 선형 시스템을 sequence layer로 이식한 것이다. 이 소절에서만 t 를 연속 시간 변수로 쓰고, discretization 후에는 책의 표준대로 token 인덱스로 되돌린다. 채널하나(스칼라 입력 $x(t) \in \mathbb{R}$)에 대해 hidden state $h(t) \in \mathbb{R}^N$ 을 두고

$$h'(t) = A h(t) + B x(t), \quad y(t) = C^\top h(t)$$

로 정의한다. $A \in \mathbb{R}^{N \times N}$, $B, C \in \mathbb{R}^N$ 은 이 소절에서는 상수다. 독자의 어휘로 먼저 번역하면: h 는 길이와 무관한 **고정 크기 buffer**이고, 이 buffer가 지금까지의 입력 스트림 전체를 lossy하게 압축해 들고 있다. Rosetta 사전(→ 1장)의 “linear-RNN state = 고정 크기 lossy 압축 memory” 행이 정확히 이 대상이다.

token 단위로 계산하려면 discretization이 필요하다. step size Δ 를 두고 **zero-order hold(ZOH)** — 각 step 동안 입력이 상수라고 가정 — 를 적용하면 두 줄로 끝난다:

$$h_t = \bar{A} h_{t-1} + \bar{B} x_t, \quad \bar{A} = \exp(\Delta A), \quad \bar{B} = (\Delta A)^{-1} (\exp(\Delta A) - I) \Delta B \approx \Delta B \quad (7-1)$$

ODE 이론은 여기까지만 필요하다. 식 (7-1) 이 말하는 것: discretization 은 연속 시스템을 **linear recurrence** — 즉 6 장의 state 갱신과 같은대수 구조 — 로 바꾸고, 그 계수 $\bar{A}$ 를 “지수 함수를 통과한 Δ ” 로 만든다.

굳이 연속 시간에서 출발하는 이유를 시스템 엔지니어의 언어로 답해 두면: 이것은 물리가 아니라 **parameterization 선택**이다. decay 를 $\exp(\Delta a)$ 꼴로 묶어 두면 $\Delta > 0$ 인 한 유지 비율이 자동으로 (0, 1) 구간에 갇혀 recurrence 가 폭주하지 않고, 초기화도 “시간상수” 라는 해석 가능한 스케일로 잡을 수 있다. 또 하나, 모델 폭 d 의 각채널이 독립적인 N 차원 state bank 를 갖는 SISO 구조라는 점을 기억해두라 — layer 전체 state 가 $d \times N$ 행렬이 되는 이 배치는 § 7.3 에서 head 단위의 $d_v \times d_k$ matrix memory 로 재조직된다.

Δ 의 역할을 뜯어보는 것이 이 장에서 가장 중요하다. A 가 음의실수부를 갖는 diagonal 이라고 하자 (실전 SSM 의 표준 설정). 채널 성분 $a < 0$ 에 대해 $\bar{a} = \exp(\Delta a) \in (0, 1)$ 이다. 그러면:

- $\Delta \rightarrow 0$: $\bar{a} \rightarrow 1, \bar{B} \rightarrow 0$. state 를 **전부 유지**하고 현재 입력을 거의 쓰지 않는다.
- Δ 큼: $\bar{a} \rightarrow 0$ 이고, 안정한 $a < 0$ 에서 $\bar{B} = ((\exp(\Delta a) - 1)/a)B \rightarrow -B/a$ 로 **포화**한다 ($\bar{B} \approx \Delta B$ 는 Δ 가 작을 때의 1 차 근사일 뿐, 큰 Δ 에서는 무한히 커지지 않는다). 옛 state 는 사라지므로 현재 입력이 상대적으로 지배한다.

즉 Δ 는 “시간 해상도” 라는 물리적 해석을 갖지만, 계산적으로는 **유지 비율과 write 강도를 한 knob 으로 묶은 gate** 다. 이 라인의 모든 gate — Mamba 의 selective Δ_t , Mamba-2·GLA 의 decay, [Titans] 의 weight decay, [Miras] 의 retention gate(→ 13 장) — 는 전부 이 한 줄의 후손이다.

S4 와 HiPPO — 한 문단. A 를 아무렇게나 두면 $\bar{A}$ 의 거듭제곱이 신호를 지수적으로 죽이거나 폭주시켜 긴 문맥을 기억하지 못한다. HiPPO(Gu et al. 2020, arXiv:2008.07669) 는 state 가 “지금까지 입력의 polynomial 근사 계수” 를 유지하도록 A 를 구조적으로 설계했고, S4(Gu, Goel & Ré 2022, arXiv:2111.00396) 는 그 구조를 diagonal-plus-low-rank 로 안정화 · 고속화해 긴 sequence 벤치마크를 처음 뚫었다. 세부는 이 책에 필요 없다. 필요한 것은 하나: S4 는 **LTI(linear time-invariant)** — A, B, C, Δ 가 token 에 무관 — 라는 사실이다.

LTI 의 계산적 귀결은 독자에게 익숙한 그림이다. 계수가 시불변이면 recurrence 전체가 convolution 으로 접힌다: $y = \bar{K} * x, \bar{K} = (C^\top \bar{B}, C^\top \bar{A} \bar{B}, C^\top \bar{A}^2 \bar{B}, \dots)$. 그래서 S4 는 훈련 · prefill 에서는 FFT convolution 으로 완전병렬, decode 에서는 식 (7-1) 의 recurrence 로 token 당 $O(N)$ 이다. **같은 모델이 phase 에 따라 두 개의 계산 모드를 갖는다** — 독자가 매일 다루는 prefill/decode asymmetry 가, 커널 구현이 아니라 모델 정의 수준에서 나타난 첫 사례다.

S5 와 associative scan — 한 문단. S5(Smith, Warrington & Linderman 2023, arXiv:2208.04933) 는 A 를 diagonal 로 두고, convolution 대신 **parallel associative scan** 으로 recurrence 를 병렬화했다 (선행: Martin & Cundy 2018, arXiv:1709.04057; scan 알고리즘 자체는 Blelloch 1990). 원리는 독자가 아는 prefix-sum 과 동일하고 monoid 만 다르다. $s_t = a_t s_{t-1} + b_t$ 꼴의 recurrence 에서 원소를 (a_t, b_t) 쌍으로 두면, 연속 구간의 합성이

$$(a_2, b_2) \circ (a_1, b_1) = (a_2 a_1, a_2 b_1 + b_2)$$

로 정의되는 associative 연산이 되어 Blelloch tree 로 $O(\log L)$ 깊이에 계산된다. 이 kernel 을 기억해 두라 — [Titans § 3.2, Eq. 18] 는 momentum recurrence $S_t = \beta_t S_{t-1} - \eta_t u_t$ (u_t 는 chunk 안에서 병렬로 구한 per-token gradient) 가 정확히 이 꼴의 linear recurrence 임을 지적하고 같은 scan 으로 푼다. 9 장에서 재회한다.

7.2 Mamba: selectivity — gate 가 input 의 함수가되다

LTI 의 대가는 표현력이다. 계수가 token 에 무관하므로 S4 는 모든 token 을 **같은 비율로** 감쇠시키고 같은 강도로 쓴다. “이 token 은 기억하고자 token 은 무시한다” 는 content 기반 선택이 원리적으

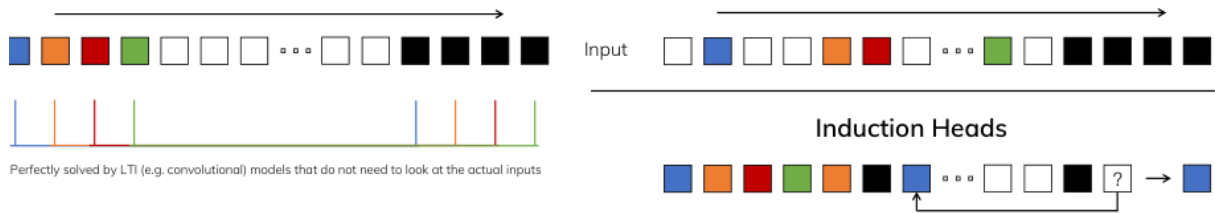


그림 7-1 — Mamba의 selectivity를 요구하는 두 합성 과제. 입력과출력 간격이 일정한 표준 copying(왼쪽)은 실제 입력을 볼 필요가 없어서 LTI convolution 모델이 완벽히 푼다. 그러나 간격이 무작위인 selective copying(오른쪽 위) 과, 문맥에 따라 답을 회수해야 하는 induction heads(오른쪽 아래)는 관련 token(색칠)을 무관 token(흰색)과 content 기준으로 구별해 유지·무시를 결정하는 시변 모델을 요구한다. 이것이 LTI를 깨고 Δ_t, B_t, C_t 를 입력의 함수로 만든 이유다. 출처: Gu & Dao, Mamba (arXiv:2312.00752), 원문 Fig.2 — 원저자의 그림 (third-party), 본서의 결과가 아님.

로 불가능하다. Gu & Dao 2023 (arXiv:2312.00752)은 selective copying과 induction head류의 합성 과제로 이 한계를 시연하고 (그림 7-1), 해법으로 **selectivity**를 제안했다: Δ_t, B_t, C_t 를 입력 x_t 의 함수로 만든다 (A 자체는 고정하되 Δ_t 를 통해 시변이된다). 이것이 Mamba다.

selectivity의 의미는 § 7.1의 Δ 해석에서 바로 나온다. $\bar{a}_t = \exp(\Delta_t a)$ 가 token마다 달라지므로, 모델은 token을 보고 “state를 유지할까, 밀어낼까”를 결정한다. Gu & Dao 2023은 특정 파라미터화 ($N = 1, A = -1, B = 1, s_\Delta = \text{Linear}, \tau_\Delta = \text{softplus}$)에서 selective SSM이 고전 RNN의 gate 식 $h_t = (1 - g_t)h_{t-1} + g_t x_t, g_t = \sigma(\text{Linear}(x_t))$ 로 환원됨을 정리로 보여 [Gu & Dao 2023 Theorem 1, § 3.5.1; 증명 App. C], 이것이 LSTM 이래의 gating과 같은 계보임을 명시한다.

계산 구조의 귀결이 독자에게 더 중요하다. 계수가 시변이 되는 순간 convolution kernel $\bar{K}$ 가 존재하지 않는다. **LTI를 깨면 FFT 모드가 소멸하고, recurrence(또는 scan)만 남는다.** Mamba가 “hardware-aware selective scan”이라는 커널 엔지니어링 — scan을 SRAM 안에서 수행하는 kernel fusion, backward를 위한 state recomputation — 에 논문 한 절을 쓰는 이유가 이것이다. 독자의 세계로 옮기면 Mamba의 커널은 FlashAttention과 같은 부류의 IO-aware 최적화다: 중간 텐서의 DRAM 왕복을 크게 줄인다. 그러나 fusion은 scan의 낮은 arithmetic intensity 자체를 없애지 못해 연산은 여전히 memory-bandwidth-bound이고 ([Mamba § 3.3.2]는 selective scan을 memory-bandwidth-bound로 규정한다), 게다가 **op mix 문제** (tensor core를 쓰지 못하는 elementwise 연산 위주)까지 겹친다. 두 제약이 공존한다는 것이 Mamba-2의 출발점이다.

recomputation 항목은 2장에서 배운 개념이 실물로 등장하는 첫 장면이므로 짚고 간다. 훈련의 backward pass는 forward의 중간값 — 여기서는 매 token의 state h_t — 을 다시 필요로 한다. 길이 L 의 sequence에서 $d \times N$ state를 전부 저장하면 activation memory가 L 에 비례해 커지므로, Mamba의 커널은 forward에서 state를 버리고 backward에서 입력으로부터 다시 계산한다. 2장의 “activation memory vs recomputation” trade-off가 커널 설계 결정으로 나타난 정확한 사례이며, FlashAttention이 attention 행렬을 저장하지 않고 backward에서 재계산하는 것과 같은 수다.

[해설] 이 라인의 어휘로 미리 번역해 두면: Mamba의 Δ_t 는 retention($\bar{a}_t$)과 write 강도($\bar{B}_t$ 의 스케일)를 **한 knob에 묶은 gate**다. 이후 논문들은 이 결합을 풀어낸다 — [Miras]는 유지 비율을 retention gate α_t 로 독립시키고 (→ 13장), [Titans]는 write 강도를 inner learning rate η_t 로 독립시킨다 (→ 12장). 반면 write rule 자체는 Mamba에서도 여전히 Hebbian 덧셈이다. 즉 Mamba의 혁신은 “무엇을 지울까”에 있지, “어떻게 쓸까”에는 없다. 후자를 바꾸는 것이 delta rule 계열 (→ 6장)과 TTT (→ 8장)다.

serving 관점의 접점 하나. Mamba layer의 decode state는 채널당 N 개 float, 모델 폭 d 전체로는 $d \times N$ 행렬이다 (Mamba-1의 기본 설정은 $N = 16$, Gu & Dao 2023). 문맥이 1M token

이어도 state 크기는 불변 — KV cache 처럼 자라지 않는다. 독자가 아는 “Mamba 는 decode 가 싸다” 의 실체가 이 고정 크기 RMW(read-modify-write) 다.

운영 측면 각주 하나를 더. Mamba block 의 decode 상태는 SSM state 만이 아니다. block 앞단의 짧은 causal depthwise convolution 이 최근 입력 몇 token 의 window 를 상태로 요구 하므로, 서빙 엔진이 실제로 관리하는 per-sequence 상태는 “SSM state + conv state” 의 묶음 이다. 그래도 총량이 문맥 길이와 무관한 상수라는 성질은 변하지 않는다 — paged KV cache 처럼 블록을 할당 · 해제 · 조각모음하는 관리 문제가 사라지는 대신, 고정 크기 상태를 요청마다 하나씩 들고 다니는 문제로 바뀐다. 이 상태 묶음이 곧 1 장의 Rosetta 사전이 “per-session weight state — 새로운 cache class” 라고 예고한 대상의 가장 온전한 형태다.

7.3 Mamba-2 와 SSD: 한 recurrence, 두 얼굴, 세 계산 경로

Mamba-2(Dao & Gu 2024, arXiv:2405.21060) 의 첫수는 **제약**이다: transition 을 per-head 스칼라 곱으로 줄인다 — $\bar{A}_t = \alpha_t I$, $\alpha_t \in (0, 1)$ 은 x_t 의 함수. 표현력을 일부 포기하는 대신, 이 제약이 두 가지를 산다. 첫째, head 의 state 들이 하나의 행렬로 묶인다. 둘째, 그 행렬 갱신이 6 장에서 이미 분석이 된다:

$$W_t = \alpha_t W_{t-1} + v_t k_t^\top, \quad y_t = \mathcal{M}(q_t; W_t) = W_t q_t \quad (7-2)$$

여기서 SSM 원 표기와 대응은 $B_t \leftrightarrow k_t$, $C_t \leftrightarrow q_t$, $x_t \leftrightarrow v_t$ 이며, $W_t \in \mathbb{R}^{d_v \times d_k}$ 다. 6 장 카탈로그의 GLA/Mamba-2 행 그대로다: α_t 가 상수면 RetNet, $\alpha_t \equiv 1$ 이면 vanilla linear attention, diagonal 이면 GLA, data-dependent 스칼라면 Mamba-2. Mamba-1 의 채널별 elementwise 감쇠도 [Miras Eq. 3] 의 general form $W_t = A_t * W_{t-1} + v_t k_t^\top$ (A_t : diagonal 또는 스칼라) 에 diagonal 사례로 포섭된다. [Miras Eq. 8] 의 논의는 이 세특수화 ($\alpha = 1$ / 학습되는 상수 / data-dependent 스칼라) 를 명시적으로 열거하며 Mamba-2 를 마지막 사례로 지목한다.

표 7-1 — SSM 원 표기 $\leftrightarrow$ 통일 표기 $\leftrightarrow$ inference 어휘 (장-국소 대응표)

SSM 원 표기 (Mamba/ Mamba-2)	통일 표기	inference 어휘
B_t (입력 투영)	k_t	write 주소 (key)
C_t (출력 투영)	q_t	read 주소 (query)
x_t (채널 값)	v_t	저장되는 값 (value)
$\bar{A}_t = \exp(\Delta_t A) = \alpha_t I$	α_t	남기는 비율 = 학습된 eviction
state h_t 의 head 묶음	$W_t \in \mathbb{R}^{d_v \times d_k}$	고정 크기 lossy 압축 KV cache
Δ_t	retention 과 write 강도의 결합 knob	cache 정책과 write 크기를 함 께 정하는 스위치

이제 이 장의 핵심 개념이다. **SSD(state-space duality)** 는 식 (7-2) 의 recurrence 가 계산하는 함수가 **masked linear attention 과 동일하다**는 동치 관계다 [Dao & Gu 2024]. 유도는 unroll 두 줄이다. $W_0 = 0$ 에서 식 (7-2) 를 풀면 $W_t = \sum_{j \leq t} (\prod_{s=j+1}^t \alpha_s) v_j k_j^\top$ 이고, 읽기를 대입하면

$$y_t = \sum_{j=1}^t \left(\prod_{s=j+1}^t \alpha_s \right) (q_t^\top k_j) v_j \iff Y = (\Gamma \odot (QK^\top))V, \quad \Gamma_{tj} = \prod_{s=j+1}^t \alpha_s \quad (t \geq j) \quad (7-3)$$

(decay mask 의 원문 기호는 L 이지만 sequence 길이 L 과 충돌하므로 이 장에서는 장-국소 기호 Γ 로 쓴다. 하삼각 바깥, 즉 $t < j$ 인 성분은 0 — causal mask 다.)

이 식이 말하는 것: **decay 누적곱을 mask 로 갖는 attention** 과, 고정 크기 state 의 recurrence 는 같은 함수의 두 표현이다. 왼쪽 (recurrent form) 은 token 당 $O(d_k d_v)$ 에 순차 계산하고, 오른

Structured State Space Model		Structured Masked Attention	
C	(contraction matrix)	Q	(queries)
B	(expansion matrix)	K	(keys)
X	(input sequence)	V	(values)
$A_{j,i}$	(state matrix)	L_{ji}	(mask)
N	(state expansion dim.)	N	(kernel feature dim.)
H = $L \cdot XB$ (hidden states (8b)) (linear mode)		SMA linear dual (15)	
SSM quadratic dual (16)		G (Gram matrix (13a)) = $Q \cdot K^T$ (quadratic mode)	

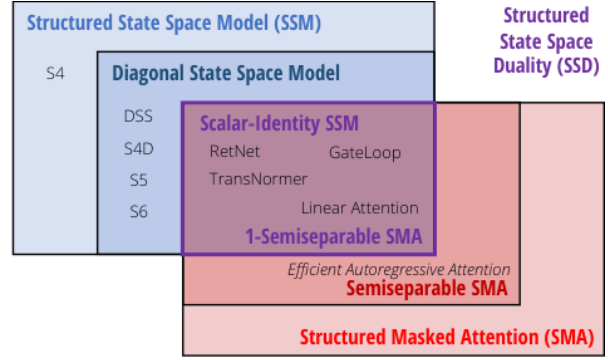


그림 7-2 — state-space duality 의 전체 지도. 왼쪽은 SSM 표기와 masked-attention 표기의 성
분별 대응 ($C \leftrightarrow Q$ read 주소, $B \leftrightarrow K$ write 주소, $X \leftrightarrow V$ 값, 그리고 state 행렬 $A \leftrightarrow$ decay
mask L) 과 각각의 linear form·quadratic form 을 나열한다. 오른쪽은 SSM 집합과 structured
masked attention 집합이 겹치는 영역이 곧 SSD 임을 보인다 — A 에 scalar-identity 구조를 준
SSM (= 이 장 식 (7-2) 의 $\bar{A}_t = \alpha_t I$) 이 1-semiseparable SMA 와 정확히 만나며, RetNet·linear
attention 이 그 교집합 안에 놓인다 (원문의 decay mask L 은 이 책 표기에서 Γ). 출처: Dao &
Gu, Mamba-2 (arXiv:2405.21060), 원문 Fig.4 — 원저자의 그림 (third-party), 본서의 결과가
아님.

쪽 (attention form) 은 $O(L^2)$ 에 완전 병렬 계산한다. 어느 쪽으로 계산할지는 **정확도가 아니라 하
드웨어 사정으로 고르는 알고리즘 선택**이다.

Dao & Gu 2024 는 이것을 행렬 구조론으로 일반화한다. sequence mixing 전체를 하삼각 행
렬 $\Gamma \odot (QK^T)$ 의 곱으로 보면 이 행렬은 **semiseparable** 구조 — 하삼각 영역에 완전히 포함된 부
분행렬 (특히대각선 아래 off-diagonal block) 이 낮은 rank 를 갖는 구조화 행렬; 대각을 가로지르
는 block 은 full rank 일 수 있다 — 를 가지며, 이를 dense 로 실체화해 곱하면 attention 모드, 인
수분해된 구조를 이용해 곱하면 recurrent 모드가 된다 [Dao & Gu 2024, Def. 3.1]. “duality”
라는 이름은 같은 구조화 행렬에 대한 이 두 곱셈 알고리즘의 쌍대성을 가리킨다. 이 대응과 두 집합
의 교집합을 한 장으로 요약한 것이 그림 7-2 다.

실전 답은 둘의 절충이다. sequence 를 크기 C 의 chunk 로 자르고 (§ 1.2 의 chunk 시작
offset $\xi(t, C) = C \lfloor (t-1)/C \rfloor$ 사용), chunk 경계에서만 state 를 전달하면:

$$y_t = \left(\prod_{s=\xi(t,C)+1}^t \alpha_s \right) W_{\xi(t,C)} q_t + \sum_{j=\xi(t,C)+1}^t \left(\prod_{s=j+1}^t \alpha_s \right) (q_t^T k_j) v_j \quad (7-4)$$

첫 항은 chunk 경계 state 의 기여 (cross-chunk: GEMV 를 chunk 단위로 모으면 $C \times d_k$ 대
 $d_k \times d_v$ GEMM), 둘째 항은 chunk 내부의 masked attention ($C \times C$ GEMM) 이다 (그림 7-3). 경
계 state 의 갱신역시 $d_v \times C$ 대 $C \times d_k$ GEMM 하나다. 결과: **모든무거운 연산이 GEMM 이 된다.**

이것이 Mamba-2 가 tensor core 를 되찾은 방법이고, [Dao & Gu 2024] 는 이 SSD 알고리즘
이 Mamba-1 의 fused selective scan 대비 $2-8 \times$ 빠르다고 보고한다. 스칼라 gate 덕에 커널이
단순해져 state 차원도 Mamba-1 보다 크게 키울 수 있게 되었다. 원문은 이를 Mamba-1 의 $8 \times$
이상이라고 표현하며 [Dao & Gu 2024 § 1], 실험은 과제별로 $N \in \{16, 64, 256\}$ 을 쓴다 — 단일
default N 은 논문에 명시가 없다 [Dao & Gu 2024 Fig. 8, Table 5].

한 가지를 지금 박아 두어야 한다. 식 (7-4) 의 chunk 분해는 **항등변형**이다. transition 이
linear 이기 때문에 결합법칙으로 항을 재배열했을뿐, 계산되는 함수는 C 와 무관하게 식 (7-3) 과
bit-exact (부동소수점재배열 오차 제외) 로 같다. FlashAttention tiling 과 정확히 같은 지위다.
9 장에서 만나는 chunkwise training 은 다르다 — state 갱신이 gradient (state 에 비선형) 가 되
는 순간 이 재배열이 불가능해지고, chunk 시작 상태에 gradient 를 고정하는 근사 (식 (M4) 의
stale-snapshot) 가 들어오며, 그때부터 C 는 함수 자체를 바꾸는 semantic hyperparameter ($\rightarrow$
9 장) 가 된다. “Mamba-2 의 chunk 는 tiling 이고, TTT 의 chunk 는 근사다” — 이 한 문장이 이
장과 9 장을 가르는 경계선이다.

$C_0^\top A_{0:0} B_0$ $C_1^\top A_{1:0} B_0 \quad C_1^\top A_{1:1} B_1$ $C_2^\top A_{2:0} B_0 \quad C_2^\top A_{2:1} B_1 \quad C_2^\top A_{2:2} B_2$		
$\begin{bmatrix} C_3^\top A_{3:2} \\ C_4^\top A_{4:2} \\ C_5^\top A_{5:2} \end{bmatrix} A_{2:2} \begin{bmatrix} B_0^\top A_{2:0} \\ B_1^\top A_{2:1} \\ B_2^\top A_{2:2} \end{bmatrix}^\top$	$C_3^\top A_{3:3} B_3 \quad C_4^\top A_{4:3} B_3 \quad C_4^\top A_{4:4} B_4$ $C_5^\top A_{5:3} B_3 \quad C_5^\top A_{5:4} B_4 \quad C_5^\top A_{5:5} B_5$	
$\begin{bmatrix} C_6^\top A_{6:5} \\ C_7^\top A_{7:5} \\ C_8^\top A_{8:5} \end{bmatrix} A_{5:2} \begin{bmatrix} B_0^\top A_{2:0} \\ B_1^\top A_{2:1} \\ B_2^\top A_{2:2} \end{bmatrix}^\top$	$\begin{bmatrix} C_6^\top A_{6:5} \\ C_7^\top A_{7:5} \\ C_8^\top A_{8:5} \end{bmatrix} A_{5:5} \begin{bmatrix} B_3^\top A_{5:3} \\ B_4^\top A_{5:4} \\ B_5^\top A_{5:5} \end{bmatrix}^\top$	$C_6^\top A_{6:6} B_6 \quad C_7^\top A_{7:6} B_6 \quad C_7^\top A_{7:7} B_7$ $C_8^\top A_{8:6} B_6 \quad C_8^\top A_{8:7} B_7 \quad C_8^\top A_{8:8} B_8$

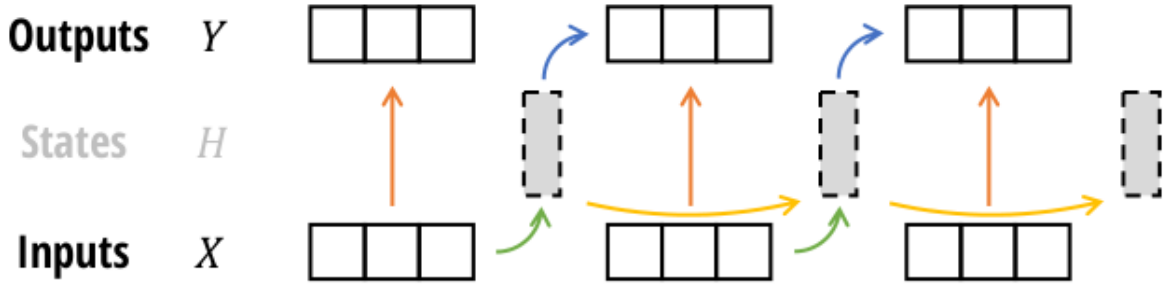


그림 7-3 — SSD의 chunk 분해 알고리즘. sequence mixing 행렬 (위)을 chunk 단위 block으로 나누면 대각 block(파랑)은 chunk 내부의 masked-attention 계산 — 식 (7-4)의 둘째 항, intra-chunk — 이 되고, off-diagonal block(주황)은 chunk 경계의 hidden state를 통해 전달되는 chunk 간 기여 — 첫 항, inter-chunk — 가 된다. 아래 그림은 같은 분해를 입력/state/출력의 chunk 흐름으로 다시 그린 것으로, chunk 안은 병렬 (수직 화살표)로, 경계 state만 순차 (수평 화살표)로 전달됨을 보인다. 이 분해가 무거운 연산을 전부 GEMM으로 바꿔 tensor core를 회복하는 것이 Mamba-2 속도 향상의 핵심이다. 출처: Dao & Gu, Mamba-2 (arXiv:2405.21060), 원문 Fig.5 — 원저자의 그림 (third-party), 본서의 결과가 아님.

decode 쪽 점점 하나. SSD 는 훈련 ·prefill 의 이야기고, decode 비용은 Mamba-1 이든 Mamba-2 든 per-token state RMW 로 같은 차수다 (상세는 § 7.6). “Mamba-2 가 빠르다” 는 주장을 서빙 엔지니어가 들을 때는 어느 **phase** 의 **throughput** 인지를 물어야 한다 — 이 질문 습관은 Part II 에서 각 논문의 효율 주장을 감사할 때 그대로 재사용된다.

7.4 계보 정리: 세 세대의 gate, 그리고 이 라인의 접속점

[Titans App. A.1] 은 linear recurrent model 의 역사를 gate 의 성격으로 3 세대로 나눈다. 이 구분은 Part II 전체의 지도이므로 표로 고정해 둔다.

표 7-2 — linear recurrent model 의 세대 구분 ([Titans App. A.1] 의 세대 구분 재구성; 모델 배치 일부는 6 장 카탈로그 기준)

세대	transition/gate	write rule	대표 모델	통일 표기 update
1 세대	data-independent decay	Hebbian 덧셈	S4, S5, RetNet, LRU, RWKV	$W_t = \alpha W_{t-1} + v_t k_t^\top$
2 세대	data-dependent gate	Hebbian 덧셈	Mamba, Mamba-2, Griffin, GLA, RWKV-6	$W_t = \alpha_t W_{t-1} + v_t k_t^\top$
3 세대	(gate 유무 다양)	delta rule / online learning	DeltaNet, Gated DeltaNet, Longhorn, TTT, RWKV-7	$W_t = W_{t-1}(I - \eta_t k_t k_t^\top) + \eta_t v_t k_t^\top$ 등
다음 세대	gate + momentum (token flow)	GD + momentum + weight decay	Titans-LMM	식 (M2)

이 표의 세로축이 곧 이 책의 서사다. 1 → 2 세대는 “무엇을 지울까” 를 학습가능하게 만들었고 (이 장), 2 → 3 세대는 “어떻게 쓸까” 를 Hebbian 덧셈에서 optimization step 으로 승격시켰으며 (6 장, 8 장), [Titans] 는 거기에 momentum — 논문의 표현으로는 token flow — 을 더해 자신을 다음 세대로 규정한다 [Titans App. A.1].

master update 와의 접속은 이렇게 읽으면 된다. 식 (M2) 는 $W_t = \alpha_t W_{t-1} + S_t$ 였다. Mamba-2 는 여기서 momentum 을 끄고 ($\beta_t = 0$) surprise 항을 Hebbian write $v_t k_t^\top$ 로바꾼 특수 사례다. 그리고 이 Hebbian write 조차 gradient 의 언어로 다시 읽힌다. [Miras] 는 dot-product attentional bias $\tilde{\ell}_t = -2\langle W k_t, v_t \rangle$ 에 GD 한 걸음을 적용하면 (step size 는 흡수) 정확히 $+v_t k_t^\top$ 의 write 가 나옴을 보이고, 이에 따라 Mamba-2 를 “dot-product bias + ℓ_2 retention + GD” 조합으로 분류한다 [Miras Eq. 8, Table 1]. 이 bias 는 W 에 대해 선형이라 gradient 가 현재 state 를 전혀 참조하지 않는다 — 옛 값을 읽고 고쳐 쓰는 correction 이 원리적으로 없고, 그래서 crosstalk(→ 5 장) 가 남는다. 이 자리 배치가 이 라인 전체의 설계 공간 (어떤 bias 를 최소화할까 × 무엇을 남길까 × 어떤 optimizer 로) 을 여는 첫 수가 된다.

gate 의 다음 진화도 여기서 예고된다. [Titans] 는 자신의 forgetting mechanism — update 앞에 곱해지는 data-dependent retention — 가 Mamba-2 류 gating mechanism 의 일반화라고 명시한다 [Titans § 1, § 3.1]. 무엇이 일반화인가? Mamba-2 의 gate 는 행렬 state 하나를 스칼라로 감쇠시키지만, Titans 의 retention 은 memory 가 “작은 모델의 weights” 로 승격된 뒤에도 같은 자리에서 작동하는 weight decay 다. 2 장에서 optimizer 객체의 성분으로 배운 weight decay — $(1 - \eta\lambda)w$ 의 그 감쇠 — 가, 여기서는 sequence 축위에서 token 마다 값이 달라지는 학

습된 eviction 으로 재해석된다. 같은수식, 다른 축이다. 이 재해석의 정식 전개는 12 장의 몫이고, 이 장은 그재료인 gate 계보를 공급했다.

의미론적 차이 하나는 기록해 둘 가치가 있다. [Miras footnote 2] 는 Mamba-2 류의 gating 과 [Titans] 의 retention 이 **완전 소거의의미**에서 다르다고 지적한다: Mamba-2 의 gate 가 0 이 되면 memory 전체가지워지고 다음 token 은 “처음 보는 데이터” 가 되는 반면, Titans 는 소거하기전의 상태 M_{t-1} 에서 surprise 를 측정하 뒤 그 update 를남긴다 (원문 표현 “cold start”). 즉 $\alpha_t \rightarrow 1$ 이어도 새 상태는 W_{init} 이 아니라 surprise 항 S_t 이며 ($M_t = (1 - \alpha_t)M_{t-1} + S_t$ 이고 S_t 의 gradient 는소거 전 M_{t-1} 에서 평가된다 [Titans Eq. 13 – 14]), 직전 memory 에서 얻은 정보가 살아남는다. (학습된 초기 상태 W_{init} 로 실제로 되돌리는 reset 은 Titans 가 아니라 [TNT] 의 local memory 에서 나타나며, 15 장에서 시스템 설계 축으로커진다.)

마지막으로 두 개의 예고. 첫째, NL 은 이 장의 전계보를 update frequency 의 스펙트럼 위에 재배열한다 — SSM state 는 token 마다 갱신되는 가장 빠른 level 일 뿐이다 ($\rightarrow$ 16 장). 둘째, 독자의 어휘로이 절 전체를 한 줄로 압축하면: **gate 는 학습된 cache eviction policy 이고, 이 장의 역사는 eviction policy 가 고정 상수에서 content-aware 함수로 진화해 온 역사다.** write policy 의 진화는 다음 장의 몫이다.

7.5 Worked micro-example: 한 recurrence 를 세 경로로계산하기

$d_k = d_v = 2$, $L = 3$ 으로 식 (7-2) – (7-4) 를 전부 손으로 확인한다. 입력은 다음과 같다 ($W_0 = 0$ 이므로 α_1 은 결과에 안 쓰인다):

t	$k_t^\top$	$v_t^\top$	α_t	$q_t^\top$
1	(1, 0)	(2, 0)	—	—
2	(0, 1)	(0, 4)	1/2	—
3	(1, 0)	(1, 1)	1/2	(1, 0)

경로 1 — recurrent form (식 7-2). decode 가 하는 계산이다.

$$W_1 = v_1 k_1^\top = \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix}, \quad W_2 = \frac{1}{2}W_1 + v_2 k_2^\top = \begin{pmatrix} 1 & 0 \\ 0 & 4 \end{pmatrix}, \quad W_3 = \frac{1}{2}W_2 + v_3 k_3^\top = \begin{pmatrix} 1.5 & 0 \\ 1 & 2 \end{pmatrix}$$

읽기: $y_3 = W_3 q_3 = (1.5, 1)^\top$.

경로 2 — attention form (식 7-3). mask 는 $\Gamma_{tj} = \prod_{s=j+1}^t \alpha_s$ 이므로 3 행은 $(\alpha_2 \alpha_3, \alpha_3, 1) = (\frac{1}{4}, \frac{1}{2}, 1)$ 이다. score 는 $q_3^\top k_j = (1, 0, 1)$. 곱하면 가중치 $(\frac{1}{4}, 0, 1)$:

$$y_3 = \frac{1}{4} v_1 + 0 \cdot v_2 + 1 \cdot v_3 = (0.5, 0)^\top + (1, 1)^\top = (1.5, 1)^\top$$

경로 3 — chunkwise form (식 7-4), $C = 2$. chunk 0 = token 1 – 2 를 처리해 경계 state W_2 를 만들고, token 3 은 chunk 1 에속한다 ($\xi(3, 2) = 2$). cross-chunk 항과 intra-chunk 항:

$$y_3 = \alpha_3 W_2 q_3 + (q_3^\top k_3) v_3 = \frac{1}{2}(1, 0)^\top + (1, 1)^\top = (1.5, 1)^\top$$

세 경로가 같은 값을 준다 — duality 가 근사가 아니라 항등임을 숫자로확인했다. 두 가지 관찰을 덧붙인다. 첫째, W_2 를 memory 로 읽어 보면 $k_1 = (1, 0) \rightarrow (1, 0)$ (원래 값 (2, 0) 이 gate 에 반감쇠됨), $k_2 = (0, 1) \rightarrow (0, 4)$ (방금 써서 온전함) 이 저장돼 있다. gate 는 eviction 의 연속 버전 — 지우는 대신 흐리게 만든다. 둘째, $k_3 = k_1$ 로 key 가 충돌하는데 write 는 Hebbian 덧셈이므로 y_3 에는 감쇠된 옛 값 $\frac{1}{4}v_1$ 이 새 값 v_3 에 **섞여** 나온다. 5 장의 crosstalk 가 gate 로 완화될 뿐 제거되지 않는 현상이다. 같은 key 를 덮어쓰고싶다면 write rule 자체를 바꿔야 한다 — 6 장의 delta rule 이 하는 일이고, 8 장의 TTT 가 일반화하는 일이다.

7.6 Systems bridge: scan 에서 GEMM 으로 — 9장리허설

이 장의 계산 이야기를 독자의 roofline 위에 정리한다.

decode. 세 모드 중 recurrent form 만 남는다. head 당 비용은 state $W \in \mathbb{R}^{d_v \times d_k}$ 의 read-modify-write: FLOPs 는 갱신 (outer product 누적) 과 읽기 (GEMV) 로 $O(d_k d_v)$, 메모리 트래픽도 state 왕복 $O(d_k d_v)$ bytes 다. FLOP/byte 비가 $O(1)$ 이므로 **decode 는 여전히 bandwidth-bound 다** — KV cache 스캔이 state RMW 로 바뀌었을 뿐, “decode 는 메모리가 지배한다” 는 독자의 세계관은 그대로 유효하다. 달라진 것은 그 트래픽이 문맥 길이와 무관하게 상수라는 점이다.

batching 과의 접점도 짚어 둔다. decode 에서 state 는 요청마다 다르지만 모양은 전부 같으므로 ($d_v \times d_k$ 행렬들), batch 전체의 읽기 · 갱신이 균일한 batched GEMV 와 rank-1 update 로 묶인다 — 길이가 제각각인 KV cache 스캔을 paged layout 으로 달래던 세계와 비교하면, 스케줄러가 다뤄야 할 자유도가 하나 줄어든 셈이다. 단, 여기서 batching 이 평화로운 이유는 memory 가 행렬 하나라서 per-request 연산이 batched GEMM 한 번으로 정규화되기 때문임을 기억해 두라. 8 장에서 memory 가 per-request 로 훈련되는 MLP 가 되는 순간, 같은 질문이 grouped GEMM 의 문제로 재등장한다 ($\rightarrow$ 10 장).

prefill/훈련 — Mamba-1 의 병목. selective scan 의 op mix 는 elementwise 곱-합이다. kernel fusion 으로 중간값의 DRAM 왕복을 크게 줄여도 scan 의 arithmetic intensity 는 낮아 연산이 여전히 memory-bandwidth-bound 이고 ([Mamba § 3.3.2]), 동시에 연산이 GEMM 이 아니므로 tensor core 가 높아 상한이 기기의 vector-ALU throughput — 최신 GPU 에서 tensor core FLOPS 의 작은 분수 — 에도 걸린다 ([Mamba-2 § 2.1]). 즉 Mamba-1 은 **bandwidth 제약** 과 **op mix 제약** 을 동시에 안는다 — fusion 은 전자를 완화할 뿐 제거하지 못하고 후자는 건드리지 못한다. roofline 그림으로 말하면: 지붕의 낮은 쪽 처마 밑에 앉아 있는 것이다.

prefill/훈련 — Mamba-2 의 답. 식 (7-4) 는 chunk 당 세 종류의 GEMM 을 만든다: intra-chunk score $QK^\top (C \times d_k \text{ 대 } d_k \times C)$, mask 적용 후 value 곱 ($C \times C \text{ 대 } C \times d_v$), 그리고 state 항 ($C \times d_k \text{ 대 } d_k \times d_v$ 와 경계 갱신). token 당 비용은 $O(Cd + d_k d_v)$, sequence 전체로는 $O(LCd + L d_k d_v) - C = 1$ 이면 순수 recurrence, $C = L$ 이면 순수 attention 으로 퇴화하는 보간식이다. C 를 키울수록 GEMM 이 두꺼워져 arithmetic intensity 가 올라가고 tensor core 활용이 회복된다. 이것이 “같은 수학을 GEMM 으로 다시 쓰기” 의 전부이며, FlashAttention 에서 tile 크기를 SRAM 에 맞추던 감각과 같은 종류의 튜닝이다.

표 7-3 — 식 (7-2)/(7-3)/(7-4) 의 세 가지 계산모드

모드	총 FLOPs 차수	주 연산 (하드 웨어 유닛)	병렬성	exact?	주 용도
recurrent ($C = 1$)	$O(L d_k d_v)$	GEMV + rank-1 RMW (vector ALU)	순차	exact	decode
quadratic ($C = L$)	$O(L^2 d)$	GEMM (tensor core)	완전 병렬	exact	짧은 prefill
chunkwise ($1 < C < L$)	$O(LCd + L d_k d_v)$	GEMM + 경 계 state 전달 (tensor core + scan)	chunk 내 병 렬, 경계 순차/ scan	exact	훈련, 긴 prefill

이 표에서 가장 중요한 열은 “exact?” 다. 이 장의 세 모드는 전부 exact — C 는 품질에 영향 없는 순수 성능 knob 이다. 9 장에서 write rule 이 gradient step 으로 바뀌면 같은 표의 chunkwise 행이 근사로 강등되고, C 가 품질 축으로 승격되며, “품질 최적 C (작음) vs MFU 최적 C (큼)” 의 긴장이 생긴다 — 그 긴장이 [TNT] 한 편의 주제다. 이 장은 그 대비의 기준선 (모든 것이 exact 였던 세계) 을 제공한다.

요약

- SSM 은 continuous 선형 시스템의 ZOH discretization 이고, step size Δ 는 유지 비율 ($\bar{a} = \exp(\Delta a)$) 과 write 강도를 한 knob 으로 묶은 gate 다 — 이 라인의 모든 gate 의 기원 이다.
- S4/HiPPO 는 LTI 라서 훈련 ·prefill 을 convolution 으로 병렬화하지만, 같은 이유로 content 기반 선택이 불가능하다.
- Mamba 의 selectivity 는 Δ_t, B_t, C_t 를 입력의 함수로 만들어 표현력을 얻는 대신 convolution 모드를 잃고, IO-aware scan kernel 에 의존한다 — bandwidth 문제는 fusion 으로 풀리지 만 op mix 문제는 남는다.
- Mamba-2 는 transition 을 per-head 스칼라 α_t 로 제약해 update 를 $W_t = \alpha_t W_{t-1} + v_t k_t^\top$ (식 7-2) 로 만들고, SSD 에 의해 이 recurrence 는 decay-mask attention $Y = (\Gamma \odot (QK^\top))V$ (식 7-3) 와 동일한 함수다.
- 같은 함수를 recurrent($C = 1$, decode) / quadratic($C = L$, 짧은 prefill) / chunkwise($1 < C < L$, 훈련) 세 모드로 계산할 수 있고, 셋 모두 exact 다 — 이 장의 chunk 는 tiling 이지 근사가 아니다.
- Mamba-2 의 chunkwise 재구성은 무거운 연산을 전부 GEMM 으로 바꿔 tensor core 를 되찾았고, [Dao & Gu 2024] 는 fused scan 대비 $2-8\times$ 의 속도향상을 보고한다. decode 비용은 두 세대가 같은 차수다.
- 제보는 gate 의 3 세대 (상수 decay $\rightarrow$ data-dependent gate $\rightarrow$ delta/online learning) 로 정리되며 [Titans App. A.1], Mamba-2 는 (M2) 에서 surprise 항을 Hebbian write 로 바꾼 특수 사례다 — write rule 의 승격이 다음 두 장의 주제다.

자가 점검체크리스트

- ZOH discretization 식 (7-1) 을 재현하고, $\Delta \rightarrow 0$ 과 Δ 가 큰 극한에서 $\bar{a}$ 와 $\bar{B}$ 가 각각 어떻게 되는지로 gate 동작을 설명할 수 있다.
- selectivity 가 왜 convolution 훈련 모드를 제거하고 scan 을 강제하는지, 그리고 kernel fusion 이 풀지 못하는 문제가 무엇인지 설명할 수 있다.
- § 7.5 의 예제를 recurrent/attention/chunkwise 세 경로로 손계산해 같은 y_3 를 얻을 수 있다.
- “Mamba-2 의 chunk 는 exact 한 tiling 이고 9 장의 chunk 는 semantic 한 근사다” 를 transition 의 선형성을 근거로 한 문장으로 정당화할 수 있다.
- Mamba-1 scan 과 Mamba-2 SSD 의 차이를 op mix 와 tensor core 활용의 언어로, decode 와 prefill 을 구분해서 설명할 수 있다.
- Mamba 의 state 와 gate 를 inference 어휘로 옮길 수 있다: state = 고정 크기 lossy 압축 KV cache, gate = 학습된 eviction policy, prefill = chunk 단위 병렬 write, decode = 상수 크기 state RMW.

다음 장으로

이 장의 끝에서 write rule 은 여전히 Hebbian 덧셈이었고, 그래서 key 충돌의 crosstalk 는 gate 로 흐려질 뿐 지워지지 않았다. 6 장의 delta rule 은 이를 “한 걸음의 gradient descent” 로 바꾸는 첫 승격이었다. 8 장은 이 승격을 끝까지 밀어붙인다: state 를 아예 작은 모델의 weights 로 선언하고, token 마다 그 모델을 self-supervised loss 로 한 걸음씩 훈련하는 TTT-Linear/TTT-MLP 를 다룬다. 그 순간 이 장의 duality 가 누리던 exactness 는 깨진다 — state 갱신이 비선형이 되므로 chunk 분해는 항등이 아니라 근사가 되고, 그 근사를 GEMM 으로 조직하는 dual form 이 8 장과 9 장의 중심 문제가 된다.

ch08. TTT lineage: test-time adaptation에서 TTT-Linear/TTT-MLP와 dual form까지

이 장의 목표 — 이 장을 마치면 다음을 할 수 있어야 한다.

1. **test-time training (TTT)** 을 “hidden state = 작은 model 의 weights, update rule = 그 model 의 learning step” 이라는 한 문장으로 정의하고, 이를 식 (M1) 의 원형으로 쓸 수 있다.
2. TTT-Linear 의 update 를 손으로 전개해 delta rule(→ 5 장)·DeltaNet(→ 6 장) 과의 동일성을 유도할 수 있다.
3. **dual form** 을 유도하고, chunk 크기 C 가 kernel tuning 파라미터가 아니라 계산되는 함수 자체를 바꾸는 근사임을 $d = 2$ 손계산으로 확인할 수 있다.
4. TTT layer 하나의 decode 비용을 GEMM/GEMV 단위로 세고, serving engine 에 무엇이 새로 필요한지 말할 수 있다.

왜 필요한가 — 6 편 전부가 이 장을 직접 전제한다. [Titans] (Titans: Learning to Memorize at Test Time, arXiv:2501.00663) 는 자신의 memory module 을 “generalized TTT layer” 로 규정하고 [Titans App. C], TTT 의 mini-batch tensorization 위에 자기 병렬화를 쌓는다 [Titans § 3.2]. [Miras] (It’s All Connected, arXiv:2504.13173; 이 책은 framework 이름 Miras 로 통칭) 는 TTT-Linear/TTT-MLP 를 taxonomy 의 기준 행으로 분류한다 [Miras Table 1]. [Atlas] (Atlas: Learning to Optimally Memorize the Context at Test Time, arXiv:2505.23735) 의 Omega rule(→ 14 장) 은 TTT 의 per-token inner objective 를 window 로 넓힌 것이고, [TNT] (TNT: Improving Chunkwise Training for Test-Time Memorization, arXiv:2511.07343) 는 자기 약어를 “TTT iNside TTT” 로도 읽을 수 있다고 밝힌다 [TNT footnote 1]. [NL] (Nested Learning, arXiv:2512.24695) 은 TTT 의 2-level 구조를 K-level 로, [Sleep] (Language Models Need Sleep, arXiv:2606.03979) 은 TTT 가 fast weights 에 쓴 것을 slow weights 로 옮기는 lifecycle 로 확장한다. 요컨대 Part II 의 여섯 장은 전부 “TTT 에서 무엇을 바꿨는가” 라는 질문으로 조직되며, 그 비교의 기준선이 이 장이다.

8.1 전사: inference 중의 weight update 는 이미 검증된 기법이었다

독자의 세계에서 가장 단단한 불변식부터 짚는다. serving engine 의 전제는 “checkpoint 를 로드한 뒤 weights 는 read-only” 다. KV cache 는 자라고 페이지가 스왑되지만, weight tensor 에 store 연산이 들어가는 일은 없다. 이라인의 논문들은 바로 그 불변식을 폐기하는데, 이 폐기는 2024 년의 발명이 아니라 vision 의 distribution-shift 문헌에서 이미 십수 년 가까이 다듬어진 기법의 연장이다. 이 절의 목적은 하나다: “production 에서 그런 걸한다고?” 라는 독자의 본능적 반응을, 선행 사례로 미리 무장해제하는 것.

test-time adaptation 은 deployment 중 만나는 입력에 맞춰 model 의 일부 parameter 를 그 자리에서 갱신하는 기법 family 를 가리킨다. 이름의 직계 조상은 Sun et al. 2020 (arXiv:1909.13231) 의 test-time training 이다. 구조는 Y자형이다: 공유 feature extractor 위에 (a) 본 task head 와 (b) self-supervised task head(입력 이미지의 회전 각도 예측) 를 함께 올려훈련한다. test 시점에는 label 이 없으므로 본 task loss 는 계산할 수 없지만, 회전 예측 loss 는 입력만으로 계산된다. 그래서 test 입력 하나가 들어올 때마다 그 loss 의 gradient 로 공유 extractor 를 몇 step 갱신한 뒤 예측한다. 핵심 통찰은 **label 없이도 학습 신호를 만들 수 있다**는 것이다 — self-supervision 이 test time 의 학습을 가능하게 하는 열쇠다.

Wang et al. 2021 (arXiv:2006.10726) 의 TENT 는 같은 아이디어의 더 가벼운 판본이다. 별도 auxiliary head 조차 없이, 예측분포의 entropy 를 loss 로 삼아 normalization layer 의 scale/shift parameter 만 갱신한다. 원 훈련 데이터에도, 훈련절차에도 접근하지 않는 “fully test-time” 설정이다. 두 기법을 한 줄로 요약하면 이렇다: test 입력 x 에 대해 어떤 보조 loss ℓ_{aux} 를 정의할 수 있는 한, $W \leftarrow W - \eta \nabla_W \ell_{\text{aux}}(W; x)$ 는 언제나 실행 가능한 연산이다. “training” 과 “inference” 의 구분은 시스템 설계의 관행이지 수학의 제약이 아니다.

systems 관점의 접점: 이 vision 기법들의 갱신 단위는 image 하나였다. 2024 년의 TTT layer 는 그 단위를 token 하나 — 독자의 어휘로는 decode step 하나 — 로 좁히고, 갱신 대상을 backbone 이 아니라 layer 안에 내장된 작은 memory network 로 한정된 것이다. 즉 이 라인은 “inference 중 weight update” 라는 이미 존재하던 기법을, sequence modeling 의 state update 자리에 이식했다.

8.2 TTT layer: hidden state 가 곧 model 이다

Sun et al. 2024 (arXiv:2407.04620, Learning to (Learn at Test Time): RNNs with Expressive Hidden States) 의 출발점은 RNN state 의 표현력 문제다. 7 장까지의 여정을 state 의 자료구조로 요약하면: Mamba 류의 vector state(d 차원), linear attention 류의 matrix state($d \times d$, 6 장) 가 있었다. 다음 단계는 무엇인가? Sun et al. 2024 의 답: **state 를 아예 작은 neural network 의 weights 전체로 승격시킨다**. 이 논문의 슬로건 — hidden state is a model, and the update rule is a step of self-supervised learning — 이 이 라인 전체의 설계 원리다. 이 책의 표기로, 그 작은 network 가 $\mathcal{M}(\cdot; W)$ 이고 그 weights W 가 fast weights($\rightarrow$ 6 장) 다.

TTT layer 는 세 요소로 정의된다. 첫째, **inner objective**. 원문은 이를 self-supervised reconstruction 으로 서술한다: token x_t 의 한 view(training view) $k_t = W_K x_t$ 를 입력으로 받아 다른 view(label view) $v_t = W_V x_t$ 를 복원하도록 memory 를 학습시킨다.

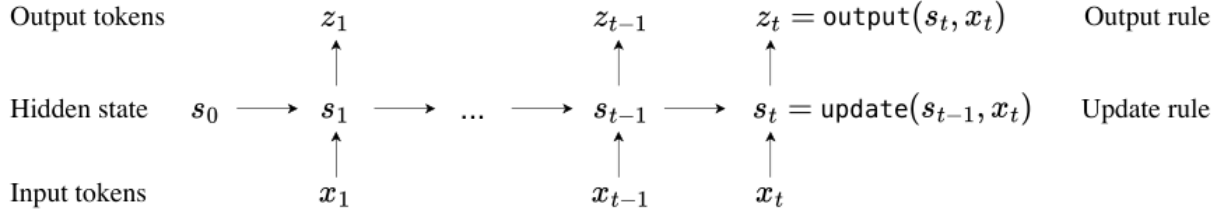
$$\ell(W; k_t, v_t) = \| \mathcal{M}(k_t; W) - v_t \|_2^2 \quad (8-1)$$

[해설] 식 (8-1) 은 5 장의 associative-memory regression 과 문자 그대로 같은 식이다. “self-supervised reconstruction” (TTT 의 언어) 과 “key $\rightarrow$ value 연상 저장” (associative memory 의 언어) 은 같은 objective 의 두 이름이며, 이 동일성이 Miras 가 여섯 모델 family 를 한 표에 넣을 수 있는 이유다 ($\rightarrow$ 13 장). 독자의 어휘로는: KV cache 가 무손실로 보관하던 (k, v) 쌍을, 여기서는 regression 으로 W 에 눌러 담는다.

둘째, **update rule**. 매 token 마다 (8-1) 에 대한 1-step gradient descent 를 실행한다. 이것이 표준형 (M1) 그대로다:

$$W_t = W_{t-1} - \eta_t \nabla_W \ell(W_{t-1}; k_t, v_t)$$

여기서 η_t 는 상수가 아니라 **학습된 data-dependent inner learning rate**, 즉 $\eta_t = \eta(x_t; \Theta)$ 형태로 slow weights 가 token 마다 산출하는 게이트다 (원문 구현은 base learning rate 에 sigmoid 를 곱한 형태이며, 원문 스스로 이를 “ $\nabla \ell$ 에 대한 gate” 로도 해석한다) [Sun et al. 2024 § 2.7]. 독자에게 이 게이트의 정확한 대응물은 Mamba 의 input-dependent gate($\rightarrow$ 7 장) 다 — 같은 역할 (“이 token 을 얼마나 강하게 state 에 반영할 것인가”) 을 optimizer 의 언어로 다시 말한 것뿐이다.



	Initial state	Update rule	Output rule	Cost
Naive RNN	$s_0 = \text{vector}()$	$s_t = \sigma(\theta_{ss}s_{t-1} + \theta_{sx}x_t)$	$z_t = \theta_{zs}s_t + \theta_{zx}x_t$	$O(1)$
Self-attention	$s_0 = \text{list}()$	$s_t = s_{t-1}.\text{append}(k_t, v_t)$	$z_t = V_t \text{softmax}(K_t^T q_t)$	$O(t)$
Naive TTT	$W_0 = f.\text{params}()$	$W_t = W_{t-1} - \eta \nabla \ell(W_{t-1}; x_t)$	$z_t = f(x_t; W_t)$	$O(1)$

그림 8-1 — 모든 sequence modeling layer 를 “hidden state + update rule” 의 한 틀로 보고, naive RNN·self-attention·naive TTT 를 세 요소 (initial state, update rule, output rule) 의 서로 다른 instantiation 으로 나란히세운 그림. TTT 의 update rule 은 self-supervised loss ℓ 에 대한 gradient step 이고, 상태가 고정 크기라 read cost 가 $O(1)$ 이다. 출처: Sun et al. 2024, Learning to (Learn at Test Time) (arXiv:2407.04620), 원문 Fig. 3 —원저자의 그림 (third-party), 본서의 결과가 아님.

셋째, **read**. 세 번째 view(test view) $q_t = W_Q x_t$ 로 갱신된 memory 를 읽는다: $y_t = \mathcal{M}(q_t; W_t)$. 원문 표기 $\theta_K, \theta_V, \theta_Q$ 는 이 책의 W_K, W_V, W_Q 에 대응한다.

이 세 요소 (그림 8-1 의 세 열 — initial state·update rule·output rule) 로 보면, 원문은 naive RNN·self-attention·TTT 를 같은 틀의 서로 다른 instantiation 으로 나란히 세운다. self-attention 은 상태가 (k, v) 쌍의 append-only list 라 read 가 $O(t)$ 인 반면, TTT 는 상태가 고정 크기 W_t 라 read 가 $O(1)$ 이라는 대비가 표의 Cost 열에 그대로 드러난다.

그림 8-1 — 모든 sequence modeling layer 를 “hidden state + update rule” 의한 틀로 보고 naive RNN·self-attention·naive TTT 를 세 요소의 instantiation 으로 정리한 그림 (Sun et al. 2024, arXiv:2407.04620, 원문 Fig. 3 의 third-party 재수록; 본서 결과 아님).

$\mathcal{M}$ 의 구조에 따라 두 instantiation 이 있다. **TTT-Linear** 는 $\mathcal{M}(k; W) = Wk$, 즉 state 가 linear attention 과 같은 $d_v \times d_k$ 행렬이다. **TTT-MLP** 는 $\mathcal{M}$ 이 2-layer MLP 다 — 이후 라인이 표준화하는 deep memory(→ 12 장) 의 원형이다. 6 장 카탈로그 (표 6-2) 의 TTT-Linear 행이 말하듯, 두 모델다 (M1) 이 전부다: momentum 도 retention gate 도 없다. 이 “없음” 이 Part II 를 여는 열쇠 구멍이다. 구현 세부 하나: 원문의 memory network 는 TTT 중의 안정성을 위해 항상 residual 과 LN 을 두른다 — 원문 표기로 $f(x) = x + \text{LN}(f_{\text{res}}(x))$ — 그리고 TTT-MLP 의 hidden 차원은 입력의 $4\times$, activation 은 GELU 다 [Sun et al. 2024 § 2.7]. 8.3 절의 유도는 이 겉옷을 벗긴 $\mathcal{M}(k; W) = Wk$ 에 대한 것이다.

Rosetta-Stone 으로 옮기면 TTT layer 의 fast weights 는 “**compression codec 이 달린 writable KV cache**” 다. KV cache 는 append-only 무손실 저장에 $O(L)$ lookup 이고, W_t 는 고정 크기 lossy 저장에 $O(1)$ lookup(GEMV 한 번) 이다. 결정적 차이는 write 경로다: cache append 는 memcpy 지만, TTT 의 write 는 read-modify-write 이며 그 “modify” 가 gradient 계산이다. 1 장에서 예고한 문장을 여기서 처음 실감하게 된다 —**backward pass 가 decode 안으로 들어온다**.

8.3 TTT-Linear 는 delta rule 이다

TTT-Linear 의 update 를 전개하면 이 layer 가 새 발명품이 아니라 5 장·6 장의 재발견임이 드러난다. $\mathcal{M}(k; W) = Wk$ 를 (8-1) 에 넣고 W 로 미분하면 $\nabla_W \ell = 2(Wk_t - v_t)k_t^\top$ — 예측 오차 벡터

와 key 의 outer product, 즉 rank-1 행렬이다. 이후 식에서는 상수 2 를 학습되는게이트 η_t 에 흡수한다 (게이트가 어차피 outer loop 가 정하는함수이므로 무해하다). 그러면 (M1) 은

$$W_t = W_{t-1}(I - \eta_t k_t k_t^\top) + \eta_t v_t k_t^\top \quad (8-2)$$

가 된다. 이 core update 는 6 장의 DeltaNet 행과 동일하다 (정확히는 $C = 1$ 이고 residual/LN 을 벗긴 core 에서의 동치다 [Sun et al. 2024 § 4.2]) — **TTT-Linear** 는 **별도 retention gate 가 없는 DeltaNet** 이고 (둘 다 학습된 write-strength gate η_t 는 갖는다), 둘 다 Widrow – Hoff delta rule(→ 5 장) 의 sequence-layer 판본이다. [Miras Table 1] 도 정확히 이렇게 분류한다: TTT-Linear = (matrix memory, ℓ_2 attentional bias, retention 없음, GD), TTT-MLP = (2-layer MLP, ℓ_2 , retention 없음, GD). 같은 표에서 DeltaNet 과의 차이는 memory 구조와 유래뿐이다.

의미론도 5 장 그대로다. Hebbian write($W += v_t k_t^\top$) 는값을 무조건 더해 crosstalk 를 쌓지만, delta write 는 **residual** $v_t - W_{t-1}k_t$ 만 쓴다. 이미 알고 있는 내용이면 gradient 가 작아거의 쓰지 않고, 어긋난 만큼만 고쳐 쓴다 — 같은 key 가 다시 오면 add 가아니라 overwrite 다. 이 “오차 기반 write” 가 Titans 가 surprise 라고 부르게될 것의 원형이다 (→ 12 장).

두 극한이 이 그림을 완성한다. 첫째, chunk 하나로 sequence 전체를 잡고모든 gradient 를 초기 상태 W_0 에서 평가하면 (다음 절의 언어로 $C = L$ 의 stale 극한), $W_0 = 0$ 일 때 TTT-Linear 는 vanilla linear attention 과 같은 함수가 된다 — [Sun et al. 2024 Theorem 1] 이 정리로제시하는 동치다 (전제: $f(x) = Wx - \text{LN/residual 제거}$ —, batch GD, 원문 표기로 $\eta = 1/2$ — 이 책은 상수 2 를 η 에 흡수 —, $W_0 = 0$; 대상은 분모 누적 없는 무정규화 linear attention). 둘째, parametric model 대신 non-parametric learner(kernel regression) 를 inner learner 로 넣으면 softmax attention 이 나온다 [Sun et al. 2024 Theorem 2 — Nadaraya – Watson estimator + exp kernel] — 5 장이 확립한 “softmax attention = ℓ_2 regression 의 non-parametric Nadaraya – Watson 해” 의 TTT 판본이다 (→ 5 장). 즉 attention 도 linear attention 도 TTT framework 의특수 사례이며, 원문의 프레임은 “새 대안” 이 아니라 “기존 layer 들을포함하는 일반화” 다.

systems 점점: state 의 shape 과 write 의 GEMM 구조가 DeltaNet 과 동일하므로, kernel 수준에서 TTT-Linear 는 새로운 비용을 만들지 않는다. 새로운 것은관점이다 — update 를 “explicit 한 학습 문제의 1 step” 으로 명명하는 순간, inner objective 를 갈아 끼우고 (→ Miras), objective 의 범위를넓히고 (→ Atlas), optimizer 를 갈아 끼우는 (→ Atlas 의 Muon) 설계 공간이 열린다.

8.4 Dual form: per-token GD 를 chunk 단위 GEMM 으로

(M1) 의 시스템적 결함은 순차성이다. W_t 는 W_{t-1} 을 필요로 하므로 token 방향으로 병렬화되지 않고, step 하나의 연산은 GEMV 한두 번과 rank-1 update — 독자의 언어로 arithmetic intensity 가 낮아 tensor core 가 노는 shape 이다. inference 의 decode 라면 이 순차성은 어차피 자명하지만, **training** 에서는 **치명적이다**: sequence 길이 L 전체를 이 속도로훑으면 wall-clock 이 무너진다. 여기서 Sun et al. 2024 의 두 번째 기여가나온다. 고정된 chunk 크기 C 의 mini-batch TTT update 를 계산하는방법이 둘이다: W_t 를 명시적으로 굴리는 **primal form** 과, 중간 W_t 를 물화하지 않고 chunk 단위 GEMM 으로 같은 출력을 얻는 **dual form** 이다 — 둘은 출력이 **정확히** 같다 (dual form 은근사가 아니다; [Sun et al. 2024 § 2.5] 는 “equivalent in output” 이라명시한다). 근사가 개입하는 지점은 primal/dual 의 선택이아니라 chunk 크기 C 자체다: $C > 1$ 이면 chunk 안의 gradient 를 chunk 시작 상태에서 평가하는 stale-snapshot 이 되어 (아래), 순수 online GD($C = 1$) 와는 다른 함수를 계산한다. Titans 는 이 재정식화를 자기병렬화의 토대로 명시한다 [Titans § 3.2].

아이디어는 mini-batch gradient descent 다. 2 장에서 굵게 강조했던 문장을소환한다: **이 라인에서 훈련의 batch 축 역할을 하는 것은 sequence 축이다**. 연속한 C 개 token 을 inner 문제의 mini-batch 로 묶고, chunk 안의 모든 gradient 를 chunk 시작 상태 $W_{\xi(t,C)}$ ($\xi(t,C) = C \lfloor (t-1)/C \rfloor$) 에서 평가한다 — 표준형 (M4) 그대로다. token τ 의 gradient 가 직전 token 들의 write 를 반영하지못하고 chunk 시작 시점의 낡은 snapshot 을 보므로, 9 장은 이를 stale-

snapshot 근사 (chunk-start anchor) 라고 부르고 일반형으로 다룬다. 이장에서는 TTT-Linear 인스턴스만 완결하자. (8-2) 에 anchor 를 적용하면 chunk 내부는 닫힌 형태가 된다:

$$W_t = W_\xi - \sum_{\tau=\xi+1}^t \eta_\tau (W_\xi k_\tau - v_\tau) k_\tau^\top, \quad \xi := \xi(t, C) \quad (8-3)$$

read $y_t = W_t q_t$ 에 (8-3) 을 대입하면 출력도 닫힌 형태다:

$$y_t = W_\xi q_t + \sum_{\tau=\xi+1}^t \eta_\tau (v_\tau - W_\xi k_\tau) (k_\tau^\top q_t) \quad (8-4)$$

식 (8-4) 를 소리 내어 읽으면 정체가 드러난다. 둘째 항은 **chunk 내부의 causal-masked attention** 이다: score 는 $k_\tau^\top q_t$, “value” 는 residual $v_\tau - W_\xi k_\tau$, mask 는 $\tau \leq t$. 행렬로 구현 하면 chunk 의 $K, V, Q \in \mathbb{R}^{C \times d}$ 를 쌓고 per-token step size 를 $D_\eta = \text{diag}(\eta_{\xi+1}, \dots, \eta_{\xi+C})$ 로 모은다: (i) $W_\xi [K^\top Q^\top]$ - residual 재로 $W_\xi K^\top$ 와 (8-4) 첫 항의 base read $W_\xi Q^\top$ 를 함께 내 는 결합 GEMM, (ii) score 행렬 $KQ^\top - (C \times d_k)(d_k \times C)$, (iii) residual $U := V^\top - W_\xi K^\top$ 에 $D_\eta \cdot \text{causal mask}$ 를 씌워 출력의 둘째 항을 만드는 곱 $U D_\eta \text{mask}(KQ^\top) - (d_v \times C)(C \times C)$, (iv) chunk 끝 상태 갱신 $W_{\text{end}} = W_\xi + U D_\eta K - (d_v \times C)(C \times d_k)$ 의 GEMM 네 개로 끝난다. 최종 출력은 $Y = W_\xi Q^\top + U D_\eta \text{mask}(KQ^\top)$ 이다 - (8-4) 의 per-token learning rate η_τ 가 D_η 로 들어가고 첫 항 $W_\xi q_t$ 가 base read 로 포함되는 것을 빠뜨리면 다른 함수가 된다. $C \times C$ score 행렬 은 독자가 FlashAttention 의 tile 내부에서 매일보는 $S = QK^\top$ 와 같은 shape 이다. dual form 의 실체는 한 줄로: **chunk 내부의 attention 처럼, chunk 경계는 RNN 처럼**.

그러나 FlashAttention 과의 유사성은 여기서 끝나고, 결정적 차이가 시작된다. FlashAttention 의 tiling 은 같은 수식의 bit-exact 한 재배열이라 tile 크기는 성능에만 영향을 준다. chunk 크기 C 는 (primal 이든 dual 이든) **계산되는 함수 자체를 바꾼다**: $C = 1$ 이면 순수 online GD, $C = L$ 이면 사실상 1-step batch GD(8.3 절의 linear-attention 극한), 그 사이의 모든 C 는 서로 다른 layer 다. 그래서 이 책은 C 를 **semantic hyperparameter** 라고 부른다 - 이 명제의 일반화와 명명은 9 장이 맡고, 이 staleness 가 train/serve 사이에서 일으키는 사고는 TNT 의 주제다 ($\rightarrow$ 15 장). 미리 정직하게 적어 두면, 이 stale 근사의 오차에 대한 형식적 bound 는 여섯 논문 어디에도 없다 ($\rightarrow$ 9 장, 15 장).

원문은 quality(작은 C 가 유리) 와 throughput(큰 C 가 유리) 사이의 실험적 절충으로 중간 크기의 chunk $C = 16$ (원문 표기 $b = 16$) 을 전 실험공통의 default 로 채택했다 [Sun et al. 2024 § 2.4, Fig. 7]. 이 절충은 그림 8-2 에서 눈으로 읽힌다: 왼쪽 panel 은 b 가 커질수록 perplexity 가 단조 상승함을 보이고 (작은 b 일수록 더 많은 GD step 을 밟아 quality 가 좋다 - 곧 앞서 말한 “ C 가 함수 자체를 바꾼다” 의 실험적 그림자다), 오른쪽 panel 은 dual form 의 forward 시간이 아주 작은 b (순차성 과다) 와 아주 큰 b (chunk 내부 attention 이 $O(C^2)$ 로 팽창) 양극단에서 모두 나빠져 중간 b 에서 최소가 됨을 보인다. 두 곡선이 만나는 절충점이 $b = 16$ 이다 - C 가 성능 knob(오른쪽) 이면서 동시에 함수 knob(왼쪽) 이라는 이 장의 주장이 한 그림에 겹쳐 있다.

그림 8-2 - chunk 크기 b 에 대한 quality(왼쪽: 작은 b 가 유리) vs throughput(오른쪽: 중간 b 가 최소 시간) 절충, $b=16$ 채택 (Sun et al. 2024, arXiv:2407.04620, 원문 Fig. 7 의 third-party 재수록; 본서 결과 아님).

8.5 Outer loop: 이 layer 자체는 누가훈련하는가

training 무경험 독자의 1 번 질문에 정면으로 답한다: “test time 에 학습한다는 이 layer 는, 그 자체로는 누가 학습시키나?” 답은 outer loop - 보통의 pretraining 이다. TTT layer 는 4 장의 bilevel 구조의 교과서적 인스턴스다. inner loop 는 sequence 하나 안에서 매 token 마다 W_t 를 움직이고, outer loop 는 corpus 전체에 대한 next-token prediction loss $\mathcal{L}$ 로 slow weights Θ 를 움직인다. 표 8-1 이 분업의 전모다.

표 8-1 - TTT layer 에서 inner 와 outer 의 분업

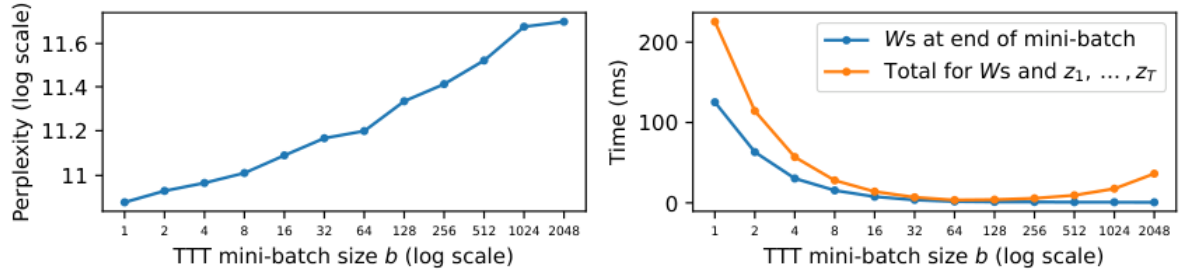


그림 8-2 — TTT mini-batch(=chunk) 크기 b 에 대한 ablation. 왼쪽: b 가 커질수록 perplexity가 상승 ($b=1$ 은 online GD, $b=T$ 는 batch GD) — 작은 b 일수록 GD step이 많아 quality가 좋다. 오른쪽: dual form의 forward 시간이 중간 b 에서 최소가 되어, quality와 throughput의 절충으로 $b=16$ 을 채택한다. 출처: Sun et al. 2024, Learning to (Learn at Test Time) (arXiv: 2407.04620), 원문 Fig. 7 —원저자의 그림 (third-party), 본서의 결과가 아님.

대상	소속	언제 움직이는가	serving 관점의 위치
W_t (memory 상태)	inner (fast weights)	매 token, 매 request	per-session state — KV cache가 있던 자리
W_K, W_V, W_Q	outer (Θ)	pretraining에서만	checkpoint, read-only
η -head ($\eta_t = \eta(x_t; \Theta)$ 의 산출기)	outer (Θ)	pretraining에서만 (출력값 η_t 는 매 token 계산)	checkpoint, read-only
W_{init} (초기 memory 상태)	outer (Θ)	pretraining에서만	checkpoint; sequence 시작 시 W_0 로 복사
backbone 나머지	outer (Θ)	pretraining에서만	checkpoint, read-only

outer 학습이 가능한 이유는 inner update 식 전체가 미분 가능한 연산의 합성이기 때문이다. W_t 는 Θ 의 함수다 — $k_\tau, v_\tau, \eta_\tau$ 가 전부 Θ 로부터 나오므로, $\mathcal{L}$ 의 gradient는 unrolled inner loop를 통과해 W_K, W_V, W_Q, η -head까지 흘러간다 (4장의 unrolling). 4장의 skeleton-key 문장이 여기서 실체를 얻는다: **inner optimizer의 hyperparameter들이, outer loop가 학습하는 data-dependent 함수가 된다.** learning rate의 token 별 multiplier는 outer gradient가 학습한다 — 단 base learning rate η_{base} (TTT-Linear는 1, TTT-MLP는 0.1로 후보군에서 사람이 선택)와 TTT-MLP의 warmup은 여전히 사람이 정한다 [Sun et al. 2024 § 2.7].

W_{init} 도 짚어 둘 가치가 있다. sequence가 시작될 때 memory는 0이 아니라 학습된 초기 상태 $W_0 = W_{\text{init}}$ 에서 출발하며, 이것 역시 Θ 의 일부다 — MAML의 “initialization이 meta-variable” (→ 4장)의 직계 대응이고, TNT에서는 local memory reset의 착지점으로 load-bearing 해진다 (→ 15장).

serving 관점에서 이 표는 안심 포인트이기도 하다. 폐기되는 불변식은 정확히 한 칸이다: W_t 만 움직이고, Θ 는 여전히 read-only checkpoint다. “weights가 움직인다”는 per-session fast-weight state의 이야기이지, 서빙 중 checkpoint가 변한다는 이야기가 아니다 — 그 경계까지 허무는 것은 [Sleep]에 가서다 (→ 17장). systems 점점 하나 더: unrolled inner loop는 정적 dataflow graph이므로 컴파일·fusion이 가능하고 (4장 systems bridge), dual form은 그 graph를 GEMM 위주로 재배열한 것에 지나지 않는다.

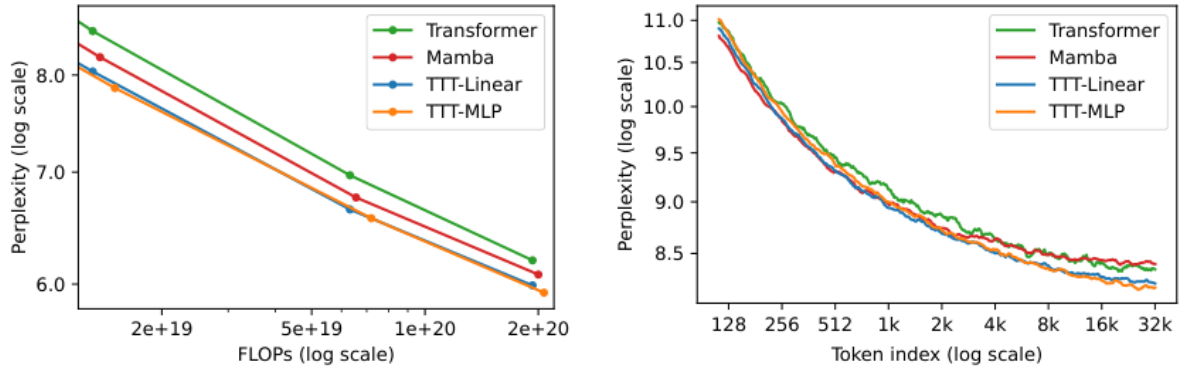


그림 8-3 — 왼쪽: Pile 8k context 에서 FLOPs 대비 perplexity scaling(350M – 1.3B 구간). 오른쪽: token index 를 x 축으로 한 perplexity — TTT-Linear·TTT-MLP·Transformer 는 문맥이 길어질수록 계속 내려가지만 Mamba 는 뒤쪽에서 정체한다 (고정 크기 state 의 capacity 한계). 출처: Sun et al. 2024, Learning to (Learn at Test Time) (arXiv:2407.04620), 원문 Fig. 2 — 원저자의 그림 (third-party), 본서의 결과가 아님.

8.6 스케일 증거와파생

[Sun et al. 2024] 는 125M – 1.3B 규모에서 TTT-Linear/TTT-MLP 를 같은 규모의 Transformer 및 Mamba 와 Pile·Books3 로 비교해, 문맥이 길어질수록 뒤쪽 token 의 perplexity 가 계속 내려가는 반면 Mamba 는 16K context 이후 개선이 정체한다고 보고한다 [Sun et al. 2024 Fig. 2] — 고정 크기 vector/matrix state 의 capacity 한계 ($\rightarrow$ 5 장) 와 정합적인 결과다. 그림 8-3 의 오른쪽 panel 이 그 정체를 그대로 보여 준다: token index 를 x 축으로 두면 TTT-Linear·TTT-MLP 는 Transformer 처럼 perplexity 곡선이 끝까지 내려가지만, Mamba 의 곡선은 오른쪽 끝에서 눕는다. 왼쪽 panel(FLOPs 대비 perplexity) 은 이 이득이 compute-매칭비교에서도 유지됨을 보인다.

그림 8-3 — scaling(왼쪽) 과 long-context 이득(오른쪽): TTT 계열은 긴문맥에서 perplexity 가 계속 내려가는 반면 Mamba 는 정체 (Sun et al. 2024, arXiv:2407.04620, 원문 Fig. 2 의 third-party 재수록; 본서 결과 아님).

다만 이 결과의 규모 감각은 정직하게 유지해야 한다. 이 라인 전체 (TTT 부터 Sleep 까지) 의 실증 상한은 1.3B parameters / 100B tokens 수준이며, 독자가 운영하는 frontier-scale serving 의 증거는 아직 없다. 이장이 확립하는 것은 “동작한다” 이지 “그 규모에서 이긴다” 가 아니다.

파생 하나가 이 라인 바깥에서 TTT layer 의 실용성을 보였다. Dalal et al. 2025 (arXiv:2504.05298) 는 3 초 clip 까지만 생성하던 pre-trained Diffusion Transformer(CogVideo-X 5B) 에 TTT-MLP layer 를 삽입·finetune 해, text storyboard 를 조건으로 1 분길이의 Tom and Jerry video 생성을 시연했다 [Dalal et al. 2025 § 1] — TTT layer 가 처음부터 함께 pretraining 되지 않아도 기존 backbone 에 graft 될수 있음을 보인 사례로, [Sleep] 의 graft 전략 ($\rightarrow$ 17 장) 을 예고한다.

systems 접점: “pre-trained backbone 에 나중에 끼워 넣을 수 있는가” 는 독자에게 배포 경로의 문제다. video 결과는 TTT layer 가 아키텍처 전면재훈련 없이 retrofit 가능한 부품임을 시사한다.

8.7 여섯 논문이 TTT에서 바꾸는 것 — Part II 의지도

Titans 가 스스로 정리한 TTT 와의 차이가 좋은 출발점이다. [Titans App. C] 는 세 가지를 꼽는다: (1) TTT layer 에는 forgetting mechanism 이 없어진 sequence 에서 고정 크기 memory 가 넘친다, (2) update 가 momentary surprise(현재 token 의 gradient) 에만 의존하고 token

흐름을 반영하는 momentum 이 없다, (3) deep memory 를 허용은 하되 그 득실을 실험적으로 검증하지 않았다. 이 세 결핍을 (M2) 의 α_t, S_t 와 deep-memory 실증으로 메운 것이 Titans 다. 같은 방식으로 여섯 편 전부를 “(M1) 의 어느성분을 바꿨는가” 로 요약할 수 있다 — 표 8-2 가 그 지도이고, 이 표가 사실상 Part II 의 목차다.

표 8-2 — 여섯 논문이 TTT((M1)) 에서 바꾸는 성분

논문 (장)	바꾸는 성분	무엇으로
[Titans] (12 장)	inner update rule	momentum S_t + retention gate α_t : (M1) $\rightarrow$ (M2); deep memory 의 실증
[Miras] (13 장)	inner objective 와 retention	ℓ 을 attentional bias family(ℓ_p , Huber, ...) 로, retention 을 정규화항으로 일반화
[Atlas] (14 장)	objective 의 범위와 inner optimizer	per-token $\rightarrow$ window 길이 c 의 Omega rule((M3)); GD $\rightarrow$ Muon(NS $_{\kappa}$); feature map ϕ_p, ϕ^*
[TNT] (15 장)	훈련 경제학	hierarchical global/local memory, periodic state reset $\rightarrow$ context parallelism, train/serve 의 C 분리
[NL] (16 장)	loop 의 층위 수	2-level $\rightarrow$ K-level(update frequency spectrum, CMS); optimizer 자체를 associative memory 로 재해석
[Sleep] (17 장)	lifecycle	wake 에서 fast weights 에 쌓인 것을 sleep phase 에 slow weights 로 consolidation

systems 독자를 위한 열쇠: 각 행은 비용 구조의 서로 다른 축을 건드린다. Titans/Miras/Atlas 는 decode step 당 FLOPs 와 state 크기를 (inner 연산이 무거워진다), TNT 는 training MFU 와 병렬화토폴로지를, NL 은 update 가 일어나는 빈도의 계층 — 곧 memory 계층 배치를, Sleep 은 serving fleet 의 background job 을 바꾼다. Part II 의 각 장 § “systems 함의” 가 이 축을 따라간다.

8.8 Worked micro-example: $d = 2$ 손계산 — chunk 가 함수를바꾼다

TTT-Linear 하나를 $d_k = d_v = 2$ 로 놓고, 같은 두 token 을 $C = 1$ (primal, 순수 online) 과 $C = 2$ (dual form, chunk-start anchor) 로 처리해 출력이달라짐을 확인하자. update 는 식 (8-2), 게이트는 상수 $\eta_t = 1$ 로 고정한다 (상수 2 는 이미 흡수됨). key 의 norm 이 1 이므로 $\eta_t = 1$ 은 “한 step 에 완전 overwrite” 를 뜻한다.

표 8-3 — 예제 입력

t	k_t	v_t	비고
1	$(1, 0)^\top$	$(2, 0)^\top$	

t	k_t	v_t	비고
2	$(1, 0)^\top$	$(4, 0)^\top$	같은 key, 다른 value —overwrite 시험

초기 상태는 $W_0 = 0$ (2×2), 질의는 $q_2 = (1, 0)^\top$ 이다.

Case A — $C = 1$ (per-token online GD). token 1: residual 은 $W_0 k_1 - v_1 = -v_1$ 이므로 $W_1 = v_1 k_1^\top = \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix}$. token 2: 이번에는 memory 가 이미 안다 — $W_1 k_2 = (2, 0)^\top$, residual 은 $(2, 0)^\top - (4, 0)^\top = (-2, 0)^\top$. update 는 그 오차만큼만: $W_2 = W_1 + (2, 0)^\top (1, 0) = \begin{pmatrix} 4 & 0 \\ 0 & 0 \end{pmatrix}$. 읽으면 $y_2 = W_2 q_2 = (4, 0)^\top = v_2$. delta rule 이 옛 값 2 를 새 값 4 로 정확히 **overwrite** 했다.

Case B — $C = 2$ (dual form, anchor W_0). 두 token 의 gradient 가 모두 $W_0 = 0$ 에서 평가된다. token 1 의 기여는 $v_1 k_1^\top$, token 2 의 기여는 residual $W_0 k_2 - v_2 = -v_2$ 에서 나온 $v_2 k_2^\top$ — token 1 이 이미 key $(1, 0)$ 에 2 를 써 두었다는 사실을 **모른 채** value 전체를 쓴다. 따라서 $W_2 = v_1 k_1^\top + v_2 k_2^\top = \begin{pmatrix} 6 & 0 \\ 0 & 0 \end{pmatrix}$ 이고, $y_2 = (6, 0)^\top$. 식 (8-4) 로 직접 계산해도 같다: $y_2 = W_0 q_2 + (v_1)(k_1^\top q_2) + (v_2)(k_2^\top q_2) = 0 + (2, 0)^\top \cdot 1 + (4, 0)^\top \cdot 1 = (6, 0)^\top$.

결과 정리: 같은 layer, 같은 입력, 같은 산술인데 $C = 1$ 은 4 를, $C = 2$ 는 6 을 돌려준다. 읽는 법이 세 겹이다. 첫째, staleness 의 정제 — delta rule 의 미덕은 residual 을 쓰는 것인데, anchor 가 낡으면 residual 계산이 틀려서 value 를 통째로 쓴다. 둘째, $6 = 2 + 4$ 는 Hebbian superposition, 곧 5 장의 crosstalk 다: anchor 가 0 인 chunk 안에서 delta rule 은 linear attention 의 additive write 로 퇴화한다 — 8.3 절의 “batch GD 극한 = linear attention” 정리를 2×2 에서 재현한 셈이다. 셋째, 이것이 C 가 **semantic hyperparameter** 라는 명제의 **최소증명**이다: FlashAttention 의 tile 크기를 바꿔서 출력이 4 에서 6 으로 변하는일은 절대 없다. GEMM shape 도 확인해 두자 — Case B 의 연산은 $U = V^\top - W_0 K^\top$ (2×2), score $KQ^\top$ (2×2 , causal mask), correction $(2 \times 2)(2 \times 2)$ 로, 전부 (아주 작은) GEMM 이다. C 를 키우면 이 행렬들이 커지며 tensor core 가 차오른다. 품질은? 방금계산했듯, 그 대가로 지불한다.

8.9 Systems bridge: decode 안의 backward pass

이 장의 새 개념을 독자의 cost model 위에 정착시키며 마친다. TTT layer 의 decode step 하나는 세 phase 이고 순서가 중요하다: (1) write 의 loss/gradient 계산 (forward $\mathcal{M}(k_t; W_{t-1})$ + backward), (2) write 적용 (update $\rightarrow W_t$), (3) 갱신된 W_t 로의 read($y_t = \mathcal{M}(q_t; W_t)$). TTT 의 causal 규칙은 현재 token 으로 memory 를 먼저 갱신한 뒤 읽는 **update-then-read** 이므로 (§ 8.2 의 $y_t = \mathcal{M}(q_t; W_t)$), W_{t-1} 을 먼저 읽으면 현재 token 의 write 가 y_t 에 반영되지 않는 다른 layer 가 된다 [Sun et al. 2024 Fig. 5, Eq. 5]. TTT-Linear($d_k = d_v = d$) 부터 세면 — read $y = Wq$ 는 d^2 MAC; write 는 Wk (d^2), residual (d), rank-1 outer-product 적용 (d^2) 으로 포함 약 $2d^2$ MAC. 즉 **write 가 read 와 같은 오더의 FLOPs** 다. KV cache append(memcpy, FLOPs 0) 에 익숙한 독자에게 이것이 첫 번째 구조 변화다. 두 번째는 traffic 이다: 매 token 마다 W 전체 (d^2 개 원소) 를 read-modify-write 하므로 per-token state traffic 이 $2d^2$ floats — arithmetic intensity 는 여전히 $O(1)$ MAC/byte 수준이고, decode 는 bandwidth-bound 로 남되 그 대상이 KV cache 에서 fast-weight state 로 바뀐다.

TTT-MLP(2-layer, hidden $4d$ 기준) 는 backward 가 명시적으로 등장한다. forward(read 또는 $\mathcal{M}(k_t)$) $\approx 8d^2$ MAC. write 는 forward(k_t) $8d^2$ + backward(VJP 연쇄, 2 장의 rule of thumb 대로 forward 의 약 2 배) $\approx 16d^2$ + parameter update(원소별 AXPY) $\approx 8d^2$ — 합계 write $\approx$ forward 의 4 배. 2 장에서 훈련 비용산식으로 배운 “backward $\approx 2 \times$ forward” 가 이제 **decode latency 산식에 직접 들어온다**. 이것이 1 장 Rosetta 의 “폐기되는 불변식” 행의정량적 의미다.

표 8-4 — per-token decode 비용의 자릿수 비교 (MAC 단위, 상수 · norm 류 생략; 정식 cheat sheet 는 $\rightarrow$ 10 장)

layer	resident state	read FLOPs	write FLOPs	state traffic/ token
softmax attention	$O(Ld)$ (KV cache)	$O(Ld)$	≈ 0 (append)	$O(Ld)$ read
TTT-Linear	d^2	d^2	$\approx 2d^2$	$2d^2$ RMW
TTT-MLP	$\approx 8d^2$	$\approx 8d^2$	$\approx 32d^2$	$\approx 16d^2$ RMW

serving engine 에의 함의를 세 가지로 못 박는다. 첫째, **backward 가 decode 에 들어온다**. 원문의 기준 구현은 범용 autograd 로 inner gradient 를 계산하고 (임의 비선형 · normalization 의 VJP 도 JAX/PyTorch autodiff 로 처리 가능하다 [Sun et al. 2024 App. A.4]), 따라서 hand-derived kernel 은 필수가 아니라 latency 최적화 선택이다 — 다만 production 서빙 stack 에는 보통 backward graph 가 없고 latency 가 중요하므로 gradient 를 손으로 유도해 하나의 fused kernel 로 하드코딩한다. TTT-Linear 라면 식 (8-2) 자체가 kernel 이고, TTT-MLP 라면 2-layer VJP 연쇄를 read·update 와 함께 fuse 한다 (이 계열의 공개 구현들이 취하는 형태다). 둘째, **shared-weight batching 이 깨진다**. W_t 가 request 마다 다르므로 batch 전체가 하나의 weight 로 GEMM 을 치는 전제가 무너지고, decode 는 grouped-GEMM (request 별 작은 GEMM 묶음) 형태가 된다 — 1 장 Rosetta 의 경고 행 그대로다. 셋째, **새로운 session-cache class 가 생긴다**. per-session W_t 는 sizing·checkpoint/restore·eviction 을 요구하는 상태다. 완성된 prefix-state snapshot 자체는 공유 가능하다 — deterministic recurrence 이므로 같은 W_{init} 같은 prefix 면 같은 W_{prefix} 가 되어 그 snapshot 을 캐시하거나 read-only 로 공유할 수 있다. 다만 continuation 이 W 전체를 수정하므로 fork 마다 copy-on-write 가 필요하고, KV cache 의 page 단위 append 공유보다 비싸다. 이 관리 문제는 Part III 의 주제다. 마지막으로 prefill/decode 비대칭은 그대로 이식된다: prefill 은 큰 C 의 dual form (GEMM 잔치), decode 는 $C = 1$ 의 primal (GEMV + rank-1) — 단, 8.8 절이 보였듯 이 둘은 **같은 함수가 아니다**. train 과 serve 의 C 가 다를 때 무슨 일이 나는가 [TNT] 의 출발점이다 (→ 15 장).

요약

- test-time adaptation (Sun et al. 2020 의 TTT, TENT) 은 “inference 중 weight update” 가 이 라인 이전부터 존재하던 검증된 기법임을 보여 준다; 2024 년의 TTT layer 는 그 갱신 단위를 image 에서 token 으로, 갱신 대상을 backbone 에서 내장 memory 로 좁힌 것이다.
- TTT layer 의 정의는 식 (M1) 그대로다: state = 작은 model 의 weights W_t , update = $\ell(W; k_t, v_t) = \|\mathcal{M}(k_t; W) - v_t\|_2^2$ 에 대한 1-step GD, read = $y_t = \mathcal{M}(q_t; W_t)$. momentum 도 retention 도 없다 — 이 결핍이 Part II 전체의 출발점이다.
- inner 문제의 모든 hyperparameter (W_K, W_V, W_Q, η_t 의 산출기, W_{init}) 는 outer loop 가 next-token prediction 으로 학습하는 slow weights Θ 다; serve 시점에 Θ 는 여전히 read-only 이고 움직이는 것은 W_t 뿐이다.
- TTT-Linear 는 gate 없는 DeltaNet, 곧 delta rule 의 sequence-layer 판본이다 [Miras Table 1]; batch-GD 극한에서 linear attention, non-parametric 극한에서 softmax attention 을 특수 사례로 포함한다.
- dual form 은 chunk-start anchor 의 stale gradient 근사로 chunk 내부 계산을 GEMM 네 개로 재배열하며, 그 내부 구조는 causal-masked attention 과 동형이다 (“chunk 내부 는 attention 처럼, chunk 경계는 RNN 처럼”).
- chunk 크기 C 는 semantic hyperparameter 다: FlashAttention tiling 과 달리 C 가 바뀌면 출력이 바뀐다 — $d = 2$ 손계산에서 $C=1$ 은 4, $C=2$ 는 6 을 반환했고, staleness 의 오차 bound 는 여섯 논문 어디에도 없다.
- decode 에 backward pass 가 들어온다: write 비용은 read 의 2–4 배 FLOPs 이고 state 전체의 RMW traffic 을 동반하며, serving engine 에는 hand-derived gradient kernel, grouped-GEMM decode, per-session weight-state 관리가 새로 필요하다.

자가 점검체크리스트

- TTT layer 의 세 요소 (state, update, read) 를 통일 표기의 (M1) 형태로 적을 수 있다.
- 식 (8-2) 를 (8-1) 에서 직접 유도하고, DeltaNet-delta rule 과의 동일성 및 Hebbian write 와의 차이 (residual write) 를 설명할 수 있다.
- “이 layer 의 learning rate 는 누가 학습하는가” 라는 질문에 inner/outer 분업표 (표 8-1) 로 답할 수 있다.
- $d = 2$ 예제를 재현해 $C = 1$ 과 $C = 2$ 의 출력이 왜 다른지 (stale residual $\rightarrow$ value write $\rightarrow$ crosstalk) 계산으로 보일 수 있다.
- dual form 의 chunk 내부 항이 causal-masked attention 과 같은 GEMM shape 임을 (8-4) 에서 읽어낼 수 있다.
- TTT layer 의 decode 를 inference 어휘로 옮길 수 있다: fast weights = codec 달린 writable KV cache, write = state 전체의 RMW + rank-1/rank- C GEMM, dual form 의 chunk = bit-exact tile 이 아닌 semantic tile.

다음 장으로

이 장은 dual form 을 TTT-Linear 한 경우에 대해서만 완결했다. 그러나 8.4 절의 요령 — chunk 경계에서 무언가를 얼려 chunk 내부를 GEMM 으로 만든다 — 은 TTT 만의 것이 아니다. GLA 는 decay 를 접어 넣고, DeltaNet 은 WY representation 으로 rank-1 곱들을 두 개의 GEMM 으로 바꾸며, Titans 는 momentum 을 associative scan 으로 풀어낸다. 9 장은 이 네 가지를 하나의 일반 scheme 의 instantiation 으로 통일하고, 이 장이 손계산으로만 보인 명제 — chunk 크기는 semantic hyperparameter 다 — 를 정식으로 세운다. 그 명제가 하드웨어 경제학과 충돌하는 지점 (품질 최적의 작은 C vs MFU 최적의 큰 C) 이 곧 TNT 의 존재 이유다.

ch09. Chunkwise-parallel training: 하나의 일반 scheme, 네 개의 인스턴스

이 장의 목표 — 독자가 이 장을 마치면 다음을 할 수 있어야 한다. (1) 임의의 fast-weight recurrence 를 받아 “이것은 exact 하게 chunk 병렬화되는가, anchored 근사가 필요한가” 를 transition 의구조 (스칼라/대각 linear, 비대각 linear, nonlinear) 만 보고 판정할 수 있다. (2) GLA·DeltaNet·TTT dual form·Titans 의 chunkwise 알고리즘을 하나의 일반 scheme(“경계에서 얼리고, 내부는 GEMM, 경계 사이는 handoff 또는 scan”) 의 네 인스턴스로 유도할 수 있다. (3) chunk 크기 C 가 왜 어떤 family 에서는 순수 성능 knob 이고 어떤 family 에서는 **계산되는 함수자체를 바꾸는 semantic hyperparameter** 인지 손계산으로 보일 수 있다. (4) arithmetic intensity 를 C 의 함수로 유도하고, 품질 최적 C 와 MFU 최적 C 가 왜 갈라지는지 roofline 위에서 설명할 수 있다.

왜 필요한가 — 이 장이 없으면 이 라인의 어떤 것도 실물이 되지 않는다. [Titans] (Titans: Learning to Memorize at Test Time, arXiv:2501.00663) 의 § 3.2 “How to Parallelize the Long-term Memory Training” 은 그 논문의 실용성 주장 전체를 담당하고, [Atlas] (arXiv:2505.23735) 는 § 3.4 에서 Omega rule 을, 부록 (Eq. 36 – 41) 에서 Muon 내장 momentum 을 같은 기법으로 병렬화한다. [Miras] (arXiv:2504.13173) 의 세 신모델 (Moneta/Yaad/Memora) 도 같은 chunkwise 기법으로 훈련된다 [Miras § 5.3]. 그리고 [TNT] (arXiv:2511.07343) 는 **이 장이 다루는 기법의 한계 자체가 논문 한 편의 주제다** — chunk 크기의 품질 $\leftrightarrow$ throughput 긴장, 비선형 inter-chunk recurrence 라는 병렬화 blocker, reset 을 통한 context parallelism. [NL] (arXiv:2512.24695) 의 CMS 는 이 장의 scheme 을 level 마다 반복 적용한 것이며 (식 (M5), $\rightarrow$ 16 장), [Sleep] (arXiv:2606.03979) 은 NL 을 경유해 간접적으로 이장에 의존한다. 독자의 FlashAttention/tiling 감각이 가장 큰 지렛대가 되는 장이자, 그 감각이 가장 위험한 오도가 되는 장이다.

9.1 문제 설정: training 안에 들어온 decode loop

8 장이 남긴 계산 문제를 정면으로 다시 세운다. 이 라인의 layer 는 정의상 per-token recurrence 다: token t 의 상태 W_t 는 W_{t-1} 없이는 계산될 수 없다. inference 의 decode 라면 이 순차성은 원래 세계의 조건이므로 놀랍지 않다. 문제는 **training** 이다. outer loop ($\rightarrow$ 2 장, 4 장) 는 sequence 길이 L 전체에 대한 forward 를 요구하는데, 이것을 per-token 으로 순차 실행하면 training 이 통째로 decode loop 가 된다 — step 당 연산은 GEMV 한두 번과 rank-1 update, 즉 arithmetic intensity 가 $O(1)$ 수준인 shape 이 L 번 직렬로 이어진다. 독자의 어휘로 정확히 옮기면: **batch 1 짜리 decode 를 32K 번 돌려서 한 개의 training step 을 만드는 것과 같다.** tensor core 는 놓고, wall-clock 은 무너진다. [TNT § 1] 은 이 문제를 정량으로 못 박는다 — 작은 chunk 의 deep-memory 훈련은 peak FLOPs 의 5 – 10% 미만밖에 쓰지 못한다는 보고 (LaCT, Zhang, Bi, et al. 2025) 를 인용하면서다.

이 문제의 해법 계보가 **chunkwise-parallel training** 이다 (축약: chunkwise training). 정의: sequence 를 크기 C 의 chunk 로 자르고, **chunk 내부의 계산은 chunk 시작 상태에 대해 병**

렬 (GEMM-rich) 로, chunk 사이의 상태 전달은 순차 (또는 linear recurrence 면 associative scan) 로 재조직하는 훈련 기법의 총칭이다. 기원은 linear-attention 계열의 mixed-chunk 기법 (Hua et al. 2022, arXiv:2202.10447) 이고, RetNet(arXiv:2307.08621) 이 recurrent/parallel/chunkwise 의 세 계산 모드를 명시적 정식화로 굳혔으며, GLA(Yang et al. 2024, arXiv:2312.06635) 가 hardware-efficient kernel 로, DeltaNet 병렬화 (Yang et al. 2024, arXiv:2406.06484) 와 TTT dual form(Sun et al. 2024, arXiv:2407.04620) 이 각각 비대각 transition 과 gradient-step recurrence 로 확장했다. 이 계열의 kernel 구현은 flash-linear-attention 라이브러리 (Yang & Zhang, GitHub, 2024) 에 집대성되어 있다.

세 모드의 전체 지형은 7 장에서 exact 한 세계 (Mamba-2/GLA) 에 대해 이미 그렸다 (표 7-3). 이 장의 임무는 그 표를 **일반화**하는 것이다. 일반화의 대가로 표에 새 열이 하나 생긴다 — “exact 인가?” — 그리고 이 열이 갈라지는 지점이 여섯 논문 전체의 pivot 이다.

표 9-1 — 세 계산 regime 의 일반형 (표 7-3 의 일반화)

모드	token 당 비용 차수	주 연산	병렬성	exact?
recurrent ($C = 1$)	$O(d_k d_v)$ 또는 $O(P_f)$	GEMV + rank-1/param RMW	순차	항상 exact (기준 정의)
parallel ($C = L$)	$O(L d)$	GEMM (attention-like)	완전 병렬	family 에 따라 다름
chunkwise ($1 < C < L$)	$O(C d +$ $d_k d_v / \text{token})$	GEMM + 경계 handoff/scan	chunk 내 병렬	family 에 따라 다름 — 이 장의 주제

(P_f 는 deep memory 한 개의 parameter 수. “exact” 의 기준은 $C = 1$ 의 per-token recurrence 가 정의하는 함수다.)

9.2 일반 scheme: 경계에서 무엇을얼리는가

네 인스턴스에 들어가기 전에, 모든 인스턴스가 공유하는 뼈대를 한 번만정확히 세운다. 이 절이 이 장의 심장이다.

분해. sequence 를 chunk $n = 0, 1, \dots$ (token $nC+1 \dots (n+1)C$) 로 자른다. 임의의 fast-weight update 를 다음두 부분으로 분해한다:

1. **state 에 linear 한 부분** — retention gate 의 곱 ($\alpha_t W_{t-1}$), momentum 의 EMA($\beta_t S_{t-1}$), Hebbian/delta write 의 state 의존항. 이들은 $S_t = a_t S_{t-1} + b_t$ 꼴의 **linear recurrence** 이고, linear recurrence 는 정확하게 병렬화된다: 계수의 누적곱과 입력의 가중합으로 닫힌 형태를 쓰거나 ($\rightarrow$ 아래), Blelloch(1990) 의 associative scan 으로 log-깊이에 계산한다. 독자가 이미 아는 prefix-sum kernel 이 바로 이것이다 (Rosetta: scan/prefix-sum kernel $\leftrightarrow$ momentum 의 associative scan — **동일**).
2. **state 에 nonlinear 한 부분** — deep memory 의 gradient $\nabla_W \ell(W_{t-1}; k_t, v_t)$ 처럼 state 가 MLP 의 forward/backward 를 **통과**하는 항. 이 recurrence 에는 (대각 · 비대각 linear 에서 쓴) 닫힌 형태도 associative scan 도 곧바로 적용되지 않는다 — 일반적인 비선형 recurrence 의 sequence 방향 exact 병렬화는 largely-unsolved 연구문제이고 (근사 · 반복 기반 병렬화 시도는 있다: Gonzalez et al. 2024, Lim et al. 2024) [TNT § 1], 이 라인이 실제로 쓰는 현실적수는 **얼리는 것이다**: chunk 안의 모든 gradient 를 chunk 시작상태 $W_{\xi(t,C)}$ 에서 평가한다 ($\xi(t,C) = C \lfloor (t-1)/C \rfloor, \rightarrow$ § 1.2). 이것이 이 책이 **stale-snapshot 근사** (chunk-start anchor) 라고 부르는 조작이며, 표준형 (M4) 가 그 일반형이다:

$$W_t = W_{\xi(t,C)} - \sum_{\tau=\xi(t,C)+1}^t \eta_\tau \nabla_W \ell(W_{\xi(t,C)}; k_\tau, v_\tau) \quad (\text{M4})$$

핵심 문장을 반복한다: **chunk 안의 모든 gradient**는 **chunk 시작 상태** $W_{\xi(t,C)}$ 에서 평가된다. 따라서 C 는 스케줄이 아니라 계산되는 함수 자체를 바꾸는 **semantic hyperparameter**다 — FlashAttention tiling(bit-exact) 과의 결정적 차이. anchor가 공유되므로 chunk 내부의 C 개 gradient는 상호 독립이고, anchor W_ξ 에 대해 batch C 로 병렬 계산된다. 단 필요한 것은 합산 gradient가 아니라 **token 별** (per-example) gradient g_t 이므로 (momentum scan (§ 9.6) 이 개별 g_t 를 요구한다), 합만 반환하는 표준 batched backward가 아니라 dual-form tensorization이나 per-example(vmap/Jacobian) 미분으로 얻는다 [Sun et al. 2024 App. A.2 – A.3]. 2장에서 굵게 강조한 문장이 여기서 kernel 수준의 실체를 얻는다: **이 라인에서 훈련의 batch 축 역할을 하는 것은 sequence 축이다** — chunk가 곧 inner loop의 mini-batch다.

intra-chunk 계산의 정체. 얼리고 나면 chunk 내부의 linear recurrence는 전부 $C \times C$ 의 삼각 구조로 물화(materialize) 된다. 두 가지 꼴이 있다. (i) 계수가 미리 알려진 explicit recurrence(decay 누적곱, momentum 가중합)는 삼각 가중 행렬과의 GEMM 한 번이 된다 — Atlas가 momentum을 β -가중 행렬 곱으로 물화하는 것이 이 꼴이다 [Atlas Eq. 39]. (ii) 입력이 과거 출력에 의존하는 implicit recurrence(delta rule의 pseudo-value, → 9.4 절)는 단위삼각 행렬의 **triangular solve**가 된다. 두 경우 모두 독자가 FlashAttention tile 안에서 매일 보는 $C \times C$ score 행렬과 같은 shape의 구조화된 연산이다. “chunk 내부는 attention 처럼, chunk 경계는 RNN 처럼” (→ 8 장) 이라는 슬로건의 일반형이 이것이다.

inter-chunk 계산의 정체. chunk 경계에서는 상태 하나 (W_{nC} , 필요시 S_{nC})만 다음 chunk로 넘긴다. 이 handoff가 linear면 (인스턴스 1, 2) chunk 사이도 scan으로 묶을 수 있고, nonlinear면 (인스턴스 3, 4) L/C 개의 순차 사슬이 남는다 — 이 사슬의 길이가 deep memory 훈련의 최종 병목이며, 이를 끊는 것이 TNT의 reset이다 (→ 9.8 절).

비용. chunk 당 intra 항 $O(C^2d)$ + state 상호작용 $O(Cd^2)$ + 경계 연산 $O(\text{poly}(d))$, 총

$$O\left(\frac{L}{C} [C^2d + Cd^2 + \text{poly}(d)]\right) \quad (9-1)$$

$C = 1$ 이면 순수 recurrence, $C = L$ 이면 순수 parallel attention — chunk 크기는 두 극한 사이를 보간한다. 여기까지는 7장의 exact한 세계와 같은 산수다. 다른 것은 단 하나, 인스턴스 3부터 이 보간이 **품질 축도 함께 움직인다**는 점이다.

9.3 인스턴스 1 — GLA / RetNet / Mamba-2: 대각 transition, exact한 folding

가장 쉬운 사례부터. 식 (6-2)와 § 1.6 카탈로그의 gate형 update $W_t = \alpha_t W_{t-1} + v_t k_t^\top$ (α_t : 스칼라, GLA는 채널별 대각 — folding은 elementwise로 동일하게 작동한다)는 state에 완전히 linear하다. 장-국소 기호로 **누적 retention** $\bar{\alpha}_{\tau \rightarrow t} := \prod_{j=\tau+1}^t \alpha_j$ ($\bar{\alpha}_{t \rightarrow t} = 1$)를 정의하면, chunk 시작 $\xi := \xi(t, C)$ 에서 출발한 닫힌 형태가 즉시 나온다:

$$W_t = \bar{\alpha}_{\xi \rightarrow t} W_\xi + \sum_{\tau=\xi+1}^t \bar{\alpha}_{\tau \rightarrow t} v_\tau k_\tau^\top, \quad y_t = \bar{\alpha}_{\xi \rightarrow t} (W_\xi q_t) + \sum_{\tau=\xi+1}^t \bar{\alpha}_{\tau \rightarrow t} (k_\tau^\top q_t) v_\tau \quad (9-2)$$

읽는 법: 출력의 첫 항은 이전 chunk까지의 압축 상태를 decay시켜 읽는 **inter-chunk 항** (GEMV, chunk당 GEMM $Q_n W_\xi^\top$ 로 묶임), 둘째 항은 decay가중치가 곱해진 **causal-masked attention** (intra-chunk 항)이다. chunk의 $K_n, V_n, Q_n \in \mathbb{R}^{C \times d}$ 를 행으로 쌓으면 score $Q_n K_n^\top$, decay-mask 적용, value 곱 — 전부 7장의 식 (7-4)에서 이미 본 GEMM 들이고, 실제로 Mamba-2의 SSD chunked algorithm이 정확히 이 인스턴스다 (→ 7 장). 경계 handoff도 linear이므로 chunk 사이마저 scan으로 처리할 수 있다.

결정적 사실: (9-2)는 **항등 변형**이다. 어떤 근사도 없다. C 를 1로 하든 64로 하든 4096으로 하든 같은 W_t , 같은 y_t 가 나온다 (부동소수점 오차 제외). 따라서 이 family에서 C 는 FlashAttention의 tile 크기와 정확히 같은 지위 — SRAM 크기와 GEMM 살찌우기에 맞춰 고르는 순수 성능 knob — 를 가진다. 구현 디테일로는 decay 누적곱의 dynamic range 때문에 GLA가 log-공간 계산과 chunk 내부의 2차 tile 분할 (secondary-level chunking)을 쓴다는 점만 언급해 둔다 [GLA § 4.3].

9.4 인스턴스 2 — DeltaNet: 비대각 transition과 WY representation

6장이 미뤄 둔 빛을 갠다. DeltaNet의 update (식 (6-3), → 6장)는

$$W_t = W_{t-1}(I - \eta_t k_t k_t^\top) + \eta_t v_t k_t^\top$$

로, transition이 generalized Householder $I - \eta_t k_t k_t^\top$ — **비대각 행렬**이다. state에 여전히 linear하므로 원리상 exact 병렬화가 가능하지만, 9.3 절의 folding은 실패한다: 대각 계수의 누적곱은 elementwise로 접히지만, Householder의 누적곱 $\prod_\tau (I - \eta_\tau k_\tau k_\tau^\top)$ 은 일반 $d_k \times d_k$ 행렬이라 elementwise로 접히지 않는다. rank-1 구조를 쓰면 factor 하나의 적용 · 누적은 $A(I - \eta k k^\top) = A - \eta(Ak)k^\top$ 로 token당 $O(d^2)$ 지만 (순진하게 dense 행렬곱으로 몰화하면 $O(d^3)$ 까지 낭비된다), 그렇게 하면 $d \times d$ 상태를 token마다 **순차**로 갱신 · 몰화해야 해 sequence 병렬성과 IO 효율이 무너진다 — 병렬화로 얻은 것을 도로 태우지 않으려면 이 순차 사슬을 chunk GEMM으로 바꿔야 한다. Yang et al. 2024 (arXiv:2406.06484)가 DeltaNet을 실용화한 열쇠가 수치선형대수의 고전인 **WY representation**이다: Householder 곱을 몰화하는 대신 **compact한 합의 형태로 유지**한다.

유도는 세 줄이다. 첫째, delta update를 다시 쓴다. **pseudo-value** $\tilde{v}_t := \eta_t (v_t - W_{t-1}k_t)$ (장-국소 기호: 교정된 value)를 정의하면

$$W_t = W_{t-1} + \tilde{v}_t k_t^\top \quad (9-3)$$

— delta rule은 “raw value v_t 대신 교정된 value $\tilde{v}_t$ 를 쓰는 Hebbian write”다 (6장의 self-limiting write 논의의 대수적 재진술). 둘째, (9-3)을 chunk 시작부터 전개하면 $W_t = W_\xi + \sum_{\tau=\xi+1}^t \tilde{v}_\tau k_\tau^\top$. 셋째, 이 전개를 $\tilde{v}_t$ 의 정의에 도로 대입하면 pseudo-value 사이의 **implicit recurrence**가 나온다:

$$\tilde{v}_t = \eta_t \left(v_t - W_\xi k_t - \sum_{\tau=\xi+1}^{t-1} (k_\tau^\top k_t) \tilde{v}_\tau \right) \quad (9-4)$$

$\tilde{v}_t$ 가 자기보다 앞선 $\tilde{v}_\tau$ 들에 의존한다 — 그러니 의존 계수가 스칼라 $k_\tau^\top k_t$ 뿐이다. chunk의 pseudo-value를 행으로 쌓아 $\tilde{V} \in \mathbb{R}^{C \times d_v}$, $D_\eta := \text{Diag}(\eta_{\xi+1}, \dots, \eta_{\xi+C})$ 라 하면 (9-4)는

$$(I + D_\eta \text{tril}(K_n K_n^\top, -1)) \tilde{V} = D_\eta (V_n - K_n W_\xi^\top) \quad (9-5)$$

라는 **단위 하삼각 $C \times C$ 선형계**다. 좌변 행렬은 대각이 1인 하삼각이므로 항상 가역이고, forward substitution (TRSM — 독자가 쓰는 BLAS의 표준 연산)으로 $O(C^2)$ 스칼라곱 $\times d_v$ 열에 풀린다. 이삼각계를 만들어 푸는 절차가 Householder 곱의 compact 누적 (수치선형대수문헌의 UT transform, Joffrain et al. 2006)에 해당한다 [Yang et al. 2024 Eq. 10–11]. 나머지는 전부 GEMM이다: score $K_n K_n^\top$ ($C \times d_k$ 대 $d_k \times C$), RHS의 $K_n W_\xi^\top$ ($C \times d_k$ 대 $d_k \times d_v$), 출력 $Y_n = Q_n W_\xi^\top + \text{tril}(Q_n K_n^\top, 0) \tilde{V}$, 경계갱신 $W_{\xi+C} = W_\xi + \tilde{V}^\top K_n$ ($d_v \times C$ 대 $C \times d_k$).

두 가지를 명시한다. 첫째, **이것도 항등 변형**이다. (9-3) → (9-5)의 어느 단계에도 근사가 없다 — $\tilde{v}_t$ 는 진짜 순차 delta rule 이만들었을 바로 그 값이고 (triangular solve가 token 순서의 인과를 정확하게 계산한다), 따라서 DeltaNet의 C 역시 순수 성능 knob이다. Gated DeltaNet (식 (6-4))은 9.3 절의 decay folding과 이 절의 WY를 한 kernel에 합성하면 된다. 둘째, 이 절이 “optimizer-step recurrence를 병렬화하는 지적 템플릿”인 이유다: delta rule은 곧 linear

memory 위의 1-step GD($\rightarrow$ 5 장, 6 장) 이므로, 방금 우리는 “gradient step 의 열 (列) 을 exact 하게 GEMM 으로 재조직” 하는 데 성공한 것이다. 그것이 가능했던 조건 $-\text{loss}$ 가 ℓ_2 이고 memory 가 linear 라서 **gradient** 가 **state** 에 **linear** — 를 기억해 두라. 다음 절에서 이 조건이 깨지는 순간 무엇을지불하게 되는지 본다.

9.5 인스턴스 3 — TTT dual form: anchor 라는 선택, 함수라는대가

8 장 § 8.4 가 TTT-Linear 에 대해 완결한 dual form — anchored 전개 (8-3) 과 닫힌 출력 (8-4) — 을 이 장의 언어로 재정의한다. dual form 은 (M4) 그 자체다: chunk 내 gradient 를 전부 W_ξ 에서 평가하고, intra-chunk 출력은 residual value 의 causal attention 꼴로, 경계 상태는 rank- C GEMM 으로 계산한다. TNT 는 이 형태를 라인의 표준 정식화로 승계했고 [TNT Eq. 3], 이 책의 $\xi(t, C)$ 표기도 TNT 의 것이다 ($\rightarrow$ § 1.2).

여기서 이 장의 관점이 주는 새 정보는 다음의 대조다. **TTT-Linear** 는 **ungated DeltaNet** 과 **같은 update rule** 이다 ($\rightarrow$ 8 장 § 8.3). 그렇다면 9.4 절의 WY 가 그대로 적용되어 exact 병렬화가 가능하다. 그런데 Sun et al. 2024 는 그 길 대신 anchored mini-batch GD 를 택했다 [Sun et al. 2024 § 2.4–2.5]. 왜인가. 두 이유가 이 라인 전체의 설계 논리를 드러낸다.

첫째, **deep memory 때문이다**. $\mathcal{M}$ 이 2-layer MLP 가 되는 순간 $\nabla_{W\ell}$ 은 W 에 nonlinear 해지고 (backward 가 MLP 를 통과한다), (9-4) 류의 implicit recurrence 는 스칼라 계수로 닫히지 않는다. WY 의 전제 조건이 사라진 자리에서 이 라인이 실제로 채택한 병렬화 수단이 anchor 다. 즉 stale-snapshot 근사는 취향이 아니라 **비선형 recurrence 의 exact 병렬화라는 largely-unsolved 문제에 대한 이 라인의 현실적 응답**이다 (일반적 exact 재조직은 알려져 있지 않다) [TNT § 1]. MLP memory 의 intra-chunk 계산도 같은 원리로 tensorize 된다는 것을 Sun et al. 2024 가 App. A 에서 보였다 — 형태는 linear 경우보다 복잡하지만 골격 (anchored gradient 의 batched 계산 + intra-chunk correction) 은 동일하다.

둘째, 더 미묘하고 더 중요하다: 근사가 **kernel** 에서 **model** 정의로 이사했다. TTT layer 는 “inner mini-batch 크기 C 의 mini-batch GD 로 update 하는 layer” 로 정의된다. dual form 은 그 정의를 exact 하게 계산하는 알고리즘이다. 비교하라 — FlashAttention 은 softmax attention 이라는 고정된 함수의 exact 한 재배열이고, DeltaNet 의 WY 는 순차 delta rule 이라는 고정된 함수의 exact 한 재배열이다. 반면 TTT 에서 C 를 바꾸는 것은 **다른 layer 를 정의하는 것이다**: $C = 1$ 이면 순차 delta rule (DeltaNet 과 일치), $C = L$ 이면 1-step batch GD — 곧 linear attention 의 additive write 로 퇴화하고 (8 장 § 8.8 의 손계산: $C=1$ 은 4, $C=2$ 는 6), 그 사이의 모든 C 는 서로 다른 함수다. 함수의 family 가 C 로 매개변수화된 것이다. 이것이 다음 명제의 정확한 의미다.

명제 (chunk 크기 = semantic hyperparameter). anchored chunkwise family (TTT, Titans, Atlas 의 deep memory) 에서 chunk 크기 C 는 계산 스케줄이 아니라 계산되는 함수의 매개변수다. 같은 slow weights Θ 라도 C 가 다르면 다른 sequence-to-sequence 함수가 실행된다. 따라서 (i) 훈련은 특정 C 의 함수에 대해 이루어지고, (ii) serving 에서 다른 C 를 쓰는 것은 훈련되지 않은 함수를 실행하는 것이다.

(ii) 의 결과가 실제로 관측된 것이 [TNT Fig. 2] 의 chunk-size mismatch 이고 ($\rightarrow$ 15 장; 수치는 9.7 절), ‘**chunk 크기 = semantic hyperparameter**’ 명제의 명명과 소유는 이 장에 있다 ($\rightarrow$ § 2.4; mismatch 현상 자체의 소유는 15 장). 반면 exact family (GLA·DeltaNet·Mamba-2) 에서는 train 과 serve 의 C 가 달라도 함수가 같으므로 mismatch 가 **정의상 존재하지 않는다** — “우리 모델은 chunk 병렬화된다” 는 문장을 논문에서만 만나면, 독자는 이제 이 두 지위 중 어느 쪽인지부터 물어야 한다.

9.6 인스턴스 4 — Titans: retention folding + momentum scan

Titans 의 update (M2) 는 세 성분의 합성이다: anchored gradient (비선형, 인스턴스 3 의 수법), retention gate α_t (스칼라 linear, 인스턴스 1 의 수법), momentum S_t (linear recurrence, scan). Titans § 3.2 의 병렬화는 정확히 이 세 수법의 조립이고, 그래서 이 장의 마지막 인스턴스로 배치했다 — 새 아이디어가 아니라 일반 scheme 의 총동원이다.

1 단계 — retention folding [Titans Eq. 16]. momentum 을 잠시 끄고 $W_t = \alpha_t W_{t-1} - \eta_t \nabla_W \ell(W_\xi; k_t, v_t)$ 를 chunk 시작부터 전개하면 (식 (9-2) 와 같은 folding, 9.3 절의 $\bar{\alpha}$ 재사용):

$$W_t = \bar{\alpha}_{\xi \rightarrow t} W_\xi - \sum_{\tau=\xi+1}^t \bar{\alpha}_{\tau \rightarrow t} \eta_\tau \nabla_W \ell(W_\xi; k_\tau, v_\tau) \quad (9-6)$$

(원문 Eq. 16 의 $\beta_i = \prod_j (1 - \alpha_j)$ 는 통일 표기의 $\bar{\alpha}_{0 \rightarrow i}$ 에 해당한다 — Titans 의 gate 기호는 이 책과 방향 · 배치가 다르므로 주의: 원문 $\theta_t \rightarrow$ 통일 η_t , 원문 $\eta_t \rightarrow$ 통일 β_t , 원문 $\alpha_t \rightarrow$ 통일 $1 - \alpha_t$. 전체 대응표는 12 장 § 1.7.1.) linear memory 라면 gradient 가 $(W_\xi k_\tau - v_\tau) k_\tau^\top$ 이므로 (9-6) 의 합 전체가 $(W_\xi K_n^\top - V_n^\top) \text{Diag}(\bar{\alpha} \eta) K_n - \text{rank-}C \text{ GEMM}$ 하나로 물화된다 [Titans Eq. 17]. deep memory 라면 같은 합이 “anchor W_ξ 에 대한 batch C 짜리 per-example 미분” 으로 계산된다 (합산 backward 가 아니라 token 별 g_t 를 내는 dual-form tensorization; § 9.2). 어느 쪽이든 2 장의 $dW = \delta x^\top$ shape 그대로다.

2 단계 — momentum scan [Titans Eq. 18]. momentum 을 켜면 $S_t = \beta_t S_{t-1} - \eta_t g_t$, $g_t := \nabla_W \ell(W_\xi; k_t, v_t)$ (anchored gradient). 관건은 anchor 덕분에 g_t 전부를 미리 계산할 수 있다는 것 — momentum recurrence 가 memory 상태와 분리된, 입력이 알려진 순수 linear recurrence 가 된다. 그러면 이것은 S5(arXiv:2208.04933) 식 associative scan 의 교과서 사례다: 쌍 $(\beta_t, -\eta_t g_t)$ 에 결합 법칙을 만족하는 연산 $(a, b) \bullet (a', b') := (a'a, a'b + b')$ 을 주면 prefix 곱의 둘째성분이 곧 S_t 다. 독자의 prefix-sum kernel 이 그대로 돌아간다. Atlas 는 같은 recurrence 를 scan 대신 삼각 가중 행렬로 물화해 푼다 [Atlas Eq. 36 – 39] — 9.2 절에서 말한 explicit recurrence 의 두 등가 구현이다.

3 단계 — 조립. chunk 하나의 훈련 kernel 은 결국: (i) anchored gradient 의 batched forward+backward(GEMM 덩어리), (ii) momentum 의 intra-chunk scan 혹은 삼각 GEMM, (iii) 식 (9-6) 의 누적 retention 적용, (iv) 출력 계산, (v) 경계 handoff — deep memory 에서는 이 handoff 가 비선형이므로 chunk 사이는 L/C 길이의 순차 사슬로 남는다. Titans 는 여기에 변형 하나를 더 제시한다: 게이트 η, β, α 를 token 의 함수가 아니라 **chunk 의 함수** (chunk 내 상수) 로 두면 intra-chunk 가 LTI 시스템이 되어 global convolution 으로도 계산할 수 있다는 것이다 — 표현력을 잃는 대신 더 빨라지는 선택지로, 실험은 token-dependent 쪽을 썼다 [Titans § 3.2].

Atlas 의 확장. Omega rule 은 gradient 의 평가 대상을 token 하나에서 window c 개로 바꾸지만 (식 (M3), → 14 장), 병렬화 구조는 동일하다: chunk 를 자르고 gradient 를 직전 chunk 의 마지막 상태에 anchor 한다 [Atlas § 3.4]. sliding-window masking 으로 window 항을 intra-chunk GEMM 에 흡수하고, Muon 을 쓸 때는 미리 계산된 momentum 항들에 NS_κ 를 적용한 뒤 update 한다 [Atlas Eq. 40 – 41] — S_t 가 전부 병렬로 나와 있으므로 Newton – Schulz 도 위치별로 병렬인 batched GEMM κ 회다. 2 장에서 “NS-5 = 작은 GEMM 다섯 번” 으로 배운비용 감각이 그대로 이식된다.

[해설] 이 조립에는 systems 독자가 즉시 봐야 할 비용이 하나 숨어 있다. momentum scan 의 원소 g_t, S_t 는 **weight 와 같은 shape** 이다. linear memory 에서는 g_t 가 rank-1 이라 (8-4) 류의 닫힌형태로 물화를 피할 수 있지만, deep memory 에서는 g_t 가 진짜 parameter-shape(P_f 개 원소) 이고, chunk 당 C 개를 만들었다 스캔한다 — traffic 이 $O(C P_f)$ 다. 작은 C 의 deep-memory 훈련이 compute-bound 가 아니라 memory-bound 라는 [TNT § 1] 의 진단은 이 구조에서 나온다.

9.7 Semantic hyperparameter로서의 C : 두지위의총정리

네 인스턴스를 한 표로 접는다.

표 9-2 — 네 인스턴스와 chunk 크기 C 의 지위

인스턴스	transition 의 구조	intra-chunk 핵심 연산	inter-chunk	exact?	C 의 지위
GLA / RetNet / Mamba-2	스칼라 · 대각 linear	decay-masked attention GEMM	스칼라 folding·scan 가능	exact	성능 knob (tile 과동급)
DeltaNet / Gated DeltaNet	비대각 linear (Householder 곱)	$C \times C$ triangular solve + GEMM	행렬 handoff	exact	성능 knob
TTT dual form	nonlinear (또는 linear 라도 anchored 정의)	residual-value causal attention (8-4)	순차 handoff	anchored 근사	semantic
Titans (+Atlas/Muon)	nonlinear + linear momentum + 스칼라 retention	batched fwd/bwd + scan + folding (+NS _{κ})	순차 handoff (비선형)	anchored 근사	semantic

위 두 행과 아래 두 행을 가르는 판정 기준은 단 하나다: **update 가 state 에 linear 한가**. linear 면 삼각 구조로 exact 하게 물화되고, nonlinear 면 열려야 하며, 얻린 결과는 함수의 정의에 들어간다.

semantic 이라는 말의 실증적 무게는 [TNT Fig. 2] 가 보여 준다. $C = 64$ 로 pretraining 한 550M Titans 를 서로 다른 inference chunk 로 돌리면 validation perplexity 가 $C=8$ 에서 36.45, 16 에서 34.15, 32 에서 24.23, **훈련값인 64 에서 13.78 로 최적**, 이후 128 에서 15.5, 256 에서 17.88, 512 에서 22.4 로 다시 나빠진다 [TNT Fig. 2]. 두 가지가 주목할 만하다. 첫째, “작은 chunk = 신선한 gradient = 항상 더 좋음” 이라는 직관이 틀렸다 — 모델은 훈련된 C 라는 **해상도에 과적응**하고, 더 신선한 update 조차 훈련 분포 밖이면 해가된다. 둘째, 이것은 serving 의 이상적 동작점인 $C = 1$ decode(per-token online write, $\rightarrow$ 1 장 Rosetta) 를 정면으로 위협한다: 큰 C 로 훈련된 모델은 이미 $C = 8$ 에서 36.45 로 크게 악화되므로 $C = 1$ 직행은 심각한 위험이다 (단 $C = 1$ 자체는 Fig. 2 에서 측정되지 않았다 — x 축은 8–512 다). 이 관측과 그 처방 (두 단계 훈련) 의 본격 분석은 15 장의 몫이며, 그 증거가 단일 설정 (550M, gating/momentum 없는 단순화 Titans) 의 그림 하나라는 정직성 caveat 도 15 장이 진다.

이 장이 짊어질 정직성 caveat 는 따로 있다: **stale-snapshot 근사의 오차에 대한 형식적 bound 는 여섯 논문 어디에도 없다**. anchored update 가 순차 update 에서 얼마나 벗어나는지 (C , gate 값, curvature 의 함수로서) 는 정리도 lemma 도 없이 전적으로 실험 (위의 Fig. 2 가 사실상 전부) 에 맡겨져 있다. staleness 를 다루는 이론 어휘 자체는 3 장의 online learning (지연된 feedback 하의 regret) 이 제공할 후보지만, 이 라인의 누구도 그 연결을 수행하지 않았다. 독자가 이 라인을 production 에 들일 때 감수해야 할 미정량 리스크의 목록 첫 줄에 이것을 적어 두라.

마지막으로 층위를 하나 올린다. [NL] 은 이 장의 그림을 뒤집어 읽는다: “chunk 경계마다만 update 한다” 를 근사의 부산물이 아니라 **update frequency 라는 설계 축**으로 승격시키고, 서로 다른 $C^{(\ell)}$ 로 도는 level 의 스펙트럼 (CMS) 으로 아키텍처를 재구성한다 (식 (M5), $\rightarrow$ 16 장). 그 관점에서 이 장의 C 는 가장 안쪽 level 의 update 주기이고, outer loop 의 optimizer step 은 가장 바깥 level 의 그것이다 — 같은 scheme 이 재귀적으로 반복된다.

9.8 Systems frontier: 그래도 deep memory 는 느리다 — TNT 의 처방 (골격)

인스턴스 3, 4 까지 조립해도 deep memory 훈련의 MFU 는 낮다. 원인은 이 장의언어로 정확히 셋이다. (1) **순차 사슬**: inter-chunk handoff 가 비선형이라 (상태가 MLP weights 그 자체이고, 구현에 따라 chunk 끝 normalization 까지 낀다 [Atlas App. E]) chunk 사이를 scan 으로 묶을 수 없고, 사슬 길이 L/C 가 그대로 critical path 다. (2) **skinny GEMM**: 품질이 요구하는 작은 $C(8-64)$ 에서 batched fwd/bwd 의 batch 축이 C 이므로 GEMM 이 말라서 tensor core 가 차지 않는다 — decode 에서 batch 를 못 채울 때의 그 현상이다. (3) **parameter-shape traffic**: 9.6 절의 [해설] 대로 scan 원소가 P_f -shape 라 memory-bound 다. 세 원인의 합산 결과가 앞서 인용한 peak 대비 5-10% 미만의 FLOPs utilization 이다 [TNT § 1].

[TNT] 의 처방은 이 장의 scheme 안에서 정확히 두 수를 둔다 (기법의골격만 여기서 세우고, 평가와 전모는 15 장). 첫 수는 **계층화**: 큰 chunk C_g (실험은 2048) 로 도는 global memory W^g 가 장거리 문맥을 맡아 순차 사슬을 L/C_g 개의 크고 dense 한 handoff 로 줄이고, 작은 chunk 의 local memory $W^{l(i)}$ 들이 세밀한 해상도를 맡는다. 둘째 수가 결정타다: local memory 의 상태를 주기 $L_s^{(i)}$ 마다 학습된 초기 상태 W_{init} 로 **reset** 한다. reset 은 shard 경계를 가로지르는 상태 의존을 (gradient 경로까지 포함해) 완전히 절단하므로, L/L_s 개의 shard 가 서로 독립이 된다 — 비선형 recurrence 에는 scan 이 없다는 벽을, recurrence 를 병렬화하는 대신 **recurrence 자체를 주기적으로 끝내버리는** 방식으로 우회한 것이다. 독립 shard 는 device 들에 분산하거나 (context parallelism — 독자에게는 data parallelism 과의 유비로: 자를 수 없던 sequence 축이 reset 덕분에 batch 축처럼 잘리게 됐다) 한 device 의 batch 축에 쌓아 kernel 을 살찌운다. reset 이 버리는 장거리 문맥은 global memory 가 줍고, W_{init} 이 outer loop 에서 학습되므로 (4 장의 MAML 유비, 8 장 표 8-1 의 그 W_{init}) 매 shard 는 0 이 아니라 meta-learn 된 prior 에서 출발한다. 여기에 훈련은 큰 C_1 로, 마지막 ~5%의 compute 로 작은 C'_1 (이상적으로 1) 에 fine-tune 하는 **two-stage 전략**이 얹혀 9.7 절의 mismatch 를 치유한다 — 훈련의 C 와 serving 의 C 를 분리해, semantic hyperparameter 가 강요하던 단일 절충값을 두 개의 knob 으로 쪼갠 것이다. 이 조합으로 TNT 는 가장 정확한 Titans baseline 대비 최대 $17.37\times$ 빠르게 같은 loss 에 도달했다고 보고한다 [TNT Table 1]. 나머지 — Q-K projection Π_t , 다중 해상도 $\{C_1^{(i)}\}$, 수치와 한계 — 는 15 장에서.

9.9 Worked micro-example: 같은 두 token, 두 개의 병렬화

8 장 § 8.8 의 예제를 그대로 이어받아 — $d_k = d_v = 2$, $k_1 = k_2 = (1, 0)^\top$, $v_1 = (2, 0)^\top$, $v_2 = (4, 0)^\top$, $W_0 = 0$, $\eta_t = 1$, $q_2 = (1, 0)^\top$ — 이 장의 두 주장 (WY 의 exactness, momentum scan 의 exactness) 을 손으로 확인한다. 8 장에서 이미 안다: 순차 delta rule ($C=1$) 의 답은 $y_2 = (4, 0)^\top$, anchored dual form ($C=2$) 의 답은 $(6, 0)^\top$ 이었다.

Part A — DeltaNet 을 $C = 2$ 로, WY/UT 로. (9-5) 를 조립한다. $K_2 = \begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix}$, $V_2 = \begin{pmatrix} 2 & 0 \\ 4 & 0 \end{pmatrix}$, $D_\eta = I$, $W_0 = 0$ 이므로 RHS = V_2 . score 는 $K_2 K_2^\top = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$, 그 strict lower 부분은 $\begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}$. 삼각계는

$$\begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} \tilde{V} = \begin{pmatrix} 2 & 0 \\ 4 & 0 \end{pmatrix} \Rightarrow \tilde{V} = \begin{pmatrix} 2 & 0 \\ 2 & 0 \end{pmatrix}$$

forward substitution 두 줄이다: 1 행 $\tilde{v}_1^\top = (2, 0)$; 2 행 $\tilde{v}_2^\top = (4, 0) - (2, 0) = (2, 0)$ — solve 가 “token 2 는 token 1 이 이미 2 를 써 두었음을 안다” 는 인과를 정확히 재현했다 (8 장의 anchored 계산은 이 빼기를 못 해서 $(4, 0)$ 을 통째로 썼다). 상태와 출력은

$$W_2 = \tilde{V}^\top K_2 = \begin{pmatrix} 2 & 2 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 4 & 0 \\ 0 & 0 \end{pmatrix}, \quad y_2 = W_2 q_2 = (4, 0)^\top$$

순차 계산과 **정확히 일치**한다. chunk 를 2 로 잘랐지만 함수는 그대로다 — exact family 의 실감이다.

Part B — Titans 의 momentum scan ($C = 2, \beta_t \equiv \frac{1}{2}, \alpha_t \equiv 1$). anchored gradient 부터: $g_t = (W_0 k_t - v_t) k_t^\top = -v_t k_t^\top$ 이므로 $g_1 = \begin{pmatrix} -2 & 0 \\ 0 & 0 \end{pmatrix}, g_2 = \begin{pmatrix} -4 & 0 \\ 0 & 0 \end{pmatrix}$ — 두 개가 동시에 (batch 2 의 GEMM 으로) 나온다. 순차로 momentum 을 돌리면: $S_1 = \frac{1}{2}S_0 - g_1 = \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix}, S_2 = \frac{1}{2}S_1 - g_2 = \begin{pmatrix} 5 & 0 \\ 0 & 0 \end{pmatrix}$. 이제 scan 으로: 쌍 $(\frac{1}{2}, -g_1)$ 과 $(\frac{1}{2}, -g_2)$ 에 9.6 절의 결합 연산 $(a, b) \bullet (a', b') = (a'a, a'b + b')$ 을 적용하면 $(\frac{1}{4}, \frac{1}{2} \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix} + \begin{pmatrix} 4 & 0 \\ 0 & 0 \end{pmatrix})$ 이고 둘째 성분이 $\begin{pmatrix} 5 & 0 \\ 0 & 0 \end{pmatrix}$ — 순차 계산과 일치한다. momentum recurrence 자체는 linear 이므로 scan 은 **아무 근사도 더하지 않는다**. 상태는 $W_1 = W_0 + S_1, W_2 = W_1 + S_2 = \begin{pmatrix} 7 & 0 \\ 0 & 0 \end{pmatrix}$, 읽으면 $y_2 = (7, 0)^\top$.

세 숫자를 나란히 놓자: 순차 delta rule 4, anchored TTT($C=2$) 6, anchored Titans($C=2$, momentum) 7. **4 → 4(Part A) 는 exact 병렬화** — C 가 함수를 건드리지 않았다. **4 → 6 은 anchor 의 staleness** — 근사가 함수를 바꿨다. **6 → 7 은 momentum** — token 1 의 surprise 절반 ($\frac{1}{2} \times 2$) 이 token 2 의 write 에 실려 들어온 것이고, 이는 근사가 아니라 (M2) 가 정의한 함수의 정당한 출력이다. 어느 차이가 알고리즘이고 어느 차이가 근사인지 — 이 장에서 가져가야 할 감각이 정확히 이 구분이다.

9.10 Systems bridge: roofline 위의 C

이 장 전체를 독자의 cost model 로 접는다. 식 (9-1) 의 chunk 당 비용을 GLA/RetNet 형 decay kernel (인스턴스 1, head 당, $d_k = d_v = d$) 에 대해 구체화하자 (DeltaNet 은 여기에 Gram $K_n K_n^\top \cdot \text{triangular solve} \cdot K_n W_\xi^\top$ 항이 더 붙는다 — 차수는 같은 $C^2 d + C d^2$ 지만 상수가 커진다). FLOPs(MAC 당 2 FLOP): score $Q_n K_n^\top$ 에 $2C^2 d$, masked score 와 value/pseudo-value 의 곱에 $2C^2 d$, 상태 읽기 $Q_n W_\xi^\top$ 에 $2C d^2$, 상태 갱신 ($U^\top K_n$ 또는 $K_n^\top V_n$ 류) 에 $2C d^2$ — 합계 $F(C) \approx 4C^2 d + 4C d^2$. bytes(bf16, 2 B/원소): K, Q, V 읽기와 Y 쓰기 $4C d$ 원소, 상태 read-modify-write $2d^2$ 원소 — 합계 $B(C) \approx 8C d + 4d^2$ bytes. 나누면 놀랄 만큼 깨끗한 식이 나온다:

$$\text{AI}(C) = \frac{F(C)}{B(C)} = \frac{4C d (C + d)}{4d (2C + d)} = \frac{C (C + d)}{2C + d} \xrightarrow{C \ll d} \approx C \quad (9-7)$$

작은 chunk regime 에서 arithmetic intensity 는 곧 chunk 크기다. roofline 의 x 축 좌표를 C 라는 단일 knob 이 직접 쥐고 있는 셈이다. 숫자를 넣어 보자 ($d = 1024$).

표 9-3 — $\text{AI}(C)$ 와 GEMM shape (GLA/RetNet 형 decay kernel 기준; $d = 1024$, bf16, head 당)

C	주 GEMM shape ($C \times d \cdot d \times d$ 등)	$\text{AI}(C)$ [FLOP/B]	ridge 대비
8	8×1024 짜리 — 극단적 skinny	7.9	~3%
16	16×1024	15.8	~5%
64	64×1024	60.4	~20%
256	256×1024	213	~72%
1024	1024×1024 — 정방	683	ridge 초과

ridge 는 대표적 최신 accelerator 의 공개 spec(NVIDIA H100 SXM datasheet: bf16 dense 약 989 TFLOP/s, HBM3 약 3.35 TB/s) 으로 약 295 FLOP/B 다. 표가 말하는 것: 품질이 원하는

$C(8-64)$ — TTT의 inner mini-batch, TNT 실험에서 품질최선이던 Titans $C=8$)에서 kernel은 ridge의 3-20% 지점, 즉 깊은 bandwidth-bound 영역에 앉아 있고, MFU가 원하는 C 는 300 이상이다. **품질 최적 C 와 MFU 최적 C 는 같은 축의 반대편 끝에 있다.** [TNT § 1]이 인용한 “peak의 5-10% 미만”은 이 표의 위쪽 행들을 그대로 읽은 것이며, exact family라면 이 긴장은 “훈련이 좀 느리다”로 끝나지만 semantic family에서는 C 가 품질 축이기도 하므로 **절충 불가능한이율배반**이 된다 — 이것이 TNT라는 논문이 존재하는 이유의 전부이고, 독자는 이제 그것을 (9-7)한 줄에서 재유도할 수 있다.

deep memory(인스턴스 3-4)는 여기에 두 겹을 얹는다. batched fwd/bwd의 GEMM들은 batch 축이 C 인 $(C \times d) \cdot (d \times 4d)$ shape이라 — decode에서 batch를 못 채운 GEMM과 동일한 병리로 — 작은 C 에서 tensor core 효율이 더 깎이고, momentum·gradient가 parameter-shape($P_f \approx 8d^2$)라 scan traffic이 $O(C P_f)$ 로 커진다 (9.6절 [해설]). 같은 C 에서 deep memory의 실효 MFU가 linear memory보다 더 나쁜 이유이며, TNT의 reset이 shard들을 batch 축으로 쌓아 공격하는 지점이 정확히 첫 번째 겹이다. 이 회계의 아키텍처 전반 cheat sheet — softmax attention부터 Titans까지 한 표 — 는 10장이 완성한다.

요약

- chunkwise-parallel training의 일반 scheme은 하나다: update를 state-linear 성분과 state-nonlinear 성분으로 분해해, linear 성분 (retention 누적곱, momentum EMA)은 folding·scan·삼각 GEMM으로 **정확히**, nonlinear 성분 (deep memory의 gradient)은 chunk-start anchor로 **열려서** 계산한다. chunk 내부는 $C \times C$ 삼각 구조의 GEMM, chunk 사이는 handoff다.
- 판정 기준은 transition의 구조다: 스칼라·대각 linear(GLA/RetNet/Mamba-2)는 decay folding으로, 비대각 linear(DeltaNet)는 WY representation의 pseudo-value 삼각계 (9-5)로 — 둘 다 **exact**이고 C 는 순수성능 knob이다.
- nonlinear transition에는 알려진 효율적 exact 재조직이 없다; stale-snapshot 근사(M4)가 이 라인의 현실적 응답이고, 그 결과 근사가 model 정의로 들어가 **chunk 크기 C 는 semantic hyperparameter가 된다** — 같은 Θ 라도 C 가 다르면 다른 함수다 (순계산: 순차 4 vs anchored 6). FlashAttention tiling(bit-exact)과 범주가 다르다.
- Titans의 병렬화는 세 수법의 조립이다: anchored gradient의 batched fwd/bwd [Titans Eq. 16-17] + momentum의 associative scan [Titans Eq. 18] + retention folding; Atlas는 같은 골격에 window masking과 NS_{κ} 를 얹는다 [Atlas § 3.4, Eq. 36-41].
- semantic의 실증: $C=64$ 로 훈련된 550M Titans는 inference chunk 64에서 ppl 13.78로 최적이고 8에서 36.45, 512에서 22.4로 무너진다 [TNT Fig. 2] — 모델은 훈련 해상도에 과적응하며, $C=1$ decode라는 이상적 serving 모드를 위협한다 ($C=8$ 에서 이미 36.45; $C=1$ 은 Fig. 2에서 미측정).
- stale-snapshot 근사의 오차에 대한 형식적 bound는 여섯 논문 어디에도 없다 — 이 라인의 최대 미정량 리스크다.
- arithmetic intensity는 $AI(C) = C(C+d)/(2C+d) \approx C$ (작은 C): 품질 최적 $C(8-64)$ 는 ridge의 3-20%, MFU 최적 C 는 300+ — 이 이율배반이 TNT의 존재 이유이고, TNT는 계층화(global C_g) + 주기적 reset(context parallelism) + two-stage 훈련으로 두 knob을 분리한다.

자가 점검체크리스트

- ☐ 임의의 update rule을 받아 state-linear/nonlinear 성분으로 분해하고, exact 병렬화 가능 여부를 판정할 수 있다.
- ☐ DeltaNet의 pseudo-value 재작성 (9-3) - (9-5)를 백지에서 유도하고, 삼각계의 forward substitution이 왜 순차 delta rule과 bit-동등한지 (부동소수점 제외) 설명할 수 있다.

- $d = 2$ 예제에서 순차 (4)/anchored(6)/momentum(7) 의 세 답을 재현하고, 각 차이가 근사인지 알고리즘인지 구분할 수 있다.
- “chunk 크기 = semantic hyperparameter” 명제를 정확히 진술하고, exact family 에는 train/serve chunk mismatch 가 정의상 없는 이유를 말할 수 있다.
- Titans chunk kernel 의 5 단계 조립 (batched fwd/bwd → scan → folding → 출력 → handoff) 과 각 단계의 GEMM/scan 지위를 나열할 수 있다.
- $AI(C)$ 식 (9-7) 을 유도하고, 자기 조직의 accelerator spec 으로 ridge 대비 utilization 을 추정할 수 있다.
- 이 장을 inference 어휘로 옮길 수 있다: chunk = semantic tile (bit-exact tile 이 아님), momentum scan = prefix-sum kernel, 작은 C 의 skinny GEMM = batch 못 채운 decode GEMM, TNT 의 reset = sequence 축을 batch 축처럼 자르는 허가증.

다음 장으로

이 장으로 Part I 의 모델 · 알고리즘 자료는 전부 갖춰졌다: memory 의 어휘 (5 장), update rule 의 계보 (6 – 8 장), 그리고 그것을 하드웨어에 올리는 유일한 방법 (이 장). 남은 질문은 회계다 — softmax attention 부터 Titans 까지, state 는 몇 byte 이고 decode token 당 FLOPs 와 traffic 은 얼마이며, prefill 의 병렬성은 어느 유형이고, 논문들의 효율 주장은 어디까지 믿을 수 있는가. 10 장은 이 장의 (9-7) 식 분석을 아키텍처 전개열의 cost-model cheat sheet 로 확장하고, Part II 에서 여섯 논문을 감사 (audit) 할 때 쓸 도구 상자와 용어집을 완성한다.

ch10. 통합 systems bridge: cost model cheat sheet, roofline vs chunk size, glossary

이 장의 목표 — (1) (M1) – (M4) 형태의 임의 layer 에 대해 decode FLOPs/token·state 트래픽·state 크기를 유도한다. (2) chunk 크기 C 를 roofline 독립변수로 놓고 품질-최적 C 와 MFU-최적 C 가 갈라지는 이유를 숫자로 보인다. (3) KV cache vs fast-weight state 의 memory-budget 승부를 자기 config 로판정한다. (4) Part II 의 efficiency 주장을 이 cheat sheet 로 환원해검산한다.

왜 필요한가 — [TNT] (Improving Chunkwise Training for Test-Time Memorization, arXiv:2511.07343) 는 사실상 이 장의 언어로 쓰인 논문이다: Challenge 1(peak FLOPs 5-10% 미만)·Challenge 3(chunk-size mismatch) 은 roofline 없이는 판독되지 않는다. [Titans § 3.2]·[Atlas § 3.4] 의 병렬화 주장도 이 도구로 audit 하며, Part II 전 장의 § “systems 합의” 는 표 10-2·10-4 를 참조 대상으로 삼는다.

나머지 논문의 축약: [Titans] (Titans: Learning to Memorize at Test Time, arXiv:2501.00663), [Atlas] (Atlas: Learning to Optimally Memorize the Context at Test Time, arXiv:2505.23735), [NL] (Nested Learning, arXiv:2512.24695), [Sleep] (Language Models Need Sleep, arXiv:2606.03979).

Part I 각 장이 말미에 깬 systems bridge 조각을 하나의 분석 도구 상자로 통합한다. 6 편 모두 “hardware 에 더 잘 맞는다” 고 주장하지만 측정은 훈련쪽에 몰리고 decode 쪽은 비어 있다 (6 편 전체에 decode wall-clock 부재; serving 주장은 전부 구조적 논증이지 측정이 아니다) — 논문 숫자를 받아적는 대신 이 장의 cost model 위에서 재계산하는 것이 현재 그 비용을 알 유일한 길이다.

10.1 완성된 Rosetta-Stone 사전

1 장의 inference ↔ learning 사전에서 상세 해설은 표 1-1(→ 1 장) 이 소유하므로, 여기서는 이 장이 새로 확립하는 행과 직관을 이식하면 틀리는 $\triangle$ 지점만 **차분표**로 신는다. 나머지 대응은 표 1-1 을 보라.

표 10-1 — 이 장이 새로 확립하는 대응 + $\triangle$ 오이식 지점 (나머지 전체대응·해설은 표 1-1(→ 1 장))

inference 세계 (독자의 어휘)	이 라인의 어휘	확립한 장
roofline / arithmetic intensity	chunk 크기 C = 새 독립변수	이 장
batching (shared weights 전제)	$\triangle$ per-request fast-weight state → grouped-GEMM decode	이 장

inference 세계 (독자의 어휘)	이 라인의 어휘	확립한 장
paged KV cache / session cache	per-session weight state — 새로운 cache class	이 장
speculative decoding 의 rollback	memory state snapshot/ rollback	이 장

△ 세 지점은 inference 직관을 그대로 이식하면 틀린다: FlashAttention 의 tile 과 달리 chunk C 는 계산되는 함수를 바꾸고 (semantic, § 10.5), fast-weight state 는 shared-weight batching 의 전제를 깨며 (§ 10.4), 이라인의 “mini-batch” 는 batch 축이 아니라 sequence 축의 chunk 다 (→ 2·9 장).

10.2 Cost-model cheat sheet

아키텍처 클래스별 비용을 한 표로 모은다. 회계 규약 (표 10-2 와 이후 모든 계산에 공통):

- **범위**: layer 1 개 · token 1 개 (decode 기준). head 병렬 · batch 축 · projection 비용은 전 행공통이라 생략.
- **FLOPs**: MAC 1 회 = 2 FLOPs; backward pass $\approx$ forward 의 $2\times$ (→ 2 장); 지배항만 남긴다.
- **bytes**: bf16 원소당 2 bytes. **state RMW 트래픽** = state 를 읽고 (read) 갱신해 (modify) 다시 쓰는 (write) 왕복 — 원소 수 s 당 $4s$ bytes.
- **기호**: L = 문맥 길이, w = sliding window, d = 차원 ($d_k = d_v = d$), c = Omega rule window, P_M = deep memory parameter 수 (표준이면 $8d^2$), N = TNT local memory 개수.

표 10-2 — 아키텍처 클래스별 cost-model cheat sheet (layer 당, token 당, 지배항)

아키텍처	state (원소 수)	decode FLOPs/ token	state 트래픽 (bytes/token)	병렬화 형태와 MFU 결정 요인
softmax attention + KV cache	$2Ld$ (증가)	$\approx 4Ld$	$4Ld$ 읽기 + $4d$ append	$O(L^2d)$ GEMM; tiling 은 exact; decode 는 cache 재읽기 지배
sliding- window attention (SWA)	$2wd$	$\approx 4wd$	$4wd$	동일하되 window-local; L 과 무관
linear attention (un-gated)	$\approx d^2$	$\approx 4d^2$	$4d^2$ RMW	chunkwise- GEMM(exact , → 9 장) 또는 scan; C 는 tile 성 knob
GLA / Mamba-2	d^2 (+gate 소량)	$\approx 5d^2$	$4d^2$ RMW	SSD block 분 해 · 2-level tiling(exact); scan 형은 bandwidth- bound (→ 7 장)

아키텍처	state (원소 수)	decode FLOPs/ token	state 트래픽 (bytes/token)	병렬화 형태와 MFU 결정 요인
DeltaNet / Gated DeltaNet	d^2	$\approx 6d^2$	$4d^2$ RMW	WY/UT 로 chunkwise- GEMM(exact , $\rightarrow 9$ 장); Householder 사슬이 kernel 복잡도 결정 dual form chunkwise- GEMM – 단 semantic : C 가 함수를 바꾼다 (M4)
TTT-Linear	d^2	$\approx 6d^2$	$4d^2$ RMW	dual form(se- mantic) + 비선 형 inter-chunk 의존이 직렬화 강제 [TNT § 1]
TTT-MLP	$P_{\mathcal{M}}$	$\approx 8 - 10 P_{\mathcal{M}}$	$4P_{\mathcal{M}}$ RMW	chunkwise(se- mantic) + momentum scan + token- dependent gate [Titans § 3.2] (chunk- 상수 gate 는 원 문이 제시한 미검 증 속도 최적화)
Titans-LMM	$2P_{\mathcal{M}} (W_t, S_t)$	$\approx 12 P_{\mathcal{M}}$	$8P_{\mathcal{M}}$ RMW	Omega rule 병 렬화 [Atlas § 3.4]; NS_{κ} = chunk 당 κ 회 소형 GEMM
Atlas (OmegaNet 계열)	$2P_{\mathcal{M}} + 2cd$	(M3) 그대로면 $\approx 6c P_{\mathcal{M}} + NS_{\kappa}$ 상각분	$8P_{\mathcal{M}} + 4cd$	global = 순차 큰 GEMM, local = reset 로 shard 병렬 [TNT § 4.1]
TNT (global + N local)	$(1+N)P_{\mathcal{M}} + Nd^2$	$\approx 2P_{\mathcal{M}} +$ $N(8P_{\mathcal{M}} + 4d^2)$ (global write 는 C_g 로 상각)	global read $2P_{\mathcal{M}} +$ local RMW $4NP_{\mathcal{M}} +$ $\Pi 4Nd^2$ (global write RMW 는 C_g 로 상각)	

KV cache 클래스 (1 – 2 행). state 가 자라고 decode 비용이 L 에비례하는 유일한 클래스 – 나머지 여덟 행의 대조군이다.

matrix-state 클래스 (3 – 6 행). state 가 d^2 고정이고 decode 의본질은 state RMW 다 (Mamba-2 의 SSM state 차원 N 은 자유 파라미터라 실제 state $d_v \times N$ 은 d^2 보다 작을 수 있으나 표는 통일 규약값). token 당 GEMV 몇 개 + rank-1 갱신뿐이라 FLOPs/byte 가 한자릿수 (§ 10.6) 이므로 decode 는 예외 없이 bandwidth-bound 다 – 승부처는 “state $4d^2$ bytes 를 매 token 왕복시키는가” . linear $\rightarrow$ GLA $\rightarrow$ DeltaNet 으로계수가 $4 \rightarrow 5 \rightarrow 6$ 이 되어도 트래픽은 동일: bandwidth-bound 에서 계수 큰 FLOPs 는 공짜라, retention gate-delta rule 의 품질 이득 이 decode 에서 거의 무료다.

deep-memory 클래스 (7-9 행). state 가 작은 신경망 weights 전체 (P_M) 가 되고 write 가 forward+backward+update 가 된다: compression $\approx 6P_M$ + retrieval $2P_M$ = 계수 8-10. Titans-LMM 은 momentum-retention 이 없혀 $\approx 12P_M$ 이고 S_t 도 session state 라 **state** 가 두 배다. Atlas 행은 경고표지판이다: 식 (M3) 을 문자대로 실행하면 decode FLOPs 가 c 배로 뛰고 최근 c 쌍 buffer($2cd$ 원소) 가 state 에 붙는다 — 이 상각이 [Atlas § 3.4] 의 주제다 ($\rightarrow$ 14 장). 게다가 Muon 의 NS_k (반복마다행렬-행렬 곱) 는 $C = 1$ decode 에 상각할 chunk 축이 없어 이 행만은 compute-bound 가 될 수 있다 [Atlas Eq. 40-41]. c 스윙 $\{2, 4, 8, 16\}$ (c 클수록 ppl 낮음) 의 headline 값 · decode window 유지 절차는 원문에 명시가 없다 [Atlas Fig. 5, Table 6].

TNT 행 (10 행). 위 회계의 합성이다: global write 는 C_g token 마다 한 번으로 상각되고, 매 token 비용은 N 개 local 의 fwd+bwd+read + 읽기 전용 global read + N 개 Π_t 갱신이라 FLOPs · 트래픽 모두 N 에 비례한다. state 가 문맥길이와 무관한 상수라는 것이 [TNT] 의 serving 논지다.

10.3 KV cache vs fast weights: memory-budget 산수

“고정 크기 state 니까 이긴다” 는 절반만 참이다 — 언제부터 이기는지는 나뉠셈 하나로 판정된다. layer 당 state 원소 수를 s , KV cache 는 token 당 $2d$ 원소를 쌓으므로 crossover 문맥 길이는

$$L^* = \frac{s}{2d} \quad (10-1)$$

이다 (layer 수 · 정밀도 소거). matrix memory ($s = d^2$) 는 $L^* = d/2$, 표준 deep memory ($8d^2$) 는 $4d$, momentum 포함 ($2P_M$) 은 $8d$. 아래는 $d = 2048 \cdot 24$ layer · bf16 · GQA 없는 MHA · 전 layer memory 인 보수적 config 다 — 자기 스택 값으로 갈아 끼우는 것이 목적이다.

표 10-3 — memory budget: KV cache vs fast-weight state ($d = 2048, 24$ layers, bf16)

항목	4K 문맥	64K 문맥	1M 문맥	비고
KV cache (전체)	768 MiB	12 GiB	192 GiB	token 당 192 KiB 씩 선형증가
matrix memory (d^2 /layer)	192 MiB	192 MiB	192 MiB	$L^* = d/2 = 1024$ tokens
deep memory ($8d^2$ /layer)	1.5 GiB	1.5 GiB	1.5 GiB	$L^* = 4d = 8192$ tokens
deep memory + momentum	3 GiB	3 GiB	3 GiB	$L^* = 8d = 16384$ tokens

첫째, **fast weights** 는 짧은 문맥에선 진다: deep memory + momentum 3 GiB $\approx$ 16K token KV cache 라, 4K 세션만 서빙하는 fleet 엔 Titans 류가 budget 손해다 — 이 라인이 long-context 를 앞세우는 것은 취향이 아니라 산수다. 둘째, **1M**에선 두 자릿수 배율로 갈린다 (192 GiB vs 3 GiB): paged · quantization · eviction 으로도 $O(L)$ 성장 자체는 못 없앤다 — 성장 곡선을 상수로 접는 대신 § 10.4 의 새 serving 문제로 대가를 내는 거래다.

따름정리: 64K softmax decode 는 매 token cache 12 GiB 를 재읽어 HBM 3 TB/s 급에서 4 ms/token. deep memory + momentum 의 state RMW 6 GiB 는 **전체** 합이고 문맥과 무관하다. 긴문맥에서 이 라인이 파는 것은 FLOPs 가 아니라 bandwidth 절감이다.

10.4 Prefill/decode 비대칭과 serving engine: backward pass 가들 어온다

prefill/decode 비대칭은 물려받되 내용물이 바뀐다. **prefill** = 큰 chunk 의 병렬 write (프롬프트를 chunk 로 묶어 (M4) 의 batched gradient 를 GEMM 으로 흘리는 compute-bound 구

간), **decode** = $C = 1$ 의 **online write + read**(token 마다 작은 net 의 fwd·bwd·update·read). [TNT § 4.2] 는 이 대응이 설계 목표다: global memory($C_g = 2048$) 가 prefill 을, Stage 2 에서 $C'_1 = 1$ 로 적응시킨 local 이 decode 를 맡고, prefill 시 local 은 reset 덕에 L/L_s 개 shard 로 병렬 채워진다.

backward pass 가 decode 에 들어온다는 사실이 serving engine 에 요구하는 변화는 네 가지다 (Part III 에서 재론; 여기서는 비용 구조만).

1. autograd 없이 gradient 를 계산해야 한다. serving engine 은 autograd graph 를 안 실행하지만 이 라인의 inner gradient 는 전부 닫힌 형태다: linear memory 는 $\nabla_W \ell = 2(Wk_t - v_t)k_t^\top$ (GEMV+벡셀+outer product), deep memory 도 손유도 GEMM 서너 개. “backward” 는 **shape 가 알려진 GEMM 들을 fused RMW kernel 로 굳히는 일** — 훈련프레임워크가 아니라 FlashAttention 을 짜던 방식이다.

2. per-session state 가 새로운 cache class 가 된다. 크기는 표 10-3 이 해결하고, paged KV cache(Kwon et al. 2023, arXiv:2309.06180) 와 달리 고정 크기라 allocator 는 단순하지만 새 문제 셋이 생긴다. (a) **checkpoint/restore**: $W_t(S_t, \Pi_t)$ 포함 를통째로 저장하거나 프롬프트를 replay — TNT 의 periodic reset 은 **local** 재생 분량을 L_s token 으로 유계화하나 reset 안 되는 global W^g 는 여전히 snapshot/replay 가 필요하다. (b) **prefix 재사용**: state 가 전체 history 의 함수라 중간 편집 불가·snapshot 분기만 가능하되, 고정 크기라 복사는 KV cache 보다 싸다. (c) **eviction**: 손으로 짜던 policy 가 모델 안 retention gate α_t 라는 학습된 정책이 된다 ($\rightarrow$ 3·13 장); fleet 수준 eviction 은 엔진 몫이다.

3. shared-weight batching 이 깨진다. B 개 request 가 같은 weights 를 읽어 트래픽을 상각한다는 전제가 무너진다: request 마다 자기 $W_t^{(r)}$ 가 있어 read/write 가 B 개의 서로 다른 GEMV/RMW — multi-LoRA 의 grouped-GEMM 패턴이다. state 트래픽이 B 에 비례해 자라 **batching 이 intensity 를 못 올린다** — compute-bound 탈출로가 막힌다. (head·layer 병렬성은 남고, shard 를 batch 축에 쌓는 훈련 트릭도 있다 [TNT § 4.1].)

4. rollback 이 optimizer-trajectory 문제가 된다. speculative decoding 에서 draft 각각 시 KV cache 는 꼬리만 자르면 되지만 fast-weight 모델은 $W_t \cdot S_t \cdot \Pi_t$ 궤적 전체를 되돌려야 한다. snapshot 은 싸지만 “몇 token 마다 어느 buffer 까지” 찍을지는 6 편 어디에도 답이 없는 열린 문제다.

10.5 FlashAttention tiling vs chunkwise training: roofline 위의 chunk 크기

FlashAttention(Dao et al. 2022, arXiv:2205.14135; FA-2 는 Dao 2023, arXiv:2307.08691) 의 tiling 은 **같은 합의순서 재배열**이라 tile 크기는 SRAM 용 구현 knob 이고 출력은 동일하다. 반면 chunkwise 의 C 는 (M4) 대로 **gradient 평가 지점을 바꾼다**: 모든 gradient 가 chunk 시작 $W_{\xi(t,C)}$ 에서 평가되는 stale-snapshot 근사라 C 가 다르면 다른 함수가 나온다 — tile 이 아니라 **semantic hyperparameter** 다 ($\rightarrow$ 9 장). 경계는 $\mathcal{M}$ 의 선형성이 아니라 **정의 방식**이다: 명시적 linear recurrence 를대수적으로 재배열하는 linear attention/GLA/DeltaNet 은 chunkwise 가 exact 지만, gradient 평가점을 W_ξ 에 고정하는 anchored mini-batch GD 로 **정의된** 모델은 linear 여도 (TTT-Linear) C 가 semantic 이다 — § 8.8 이 linear memory 에서 $C=1$ 은 4, $C=2$ 는 6 을 낸 것이증거다 ($\rightarrow$ 표 9-2).

실증은 [TNT Fig. 2]: $C = 64$ 로 pre-train 한 550M Titans 를 inference chunk 만 바꾸면 perplexity 가 8/16/32/64/128/256/512 에서 36.45/34.15/24.23/13.78/15.5/17.88/22.4 — 훈련 chunk 에서 최적이고 양쪽으로 무너진다. tile 을 바꿔 품질이 $2.6\times$ 나빠지는 kernel 은 없다 — 이 그림이 “ C 는 tile 이 아니다” 의증명이다 (단일 설정 한계는 15 장).

이제 C 를 roofline(Williams, Waterman & Patterson, CACM 2009) 위에놓는다. delta 계열 chunk 하나를 fused kernel 이 처리하면 chunk 당 FLOPs = state 처리 (계수 6) $\approx 6Cd^2$ + chunk 내부 $C \times C$ attention 형 항 (dual form (8-4)) $\approx 4C^2d$, 트래픽 = state RMW $4d^2$ + token IO(k, v, q, y) $8Cd$ bytes:

$$AI(C) = \frac{4C^2d + 6Cd^2}{4d^2 + 8Cd} \text{ [FLOP/byte]} \quad (10-2)$$

두 극한: $C \ll d$ 면 intra-chunk 항이 무시돼 $AI \approx 1.5C$ (decode 영역), $C \gg d$ 면 $4C^2d$ 항이 지배해 $AI \approx 0.5C$ — 둘 다 C 에 따라 자라 **bandwidth 천장이 없고**, $C = L$ 극한에서 총 FLOPs $O(L^2d)$ 의 순수 attention(compute-bound) 으로 이어진다. 작은 d 의 head 도 C 를충분히 키우면 원리적으로 ridge 를 넘지만 그 C 는 gradient 를 파괴적으로 stale 하게 만든다 (§ 10.6). deep memory 는 state 항이 $\sim P_M/d$ 로 커지는 데다 비선형 inter-chunk 의존이병렬화를 막는다 [TNT § 1]. 세 영역 ($C = 1$ recurrent = bandwidth-bound, $1 < C < L$ chunkwise = ridge 로 이동, $C = L$ 병렬 = compute-bound $\cdot O(L^2d)$) 은 하나의 roofline 위 연속보간이고 C 가 보간 위치다.

여기서 근본 긴장이 나온다. **품질-최적 C 는 작고 MFU-최적 C 는 크다**. 품질: chunk 를 일치시키면 $C = 8$ 이 $C = 256$ 보다 낮다 (avg ppl 25.07 vs 27.13 [TNT Table 2]) — 신선한 gradient 가 이긴다. throughput: 같은 $C = 8$ 은 목표 loss 까지 19.48 시간으로 가장 느린 baseline 이다 [TNT Table 1]. [TNT § 3] 의 “작은 chunk 훈련은 peak FLOPs 5-10% 미만” (Zhang, Bi et al. 2025 = LaCT) 의 이유가 식 (10-2) 다 — FLOPs 가 아니라 intensity 부족이다. TNT 의 답은 두 knob 분리다: Stage 1 은 계층 memory(큰 C_g global + reset local) 로 훈련을 throughput-최적점에서 돌리고, Stage 2(추가 약 5-8% compute, 4-local 약 8.3% [TNT Table 4]) 가 chunk-1 decode 를 품질-최적점으로 만든다. 결과는 서로 다른 구성의 두 최고치 — 최대 $17.37\times$ 빠른 목표-loss 도 달 ({64} [TNT Table 1]) 과 다중-local Stage 2({2,4,8,16}) 의 평균 ppl 23.09(best Titans 25.07 대비 [TNT Table 2]).

[평가] stale-snapshot 근사 오차의 정량적 한계는 6 편어디에도 없다. 품질-최적 C 가 작다는 것도 mismatch 가 위험하다는 것도전부 경험적 관찰이라 이 knob 엔 아직 “numerics 문서” 가 없다.

10.6 Worked micro-example: chunk 하나의 arithmetic intensity 손계산

식 (10-2) 를 숫자로 확인한다 (모든 값 2 의 거듭제곱): TTT-Linear 형 matrix memory, $d = 64$, bf16, H100 급 1 장 — bf16 peak 989 TFLOP/s/HBM3 3.35 TB/s [NVIDIA H100 SXM datasheet] 를 1000 TFLOP/s 로 근사, ridge ≈ 300 FLOP/byte(장비값으로 교체 가능).

Step 1 — token 1 개 FLOPs. delta rule 한 step = forward $Wk_t(2d^2)$ + update W — $\eta_t(Wk_t - v_t)k_t^\top(2d^2)$ + read $Wq_t(2d^2) = 6d^2 = 6 \times 4096 = 24,576$ FLOPs.

Step 2 — $C = 1$ (decode). state RMW $4d^2 = 16,384$ + token IO $8d = 512$ = 합계 $\approx 16,896$ bytes. $AI(1) = 24,576/16,896 \approx 1.5$ FLOP/byte — 식 (10-2) 의 $1.5C$ 와 일치. 달성 성능 $3.35 \text{ TB/s} \times 1.5 \approx 5 \text{ TFLOP/s}$ = **peak 약 0.5%**. [TNT § 3] 의 “5-10% 미만” 이 손끝에서재현된다 (실제 구현은 head·batch 축으로 살피워 이보다 낮다).

Step 3 — $C = 64$ 의 AI. FLOPs = intra-chunk $4C^2d$ + state $6Cd^2 = 4 \cdot 262,144 + 6 \cdot 262,144 = 2,621,440$ ($C = d = 64$ 이라도 항이 같은 크기 — intra-chunk 를 빠뜨리면 안 되는 이유). 트래픽 $4d^2 + 8Cd = 16,384 + 32,768 = 49,152$ bytes. $AI(64) \approx 53$ FLOP/byte — $C = 1$ 대비 약 $36\times$, 성능 상한 $3.35 \times 53 \approx 178 \text{ TFLOP/s} \approx \text{peak } 18\%$ 다.

Step 4 — 큰 C . AI 는 큰 C 에서 $\approx 0.5C$ 로 계속 자라 (bandwidth 천장 없음) $d = 64$ 에서 ridge(≈ 300) 를 넘기려면 $C \approx 600$ 이 필요한데, 그만한 C 는 gradient 를 파괴적으로 stale 하게 만든다 [TNT Fig. 2]. MFU 를 얻는 현실적 경로는 C 가 아니라 **다른축** (head·request 병합, shard 를 batch 축에 쌓기) 이다.

Step 5 — 교훈. $1 \rightarrow 64$ 의 intensity 이득은 공짜가 아니다: (M4) gradient 가 그만큼 stale 해져 품질이 훈련 C 에 과적합된다 [TNT Fig. 2]. 이 계산이 “왜 chunkwise 인가, 왜 아픈가, 왜 TNT 인가” 를 요약한다. 숫자는 설명용이며 논문 측정치가 아니다.

10.7 Glossary: Part II 에 등장하는 training 용어의 systems 한 줄사전

Part II 에서 낯선 용어를 만나면 이 표에서 찾는다. 정의 소유 장을 병기했다— gloss 는 환기용 한 줄이지 정의가 아니다.

표 10-4 — training 용어 → systems 한 줄 gloss

용어	장	systems 한 줄 gloss
backward pass	2	역방향 VJP 사슬; FLOPs $\approx$ forward $2\times$
gradient ($\nabla_W \ell$)	2	오차 감소 방향; (오차) $k^\top$ write
inner ℓ / outer $\mathcal{L}$ loss	2	write / pretraining 목적함수
learning rate (η, η_t)	2	update 크기; inner 에션 write 강도 gate
momentum (S_t)	2	gradient EMA; session state
weight decay / retention gate (α_t)	2, 13	매 step state $\times \alpha_t$; 학습된 eviction
optimizer state (m_t, h_t)	2	step 간 살아남는 buffer; 훈련 register file
AdamW	2	좌표별 lr + decoupled decay
Muon / NS $_{\kappa}$	2, 14	update 행렬 semi-orthogonalization; κ 회 소형 GEMM
mini-batch	2	GEMM 살피우는 묶음; 여기선 sequence chunk
online learning / OGD	3	도착 즉시 predict-loss-update; decode 와 동형
regret	3	최적 고정 상태 대비 누적 손실 격차
FTRL	3	과거 loss 합 + regularizer argmin; retention 의 모체
inner / outer loop	1, 4	write 루프 / pretraining 루프 (W vs Θ)
bilevel optimization	4	inner 를 관통해 outer 최적화
meta-learning / MAML / W_{init}	4, 15	초기 상태를 outer 가 학습; TNT reset 의 착지점
unrolling / hypergradient	4	inner 궤적 펼쳐 미분; fuse 가능 static dataflow
associative memory	5	$k \rightarrow v$ 매핑 저장 장치
Hebbian write / outer product	5	$W += v_t k_t^\top$; append $\rightarrow$ crosstalk
delta rule	5	오차만큼만 overwrite; RMW 의 원형
crosstalk / capacity	5, 14	비직교 key 간섭/ 저장 쌍 수
fast / slow weights	6	움직이는 W / 고정 Θ ; batching 경계
linear attention	6	KV cache 를 $d \times d$ 로 압축한 극한

용어	장	systems 한 줄 gloss
DeltaNet / Gated DeltaNet	6	delta rule 의 layer 화; Householder transition
SSD (state-space duality)	7	SSM $\equiv$ masked linear attention; scan·GEMM 등가
TTT layer / dual form	8	state = 작은 net weights, update = GD step / 그 GEMM 형
chunkwise-parallel training	9	chunk 안 병렬 + chunk 간 순 차 (state 전달)
stale-snapshot 근사	9	gradient anchor 를 chunk 시작 $W_{\xi(t,C)}$ 에 고정
semantic hyperparameter C	9	C 가 함수를 바꿈; tile(exact) 과의 차이
WY / UT transform	9	rank-1 사슬 $\rightarrow$ chunk 당 GEMM 2 개; exact
associative scan	7, 9	log-depth prefix; momentum· Π_t 누적
surprise (momentary / past)	12	inner gradient / momentum buffer [Titans]
deep memory	12	state 가 2-layer residual MLP weights
persistent memory	12	입력 앞 학습된 token; Θ 의 일부
MAC / MAG / MAL	12	memory·attention 결합 topology 세 종
attentional bias	13	inner objective 의 family – “무엇을 loss 로”
Omega rule / window gate	14	최근 c 쌍 window objective / 쌍별 가중 gate
$\gamma_{t,i}$ test-time memorization	14	[Atlas] 재명명: learning 아닌 문맥 기억
chunk-size mismatch	15	train $C \neq$ serve C 면 품질 붕괴 [TNT Fig. 2]
hierarchical memory (global/local)	15	순차 global(큰 C_g) + reset 병렬 local
periodic reset / context parallelism	15	L_s 마다 W_{init} 복귀 $\rightarrow$ chain 절단 $\rightarrow$ shard 병렬
Q-K projection (Π_t)	15	key subspace 로 사영하는 $d \times d$ 보조 state
two-stage training	15	큰 chunk pretrain(throughput) + 작 은 chunk finetune(품질)
update frequency / level	16	컴포넌트별 갱신 주기; cache 계층의 시간축 유비
Continuum Memory System (CMS)	16	갱신 주기 스펙트럼 위의 memory 사슬
Local Surprise Signal (LSS)	16	layer 출력의 국소 오차 신호 u_t
self-modifying Titans	16	자기 update rule 을 스스로 생성하는 memory

용어	장	systems 한 줄 gloss
catastrophic forgetting	11	새 SGD 가 옛 매핑 파괴 (crosstalk 의 혼련판)
complementary learning systems (CLS)	11	빠른 hippocampus / 느린 cortex 이중계
consolidation	11, 16, 17	fast state $\rightarrow$ slow weights; background compaction
knowledge distillation / GKD	11	teacher 분포로 student 혼련; GKD 는 on-policy
replay	11	과거 데이터 재주입으로 forgetting 완화
wake/sleep lifecycle	17	온라인 서빙 / 주기적 offline 통합 phase 의 교대
Knowledge Seeding (KS/ SKS) / Dreaming	17	fast $\rightarrow$ slow distillation / RL 합성 커리큘럼
MoE / router	이 장	expert 부분망 일부 실행/ 선택 gate; [Sleep] parameter expansion 재료 ($\rightarrow$ 17 장)

요약

- decode 비용은 세 숫자다: state 원소 수 s , token 당 FLOPs, state RMW bytes($4s$). 고정-state decode 는 대체로 bandwidth-bound(예외: Atlas/Muon 의 NS_κ 는 compute-bound 가능) 이고, 그 영역에서 FLOPs 계수 차이 (4 vs 12) 는 공짜다 (표 10-2).
- memory 승부는 식 (10-1) $L^* = s/2d$: matrix $d/2$, 표준 deep memory $4d$, momentum 포함 $8d$ token 부터 이긴다 — 짧은 문맥에선 fast weights 가 진다.
- chunk C 는 roofline 의 독립변수다: AI 가 작은 C 에서 $\approx 1.5C$, intra-chunk $O(C^2d)$ 때 문에 큰 C 에서도 $\approx 0.5C$ 로 계속 자라 $C = L$ 의 compute-bound($O(L^2d)$) 로 이어진다 — bandwidth 천장 없음 (식 10-2).
- FlashAttention tiling 은 exact 재배열, (M4) 의 C 는 semantic knob — [TNT Fig. 2] 의 mismatch(13.78 vs 36.45) 가 실증. 경계는 선형성이 아니라 정의 방식이다: 명시적 linear recurrence 의 chunkwise 는 exact 지만 anchored mini-batch GD 로 정의된 TTT-Linear 는 linear 여도 semantic.
- 품질-최적 C (작음) 와 MFU-최적 C (큼) 는 반대 방향이며, TNT 는 계층 memory + 두 단계 혼련으로 분리해 (서로 다른 구성에서) 최대 $17.37\times$ 빠른 혼련과 개선된 perplexity 를 얻는다 [TNT Table 1, 2].
- backward-in-decode 요구 넷: closed-form gradient kernel, per-session state 라는 새 cache class(checkpoint/restore·prefix 분기 · 학습된 eviction), grouped-GEMM decode(batching 이 intensity 를 못 올림), optimizer-trajectory rollback.
- 6 편 모두 decode wall-clock 을 보고하지 않는다 — Part II 의 serving 주장은 이 장의 도구로 각자 재계산하는 것이 기본이다.

자가 점검체크리스트

- ☐ (M1) – (M4) 형태의 임의 layer 에 대해 decode FLOPs/token·state RMW bytes/token 을 지배항 수준에서 유도한다.
- ☐ 식 (10-1) 로 자기 서빙 config 의 KV-cache-vs-state crossover 길이를 계산한다.
- ☐ 식 (10-2) 로 $AI(C)$ 를 계산해 자기 hardware ridge point 와 비교, 어느 C 부터 compute-bound 인지 판정한다.

- tiling vs chunkwise 를 “exact 재배열 vs semantic 근사” 로 설명하고 어느클래스부터 적용되는지 말한다.
- backward 가 decode 에 들어올 때 serving engine 의 네 가지 요구를 나열한다.
- Part II 임의 장의 § “systems 합의” 를 표 10-2 행과 식 (10-1)·(10-2) 로 환원해 계산한다.

다음 장으로

여기까지의 cost model 은 전부 한 세션 · 한 inner loop 의 이야기였다: request 가 시작되면 W_t 가 움직이고 끝나면 버려진다. [NL] 과 [Sleep] 은 이를 확장한다 — update 주기를 token 부터 pretraining 까지 연속 스펙트럼으로 펼치고 (Continuum Memory System), 세션후 fast state 를 slow weights 로 옮기는 offline 단계 (sleep, consolidation) 를 lifecycle 에 넣는다. 이를 읽으려면 마지막 배경이 필요하다: update 가 왜 옛 지식을 부수는가 (catastrophic forgetting), 뇌는 왜 빠른/느린 기억계를 분리했는가 (CLS), 지식 이전의 표준도구 (distillation, replay, RL-lite). 11 장이 이 조각을 채워 Part II 준비를 끝낸다.

ch11. Continual learning, complementary learning systems, distillation, RL-lite

이 장의 목표 — 이 장을 마친 독자는 다음을 할 수 있다. 1. catastrophic forgetting 을 공유 weight 위의 gradient 간섭으로 정의하고, 2-parameter 예제에서 forgetting 이 일어나는 조건 (입력 겹침) 과 일어나지않는 조건 (orthogonality) 을 손으로 계산 할 수 있다. 2. replay-regularization(EWC)-parameter isolation 세 완화책 가족을 (state, update, cost) 객체로 비교하고, 새 기법을 만나면 어느 가족의 조합인지 분류할 수 있다. 3. complementary learning systems 의 fast/slow 분업 구도를 update-frequency spectrum 으로 확장해읽고, online consolidation 과 offline consolidation 을 구분해 [NL] 과 [Sleep] 이 각각 어느 반쪽을 지었는지 말할 수 있다. 4. Hinton KD, on-policy GKD, REINFORCE, ReST[△]EM 을 구분하고, REINFORCE 한 스텝을손으로 계산하며, [Sleep] 의 sleep phase 가 이 부품들의 어떤 조립인지지도 위에 놓을 수 있다.

왜 필요한가 — 이 장은 6 편 중 오직 [NL] (Nested Learning, arXiv:2512.24695) 과 [Sleep] (Language Models Need Sleep, arXiv:2606.03979) 두 편을 위해 존재한다. [NL § 1] 의 anterograde-amnesia framing 과 continual-learning 실험 (CLINC-CTNL, 16 장에서 상술), 그리고 [NL § 4.3] 의 “momentum 도 잇는다” 논증은 catastrophic forgetting 과 complementary learning systems 의 어휘 없이는 수사로만 읽힌다. [Sleep] 은 기제전체가 이 장의 재료로 조립된다: wake/sleep 구분과 online/offline consolidation 의 이분법 [Sleep § 3.1], catastrophic forgetting 을 capacity 문제로 재정의하고 parameter expansion 으로 답하는 [Sleep § 3.2], Knowledge Seeding = GKD 기반 on-policy distillation + RL 보상 [Sleep § 3.3], Dreaming = ReST[△]EM 기반 self-training [Sleep § 3.4]. 반대로 ch12 - ch15 의 네 논문에는 이 장이 필요 없다 — 그래서 이 장은 Part I 의 맨 끝, Part II 직전에 있다.

11.1 Catastrophic forgetting: 쓰기가 곧 파괴인세계

독자의 세계에는 forgetting 이 없다. serving 중인 모델의 weight 는 움직이지않고 (“weight update 없음” 이라는 추론 불변식, → 1 장), KV cache 는 append-only 여서 어제 넣은 entry 가 오늘의 write 때문에 변질되는 일이없다. 지식이 파괴되려면 누군가 명시적으로 지워야 한다. 이 라인이 그불변식을 폐기한 순간 (→ 8 장), 훈련 쪽 세계의 기본 병리 하나가 함께들어온다. 이 절은 그 병리를 정의한다.

catastrophic forgetting(CF) 은 task A 를 이미 학습한 network 를 새 task B 의 데이터 만으로 계속 훈련하면, task A 의 성능이 점진적이 아니라급격하게 — 종종 거의 전부 — 무너지는 현상이다 (McCloskey & Cohen 1989, Psychology of Learning and Motivation; French 1999, Trends in Cognitive Sciences). 메커니즘은 두 사실의 합이다. 첫째, neural network 의 지식은 국소화되어 있지 않다. 하나의 mapping 이 수많은 weight 에분산 저장되고, 하나의 weight 가 수많은 mapping 에 참여한다. 둘째, gradient descent 는 현재 objective 만 본다.

$\Theta \leftarrow \Theta - \eta \nabla_{\Theta} \mathcal{L}_B(\Theta)$ 의 어디에도 “task A의 loss를 지켜라”는 항이 없다. 그래서 B를 위한 update가 A가 쓰던 좌표를 지나가면, A의 함수값은 A의 데이터를 한 번도 다시 보지 않은 채로 바뀐다. 파괴는 부작용이 아니라 update rule의 정의 그 자체다.

이 구조는 이미 본 것이다. 5장의 crosstalk은 하나의 matrix memory에 여러 (k, v) 쌍을 겹쳐 쓸 때 모든 read가 점진적으로 오염되는 현상이었다 (→ 5장). catastrophic forgetting은 같은 병리의 시간축 확대판이다: crosstalk이 한 시퀀스 안에서 write 들끼리 간섭하는 것이라면, CF는 모델의 배포 수명 동안 task 들끼리 간섭하는 것이다. 뿌리가 같으므로 (“겹치는 표현+ 파괴적 write”) 해법의 문법도 같다 — 간섭을 피하도록 좌표를 분리하거나 (isolation), 지킬 것을 명시하거나 (regularization·retention), 옛것을 다시 써넣거나 (replay). 이 세 가족이 § 11.2의 전부다.

간섭의 양은 기하가 정한다. linear 모델 $y = \Theta^T x$ 에서 task B의 gradient는 B의 입력 방향으로만 뻗으므로, A와 B의 입력이 orthogonal하면 B의 학습은 A가 쓰는 좌표를 건드리지 않고, 겹치면 겹친 만큼 A를 파괴한다 — § 11.7에서 숫자로 확인한다. deep network에서는 표현이 학습되면서 겹침 자체가 움직이므로 이렇게 깔끔하게 쪼개지지 않지만, “forgetting = gradient 부분 공간의 겹침”이라는 1차 근사는 이 라인의 논증을 읽는 데 충분하다.

[NL]은 여기에 한 겹을 더 얹는다: 잊는 것은 weight만이 아니다. 2장에서 momentum을 “gradient들의 EMA를 유지하는 상태 객체”로 배웠다. [NL § 4.3]은 decay $\beta = 0.9$ 의 momentum이 최근 gradient에 기여가 물리는 low-pass filter임을 지적한다: 정상 상태에서 최근 n 개의 비중은 $1 - \beta^n$ 이라 50%를 넘으려면 7개, 99%를 넘으려면 44개가 필요하다 (원문은 6/43으로 적었으나 $1 - 0.9^6 = 46.9\%$, $1 - 0.9^{43} = 98.9\%$ 로 각각 임계값에 살짝 못미친다). orthogonal한 task가 이어지는 continual learning에서 optimizer는 옛 task의 gradient 부분공간을 잊는데, [NL § 4.3]은 이것을 모델 capacity의 실패가 아니라 optimizer의 memory 관리 실패로 규정한다 (NL의 논지). 단 이 optimizer-state의 망각이 곧 task 성능의 CF인 것은 아니다 — § 11.7(a)에서 보듯 순수 선형·orthogonal 예제에서는 이전 예측이 $x_A^T \Delta \Theta = 0$ 으로 불변이라 task는 잊히지 않는다. task-level CF가 성립하려면 학습된 표현의 이동, 비직교 gradient 성분, task 간 Hessian 결합 같은 추가 조건이 필요하다. 즉 forgetting은 특정모델의 결합이 아니라 **모든 EMA 류 상태의 공통 성질**이다 — 2장에서 optimizer를 상태 객체로 배운 것이 여기서 처음 배당을 지급한다. [Sleep § 1]은 같은 문제를 배포 관점의 딜레마로 요약한다: 갱신하지 않으면 지식이 낡고, 갱신하면 CF가 온다.

systems 어휘로 옮기면 CF는 **policy 없는 eviction**이다. cache의 eviction은 무엇을 버릴지 명시적 정책 (LRU, LFU)이 고르지만, 공유 weight 위의 gradient update는 “새 데이터의 loss를 줄이는 방향”이라는 단일기준으로 낡은 entry를 암묵적으로 덮어쓴다 — 무엇이 지워졌는지 로그도 남지 않는다. retention gate (→ 13장)는 이 eviction을 학습된 정책으로 바꾸지만, 여전히 무엇을 파괴할지 고르는 정책이다. 파괴 없이 새 지식을 넣으려면 저장소가 자라야 한다. 이 직관이 [Sleep § 3.2]의 재정의 — CF는 근본적으로 capacity 문제이며, parameter가 덮어써지는 이유는 새 지식을 넣을 자리가 없기 때문이라는 — 로 곧장 이어진다.

11.2 고전 완화책 3종: replay, EWC, parameter isolation

30년치 continual-learning 문헌은 세 가족으로 압축된다. 이 책의 관행대로 각 가족을 (state, update, cost)를 갖는 객체로 제시한다 (→ 2장). 이 분류의 실용적 가치는 [NL]·[Sleep]의 기제를 만났을 때 “새발명”과 “고전의 재조합”을 구분하게 해 준다는 데 있다.

replay는 옛 데이터의 일부를 buffer에 보관했다가 새 task의 배치에 섞어 넣는 것이다. update rule은 바뀌지 않고 데이터 분포만 교정된다: 매 스텝의 gradient가 옛 task의 loss 항을 계속 포함하므로, § 11.1의 “A를 지켜라는 항이 없다”는 문제를 데이터 쪽에서 해결한다. state는 buffer 자체 (모델 밖의 byte), update는 배치 구성 규칙, cost는 저장 용량 + 옛 샘플만큼의 추가 FLOP + 원본 데이터를 계속 보관하는 데 따르는 거버넌스 부담이다. 변형으로, buffer 대신 **생성된 데이터로 하는 replay** (pseudo-rehearsal; French 1999가 정리한 계보)가 있다: 모델 자신이나 생성기가 옛 지식을 표본화해 그것으로 되새김한다. [Sleep]의 sleep phase가 teacher(자기 자신의 이전 버전)에게서 corpus를 표본화해 consolidation에 쓰는 것은 정확히 이 계보다 — 원본 데이터가 없어도 되는 replay (→ 17장).

EWC(elastic weight consolidation; Kirkpatrick et al. 2017, PNAS, arXiv:1612.00796) 는 regularization 가족의 대표다. task A 를 끝낸 시점의 parameter Θ^{*A} 를 닳으로 삼고, task B 의 loss 에 좌표별 가중이차 penalty 를 더한다:

$$\mathcal{L}_{\text{EWC}}(\Theta) = \mathcal{L}_B(\Theta) + \sum_i \frac{\lambda_{\text{EWC}}}{2} F_i (\Theta_i - \Theta_i^{*A})^2. \quad (11-1)$$

여기서 F_i 는 task A 에서의 diagonal **Fisher information** — “좌표 i 가 task A 의 예측에 얼마나 하중을 받는가” 의 국소 추정 — 이고, λ_{EWC} 는 옛 task 보존과 새 task 학습의 트레이드오프계수다 (둘 다 장-국소 기호). 직관은 단순하다: A 에 하중이 큰 좌표는 못움직이게 잡아 두고, update 를 A 가 쓰지 않는 좌표로 흘려보낸다. state 는 parameter 크기의 buffer 두 개 (Θ^{*A} 와 F) — 2 장에서 Adam 이 이미 m_t, h_t 두 buffer 를 갖고 다니는 것을 본 독자에게 익숙한 자원 등급이다. cost 는 FLOP 으로는 미미하고, memory 로는 —diagonal 이차 penalty 의 합은 하나의 이차식으로 합쳐지므로 (누적 precision + precision-가중 닳) 기본 구현은 $O(P)$ state 이고 task 별 posterior 를 따로 보존하는 변형에서만 $O(TP)$ 로 자란다 — 그리고 더 근본적으로는 F 가 국소 이차 근사일 뿐이라 task 가 여럿 쌓이면 닳들이 서로충돌한다는 것이다. 식 (11-1) 을 이 책의 표기로 다시 읽으면 정체가드러난다: weight decay 가 원점으로 당기는 무차별 retention 이라면 ($\rightarrow$ 2 장), EWC 는 옛 최적점으로 당기는 **선택적 retention** 이다. Miras 가 retention gate 를 “무엇을 남길지의 regularizer” 로 재이론화하는 것 ($\rightarrow$ 13 장) 과 같은 문법 위에 있다.

parameter isolation 은 간섭을 원천 차단한다: 옛 task 의 parameter 를 동결하고, 새 task 에는 새 capacity(새 모듈, mask, adapter) 를배정한다. state 는 task 별 parameter 구획과 routing 정보, update 는 “새구획만 훈련”, cost 는 task 수에 비례해 자라는 parameter 와 어느 구획을쓸지 고르는 routing 이다. 동결된 구획은 구조적으로 파괴 불가능하므로 CF 가정의상 없다 — 대신 문제가 “무엇을 지킬까” 에서 “성장을 어떻게 관리할까” 로이동한다. [Sleep § 3.2] 의 parameter expansion 은 정확히 이 가족이다: consolidation 때마다 저주파 블록에 low-rank expert 하나를 새로활성화하고, 전이되는 지식은 그 새 expert 에만 쓰며 (나머지 전부 동결 —구조적 CF 방지), 모든 expert 를 초기화 시점에 미리 할당해 두고 mask 만벗기는 구현으로 tensor shape 을 고정한다. 성장 관리는 빠른 (source) 블록의옛 low-rank expert 를 주기적으로 reset 해 그 capacity 를 재활용하는 것으로**부분적으로** 답한다 (synaptic-pruning reset, $\rightarrow$ 17 장). 단 이 reset 은 지식을 넘긴 고주파 source 의 임시 parameter 만 회수할 뿐, 지식을받는 저주파 target 에는 consolidation 마다 새 expert 가 남으므로 전체성장 (특히 최저주파 블록의 누적) 은 bound 하지 못한다 — 사전 할당 mask 도 tensor shape 만 고정하지 유한 pool 의 고갈을 없애지 않는다.

표 11-1 — 세 완화책 가족의 (state, update, cost) 와 이 라인에서의 화신

가족	state	update	cost	대표 실패 모드	이 라인에서의 화신
replay	옛 데이터 buffer (모델 밖 byte)	새 배치에 옛 샘플 혼합	저장+ 추가 FLOP + 데이터 거버넌스	buffer 가 옛 분포를 대표하지 못함	[Sleep] 의 teacher-표본 corpus (생성형 replay, $\rightarrow$ 17 장)
regularization (EWC)	누적 Fisher + 결합 닳 (기본 $O(P)$; task 별 보존시 $O(TP)$)	loss 에 이차 penalty 추가, 식 (11-1)	memory (기본 $O(P)$), 근사 오차	국소 근사 붕괴, 닳 간 충돌, capacity 경합	retention gate 의 사춘 (선택적 retention, $\rightarrow$ 13 장)

가족	state	update	cost	대표 실패 모드	이 라인에서의 화신
parameter isolation	task 별 구획 · mask · adapter	옛 구획 동결, 새 구획만 훈련	parameter 성장 + routing	무한 성장 (pruning 없 이는)	[Sleep § 3.2] low-rank expert expansion + reset (→ 17 장)

systems 접점을 세 줄로 요약한다. replay buffer 는 **data-plane artifact** 다 — 모델 파일이 아니라 저장 · 표본화대역폭 · 보존 정책을 갖는 데이터 자산이며, fleet 설계에서별도 수명주기를 가진다. EWC 의 buffer 는 optimizer state 와 같은 자원등급이므로 “훈련 상태 = 추가 parameter 몇 벌” 이라는 2 장의 회계에 그대로편입된다. parameter isolation 의 서빙 형태를 독자는 이미 운영하고 있다: per-tenant LoRA adapter 를 base weight 위에 얹어 서빙하는 multi-LoRA 스택이 바로 그것이다 — continual learning 의 격리 전략과 serving 의 격리전략이 같은 물건임을 [Sleep] 이 이용한다.

11.3 Complementary learning systems: 두 학습자의분업

complementary learning systems(CLS) 는 하나의 학습 시스템이 “빠른 획득” 과 “안정된 보존” 을 동시에 잘할 수 없으므로, 뇌가 학습을 두시스템으로 분업시켰다는 이론이다 (McClelland, McNaughton & O’ Reilly 1995, Psychological Review). hippocampus 는 빠른 학습자다: 새경험을 한 번에, 서로 겹치지 않는 sparse 한 표현으로 즉시 기록한다. neocortex 는 느린 학습자다: 겹치는 분산 표현 위에서 구조화된 일반 지식을축적하며, 그래서 갑작스러운 write 에 취약하다 — § 11.1 의 CF 가 바로 겹치는표현 + 빠른 write 의 조합에서 왔음을 상기하라. 분업의 핵심 고리는 전이다: hippocampus 에 임시 저장된 새 기억이 수면 · 휴식 중 **replay** 되어 옛 지식과 섞여 (interleave) neocortex 에 천천히통합된다. 즉 CLS 의 처방은 § 11.2 의 세 가족 중 두 개의 합성이다 — 빠른쪽과 느린 쪽을 분리 (isolation) 하고, 둘 사이를 replay 로 연결한다.

이 이론이 없으면 [NL] 과 [Sleep] 의 신경과학 수사는 장식으로읽힌다. [NL § 1] 은 배포된 LLM 을 anterograde amnesia — 새 장기 기억을만들지 못해 영원한 현재를 사는 환자 — 에 비유하는데, CLS 어휘로 옮기면정확한 진단이 된다: context window(즉시, 세션이 끝나면 소멸) 와 동결된 weight(pre-training 시점에 고정) 사이에 hippocampus → neocortex 전이에해당하는 경로가 없다. [NL § 1] 은 또한 뇌가 brain-wave 주파수별로 여러시간 척도에서 기억을 굳히는 것을 들어 “deep network 의 update 주파수가 ∞ (attention state) 와 0(동결 MLP) 둘뿐” 임을 문제 삼는다 — 이관찰이 § 11.4 의 spectrum 으로 이어진다. [Sleep § 1] 은 한 단계 더 내려가수면의 두 단계를 각각 계산 절차로 번역한다: NREM(느린 파형) 수면의 hippocampus → neocortex consolidation 과 synaptic homeostasis(전역 downscaling) 는 Memory Consolidation 단계로, REM 수면의통합 · 시뮬레이션 (꿈) 은 Dreaming 단계로. 두 논문에서신경과학은 동기이지 메커니즘이 아니다 — 검증되는 것은 계산 절차 쪽이다.

표 11-2 — CLS ↔ 이 라인의 대응

CLS (생물학)	이 라인의 대응물	정의 장
hippocampus (빠른 학습자, sparse, 즉시 기록)	fast weights W_t , 고주파 블록	→ 6·8 장, 16 장
neocortex (느린 학습자, 분산 표현, 구조화)	slow weights Θ , 저주파 블록	→ 2 장, 16 장
수면 중 interleaved replay	self-generated data 로 하는 offline consolidation	§ 11.4, → 17 장

CLS (생물학)	이 라인의 대응물	정의 장
synaptic homeostasis / pruning	빠른 블록의 low-rank expert reset	→ 17 장
brain-wave 주파수 사다리	CMS 의 update-frequency 사다리	→ 16 장

systems 관점에서 CLS 는 신경과학이기 이전에 낮은 아키텍처 패턴이다: write 에 최적화된 작은 front store 와 read 에 최적화된 큰 back store 를두고, 주기적 background 작업이 앞에서 뒤로 데이터를 재조직하며 옮긴다. 독자가 storage 스택에서 매일 보는 구도이며, § 11.8 에서 이 유비를 정색하고편다.

11.4 Online vs offline consolidation, 그리고 update-frequency spectrum

지금까지 이 책은 두 개의 시간 척도만 썼다: token 마다 움직이는 fast weights W_t (→ 6·8 장) 와, pre-training 이 끝나면 동결되는 slow weights Θ (→ 2 장). CLS 의 두 학습자와 정확히 포개지는이분법이다. [NL] 의 첫 수는 이 이분법을 **spectrum** 으로 바꾸는것이다: 배포된 Transformer 에서 attention 의 state 는 매 token 다시만들어지므로 update frequency 가 사실상 ∞ 이고, MLP 는 0 이다 — 그사이 가 통째로 비어 있다 [NL § 1]. 그 사이를 채우는 것, 즉 “1K token 마다” , “10K token 마다” 갱신되는 중간 주파수의 memory 블록 사다리를두는 것이 CMS 다 (정식 정의 와 update rule 식 (M5) 는 → 16 장; [Sleep] 의실험은 주기 1K → 5K → 10K 같은 사다리를 쓴다 [Sleep Fig. 7]). update 주기가 곧 기억의 수명 등급이 된다: 고주파 블록은 최근 문맥의 단기 기억, 저주파 블록은 장기 기억.

이 spectrum 위에서 이 장이 소유하는 구분 하나를 정의한다. **online consolidation** 은 모델이 입력을 받는 동안 (wake), 상시 학습 과정 자체를통해 고주파 블록의 지식이 저주파 블록으로 전이되는 것이다 — [NL] 의 CMS 와 Hope 가 하는 일이며, backprop 이 층위를 관통해 흐르는 end-to-end 갱신의 부산물로 전이가 일어난다. **offline consolidation** 은 입력이멈춘 기간에, self-generated data 를 매개로 지식을 명시적으로전이 · 추상화하고 필요하면 capacity 를 늘리는 것이다 — [Sleep] 의 sleep phase 가 하는 일이다. 이 이분법 자체는 [Sleep § 3.1] 이 wake/sleep lifecycle 을 정의하면서 정식화했다 (그 lifecycle 의 상세는 → 17 장). [Sleep § 1] 은 online 만으로 부족한 이유를세 가지로 든다: 전이가 같은 추상화 수준에서 일어나 lossy 압축이 없으므로 capacity 를 그대로 소모하고, 능동적으로 retrieval 되는 기억만 강화되는선택적 과정이며, 현재 context 에 갇혀 새 지식과 기존 지식의 상위 수준통합을 놓친다.

spectrum 은 CF 를 없애지 않고 유예한다. 저주파 블록도 언젠가는 갱신되고, 그 갱신은 여전히 파괴적 write 다. [Sleep § 3.2] 의 관찰이 이 장 전체의매듭이다: 주기 1K 인 블록 뒤에 주기 10K 인 블록이 있으면, 느린 블록이 한번 갱신되기 전에 빠른 블록의 지식이 10 번 그 안으로 consolidate 되어야하고, 고정 크기 저장소에 대한 이 반복 write 가 정확히 CF 가 터지는지점이다. 그래서 consolidation 은 **덮어쓰기 직전에 fire** 해야하고 ([Sleep § 3.2] 의 스케줄: 각 블록의 update 경계마다), 반복 write 를받는 쪽은 capacity 가 자라야 한다 — § 11.2 의 isolation 가죽이 다시등장하는 이유다.

systems 접점: update frequency 는 memory 계층 배치의 언어로 즉시번역된다. 매 token 갱신되는 상태는 연산 유닛가까이 (register/SRAM 감각의 자리) 에 있어야 하고, 1K token 마다 갱신되는 상태는 write 트래픽이 3 자릿수 낮으므로 더 멀고 싼계층에 두어도 된다. 주기 $C^{(\ell)}$ 이 곧 그 블록의 **write-traffic 예산**이다: parameter 크기 P byte 인 블록의 상각 write 대역폭은 $P/C^{(\ell)}$ byte/token 으로, 사다리의 각단이 자기 자리를 스스로 말해 준다. 이 계산은 Part II 에서 [NL]·[Sleep] 의 serving 함의를 논할 때의 표준 도구가된다 (→ 16·17 장).

11.5 Knowledge distillation: soft target 에서 on-policy GKD 까지

offline consolidation 의 실행 수단이 필요하다. 지식을 한 parameter 덩어리에서 다른 덩어리로 “옮긴다” 는 것은 물리적 복사가 아니다 — 두 덩어리는 shape 부터 다르다. 옮길 수 있는 것은 행동 뿐이다. **knowledge distillation**(KD) 은 teacher 모델의 출력 분포를 target 으로 삼아 student 모델을 훈련하는 기법이다 (Hinton, Vinyals & Dean 2015, arXiv:1503.02531). LM 이라면 위치 t 마다 teacher 의 다음-token 분포 $p^T(\cdot | y_{<t}, x)$ 를 통째로 맞추게 한다:

$$\mathcal{L}_{\text{KD}} = \mathbb{E}_{(x,y) \sim \mathcal{D}} \sum_t \text{KL}(p^T(\cdot | y_{<t}, x) \parallel p^S(\cdot | y_{<t}, x)). \quad (11-2)$$

hard label 대비 이득은 분포의 꼬리에 있다: teacher 가 세 후보에 (0.7, 0.2, 0.1) 을 주면, 정답 하나만 남기는 hard label 이 버리는 “오답들 사이의 2:1 비율” 까지 student 가 물려받는다 — Hinton 이 dark knowledge 라고 부른 정보다. temperature 로 분포를 녹여 꼬리를 키우는 변형이 표준이지만, 이 라인을 읽는 데는 “teacher 분포 전체가 target” 이라는 뼈대면 충분하다.

식 (11-2) 의 급소는 기대값이 걸리는 분포 $\mathcal{D}$ 다. teacher 가 만든 (또는 원본 corpus 의) 시퀀스 위에서만 KL 을 재면, student 는 자기가 생성할 때 실제로 방문하는 상태에서 한 번도 교정받지 못한다. autoregressive 생성은 오차가 누적되는 루프라서, 훈련분포 (teacher 의 궤적) 와 서빙 분포 (student 자신의 궤적) 의 괴리가 성능을 갉아먹는다. 독자의 어휘로 정확히 옮겨진다: 합성 벤치 트래픽으로만 검증한 시스템이 live 트래픽에서 처음 보는 상태를 만나는 문제다. off-policy 훈련/on-policy 서빙의 불일치라는 이 진단에서 **on-policy distillation** 이 나온다: student 가 직접 생성한 시퀀스 위에서 teacher 가 token-level 로 채점한다.

GKD(generalized knowledge distillation; Agarwal et al., On-Policy Distillation of Language Models, ICLR 2024, arXiv:2306.13649) 는 두 극단을 하나의 목적함수로 섞는다:

$$\mathcal{L}_{\text{GKD}} = (1 - \lambda_{\text{on}}) \mathbb{E}_{(x,y) \sim \mathcal{D}} [\mathcal{F}(p^T \parallel p^S)(y | x)] + \lambda_{\text{on}} \mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim p^S(\cdot | x)} [\mathcal{F}(p^T \parallel p^S)(y | x)]. \quad (11-3)$$

$\lambda_{\text{on}} \in [0, 1]$ 은 student 자신의 rollout 이 차지하는 비율이고 (이 책의 통일 기호; 원문들의 λ), $\mathcal{F}$ 는 teacher/student token 분포 사이의 divergence 다. $\mathcal{F}$ 의 선택은 자유 축이다: forward KL $\text{KL}(p^T \parallel p^S)$ 은 teacher 가 질량을 둔 곳을 student 가 모두 덮게 만들고 (mode-covering), reverse KL $\text{KL}(p^S \parallel p^T)$ 은 teacher 질량이 희박한 곳에 student 가 질량을 두는 것을 강하게 벌해 소수 mode 에 집중하게 만든다 (mode-seeking). 실무 관행 하나가 중요하다: 둘째 항에서 student 의 표본화 분포 자체로는 backprop 하지 않는다 — 표본은 상수 취급하고 그 위의 divergence 만 미분한다. 분산과 불안정성을 피하는 표준 선택이며, [Sleep § 3.3] 도 이 관행을 그대로 계승한다.

[Sleep] 이 이 위에 없는 뒤틀림 하나만 예고한다. KD 의 통상 방향은 큰 teacher $\rightarrow$ 작은 student 의 압축이다. [Sleep § 3.3] 의 Knowledge Seeding 은 방향을 뒤집는다: 작은 teacher(갱신 직전의 고주파 블록을 품은, 확장 전의 자기 자신) 가 더 큰 student(low-rank expert 가 새로 열린 확장후의 자기 자신) 에게 가르친다 — 그래서 별칭이 upward distillation 이다. 외부 corpus 없이 teacher 에게서 표본화한 $\mathcal{D}$ 로 식 (11-3) 형 objective 를 돌리고, student 쪽은 새 expert 만 훈련 가능하다. 상세와 정식정의는 $\rightarrow$ 17 장.

systems 접점: distillation job 의 자원 프로파일은 훈련보다 추론에 가깝다. 지배 비용은 corpus 생성 (teacher 표본화) 과 on-policy rollout(student 생성) — 즉 decode-heavy, 독자의 fleet 가 가장 잘하는 일 — 이고, backward 는 student(Sleep 에서는 low-rank expert 하나) 에만 걸린다. “데이터셋을 모델이 만든다” 는 이 구도가 § 11.8 의 lifecycle 그림에서 핵심부품이 된다.

11.6 RL-lite: REINFORCE, imitation, 그리고 post-training 의 모양

distillation 은 target 분포가 있을 때의 이야기다. [Sleep] 의 Dreaming 은 target 분포가 없는 신호 — “이 합성 데이터로 fine-tune 했더니 벤치마크가 올랐는가” 같은, 시퀀스가 끝난 뒤 도착하

는 스칼라 — 로 생성정책을 개선해야 한다. 이것이 RL 이 필요한 전부이므로, 이 절은 RL 을엔지니어 컷으로만 자른다: 추정기 하나, 구분 하나, 조립 패턴 하나.

어휘부터. **policy** 는 상태를 받아 행동의 분포를 내는 함수다 — LM 이 곧 policy 다: $\pi_{\Theta}(y|x)$ 는 prompt x 에서 시퀀스 y 를 생성할 확률이고, 독자가 매일 돌리는 sampler 가 policy 의 실행이다. **reward** $r(y)$ 는 완성된 시퀀스에 대해 환경이 주는 스칼라다. 요점은 r 이 미분 가능할 필요도, 모델일 필요도 없다는 것: 채점기, Levenshtein distance, “정답 여부” 모두 된다. 목표는 기대 reward $J(\Theta) = \mathbb{E}_{y \sim \pi_{\Theta}}[r(y)]$ 의 최대화인데, r 을 미분할 수 없으니 gradient 를 어떻게 얻는가가 유일한 기술적 문제다.

REINFORCE(Williams 1992, Machine Learning) 가 답이다. 항등식 $\nabla_{\Theta} \pi_{\Theta} = \pi_{\Theta} \nabla_{\Theta} \log \pi_{\Theta}$ 를 기대값 안에 넣으면

$$\nabla_{\Theta} J(\Theta) = \mathbb{E}_{y \sim \pi_{\Theta}(\cdot|x)} \left[(r(y) - \bar{r}) \nabla_{\Theta} \log \pi_{\Theta}(y|x) \right], \quad \log \pi_{\Theta}(y|x) = \sum_t \log \pi_{\Theta}(y_t | y_{<t}, x). \quad (11-4)$$

reward 의 gradient 는 등장하지 않는다 — 표본화한 시퀀스의 log-likelihood gradient 에 reward 를 곱해 평균할 뿐이다. baseline $\bar{r}$ (보통 표본평균 reward; 장-국소 기호) 은 기대값을 바꾸지 않으면서 분산만 줄이는 상수다. 구현 관점의 번역이 이 절의 요지다: 식 (11-4) 는 **reward 로 가중된 SFT gradient** 다. kernel 수준에서 하는 일은 cross-entropy 훈련과 동일하고, 다른 것은 데이터가 자기 생성물이라는 것과 시퀀스별 가중치가 $(r - \bar{r})$ 이라는 것뿐이다. § 11.7 에서 두 행동 짜리 예제로 한 스텝을 손으로 돌린다.

구분 하나: **imitation learning** 은 시연자의 행동 분포를 맞추는 것 (target 분포가 있음 — SFT 와 KD 가 이 가족), RL 은 스칼라 reward 를 최대화하는 것 (target 분포가 없음) 이다. 피드백의 밀도로 다시 읽으면 스펙트럼이 된다: KD 는 위치마다 분포 전체 (가장 진한 피드백), on-policy GKD 는 자기 궤적 위에서 위치마다 분포, REINFORCE 는 시퀀스 전체에 스칼라하나 (가장 희박한 피드백) 다. [Sleep § 3.3] 의 Learning to Imitate 는 이름 그대로 이 사이에 있다: distillation 으로 지식을 넣은 student 에게 teacher 시퀀스의 prefix 를 주고 나머지를 완성하게 한 뒤, 의미 동등성 reward 와 Levenshtein 기반 reward 의 혼합 (이 책의 통일 기호로 ρ 와 λ_{KD} 가 관장하는 항들, → 17 장) 으로 강화한다 — “아는 것” 과 “그렇게 행동하는 것” 의 간극을 RL 로 메우는 부품이다.

조립 패턴: 현대 post-training 의 두 축은 reward 의 출처다. RLHF 는 사람선호로 학습한 reward model 이 채점하고 (Ouyang et al. 2022, arXiv:2203.02155; 최적화기로는 PPO 계열), RLVR 은 검증기 (테스트 통과, 정답 일치) 가 채점한다. 이 장에서 이름만 알면 되는 이유는, [Sleep] 이 실제로 쓰는 루프가 그보다 훨씬 단순한 **ReST⁺EM** (Singh et al. 2024, arXiv:2312.06585) 이기 때문이다: 생성한다 → binary reward 로 거른다 → 생존한 표본에 SFT 한다 → 반복한다. value network 도, step 별 보정도 없다 — “필터링된 자기 생성물에 대한 반복 SFT” 라는 EM 품의 루프이며, gradient 추정기로서는 식 (11-4) 의 reward 가 0/1 이라 생존자만 남는 특수형으로 읽어도 무방하다. [Sleep § 3.4] 의 Dreaming 은 SEAL (Zweiger et al. 2025, arXiv:2506.10943) 의 self-edit 루프를 이스케줄에 얹고, 후보 선별과 novelty 주입을 추가한 것이다 (→ 17 장).

마지막 부품은 **LoRA** (Hu et al. 2022, arXiv:2106.09685) 다. weight 행렬에 low-rank 보정 $\Delta W = AB$ ($A \in \mathbb{R}^{d \times d_{\text{low}}}$, $B \in \mathbb{R}^{d_{\text{low}} \times d}$, $d_{\text{low}} \ll d$) 만 학습하는 fine-tuning 이다. 독자는 이것을 serving 쪽에서 이미 안다 — multi-LoRA 스택의 그 adapter 다. 훈련 관점에서 정확히 하자: LoRA 는 훈련 가능한 parameter 수 (따라서 optimizer state · 전송 · 저장량) 를 크게 줄이고, frozen weight 의 **parameter-gradient GEMM** ($dW = \delta x^{\top}$) 을 생략하므로 backward 연산도 full fine-tuning 보다 준다 (LoRA 원문이 throughput 향상으로 보고). 다만 activation gradient δ 는 여전히 backbone 전체를 관통해 흘러야 A, B 에 도달하므로, backward 비용이 adapter 크기에 비례하는 것도 아니다. [Sleep § 3.4] 가 dream 마다 격리된 LoRA SFT 를 쓰는 이유는 훈련이 싸서라기보다 **병렬 · 격리 · 폐기**가 쉬워서다: adapter 하나 = 실험 하나, 실패하면 버린다.

systems 점점: RL 루프의 병목은 학습 스텝이 아니라 rollout 생성이다. 식 (11-4) 의 기대값을 채우는 표본이 전부 autoregressive 생성이므로, 생성 처리량이 곧 학습 처리량이고, RL 인프라의 대부분은 사실상 대규모 decode 서비스다. distillation (§ 11.5) 과 같은 결론의 다른 얼굴: post-training 시대의 워크로드는 독자의 전문 분야 쪽으로 이미 넘어와 있다.

11.7 Worked micro-example: forgetting, EWC, REINFORCE 를 손으로

세 부품을 각각 최소 크기로 돌려 보자. 코드 없이 표와 수식만으로 따라갈수 있다.

(a) **forgetting 이 일어나는 계산.** 모델은 $y = \Theta^\top x$, $\Theta = (\theta_1, \theta_2)^\top \in \mathbb{R}^2$ (성분 θ_1, θ_2 는 장-국소 기호), loss 는 제곱 오차다. task A 는 예제하나: 입력 $x^A = (1, 0)^\top$, 목표 1, 즉 $\mathcal{L}_A(\Theta) = (\theta_1 - 1)^2$. task B 도 하나: $x^B = (1, 1)^\top$, 목표 0, 즉 $\mathcal{L}_B(\Theta) = (\theta_1 + \theta_2)^2$. A 를 끝내면 $\Theta = (1, 0)$ 이고 $\mathcal{L}_A = 0$ 이다. 이제 B 만으로 gradient step 하나를 밟는다. $\nabla_{\Theta} \mathcal{L}_B = 2(\theta_1 + \theta_2)(1, 1)^\top$ 이므로 $(1, 0)$ 에서 gradient 는 $(2, 2)^\top$, $\eta = 0.25$ 로

$$\Theta \leftarrow (1, 0) - 0.25(2, 2) = (0.5, -0.5).$$

결과: $\mathcal{L}_B = (0.5 - 0.5)^2 = 0$ — B 는 한 스텝에 완벽히 풀렸다. 그리고 $\mathcal{L}_A = (0.5 - 1)^2 = 0.25$ — A 의 오차가 0 에서 0.25 로 뛰었다. A 의 데이터는 등장한 적도 없다. 원인은기하다: $x^A \top x^B = 1 \neq 0$ 이라 B 의 gradient 가 A 의 하중 좌표 θ_1 을 관통했다. 대조 실험으로 $x^B = (0, 1)^\top$ 이었다면 B 의 gradient 는 θ_2 축에만 실리고 θ_1 은 손끝 하나 안 다친다 — 간섭은 겹침에서만 온다 (5 장의 crosstalk 조건과 동일). [NL § 4.3] 이 orthogonal task 시나리오를 표준예로 쓰는 이유가 이 대비에 있다.

(b) **EWC 가 구조하는 계산.** task A 의 Fisher 를 추정한다. 고정 분산 Gaussian likelihood $y \sim \mathcal{N}(\Theta^\top x, \sigma^2)$ 를 가정하면 score 는 $(y - \Theta^\top x)x/\sigma^2$ 이고 true Fisher 는 그기대 외적 $x^A x^A \top / \sigma^2$ 이다 (주의: Fisher 는 prediction Jacobian $\nabla_{\Theta} \hat{y} = x$ 의 외적이 아니라 score 의 외적이다 — 최적점에서 관측 label 로 재는 empirical Fisher 는 score 가 0 이라 0 이 될 수 있다). 상수 σ^{-2} 를 λ_{EWC} 에 흡수하면 $F = \text{diag}(1, 0)$: “A 는 θ_1 에만 하중을 싣는다.” $\lambda_{\text{EWC}} = 2$ 로 식 (11-1) 을 쓰면 목적함수는

$$(\theta_1 + \theta_2)^2 + (\theta_1 - 1)^2.$$

두 편미분을 0 으로 놓으면 $\theta_1 + \theta_2 = 0$ 과 $\theta_1 = 1$, 즉 $\Theta^* = (1, -1)$. 계산: $\mathcal{L}_A = (1 - 1)^2 = 0$, $\mathcal{L}_B = (1 - 1)^2 = 0$ — 두 task 모두 정확히 풀렸다. EWC 가 한 일은 update 를 A 의 null 방향 (θ_2) 으로 흘려보낸 것이다: Fisher 가 θ_2 를 “공짜 좌표” 로 판정했고, penalty 가 θ_1 을 닦아 줬다. 단서 하나가 이 장난감에서도 보인다: 구조가 성립한 것은 남는 **capacity**(θ_2) 가 있었기 때문이다. 두 task 가 같은 좌표를 서로 다른 값으로 요구하면 어떤 penalty 계수도 둘 다를 살릴 수 없다 — CF 를 capacity 문제로 재정의하고 확장으로 답하는 [Sleep § 3.2] 의 논리가 2 차원에서 이미 작동한다.

(c) **REINFORCE 한 스텝.** 행동이 두 개뿐인 한 상태 문제 (bandit) 를 보자. policy 는 $\pi_{\Theta} = \text{softmax}(\Theta)$, $\Theta = (0, 0)$ 에서 시작하므로 $p = (0.5, 0.5)$. reward 는 $r(a^{(1)}) = 1$, $r(a^{(2)}) = 0$ 이고 policy 는 이를 모른다. 표본 하나를 뽑아 $a^{(1)}$ 이 나왔다고 하자. softmax 의 성질에서 $\nabla_{\Theta} \log \pi_{\Theta}(a^{(1)}) = (1 - p_1, -p_2) = (0.5, -0.5)$ 이고, baseline 없이 $\eta = 1$ 로 식 (11-4) 를 적용하면

$$\Theta \leftarrow (0, 0) + 1 \cdot 1 \cdot (0.5, -0.5) = (0.5, -0.5), \quad p_1 = \frac{1}{1 + e^{-1}} \approx 0.731.$$

좋은 행동의 확률이 $0.5 \rightarrow 0.731$ 로 올랐다 — reward 를 미분한 적은 없다. 이번엔 $a^{(2)}$ 가 뽑혔다면? $r = 0$ 이라 baseline 없는 update 는 정확히 0 이다: 실패 표본은 아무것도 가르치지 못하고 버려진다. baseline $\bar{r} = 0.5$ 를 넣으면 달라진다: 가중치가 $r - \bar{r} = -0.5$, $\nabla_{\Theta} \log \pi_{\Theta}(a^{(2)}) = (-0.5, 0.5)$ 이므로 update 는 $(-0.5) \cdot (-0.5, 0.5) = (0.25, -0.25)$, 즉 $p_1 = \frac{1}{1 + e^{-0.5}} \approx 0.622$ — 실패 표본에서도 좋은 행동 쪽으로 확률이 이동한다. baseline 이 분산을 줄인다는 말의 손에 잡히는 형태다. 이제 [Sleep § 3.4] 의 Dreaming reward — “이 dream 으로 LoRA-SFT 한 모델이 벤치마크에서 나아졌는가” 의 0/1 — 를 보라. 정확히 이 예제의 reward 자리에 들어가는, 미분 불가능한 black-box 스칼라다.

11.8 Systems bridge: wake/sleep 은 serving fleet 의 lifecycle 이다

1 장의 Rosetta 사전에는 이 장을 위해 예약된 행이 하나 있었다: “서빙 fleet 의 백그라운드 job ↔ sleep phase = 주기적 offline consolidation/dreaming job.” 이제 그 행을 정색하고 채운다.

[해설] 이 책이 제안하는 가장 압축적인 유비는 LSM-tree 다. write 에 최적화된 작은 memtable 이 실시간 트래픽을 받고, 배경의 compaction job 이 주기적으로 그것을 read 에 최적화된 큰 하위 계층으로 재조직해내려보낸다. 대응은 항목별로 성립한다: memtable ↔ fast weights·고주파 블록(빠른 write, 작음, 휘발성), compaction ↔ consolidation(배경에서 재조직·병합, 전면 트래픽과 격리), 계층별 크기·갱신 빈도의 사다리 ↔ update-frequency spectrum (§ 11.4), compaction 의 write amplification ↔ 빠른 블록이 느린블록에 반복 consolidate 되는 비용 ([Sleep § 3.2] 의 주기 1K → 10K 에서 10 회). 유비의 한계도 명시한다: LSM compaction 은 무손실 재배치지만 consolidation 은 distillation 을 거치는 **lossy 추상화**이고, 성공기준이 byte 보존이 아니라 행동 보존이다.

이 그림에서 sleep phase 의 실체는 신비로운 것이 아니라 **serving 배포에 부착된, 스케줄된 오프라인 job 의 묶음**이다. 프로파일별로 뜯으면독자에게 전부 낮익다. (1) corpus 생성과 on-policy rollout 은 decode-heavy 추론 워크로드다 — 훈련 클러스터가 아니라 fleet 의 비수기 용량이 자연서식지다. (2) backward 의 trainable state 는 low-rank expert 또는 LoRA 크기이지만, activation gradient 는 전체 그래프를 관통하므로 backward FLOP 은 adapter 배치 위치에 좌우된다 (§ 11.5 – 11.6). (3) dream 별 LoRA SFT 는서로 격리된 작은 job 이라 embarrassingly parallel 하다. (4) reward 채점은별도 서비스 (reward model) 호출 또는 문자열 연산 (Levenshtein) 이다. 비용의 1 차 회계도 논문에 있다: [Sleep App. B.5] 는 스텝당으로는 Sleep 이 SFT 의 4 배 비싸지만, 같은 정확도에 도달하는 wall-clock 으로는 SFT 쪽이 AIME-24/AIME-25/HMMT-25 에서 각각 $4.3 \times / 3.6 \times / 4.8 \times$ 더 걸린다고 보고한다 — 스텝은비싸고 스텝 수가 적은 profile 이다.

운영 관점의 파생 문제들이 이 lifecycle 의 진짜 청구서다. sleep 한 번이끝날 때마다 fleet 에는 **새 모델 버전**이 생긴다 — checkpoint 관리, cache 무효화, 회귀 테스트, rollback 이 릴리스 이벤트가 아니라 상시 운영문제가 된다. replay/teacher corpus 는 data-plane artifact 로서 저장·보존·감사의 대상이되고 (§ 11.2), per-session fast-weight state 와 per-tenant adapter 는 이 미논한 새 cache class 의 문제다 (→ 10 장). 그리고 정직하게: 이 절의 lifecycle 그림은 1B – 8B 규모의 실험에서 외삽한 것이다. 6 편 전체에 serving 규모의 wall-clock·에너지 회계는 없으며, sleep 스케줄을 실제 트래픽아래에서 언제 어떻게 돌릴지는 열린 문제로 남아 있다 (→ 17 장).

요약

- catastrophic forgetting 은 공유 weight 위에서 gradient update 가 현재 objective 만 보기 때문에 생기는 파괴적 write 이며, 5 장 crosstalk 의시간축 확대판이다. 간섭량은 task 간 입력/gradient 겹침의기하가 정한다.
- 잊는 것은 weight 만이 아니다: $\beta = 0.9$ 인 momentum 은 최근 gradient 7 개가 기여의 50%, 44 개가 99%를 넘는 low-pass filter 라서 (정상상태 $1 - \beta^n$; 원문은 6/43 으로 근사) optimizer 의 memory 도잇는다 [NL § 4.3].
- 완화책은 세 가족으로 압축된다 — replay(데이터 재주입), regularization/EWC(선택적 retention, 식 (11-1)), parameter isolation(동결 + 새 capacity). [Sleep] 의 parameter expansion 은 isolation 가족이고, teacher-표본 corpus 는 생성형 replay 계보다.
- CLS 는 빠른 학습자 (hippocampus) 와 느린 학습자 (neocortex) 의 분업 + replay 전이로 CF 를 피하는 구도이며, [NL]·[Sleep] 의 신경과학 수사를 해독하는 열쇠다.
- fast/slow 이분법은 update-frequency spectrum 으로일반화된다. online consolidation 은 wake 중 end-to-end 학습에 의한전이 ([NL] 의 CMS), offline consolidation 은 입력 없는 기간에 self-generated data 로 하는전이·추상화·확장 ([Sleep] 의 sleep) 이다 [Sleep

§ 3.1]. multi-frequency 는 CF 를 유예할 뿐이므로 consolidation 은 덮어쓰기 직전에 fire 해야 한다 [Sleep § 3.2].

- KD 는 teacher 분포를 target 으로 하는 훈련 (식 (11-2)), GKD 는 student 자신의 rollout 위에서 teacher 가 token-level 로 채점하는 on-policy 혼합 (식 (11-3)) 이다. [Sleep] 의 Knowledge Seeding 은 GKD 의 방향을 뒤집은 upward distillation 이다.
- REINFORCE 는 reward-가중 SFT gradient 다 (식 (11-4)): reward 는 미분불가능한 black-box 여도 되고, baseline 은 분산만 줄인다. ReST^{EM} 은 “생성→ binary 필터→ SFT →반복” 의 단순 루프이며 [Sleep] 의 Dreaming 이그 위에 선다.
- systems 렌즈: sleep phase = serving fleet 에 부착된 주기적 배경 job (LSM compaction 유비), distillation·RL 워크로드의 지배 비용은 생성 (decode-heavy), update frequency 는 write-traffic 예산으로서 memory 계층 배치를 스스로 지정한다.

자가 점검체크리스트

- ☐ catastrophic forgetting 이 왜 gradient descent 의 정의에서 곧바로 따라나오는지, 그리고 crosstalk(→ 5 장) 과 어떤 관계인지 설명할 수 있다.
- ☐ § 11.7(a) – (b) 를 다른 숫자 (예: $x^B = (1, 2)^T$) 로 다시계산해 forgetting 양과 EWC 해를 구할 수 있다.
- ☐ replay·EWC·parameter isolation 의 state/update/cost 를 각각 말하고, [Sleep] 의 parameter expansion + expert reset 이 어느 가족의 어떤변형인지 판별할 수 있다.
- ☐ online consolidation 과 offline consolidation 을 정의하고, [NL] 과 [Sleep] 이 각각 어느 쪽을 다루며 offline 이 왜 별도로필요한지 ([Sleep § 1] 의 세 가지 한계) 설명할 수 있다.
- ☐ 식 (11-2) 와 (11-3) 의 차이를 “훈련 분포 vs 서빙 분포” 의 언어로 설명하고, forward/reverse KL 의 행동 차이를 말할 수 있다.
- ☐ REINFORCE 한 스텝 (§ 11.7(c)) 을 손으로 재현하고, baseline 이 무엇을 바꾸고 무엇을 바꾸지 않는지 말할 수 있다.
- ☐ sleep phase 를 inference 어휘로 옮길 수 있다: 어떤 sub-job 이 decode-heavy 이고 어떤 것이 backward 를 요구하며, 왜 전체가 “weights 의 background compaction” 으로 읽히는지.

다음 장으로

Part I 이 여기서 끝난다. 이 장의 재료 — forgetting, CLS, consolidation, distillation, RL — 는 잠시 서랍에 들어가고, 16 장 (NL) 과 17 장 (Sleep) 에서 회수된다. 당장 다음 장은 라인의 본편이 시작되는 12 장이다: 2 장의 optimizer 객체 (momentum, weight decay), 5 장의 associative loss, 8 장의 TTT, 9 장의 chunkwise 병렬화가 하나의 아키텍처로 조립된 [Titans] (Titans: Learning to Memorize at Test Time, arXiv:2501.00663) 를, 이 책의 master update (M2) 를 기준 삼아 읽는다. 이 장이 심어 둔 질문 하나를 들고 가라: Titans 의 retention gate 가 하는 “학습된 eviction” 은 결국 파괴적 write 다 — 파괴없이 축적하는 시스템은 어떻게 생겼는가. 그 답이 이 라인의 종착지 (16·17 장) 다.

제 2 부 — 여섯 편의 논문

Part II — 여섯 편의 정밀독해

Part II 는 여섯 편을 순서대로 해부한다. 각 장은 같은 8 절 구조를 따른다: 전작이 남긴 open question(bridge-in) → 이 논문의 문제의식 → 통일 표기로 쓴 core mechanism 과 표기 대응표 → outer-loop 학습과 inner-loop test-time 학습의 분리 → concept ledger delta → 실험과 스케일 → systems/serving 함의 → 한계와 다음 논문으로의 인계 (bridge-out). 이 격자가 여섯 편을 비교 가능하게 만든다.

두 원칙이 Part II 를 관통한다. 첫째, **모든 수식은 통일 표기로 환원된다**. 여섯 편은 같은 대상을 서로 다른 (때로 상충하는) 기호로 쓰므로, 이 책은 fast/slow weights 를 W/Θ 로, 게이트 3 종을 η_t (inner learning rate)· β_t (momentum decay)· α_t (retention, 남기는 비율) 로 고정하고, 각 장은 자기 논문의 원 표기와 대응표를 실어 독자가 원문과 대조할 수 있게한다. 기준이 되는 것은 master update — “GD + momentum + weight decay 가 sequence layer 다” 를 한 줄로 요약하는 식 (M) — 이며, 각 논문은 (M) 의 어느성분을 바꾼 것으로 서술된다. 둘째, **정직성 계약이 본문에 박혀있다**. Muon 제거가 오히려 perplexity 를 개선한 ablation, 달하지 않은 retrieval 격차, momentum·gating 을 벗겨 낸 TNT 의 단순화, pre-trained backbone 위에 얹힌 Sleep 의 graft — 논문에 불리한 사실은 해당장이 반드시 담는다.

장들은 하나의 연속 서사로 이어진다. **12 장 [Titans]** — deep memory 를 GD-with-momentum-and-decay 로 갱신하는 sequence layer, 그리고 attention 과의 세 결합 (MAC/MAG/MAL). **13 장 [Miras]** — 그 점 설계를 4 축 설계 공간 (architecture × attentional bias × retention × algorithm) 으로 일반화하고 forget gate 를 retention 으로 재이론화; Moneta/Yaad/Memora. **14 장 [Atlas]** — 그 공간의 각 축을 최적으로 밀어붙인다: capacity 이론, windowed Omega rule, inner Muon. **15 장 [TNT]** — 훈련 비용을 계산해 지불하는 systems 편: chunk 경제학, reset 과 context parallelism, Q-K projection, train/serve chunk-size mismatch. 이 라인이 내놓은 **유일한** wall-clock 증거가 여기 있다. **16 장 [NL]** — 그 수가 보편적임을 선언한다: 모델도 optimizer 도 backprop 도 update frequency 로 색인된 nested associative memory 이고, CMS 와 self-modifying Titans 와 Hope 가 그 종합이다. **17 장 [Sleep]** — train/test 경계를 지운다: wake/sleep lifecycle, upward consolidation(Knowledge Seeding), Dreaming.

여섯 장을 겹치면 layer 여섯 개가 아니라 **불변식 하나의 폐기** — “추론 중 weights 는 변하지 않는다” 의 소멸 — 가 남는다. 그 폐기가 만드는배포 형태와 그것이 systems 엔지니어에게 던지는 질문을 정식화하는 것이 Part III 의 몫이며, 그 인계는 18 장이 이어받는다.

ch12. Titans: Learning to Memorize at Test Time

12.1 Bridge-in: TTT가 남긴문제

Part II의 첫 논문은 [Titans] (Titans: Learning to Memorize at Test Time, arXiv:2501.00663; Ali Behrouz, Peilin Zhong, Vahab Mirrokni, Google Research, 2024-12-31 v1)이다. 이장의 출발점은 8장이 멈춘 자리다. TTT-Linear/TTT-MLP(→ 8장)는 “hidden state는 작은 모델의 weights이고, state update는 그 모델에대한 gradient descent step이다”라는 등식을 language modeling 스케일에서처음 작동시켰고, chunk안의 gradient를 chunk 시작 상태에서 평가하는 mini-batch 근사(→ 8장의 dual form, → 9장의 일반화)로 훈련을 GEMM 위에올려놓았다. 그러나 TTT가 남긴 결핍은 명확했다.

첫째, **지우는 방법이 없다**. TTT의 inner optimizer는 순수 SGD다. state는 고정 크기인데 write는 무한히 쌓이므로, 문맥이 길어지면 언젠가포화한다. 5장의 어휘로 말하면 crosstalk이 누적되는데 eviction이 없다. 둘째, **optimizer가 기억을 갖지 않는다**. 매 token의 update는 그 token 하나의 gradient만 반영한다. token 사이의 흐름 — 어떤 사건이중요했다면 그 직후 token도 함께 저장해야 한다는 시간적 구조 — 이 update rule에 존재하지 않는다. 셋째, **deep memory의 가치가 미검증이다**. TTT는 MLP state를 허용했지만 깊이가 실제로 무엇을 사주는지 실험으로답하지 않았다.

한편 6장의 DeltaNet 계열은 정반대의 트레이드를 택했다. DeltaNet과 Gated DeltaNet은 memory를 matrix로 묶어 두는 대가로 정확한 closed-form chunkwise recurrence를 얻었고, Gated DeltaNet은 여기에 retention gate(원문 표현 forget gate)까지 더했다. 즉 pre-Titans 지형에서는 “표현력있는 inner optimizer + 깊은 nonlinear memory”와 “병렬화 가능한 훈련”이양립 불가능한 것처럼 보였다. 한쪽 끝에 retention gate를 가진 linear memory(Gated DeltaNet), 다른 쪽 끝에 forgetting도 momentum도 없는 gradient 기반 deep memory(TTT)가 있었고, 가운데가 비어 있었다.

[Titans]는 그 가운데를 채우겠다는 논문이다: momentum과 weight decay를 모두 갖춘 inner optimizer로 깊은 MLP memory를 test time에훈련하되, chunkwise 병렬화와 associative scan으로 훈련 throughput을지킨다. 그리고 Appendix C에서 Gated DeltaNet, Longhorn, RWKV-7, TTT전부를 자신의 특수 사례로 회수한다 — 이 회수가 다음 장 [Miras] (It's All Connected, arXiv:2504.13173; 이 책은 framework 이름 Miras로 통칭한다)의 출발점이 된다.

12.2 문제의식과 논문의 핵심주장

논문이 스스로 정의한 문제는 linear model의 자기모순이다 [Titans § 1]. attention은 정확하지만 $O(L^2)$ 이고, linear recurrent model은 $O(L)$ 이지만 고정 크기 state에 역사를 압축한다. 그런데 linear cost가가장 절실한 구간이 바로 very long context이고, very long context야말로작은 vector·matrix state에 제대로 압축될 수 없는 구간이다. 효율이 필요한 곳에서 품질이 무너지도록 설계되어 있는 셈이다.

[Titans]는 이 모순을 memory의 언어로 재서술한다. 모든 sequence model은 memory 구조, write(update) 연산, read(retrieval) 연산의 3요소로분해된다 [Titans § 2]. Transformer는 압축 없이 KV 쌍을 append하는 growing memory이고 (read는 유사도 검색), linear attention은 $v_t k_t^T$ 를 하나의 matrix에 더해 쓰는 additive write라 overflow가예정되어 있으며, GLA·Mamba-2 계열은 data-dependent erase를, DeltaNet 계열은 replace-then-write를 더한 것이다 (→ 6 장, 7 장). 이관점에서 논문은 다섯 질문을 세운다: (Q1) 좋은 memory 구조는 무엇인가, (Q2) 좋은 update rule은 무엇인가, (Q3) 좋은 retrieval은 무엇인가, (Q4) 서로 다른 memory 시스템 여러 개를 어떻게 합성하는가, (Q5) memory는 deep해야 하는가 [Titans § 1, § 2].

또 하나의 문제의식이 이 라인 전체의 성격을 결정한다. training data를외우는 것은 일반화·privacy·OOD 관점에서바람직하지 않으므로, pre-training에서는 **외우는 방법**을 meta-learn하고 실제 memorization은 test context 위에서만 일어나야 한다는것이다 [Titans § 3.1]. 즉 “learning to memorize at test time”이라는제목 자체가 4 장의 bilevel 구조 선언이다.

핵심 주장은 넷이다. (1) 무엇을 외울지는 **surprise**가 결정하며, surprise는 inner loss의 gradient다. (2) surprise에는 관성이 필요하며, 그것이 곧 momentum이다. (3) 잊기는 per-token weight decay이며, 이는 현대 linear RNN의 gate전부의 일반화다. (4) memory는 matrix가 아니라 깊은 MLP여야 한다. 그리고 이 네 가지를 다 넣고도 훈련은 chunkwise로병렬화된다는 것이 시스템 측 주장이다.

12.3 Core mechanism (통일표기)

이 절은 책 전체의 기준 수식 (M2)를 유도하는 절이다. 이후 모든 Part II 장은 여기서 확정되는 형태와의 차이로 자기 논문을 서술한다.

12.3.1 memory는 weights다: 상태와 함수의분리

neural long-term memory module (LMM)은 작은 신경망이며, 그 weights 자체가 sequence model의 recurrent state다. 이 책의 표기로 상태는 fast weights W_t , 읽기는 함수 $\mathcal{M}(\cdot; W_t)$ 이다. [Titans]는 $L_{\mathcal{M}} \geq 1$ 층 MLP를 쓴다(입출력 폭 d) [Titans § 3.1]. 원문은 $\mathcal{M}(x)$ (write를 동반하는 forward)와 $\mathcal{M}^*(x)$ (read 전용 forward)를 한 기호에 겹쳐 쓰지만, 이 책은상태와 함수를 분리한다 (§ 12.3.10의 대응표 참조) — write는 $W_{t-1} \rightarrow W_t$ 의 명시적 update 식으로, read는 $y_t = \mathcal{M}(q_t; W_t)$ 로 쓴다.

memory가 풀 문제는 associative memory(→ 5 장) regression이다. token $x_t \in \mathbb{R}^d$ 를 slow weights의 projection으로 key와 value로바꾸고 —

$$k_t = W_K x_t, \quad v_t = W_V x_t, \quad W_K, W_V \in \mathbb{R}^{d \times d}$$

— memory가 key에서 value를 재생하도록 하는 inner objective를 세운다 [Titans Eq. 11, Eq. 12]:

$$\ell(W; k_t, v_t) = \|\mathcal{M}(k_t; W) - v_t\|_2^2. \quad (12-1)$$

attention의 KV lookup과 같은 추상 — key와 비슷한 query가 오면 연관된 value를 돌려준다 — 을 weights안에 저장하겠다는 것이다. 여기서 결정적인문장이 나온다: “training of the memory is in the inner-loop, and so parameters W_K and W_V are hyperparameters in the above loss function” [Titans § 3.1]. 즉 W_K, W_V 는 inner 문제의 **hyperparameter**로, inference 중에는 절대 움직이지 않고 outer loop에서만 학습된다. 이 구분은 § 12.4에서 전면 전개한다.

12.3.2 momentary surprise와 기본 update

무엇을 얼마나 세게 쓸 것인가? [Titans]의 답: 지금 들어온 token이이미 저장된 내용을 얼마나 위반하는가, 즉 inner loss의 gradient크기다. **momentary surprise**는 $g_t^{\text{in}} = \nabla_W \ell(W_{t-1}; k_t, v_t)$

로 정의되며, gradient 가 클수록 새롭고 예상 밖인 입력이므로 더 세게 외운다 [Titans § 3.1]. 기본 update 는 1-step gradient descent, 즉 표준형 (M1) 그대로다 [Titans Eq. 8]:

$$W_t = W_{t-1} - \eta_t g_t^{\text{in}}, \quad g_t^{\text{in}} = \nabla_W \ell(W_{t-1}; k_t, v_t), \quad (12-2)$$

여기서 η_t 는 data-dependent inner learning rate — token x_t 의 함수로 slow head 가 산출하는 게이트다. 주의: 원문에서 이 학습률의 기호는 θ_t 이고, 원문의 η_t 는 아래에서 momentum decay 로 쓰인다. 이 책은 기호를 교차 정리했다 (표 12-1).

[해설] $\mathcal{M}$ 이 linear($W \in \mathbb{R}^{d \times d}$) 이면 식 (12-2) 는 정확히 delta rule(→ 5 장) 이고, 따라서 (12-2) 는 DeltaNet(→ 6 장) 과 TTT(→ 8 장) 가 이미 서있던 자리다. inference 독자의 어휘로: KV cache append 가 rank-1 GEMM write 로 바뀐 것이며, “쓰기 전에 현재 저장값을 읽어 오차만 쓴다” 는 점에서 append 가 아니라 read-modify-write 다 (→ 1 장 Rosetta).

12.3.3 past surprise: momentum 이 sequence layer 안으로들어오다

momentary surprise 만으로는 실패하는 상황이 있다. 큰 surprise 직후에는 loss 가 이미 낮아져 gradient 가 급감하므로, 놀라운 사건 **뒤에이어지는** 중요한 token 들이 약하게 저장된다. 논문의 비유로, 놀라운 순간은한동안 주의를 붙들어 그 시간 구간 전체를 기억하게 만든다 [Titans § 3.1]. 그래서 surprise 를 둘로 분해한다: **past surprise** S_t (최근 과거의 surprise 누적) 와 momentary surprise g_t^{in} [Titans Eq. 9, Eq. 10]:

$$S_t = \beta_t S_{t-1} - \eta_t g_t^{\text{in}}, \quad W_t = W_{t-1} + S_t. \quad (12-3)$$

S_t 는 weights 와 같은 shape 의 벡터이며, 이것은 문자 그대로 SGD-with-momentum 의 momentum buffer 다 (→ 2 장의 optimizer-as-object: state S_t , update 식 (12-3), cost 는 elementwise 연산). 즉 [Titans] 의 첫 번째 발명은 “optimizer state 를 sequence layer 의 recurrent state 로 승격” 시킨 것이고, 논문은 이를 momentum 이 “시간축 위의 surprise 에 대한 memory” 로 작동한다고 읽는다. **data-dependent surprise decay** $\beta_t \in [0, 1]$ 은 token 의 함수다: $\beta_t \rightarrow 0$ 이면 과거 surprise 의 전파를 끊고 (문맥 전환), $\beta_t \rightarrow 1$ 이면 온전히전파한다 (현재 token 이 직전 문맥과 강하게 결속) [Titans § 3.1]. β_t, η_t 를 상수가 아니라 token 의 함수로 두는 것이 설계의요점이다 — 과거 surprise 가 지금의 write 에 영향을 줘야 하는지는 두 token 이 같은 문맥에 있는지에 달려 있기 때문이다.

12.3.4 forgetting: weight decay 가 per-token retention 이되다

수백만 token 을 다루려면 deep memory 라도 언젠가 가득 찬다. 그래서 마지막성분으로 **forgetting mechanism** 을 넣는다 — 그리고 그것의 정체는 per-token **weight decay** 다 [Titans Eq. 13, Eq. 14]. 통일표기로 쓰면 이것이 이 책의 master update (M2) 다:

$$S_t = \beta_t S_{t-1} - \eta_t \nabla_W \ell(W_{t-1}; k_t, v_t), \quad W_t = \alpha_t W_{t-1} + S_t. \quad (12-4)$$

읽는 법: $\alpha_t \in [0, 1]$ 은 **남기는 비율**이다. $\alpha_t \rightarrow 1$ 이면 과거의 추상을 전부 보존한 채 덧쓰고, $\alpha_t \rightarrow 0$ 이면 memory 를 통째로 소거한다. **방향 주의**: Titans 원문의 α_t 는 “잊는 비율” 이라 $(1 - \alpha_t)\mathcal{M}_{t-1}$ 로 등장하며, 이 책의 $\alpha_t^{(\text{통일})} = 1 - \alpha_t^{(\text{Titans})}$ 이다 (표 12-1). 이 게이트가 뒤에 [Miras] 가 retention gate(→ 13 장) 로재이론화하는 대상이고, Titans 문맥에서는 “forgetting mechanism” 이라는원문 표현을 그대로 인용할 수 있다. 논문은 이 weight decay 가 Mamba-2, GLA, Gated DeltaNet 의 gating 을 임의의 (deep) memory 로 일반화한 것임을보인다 [Titans § 3.1, App. C]. 게이트 차원 주의: 식 (12-4) 는 $\eta_t, \beta_t, \alpha_t$ 를 스칼라로 쓴 형태이고, 일반형 (식 (12-6), [Titans App. C Eq. 32 – 33]) 은 채널별 대각 게이트 $\text{diag}(\cdot)$ 로 W, S 에 원소별로 작용한다 — 스칼라는 그특수형이다.

정리하면, [Titans]의 write 연산 전체는 “**mini-batch gradient descent + momentum + weight decay**” 한 step 이고, **optimizer**의 세스칼라 ($\eta_t, \beta_t, \alpha_t$)가 모두 token마다 게이트로 산출된다. 훈련을 한 번도 본 적 없는 독자를 위해 다시 말하면: 2장에서 optimizer를 (state, update, cost)를 가진 객체로 배웠는데, [Titans]는 그 객체를통째로 모델 내부에 넣고 그 hyperparameter 세 개를 학습된 함수로 바꾼것이다. 뒤의 ablation (§ 12.6)은 세 성분의 기여를 weight decay > momentum > convolution > persistent memory 순으로 매긴다 [Titans Table 5].

12.3.5 read: 갱신 없는 forward

읽기는 weight 갱신이 없는 순수 forward pass 다 [Titans Eq. 15]:

$$q_t = W_Q x_t, \quad y_t = \mathcal{M}(q_t; W_t). \quad (12-5)$$

W_Q 역시 outer 에서 학습되는 projection 이다. read 가 상태를 바꾸지않는다는 것은 KV cache lookup 과 동일한 성질이며, write 와 read 가 명시적으로 분리된 두 연산이라는 것이 이 라인의두-연산 (compression/retrieval) 추상의 원형이다.

12.3.6 왜 deep memory 인가

$\mathcal{M}$ 을 matrix $W \in \mathbb{R}^{d \times d}$ 로 두면 식 (12-1)은 online **linear** regression 이고, 그 최적해는 역사가 선형사상으로 압축 가능하다는 가정을 깔고 있다 [Titans § 3.1]. **deep memory** — $L_{\mathcal{M}} \geq 2$ 층 MLP — 는 key-value 구조가 비선형이어도 저장할 수 있다는 것이 논문의 논거이며 (2층 이상 MLP의 표현력 우위를 인용), § 12.6의 깊이 실험과 ablation(linear memory로 바꾸면 long-context 점수가 $92.68 \rightarrow 85.34$ 로 하락 [Titans Table 5])이 실증을 맡는다. 이 라인의 후속작들이 표준형으로 굳히는 2-layer residual MLP (§ 1.3의 “표준 deep memory”)의 출발점이며이다. 단, [Titans] v1은 memory MLP의 폭·activation 등세부를 명시하지 않는다.

12.3.7 persistent memory

LMM은 **contextual memory** — 내용이 전적으로 입력 문맥에 의존하는 memory — 다 (attention의 KV도 마찬가지다). [Titans]는 여기에 입력과무관한 세 번째 memory를 더한다. **persistent memory**는 $N_p \geq 1$ 개의 학습되는 input-independent 벡터 $P = [p_1 \dots p_{N_p}]$ 를 sequence 앞에 붙이는 것이다 [Titans Eq. 19]:

$$x_{\text{new}} = P \parallel x.$$

(sequence 결합에서 P 는 행-쌓기 $[p_1^\top; \dots; p_{N_p}^\top] \in \mathbb{R}^{N_p \times d}$ 로 $X \in \mathbb{R}^{L \times d}$ 앞에 붙는다 — 열벡터 정의 $P \in \mathbb{R}^{d \times N_p}$ 의 전치. 이하 MAC의 $P \| r \| X$ 도 같은 행 방향 결합이다.)

세 가지 정당화가 제시된다 [Titans § 3.3]. (1) memory 관점: task를 수행하는 방법에 대한 지식은 입력에 따라 변하면 안 되므로 input-independent parameter에 살아야 한다. (2) FFN 관점: $\text{FFN}(x) = W_V \text{Softmax}(W_K x)$ [Titans Eq. 20] — FFN은 K/V가 data-independent한 attention이며 (Sukhbaatar et al. 2019, arXiv:1907.01470 인용), persistent token은 attention 내부에서 같은 역할을 한다. (3) 기술적 관점: causal attention은 초반 token에 과도한 weight를 주는 attention-sink 편향이있는데, 학습 가능한 prefix token이 그 편향을 흡수·재분배한다.

12.3.8 세 가지 합성: MAC / MAG / MAL, 그리고 LMM 단독

Q4의 답으로 [Titans]는 attention을 정밀한 short-term memory, LMM을 서서히 잊는 long-term memory로 두고 세 가지 합성을 제시한다 [Titans § 4]. 세 변형 모두 persistent token을 포함한다. 풀네임은 **Memory as Context (MAC)**, **Memory as Gate (MAG)**, **Memory as Layer (MAL)**이며 이하 약어로 쓴다.

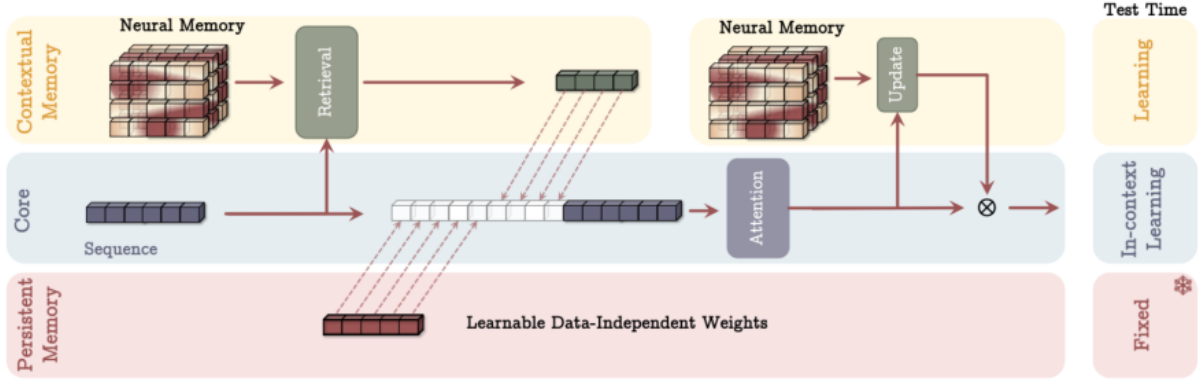


그림 12-1 — MAC 아키텍처의 데이터 흐름. 세 branch 가 나란히있다: contextual(long-term) memory 는 retrieval 로 과거를 읽어 core branch 입력에 concat 되고, attention 을 통과한 출력이 다시 memory 에 write 되며 (오른쪽 Update), persistent memory 는 data-independent 학습 weight 로 test time 에 동결 (눈송이) 된다. 오른쪽 라벨이 test-time 역할분담을 요약한다 — memory 는 여전히 learning, core(attention) 는 in-context learning, persistent 는 fixed. 출처: Behrouz et al., Titans (arXiv:2501.00663), 원문 Fig.2 —원저자의 그림 (third-party), 본서의 결과가 아님.

MAC [Titans § 4.1, Eqs. 21 – 25]. sequence 를 크기 C 의 segment 로 자른다 (원문의 segment 크기 C — 이 책은 chunk 크기와동일시한다, 표 12-1). n 번째 segment 의 token 행렬을 $X^{(n)} \in \mathbb{R}^{C \times d}$ (token $nC+1 \dots (n+1)C$) 라하자. segment 가 들어오면, 직전 segment 까지 갱신된 memory 상태 W_{nC} 에서 읽고, attention 을 돌린 뒤, 그 출력을 memory 에 쓴다:

$$r_\tau = \mathcal{M}(q_\tau; W_{nC}) \quad (\text{segment 의 각 token } \tau, q_\tau = W_Q x_\tau),$$

$$\tilde{X}^{(n)} = P \parallel r \parallel X^{(n)}, \quad a = \text{Attn}(\tilde{X}^{(n)}),$$

$$W_\tau \leftarrow \text{식 (12-4) 를 } a_{nC+1..\tau} \text{ 에 prefix 로 실행, } y_\tau = \text{gate}(a_\tau, \mathcal{M}(a_\tau; W_\tau)).$$

각 위치 τ 의 최종 read 는 자기까지의 prefix 갱신으로 얻은 상태 W_τ 에서 이뤄져야 causal leakage 가 없다 — segment-end 상태 $W_{(n+1)C}$ 에서 모든 위치를 읽으면 앞쪽 token 이 뒤 token 의 write 를보게 된다 ([Titans Eq. 16] 은 chunk 안에서 M_t 를 prefix 누적으로정의한다; [Titans Eq. 24–25] 의 압축 표기가 이 token 별 causal 계산을제거하는 것은 아니다). chunkwise 구현에서는 causal triangular/prefix dual form 으로 모든 W_τ 를 계산하고, $W_{(n+1)C}$ 는 다음 segment 로의 handoff 로 쓴다.

즉 attention window 안에는 persistent token, memory 에서 검색해 온 역사 r (원문 표기 h_t — outer 2 차 moment 와의 충돌을 피해 개명, 표 12-1; 원문 mask 표기로는 N_l 개의 long-term memory token 인데, [Titans Eq. 21] 의 query 가 segment 전체 — C 개 token — 이므로차원상 $N_l = C$ 가 따라 나온다; 단 [Titans Fig. 3a] 캡션은 별도상수 N_l 표기를 유지한다), 현재 segment 가 나란히 놓이고, 그 안에서 full causal attention 이 돈다 (전체 sequence 관점의 attention mask 는 segment 별 block-diagonal 이다 [Titans Fig. 3a]). 설계 이유가중요하다 (그림 12-1 의 “read $\rightarrow$ attention $\rightarrow$ write” 흐름이 이 세 branch 배치를 그대로 보여준다): (i) attention 이 검색된 역사와 현재 데이터를동시에 보므로 지금 long-term 정보가 필요한지를 token 단위로 판단할 수있고, (ii) memory 에는 attention 이 처리한 표현만 쓰이므로 attention 이 **write filter** 역할을 해 쓸모없는 token 으로 인한 memory overflow 를 줄인다 [Titans § 4.1]. 원문의 write 식 $\mathcal{M}_t = \mathcal{M}_{t-1}(y_t)$ [Titans Eq. 24] 는 “forward pass 를 통해 weight 를 갱신한다” 는 관행 표기로, segment 내부의 write 세분 (granularity) 은 명시되어 있지 않다. 최종 read 도 attention 출력 a_τ 를 W_Q 재적용 없이 그대로 query 로 쓴다 [Titans Eq. 25]. 또한 retrieval 이 갱신 전 상태 W_{nC} 를 읽으므로 read 는 항상 한 segment 만큼 stale 하다.

MAG [Titans § 4.2, Eqs. 26–28]. segmentation 없이 두 branch를 병렬로 돌린다. branch 1 은 persistent prefix 가 항상 보이는 sliding window attention(SWA), branch 2 는 prefix 가 붙은 전체 sequence 위를 식 (12-4) 로 도는 LMM 이다. 두 branch 의 출력을 학습된 elementwise gate 로 결합한다 — 실험에서는 각 branch 출력을 학습 가능한 벡터 weight 로 normalize 한 뒤 비선형 $\sigma(\cdot)$ 를 거쳐 곱한다 [Titans § 4.2]. 원문은 이 결합을 $\otimes$ 로 쓰지만 이 책은 $\text{gate}(\cdot, \cdot)$ 로 쓴다 ($\otimes$ 금지, § 1.3). SWA 가 선명한 short-term, LMM 이 fading long-term 이며, 구조가 다른 head 들을 가진 multi-head layer 로도 읽을 수 있다 (Hymba 인용).

MAL [Titans § 4.3, Eqs. 29 – 31]. LMM 을 하나의 layer 로 SWA 앞에 직렬로 쌓는다: $\tilde{x} = P \parallel x$, $z = \text{LMM}(\tilde{x})$, $y = \text{SW-Attn}(z)$ (z 는 LMM branch 의 per-token read 를 쌓은 중간 출력, 장-국소 기호). 이는 H3 이래사실상 모든 hybrid(Samba, Griffin 등) 가 쓰는 관행 패턴인데, 논문은 이 설계가 **가장 약하다**고 명시한다 — 파이프라인의 힘이 각 단의 힘으로 상한되어 attention 과 memory 의 상보성을 살리지 못한다는 것이다 [Titans § 4.3]. 기존 hybrid 전부가 MAL 모양이라는 점에서, $\text{MAC/MAG} > \text{MAL}$ 이라는 § 12.6 의 결과는 개별 모델비교를 넘어 hybrid 설계 관행 자체에 대한 기초장이다.

LMM 단독: attention 없이 memory module 만 sequence model 로 쓰는 변형. memory 시스템의 각 부분은 독립적으로도 작동해야 한다는 § 12.2 의 Q4 철학에 따른 대조군이자, long-term memory 가 홀로도 강하다는 주장의 시험대다 [Titans § 4.3].

블록 세부 [Titans § 4.4]: 모든 블록에 residual connection; q/k/v 계산에 SiLU activation; query 와 key 의 ℓ_2 -normalization; q/k/v projection 뒤 1D depthwise-separable convolution(효과는 작지만 양수, 비용 저렴); 최종 출력 projection 앞에 normalization + linear gating.

표현력 주장: [Titans] 는 Transformer·diagonal linear RNN·DeltaNet 이 TC^0 에 갇히는 반면 (Merrill et al. 인용) Titans 는 TC^0 너머의 문제를 풀 수 있어 state tracking 에서 이론적으로 더 표현력 있다고 주장한다 (Thm 4.1). **v1 에는 이 정리의 증명이 없다** — 라인 끝 (NL) 까지도 이 주장은 정리가 아닌 실증으로만 뒷받침된다는 점을 이 책은 반복해 기록한다. 추가 표현력의 출처로 지목되는 것은 § 12.4 에서 보게 될 inter-chunk nonlinear recurrence 다.

12.3.9 특수 사례 회수: 이 update 는 기존 계보 전부를 포함한다

[Titans App. C] 는 라인 전체에서 가장 많이 인용되는 부록이다. memory 를 $\text{linear}(W_t \in \mathbb{R}^{d \times d})$ 로, loss 를 $\ell = \frac{1}{2} \|Wk_t - v_t\|_2^2$ 로 두면 식 (12-4) 는 다음이 된다 [Titans Eq. 32, Eq. 33]:

$$W_t = \text{diag}(\alpha_t) W_{t-1} + S_t, \quad S_t = \text{diag}(\beta_t) S_{t-1} - \text{diag}(\eta_t) (W_{t-1} k_t k_t^\top - v_t k_t^\top). \quad (12-6)$$

식 (12-6) 에서 $\beta_t = 0$ (momentum 차단) 으로 두면 Gated DeltaNet 의 rule $W_t = W_{t-1}(I - \eta_t k_t k_t^\top) + \eta_t v_t k_t^\top$ (+ decay) [Titans Eq. 34] 와 일치한다. Longhorn 은 같은 loss 를 implicit online learning 으로 풀어 step size 가 $\eta_t / (1 + \eta_t k_t^\top k_t)$ 로 바뀐 delta 형태이되 retention gate 가 없고 [Titans Eq. 35], RWKV-7 은 같은 loss·형식을 쓴다는 이유로 명시적으로 포함되며 (“similar approaches such as RWKV-7 ... LMM is generalizing all such models” [Titans App. C]), TTT 는 forgetting 도 momentum 도 없는 gradient 기반 특수 사례다 [Titans App. C]. 결국 LMM 은 이 계보 전체를 네 축에서 일반화한다: (1) momentum 기반 (token 흐름 인지) update — 논문은 자신이 linear-recurrent 계열 최초의 momentum rule 이라고 주장한다, (2) deep memory, (3) inter-chunk nonlinear recurrence, (4) retention gate. 6 장의 카탈로그 (§ 1.6) 한 줄 요약으로: Titans-LMM = (M2) + 표준 deep memory.

[해설] 이 부록의 함의는 양방향이다. 앞으로는 “기존 모델전부가 (M2) 의 성분을 끈 것”이라는 통일이고, 뒤로는 “그럼 왜 하필 ℓ_2 loss, 왜 하필 momentum+decay 인가” 라는 질문이다. [Titans] 는 전자만 답하고 후자를 열어 둔다 — 그 열린 질문이 13 장의 [Miras] 다.

12.3.10 표기 대응표

표 12-1 — [Titans] 원 표기 $\leftrightarrow$ 이 책의 통일 표기 (STYLE-NOTATION § 1.7.1 사본 + 장-국소 형)

Titans 원 표기	통일 표기	의미 / 주의
$\mathcal{M}_t$ (상태 점 함수) $\mathcal{M}(x)$ – write 동반 forward	W_t (상태), $\mathcal{M}(\cdot; W_t)$ (함수) update 식 + read 로 분해해 표기	상태/함수 분리 (P1)
$\mathcal{M}^*(x)$ – read-only forward	$\mathcal{M}(x; W_t)$	★는통일 체계에서 argmin 전용
θ_t – inner learning rate	η_t	△ 기호 교차 주의
η_t – surprise(momentum) decay	β_t	△ Titans 의 η 는 lr 가 아니다
α_t – forget gate, $(1 - \alpha_t)\mathcal{M}_{t-1}$	$\alpha_t^{(\text{통일})} = 1 - \alpha_t^{(\text{Titans})}$	방향 반전: 통일 α_t 는 남기는 비율
S_t – past surprise (momentum buffer)	S_t	동일
$\nabla \ell(\mathcal{M}_{t-1}; x_t)$ – momentary surprise	$g_t^{\text{in}} = \nabla_W \ell(W_{t-1}; k_t, v_t)$	
$\mathbf{k}_t = x_t W_K$ (행벡터 관행)	$k_t = W_K x_t$ (열벡터)	
b – inner mini-batch(chunk) 크기	C	
segment $S^{(i)}$, 크기 C (MAC)	“segment” (산문), 크기 C	segment 크기 = chunk 크기로 동일시함을 명시
N_p, P, p_i – persistent memory	동일	
d_{in}	d	
$L_{\mathcal{M}}$ – memory 깊이	동일	
M_0 – 초기 상태	W_{init}	meta-learn 됨 (암묵적)
$y_t = \mathcal{M}^*(q_t)$	$y_t = \mathcal{M}(q_t; W_t)$	
$\otimes$ – MAG 의 gating 결합	$y \odot g$ 또는 $\text{gate}(\cdot, \cdot)$	⊗ 금지
$\beta_i = \prod_{j \leq i} (1 - \alpha_j)$ – 누적 decay (Eq. 16)	$\bar{\alpha}_i = \prod_{j \leq i} \alpha_j$ (장-국소)	△ 통일 β_t (momentum decay) 와 충돌 방지
$u_t = \nabla \ell(M_{t'}; x_t)$ – chunk-anchored gradient (Eq. 18)	$\hat{g}_\tau$ (장-국소)	△ NL 의 u_t (LSS) 와 충돌 방지
$\Theta_b, \mathbf{B}_b$ – chunk 별 diagonal (Eq. 17)	$\text{diag}(\tilde{\eta}_1, \dots, \tilde{\eta}_C),$ $\tilde{\eta}_i = \eta_i \bar{\alpha}_C / \bar{\alpha}_i$ (장-국소)	두 diagonal 을 하나로 합쳐 표기; △ 예약 기호 c (Omega window 길이) 를 피해개명
$t' = t - \text{mod}(t, b)$ – chunk 시작 (Eq. 16)	$\xi(t, C) = C \lfloor (t - 1) / C \rfloor$	원문 정의는 chunk 마지막 token($t = b$) 에서 $t' = b$ 가 되는 경계 슬립 – § 1.2 정의로 교정
h_t – MAC 의 retrieved history (Eq. 21)	r_τ (장-국소)	△ outer 2 차 moment h_t 와 충돌 방지
$S^{(t)}, \tilde{S}^{(t)}$ – segment, 증강 segment (Eq. 22)	$X^{(n)}, \tilde{X}^{(n)}$ (장-국소)	△ momentum S_t 와 충돌 방지
y_t – MAC 의 attention 출력 (Eq. 23)	a_τ (장-국소)	layer 최종 출력 y 와 구분
o_t – 최종 출력 (Eq. 25)	y_τ	통일 규약: layer 출력은 y

12.4 Outer-loop training vs inner-loop test-time learning

이 절이 이 장의 심장이다. training 경험이 없는 독자의 첫 질문 — “그게이트는 누가 학습하는가?” — 에 성분 단위로 답한다.

표 12-2 — [Titans] 에서 무엇이 어느 loop 에 사는가

구성 요소	소속	갱신 시점 · rule	크기/비용 감각
projection W_K, W_V, W_Q	Θ (slow)	pre-training 에서만, AdamW 로	inner loss 의 hyperparameter
게이트 산출 head ($\eta_t, \beta_t, \alpha_t$ 를 emit)	Θ (slow)	pre-training 에서만	inference 에서 token $\rightarrow$ 게이트 3 개 산출 (스칼라 또는 채널별 벡터; § 12.3.4)
persistent tokens P	Θ (slow)	pre-training 에서만; test time 에 동결	$N_p \times d$
attention 블록, conv, normalization, LM head	Θ (slow)	pre-training 에서만	통상적 backbone
초기 memory 상태 W_{init}	Θ (slow)	pre-training 에서 암 묵적으로	v1 은 설정을 명시하지 않음
memory weights W_t	inner (fast)	매 token, 식 (12-4) 로, inference 중에도	$P_M \approx L_M d^2$ per layer per sequence
momentum buffer S_t	inner (fast)	매 token, 식 (12-4) 로	W_t 와 같은 shape — state 2 배의 주범

outer loop: 무엇이 meta-learn 되는가. memory 의 일시적 상태를 제외한 전부다. outer objective 는 평범한 next-token cross-entropy $\mathcal{L}$ 이고, 논문은 이 구조를 Reptile/CAVIA 계열 meta-learning 으로 명시적으로 자리매김한다 [Titans § 3.1]: inner 에서는 $\mathcal{M}$ 의 weights 를 최적화하고, outer 에서는 아키텍처의 나머지를 최적화한다. 기술적으로 결정적인 사실은 이것이다 — **outer gradient 는 unroll 된 inner update 를 관통한다.** 아래 chunkwise closed form 이 곧 forward graph 이며, autodiff 가 그 그래프를 그대로 backprop 한다. 4 장의 어휘로 “optimizer 를 미분한다” : W_K 를 바꾸면 매 token 의 inner gradient 가 바뀌고, 그것이 W_t 의 전 궤적을 바꿔 최종 $\mathcal{L}$ 을 바꾼다. 그 연쇄 전체에 대한 gradient 가 W_K 를 학습시킨다. 그래서 게이트에 대한 답은: $\eta_t, \beta_t, \alpha_t$ 의 값은 inference 중 매 token 새로 계산되지만, 그 값을 만드는 함수 (head 의 weights) 는 pre-training 에서만 학습되고 이후 동결된다. 단, v1 은 이 head 의 함수형 · 초기화를 명시하지 않는다 — 재구현자가 채워야 하는 공백이다.

chunkwise 병렬화: inner loop 를 GEMM 으로. 순수 online rule (12-4) 는 $O(L)$ FLOPs 이지만 엄격히 순차적이고 matmul 이 없다 — 가속기에서 최악의 형태다. [Titans § 3.2] 는 TTT($\rightarrow$ 8 장) 를 따라 sequence 를 크기 C 의 chunk 로 자르고, chunk 안의 모든 gradient 를 chunk 시작 상태에서 평가한다 (9 장의 stale-snapshot 근사, 표준형 (M4)). momentum 을 잠시 끄고 첫 chunk($W_0 = W_{\text{init}}, t \leq C$) 에 집중하면 recurrence 가 닫힌 꼴로 풀린다 [Titans Eq. 16]:

$$W_t = \bar{\alpha}_t W_0 - \sum_{i=1}^t \eta_i \frac{\bar{\alpha}_t}{\bar{\alpha}_i} \nabla_W \ell(W_0; k_i, v_i), \quad \bar{\alpha}_i = \prod_{j=1}^i \alpha_j. \quad (12-7)$$

retention 은 누적 곱 $\bar{\alpha}$ 로 접히고, 모든 gradient 가 같은 동결상태 W_0 에서 평가되므로 C 개를 동시에 계산할 수 있다. 일반위치에서는 anchor 가 $W_{\xi(t,C)}$ 가 된다 — 이 근사가 계산되는 함수 자체를 바꾸는 semantic 한 선택이라는 것, 즉 FlashAttention tiling(bit-exact) 과 결정적으로 다르다는 것은 9 장의 명제다. linear memory 로 구체화하면 chunk 하나의 gradient 합이 GEMM 두 개로 tensorize 된다 [Titans Eq. 17]:

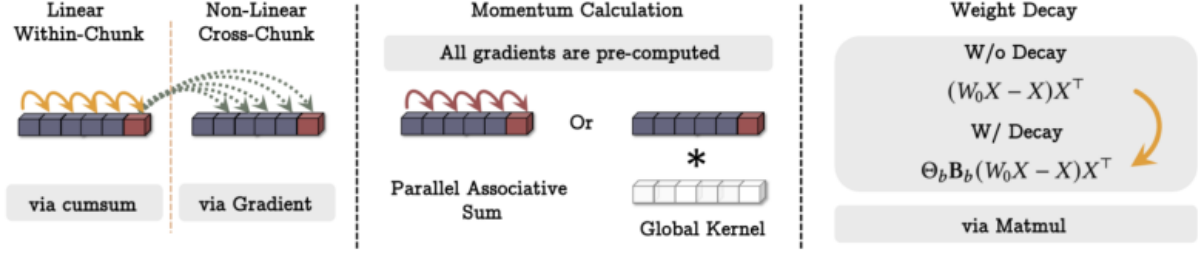


그림 12-2 — inner-loop 갱신을 가속기 친화적 연산으로 병렬화하는 세 조각. (왼쪽) chunk 안 갱신은 linear 라 cumsum 으로, chunk 경계를 넘는 재진입은 nonlinear 라 gradient 로 처리된다. (가운데) momentum buffer 는 gradient 를 미리 계산해 두면 1 차 선형 recurrence 가 되어 parallel associative sum 또는 global(convolution) kernel 로 얻는다. (오른쪽) weight decay 는 누적 곱을 대각으로 접어 넣은 matmul 로 흡수된다 — 본문 식 (12-7) – (12-9) 의 GEMM·scan 구조가 그림 그대로다. 출처: Behrouz et al., Titans (arXiv:2501.00663), 원문 Fig.1 — 원저자의 그림 (third-party), 본서의 결과가 아님.

$$\sum_{i=1}^C \eta_i \frac{\bar{\alpha}_C}{\bar{\alpha}_i} \nabla_W \ell(W_0; k_i, v_i) = (W_0 K^\top - V^\top) \text{diag}(\tilde{\eta}_1, \dots, \tilde{\eta}_C) K, \quad \tilde{\eta}_i = \eta_i \bar{\alpha}_C / \bar{\alpha}_i, \quad (12-8)$$

여기서 $K \in \mathbb{R}^{C \times d_k}$, $V \in \mathbb{R}^{C \times d_v}$ 는 chunk 의 key/value 행-쌍기다. (식 (12-8) 은 § 12.3.9 처럼 $\ell = \frac{1}{2} \|Wk - v\|^2$ 관행을따르므로 gradient 에 계수 2 가 나타나지 않는다. 식 (12-1) 의 무- $\frac{1}{2}$ loss 를 그대로 쓰면 각 항에 계수 2 가 붙고, 그 상수는 학습되는 $\tilde{\eta}_i$ 에 흡수된다.) 원문 표기 $\Theta_b \mathbf{B}_b(W_0 X - X) X^\top$ 는 행벡터 관행에 더해 (k, v) 자리에 x 를 쓰는 축약이라 재구현 시 되돌려야 한다. chunk 마다 diagonal 계수만 materialize 하면 되고 L/C 개를 모두 들고 있을 필요가 없다 [Titans § 3.2]. 식 (12-8) 을 GEMM shape 로 읽으면 ($\rightarrow$ 1 장 Rosetta) 이것은 $(d_v \times C) \cdot (C \times d_k)$ 의 rank- C write 다: **chunk 크기 C 가 곧 write GEMM 의 내적 차원이고, 따라서 arithmetic intensity 의 손잡이다.** MLP memory($L_M \geq 2$) 의 경우도층별 gradient 를 chunk-start weights 에서 batch 로 평가하는 같은 구조라고만 서술된다 [Titans § 3.2].

momentum 은 어떻게 되는가? chunk 안에서 $\hat{g}_\tau = \nabla_W \ell(W_{\xi(\tau, C)}; k_\tau, v_\tau)$ 를 전부 미리 계산해 두면 [Titans Eq. 18]

$$S_\tau = \beta_\tau S_{\tau-1} - \eta_\tau \hat{g}_\tau \quad (12-9)$$

는 input-dependent 전이 계수 β_τ , 입력 $\hat{g}_\tau$, hidden state S_τ 의 1 차 선형 recurrence 다 — 정확히 S5/Mamba 의 selective scan 과 같은 모양이므로 parallel associative scan($\rightarrow$ 7 장, 9 장) 으로 chunk 내 모든 S_τ 를 log-depth 에 얻는다. 독자가 이미 아는 scan/prefix-sum kernel 이 그대로 훈련 그래프의 한가운데에 등장하는 것이다.

결과 구조가 [Titans] 의 표현력-병렬성 협상이다 (그림 12-2): **intra-chunk** 는 linear(matmul + scan 으로 병렬), **inter-chunk** 는 nonlinear(chunk 경계에서 갱신된 MLP weights 가 다음 chunk 의 loss 에 비선형으로 재진입). 순차 임계 경로가 L 이 아니라 L/C 번의 MLP-weight 적용 으로 줄고, 동시에 이 비선형 경계가 Thm 4.1 의 표현력 주장의 근거로 지목된다 — DeltaNet 계열 이 정확한 closed form 을위해 전부 linear 로 남은 것과 의도적으로 다른 트레이드다. 추가 가속 옵션으로 α, η, β 를 token 이 아니라 chunk 의 함수로 두면 식 (12-9) 가 LTI(linear time-invariant $\rightarrow$ 7 장) 시스템이 되어 S4 식 global convolution 으로 계산 가능하지만 [Titans § 3.2], 실험은 표현력을 위해 per-token 게이트를 유지했고 이 fast path 는 끝내 평가되지 않았다.

outer 레시피 [Titans § 5.1]: 훈련 길이 4K, Llama-2 tokenizer(32K vocab), AdamW lr $4e-4$ cosine annealing, batch 0.5M tokens, weight decay 0.1 — Gated DeltaNet 의 프로토콜을 따른다. 여기서 두층위가 한 문장에 겹친다: outer optimizer(AdamW, weight decay

$\lambda = 0.1$) 는 Θ 를 학습하는 통상의 훈련이고, inner optimizer(식 (12-4)) 는 모델 그 자체다. 기호로도 구분된다 — outer 는 무침자 상수 (η, λ), inner 는 시간 침자게이트 ($\eta_t, \beta_t, \alpha_t$).

inner loop: inference 에서 실제로 움직이는 것. sequence 마다 진화하는 tensor 는 단 둘, W_t 와 S_t 다. 나머지 전부는 동결이다. per-token 비용을 세어 보자.

[해설] P_M 을 memory parameter 수라 하면 (폭 d MLP 에서 $P_M \approx L_M d^2$), 단위를 MAC 로 통일해 세면: write 한 번 = k/v projection $2d^2$ MAC + M forward $\approx P_M$ MAC + weight gradient 를 위한 backward $\approx 2P_M$ MAC(backprop 의 표준 $2 \times$ forward 비용; linear memory 에서는 rank-1 outer product $(Wk_t - v_t)k_t^\top$ 로 퇴화) + S, W 의 elementwise 갱신 $\approx 2P_M$ 연산 + 게이트 head 3 개 (미미). read 한 번 = projection + forward $\approx d^2 + P_M$ MAC. 합계 token 당 $\approx 4-5 \times P_M$ MAC(1 MAC = 2 FLOP 이므로 **FLOP** 로는 약 $8-10 \times P_M$) + projection·elementwise — **context 길이와 무관한 상수**이며, 같은 state 크기의 linear-attention/DeltaNet layer 대비 작은 상수배, 그 상수는 L_M 에 선형이다 (10 장 표 10-2 의 $\approx 12P_M$ 과 같은 자릿수). 이 산정은 이 책의 계산이고, 논문의 실측은 “LMM 이 Mamba-2/Gated DeltaNet 보다 약간 느리다” [Titans Fig. 9] 까지다.

여기서 독자의 세계가 실제로 뒤집히는 지점을 명시한다: **backward pass 가 decode 안으로 들어온다**. decode = “read-only forward + KV append” 라는 불변식은 이 라인에서 폐기되고, decode 한 step 은 forward + backward + optimizer step + read(식 (12-5)) 가 된다. 다만 주의할 것이 있다: 훈련은 $C > 1$ 의 chunk-start anchor 로 stale gradient 를 쓰고 decode 는 $C = 1$ online write 를 쓰는데, C 는 계산되는 함수를 바꾸는 semantic hyperparameter 이므로 ($\rightarrow$ 9 장) $C_{\text{train}} \neq C_{\text{decode}}$ 이면 두 궤적은 일반적으로 다르다 — 이 train/serve mismatch 가 § 12.8 한계 3 이자 [TNT] 의 출발점이다 ($\rightarrow$ 15 장). 일치시키려면 decode 도 훈련과 같은 chunk anchor 를 순차로 유지해야 한다. prefill 은 훈련 때의 chunkwise 공식 그대로긴 prompt 를 chunk 병렬로 흡수하고, decode 가 outer gradient 를 함께 계산하지 않는다는 점만 훈련과 다르다. 훈련의 batch 축 역할을 sequence 축이 대신한다는 것 (inner loop 의 mini-batch = chunk) 도 여기서 처음 실물로 확인된다 — 2 장과 9 장이 예고한 최대 혼동 지점이다.

변형별 흐름: LMM/MAG/MAL 은 (prefix 가 붙은) 스트림의 모든 token 에서 memory 를 갱신하고, MAC 은 segment 단위로 “읽기 $\rightarrow$ attention $\rightarrow$ attention 출력만 쓰기” 를 반복한다. test time 에 persistent parameter 는 동결 (task 지식), attention weights 는 window 안의 in-context learner, LMM 만이 “여전히 학습 중” 이다 [Titans § 4.1].

12.5 Concept ledger delta

[Titans] 는 이 라인의 시조이므로 폐기하는 개념은 없고, ledger 의 첫행들을 만든다. 아래 표는 누적 관점 — 각 개념이 이후 어느 장에서 어떻게 진화하는지 — 을 함께 적는다.

표 12-3 — [Titans] 가 ledger 에 추가한 개념

개념 (이 논문이 도입)	내용	이후의 운명
surprise (momentary / past)	momentary = g_t^{in} , past = S_t	Miras: 임의 attentional bias 의 gradient 로 일반화 ($\rightarrow$ 13 장); Atlas: 문맥의 surprise(window) 로 ($\rightarrow$ 14 장); NL: Local Surprise Signal($\rightarrow$ 16 장)

개념 (이 논문이 도입)	내용	이후의 운명
momentum-as-memory (정의 소유: 2 장 · 16 장)	optimizer state S_t 의 recurrent state 승격; linear-recurrent 계열 최초의 momentum rule 주장 [Titans App. C]	Miras 의 출시 모델들은 제거; Atlas 가 Muon 으로 부활 (→ 14 장); NL 이 정리로 격상 (→ 16 장)
weight decay = per-token retention	$\alpha_t W_{t-1}$; Mamba-2/GLA/ Gated DeltaNet gate 의 deep-memory 일반화	Miras 가 retention gate 로 공식 재이론화 (→ 13 장)
deep memory	$L_M \geq 2$ MLP; linear memory = 선형 압축가정의 거부	2-layer residual MLP 가 라인 표준으로 (→ 13 장); capacity 이론은 Atlas(→ 14 장)
persistent memory	N_p 개 학습 prefix token; task 지식의 input-independent 저장	NL/CMS 에서 frequency-0 극점으로 재해석 (→ 16 장)
contextual memory	내용이 입력 문맥에 의존하는 memory 의 총칭 (LMM + attention KV)	persistent 와의 이분법이 NL 의 frequency 연속체로 확장 (→ 16 장)
MAC / MAG / MAL	합성 3 형; MAL(관행 hybrid) 의 열등 판정	Atlas·NL 의 hybrid 실험이 재사용 (→ 14, 16 장)
intra-chunk linear / inter-chunk nonlinear recurrence	병렬성과 표현력의 협상 지점; Thm 4.1 의 근거	TNT 가 reset 으로 이 비선형 사슬을 끊어 context parallelism 획득 (→ 15 장)
meta-learned W_{init} (암묵)	W_0 이 outer 학습대상이라는 함의; v1 은 명시하지 않음	TNT 에서 load-bearing 으로 승격 (→ 15 장), 개념 소유는 4 장 · 15 장
chunk-당-상수 게이트 (LTI fast path)	제안만 되고 미평가	라인어디에서도 재평가되지 않은 열린 갈래

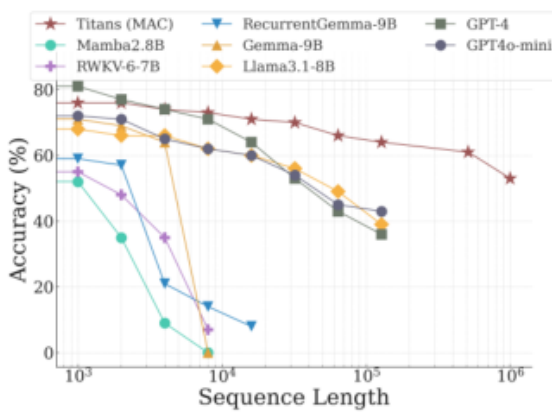
표기 차원의 유산도 있다: Titans 의 forget-방향 α_t 는 [Miras] 가 retention 방향으로 뒤집어 재정의하며, 이 책은 처음부터 Miras 방향 (남기는 비율) 으로 통일했다 (표 12-1).

12.6 실험과 스케일

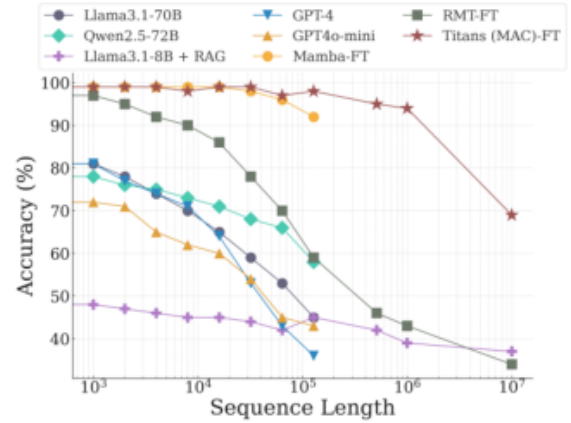
설정 [Titans § 5.1]: 170M/340M/400M 모델은 FineWeb-Edu 15B tokens, 760M 은 30B tokens 로 훈련. 훈련 길이 4K. baseline 은 Transformer++, RetNet, GLA, Mamba, Mamba-2, DeltaNet, TTT, Gated DeltaNet 과 hybrid(Samba, Gated DeltaNet-H2). 400M baseline 수치는 재실행이 아니라 Gated DeltaNet 논문의 보고치를 재사용했다 [Titans App. B].

Language modeling + commonsense reasoning [Titans Table 1]. 모든 스케일에서 LMM 단독이 비-hybrid baseline 전부를 이긴다. 340M: LMM 평균 46.17 vs Gated DeltaNet 45.42, TTT 44.51 — 논문은 TTT 와의 격차를 momentum+forgetting 과, Gated DeltaNet 과의 격차를 deep nonlinear memory 와 연결짓는다 [Titans § 5.2]. 다만 이 비교들은 objective·architecture·parameterization 이동시에 다른 비통제 비교이므로 개별 성분의 인과 효과를 식별하지 못한다 — 성분별 순수 기여는 아래 Table 5 의 통제된 ablation 이 측정하는 범위에서만 주장할 수 있다. 760M: LMM Wiki ppl 20.04 / 평균 51.56 vs Gated DeltaNet 21.18 / 49.69. hybrid 에서는 MAC/MAG/MAL 셋 모두 Samba 와 Gated DeltaNet-H2 를 이긴다 — 760M 에서 MAG Wiki ppl 18.61, MAC 평균 52.51 vs Gated DeltaNet-H2 19.88 / 51.49. 일관된 순서는 $MAC \approx MAG > MAL$ 이며, 모듈이 같고 배치만 다르므로 이 격차는 순수하게 합성 설계의 몫이다 [Titans § 5.2].

Model	S-NIAH-PK				S-NIAH-N				S-NIAH-W			
	2K	4K	8K	16K	2K	4K	8K	16K	2K	4K	8K	16K
TTT	98.4	98.8	98.0	88.4	60.2	36.6	10.2	4.4	78.8	28.0	4.4	0.0
Mamba2	98.6	61.4	31.0	5.4	98.4	55.8	14.2	0.0	42.2	4.2	0.0	0.0
DeltaNet	96.8	98.8	98.6	71.4	47.2	15.4	12.8	5.4	46.2	20.0	1.6	0.0
Titans (LMM)	99.8	98.4	98.2	96.2	100.0	99.8	93.4	80.2	90.4	89.4	85.8	80.6
Titans (MAC)	99.2	98.8	99.0	98.4	99.6	98.2	97.6	97.4	98.2	98.2	95.6	95.2
Titans (MAG)	99.4	98.0	97.4	97.4	99.2	98.8	97.2	98.6	98.0	98.0	90.2	88.2
Titans (MAL)	98.8	98.6	98.8	97.8	99.8	98.1	96.8	96.4	98.0	97.4	92.0	90.4



(a) Few-shot Setup



(b) Fine-Tuning Setup

그림 12-3 — (위) S-NIAH(RULER) 정확도 표: Titans 계열이 2K – 16K 전 구간을 유지하는 반면 Mamba-2/DeltaNet 은 N·W 하위과제에서 붕괴한다. (아래) BABILong 정확도 대 sequence 길이 — (a) few-shot 에서 Titans(MAC) 가 GPT-4 등 훨씬 큰 모델을 앞서고, (b) fine-tuning 에서 작은 MAC 가 10^6 token 너머까지 정확도를 유지한다. 출처: Behrouz et al., Titans (arXiv: 2501.00663), 원문 Fig.6(및 Table 2) — 원저자의 그림 (third-party), 본서의 결과가 아님.

S-NIAH (RULER, 2K – 16K) [Titans Table 2]. 기제별 귀속이 가장선명한 실험이다. Titans 계열은 전 구간 80 – 99%를 유지한다 (MAC PK-16K 98.4, MAG N-16K 98.6). 대조: Mamba-2 는 PK-16K 5.4, W-8K/16K 0.0 으로 붕괴 — erase 는 있으나 얇은 state 로는 부족하다; DeltaNet 은 PK-16K 71.4 까지 버티지만 N/W 에서 무너진다 — replace 는 해도 진짜 erase(forgetting) 가 없다; TTT 는 16K 에서 처진다 (PK-16K 88.4) — retention gate 부재 [Titans § 5.3]. 즉 momentum+forgetting 이 TTT 를, deep nonlinear memory + erasure 가 Mamba-2 를, forgetting 이 DeltaNet 을 각각이기에 만든 성분이라는 **가설**과 표의 패턴이 일관된다 — 단 이 표 자체는 성분별 ablation 이 아니라 서로 다른 모델의 비교이므로, 인과 귀속은 Table 5 의 통제 ablation 이 뒷받침하는 범위로 한정된다.

BABILong [Titans Fig. 6]. few-shot 설정에서 Titans (MAC) 는 Mamba-2.8B, RWKV-6-7B, RecurrentGemma-9B, Gemma-9B, Llama3.1-8B, GPT-4, GPT-4o-mini 를 모두 이긴다 — 훨씬 적은 parameter 로. fine-tuning 설정에서는 작은 MAC 가 fine-tune 된 RMT·Mamba, RAG 를 단 Llama3.1-8B(약 $70\times$ 더 많은 parameter [Titans § 5.4]), 그리고 GPT-4, Qwen2.5-72B, Llama3.1-70B 를 넘어서며 2M tokens 너머까지 정확도를 유지한다. 이 라인의 “>2M context” 헤드라인은 전부 이 실험 (MAC, fine-tuned, baseline 수치는 벤치마크 저자 보고) 에 얹혀 있다 (그림 12-3).

memory 깊이 [Titans Fig. 7, Fig. 8]. Pile 부분집합, 170M/360M/760M 에서 $L_M = 1, 2, 3, 4$ 비교: 깊을수록 전 길이에서 perplexity 가 좋고 길이에 강건하며 (효과는 작은스케일에서

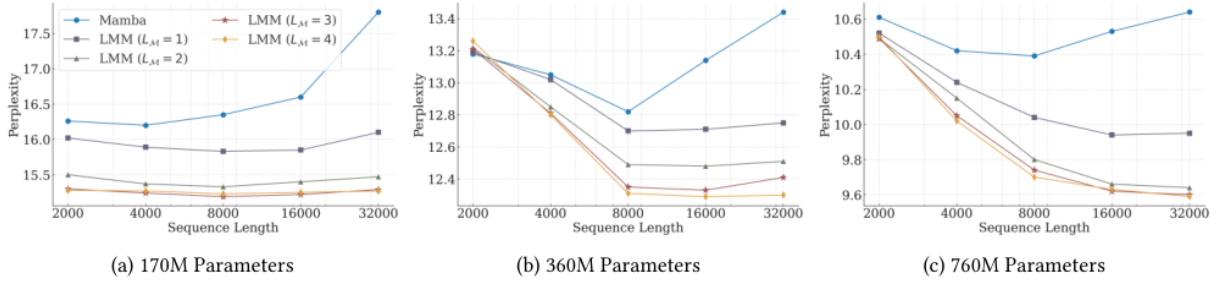


그림 12-4 — memory 깊이 $L_M \in \{1, 2, 3, 4\}$ 가 perplexity에 주는 효과 (170M/360M/760M). 깊은 memory 일수록 모든 sequence 길이에서 perplexity가 낮고, 특히 길이가 늘때의 열화가 완만하다 — linear memory($L_M = 1$)의 선형 압축가정을 넘어서는 이득이 § 12.3.6 deep-memory 논거의 실증이다. 비교 baseline은 Mamba. 출처: Behrouz et al., Titans (arXiv:2501.00663), 원문 Fig.7 —원저자의 그림 (third-party), 본서의 결과가 아님.

최대), 대가는 훈련 throughput의 선형 하락이다. 모든 깊이에서 tokens/sec은 sequence 길이에 대해 일정 — 즉 훈련 비용은 길이에 선형이다 (그림 12-4).

효율 [Titans Fig. 9]. LMM은 Mamba-2/Gated DeltaNet보다 약간 느리다 — deep memory의 본질 비용에 더해 fused kernel 부재가 원인으로 지목된다. MAL이 전체에서 가장 빠르는데, 이는 FlashAttention의 성숙도 덕이다 [Titans § 5.8].

ablation [Titans Table 5]. base LMM: ppl 27.01 / reasoning 47.83 / long-context 92.68. 성분 제거 시 ppl: weight decay 제거 29.04(최악), momentum 제거 28.98, convolution 제거 28.73, deep → linear memory 28.49(long-context는 85.34로 급락), persistent memory 제거 27.63. attention을 붙이면 전부개선: MAC 26.67/48.65/97.95, MAG 25.70/48.60/96.70, MAL 25.91/47.87/96.91. 기여 순위: weight decay > momentum > convolution > persistent memory [Titans § 5.9].

[해설] 원문은 § 5.9에도 Table 5 캡션에도 이 ablation의 모델규모를 명시하지 않는다. 다만 Table 5의 ppl 열은 Table 1의 Wiki·LMB perplexity 평균과 자릿수까지 일치하므로 (예: LMM 400M — $(25.03 + 28.99)/2 = 27.01$; MAC $(25.61 + 27.73)/2 = 26.67$; 4행모두 확인), ablation은 400M 스케일 수치로 읽는 것이 정합적이다.

보조 도메인: time series forecasting에서는 Simba 프레임워크의 Mamba 자리에 LMM을 넣어 ETT/ECL/Traffic/Weather에서 최고 MSE/MAE를 보고하고 (예: ETTm1 0.358/0.387 vs PatchTST 0.387/0.400) [Titans Table 3], DNA modeling(GenomicsBenchmarks)에서는 HyenaDNA·Based·Mamba와 경쟁 수준이다 (Enhancer Cohn 75.2 최고) [Titans Table 4].

스케일의 정직한 자리매김. 이 논문의 증거는 전부 $\leq 760M$ parameters / $\leq 30B$ tokens / 훈련 길이 4K에서 나왔다. v1 각주는 “더 큰 모델의 결과를 다음 버전에 싣겠다”고 약속하지만 [Titans § 5], 이 라인 여섯 편 전체의 실증 상한은 끝내 1.3B params / 100B tokens에 머문다. 또한 위의 모든 효율 수치는 훈련 throughput이다 — decode wall-clock 수치는 이 논문에도, 라인 여섯 편 어디에도 없다.

12.7 Systems/serving 합의

state 수학: KV cache와의 교환. sequence 당 LMM layer 하나의 recurrent state = memory weights + momentum buffer = $2P_M$ 개의 수다. 아래 비교가 이 논문의 존재 이유를 요약한다.

표 12-4 — state 크기: LMM vs KV cache (bf16, layer 당 · sequence 당) **[해설]** — 수치는 이 책의 계산

항목	크기 식	$d = 1024$, 2-layer memory, $L = 2M$ tokens
LMM state ($W_t + S_t$)	$2P_M \approx 2L_M d^2$	약 4M 값 $\approx$ 8MB — 길이 불변
attention KV cache	$2Ld$	약 4G 값 $\approx$ 8GB — 길이 비례

2M-token 문맥에서 attention의 KV cache는 layer 당 GB 단위로 자라 사정권밖이지만 LMM state는 그대로다 — BABILong의 >2M 능력은 정확히 교환 위에서 있다. 대가도 명확하다: momentum buffer가 LMM 자신의 fast-weight state를 2배 (W_t 에 같은 크기 S_t 가 붙는다)로 만들고 — 같은 폭 Gated DeltaNet의 단일 $d \times d$ state 대비로는 약 $2L_M$ 배 (표의 2-layer 가정에서 약 4배)이며 — 그 buffer는 decode step 사이에 반드시 이월되어야 한다. MAC에서는 attention이 한 segment(C tokens) + N_p persistent + 검색된 history token 만보므로 attention 쪽 KV cache가 segment 크기로 유계이고, MAG/MAL에서는 SWA window가 그 역할을 한다.

decode의 재정의와 state 트래픽. decode 한 step은 이제 read-only lookup이 아니라 $2P_M$ 전체에 대한 read-modify-write다: W_t 와 S_t 를 읽고, elementwise로 갱신해, 도로 쓴다. token 당 FLOPs가 $O(P_M)$ 상수 (§ 12.4)인 동시에 token 당 state 트래픽도 $O(P_M)$ 이므로, decode에서 이 layer의 arithmetic intensity는 낮은 상수에 고정된다 — KV cache 스트리밍이 지배하던 자리에 weight-state RMW 스트리밍이 들어선 것뿐이라는 점에서 독자에게 익숙한 memory-bound 그림이지만, 이제 그 트래픽이 문맥 길이와 무관하게 일정하다는 점이 다르다. 정량적 roofline 분석은 10 장이 맡는다.

병렬화와 kernel. prefill은 훈련 시의 chunkwise 공식 (식 (12-7) – (12-9))을 그대로 써서 prompt를 chunk 병렬로 흡수한다 — prefill = 큰 chunk의 병렬 write라는 Rosetta 대응이 문자 그대로 성립한다. MAC의 attention은 segment 별 block-diagonal mask이므로 [Titans Fig. 3a] $C + N_p + N_l$ 크기 window들에 대한 FlashAttention이고 총비용 $O(L \cdot C)$, segment 간 의존은 memory의 순차 사슬뿐이다. kernel 현실: 발표 시점의 LMM에는 fused kernel이 없고 (batched MLP forward + backward + scan + AXPY를 묶은 chunk kernel이 필요), 그런데도 Mamba-2 급 throughput의 사정권에 있다 [Titans Fig. 9]. MAL이 가장 빠른 것은 순전히 FlashAttention의 성숙도다 — 즉 “MAL vs MAC/MAG”는 오늘 측정되는 throughput과 품질의 교환이며, LMM kernel이 성숙하면 순위가 뒤집힐 수 있는 sociotechnical한 순위다.

batching의 붕괴와 serving 신질문. shared weights 전제가 깨진다: request마다 W_t, S_t 가 다르므로 naive multi-tenant batching은 per-request weight 사본을 요구하고, batched decode는 shared-weight GEMM이 아니라 per-sample weights의 grouped GEMM/bmm workload가 된다 — TTT가 이미 강제한 패턴이지만 state가 MLP 전체 + momentum이라 더 무겁다. retention gate α_t 는 **학습된 eviction policy**다: paged-KV 같은 runtime 관리 기제는 사라지지만, 무엇이 언제 지워지는지가 runtime에 불투명해진다. state가 optimizer 궤적의 산물이므로 accumulation 순서·scan 정밀도 같은 numerics가 “cache” 자체를 표류시킬 수 있고 — KV cache에는 없던 결정성 문제 — speculative decoding의 rollback은 (W_t, S_t) 궤적의 snapshot/복원이라는 비자명한 연산이 된다. test-time weight 변이는 multi-tenant 격리와 prompt privacy(문맥이 weights에 흡수된다) 질문도 연다. 논문은 이 중 무엇도 다루지 않는다.

update frequency 관점의 예고. 이 아키텍처에는 이미 세 개의 갱신주기가 공존한다: 매 token 갱신되는 W_t (가장 빠름), window 안에서만 유효한 attention KV, 그리고 절대 갱신되지 않는 P 와 Θ (frequency 0). 어떤 상태를 얼마나 자주 갱신하며 memory 계층 어디에 둘 것인가라는 질문의 원형이 여기 있고, 이 삼분법을 연속체로 퍼는 것이 16 장 (NL)의 일이다.

마지막으로 의무 caveat를 반복한다: 이 논문을 포함해 라인 전체에 decode wall-clock 수치가 없다. 위 문단의 serving 논의는 구조에서 연역한 것이지 측정이 아니다.

12.8 한계와 bridge-out

논문 스스로 인정하거나 본문 검토에서 드러나는 한계를 모은다.

1. **point design**이다. 왜 ℓ_2 regression 인가, 왜 SGD+momentum+decay 인가에 대한 논거가 없다. Appendix C가 기존 모델전부를 특수 사례로 회수하는 순간, “그 축들 위의 다른 점

은?” 이라는 질문이 자동으로 생긴다. 논문도 더 나은 inner objective 와 inner optimizer, memorization 특화 아키텍처 (Memory Mosaics 등) 를 명시적 future work 로 남긴다 [Titans § 3.1, App. C].

2. **미명시 세부가 많다.** 게이트 $\eta_t, \beta_t, \alpha_t$ 의 함수형 · 초기화, memory MLP 폭 · activation, chunk 크기 C 와 MAC segment 크기의 실험값, W_{init} 의 설정, chunk 경계에서 momentum buffer 의 훈련/decode 시 처리 — v1 에 없다. 학습된 optimizer 궤적이 수백만 step 동안 안정적이라는 보장도 없다 ($\|W_t\|$ 를 묶는 것은 학습된 decay 뿐이다).
3. **근사의 비용이 미측정이다.** chunk 안 gradient 는 stale 하다. C 에 대한 ablation 이 없고, 훈련 시 chunk 단위 갱신과 decode 시 token 단위 갱신의 정합 (train/serve 일치) 문제도 제기되지 않는다. staleness 의 오차 한계는 이 논문은 물론 라인 여섯 편 어디에도 없다.
4. **Thm 4.1 은 증명이 없다** (§ 12.3.8). 표현력 주장은 v1 에서 단언이다.
5. **스케일과 kernel.** $\leq 760\text{M}/30\text{B}$ 의 증거로는 momentum 과 deep memory 의 우위가 7B+ · 최적화된 DeltaNet 계열 kernel 앞에서도 유지되는지 알 수 없다. 약속된 “다음 버전” 의 대형 모델과 코드 공개 (PyTorch/JAX) 는 v1 시점에 없다.
6. **outer 훈련의 memory 비용.** 4K 문맥의 unrolled inner loop 를 backprop 하려면 chunk 별 inner state/activation 을 저장해야 하는데, activation memory 회계나 gradient-checkpointing 레시피가 없다 — 문맥이나 $L_{\mathcal{M}}$ 을 키울 때 실질 병목이 될 수 있는 항목이다.

Bridge-out → 13 장 [Miras]. [Titans] 가 남긴 가장 생산적인 유산은 결핍이 아니라 회수다. Appendix C 가 “Gated DeltaNet = $\beta_t=0$, Longhorn = implicit step + no-forget, TTT = no-forget no-momentum, RWKV-7 = 같은 loss” 를 보인 순간, 이 계보 전체가 하나의 설계공간의 점들이 드러났다 — 그런데 그 공간의 좌표축이 무엇인지는 [Titans] 가 명명하지 않았다. 모두가 ℓ_2 (또는 dot-product) objective 와 ℓ_2 retention 만 쓰고 있었다는 관찰, 그리고 “objective / retention / 아키텍처 / 알고리즘” 이라는 4 축의 명명이 [Miras] 의 출발점이다. 그 과정에서 [Miras] 의 출시 모델들은 [Titans] 의 자랑인 momentum 을 오히려 떼어내고 (단순 GD 로 회귀), optimizer 측은 비워 둔 채 후속작 (Atlas) 에 넘긴다 — 시조의 point design 이 분해되어 좌표계가 되는 과정을 다음 장에서 본다.

ch13. Miras: It' s All Connected — sequence model 설계 공간의 통일 이론

13.1 Bridge-in: Titans가 남긴 질문 — “왜 하필 그선택인가”

12 장의 [Titans] (Titans: Learning to Memorize at Test Time, arXiv:2501.00663) 는 하나의 point design 을 출하했다. deep MLP memory 의 weights 를 token 마다 gradient descent 로 갱신하되, data-dependent momentum(“past surprise”) 과 weight decay(“forgetting”) 를 붙이고, inner objective 는 ℓ_2 associative regression 으로 고정하며, attention 과는 MAC/MAG/MAL 세 방식으로 합성한다 (→ 12 장). 그리고 Titans 원문 스스로가 자기 설계의 일반성을 암시했다: [Titans App. C] 는 Gated DeltaNet(이하 GDN), Longhorn, RWKV-7, TTT 가 모두 Titans update 식의 특수 사례임을 보인다. 그런데 바로 그 지점에서 논문이 끝난다 — 특수 사례가 있다면 일반형이 있을 텐데, 그 설계 공간이 무엇인지, 각 축에서 Titans 의 선택이 왜 좋은지는 답하지 않았다.

구체적으로 세 가지 질문이 열려 있었다. 첫째, 왜 ℓ_2 인가? inner loss $\ell(W; k_t, v_t) = \|\mathcal{M}(k_t; W) - v_t\|_2^2$ 는 관행이지 논증이 아니다. 둘째, 왜 momentum + weight decay 인가? inner optimizer 로 SGD with momentum 을 쓴 것 역시 outer 세계의 관행 이식이며, 다른 optimizer 가 더 나은지는 묻지 않았다. 셋째, “forgetting” 이 좋은 추상인가? Titans 는 α_t 를 forgetting mechanism 이라 불렀지만, gate 가 실제로 하는 일이 무엇인지에 대한 이론은 없었다. Titans 는 “ ℓ_2 regression 너머의 inner objective” 와 “더 나은 inner optimizer” 를 명시적인 open problem 으로 남겼다.

이 장의 논문 [Miras] (It' s All Connected: A Journey Through Test-Time Memorization, Attentional Bias, Retention, and Online Optimization, arXiv:2504.13173, Behrouz, Razaviyayn, Zhong, Mirrokni, Google Research, 2025-04) 는 이 질문에 새아키텍처가 아니라 분류학으로 답한다. 논문의 실제 제목은 It' s All Connected 지만, 이 책은 논문이 제안한 framework 의 이름인 **Miras** 로 통칭한다. Miras 는 “유산 (Legacy)” 을 뜻하는 단어로, 후속연구가 밭을 설계 단계를 물려준다는 의도의 명명이다 [Miras 각주 1].

13.2 문제의식과 논문의 핵심주장

Miras 가 정리한 지형은 이렇다. Transformer 는 in-context learning 과 스케일링 능력으로 SOTA 지만 quadratic 시간과 선형 증가하는 KV cache 가 long-context 적용을 막는다. efficient recurrent 대안들은 context 를 고정크기 state 로 압축하며, 이 계열은 세 개의 경험적 전선에서 개선했다 [Miras § 1]: (1) learning rule — Hebbian rule 에서 delta rule 로 (→ 5 장, 6 장); (2) (당시 명칭) forget gate — LSTM 에서 Mamba-2 를 거쳐 Titans 의 gate 로; (3) memory 구조 — RetNet 류의 vector memory 에서 Titans/TTT 의 deep memory 로. 그러나 이 세 전선을 관통하는 설계 프레임은 없었다. 기존의 통일 시도들 — Mamba-2 의 SSD framework(→ 7 장), Longhorn 의 online-learning 관점, Sun et al. 2024 의 TTT regression 관점 (→ 8 장), 그리고 동시기 작업인 Wang et al. 2025 의 test-time regression framework — 은 각각 일부만 포괄하거나, forget gate(당시 명칭) 를 표현하지 못하거나, 모든 모델을 단일 regression objective 에강제로 끼워 넣어 Hebbian 과 delta 의 실질적 차이를 지워버린다 [Miras § 2]. 특히 Wang et al.

에 대해 Miras 는 부록에서, RetNet/Mamba 를 regression solver 로 근사해야만 하고 HGRN2·Moneta·Yaad·Memora 는 아예표현할 수 없다고 반박한다.

이 지형 위에서 Miras 의 주장은 두 개의 관찰과 하나의 프레임으로 요약된다.

관찰 1 — 모두가 같은 objective 를 쓰고 있었다. 거의 모든 기존 sequence model 은 associative memory 이며, 그 내부 objective 는 단 두 종류— dot-product similarity 아니면 ℓ_2 regression — 뿐이다 [Miras Table 1]. 이 내부 objective 를 논문은 **attentional bias** 라명명한다: 인지과학에서 특정 자극을 우선시하는 성향을 가리키는 용어를 빌려, “이 memory 가 무엇을 우선해 기억하는가” 를 정의하는 loss 로 정식화한것이다.

관찰 2 — forgetting 은 존재하지 않는다. retention 이 있을 뿐이다. 기존 forget gate 들은 전부 ℓ_2 류 regularization 의 특수형이며, gate 가 하는 일은 “지우기” 가 아니라 “새 association 을 배우는 것과 이전 state 에 머무르는 것 사이의 trade-off” 다. 모델은 memory 를 소거하는 것이 아니라 유지하지 않기로 결정할 뿐이며, 이는 뇌가 기억을 지우는 것이 아니라 retrieval failure 로 접근 불가능해진다는 신경과학의 관점과 일치한다 [Miras Remark 3]. 그래서 논문은 forget gate 를 **retention gate** 로 개명한다 — 이 책이 이 용어를 공식 명칭으로 채택한 근거가 이 절이다 (역사적 별칭 “forget gate” 는 이 문장 한 번으로 병기를 마친다).

프레임 — Miras framework. 위 관찰은 sequence model 설계를 네개의 독립적인 축으로 분해한다: (i) memory architecture, (ii) attentional bias, (iii) retention gate, (iv) memory learning algorithm. 기존 모델전부가 이 4-tuple 의 한 점이고, 지금까지의 발전은 이 네 축을 설계 축으로 인식하지 못한 채 주로 축 (i)(architecture) 와 축 (iii)(retention) 를 움직였다 — 축 (ii) 는 dot-product/ ℓ_2 두 선택지사이에서만, 축 (iv) 는 Longhorn·DeltaProduct 정도로만 제한적으로 탐사했다. 논문은 축 (ii) 와 (iii) 에 새 선택지를 채워 넣은 세모델 — **Moneta, Yaad, Memora** — 를 출하해프레임의 생산성을 실증하고, 1.3B/100B tokens 스케일에서 attention-free 순수 recurrent 모델이 attention hybrid 까지 이기는 결과를 headline 으로 내세운다 [Miras § 6.1].

이 장의 재구성 관점에서 말하면: Titans 가 “optimizer 를 sequence layer 로만들 수 있다” 는 존재 증명이었다면, Miras 는 그 move 가 열어놓은 공간의좌표계이며, 이후 장들의 논문이 전부 이 좌표계 위에서 자신의 위치를서술한다.

13.3 Core mechanism (통일표기)

13.3.1 Associative memory 의 재정식화와 attentional bias

Miras 의 출발점은 5 장의 고전적 associative memory 를 optimization 문제로다시 정의하는 것이다 — Hopfield energy 와 capacity 의 관점 대신, **무엇을 최소화하는 객체인가가 정의의 전부다.**

Definition (associative memory 와 attentional bias) [Miras Def. 3.1]. key 집합 $\mathcal{K} \subseteq \mathbb{R}^{d_k}$ 와 value 집합 $\mathcal{V} \subseteq \mathbb{R}^{d_v}$ 가 주어질 때, associative memory 는 operator $\mathcal{M} : \mathcal{K} \rightarrow \mathcal{V}$ 이고, 그 mapping 의 학습은 objective L — **attentional bias** — 의최소화다:

$$\mathcal{M}^* = \arg \min_{\mathcal{M}} L(\mathcal{M}(K); V). \quad (13-1)$$

여기서 k_t, v_t 는 attention 에서와 정확히 같은 방식으로 입력 token 의 linear projection 으로 만들어진다 ($k_t = W_K x_t, v_t = W_V x_t$). $\mathcal{M}^*$ 는 argmin 의 최적해를 가리키는 전용 기호이며 (§ 1.2 예약), Titans 의 read-only pass 표기와 무관하다. memory 를 parameter W 로 매개화하면 최적화는 W 위에서 수행되고, 과거 데이터의 retention 을 제어하는 regularizer $R(W)$ 를 추가할 수 있다 [Miras Remark 1]. 이후 per-token 축약형으로 $\ell(W_{t-1}; k_t, v_t) := L(\mathcal{M}(k_t; W_{t-1}), v_t)$ 를 쓴다.

결정적인 것은 [Miras Remark 2] 다: 식 (13-1) 의 학습은 **meta-learning / bilevel** 문제다. attentional bias 는 **inner loop** 에서 — 즉 test time 에, token 마다 — 최적화되고, 나머지 모든 parameter(projection, convolution, gate 생성층) 는 **outer loop** 에서 통상의 pre-training 으로 최적화된다 (→ 4 장). 이 한 문단이 이라인 전체의 두-loop 구조를 논문 안에서 처음으로 명시적 정의로 못박은지점이며, 이 장 § 13.4 에서 전면 전개한다.

식 (13-1) 을 online 으로 푸는 가장 단순한 방법은 새 (k_t, v_t) 쌍이 도착할 때마다 gradient descent 한 걸음을 걷는 것이다:

$$W_t = W_{t-1} - \eta_t \nabla_W \ell(W_{t-1}; k_t, v_t),$$

즉 표준형 (M1) 그대로다 [Miras Eq. 5]. Titans 가 momentary surprise 라 불렀던 양 ($\rightarrow$ 12 장) 은 이 관점에서는 그냥 임의의 attentional bias 에 대한 online GD 의 gradient 다 — surprise 의 일반화가 여기서 완료된다.

13.3.2 두 개의 optimization 관점: FTRL 과 Learning – Retaining

Miras 의 이론적 척추는 (M1) 을 해석하는 두 관점이다. 이 두 관점이 있어야 “retention gate 가 정확히 무엇인가” 가 정의되고, 새 gate 의 유도가기계적으로 가능해진다.

관점 1 – FTRL viewpoint. (M1) 은 loss 열 $\ell(W; k_1, v_1), \ell(W; k_2, v_2), \dots$ 위의 online gradient descent 한 걸음이고, OGD 는 FTRL($\rightarrow$ 3 장) 의 특수 사례다. $W_0 = 0$ 일 때 (M1) 은 다음과 동치다 [Miras Eq. 7]:

$$W_t = \arg \min_{W \in \mathcal{W}} \underbrace{\sum_{i=1}^t \hat{\ell}_i(W; k_i, v_i)}_{\text{attentional bias}} + \underbrace{\frac{1}{\eta_t} R_t(W)}_{\text{memory stability}}, \quad (13-2)$$

고전형에서는 $\hat{\ell}_i(W) = \langle W - W_{i-1}, \nabla_W \ell(W_{i-1}; k_i, v_i) \rangle$ (각 시점 loss 의 국소 선형화) 이고 $R_t(W) = \frac{1}{2} \|W\|_2^2$ 다. 첫 항은 **모든 과거 token** 을 잘 기억하는가를 재고, 둘째 항은 memory 의 크기를 벌한다. 주의: (M1) 과 FTRL 의 **정확한** 동치는 $\eta_t = \eta$ 상수일 때다 ([Miras Eq. 7] 도 상수 η 와 $\frac{1}{2\eta} \|W\|^2$ 를 쓴다). data-dependent η_t 에서는 $\frac{1}{\eta_t} R_t$ 를 단일 계수로 앞에 둔 (13-2) 는 variable-step OGD($W_t = -\sum_i \eta_i g_i$) 를 정확히 재현하지 못하고 ($-\eta_t \sum_i g_i$ 가 된다) 일반화된 FTRL 관점으로만 성립한다 — 정확히 맞추려면 η_i 가중을 선형화 loss 항 안에 넣어야 한다. $\hat{\ell}_i$ 와 R_t 를 일반화하면 mirror descent 류 알고리즘이 나온다 — 이 자리가 뒤에서 dual-accumulator 형 update(Moneta) 가 태어나는 자리다.

관점 2 – Learning – Retaining viewpoint. 같은 (M1) 을 “최신 쌍을 배우되 이전 state 근처에 머무르기” 로 읽을 수도 있다:

$$W_t = \arg \min_{W \in \mathcal{W}} \underbrace{\tilde{\ell}_t(W; k_t, v_t)}_{\text{learning}} + \underbrace{\text{Ret}_t(W, W_{t-1})}_{\text{retention}}, \quad (13-3)$$

여기서 $\tilde{\ell}_t$ 는 ℓ 의 근사 (선형화 등) 이고, retention 항은 **local** 과 **global** 로 분해된다 [Miras § 3.3]:

$$\text{Ret}_t(W, W_{t-1}) = \frac{1}{\eta_t} D_t(W, W_{t-1}) + \frac{1}{\alpha_t} G_t(W). \quad (13-4)$$

local retention D_t 는 W_{t-1} 로부터의 이탈을 벌하는 premetric 으로, 이미 배운 것을 유지하는 힘이다. 그 계수 η_t 를 논문은 **meta in-context learning rate** 라 부른다: η_t 가 크면 새것을 많이 배우고 옛것을 많이 놓아준다. **global retention** G_t 는 state 자체의 크기를 제한한다 — 고전적 gate 가 사는 곳이 바로 이항이다. 두 관점의 관계는 다음이 정리한다.

Proposition [Miras Prop. 3.2, 증명은 App. B]. $\eta_t = \eta$ 상수, $\mathcal{W} = \mathbb{R}^d$ 이고, $h_t(W) := \sum_{i=1}^{t-1} \hat{\ell}_i(W) + \frac{1}{\eta} R(W)$ 가 strictly convex 라 하자. Ret_t 를 h_t 의 Bregman divergence 로 놓으면 Learning – Retaining update 는 FTRL update 와 정확히 일치한다.

즉 Learning – Retaining 이 더 일반적인 렌즈이고, 논문의 모든 유도가 이관점에서 나온다. 단, 정직하게 짚어야 한다:

[평가] Prop. 3.2 의 가정 — 상수 η , 무제약 $\mathcal{W}$, 누적 objective 의 strict convexity — 는 실제 출하된 모델에서 전부 깨진다. gate 는 data-dependent(η_t) 이고, Memora 의

$\mathcal{W}$ 는 simplex 제약이 있으며, 2-layer MLP memory 는 convexity 를 깨뜨린다. 따라서 이 동치는 설계의 동기이지 보증이 아니다. 3장에서 배운 regret 보증이 여기 자동으로 이식되지 않는 이유가 정확히 이것이다.

마지막으로 [Miras Remark 4]: 두 관점은 GD 기반 유도를 위한 것일 뿐, 식 (13-1)의 정의 자체는 optimizer-agnostic 이다. Newton 법으로 풀면 Mesa-layer 가 되고, non-parametric 하게 풀면 softmax attention 이 된다. 이개방성이 다음 절의 통일표를 가능하게 한다.

13.3.3 Miras 의 네 축

이제 framework 를 조립할 수 있다. **Miras framework** 는 sequence model 을 다음 4-tuple 로 정의한다 [Miras § 4]:

1. **Memory architecture** — memory 의 구조: vector, matrix($\mathcal{M}(k; W) = Wk$), MLP, 또는 그 이상. 선택적으로 W 에 제약을 건다 (ℓ_2 ball, scaled probability simplex 등 — 제약 자체가 안정성 장치다).
2. **Attentional bias** — objective L (또는 그 근사 $\hat{\ell}/\tilde{\ell}$). memory 가 입력을 어떻게 mapping 하고 어떤 사건을 우선하는지 결정한다.
3. **Retention gate** — R_t (FTRL) 또는 Ret_t (Learning - Retaining; 식 (13-4)의 local+global 분해). plasticity 와 stability 의 균형. 논문은 long-context 성능의 결정적 레버가 이 축이라고 주장한다.
4. **Memory learning algorithm** — GD, GD+momentum, implicit GD, Newton, multi-step GD, closed-form/non-parametric 해.

부수 축으로 data-dependent 여부, scalar vs channel-wise parameter 가 있다 — 같은 4-tuple 좌표의 모델들이 부수 축에서만 갈리는 경우가 많다.

이 네 축과 그 축 위의 대표 선택지, 그리고 한 layer 가 입력을 memory 에 쓰고 (attentional bias + retention gate 로 구성된 목적함수를 gradient descent 로 최소화) 다시 읽어내는 흐름을 논문의 overview 그림이 한 장으로 요약한다 (그림 13-1). 이 그림에서 memory 는 associative memory 하나이고, “attentional bias $\mathcal{L}(\mathcal{M}(\mathcal{K}); \mathcal{V})$ + retention $\text{Ret}_t(\mathcal{M}, \mathcal{M}_{t-1})$ ” 라는 합이 곧 매 token 최소화되는 inner objective 이며, 그 최소화 알고리즘의 선택이 네 번째 축이다 — 이 장의 나머지 전개는 이 한 그림의 각 칸을 채워 넣는 작업이다.

13.3.4 기존 모델의 재유도: 전부가 이 공간의 점이다

Miras 의 통일 결과 [Miras Table 1, § 4] 를 통일 표기로 재구성한다. 유도의 뼈대만 한 번 보이면 나머지는 기계적이다.

Hebbian 계열. attentional bias 를 dot-product similarity $\tilde{\ell}_t = -2\langle Wk_t, v_t \rangle$ 로, local retention 을 $\|W - \alpha W_{t-1}\|_F^2$ 로 놓고 (13-3)을 GD 로 풀면, 일차 조건 $-2v_t k_t^\top + 2(W - \alpha W_{t-1}) = 0$ 에서 곧바로

$$W_t = \alpha W_{t-1} + v_t k_t^\top \quad (13-5)$$

가 나온다 [Miras Eq. 8]. $\alpha = 1$ 이면 linear attention, α 가 학습 상수면 RetNet($n=1$ vector state)/Lightning Attention($n>1$), α_t 가 data-dependent scalar 면 Mamba-2, diagonal 이면 GLA 다. dot-product bias 는 “비슷한 key 에 비례해 크게 쓰라” 는 objective 이므로 최소화의 해가 순수가산 write 가 된다 — Hebbian write 에 replacement 가 없는 이유가 objective 수준에서 설명된다.

delta 계열. bias 를 ℓ_2 regression $\|Wk_t - v_t\|_2^2$ 로 바꾸고 같은 retention, (stochastic) GD 를 쓰면 6 장 카탈로그의 기준형

$$W_t = \alpha_t W_{t-1} (I - \eta_t k_t k_t^\top) + \eta_t v_t k_t^\top \quad (13-6)$$

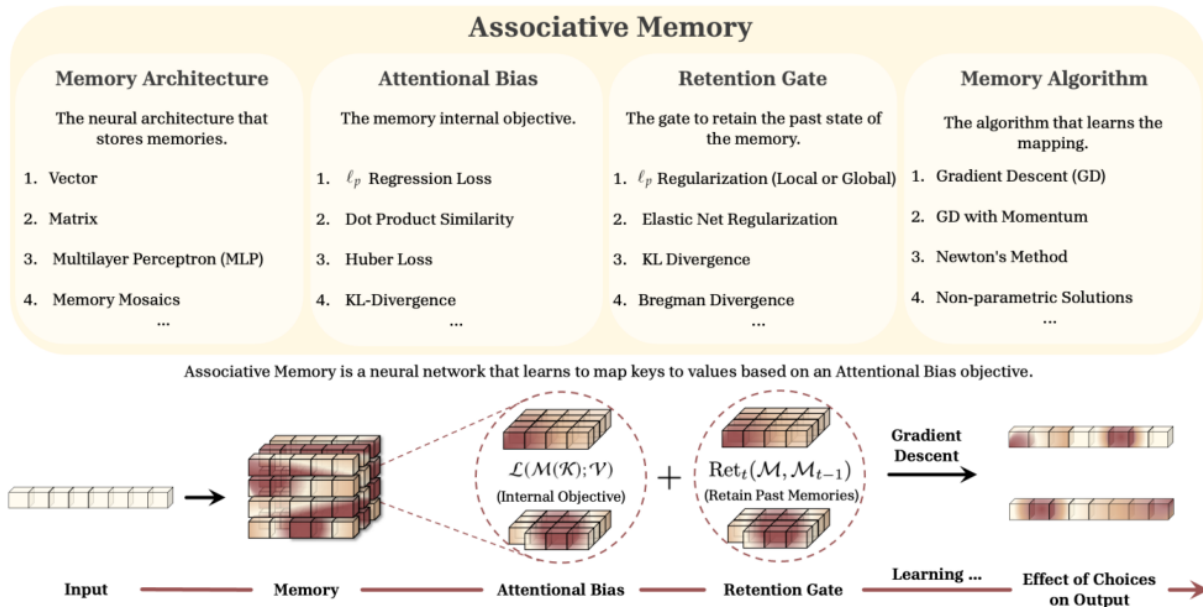


그림 13-1 — Miras framework 의 개요. associative memory 설계를 네 개의 독립 축 (memory architecture / attentional bias / retention gate / memory algorithm) 으로 분해하고, 각 축의 대표 선택지와 “attentional bias + retention gate = 매 token 최소화되는 목적함수 → gradient descent” 흐름을 보인다. 출처: Behrouz et al., It’s All Connected (Miras, arXiv:2504.13173), 원문 Fig.1 — 원저자의 그림 (third-party), 본서의 결과가 아님.

이 된다 [Miras Eq. 9; 행렬 곱 순서와 write 항 계수는 표기 관행 차이, 알고리즘 동일]. $\alpha = 1$ 이면 DeltaNet, data-dependent scalar α_t 면 GDN, channel-wise vector 면 RWKV-7 이다. Longhorn 은 같은 objective 를 **implicit GD**(closed-form proximal step) 로 풀 것 — algorithm 축만 다른 점이고, DeltaProduct 는 token 당 여러 GD step 을 달는 multi-step 변형이다. Hebbian → delta 의 개선이 “덧쓰기 → 고쳐쓰기” 였다는 6 장의 서사가, 여기서는 “objective 의 교체” 라는 한 문장으로 압축된다.

delta 너머. Titans-LMM 은 nonlinear ℓ_2 bias(deep MLP memory) + local $D_t = \|W - W_{t-1}\|_F^2$ 와 global $G_t = \|W\|_2^2$ 둘다 + **GD with momentum**, 즉 표준형 (M2) 그대로다. Miras 는 각주에서 Titans gate 와 Mamba-2/GDN gate 의 차이를 짚는다: 완전 소거 ($\alpha_t \rightarrow 0$) 의 극한에서 Mamba-2 류는 다음 token 을 “생애 첫 데이터” 로 취급하지만, Titans 는 소거 직전의 memory 로 새 token 의 surprise 를 먼저 측정하는 cold-start 전략을 쓴다 [Miras Table 1 각주 2]. Mesa-layer 는 전체 이력 objective $\sum_{i \leq t} \|\mathcal{M}(k_i; W) - v_i\|_2^2 + \|W\|_2^2$ 를 Newton 법으로 정확히 풀 극한이다. 그리고 **softmax attention**: ℓ_2 regression loss 의 non-parametric Nadaraya-Watson 해 (→ 8 장) 로, retention 이 없고 state 가 곧 커지는 집합 $\{(k_t, v_t)\}$ — 즉 KV cache 그 자체다. attention 이 선형 state 증가를 갖는 이유는 과거 KV 를 **압축 없이 보존하는 associative memory** 이기 때문이다 — 단 retrieval 정확도는 softmax kernel weighting 에 달려 있어 (key 충돌 · 가중 평균) ‘완벽한 recall’ 이 보장되는 것은 아니다.

표 13-1 — 기존 모델의 Miras 좌표 ([Miras Table 1] 재구성; update 식은 식 (13-5)·(13-6) 계열의 § 1.6 카탈로그 기준형)

모델	architecture	attentional bias	retention	algorithm
linear attention	matrix	dot-product	없음 ($\alpha = 1$)	GD
RetNet	vector	dot-product	ℓ_2 (상수 α)	GD

모델	architecture	attentional bias	retention	algorithm
GLA /	matrix	dot-product	ℓ_2 (data-dep. α_t)	GD
Mamba-2	matrix	ℓ_1 류 (tied write $v_t(1 - \alpha_t)^\top$)	ℓ_2	GD
DeltaNet	matrix	ℓ_2 regression	없음 ($\alpha = 1$)	GD
Longhorn	matrix	ℓ_2 regression	—	implicit GD
GDN	matrix	ℓ_2 regression	ℓ_2 (scalar α_t)	GD
RWKV-7	matrix	ℓ_2 regression	ℓ_2 (channel-wise)	GD
DeltaProduct	matrix	ℓ_2 regression	ℓ_2	multi-step GD
TTT-Linear /	matrix / 2-layer	ℓ_2 regression	없음	GD
TTT-MLP	MLP			
Titans-LMM	deep MLP	nonlinear ℓ_2	local+global ℓ_2	GD + momentum — (M2)
Mesa-layer softmax attention	matrix non- parametric (KV cache)	전체 이력 ℓ_2 ℓ_2 regression	ℓ_2 없음	Newton Nadaraya – Watson 해
Moneta	2-layer MLP	ℓ_p ($p=3$)	ℓ_q ($q=4$) + ℓ_2	GD
Yaad	2-layer MLP	Huber (mixture)	local+global ℓ_2	GD
Memora	2-layer MLP	ℓ_2	KL / simplex	GD

단, 논문 자신의 각주가 이 표의 해상도 한계를 인정한다: retention 열의 “ ℓ_2 ” 는 세부가 다른 ℓ_2 류 gate 들을 뭉뚱그린 것이고, 유도된그대로의 ℓ_2 retention 을 쓰는 것은 엄밀히는 Titans 와 RWKV-7 뿐이다 [Miras Table 1 각주 †]. 또한 4-tuple 좌표는 backbone 을 유일하게식별하지 않는다 — conv, hybrid attention, channel-wise 화 같은 부수 축이남는다.

13.3.5 새로운 attentional bias: ℓ_p , Huber, value-shift robustness

축 (ii) 를 여는 세 변형이다 [Miras § 5.1]. 공통 동기는 ℓ_2 의노이즈 민감성이다.

변형 1 — ℓ_p bias. $L = \|\mathcal{M}(k_t; W) - v_t\|_p^p$ ($p \geq 1$) [Miras Eq. 10]. matrix memory 에서 GD step 의 gradient 는

$$\nabla_W \ell_p = p [\text{Sign}(Wk_t - v_t) \odot |Wk_t - v_t|^{p-1}] k_t^\top \quad (13-7)$$

이다 [Miras Eq. 11]. 여기서 $\text{Sign}(\cdot)$ 과 $|\cdot|$ 는 element-wise 부호 · 절댓값 연산자다 (장-국 소 정의). p 는오차 민감도의 다이얼이다: $p < 2$ 는 큰 residual 의 영향을 눌러 노이즈에강건해지고, $p > 2$ 는 큰 오차 — 즉 심하게 놀란 token — 를 증폭해기억한다. 극한 $p = 1$ 에서는 update 가 $W_t = W_{t-1} - \eta_t \text{Sign}(W_{t-1}k_t - v_t) k_t^\top$ 로단순화되어 [Miras Eq. 12], residual 의 크기를 버리고 부호만 write 신호로 쓴다 (residual 이 ± 1 로 양자화될 뿐, η_t 와 k_t 가공해져 누적되므로 W 의 entry 와 memory 출력 자체는 일반 실수다). 논문은 이를 **value-less associative memory** 라 부른다: 어떤 key 가왔었다는 사실은 저장하되 value 의 크기는 저장하지 않는, 인간이 극단적사건의 세부를 기억에서 눌러버리는 coping mechanism 의 유비다.

여기에 시스템적으로 중요한 조건 하나가 붙는다 [Miras Remark 5]: Sign 과 $|\cdot|$ 는 미분 불가능 하므로 그대로 두면 outer-loop backprop 이 죽는다. 그래서 $\text{Sign}(x) \approx \tanh(\nu x)$, $|x| \approx \sqrt{x^2 + \epsilon}$ ($\epsilon = 10^{-6}$) 로 매끈하게바꾼다 (smoothing 계수는 원문의 α 를 retention gate 와의 충돌때문에 ν 로 개명, 표 13-2). 왜 이것이 사활적인지는 § 13.4 에서명확해진다.

변형 2 — Huber bias: coping mechanism의 정식화. robust regression의 Huber loss를 attentional bias로 쓴다. 세 가지 적용법이 있다 [Miras § 5.1 Variant 2]: (i) 좌표별 Huber 합 — threshold δ_t 이내 좌표에는 ℓ_2 gradient, 밖의 좌표에는 $\text{sign}(\text{value-less})$ gradient를 쓰는 좌표별 혼합 [Miras Eq. 14]; (ii) residual norm의 Huber $\ell = \mathcal{H}(\|\mathcal{M}(k_t; W) - v_t\|_2)$ — norm이 δ_t 를 넘으면 residual을 정규화된 방향벡터로 바꿔 δ_t 배율로 쓰는, 사실상 **per-token gradient clipping** [Miras Eq. 15]; (iii) 매끈한 혼합 — token 단위로 ℓ_2 와 ℓ_1 bias 중하나를 선택:

$$W_t = W_{t-1} - \begin{cases} \eta_t \nabla_W \ell_2(W_{t-1}; k_t, v_t) & \|\mathcal{M}(k_t; W_{t-1}) - v_t\|_2 \leq \delta_t, \\ \eta_t \delta_t \nabla_W \ell_1(W_{t-1}; k_t, v_t) & \text{otherwise.} \end{cases}$$

[Miras Eq. 16] 형태 (iii)이 Yaad에 실린다. 핵심 설계는 δ_t 가 **입력 의존적으로 학습된다는 것**: 어떤 사건이 outlier여서 크기를 깎아 기억해야 하는지를 memory 스스로 token마다 판단한다.

변형 3 — value shift에 강건한 memory. worst-case 정식화 $L = \max_{\|\delta_v\|_2 \leq \Delta} \frac{1}{2} \|\mathcal{M}(k_t; W) - (v_t + \delta_v)\|_2^2$ [Miras Eq. 17]. 내부 max는 닫힌 해가 있어 $L = \frac{1}{2} \|r\|_2^2 + \Delta \|r\|_2 + \frac{1}{2} \Delta^2$ (r 은 residual)로 정리되고, update는 통상 ℓ_2 항에 정규화된 residual 방향 항이 Δ 배율로 더해진 형태가 된다. Δ 는 학습가능하다. 결과적으로 이 변형은 ℓ_2 와 ℓ_2 -norm 항의 혼합 — Huber (ii)의 매끈한 친척이다.

13.3.6 새로운 retention gate: 정규화의동물원

축 (iii)이 이 논문의 가장 독창적인 절이다 [Miras § 5.2]. 3장의 online optimization 도구함 — mirror descent, Bregman divergence — 이 통째로 sequence layer 설계로 수입되고, 여기에 **elastic net**($\ell_1 + \ell_2$ 결합 별점; 3장의 ℓ_1/ℓ_2 eviction 구도의 결합형으로, 이 장에서도 도입한다)이 더해진다.

변형 1 — f -divergence retention과 scaled probability simplex. state를 유계 영역에 가두는 것은 수치 안정성의 고전적 처방이다. $\mathcal{W} = \{W : \|W\|_1 = c, W_{jl} \geq 0\}$ — scaled probability simplex — 로 제약하면 W 를 measure로 볼 수 있고, local retention D_t 를 f -divergence $\sum_{jl} W'_{jl} f(W_{jl}/W'_{jl})$ 로 정의할 수 있다. 선형화된 loss와 결합하면 곱셈형 update $W_t = W_{t-1} \odot g(-\zeta_t - \eta_t \nabla_W \ell(W_{t-1}; k_t, v_t))$ 가 나온다 [Miras Eq. 18]. $g = (f')^{-1}$ 이고 ζ_t 는 $\|W_t\|_1 = c$ 를 강제하는 정규화 상수다. KL 특수화 ($f(\tau) = \tau \ln \tau$)에 Shannon entropy를 global retention $G_t(W) = \sum_{jl} W_{jl} \log W_{jl}$ 로 더하면, KKT 조건 (제약 최적화의 1차 최적성 조건)에서

$$W_t \leftarrow c \text{softmax}((1 - \lambda_t) \log W_{t-1} - \eta'_t \nabla_W \ell(W_{t-1}; k_t, v_t)), \quad \lambda_t = \frac{1/\alpha_t}{1/\alpha_t + 1/\eta_t}, \quad \eta'_t = \frac{1}{1/\alpha_t + 1/\eta_t} \quad (13-8)$$

가 유도된다 [Miras Eq. 21]. 이것은 online learning의 exponentiated gradient / multiplicative weights를 memory rule로 만든 것이다. softmax가 매 step state를 양수 · 합-정규화 상태로 유지하므로 **state는 context 길이와 무관하게 절대 발산할 수 없다**. 잊기는 decay가 아니라 확률 질량의 경쟁에서 나온다 — 이 책은 이 질적으로 다른 안정화 기제를 **retention-as-renormalization**이라 부른다 (Miras에 암묵적인 개념의 명시화).

변형 2 — elastic net retention: hard와 soft forgetting. LASSO(ℓ_1)와 Ridge(ℓ_2)의 혼합을 global retention $G_t(W) = \frac{1}{2c_2} \|W\|_2^2 + \frac{1}{c_1} \|W\|_1$ 로, local은 $D_t = \frac{1}{2} \|W - W_{t-1}\|_2^2$ 로 놓으면 (Learning - Retaining에서)

$$W_t = \mathcal{S}_\gamma(\lambda W_{t-1} - \zeta \nabla_W \ell(W_{t-1}; k_t, v_t)), \quad \gamma = \frac{\eta c_2}{c_1(\eta + c_2)}, \quad \lambda = \frac{c_2}{c_2 + \eta}, \quad \zeta = \eta \lambda \quad (13-9)$$

가 나온다 [Miras Eq. 22; elastic-net 계수는 원문 β, α 를 c_2, c_1 로 장-국소 개명]. $\mathcal{S}_\gamma(z) = \text{sign}(z) \max\{0, |z| - \gamma\}$ 는 element-wise soft-thresholding 연산자다 (장-국소 선연; Atlas의 window gate $\gamma_{t,i}$ 와 무관). 해석이 아름답다: $\lambda \in (0, 1)$ 의 곱셈 감쇠는 **soft forgetting** — 고전적 gate 그 자체 — 이고, thresholding은 **hard forgetting** — γ 이하로 작아진 entry를 정

확히 0 으로 스넵해 용량을 해방한다. 미분 불가능성은 여기서도 smooth surrogate $\mathcal{S}_\gamma(z) \approx |z| \arctan(z/\gamma)/(\pi/2)$ 로 처리한다.

변형 3 — elastic net 의 FTRL 형. 같은 regularizer 를 FTRL viewpoint(식 (13-2)) 에 넣으면 이중 변수 구조가 나온다: $A_t = A_{t-1} - \eta \nabla_W \ell(W_{t-1}; k_t, v_t)$, $W_t = \mathcal{S}_{\eta/c_1}(A_t)$ [Miras Eq. 23]. memory 가 날 것의 gradient accumulator A_t 와 노출용 뷰 W_t 의 두 층으로 갈라진다.

변형 4 — 일반 ℓ_q stability. $1 < q \leq 2$ 에 대해 $R(W) = \frac{1}{2\eta(q-1)} \|W\|_q^2$ 로 놓으면 (FTRL) 고전적 결과 (Shalev-Shwartz 2012, § 2.6) 에 의해 $A_t = A_{t-1} - \eta \nabla_W \ell$, $W_t = A_t / \|A_t\|_p^{p-2}$ ($p = q/(q-1)$, dual norm) 가 된다. accumulator 는 자유롭게 크지만 노출되는 memory 는 norm 이 통제된 껍질 위에 산다.

변형 5 — Bregman divergence retention = mirror descent. strictly convex f 로 $F(W) = \sum_{jl} f(W_{jl})$ 를 만들고 D_t 를 F 의 Bregman divergence 로 놓으면 $W_t = g(-\eta \nabla_W \ell(W_{t-1}; k_t, v_t) + F'(W_{t-1}))$, $g = (F')^{-1}$ element-wise. $f(\tau) = \tau^2/2$ 면 plain GD 로 퇴화하고, f' 를 inverse sigmoid 로 고르면 $W_t = \sigma(\ln \frac{W_{t-1}}{1-W_{t-1}} - \eta \nabla_W \ell)$ — 모든 state entry 가 $(0, 1)$ 에 갇힌다. retention gate 의 선택이 곧 **recurrence 에 주입되는 nonlinearity** 의 선택이 된다는 것, 즉 gate 축과 expressivity 축이 붙어 있다는 것이 이 변형의 교훈이다.

13.3.7 출하된 세 모델: Moneta, Yaad, Memora

세 모델의 공통 사양 [Miras § 5.3]: memory architecture 는 12 장의 표준 deep memory 에 LayerNorm 을 더한 2-layer residual MLP, expansion 4, GELU — $\mathcal{M}(z; W) = z + \text{LN}(W_1 \sigma(W_2 z))$. memory algorithm 은 셋 다 **plain GD** — momentum 없음 (Titans 에서 후퇴한 유일한 축이며, 의도된 후퇴다: expressivity 를 optimizer 가 아니라 objective 와 retention 에 실는 실험 설계다). inner learning rate η_t 와 retention α_t 는 분리 (decoupled) 되고, 모든 $\text{gate}(\eta_t, \alpha_t, \delta_t \in \mathbb{R}^d)$ 는 channel-wise 이며, parameter 비용을 묶기 위해 RWKV-7 을 따라 rank 32 – 64 의 low-rank projection 이 emit 한다.

Moneta(p, q) — ℓ_p bias + ℓ_q -stability(+ ℓ_2) retention 의 조합:

$$A_t = \alpha_t A_{t-1} - \eta_t \nabla_W \ell_p(W_{t-1}; k_t, v_t), \quad W_t = \frac{A_t}{\|A_t\|_q^{q-2}}, \quad (13-10)$$

gradient 는 식 (13-7) [Miras Eq. 24 – 25]. 출하 값은 $(p, q) = (3, 4)$ 다. 단 $q = 4$ 는 § 13.3.6 변형 4 의 FTRL 유도 범위 ($1 < q \leq 2$) 를 벗어난 경험적 확장이며, $q = 4$ update 에 대한 별도 유도나 보증은 원문에 없다. 설계 논리: $p = 3$ 은 잘 회상되지 않는 (놀라운) token 에 대한 기억 압력을 ℓ_2 보다 날카롭게 하고, ℓ_q 정규화는 노출 memory 를 norm-통제 껍질에 유지해 곱셈 감쇠보다 강한 안정화를 제공한다.

Yaad — Huber mixture bias + Titans 식 local+global ℓ_2 retention:

$$W_t = \alpha_t W_{t-1} - \begin{cases} \eta_t \nabla_W \ell_2(W_{t-1}; k_t, v_t) & \|\mathcal{M}(k_t; W_{t-1}) - v_t\|_2 \leq \delta_t, \\ \eta_t \delta_t \nabla_W \ell_1(W_{t-1}; k_t, v_t) & \text{otherwise,} \end{cases} \quad (13-11)$$

δ_t 는 학습된 입력 의존 threshold 다 [Miras Eq. 26]. 논리: 노이즈 · 적대적 스펠 같은 outlier token 이 자기 크기만큼 memory 를 흔들게 두어서는 안 된다.

Memora — plain ℓ_2 bias + KL retention, 즉 (13-8) 의 기계 그대로:

$$W_t = \text{softmax}(\alpha_t \log W_{t-1} - \eta_t \nabla_W \ell_2(W_{t-1}; k_t, v_t)). \quad (13-12)$$

[Miras Eq. 27; (13-8) 의 $1 - \lambda_t$ 역할을 α_t 가 맡는다.] 논리: simplex-제약 (softmax-재정규화) state 는 증명 가능하게 유계이며, context 가 아무리 길어도 state 가 폭발할 수 없다. W 가 MLP weights 일 때는 같은 rule 이 slice 별로 적용된다.

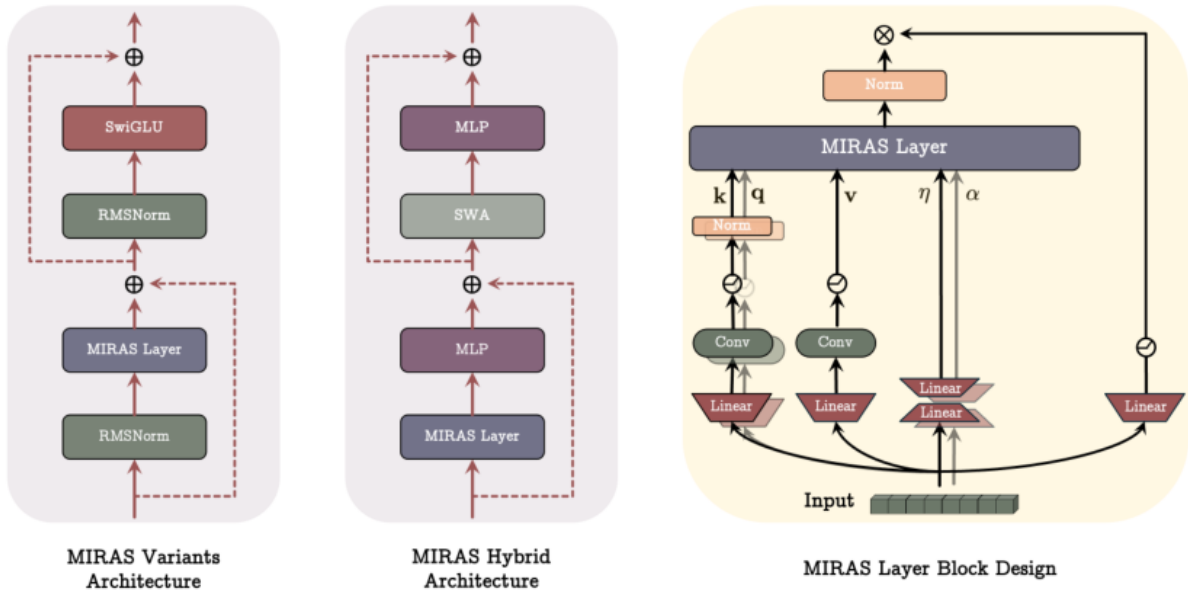


그림 13-2 — Miras 변형의 아키텍처. (왼쪽) 순수 recurrent 변형의 residual 블록, (가운데) Miras layer와 Sliding Window Attention 을교차하는 Samba 식 hybrid, (오른쪽) Miras layer 내부 블록 설계: k/q/v projection 뒤의 depthwise conv, q·k 의 normalization, low-rank 로 emit 되는 $\eta \cdot \alpha$ 게이트, 출력 normalization + linear output gate. 출처: Behrouz et al., It’ s All Connected (Miras, arXiv:2504.13173), 원문 Fig.2 —원저자의 그림 (third-party), 본서의 결과가 아님.

블록 구조와 hybrid [Miras § 5.3]. Miras layer 는 Llama macro 구조에서 attention 자리에 들어간다: SwiGLU 채널 MLP, RoPE, RMSNorm. token-mixing 블록 내부는 q/k/v projection 각각뒤에 depthwise-separable 1D conv(kernel 4), 훈련 안정성을 위한 q·k 의 ℓ_2 normalization, 그리고 memory 읽기 출력의 normalization + linear output gate. hybrid 변형 (Moneta-H/Yaad-H/Memora-H) 은 Samba 를 따라 Miras layer 와 Sliding Window Attention layer 를 순차 교차한다. 그림 13-2 가 이 세 층위를 한눈에 보인다: 왼쪽은 순수 변형 (RMSNorm → Miras layer → SwiGLU 의 residual 블록), 가운데는 Miras layer 와 SWA 를 번갈아 쌓는 hybrid, 오른쪽은 layer 내부 — k·q·v 를 각각 conv 로 만들고 q·k 만 normalization 하며, η 와 α 게이트는 별도 low-rank(사다리꼴로 표시된 축소→확장) projection 이 emit 하고, 출력은 normalization 후 linear gate 로 곱해지는블록 설계다 (그림 13-2 오른쪽의 η, α 분기가 § 13.4 에서 다룰 “gate 를 만드는 정책은 outer loop 가 학습한다” 의 그림 대응물이다).

13.3.8 표기 대응표

표 13-2 — [Miras] 원 표기 ↔ 통일 표기 (§ 1.7.2 사본 + 장-국소 행)

Miras 원 표기	통일 표기	의미 / 주의
$\mathcal{M}, W$ (혼용)	$W_t / \mathcal{M}(\cdot; W_t)$	
$\mathcal{M}^* = \arg \min_{\mathcal{M}} L(\cdot)$	$\mathcal{M}^*$	Titans 의 read-star 와 무관
L — attentional bias	L	동일 (공식 용어)
$\ell(W_{t-1}; \mathbf{k}_t, \mathbf{v}_t)$	$\ell(W_{t-1}; k_t, v_t)$	동일
$\hat{\ell}_i$ (선형화), $\tilde{\ell}_t$ (surrogate)	동일	
η_t — inner lr (“meta in-context lr”)	η_t	동일

Miras 원 표기	통일 표기	의미 / 주의
α_t — global retention 계수 ($\alpha \mathcal{M}_{t-1}$)	α_t	동일 방향 (통일안의 기준)
$\beta_t \in [0, 1]^d$ — decoupled retention gate (§ 5.3)	α_t 계열로 흡수; 병기 필요시 α_t^g	$\triangle$ 통일 β_t (momentum decay) 와 무관
R_t (FTRL), Ret_t , D_t (local), G_t (global)	동일	
A_t — dual accumulator (Moneta, FTRL 형)	A_t	장-국소
$\mathcal{S}_\gamma$ — soft-thresholding	$\mathcal{S}_\gamma$ (장-국소 선언 후 사용)	$\triangle$ window gate $\gamma_{t,i}$ 와 구분
$p, q - \ell_p$ bias / ℓ_q stability 차수	동일	query q_t 와 혼동 금지
δ_t — Huber threshold (learned)	δ_t	backprop δ_ℓ 과 첨자로 구분
$\lambda_t, \eta'_t, \zeta$ — KKT 유도 파생 계수 GD with momentum 행 (Titans 행)	장-국소 유지 (M2)	
Remark 5 의 smoothing 계수 α ($\tanh(\alpha x)$)	ν (장-국소)	$\triangle$ retention gate α_t 와 충돌 방지
Eq. 22 의 elastic-net 계수 β, α	c_2, c_1 (장-국소)	$\triangle$ momentum $\beta_t \cdot \text{retention}$ α_t 와 충돌 방지
Eq. 28 의 누적 곱 $\beta_i = \prod_{j \leq i} \alpha_j$	$\bar{\alpha}_i$ (장-국소)	$\triangle$ momentum β_t 와 충돌 방지
b — chunk 크기	C	§ 1.2
E_b, B_b — chunk 내 broadcast 행렬	E_C, B_C (장-국소)	
lag token 인덱스 $i = kb + 1$	$t = nC + 1$	chunk 인덱스 n 은 0-based

13.4 Outer-loop training vs inner-loop test-time learning

이 절이 training 무경험 독자에게 가장 중요한 절이다. Miras 에는 loss 가 두개, learning rate 가 두 개, 학습이 두 개 있다. 표 하나로 먼저 가른다.

표 13-3 — Miras 의 두 loop

	outer loop (pre-training)	inner loop (test time)
최적화 대상	Θ : W_Q, W_K, W_V , depthwise conv, output gate, RMSNorm/LN scale, SwiGLU 채널 MLP, gate hypernetwork(low-rank), 초기 memory W_{init}	W_t (2-layer MLP fast weights); Moneta 는 accumulator A_t 추가
objective	$\mathcal{L}$: next-token prediction	ℓ : 각 layer 의 attentional bias
update rule	표준 pre-training (unrolled inner loop 관통 backprop; optimizer 종류 · batch · 스케 줄은 원문 미명시 [Miras App. C — Table 5 외 훈련 레시피 부재])	(13-10)/(13-11)/(13-12) — smoothed GD 1 step/token

	outer loop (pre-training)	inner loop (test time)
갱신 시점	training step 마다; inference에서는 동결	매 token (훈련·prefill에서는 chunk 단위 근사)
learning rate	outer η — 원문은 peak LR 만 명시 (3e-3/1.5e-3/1.25e-3 [Miras Table 5]); 스케줄· optimizer 미명시	inner η_t (channel-wise, token 마다 Θ 가 생성)

outer loop에서 meta-learn 되는 것. key/value/query 를 제조하는 projection, conv, output gate, 채널 MLP — 여기까지는 Transformer 훈련과 다르지 않다. Miras 고유의 것은 두 가지다. 첫째, **inner-loop hyperparameter**를 emit 하는 **hypernetwork**가 학습된다: $\eta_t, \alpha_t, \delta_t$ 는 사람이정하는 상수가 아니라 현재 token의 함수 $\eta_t = \eta(x_t; \Theta)$ 로서, rank 32 – 64 low-rank projection이 채널별로 뽑는다. 즉 “이 gate는 누가학습하는가?”의 답은: **gate의 값은 inner loop에서 소비되지만, gate를 만드는 정책은 outer loop가 학습한다**. outer loop는 “inner loop가 얼마나 공격적으로 쓰고 얼마나 유지할지”의 per-token-per-channel 정책을 배우는 것이다. 둘째, memory의 초기 상태 W_{init} ($W_0 = W_{\text{init}}$)도 Θ 의 일부로 학습된다.

gradient는 어떻게 inner loop를 관통하는가. inference 어휘로말하면, inner update의 궤적 전체가 forward graph의 일부다. 매 inner step $W_t = \alpha_t W_{t-1} - \eta_t \nabla_W \ell(\dots)$ 은 (a) 그 token의 k_t, v_t , (b) emit된 gate들, (c) 직전 state의 미분 가능한 함수이므로, $\partial \mathcal{L} / \partial \Theta$ 는 penultimate token의 memory 읽기에서 출발해 unrolled recurrence를 거슬러 첫 token까지 흐른다. 이것이 4장에서 본 MAML 류 “backprop through an optimizer”이며, 훈련 비용과 메모리에 unrolled inner step이 포함되는 이유다. 그리고 이제 § 13.3.5의 smooth surrogate가 왜 사활적인지 명확해진다: Sign은 거의 모든 곳에서 도함수가 0이라 outer gradient가 죽고, $|\cdot|$ 와 $\mathcal{S}_\gamma$ 는 원점(꺾인 점)에서 미분 불가능하다 — 폭주가 아니라 이 비미분 가능성이 문제다 ($|\cdot|$ 의 도함수는 ± 1 로 유계여서 소실·폭주를 일으키지 않는다). $\tanh(\nu x)$, $\sqrt{x^2 + \epsilon}$, arctan-thresholding은 **inner의 수학을 outer가 미분할 수 있게 만드는 접착제**다. inference-only 세계에는 대응물이 없는, 이 라인 고유의 설계 제약이다.

chunkwise-parallel training. 이 unrolled 훈련을 가속기에서 실행가능하게 만드는 것이 9장의 chunkwise 기법이고, Miras는 Titans/TTT의 레시피를 계승한다 [Miras § 5.3]. sequence를 크기 C (논문 설정 16 또는 64)의 chunk로 나누고, chunk 안 모든 token의 gradient를 직전 chunk의 마지막 state에서 평가한다 — 즉 $\nabla_W \ell(W_{t-1}; k_t, v_t)$ 대신 $\nabla_W \ell(W_{\xi(t, C)}; k_t, v_t)$, 표준형 (M4)의 stale-snapshot 근사 그대로다. retention까지 포함해 recurrence를 전개하면 ($\bar{\alpha}_i := \prod_{j \leq i} \alpha_j$)

$$W_t = \bar{\alpha}_t W_0 - \sum_{i=1}^t \frac{\bar{\alpha}_t}{\bar{\alpha}_i} \eta_i \nabla_W \ell(W_{\xi(i, C)}; k_i, v_i) \quad (13-13)$$

[Miras Eq. 28; 첫 chunk 기준, W_0 는 chunk 시작 state]. chunk 안에서 anchor가 고정되므로 C 개의 gradient는 서로 독립이고, ℓ_p loss의 경우 chunk 전체의 gradient 합이 $p E_C \odot B_C \odot \text{Sign}(W_0 K_{(n)} - V_{(n)}) \odot |W_0 K_{(n)} - V_{(n)}|^{p-1} K_{(n)}^\top$ — column-stack된 chunk 행렬 $K_{(n)}, V_{(n)}$ 위의 **batched GEMM 하나 + element-wise epilogue**로 붕괴한다 [Miras Eq. 29]. E_C 는 η_i 의, B_C 는 감쇠비 $\bar{\alpha}_C / \bar{\alpha}_i$ 의 broadcast 행렬이다.

선형성을 깨는 비선형은 chunk 경계로 밀어낸다. Moneta의 ℓ_q 정규화 $W_t = A_t / \|A_t\|_q^{q-2}$ 는 token마다가 아니라 chunk 끝에서 한 번 적용한다. Yaad는 $\ell_1 \cdot \ell_2$ 두 branch의 chunk gradient를 모두 계산한 뒤 $\|\mathcal{M}(k_t; \cdot) - v_t\|_2 \leq \delta_t$ mask로 혼합한다. Memora가 어려운 경우다: 경계에서 log와 softmax를 연달아 적용하면 $\text{softmax}(\log(x) - g)$ 꼴이 되어 log가 상쇄된다. 그래서 **lag token**을 도입한다 — 각 chunk의 첫 token($t = nC + 1$)에서만 진짜 비선형 step (13-12)를 한번 밟고, chunk의 나머지는 그 결과에서 선형으로 달린다 [Miras Eq. 30].

훈련된 (그리고 chunkwise로 서빙되는) 모델은 per-token 정의 (13-10) – (13-12)의 근사물이다: gradient는 최대 $C - 1$ token 묶은 state에서 평가되고, state 비선형은 chunk당 한 번만 발화한다. chunk 크기가 스케줄이 아니라 계산되는 함수 자체를 바꾸는 semantic hyperparameter

라는 9 장의 명제가 여기서도 성립하며, Miras 는 이 근사의오차를 정량화하지 않고 Titans/TTT 의 경험적 정당화를상속한다 — 6 편 어디에도 이 staleness 의 오차 한계는 없다 (→ 9 장, 15 장).

inference에서 움직이는 것. layer 당 · head 당 fast weights $W = \{W_1, W_2\}$ (그리고 Moneta 의 A_t) 만 갱신되고, 나머지전부 — projection, conv, gate hypernetwork, 채널 MLP — 는 동결 이다. gate 들은 동결된 projection 이 현재 token 에서 계산해 내놓는 값이므로, inference 시에 도 token 마다 다르지만 **학습되고 있지는 않다**. per-token 갱신은 forward(예측 $\mathcal{M}(k_t; W_{t-1})$ 형 성) → backward(2-layer MLP 관통 $\nabla_{W_1}, \nabla_{W_2}$) → element-wise retention/정규화 → query 로 읽기 $y_t = \mathcal{M}(q_t; W_t)$ 의 네 단계이고, 비용은 context 길이와 무관한상수다. 정량은 § 13.7 에서 다룬다. 12 장과의 대비: Titans decode 는 momentum buffer S_t 까지 끌고 다녔지만, Miras 세 모델은 plain GD 라 S_t 가없다 — 대신 Moneta 는 FTRL 이중 구조의 A_t 라는 두 번째 state 를 끌 고다닌다.

13.5 Concept ledger delta

표 13-4 — Miras 가 ledger 에 가한 변경

개념	변경 유형	내용
attentional bias	신규 (공식 용어)	inner objective 를 설계측으 로 명명 · 정식화. Titans/TTT 의 암묵적 ℓ_2 는 한 선택지로 강등
retention gate	재이론화 + 개명	forget gate → retention regularization; local D_t + global G_t 분해; “모델은 지우 지않는다, 유지하지 않을 뿐”
Miras framework (4 축)	신규	architecture × bias × retention × algorithm; 이 후 논문 전부가 이 좌표계를 전제
Learning – Retaining / FTRL viewpoint	신규	두 유도 렌즈 + Prop. 3.2 포함 관계; FTRL 자체는 3 장 소유
meta in-context learning rate	신규 (해석)	η_t = 새것 학습량과옛것 방출량 을 동시에 정하는 다이얼
value-less associative memory	신규	$p=1$ 극한: key 의 발생만 ± 1 로 저장
hard / soft forgetting	신규	elastic net 에서: 곱셈감쇠 (soft) vs soft-thresholding 의 0-스냅 (hard)
retention-as-renormalization	신규 (암묵 → 이 책이 명명)	simplex/norm 구속에 의한 안정성: decay 없는 forgetting
lag token	신규 (훈련 기법)	Memora 의 chunk 경계 1 회 비선행 step
Moneta / Yaad / Memora surprise (Titans, → 12 장)	신규 (모델) 흡수 · 일반화	축 (ii)·(iii) 의 세 실증점 momentary surprise = 임의 attentional bias 의 gradient
Titans-LMM	지위 변화	독립 아키텍처 → Table 1 의 한 행 ((M2) 좌표)

개념	변경 유형	내용
momentum S_t (inner)	보류 (사실상 일시 폐기)	세 모델모두 plain GD; algorithm 측은 의도적으로 미탐사 — Atlas 가 상속

누적 관점에서 가장 큰 변화는 어휘의 세대 교체다. 12 장까지 서사의 주인공이던 “surprise” 는 이 장 이후 특정 $\text{bias}(\ell_2)$ 의 gradient 에 대한 별명이 된다. 반대로 Titans 의 inner momentum 은 ledger 에서 일시되장한다 — 바로 그 빈 축이 14 장의 Atlas 가 여는 문이다.

13.6 실험과 스케일

설정 [Miras § 6, App. C]. 훈련 context window 4096. 언어모델링 · 상식추론은 FineWeb-Edu, scaling 곡선은 C4. 모델크기는 본문 기준 120M/340M/760M/1.3B 이고, token 수는 소형 (120M·340M) 15B, 760M 30B, 1.3B 100B 다. 아키텍처세부는 [Miras Table 5]: 12 block/dim 768/16 head(peak LR $3e-3$), 24/1024/16($1.5e-3$), 24/1536/16($1.25e-3$). 단 App. C 표는 크기를 170M/340M/780M 으로 적어본문의 120M/760M 과 표기가 어긋난다 — 원문 자체의 불일치이므로 이 책은 본문 수치 (760M 등) 로 인용하되 여기 한 번 기록해둔다. baseline 은 Transformer++, RetNet, GLA, Mamba, Mamba2, DeltaNet, TTT, GDN 과 hybrid 인 Samba, GDN-H2 이며, **baseline** 수치는 **Titans 논문이 보고한 값을 그대로 가져온 것이다** [Miras § 6 Setup] — 같은 저자 라인의 재사용이므로 훈련 설정은 정합하지만, 독립 재현이 아니라는 점은 기억할 것. 같은 저자 라인의 Atlas 가 AdamW·cosine·batch 0.5M tokens 를 명시하는 것과 달리 [Atlas App. E], Miras 에는 outer optimizer 의 종류 · batch · 스케줄이 없다 [Miras App. C — Table 5 외 훈련 레시피 부재] — 재현 시도자는 이 부재를 알고 시작해야 한다.

언어모델링과 상식추론 [Miras Table 2]. 340M/15B 에서 Moneta 가 WikiText ppl 26.19 로 GDN 27.01, TTT 27.44, Transformer++ 31.52 를 앞선다. 760M/30B 에서는 Yaad 20.99 / Moneta 21.18 / Memora 22.28 로, 순수 recurrent 끼리는 GDN(21.18) 과 대등하거나 앞서지만 hybrid Samba(20.63)·GDN-H2(19.88) 에는 밀린다 — 이 스케일에서는 hybrid 가 여전히 우세하다. 그런데 Miras 쪽도 hybrid 를 만들면 (Samba 식 SWA 교차) Memora-H 18.24 / Yaad-H 18.59 / Moneta-H 18.72 로 GDN-H2 를 다시 앞선다. 그리고 flagship 인 1.3B/100B: **순수 recurrent 인 Yaad 가 ppl 15.18, Moneta 15.52 로, GDN(16.42) 만이 아니라 hybrid 인 Samba(16.13) 와 GDN-H2(15.91) 까지 이긴다** [Miras Table 2]. LAMBADA ppl 도 같은 그림이다 (Moneta 11.47 vs GDN 12.17). “더 나은 attentional bias + retention 이면 attention 없이 attention hybrid 를 이긴다” 가 이 논문의 헤드라인 주장이고, 적어도 이 스케일 · 이 벤치마크에서는 수치가 그것을 지지한다.

S-NIAH (RULER) [Miras Table 3]. 1K – 8K 길이의 single needle-in-haystack 세 변형에서 평균 Moneta 93.5 / Yaad 92.9 / Memora 92.1 — GDN 75.8, TTT 66.1, DeltaNet 57.9, Mamba2 52.0 과의 격차가 크다. 특히 haystack 이 합성 노이즈인 S-NIAH-PK 에서 Moneta 는 8K 에서도 98.8 을 유지하는데, 논문은 이를 노이즈에 강건한 p -norm objective 의 효과로 귀속한다 [Miras § 6.3].

scaling 곡선 [Miras Fig. 3]. FLOPs-매칭 ppl(모델 크기 축) 과 context 길이 축 (2K – 32K, 340M·760M) 모두에서 세 변형이 baseline 보다 좋은 기울기를 보인다 (그림 13-3). 왼쪽 패널은 같은 FLOPs 예산에서 Moneta·Yaad·Memora 가 Transformer·Mamba2·TTT 보다 낮은 perplexity 에 도달함을, 가운데 · 오른쪽 패널은 context 길이를 2K 에서 32K 로 늘릴 때 baseline 이 16K 이후 perplexity 가 도로 치솟는 반면 세 변형은 완만하게 유지 · 개선됨을 보인다 — 논문은 이 “긴 context 에서 더 낮게 scale 한다” 를 세 변형 공통의 효과로 귀속한다. 다만 논문의 효율주장 전체가 이 FLOPs 기준 그림 하나에 얹혀 있다는 점 (wall-clock·throughput 부재) 은 § 13.7 에서 다시 짚는다.

ablation [Miras § 6.4]. 세 개가 실했고 각각이 설계 축하나씩을 겨냥한다. (1) $p \in \{1, 1.5, 2, 2.8, 3, 3.2, 4\}$ sweep: 성능은 p 에 대해 **비단조**이고 최적은 $p = 3$, 최악은 $p = 4$ 다. 흥미롭게도 p 는 context-길이 scaling 의 모양은 바꾸지 않는다. (2) $q \in \{2, 3, 4, 5\}$ sweep: 반대

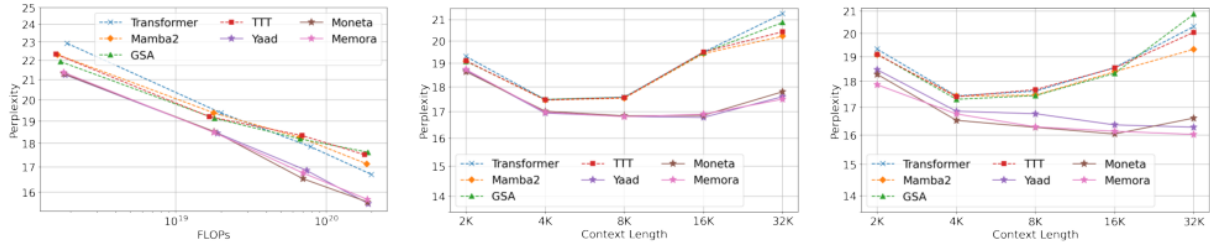


그림 13-3 — C4에서의 scaling 패턴. (왼쪽) 모델 크기를 키울 때의 #FLOPs 대 perplexity, (가운데) 340M에서 context 길이 2K → 32K, (오른쪽) 760M에서 context 길이 2K → 32K. Moneta·Yaad·Memora 세 변형이 baseline 보다 좋은 기운을 보이며, 특히 긴 context에서 baseline의 perplexity 반등을 겪지 않는다. 출처: Behrouz et al., It’s All Connected (Miras, arXiv: 2504.13173), 원문 Fig.3 — 원저자의 그림 (third-party), 본서의 결과가 아님.

로 q 는 **scaling 패턴 자체를 바꾼다** — retention gate의 품질이 long-context 거동을 지배한다는 논문 주장의 가장 직접적인 증거다. (3) Yaad 구성요소 제거 [Miras Table 4]: 평균 LM 점수 53.98에서 retention gate 제거 시 50.63(− 3.35), δ 입력-독립화 52.19(− 1.79), ℓ_2 branch 제거 52.86(− 1.12), ℓ_1 branch 제거 53.04(− 0.94), MLP를 linear memory로 교체 51.57(− 2.41). 기여 순위가 retention > deep memory > threshold의 입력의존성 > bias 세부라는 것 — 논문이 “retention이 결정적레버”라 주장한 순서 그대로다.

스케일의 정직한 상한. 이 라인 전체가 그렇듯 실증은 1.3B params / 100B tokens에서 끝난다. 7B+에서, SFT/RLHF 이후에, exact-copy recall이 지배하는 실무 부하에서 순수 recurrent가 hybrid를 이기는 순서가 유지되는지는 이 논문이 답하지 않는 질문이며 (§ 13.8), long-context 검증도 single-needle S-NIAH 하나뿐이다. multi-needle, multi-hop, 32K 초과 외삽은 미검증이다.

13.7 Systems/serving 함의

이 절은 논문 내용을 독자의 세계 — decode 비용, state 크기, kernel — 로 번역한다. 논문 자신은 이 산수를 하지 않으므로, 별도 표기 없는 정량은 전부 이 책의 계산이다.

[해설] state 크기. Miras layer의 recurrent state는 head당 2-layer MLP 전체다: $W_1 \in \mathbb{R}^{d \times 4d}$, $W_2 \in \mathbb{R}^{4d \times d}$ (d = head 차원), 합계 $8d^2$ 개의 fp 스칼라. matrix-state 모델 (DeltaNet/GDN)의 d^2 의 $8 \times$ 이고, Moneta는 accumulator A_t 가 같은 모양이므로 $16 \times (16d^2)$ 다. $d = 64$ 면 head당 32K entry (Moneta 64K) vs matrix state 4K. KV cache와의 손익분기: cache는 head당 $2Nd$ entry이므로 $8d^2 = 2Nd$ 에서 $N = 4d$ — $d = 64$ 기준 **약 256 token** (Moneta 512)만 넘으면 state가 cache보다 작고, 이후는 context가 아무리 길어도 상수다. linear-RNN 계열의 표준 serving이 점 ($O(1)$ decode 메모리·FLOPs, cache paging 불필요)은 그대로 성립하되, state가 선택들보다 8–16× 뚱뚱하다는 것이 Miras의 세금이다. per-sequence state 상주 비용, prefix caching 용 state checkpoint, speculative branch당 state 복제가 전부 그 배율로 커진다.

decode: RMW 트래픽이 지배한다. per-token 갱신은 memory MLP의 forward($W_2 z$ 와 $W_1 h$ 두 층 합쳐 $8d^2$ MAC) + backward($\approx$ forward의 2배, $16d^2$ MAC) + query 읽기 forward($8d^2$ MAC)에 element-wise retention/정규화가 더해져, head-layer당 대략 $28 - 32d^2$ MAC (FLOP로는 약 $56 - 64d^2$) — 같은 범위로 센 GDN 류 update(약 $3 - 4d^2$ MAC)의 대략 8–10× 수준이다 (이 책의 추산; 1 MAC = 2 FLOP 단위를 섞지 않도록 주의). 그러나 결정 변수는 FLOPs가 아니다. **매 token마다 $8d^2(16d^2)$ state 전체를 read-modify-write** 해야 하므로 decode는 memory-bandwidth-bound이고, 이 라인의 decode state RMW 트래픽 논증 (→ 10 장)이 Miras에서 8

- $16\times$ 배율로 적용된다. attention decode 가 KV cache 를 read-only 로 스트리밍하는 것과 달리, 여기는 write 가 절반이다.

prefill/훈련: chunkwise = GEMM 골격. (13-13) 의 chunkwise 형태 덕에 prefill 과 훈련은 GLA/Titans/TTT 식 chunkwise kernel 과 같은 골격이다: chunk 당 $|W_0 K_{(n)} - V_{(n)}|^{p-1} K_{(n)}^\top$ 류 batched GEMM + element-wise epilogue, 순차 의존은 chunk 경계에만 남는다 (L/C step). Miras 고유의 kernel 고려사항 네 가지: (1) Moneta 의 $\|A_t\|_q$ 는 경계마다 $8d^2$ entry 전체에 대한 global norm reduction — GEMM phase 사이에 끼는 reduction kernel 이다. (2) Memora 의 softmax/log-sum-exp 는 token 축이 아니라 지정된 parameter slice 마다 수행된다 ([Miras Eq. 21] 과 § 13.3.7: W 가 MLP weights 일 때 slice 별 적용) — reduction 범위와 비용은 구현 선택에 달렸지 단일 $8d^2$ 전역 reduction 이 강제되는 것은 아니다; 대신 state 가 유계 · 정규화되어 저장될 저장에 원리적으로 우호적이다 (단 log 가 소값의 양자화 오차를 증폭한다). (3) Yaad 는 $\ell_1 \cdot \ell_2$ 두 branch gradient + mask 로 element-wise 작업이 약 $2\times$, token 당 residual-norm reduction 하나가 추가된다. (4) smooth surrogate 들 ($\tanh, \sqrt{x^2 + \epsilon}, \arctan$) 은 값싼 element-wise epilogue 다. gate hypernetwork(rank 32 – 64) 의 비용은 무시 가능하다.

batching 과 hybrid. per-request fast-weight state 는 shared-weight batching 을 깨뜨린다는 1 장의 Rosetta 항목이 그대로 적용되고, state 가 $8 - 16\times$ 커진 만큼 grouped-GEMM decode 의 per-request working set 도 그 배율로 커진다. hybrid 변형 (-H) 은 SWA 를 교차하므로 window 크기의 KV cache($O(wd)$) 가 recurrent state 위에 다시 얹힌다 — “cache 없는 serving” 은 순수 변형에만 해당한다.

증거의 한계. 논문에는 wall-clock 수치, kernel 구현, decode throughput 비교가 전혀 없다 — 효율 증거는 FLOPs-매칭 perplexity [Miras Fig. 3] 하나다. “fast parallelizable training” 은 측정이 아니라 Titans/TTT chunkwise 레시피에서 상속된 구조적 · 점근적 주장으로 읽어야 한다. 이 부재는 Miras 만의 흠이 아니라 6 편 전체의 공백이며 (decode wall-clock 은 라인 어디에도 없다), 15 장의 TNT 가 처음으로 훈련 쪽 wall-clock 을 내놓는다.

[평가] 설계 관점에서 아까운 지점 하나: elastic net(식 (13-9)) 의 hard forgetting(0-스냅) 은 state sparsity 를 만들어 내는 gate — 즉 state 압축 · 양자화의 자연스러운 후 — 인데, 출하된 세 모델 어디에도 실리지 않았다. serving 경제학에 가장 직접적으로 닿는 변형이 실증 없이 카탈로그에만 남은 셈이다.

13.8 한계와 bridge-out

논문이 남긴 것. 첫째, **memory algorithm 축이 비어 있다.** 세 모델 모두 plain GD 다. momentum(Titans), implicit GD(Longhorn), multi-step(DeltaProduct), Newton(Mesa) 이 새 bias·gate 와 합성되는지는 열린 문제로 명시된다 — 논문 스스로 Newton 과 다른 optimizer 를 future work 로 지목한다. 둘째, **inner objective 가 여전히 per-token single-pair 다.** 모든 bias 가 “지금 이 (k_t, v_t) 하나” 를 얼마나 잘 저장하는가만 묻고, 최근 c 개 token 이 함께 잘 저장되어 있는가는 아무도 묻지 않는다. 셋째, **이론이 없다.** 새 bias·gate 어느 것에도 capacity, regret, state-tracking 급의 결과가 없다 — online convex optimization 의 도구함을 통째로 수입했지만 그보증이 요구하는 convexity 를 2-layer MLP memory 가 깨기 때문이다 (§ 13.3.2 의 [평가] 참조). 넷째, chunkwise 근사 (stale gradient, 경계 비선형, lag token) 의 오차는 미정량이고 C -ablation 도 없다. 다섯째, $(p, q) = (3, 4)$ 는 sweep 의 승자일 뿐 데이터 통계와 잇는 이론이 없으며, 과제별 · 적응형 p 는 미답사다. 여섯째, $8 - 16\times$ state 와 $3 - 4\times$ update 비용이 production batch 에서 품질 이득을 정당화하는지 — serving 경제학 — 는 수치가 전무하다.

[평가] 이 논문의 실질 기여는 세 모델보다 좌표계다. Moneta/Yaad/Memora 의 수치는 1.3B 에서 인상적이지 않지만 그 스케일에서 멈추고, 반면 attentional bias / retention gate / 4 축이라는 어휘는 이후 모든 논문이 무상으로 사용한다. 통일 주장 자체에도 결이 있다: Table 1 은 각 모델의 “원 설계방정식” 수준의 동일시이며, 논문 각주가 인정하듯 retention 열은 ℓ_2 류를 뭉뚱그린 것이다. 분류학적으로는 정확하되, 각 칸의 등호를 구현 수준의 등가로 읽으면 과독이다.

Bridge-out → 14 장 (Atlas). Miras 가 열어 둔 세 문제 — (i) 비어있는 optimizer 축, (ii) per-token 단일 쌍 objective, (iii) 부재하는 capacity 이론 — 는 그대로 다음 논문의 목차가 된다. [Atlas] (Atlas: Learning to Optimally Memorize the Context at Test Time, arXiv: 2505.23735) 는 셋을 한꺼번에 공격한다: (i) inner loop 에 Muon 을 이식해 — momentum 을 Newton – Schulz 로 semi-orthogonalize — 병렬화 가능한 근사 2 차 memory update 를 만들고, (ii) 최근 c token 의 window 전체를 함께 최적화하는 Omega rule 로 “token 의 surprise” 를 “context 의 surprise” 로 바꾸며 ($c=1$ 이 delta rule/Titans, $c=L$ 이 Mesa-layer 로 퇴화하는 스펙트럼), (iii) matrix memory $O(d_k) \rightarrow$ polynomial feature $O(d_k^p) \rightarrow \phi^*$ 무한 capacity(= softmax attention) 의 정식 capacity 이론으로 § 13.3.4 의 attention 행 — “attention 은 압축하지 않는 memory” — 에 정리를붙인다. Miras 가 지도를 그렸다면, Atlas 는 그 지도의 빈 축들을 최적화이론으로 채우러 간다. 명칭도 이때 바뀐다: test-time training 이아니라 test-time memorization 이라는 Atlas 의 고집이 왜 단순한수사가 아닌지 — 14 장에서 본다.

ch14. Atlas: Learning to Optimally Memorize the Context at Test Time

14.1 Bridge-in: Miras 가 남긴문제

이 장의 논문은 [Atlas] (Atlas: Learning to Optimally Memorize the Context at Test Time, arXiv:2505.23735; Ali Behrouz, Zeman Li, Praneeth Kacham, Majid Daliri, Yuan Deng, Peilin Zhong, Meisam Razaviyayn, Vahab Mirrokni, Google, 2025-05-29 v1) 다. 출발점은 13 장이 닫으며 남겨 둔 세 개의 공백이다.

[Miras] (It's All Connected, arXiv:2504.13173; 이 책은 framework 이름 Miras 로 통칭한다) 는 sequence model 전체를 (memory 구조 $\times$ attentional bias $\times$ retention gate $\times$ learning algorithm) 의 4 축 설계 공간으로 재편했다. 그러나 그 4 축 중 네 번째 축 — inner loop 의 **learning algorithm** — 은 선언만 되고 사실상 비어 있었다. Miras 가 실제로 출하한 세 모델 Moneta·Yaad·Memora 는 전부 plain gradient descent 로 inner objective 를 최적화하며, Newton 법류의 고차 optimizer 는 명시적으로 future work 로 미뤄졌다 ($\rightarrow$ 13 장). 첫 번째 공백이다.

두 번째 공백은 objective 의 시야다. Titans 도 Miras 의 세 모델도, 그리고 DeltaNet 계열 전부가, inner loss 를 **현재 token 하나의 (k_t, v_t)** 쌍에 대해서만 세운다. 식 $(M1)/(M2)$ 의 $\ell(W_{t-1}; k_t, v_t)$ 가 그것이다. 어떤 모델도 “직전 c 개 token 이 함께 잘 저장되어 있는가” 를 objective 수준에서 묻지 않았다. [Titans] (Titans: Learning to Memorize at Test Time, arXiv:2501.00663) 의 momentum 이 부분적보완이기는 하다 — [Atlas § 4.1] 의 각주는 Titans 를 “모든 과거 token 에 대해 momentum 이 만들어 내는 implicit decay 가중치로 memory 를 최적화하는, 병렬화를 유지한 예외” 로 자리매김한다. 그러나 momentum 은 **per-token gradient 들의 지수 감쇠 합일 뿐**, loss 자체는 여전히 token 하나짜리다. surprise ($\rightarrow$ 12 장) 의 언어로 말하면, 여러 token 에 걸친 사건은 어느 한 token 도 개별적으로는 놀랍지 않으면서 전체로는 기억할 가치가 있을 수 있는데, per-token loss 는 이 경우를 구조적으로 놓친다.

세 번째 공백은 이론이다. 이 라인은 “고정 크기 state 에 문맥을 압축한다” 를 반복해 왔지만, 그 state 가 **몇 개의 연상을 저장할 수 있는지에 대한** 정량적 정리는 Titans 에도 Miras 에도 없다. 5 장의 고전 Hopfield capacity 는 binary pattern 과 outer-product write 에 대한 결과라서, deep MLP memory 와 gradient 기반 write 에는 그대로 이식되지 않는다.

[Atlas] 는 이 세 공백을 한 논문에서 전부 메우겠다는 논문이다. optimizer 축에는 Muon ($\rightarrow$ 2 장) 을 inner loop 안으로 이식하고, objective 축에는 sliding-window inner loss (**Omega rule**) 를 도입하며, 이론축에는 memory capacity 의 형식적 정의와 세 개의 정리를 제공한다. 그리고 부산물로 — 사실은 부산물 이상인데 — softmax attention 자체를 “capacity 가 무한한 associative memory” 로 회수하고, 그 자리에서 Transformer 의 strict generalization 두 가족 (DeepTransformers, Dot) 을 파생시킨다.

14.2 문제의식과 논문의 핵심주장

논문이 스스로 정의한 문제는 현대 recurrent model 의 세 가지 설계 결함이다 [Atlas § 1]. (1) **online 갱신**: memory 가 현재 token 만 보고 최적화되고 이전 state 는 retention 으로만 유지된

다 — 개별 token 의 greedy memorization 이며, 문맥 전체가 잘 저장되었는지는 아무도 묻지 않는다. (2) **제한된 memory capacity**: 구조와 key-value feature mapping 이 “완벽하게 매핑 가능한 쌍의 수” 를 제한한다. (3) **표현력 없는 memory 관리**: inner optimizer 가 거의 전부 1 차 gradient descent 라서, token 동역학의 1 차 정보에만 의존해 나쁜 local minima 에 수렴하고 질 낮은 key $\rightarrow$ value 매핑을 배울 수 있다. 이 셋이 각각 Omega rule, feature map, Muon 으로 대응된다는 것이 논문의 구도다.

이 논문에는 라인 전체의 어휘를 바꾸는 용어 주장이 하나 있다. [Atlas § 1] 은 “**test-time memorization**” 이라는 표현을 “test-time training” 대신 쓰겠다고 명시한다. 근거는: inner loop 가 하는일은 현재 global context 안에서의 저장과 인출뿐이고, pre-training 으로 학습된 core parameter(outer loop) 와 초기 state 는 전혀 갱신되지 않으며, memory 를 비우고 나면 새로운 독립 context 로 이월되는 persistent learning 이나 skill 습득이 없다는 것이다. 이 책도 Atlas 이후 문맥에서 이구분을 존중한다 — inference 시점에 weights 가 변한다고 해서 checkpoint 가 학습되는 것이 아니다. 이 구분 자체가 이 장의 논점 중 하나이며, § 14.4 에서 두 loop 의 경계로 다시 정확히 그린다.

핵심 주장을 논문 자신의 분류표로 요약하면 이렇다. [Atlas Table 1] 은 현대 recurrent model 들을 다섯 성질로 비교한다: (1) dynamic decay(data-dependent retention), (2) deep neural memory, (3) non-linear capacity(feature map 에 의한 초선형 capacity), (4) **locally optimal**(token 에 대한 근사 2 차 정보로 memory 를 관리), (5) **flexible context**(문맥의 어느 부분을 기억할지 유연하게 선택). attention 과 SWA 는 (4)(5) 를 non-parametric 하게 갖지만 state 가 자라고, 기존 recurrent 계열은 (1) – (3) 의 부분집합만 갖는다. Atlas 는 다섯을 전부 체크하는 유일한 parallelizable recurrent model 이라는 것이 논문의 자기위치 규정이다.

14.3 Core mechanism (통일표기)

이 절은 세 층으로 전개된다: capacity 이론 (왜 feature map 인가) $\rightarrow$ Omega rule(왜 window 인가) $\rightarrow$ Muon(왜 2 차 근사인가). 모든 수식은 통일 표기이며, 표준형 (M1) – (M4) 와의 차이로 서술한다. Atlas 의 통일형은 식 (M3) 다.

14.3.1 Capacity 이론: 고정 크기 memory 는 몇 쌍을 저장하는가

[Atlas § 3.1] 은 이 라인 최초로 **memory capacity** 를 형식적으로 정의한다: memory 가 정확히 (inner loss 0 으로) 매핑할 수 있는, 선형독립 key 를 가진 (k_i, v_i) 쌍의 최대 개수 m . 5 장의 고정 capacity(Hopfield 의 확률적 저장 한계) 와 달리, 이것은 정확 보간 (exact interpolation) 기준의 결정론적 정의다.

Proposition 1 [Atlas Prop. 1]: matrix memory $W \in \mathbb{R}^{d_v \times d_k}$ 가 $\ell(W; k_t, v_t) = \|Wk_t - v_t\|_2^2$ 를 gradient descent 로 최적화하면, 저장 가능한 쌍은 최대 $O(d_k)$ 개다. 증명의 뼈대는 순수한 rank 논증이다 [Atlas App. C]: 정확 저장은 $WK = V$ ($K = [k_1 \cdots k_m]$) 를 요구하고, vectorize 하면 $(K^\top \otimes I_{d_v}) \text{vec}(W) = \text{vec}(V)$ — 미지수 $d_k d_v$ 개에 방정식 $m d_v$ 개 — 이므로 $m \leq d_k$. 이 한계는 tight 하다: $m \leq d_k$ 이고 K 가 full column rank 이면 Moore – Penrose pseudoinverse 로 정확 보간해를 지을 수 있고, full-batch GD 는 (loss $\|Wk - v\|_2^2$ 의 계수 2 를 반영한) step size $0 < \eta < 1/\lambda_{\max}(KK^\top)$ 에서, 그리고 $W_0 = 0$ 초기화아래에서 minimum-norm 보간해로 수렴한다 (GD 의 implicit bias; $\eta < 2/\lambda_{\max}$ 는 $\frac{1}{2}$ -loss 관행의 값이다). 같은 rank 제약이 multi-head attention 의 “low-rank bottleneck” 이기도 하다는 지적이 붙는다.

이 명제의 함의를 독자의 언어로 옮기면: $d_k \times d_v$ 개의 파라미터를 가진 memory 가 d_k 개의 연상밖에 저장하지 못한다 — d_k, d_v 가 함께 커진다고 가정할 때 capacity 는 파라미터 수 $P = d_k d_v$ 에 대해 **sub-linear** 다 (예: $d_v = \Theta(d_k)$ 이면 $O(\sqrt{P})$). d_v 를 고정하면 $O(d_k) = O(P/d_v)$ 로 P 에 선형임에 주의. state 크기를 늘리는 것 (파라미터를 더 쓰는 것) 과 capacity 를 늘리는 것은 같은 일이다.

Theorem 1 [Atlas Thm. 1]: $L_{\mathcal{M}} \geq 2$ 층 MLP memory(입력 차원 d_k , hidden 차원 $d_h^{(j)}$) 는 최소 $O(d_k d_v)$, 최대 $O(d_k d_v \sum_{i=1}^{L_{\mathcal{M}}} \min_{j \geq i} d_h^{(j)} d_h^{(i+1)})$ 쌍을 저장한다. 증명은 ReLU MLP 의 piecewise-affine 구조를 쓴다: 고정된 activation pattern 위에서 MLP 는 하나의 affine 사

상 $A(\cdot) + B$ 이고, $m \leq \text{rank}(A)$ 는 합성 경로의 최소 폭들로 위에서 늘린다 [Atlas App. C]. 논문은 두 방향으로 결론짓는다: depth 는 표현력만이 아니라 **capacity 자체**를 올리며 (Titans 의 deep memory ablation 이 경험적으로 보였던 것의 이론적 대응), 그러나 상한은 여전히 (d_k, d_v) 에 대해 subquadratic 이라 deep memory 만으로는 초선형 capacity 에 도달할 수 없다고.

[평가] Theorem 1 을 검증된 정리로 읽어서는 안 된다. Prop 1 의 capacity 정의는 “ $\mathbb{R}^{d_k}$ 의 선형독립 key” 를 요구하므로 입력 key 에 대해서는 어떤 memory 든 $m \leq d_k$ 인데, Theorem 1 의 하한 $O(d_k d_v)$ 는 $d_v > 1$ 이면 이를 초과한다 — 즉 Theorem 1 은 입력 key 가 아니라 MLP 가 내부에서 lift 한 표현의 선형독립성을 세는, 원문이 명시하지 않은 다른 capacity 개념을 암묵적으로 쓴다. 또한 App. C 증명의 $m \leq \text{rank}(A)$ 는 value 의 선형독립을 가정하지 않아 하한을 깔끔히 세우지 못한다 (AK=V 에서 $\text{rank}(V) \leq \text{rank}(A)$ 는 나와도 $m \leq \text{rank}(A)$ 는 따라오지 않는다). 따라서 “depth 가 capacity 자체를 올린다” 는 검증된 정리가 아니라 원문의 주장이며, 이 책은 그 방향성만 (Titans 의 deep-memory ablation 과 정합하는 선에서) 인용한다.

남은 손잡이가 key 의 차원이다. key·value 차원을 직접 키우면 projection 파라미터가 차원 당 $O(d)$ 씩 늘고 긴 문맥에서 메모리 사용량이 커진다. 대신 [Atlas § 3.1] 은 separable kernel $\sigma(x, y) = \phi(x)^\top \phi(y)$ 를 key 와 query 에 적용한다. **polynomial feature map** $\phi_p(x) = [x^\beta]_{|\beta| \leq p}$ — 차수 p 이하의 모든 monomial 을 쌓은 벡터 — 를 쓰면 lifted 차원은 $D = \binom{d_k+p}{p} = \Theta(d_k^p)$ 가 된다.

Proposition 2 [Atlas Prop. 2]: lifted key 위의 matrix memory 가 $\ell(W; \phi_p(k_t), v_t) = \|W \phi_p(k_t) - v_t\|_2^2$ 를 최적화하면 capacity 는 최대 $O(d_k^p)$. 그리고 증명 [Atlas App. C] 은 더 강한 사실을 준다: $\text{rank}(W\Phi) \leq \text{rank}(\Phi) \leq D$ 이므로, **어떤 최적화 방법을 쓰든 D 쌍을 넘길 수 없다**. 즉 feature map 은 optimizer 와 독립인 capacity 상한 그 자체를 옮기는 손잡이고, optimizer (다음의 Muon) 는 그 상한 안에서 실제 도달 품질을 올리는 손잡이다 — 두 축이 직교한다는 것이 이 논문 설계의 논리적 골격이다. 그리고 이 정리 사슬은 5 장 § 5.4 의 dense-Hopfield 사슬 (Krotov $\rightarrow$ Ramsauer: energy 차수 = feature lift = capacity) 의 정식화다 — 그 사슬을 통과한 독자에게 Prop 2 는 corollary 처럼 읽히며, 이미 가진 $\phi_p \cdot \phi^*$ 직관을 그대로 재사용하면 된다.

polynomial map 에는 두 가지 추가 해석이 붙는다 [Atlas § 3.1]. 첫째, **Taylor 근사로서의 softmax**: $\exp(q^\top k) \approx a_0 + a_1 q^\top k + a_2 (q^\top k)^2 + \dots + a_p (q^\top k)^p$ [Atlas Eq. 5]. 계수 a_i 를 $1/i!$ 로 초기화하되 **학습 가능하게** 두면, polynomial kernel 은 “절단된, 학습 가능한 softmax kernel” 이 된다. 둘째, **input feature gating**: $a_i \rightarrow 0$ 은 차수 i 의 feature block 전체를 잘라내고, $a_1 \rightarrow 1$ 에 나머지 0 이면 $\phi(x) = x$ 로 돌아간다 — RNN 의 gate 를 memory 가 아니라 **입력 표현**에 적용한 것이다.

극한이 이 절의 결론이다. Kronecker self-tensoring 으로

$$\phi^*(x) = \left(1, x, \frac{x^{\otimes 2}}{\sqrt{2!}}, \frac{x^{\otimes 3}}{\sqrt{3!}}, \dots\right)^\top$$

를 정의하면 $\exp(q^\top k) = \phi^*(q)^\top \phi^*(k)$ 가 **정확히** 성립한다 [Atlas Eq. 22–23]. 따라서 softmax attention 은 무한 차원 feature 공간 위의 associative memory 이고, **capacity 가 unbounded 다** [Atlas § 4.2] ($\rightarrow$ 5 장 § 5.4: exponential 극한 = softmax attention). 1 장의 Rosetta 사전에서 “KV cache = 압축하지 않는 memory” 라고 썼던 대응이 여기서 정리의 형태를 얻는다: attention 이 긴 문맥 recall 에서 고정 state 모델을 이기는 이유는 신비가 아니라 capacity 상한 의 차이이다.

한 가지 표기 주의: 원문은 ϕ_p 를 § 3.1 에서는 “차수 $\leq p$ 의 모든 monomial” 로, [Atlas Eq. 22] 에서는 self-tensoring $x^{\otimes p}$ 로 두 번 다르게 정의한다. 두 정의는 lifted 차원의 스케일 ($\Theta(d_k^p)$) 에서는 같은 급이며, 이 책은 § 3.1 의 정의를 기본으로 쓴다.

14.3.2 Omega rule: token 이 아니라 context 를 memorize 한다

이 절 전체의 대비를 한 장으로 요약한 것이 원논문의 첫 그림이다 (그림 14-1). 왼쪽은 현재 token 하나의 surprise 로 memory 를 갱신하는 계열 (Titans·RWKV·DeltaNet·Longhorn·Moneta) 이

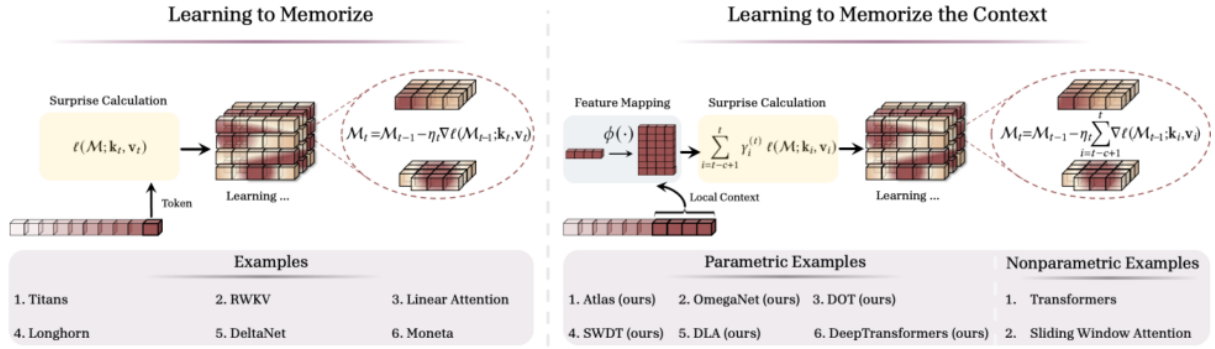


그림 14-1 — 개별 token 을 기억하는 것 (왼쪽) 과 context 를 기억하는것 (오른쪽) 의 대비. 왼쪽은 per-token surprise $\ell(\mathcal{M}; k_t, v_t)$ 로 memory 를 갱신하고, 오른쪽은 $\phi(\cdot)$ 로 lift 한 뒤 마지막 c 개 token 의 합 $\sum_{i=t-c+1}^t \gamma_i^{(t)} \ell(\mathcal{M}; k_i, v_i)$ 을최소화한다 (Omega rule). 하단은 각 방식에 속하는 모델 목록이며, 오른쪽의 “Nonparametric Examples” 가 Transformer·SWA 다. 출처: Behrouz et al., Atlas 축약 (arXiv:2505.23735), 원문 Fig.1 — 원저자의 그림 (third-party), 본서의 결과가 아님.

고, 오른쪽은 feature map $\phi(\cdot)$ 을 통과시킨 마지막 c 개 token 의 windowed loss 로 갱신하는 Atlas·OmegaNet·Dot 계열이다 — 그리고 오른쪽 맨끝에 그 windowed regression 의 비모수대응으로 Transformer 와 SWA 가 놓인다 (§ 14.3.5 의 논지를 미리 그린 배치다).

기존 online 모델들의 inner 문제는 retention gate(→ 13 장) 를 붙인 per-token 최적화다 [Atlas Eq. 6]:

$$\min_W \ell(W; k_t, v_t) + \text{Ret}_t(W, W_{t-1}).$$

반대쪽 극단은 전체 문맥에 대한 global 최적화 $\min_W \sum_{i=1}^t \ell(W; k_i, v_i)$ 다 [Atlas Eq. 7]. global 형태의 문제는 두 가지다 [Atlas § 3.2]: (1) **효율** — 매 step 최적화 제약이 늘고, 일반적인 비선형 memory 나 충분통계가 없는 objective 에서는 test time 에 모든 과거 key·value 를 cache 해야 하며 (고정 state 의 존재 이유 소멸; 단 linear least-squares 는 Sherman – Morrison RLS 로 충분통계+inverse state 만 유지하면 되는 예외 — 그실제 결격 사유는 순차 inverse update·추가 $O(d_k^2)$ state-linear memory 한정이다, 아래 Mesa-layer), 정확해를원하면 병렬화 불가능한 solver 가 필요하다. (2) **context pruning 불가** — 문맥이 중간에 바뀌거나 무관한 구간이 끼면 “전부에 대한 최적” 이오히려 해가 되는데, global objective 에는 특정 token 을 잘라낼 직접적 gate 가 없다.

Atlas 의 중간해가 **Omega rule** 이다. window 길이 $c \geq 1$ 에 대해 inner objective 를 마지막 c 개 token 의 gated 합으로 세운다 [Atlas Eq. 8 – 9]:

$$\min_W \sum_{i=t-c+1}^t \gamma_{t,i} \|\mathcal{M}(k_i; W) - v_i\|_2^2 \quad (14-1)$$

여기서 $\gamma_{t,i} \in [0, 1]$ 이 **window gate** 다: step t 의 window 안에서 i 번째 token 이 최적화에 참여하는 정도를 정하는 input-dependent gate 로, $\gamma_{t,i} \rightarrow 0$ 이면 그 token 을 최적화에서 **직접 (hard) 잘라내고**, $\gamma_{t,i} \rightarrow 1$ 이면 온전히 포함한다 — 논문의 표현으로 **in-context pruning** 이다 [Atlas § 3.2]. 이 gate 가 감당 가능한 이유가 바로 sliding window 구조다: step 당 필요한 gate 수가 c 개로 상수다. window c 를 키우면 SWA 의 얇은 banded 의존이 하삼각 전체로 번지는데 (그림 14-2), SWA 는 그 폭 안을 비모수로 훑는반면 Omega rule 은 같은 폭을 parametric memory 에 **누적**한다는 것이두 방식의 갈림이다. global 최적화 (Eq. 7) 에 input-dependent gate 를 달려면 prefix 길이만큼의 gate 값이 필요해 — 공유 gate-producer 의 파라미터는고정이지만 gate 값의 수와 이를 계산·저장하는 비용이 문맥길이에 따라 자라 — recurrent model 의 장점이 사라진다.

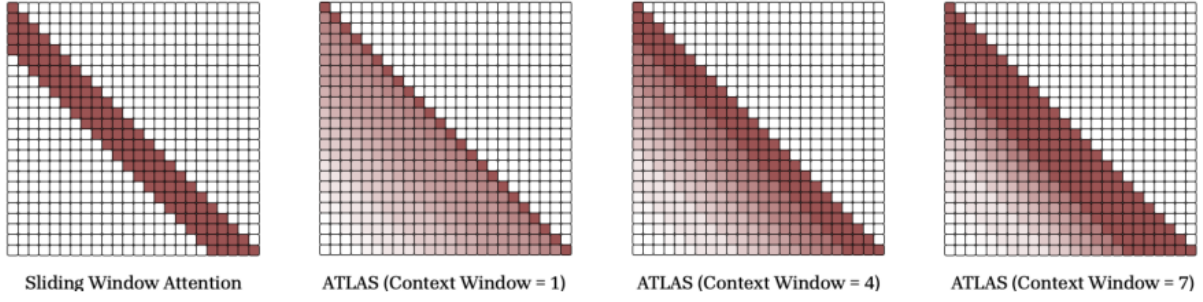


그림 14-2 — SWA 와 Atlas/OmegaNet 의 token 의존 구조비교 (하삼각 causal mask). SWA(맨 왼쪽) 는 폭이 고정된 banded mask 라 각 query 가 인접 c 개만 본다. Atlas 는 window c 를 키울 수록 ($c = 1, 4, 7$) 의존이 하삼각 전체로 번지는데, 이는 windowed loss 의 gradient 가 chunk 를 거쳐 이전 상태로 전파되기 때문이다 — 같은 banded 구조를 parametric memory 가 어떻게 “누적” 으로 바꾸는지 보여준다. 출처: Behrouz et al., Atlas 축약 (arXiv:2505.23735), 원문 Fig.2 — 원저자의 그림 (third-party), 본서의 결과가 아님.

Omega rule 은 계보 전체를 극한으로 회수한다 [Atlas § 3.2].

- $c = 1$: online delta rule($\rightarrow$ 5 장, 6 장), 즉 (M1) 이다. 여기에 momentum 을 더하면 **Titans 의 LMM update 가 정확히 나온다** [Atlas Eq. 12–13] — 통일 표기로 $S_t = \beta_t S_{t-1} - \eta_t \nabla_W \ell(W_{t-1}; k_t, v_t)$, $W_t = \alpha_t W_{t-1} + S_t$, 곧 (M2) 다. Titans 는 Omega 의 window-1 특수 사례다.
- $c =$ 문맥 전체, linear memory, $\gamma \equiv 1$: (regularized) least-squares $\min_W \sum_{i=1}^t \|W k_i - v_i\|_2^2$ 가 되고 [Atlas Eq. 14], 이를 Sherman – Morrison 재귀로 정확히 푸는 것이 Mesa-layer 다 ($\rightarrow$ 이 책 § 1.6 카탈로그의 대조군). 정확하지만 병렬화 불가, linear memory 한정, 그리고 hard gate 가 없어 pruning 불가 — Atlas 가자신을 차별화하는 세 가지 결격 사유다.

memory 의 은유로 요약하면 [Atlas § 3.2]: window loss 의 gradient 는 개별 token 의 surprise 가 아니라 **surprise of the context** — 마지막 c 개 token 의 context-aware 결합 — 이다. 12 장의 momentary/past surprise 2 분법이 여기서 세 번째 항을 얻는셈이다.

[해설] inference 어휘로 옮기면 이렇다. per-token delta rule 이 “cache 의 마지막 항목 하나에 대해서만 정합성을 유지하는 write” 라면, Omega rule 은 “최근 c 개 항목에 대한 batch write-repair” 다. 그리고 $\gamma_{t,i}$ 는 학습된 admission control 이다: retention gate α_t 가 이미 저장된 것 중 무엇을 남길지 (eviction) 를 정한다면, $\gamma_{t,i}$ 는 애초에 무엇을 저장대상에 넣을지 (admission) 를 정한다. 두 gate 는 같은 축이 아니다.

14.3.3 OmegaNet: rank-c 전이로 일반화

Omega rule + polynomial feature + GD + weight decay 가 **OmegaNet** 이다 [Atlas Eq. 10]:

$$W_t = \alpha_t W_{t-1} - \nabla_W \sum_{i=t-c+1}^t \gamma_{t,i} \|\mathcal{M}(\phi(k_i); W_{t-1}) - v_i\|_2^2 \quad (14-2)$$

식 (14-2) 는 정확히 표준형 (M3) 다. 기호를 전부 확정하면: W_t 는 fast weights(기본형은 2-layer residual MLP 의 weights, § 14.3.6), $\alpha_t \in [0, 1]$ 은 retention gate(남기는 비율; Atlas 원문도 같은방향), gradient 는 W_{t-1} 에서 평가되며, per-token step size 는 $\gamma_{t,i}$ 에 흡수된다(관행은 (M3) 아래 § 1.4 참조). linear memory($\mathcal{M}(z; W) = Wz$, $W \in \mathbb{R}^{d_v \times D}$) 로 특수화하면 closed form 이 나온다 [Atlas Eq. 11] — 이 책의 열벡터관행으로:

$$W_t = W_{t-1} \left(\alpha_t I - \sum_{i=t-c+1}^t \gamma_{t,i} \phi(k_i) \phi(k_i)^\top \right) + \sum_{i=t-c+1}^t \gamma_{t,i} v_i \phi(k_i)^\top. \quad (14-3)$$

(원문은 행벡터 관행이라 전이 항을 좌곱 ($\text{diag}(\alpha_t) - \sum \gamma \phi \phi^\top$) M_{t-1} 로 쓰며, $\text{diag}(\alpha_t)$ 표기는 retention gate 가 채널별 벡터일 수있음을 시사한다 — 스칼라 gate 이면 두 표기는 동치다.)

식 (14-3) 을 6 장의 카탈로그 옆에 놓으면 이 rule 의 정체가 선명해진다. DeltaNet 의 전이는 $W_{t-1}(I - \eta_t k_t k_t^\top)$ — rank-1 수정이다. Omega rule 의 전이는 c 개 key 의 gated 합 $\sum_i \gamma_{t,i} \phi(k_i) \phi(k_i)^\top$ — **rank- c 수정**이다. (gated) delta rule 의 rank-1 구조를 window 크기만큼의 rank 로 일반화한 것이 Omega rule 의 대수적 내용이다.

원문 대조 주의가 하나 있다. [Atlas Eq. 11] 은 value 항 $\sum \gamma v_i \phi(k_i)^\top$ 앞에 음 (−) 부호를 인쇄하는데, [Atlas Eq. 10] 의 ℓ_2 loss 를 실제로 전개하면 value 항의 부호는 양 (+) 이다 (delta rule 전개와 동일; → 5 장). 이 책은 전개가 맞는 (14-3) 의 부호로 쓰고, 원문의 부호는 [Atlas App. D.4] 와 Table 1 에도 걸쳐 있는 표기 관행 흔들림의 일부로 처리한다 (§ 14.3.4 말미).

[해설] 식 (14-3) 은 GEMM 으로 읽힌다. 전이 항은 $[\phi(K)]_{c \times D}$ 꼴 행렬의 gated Gram matrix 이고, write 항은 $(d_v \times c) \times (c \times D)$ GEMM — rank- c write 다. 1 장 Rosetta 의 “훈련 수식을 GEMM shape 로 읽는다” 의 전형으로, window 를 키우는 것은 write GEMM 의 내적 차원 c 를 키우는 것이다.

14.3.4 Atlas: inner loop 의 Muon — 근사 2 차 memory 관리

Omega rule 과 feature map 까지 갖춰도 optimizer 가 GD 면 1 차다. windowed loss 의 표면에서 나쁜 local minimum 에 앞으면 낮은 품질의 key $\rightarrow$ value 매핑이 저장된다는 것이 세 번째 진단이었다 [Atlas § 5]. Atlas 는 같은 objective (14-1) 을 **Muon**(→ 2 장; Jordan et al. 2024) 으로 최적화한다. 통일 표기로:

$$S_t = \beta_t S_{t-1} - \nabla_W \sum_{i=t-c+1}^t \gamma_{t,i} \|\mathcal{M}(\phi(k_i); W_{t-1}) - v_i\|_2^2, \quad W_t = \alpha_t W_{t-1} + \eta_t \text{NS}_\kappa(S_t) \quad (14-4)$$

[Atlas Eq. 32 – 33]. 기호: S_t 는 momentum buffer 로 memory 의 각 weight 행렬과 같은 shape 의 텐서 (12 장의 past surprise 와 같은 객체이되, 이제 **windowed** gradient 를 누적한다), β_t 는 momentum decay gate(원문 기호 θ_t), η_t 는 learning-rate gate, $\text{NS}_\kappa(\cdot)$ 는 Newton – Schulz 행렬 반복 κ 회다. NS_κ 는 $\kappa \rightarrow \infty$ 에서 S 의 SVD $S = U \Sigma V^\top$ 에 대한 **nearest semi-orthogonal matrix** $UV^\top$ 로 수렴한다 — raw momentum 대신 그 orthogonalization 을 update 로 쓰면 update 의 singular value 가 균등화되고, 논문은 이를 “token 에 대한 2 차 정보의 근사” 로 읽는다 [Atlas § 5]. 이것이 Table 1 의 **locally optimal memory** 성질이다: attention/SWA 는이 성질을 non-parametric 하게 (regression 을 그냥 풀어서) 갖고, GD 기반 RNN 전부는 갖지 못하며, Atlas 는 병렬화를 유지한 채 parametric 하게 달성한 최초의 사례라는 것이 논문의 주장이다 [Atlas Table 1, § 1].

κ 는 이 라인에서 처음 등장하는 종류의 손잡이다. [Atlas § 5] 는 κ 를 명시적으로 “internal **test-time compute** parameter” 로 규정한다: 반복을 더 돌리면 semi-orthogonalization 이 정확해지고 잠재적으로 memorization 이 좋아지되, inference FLOPs 가 선형으로 는다. state 크기를 전혀 바꾸지 않고 decode 연산량과 품질을 교환하는 dial 이다 (§ 14.7 에서 serving 관점으로 재론). 실전값은 $\kappa = 5$ (“NS-5”) 다.

재구현자를 위한 정직한 경고를 여기 두어야 한다. Atlas 의 recurrence 는 원문 안에서 세 곳의 표기가 서로 다르다. [Atlas Table 1] 의 행은 $S_t = \theta_t S_{t-1} - \nabla \ell$, $M_t = \alpha_t M_{t-1} - \eta_t \text{NS-5}(S_t)$ 로 쓰고 (심지어 Table 1 의 Titans 행은 η 와 θ 의 역할을 [Atlas Eq. 12 – 13] 과 맞바꿔 인쇄한다), [Atlas Eq. 33] 은 momentum 에 gradient 를 더하며 ($+\nabla$), [Atlas App. D.4 Eq. 57 – 58] 은 per-token 계수 $\eta_i^{(t)}$ 를 합 안에 넣고 $M_t = \alpha_t M_{t-1} + \text{NS-5}(S_t)$ 로 η_t 를 밖에서 제거한다. 이 중 **Table 1 의 행은 단순 표기 관행이 아니라 부호오탈자**다: S_t 에 $-\nabla \ell$ (descent 방향) 를 쌓아 놓고

M_t 에서 그 $NS(S_t)$ 를 다시 빼면 두 음부호가 겹쳐 loss ascent 가 된다. 서로 동등한 descent 관행은 Atlas Eq. 33 과 Atlas App. D.4 둘뿐이며, 알고리즘의 의미 (windowed gradient 의 decayed momentum 을 쌓고 orthogonalize 하고 retention 을 곱한 뒤 step) 는 이 둘에서만 동일하다. 이 책은 App. D.4 관행, 즉 (M2)/(M3) 의 부호로 식 (14-4) 를 고정한다.

14.3.5 Transformer 일반화 가족: DLA/SWLA → DeepTransformers → Dot

Atlas 의 두 번째 기둥은 같은 기계로 Transformer 를 다시 유도하는 절이다. 출발점은 softmax attention 의 재서술이다: attention 은 Nadaraya – Watson kernel regression

$$\mathcal{M}^*(q) = \arg \min_z \sum_{i=1}^L s(k_i, q) \|v_i - z\|_2^2 = \sum_{i=1}^L \frac{s(k_i, q)}{\sum_j s(k_j, q)} v_i$$

의 **non-parametric** 해다 [Atlas Eq. 17] (z 는 최적화 변수, $s(\cdot, \cdot)$ 는 exp kernel 유사도, $\mathcal{M}^*$ 는 argmin 최적해 — § 1.2 의 예약 의미 그대로다). 합을 마지막 c 개 token 으로 제한하면 그대로 sliding window attention(SWA) 이 나온다 [Atlas Eq. 18]. 이 대응이 주는 통찰이 이 절의 논지다: attention 은 자기 attentional bias 를 **global 하게, 비모수적으로** 최적화하고, 현대 recurrent model 은 **parametric online learner** 다. Omega rule 은 attention 의 window 수준 최적화를 parametric 세계로 수입한 것이고, SWA 는 Atlas 가 parametric 하게 푸는 것과 같은 windowed regression 의 비모수 해다 — 같은 문제, 다른 해 공간.

이 대응 위에 논문은 controlled baseline 두 개와 본 모델 두 개를 세운다.

DLA(Deep Linear Attention) [Atlas Eq. 19]: dot-product attentional bias $\ell = -\langle \mathcal{M}(\phi(k_t); W), v_t \rangle$ (최소화) + deep MLP memory + GD/decay. 부호 주의 — 원문 Eq. 19 는 양의 내적 $\langle \cdot \rangle$ 을 쓰고도 양의 Hebbian write 를 적어 부호 모순을 보인다: $+v_k^\top$ write 가 나오려면 loss 가 음의 내적이거나 gradient ascent 여야 한다. $\phi = \text{identity}$ 로 특수화하면 gated linear attention $W_t = \alpha_t W_{t-1} + v_t k_t^\top$ 로 붕괴하므로 ($\rightarrow$ 6 장), DLA 는 Hebbian/linear-attention 설정에서 **deep memory 의 기여만** 분리해 재는 baseline 이다. **SWLA** [Atlas Eq. 20] 는 같은 bias 를 window 합으로 바꾼 windowed gated Hebbian rule — linear memory closed form 은 $W_t = \alpha_t W_{t-1} + \sum_{i=t-c+1}^t \gamma_{t,i} v_i \phi(k_i)^\top$ — 로, **windowed objective 의 기여만** 분리한다.

DeepTransformers: DLA 의 ϕ 를 정확한 exponential map ϕ^* 로 바꾼 것이다 [Atlas Eq. 25]. linear memory 이면 $W_t = \sum_{i \leq t} v_i \phi^*(k_i)^\top$ 이고 읽기가

$$y_t = W_t \phi^*(q_t) = \sum_{i \leq t} v_i \exp(q_t^\top k_i)$$

— 정확히 **unnormalized softmax attention** 이다 [Atlas Eq. 26]. 따라서 DeepTransformers(deep memory + ϕ^*) 는 **unnormalized** exponential attention 의 strict generalization 이고, unnormalized Transformer 는 그 특수 사례 (linear memory + Hebbian rule + exp kernel) 다 — 정규화된 표준 Transformer 까지 포함하려면 softmax 분모를 계산하는 별도 state·정규화 연산이 필요하다 (아래 한계). sliding-window 판이 SWDT 다.

Dot(Deep Omega Transformer): Hebbian rule 자리에 Omega rule 을 넣는다 [Atlas Eq. 27]:

$$W_t = W_{t-1} - \nabla_W \sum_{i=t-c+1}^t \gamma_{t,i} \|\mathcal{M}(\phi^*(k_i); W) - v_i\|_2^2.$$

linear memory closed form 에서 $c = 1$ 로 두면 [Atlas Eq. 30 – 31] 이 나오는데, 논문은 이를 “**Transformers with Delta rule**” 이라 부른다: unbounded memory 가 새 (k, v) 를 attention 처럼 append 하는 동시에, 그 key 에 대해 이전 state 가 예측하던 value 를 빼서 교정한 다 — error-correcting attention 이다. 단, DeepTransformers 와 Dot 모두 softmax 의 분모 (normalizer) 가 빠진 **unnormalized** 형태로만 분석·정의된다 [Atlas Table 1 각주]. 정규화가 훈련된 모델에서 어떻게 복원되는지는 논문에 명시가 없다 — § 14.8 의 한계 목록에 다시 올린다.

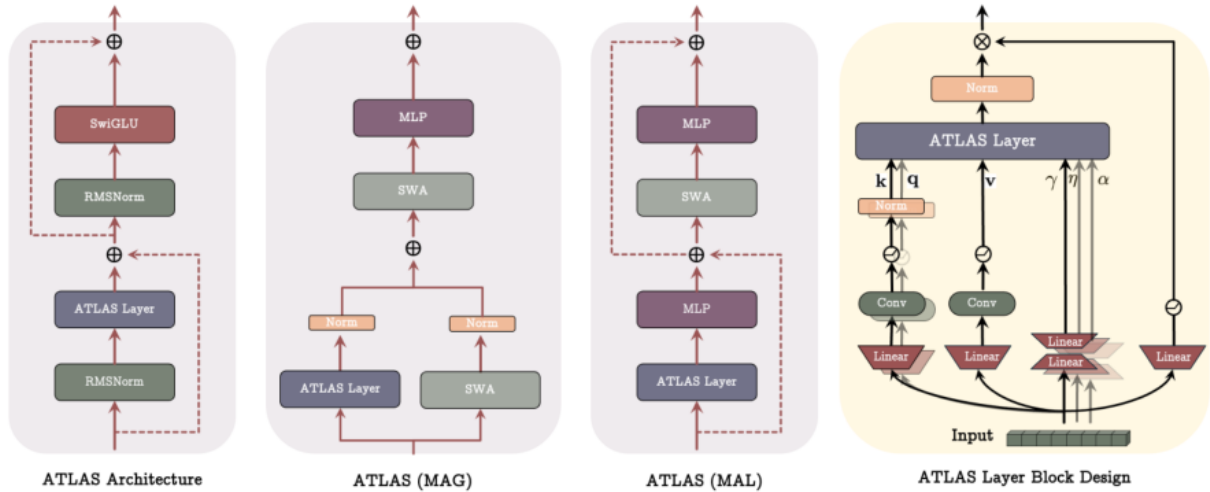


그림 14-3 — Atlas 아키텍처 (맨왼쪽)와 두 hybrid 구성 (MAG·MAL), 그리고 Atlas layer 의 block 설계 (맨오른쪽). block 도해는 입력에서 Q/K/V projection(linear + 크기-4 short conv) 과 세 gate γ, η, α 의 producer 가갈라져 나와 “ATLAS Layer” 로 들어가는 배선을 보여준다. § 14.7 의 회계가지적하듯 이 도해에는 feature map 차수 p ·sketch 차원·memory head 분할이 표기되지 않는다. 출처: Behrouz et al., Atlas 축약 (arXiv:2505.23735), 원문 Fig.3 — 원저자의 그림 (third-party), 본서의 결과가 아님.

이 가족이 개념적으로 사주는 것은 Table 1 의 다섯 번째 성질 **flexible context** 의 계보다: window + 학습된 γ gate 로 “무엇을 기억할지 고르는” 능력은 SWA, SWDT, OmegaNet, Dot, Atlas 가공유하며, 이 성질을 기준으로 보면 Atlas 계열은 “SWA 의 parametric 쌍둥이” 다.

14.3.6 Backbone, 표준 memory, Atlas++

architecture backbone 은 현대 recurrent LM 관행을 따른다 [Atlas § 5.1]: 층마다 Q/K/V linear projection + 크기 4 의 short convolution, 그리고 학습 안정화를 위한 key·query normalization. memory module 의 기본형은 이 라인의표준 deep memory 다:

$$\mathcal{M}(z; W) = z + W_1 \sigma(W_2 z)$$

— 2 층, expansion 4, GELU, residual [Atlas Eq. 42, App. E]. App. E 는 chunk 끝마다 layer norm 을 더한다. **Atlas++**는 memory 를 gated MLP 로 올린 변형이다 [Atlas Eq. 43]:

$$\mathcal{M}(z; W) = z + W_1 (\sigma(W_2 z) \odot W_3 z),$$

W_1, W_2, W_3 전부가 inner loop 에서 갱신되는 fast weights 다. 합성은 Titans 의 문법을 그대로 쓴다 ($\rightarrow$ 12 장): MAG(SWA 브랜치와 gate 결합), MAL(memory block 다음 SWA block), 그리고 BABILong 실험에서는 MAC 을 persistent memory tokens 없이 쓴다 [Atlas § 6.3]. 세 구성과 layer 내부 배선을 원논문이 한 장에 그려 둔다 (그림 14-3).

설계 선택과 담당 결함의 대응을 한 줄씩 정리하면 — window loss(14-1) 는 online 결함 (1) 을, ϕ_p/ϕ^* 와 deep/gated memory 는 capacity 결함 (2) 을, $NS_\kappa(S_t)$ 는 관리 결함 (3) 을 맡고, α_t (retention) 와 β_t (momentum) 는 Titans 에서 상속된 상태유지 장치다.

14.3.7 표기 대응표

표 14-1 — [Atlas] 원 표기 $\leftrightarrow$ 통일 표기 대응 (§ 1.7.3 기준; 아래쪽 6 행은 이 장의 장·국소 추가)

Atlas 원 표기	통일 표기	의미 / 주의
M_t — memory 상태	W_t	
η_t — learning-rate gate	η_t	동일
θ_t — momentum decay	β_t	△ Titans 와 정반대 배치였음 — 통일로 해소
α_t — weight-decay(forget) gate, $\alpha_t M_{t-1}$	α_t	동일 방향
S_t — momentum (windowed gradient 누적)	S_t	동일
$\gamma_i^{(t)}$ — window gates	$\gamma_{t,i}$	동일 (첨자 표기만정리)
c — sliding window 길이	c	동일. chunk C 와 구분
ϕ_p, ϕ^*	동일	
NewtonSchulz- k / NS5	NS $_{\kappa}, \kappa = 5$	반복 횟수 기호 k 금지
Eq. 32 – 33 vs App. D.4 의 부 호 ·step-size 차이	(M3) 의 부호로고정	“부호 관행 차이 (알고리즘 동 일)” 로 처리
capacity 의 m (저장 쌍 수), $D = \binom{d_k+p}{p}$	동일	장-국소
DLA/SWLA/OmegaNet/ DeepTransformers/Dot/ SWDT	고유명사 유지	§ 1.6 카탈로그 형태로 소개
b — 병렬화 chunk 크기 [Atlas § 3.3]	C	Titans 의 b 와같은 knob (§ 1.5: b 금지)
$t' = t - \text{mod}(t, b)$ — chunk 시작	$\xi(t, C)$	(M4) 의 gradient anchor
$u_t = \nabla \ell(M_{t'}; k_t, v_t)$ — 사전 계산 gradient [Atlas § 5.1]	$\tilde{g}_t$ (장-국소)	△ u_t 는 NL 의 Local Surprise Signal 로예약
M_s — sliding-window mask [Atlas § 3.3]	M_s (장-국소)	banded 0/1 mask
a_i — 학습 Taylor 계수 [Atlas Eq. 5]	동일	init $1/i!$, outer-loop 학습 대상
Θ, E — gate 대각 행렬 [Atlas Eq. 39]	산문으로 서술	broadcast scan 의 구현 세부

14.4 Outer-loop training vs inner-loop test-time learning

Atlas 에서 “이 gate 는 누가 학습하는가?” 라는 질문의 답은 예외 없이 하나다: **전부 outer loop 가 학습한다**. inner loop 가 학습하는 것은 memory 의내용물뿐이다. 논문 자신이 [Miras] 의 Definition 1 을 그대로 이어받아두 loop 를 정의한다 [Atlas Def. 1]: inner loop 는 $\theta_{\mathcal{M}} = \{W_1, W_2, \dots\}$ — memory module 의 파라미터 —만을 최적화하고, 그 동안 모델의 다른 모든 파라미터는 고정된 hyperparameter 다; outer loop 는 그 나머지 전부를 최적화한다.

표 14-2 — Atlas 의 두 loop: 무엇이 어디서 움직이는가

대상	loop	갱신 rule / 시점	비고
W_Q, W_K, W_V (projection), 크기-4 conv, backbone MLP·norm	outer (Θ)	AdamW, pre-training 중	inner loss 의 “hyperparameter”

대상	loop	갱신 rule / 시점	비고
gate-producer: $\alpha_t, \eta_t, \beta_t$ 와 c 개의 $\gamma_{t,i}$ 를 산출하는 outer-학 습 메커니즘 (입력 · 구 조 — 특히 $\gamma_{t,i}$ 가 x_t 만 의 함수인지 window token-pairwise feature 에도 의존하 는지 — 는 원문 미 공개)	outer (Θ)	AdamW	gate 값은 매 token 바뀌지만 gate 를만드 는 함수는 frozen
Taylor 계수 a_i (feature map gating)	outer (Θ)	AdamW, init $1/i!$	[Atlas Eq. 5]
memory 초기 상태 W_{init} (memory MLP 의 초기화)	outer 상태	pre-training 이 결정	매 context 시작 시 $W_0 = W_{\text{init}}$ 로 재시작
memory weights W_t (W_1, W_2 ; Atlas++ 는 W_3 까지)	inner	식 (14-4), 매 token/ chunk	context 가 끝나면 폐기
momentum buffer S_t	inner	식 (14-4), 매 token/ chunk	weight 행렬당 하나, 같은 shape
window buffer: 최 근 $c - 1$ 개의 $\phi(k), v$ 와 gate	inner (rolling)	ring-buffer append	크기 $O(c(D + d_v))$, $D = \binom{d_k+p}{p}$ (lifted $\phi(k)$ 차원), 길이 무관

training 무경험 독자가 가장 헷갈리는 지점을 짚는다. “memory 가 test time 에 훈련된다”는 문장은 **pre-trained checkpoint 가 변한다는 뜻이 아니다**. outer loop 는 pre-training 에서 단 한 번, “inner loop 가 어떻게갱신해야 하는가” 를 — gate-producer, projection, feature 계수, 초기상태의 형태로 — 학습한다. inference 에서는 그 배운 update 절차가 매 context 마다 W_{init} 에서 다시 실행될 뿐이다. 이것이 [Atlas § 1] 의 test-time memorization 명명이 가리키는 정확한 사실이고, 12 장의 bilevel 구조 ($\rightarrow$ 4 장) 가 Atlas 에서도 그대로 유지된다는 뜻이다.

outer gradient 는 inner loop 를 관통한다. update 식 (14-4) 는 그 자체로 미분 가능한 계산 그래프다 — gradient 평가 ($\nabla_W \ell$), gate 곱, momentum 누적, 그리고 Newton–Schulz 행렬 다항식까지 전부. outer loss $\mathcal{L}$ (next-token prediction) 의 backward 는 이 unrolled recurrence 전체를 거꾸로 타고 내려가므로, 예컨대 W_K 의 outer gradient 에는 $\nabla_W \|\mathcal{M}(\phi(k_i); W) - v_i\|^2$ 를 다시 k_i 로 미분하는 2 차 미분 성격의 항이들어간다 (TTT-Titans 와 같은 기제; $\rightarrow$ 8 장, 12 장). Atlas 특유의추가분은 backward 가 NS_κ 의 행렬 다항식까지 통과해야한다는 점이다. 따라서 아래의 chunkwise 형태는 단순한 추론 최적화가아니라 **실제로 미분되는 그래프의 정의**다.

chunkwise 병렬화 1 — window 는 banded mask 하나 값이다 [Atlas § 3.3]. 소박한 구현은 위치마다 c 개의 gradient 행렬 $\nabla_W \ell \in \mathbb{R}^{d \times d}$ 를 실체화해야 해서메모리 · IO 가 폭발한다. 대신 시퀀스를 크기 C (원문 b) 의 chunk 로 나누고, chunk 안 모든 gradient 를 chunk 시작 상태 $W_{\xi(t,C)}$ 에서 평가한다 — 9 장의 stale-snapshot 근사 그대로다. 설명편의상 $\gamma_{t,i} = \eta_t$ 로 두면, chunk 내부의 t ($\xi(t, C) < t \leq \xi(t, C) + C$) 에 대해

$$W_t = \left(\prod_{\tau=\xi+1}^t \alpha_\tau \right) W_\xi - \underbrace{\sum_{n=\xi+1}^t \left(\prod_{\tau=n+1}^t \alpha_\tau \right) \eta_n \sum_{i=n-c+1}^n \nabla_W \ell(W_\xi; k_i, v_i)}_{G_t}, \quad \xi := \xi(t, C) \quad (14-5)$$

[Atlas Eq. 16]. G_t 의 계산은 Titans 의 병렬 gradient 계산과동일하되, einsum broadcast

단계에 **sliding-window mask** M_s 를 하나 더 곱한다: $c = 1$ 이면 M_s 는항등 행렬이고, $c > 1$ 이면 각 대각 성분 바로 앞 $c - 1$ 개 위치를 추가로 1 로 둔 banded 하삼각 0/1 행렬이다. 각 token 의 gradient 는 한번만 계산되고, 자신이 속한 c 개 window 로의 기여는 mask 가 합산한다 — per-position gradient 텐서를 실체화하지 않고. 논문은 이 때문에 window 가 online($c = 1$) 버전 대비 유의미한 계산 overhead 를 더하지 않는다고주장한다 [Atlas § 3.3]. FlashAttention tiling 과의 차이는 9 장의 명제그대로다: tiling 은 bit-exact 지만, 여기의 C 는 gradient anchor 를 바꾸므로 계산되는 함수 자체를 바꾸는 **semantic hyperparameter** 다 (식 (M4); → 9 장). window c 는 objective 의 파라미터, chunk C 는 병렬화의 파라미터 — 둘은 독립 knob 이며 반드시 구분해야한다.

chunkwise 병렬화 2 — momentum 이 memory 에서 분리된다 [Atlas § 5.1]. Muon 까지 병렬화하는 열쇠는 구조적 관찰 하나다: gradient 를 chunk 경계 상태에서 평가하기로 한 순간, $\tilde{g}_t := \nabla_W \ell(W_{\xi(t,C)}; k_t, v_t)$ 는 chunk 전체에 대해사전 계산 가능하고, 그러면 momentum recurrence $S_t = \beta_t S_{t-1} - \eta_t \tilde{g}_t$ 는 진화하는 memory W 와 완전히독립인 linear scan 이 된다. unroll 하면

$$S_t = \left(\prod_{j \leq t} \beta_j \right) S_0 - \sum_{i \leq t} \left(\prod_{j=i+1}^t \beta_j \right) \eta_i \tilde{g}_i \quad (14-6)$$

[Atlas Eq. 39] — gate 들의 대각 행렬과 쌓아 둔 gradient 의 broadcast-and-matmul 로 chunk 내 모든 S_t 를 한 번에 얻는다 (1 장 Rosetta 의 scan/prefix-sum 대응 그대로). 그다음 NS_5 는 per-matrix 다항식 — 반복마다 $X \leftarrow aX + b(XX^\top)X + c'(XX^\top)^2X$ 꼴의 서너 개 matmul(계수는→ 2 장) — 이므로 chunk 내 전 위치의 S_t 에 **batched matmul 로동시에** 적용된다 [Atlas Eq. 40]. 마지막 남은 순차 부분은 $W_t = \alpha_t W_{t-1} + \eta_t NS_5$ -항의 gated 누적 scan 하나다 [Atlas Eq. 41]. 이 세 단계 — gradient 병렬, momentum scan, batched NS — 가 “근사 2 차 inner optimizer 를 가진 최초의 parallelizable recurrent architecture” 라는 주장 [Atlas § 1] 의 실체다.

outer 훈련 recipe [Atlas App. E]: FineWeb, T5 tokenizer(32K vocab), 훈련 문맥 4K(SWA 성분은 2K), outer optimizer AdamW(learning rate 4e-4, cosine schedule, batch 0.5M tokens, weight decay 0.1), 스케일별 peak LR 은 표 14-4 참조.

14.5 Concept ledger delta

표 14-3 — [Atlas] 가 ledger 에 더하고 바꾼 것

구분	개념	내용
신규	Omega rule (이 장 소유)	windowed inner objective (14-1); delta rule($c = 1$) 의 strict generalization, Mesa-layer($c = L$) 로의 보간
신규	window gate $\gamma_{t,i}$ (이 장 소유)	in-context pruning; step 당 상수 (c) 개라서 학습 가능한 hard gate 가 성립
신규	memory capacity (formal) (이 장 소유)	선형독립 key 의정확 저장 쌍수; matrix $O(d_k) \rightarrow$ deep MLP subquadratic $\rightarrow \phi_p$ 로 $O(d_k^p) \rightarrow \phi^*$ 는 unbounded

구분	개념	내용
신규	ϕ_p + 학습 Taylor 계수 a_i, ϕ^*	capacity 손잡이로서의 feature map; softmax attention = unbounded-capacity associative memory 의 정식화
신규	locally optimal memory	근사 2 차 정보에 의한 memory 관리 (Muon inner); Table 1 의 새 열
신규	flexible context	무엇을 기억할지 고르는 능력 (window + γ); Table 1 의 새 열
신규	NS_κ 반복수 = test-time compute dial	state 불변으로 decode FLOPs $\leftrightarrow$ 품질 교환하는 최초의 layer 내부 손잡이
신규	sliding-window mask M_s	window 합을 banded 0/1 mask 하나로 처리하는 chunkwise 구현 장치
신규 (모델)	OmegaNet, Atlas, Atlas++, DLA, SWLA, DeepTransformers, SWDT, Dot	§ 1.6 카탈로그에 등재된 여덟 이름
확장	surprise ($\rightarrow$ 12 장)	momentary/past 에 이어 세 번째형태: surprise of the context = windowed loss 의 gradient
확장	momentum-as-memory ($\rightarrow$ 12 장, 2 장)	raw momentum $\rightarrow$ NS_κ -orthogonalized momentum; optimizer-as-architecture
확장	attentional bias ($\rightarrow$ 13 장)	축의 두 번째 사례 per-token family 에 window 축이추가됨; Miras Table 의 두 열 확장판이 [Atlas Table 1]
확장	retention ($\rightarrow$ 13 장)	α_t (저장분의 eviction) 와 별개로 $\gamma_{t,i}$ (저장 전 admission) 라는 직교 gate 가 생김

구분	개념	내용
개명	test-time training → test-time memorization (이 장 소유)	inner loop 는 저장 · 인출일 뿐 학습이 아니라는 용어 교정; “진짜 continual learning 은 따로 필요하다” 는 후속작 — [NL] (Nested Learning: The Illusion of Deep Learning Architecture, arXiv:2512.24695) 과 [Sleep] (Language Models Need Sleep: Learning to Self-Modify and Consolidate Memories, arXiv:2606.03979) — 의복선

14.6 실험과 스케일

스케일과 구성 [Atlas Table 7, App. E]:

표 14-4 — 훈련 구성 ([Atlas Table 7])

모델	blocks	dim	heads	peak LR	tokens
170M	12	768	16	3e-3	15B
340M	24	1024	16	1.5e-3	15B
760M	24	1536	16	1.25e-3	30B
1.3B	18	2048	8	7e-4	100B

훈련 문맥은 4K, 데이터는 FineWeb 이다. 원문 § 6 의 Setup 문단은 스케일을 “340M, 400M, 790M, 1.3B” 로 적는데 [Atlas § 6], App. E 의 Table 7 과결과 표들의 열 제목 (760M/1.3B) 은 위 표와 같다 — 원문 내부불일치이며, 이 책은 Table 7 을 따른다. baseline 수치의 대부분은 재실행이 아니라 Titans·Miras·Gated DeltaNet 논문들에서 상속되었다는 점도 원문이 명시한다 [Atlas § 6, App. E].

Language modeling + commonsense reasoning [Atlas Table 2]. 1.3B / 100B tokens 에서: Atlas 는 Wikttext ppl 14.97 / LAMBADA ppl 10.98 / 평균 acc 57.62, Atlas++는 14.40 / 10.72 / 58.03, OmegaNet 은 14.91 / 11.26 / 57.23. 비교군은 Titans (LMM) 15.60 / 11.41 / 56.82, Gated DeltaNet 16.42 / 12.17 / 55.32, Samba(hybrid) 16.13 / 13.29 / 54.00, Transformer++ 18.53 / 18.32 / 52.25. Transformer 일반화 가족도 자기 비교군을 이긴다: DeepTransformers 평균 56.19, Dot 57.35 vs Transformer++ 52.25. 같은 순서가 760M 에서도 유지된다: OmegaNet 52.56 / Atlas 52.77 / Atlas++ 53.09 vs Titans 51.56, Transformer++ 48.69; hybrid 는 Atlas(MAG) 가 Wikttext ppl 18.62, 평균 53.08 로 MAL(19.07, 52.63) 보다 낫다 [Atlas Table 2].

S-NIAH (RULER) [Atlas Table 3]. 4K 로 훈련된 모델을 2K – 16K needle-in-haystack 에서 평가한다. 순수 recurrent 비교에서 Atlas 는 S-NIAH-N 16K 에서 84.0 으로 Titans 80.2 를 앞서고, DeltaNet(5.4)·TTT(4.4) 와는 자릿수가 다르다. hybrid 와 Transformer-like 가족은 더 강하다: Dot 은 전 설정에서 93.2 – 100(S-NIAH-W 16K 의 93.2 가 최솟값), Atlas(MAG) 는 S-NIAH-PK 16K 에서 98.6 — 훈련 문맥의 4× 외삽이다.

BABILong [Atlas § 6.3, Fig. 4]. MAC backbone(persistent memory tokens 없이) 으로 benchmark protocol 에 따라 fine-tune 한 설정이다. Atlas 는 1M token 까지 Titans 와 동급이 다가, 10M 에서 Titans 가 무너지는 지점에서 **+80% accuracy 를 유지한다** — 이 논문의 헤드라인

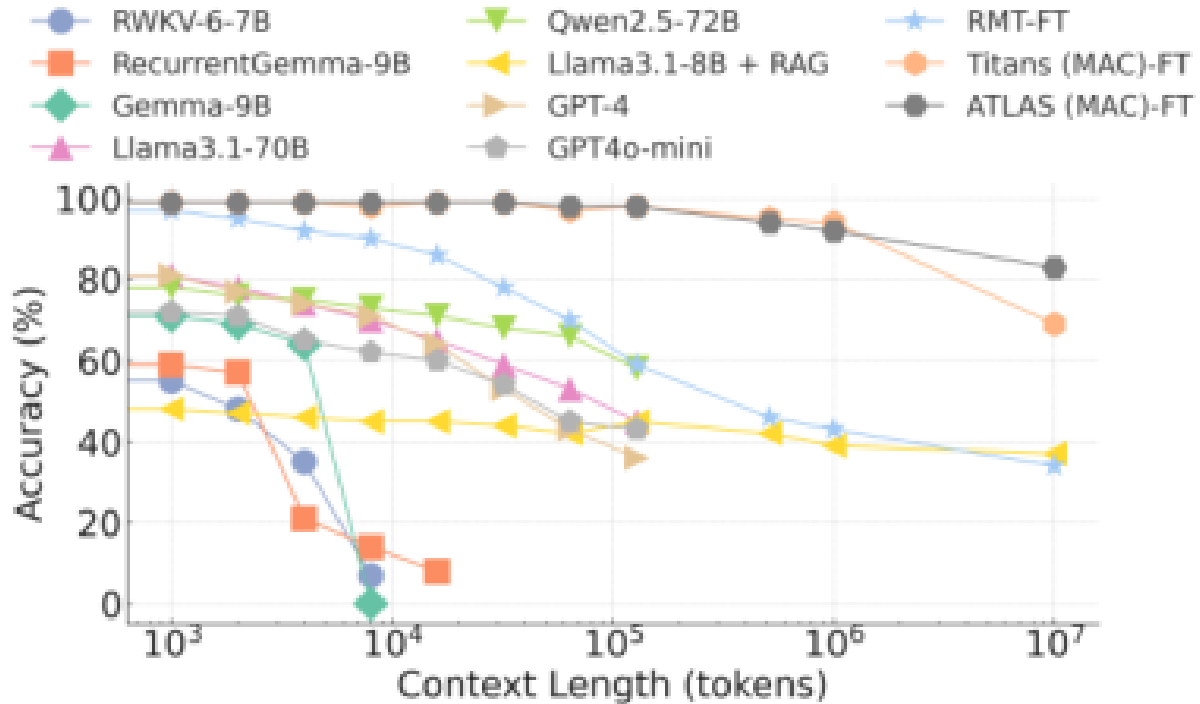


그림 14-4 — BABILong benchmark 에서 context length(가로축, 로그스케일) 에 대한 정확도. Atlas(MAC)-FT 가 10M token 까지 높은 정확도를유지하며, 1M 부근에서 무너지는 recurrent baseline(RWKV·RecurrentGemma·Gemma·Llama+RAG 등) 과 갈린다. 4K 훈련 문맥의 수 천 배를 외삽하는 구간이라는 점이 요지다. 출처: Behrouz et al., Atlas 축약 (arXiv:2505.23735), 원문 Fig.4 —원저자의 그림 (third-party), 본서의 결과가 아님.

long-context 주장이다 (그림 14-4). 논문은 이를 Muon(관리), polynomial kernel(capacity), context memorization(objective) 의 합작으로 귀속시킨다 [Atlas § 6.3].

MAD synthetic suite [Atlas Table 4]. 평균 Atlas 79.50 / OmegaNet 78.98 vs Titans 76.44, Transformers 75.46, Gated DeltaNet 71.04. 최대 격차는 memorization(91.4) 과 fuzzy recall 축이다.

In-context recall — 의무 caveat [Atlas Table 5]. SWDE/NQ/DROP/FDA/SQuAD/TQA 평균에서 **Transformer 가 여전히 이긴다**: 53.55 vs Atlas 43.70, OmegaNet 43.13, Titans 42.31, Gated DeltaNet 40.28. 특히 FDA 에서 72.5 vs 40.7 로 격차가 크다. Atlas 는 recurrent 중 최고이고 gap 을 좁혔지만 닫지 못했다 — 이것은 § 14.3.1 의 capacity 이론이 예측하는 방향 그대로다: ϕ^* 의 unbounded capacity 를 가진 비모수 memory(attention) 와 유한 state 의 parametric memory 사이의 격차는, Atlas 의 모든 장치를 넣고도 남는다.

Ablation — 의무 caveat [Atlas Table 6, 760M]. 완전한 Atlas: language modeling ppl 19.97 / reasoning acc 52.77. 성분 제거의 효과는: linear memory 로강등 21.03 / 49.74, $c = 1$ (window 제거) 21.98 / 49.26, polynomial mapping 제거 22.14 / 50.57, **Muon 제거 19.65 / 52.56**, gated-MLP memory 추가 19.53 / 53.09, +Attn(MAG) 19.90 / 53.08, +Attn(MAL) 20.26 / 52.63. 숫자를 정직하게 읽어야 한다: window($c = 1$ 대비 - 2.01 ppl) 와 feature map(- 2.17 ppl) 과 deep memory(- 1.06 ppl) 는 명확히 기여하지만, **Muon 을 제거하면 perplexity 는 오히려 개선된다** ($19.65 < 19.97$). Muon 이 산 것은 reasoning accuracy 0.21pt 뿐이다.

[평가] 이 ablation 은 Atlas 의 세 축 중 optimizer 축의 가치가 760M 스케일에서 미결임을 뜻한다. 논문 제목이 약속하는 “optimally memorize” 의 optimality 는 window 와 capacity 축이 대부분 벌어들인 것이고, “locally optimal” 을 담당하는 Muon 의 기여는 과제 의존적이며 perplexity 기준으로는 음수다. κ 를 test-time

compute dial 이라 부르려면 κ 에 대한 품질 곡선이 단조임을 보여야 하는데, 그 측정
은 논문에 없다.

window 크기와 scaling [Atlas Fig. 5, Fig. 8]. 고정된 global context 에서 window c 를
키우면 성능이 단조 개선된다 — γ gate 가 필요할 때 잘라내 주기 때문이라는 것이 논문의 설명이다
[Atlas § 6.6]. 파라미터 수와 훈련 문맥 길이에 대한 scaling 도 baseline 보다 유리하다 [Atlas Fig.
8]. MQAR 에서는 memory 크기당 정확도 기준으로 DeltaNet 류 대비 최고를 보고한다 [Atlas
§ 6.5, Fig. 7].

Learnability micro-study [Atlas § 6.4]. $d = 256$ 의 온라인 regression 으로 memory 후
보 (작은 MLP + Adam/RMSprop/SGD) 의 학습 능력 자체를 쟀 부속실험이다. 다섯 종류의 목표
사상 중 low-rank·MLP·attention·출력 사상은 잘배우지만, 과거 입력의 기억을 요구하는 두 설정
(attention+MLP, SWA+MLP) 에서 최악이고, 특히 **SWA 목표가 full-attention 목표보다 더 어렵**
다. 저자들의 가설: online 학습기는 오래된 입력을 ‘잊지’ 못해서, 잊기가 필수인 sliding-window
사상에서 더 크게 실패한다 — gated windowed objective 의 필요성을 뒷받침하는 정황 증거다.
단, 이것은 γ gate 가 그 문제를 실제로 푼다는 분리 실험이 아니라 동기 부여 실험이다.

스케일의 정직한 상한. 이 논문의 모든 품질 주장은 1.3B params / 100B tokens 에서 끝난
다. 라인 전체의 실증 상한이기도 하다 ($\rightarrow$ 12 장, 15 – 17 장 공통). 그리고 훈련이든 decode 든
wall-clock 수치는 한 건도 없다 — “significant overhead 가 없다”, “병렬화된다” 는 전부 구조
논증이지 측정이 아니다.

14.7 Systems/serving 합의

state 크기의 회계. Atlas 의 per-layer state 는 세 덩어리다: (1) memory weights W — 표준형
이면 $W_1 \in \mathbb{R}^{d_m \times 4d_m}$, $W_2 \in \mathbb{R}^{4d_m \times d_m}$ 로 $8d_m^2$ 원소 (d_m = memory 폭), (2) momentum S — 같은
shape 로 또 $8d_m^2$, (3) window buffer — $O(c(d_k + d_v))$ 로 무시 가능. 합계 $\approx 16d_m^2$ 원소이며 문맥
길이와 무관하다. Transformer KV cache 는 layer 당 $2L_{\text{ctx}}d$ 원소로 문맥에 비례한다.

[해설] 같은 정밀도를 가정하고 등호를 놓으면 crossover 문맥길이는 $L^* = 16d_m^2/(2d)$
다. memory 폭을 model 폭과 같게잡으면 ($d_m = d$) $L^* = 8d = 1.3\text{B}$ 구성 ($d = 2048$)
에서 약 16K tokens 다. 즉 16K 보다 짧은 문맥에서는 KV cache 가 더 작은 상태이
고, BABILong 의 10M-token 구간에서는 Atlas 의 state 가 KV cache 의 수백분의일
이다. 단 이 계산에는 두 개의 미지수가 있다. 첫째, 실제 구현의 memory 가 head 별
로 쪼개지는지, d_m 이 얼마인지 논문에 없다. 둘째, ϕ_p 는 memory 첫 층의 입력 폭을
 $\Theta(d_k^p)$ 로 불리므로 (차수 2 만 해도 sketch 전 $\sim d_k^2$), **구현된 차수 p 와 sketch 차원이
공개되지 않은 한 state 크기는 계산할 수 없다** — 구현 p -sketch 차원 · memory head
분할은 원문본문에도 [Atlas App. C], [Atlas App. E], [Atlas Fig. 3] 의라벨에도 없
다. capacity 의 이득은 cache 성장이 아니라 state 크기와 matmul 폭으로 지불된다
— 그 청구서의 액수가 논문에 없다. 5 장 § 5.4 가 예고한 “비용 계산” 은 그래서 여기
서 **구조** — capacity 의 지불 통화가 state byte 와 matmul 폭이라는 것 — 로 확정된
다; 절대값 계산은 구현 차수미공개로 여기서도 불가하다는 것까지가 이 장의 결론이다.

decode 의 state 트래픽. 매 token, 각 layer 의 memory 는 read-modify-write 다: W 와 S
를 읽고, gradient·momentum·NS 를 계산하고, 둘 다 다시 쓴다. 트래픽은 token 당 $\approx 2 \times 16d_m^2$
원소의 읽기+쓰기로, DeltaNet 류의 $d \times d$ matrix state 대비 (expansion 4 의 2 층 + momentum
때문에) 원소 수로 16 배 급이다 — 단 이는 표준 memory · 명시적 ϕ_p lift 없을 때의 값이고, lift 가
있으면 첫 층 입력이 $D = \binom{d_k+p}{p}$ 로 커져 더 크다 (§ 14.7 [해설] 의 미공개 차원 caveat). 이 RMW
스트림이 decode 의 실질 대역폭예산을 정하며, KV cache 처럼 append-only 가 아니므로 캐시
계층에상주시키는 전략이 달라진다 — 상세한 배치 논의는 10 장과 Part III 의몫이다.

연산의 성격은 전부 dense matmul 이다. windowed gradient 는 banded einsum(M_S ;
식 (14-5)), momentum 은 broadcast scan(14-6), NS_5 는 행렬 다항식 — 논문이 명시적으로
“tensorize computations and maximize matmuls” 를 설계 목표로 선언한다 [Atlas § 3.3].

Titans 대비 추가 상수는 NS_5 다: update 당 $\cdot$ weight 행렬당 반복 5 회 $\times$ 서너 개의 정방 matmul $\approx 15 - 20$ 개의 추가 matmul. chunk 안에서 전 위치에 batch 되므로 GEMM shape 는 좋다. window 자체는 mask 하나라서 훈련 시 거의 공짜다 — c 는 훈련 비용을 거의 바꾸지 않는 품질 knob 이라는 것이 Atlas 의 병렬화가 만든 특이한 경제학이다.

병렬화 구조. intra-chunk 는 (gradient, momentum, NS) 3 단 전부 병렬이고 inter-chunk 만 gated scan 이다. momentum recurrence 가 memory state 에서 분리된다는 § 14.4 의 관찰이 구조적 핵심으로, sequence-parallel 훈련이 자연스럽게 얻힌다. 단 chunk 사슬 자체는 여전히 순차다 — 이 사슬을 끊는 것은 [TNT] (TNT: Improving Chunkwise Training for Test-Time Memorization, arXiv:2511.07343) 의 reset 이 처음이다 ($\rightarrow$ 15 장).

serving 관점의 새 dial: κ . NS 반복수는 state 크기 $\cdot$ checkpoint 형식을 전혀 건드리지 않고 decode FLOPs 와 (주장되는) memorization 품질을 교환한다. speculative decoding 이 요청 단위의 품질/지연 트레이드라면 κ 는 layer 내부의 트레이드다 — 다만 § 14.6 의 [평가] 대로 품질 곡선의 단조성이 이미 측정이므로, 현재로서는 “존재하는 knob” 이지 “검증된 knob” 이 아니다.

batching 과 hybrid. per-request 로 변하는 fast weights 는 shared-weight batching 을 깨뜨린다 — 요청마다 다른 W_t, S_t 로 같은 layer 를 실행해야 하므로 decode 는 grouped-GEMM 형태가 된다 ($\rightarrow$ 1 장 Rosetta, 10 장). MAG/MAL hybrid 는 2K SWA 브랜치를 달고 있어 serving 이 rolling KV cache 와 recurrent state 를 동시에 관리해야 한다. 반대로 DeepTransformers/Dot 은 정확한 ϕ^* 아래에서 유한한 recurrent state 가 없으므로 attention 과 같은 비용 구조다 — 이들은 serving 효율이 아니라 Transformer 대비 품질로 경쟁하는 갈래다.

요약하면: kernel 표면의 신규 워크로드는 (i) chunk 전체에 걸친 batched NS-5 fusion, (ii) banded gradient mask, (iii) outer 훈련의 backprop-through-NS 세 가지이며, 어느 것도 공개 구현이 없다. 그리고 다시 — throughput/latency 수치가 논문에 전무하므로, serving 시점의 quality-per-FLOP 이야기는 전부 미정량이다.

14.8 한계와 bridge-out

논문 안팎의 한계를 정리한다.

1. **retrieval gap 미해소** [Atlas Table 5]: 53.55 vs 43.70. capacity 이론은 격차의 방향을 설명하지만, 남은 격차가 capacity 한계인지, inner 최적화의 한계인지, lossy 고정 state 의 본질인지는 이 논문으로 판별되지 않는다.
2. **Muon 의 기여가 equivocal** [Atlas Table 6]: perplexity 는 제거 시 개선. κ -품질 곡선 미측정. optimizer 측의 실증은 후속 (NL 의 레벨 관점; $\rightarrow$ 16 장) 으로 넘어간다.
3. **feature map 의 구현 미공개:** 실제 차수 p , sketch, 결과차원이 없어 state $\cdot$ FLOPs 회계가 불가능하다 (§ 14.7). ϕ^* 와 deep memory 의 결합도 linear closed form 밖에서는 정의되지 않는다 — 무한 차원 feature 는 실체화할 수 없으므로, “w/o Polynomial Mapping” ablation 의 존재는 실제 Atlas 가 ϕ_p 를 쓴다는 정황이다.
4. **unnormalized 일반화:** DeepTransformers/Dot 은 softmax 분모를 버린 채 분석된다. softmax 의 안정성 $\cdot$ 품질 중분모의 뭉치 얼마인지는 열린 문제다.
5. **capacity 이론의 이상화:** 정리들은 “선형독립 key 의 정확 보간” 기준이고, Theorem 1 은 단일 activation region 논증이다. 실전 retrieval 품질의 근사적 대리 지표이지 그 자체가 아니다.
6. **표기 삼중 불일치** (§ 14.3.4): 재구현자는 Table 1 / Eq. 32 – 33 / App. D.4 중 하나의 관행을 골라야 한다. 의미는 하나다.
7. **chunkwise staleness 무정량:** gradient anchor $W_{\xi(t,C)}$ 근사의 오차 한계가 없고 (라인 공통; $\rightarrow$ 9 장), chunk C 와 window c 의 상호작용도 미탐구다.
8. **window 스케줄:** Fig. 5 는 “클수록 좋다” 까지만 말한다. c 의 비용 최적, 적응적 선택, 학습된 스케줄은 없다.

Bridge-out — [TNT] 로. Atlas 까지 와서 이 라인은 objective(Omega), capacity(feature map), optimizer(Muon), 이론(capacity 정리) 을 다 갖췄다. 그런데 여섯 편을 통틀어 이 시점까

지 **wall-clock** 수치가 0 건이다. 모든 것이 chunkwise 트릭 — stale snapshot에서의 gradient — 위에 서 있는데 그 근사는 무정량이고, chunk 크기 C 는 throughput knob으로만 취급될 뿐 계산되는 함수를 바꾸는 **semantic knob**이라는 사실 (→ 9 장)은 아무도 논점으로 삼지 않았다. 특히 Atlas는 “훈련한 C 와 다른 C 로 serve 하면 무슨 일이 일어나는가”를 묻지 않는다 — decode는 $C = 1$ 의 세계인데 훈련은 큰 C 에서 이뤄지는 구조적 mismatch가 방치되어 있다. [TNT]는 정확히 여기서 시작한다: 훈련·추론 chunk-size mismatch를 발견하고 (550M Titans가 $C = 64$ 훈련 후 $C = 64$ 평가에서 ppl 13.78, $C = 8$ 평가에서 36.45 [TNT Fig. 2]), 계층적 memory와 학습된 W_{init} 로의 주기적 reset, 그리고 two-stage 훈련으로 chunk 경제학 전체를 재설계한다 (→ 15 장). Atlas가 “무엇을 얼마나 잘 기억하는가”의 정점이라면, TNT는 “그걸 만들고돌리는 데 얼마가 드는가”라는, 이 책의 독자가 처음부터 묻고 있던 질문의장이다.

그리고 하나 더, 조용한 복선이 있다. Atlas가 “test-time **memorization**”이라는 개명을 고집한 순간 — in-context 적응은 학습이 아니라고 선을 그은 순간 — “그러면 진짜 continual learning은 어디서 오는가”라는 질문이 라인의 장부에 미결로 올라갔다. 그 답이 [NL]의 nested level들과 [Sleep]의 wake/sleep lifecycle이다 (→ 16 장, 17 장).

ch15. TNT: Improving Chunkwise Training for Test-Time Memorization

15.1 Bridge-in: 전작에서 남은문제

[Atlas] (Atlas: Learning to Optimally Memorize the Context at Test Time, arXiv:2505.23735, → 14 장) 까지 이라인은 update rule 의 “무엇” 을 완성했다. inner objective 는 [Miras] (It’s All Connected, arXiv:2504.13173) 가 attentional bias 라는 family 로 일반화했고, retention 도 같은 논문이 재이론화했으며, Atlas 는 objective 의 범위를 window 로 넓히고 (Omega rule) inner optimizer 를 Muon 으로 올렸다. 식 (M2) 와 (M3) 의 성분표는 사실상 채워졌다. 그런데 Titans 부터 Atlas 까지 세 편 어디에도 **wall-clock 수치가 없다** — perplexity·capacity 정리·ablation 은 있어도, systems 엔지니어가 첫 페이지에서 찾을 “훈련에 몇 시간” 이라는 숫자가 없다.

없는 데는 이유가 있다. 이 라인의 모든 모델은 chunkwise-parallel training(→ 9 장) 이라는 트릭 — chunk 안의 모든 inner gradient 를 chunk 시작상태 $W_{\xi(t,C)}$ 에서 평가하는 stale-snapshot 근사, 즉 식 (M4) — 위 에 서 있 고, 이 트릭은 두 가지를 미해결로 남겼다. 첫째, 근사의 품질: chunk 크기 C 를 키우면 gradient 가 낡아 (staleness) 품질이 떨어지고, 줄이면 kernel 이 잘게 쪼개져 hardware 가 논다. 실무는 C 를 16-64 에 고정해 왔지만 이 타협의 비용은 아무도 정량화 하지 않았다. 둘째, C 의 이중 신분: 병렬화 knob 이면서 동시에 — 식 (M4) 가 계산하는 함수 자체를 바꾸므로 — semantic hyperparameter 다 (→ 9 장). 9 장에서 명제로 세운 이 이중성의 실증적 발견자가 바로 이 장의 논문이며, Atlas 까지는 훈련 C 와 다른 C 로 serving 하면 어떻게 되는지 묻은 적조차 없다.

한 가지 수치가 사태의 심각성을 요약한다. deep memory(→ 12 장) 계열의 훈련은 품질이 좋은 작은 chunk 에서 peak FLOPs 대비 5-10% 미만의 utilization 으로 돌아간다 — [TNT] (TNT: Improving Chunkwise Training for Test-Time Memorization, arXiv:2511.07343) 가 LaCT(Zhang, Bi, et al. 2025, arXiv:2505.23884) 를 인용해 보고하는 값이다 [TNT § 3]. 독자의 어휘로 말하면, 이 라인의 모델들은 지금까지 MFU 한 자릿수의 workload 였다. 표현력 논쟁 이전에, 이 훈련 경제학이 해결되지 않으면 어떤 Titans 후속도 대규모로 갈 수 없다.

[TNT] 는 이 지점을 정면으로 겨냥한 라인의 systems 편이다. 새아키텍처가 아니라 **훈련 paradigm** 이고, 주장하는 것도 표현력이 아니라 throughput 과 chunk 경제학이다. 논문 제목의 TNT 는 “Titans iNside Titans” 또는 “TTT iNside TTT” 의 약자다 [TNT § 1 각주 1] — 이름부터가 memory 안에 memory 를 중첩하는 계층 구조를 가리킨다.

15.2 문제의식: 세 개의 challenge

[TNT] 는 deep memory module 의 실용화를 막는 문제를 세 개의 challenge 로 명명해 분해한다 [TNT § 3]. 이 분류법 자체가 논문의 기여중 하나다.

Challenge 1 — 효율적 훈련 구현의 부재. deep memory 는 fine-grained learning signal 을 위해 작은 chunk(16-64 tokens) 를 요구하는데 [TNT § 3, Sun et al. 2024 (arXiv:2407.04620)]

인용], 작은 chunk 는 작은 연산을 대량 생산해 훈련을 compute-bound 가 아니라 memory-bound 로 만든다. linear memory 계열 (GLA, DeltaNet, Mamba-2, → 6.7 장) 은 SRAM 상주 chunkwise kernel 로 이를 회피하지만, 그 kernel 들은 **linear state transition** 과 **chunk 간 closed-form 분해**에 근본적으로 의존한다. deep memory 의 recurrence 는 MLP forward/backward 와 chunk 사이 LayerNorm 같은 비선형을 통과하므로 그 kernel 이 이식되지 않는다 [TNT § 1, § 3]. 비선형 recurrence 를 sequence 길이 방향으로 병렬화하는 것은 parallel scan 이 적용되지 않는, 오래된 미해결 문제다 [TNT § 1].

Challenge 2 — compression 과 retrieval 의 domain 불일치. inner loop 는 $\mathcal{M}(\cdot; W)$ 를 key 를 입력으로 value 를 맞히도록 적합시킨다 (write 는 $k_t \mapsto v_t$). 그런데 읽기는 q_t 로 한다. 학습된 함수의 입력 domain 밖에서 함수를 평가하는 셈이고, 이 train/use domain shift 가 retrieval 품질을 깎는다는 것이 논문의 진단이다 [TNT § 3 Challenge 2].

[해설] 이것은 memory 내부에서 일어나는 미니어처 train/test mismatch 다. attention 에는 이 문제가 구조적으로 없다 — softmax attention 의 read 는 q 와 모든 k 의 내적을 명시적으로 계산하므로 “key domain 에 적합된 파라미터 함수” 라는 중간물이 아예 없다. 압축하는 memory 로 넘어오는 순간에만 생기는 세금이다.

Challenge 3 — 고정 pre-training chunk 크기에 대한 성능 민감성. 논문의 새 실증 발견이다. 550M Titans 를 $C = 64$ 로 pre-train 한 뒤 inference chunk 크기를 바꿔 가며 validation perplexity 를 재면: $C = 8$ 에서 36.45, 16 에서 34.15, 32 에서 24.23, **64 에서 13.78(최적)**, 128 에서 15.5, 256 에서 17.88, 512 에서 22.4 [TNT Fig. 2]. 훈련 때 쓴 chunk 크기에서만 최적이고, 양쪽으로 벗어나면 급격히 나빠진다. 특히 왼쪽이 인상적이다 — 더 작은 chunk 는 더 신선한 gradient 를 뜻하므로 직관적으로는 inference 에서 더 좋아야 하는데, 실제로는 ppl 이 $2.6\times$ 이상 폭발한다. 모델이 훈련 해상도에 **over-specialize** 된 것이다 [TNT § 3 Challenge 3]. 이것이 이 책이 **chunk-size mismatch** 라 부르는 현상이다: 같은 checkpoint 가 serving 때 memory update 를 얼마나 자주 적용하느냐에 따라 전혀 다른 품질을 낸다. 이 발견은 이상적 serving 구성 — decode 에서 chunk 크기 1, 즉 매 token online update — 을 위협한다. 큰 chunk 로 싸게 훈련한 baseline 은 이미 $C = 8$ 에서 ppl 이 36.45 로 폭발하므로 (Fig. 2 의 최소 inference chunk 가 8 이다 — $C = 1$ 자체는 측정되지 않았다), chunk 1 로 직행하기 어렵기 때문이다.

위 문단이 나열한 perplexity 값들이 그리는 곡선이 그림 15-1 이며, V 자바닥이 정확히 훈련 chunk 크기 $C = 64$ 에 걸려 있다 — 이 비대칭 절벽, 특히 더 신선한 gradient 를 뜻하는 작은 chunk 쪽이 오히려 더 나쁜 것이 chunk-size mismatch 의 시각적 정의다.

세 challenge 를 관통하는 논문의 핵심 주장은 이렇다: **훈련 효율과 inference 성능을 한 개의 chunk 크기가 동시에 결정하도록 놔두지 말고, 두 단계로 분리 (decouple) 하라.** Stage 1 은 hierarchical memory 로 최대 throughput 의 pre-training 을 하고, Stage 2 는 전체 비용의 약 5–8%(구성에 따라; 최고 품질 4-local 구성은 약 8.3%)로 작은 chunk 에 fine-tune 해서 chunk-1 decode 를 품질과 정렬한다. 결과 요약: 150M Titans 기준, 가장 정확한 Titans baseline($C = 8$) 대비 목표 loss 도달까지 최대 $17.37\times$ 빠르면서 평균 perplexity 는 오히려 개선 (23.09 vs 25.07) 되고 vanilla Transformer(23.58) 도 이긴다 [TNT Table 1, Table 2].

선행 완화책에 대한 논문의 비판도 기록해 둔다 [TNT § 1]. LaCT 는 큰 chunk 를 window attention 과 결합하지만, 이는 비효율을 우회할 뿐 해결이 아니고, memory 와 attention 을 섞어 분석을 흐리며, decode 에 필요한 chunk ~ 1 을 외면한다. log-linear attention (Guo et al. 2025, arXiv:2506.04761) 은 계층적이지만 linear memory 에 국한된다.

15.3 Core mechanism (통일표기)

15.3.1 두 연산으로 정식화된 deep memory

[TNT § 2.1] 은 라인 전체가 암묵적으로 쓰던 구조를 두 개의 per-token 연산으로 깔끔하게 정식화한다. fast weights W 가 sub-network $\mathcal{M}(\cdot; W) : \mathbb{R}^d \rightarrow \mathbb{R}^d$ 를 파라미터화하고, 매 token 마다:

$$W_t = W_{t-1} - \eta_t \nabla_W \ell(W_{t-1}; k_t, v_t), \quad y_t = \mathcal{M}(q_t; W_t) \quad (15-1)$$

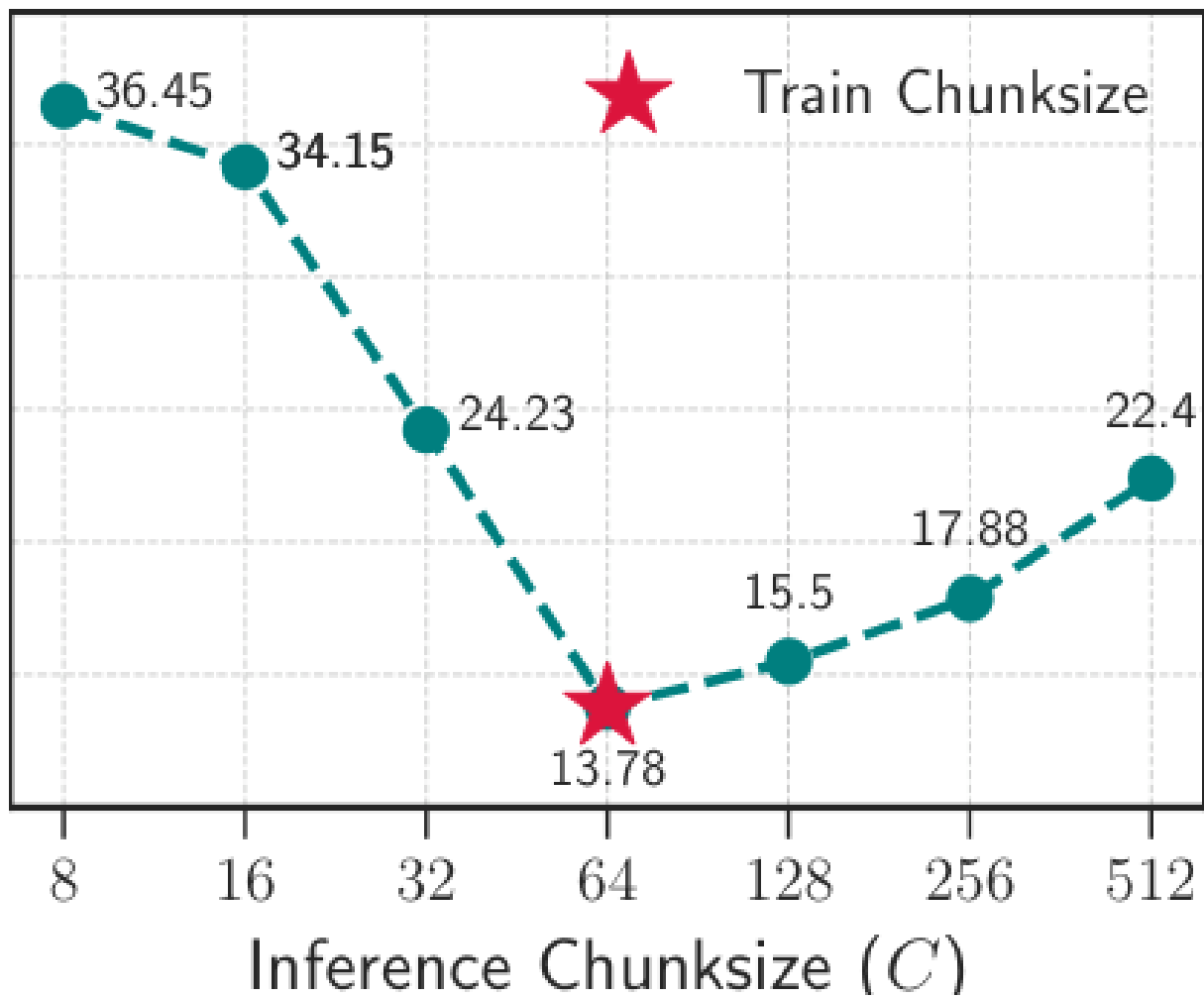


그림 15-1 — 550M Titans 를 $C = 64$ 로 pre-train 한 뒤 inference chunk 크기만 바꿔 가며 validation perplexity. 훈련 chunk 크기 (별표, $C = 64$) 에서 13.78 로 최적이고, 양쪽으로 벗어나면 급격히나빠진다 — 특히 더 작은 chunk(왼쪽) 가 직관과 반대로 36.45 까지 폭발한다. 출처: [TNT] (arXiv:2511.07343), 원문 Fig.2 —원저자의 그림 (third-party), 본서의 결과가 아님.

첫 식이 **Memory Compression**(write) [TNT Eq. 1], 둘째가 **Memory Retrieval**(read) [TNT Eq. 2] 다. $x_t \in \mathbb{R}^d$ 는 입력 token 표현, slow-weight projection 이 $q_t, k_t, v_t \in \mathbb{R}^d$ 를 만들고, ℓ 은 self-supervised inner loss 로 기본형은 associative-memory regression $\ell(W; k, v) = \|\mathcal{M}(k; W) - v\|_2^2$ 이다 (원문 예시는 MSE [TNT § 2.1]). η_t 는 학습된 per-token inner learning rate 로 [TNT § 2.1], 시간 첨자가 붙었으니 § 1.2 상 data-dependent 게이트 — 라인의관행 ($\rightarrow$ 12 장) 대로 slow weights 가 token 마다 산출하는 값이다 (다만 [TNT] 는 산출 head 구조를 명시하지 않는다). 식 (15-1) 은 정확히 표준형 (M1) 이고, (M2) 와 달리 momentum β_t 도 retention gate α_t 도 없다 — 실수가 아니라 의도된 단순화이며 § 15.6·§ 15.8 에서 다시 다룬다 [TNT App. D].

recurrence $W_t = W_{t-1} - \dots$ 는 W 에 대해 비선형이다. $\nabla_W \ell$ 이 deep net $\mathcal{M}$ 의 forward 와 backward 를통과하기 때문이다. 이 한 문장이 Challenge 1 의 근원이다: 상태 전이가 비선형이면 linear attention 계열의 chunk 간 closed-form 전파가 성립하지 않는다.

15.3.2 chunkwise parallel training 복습

token-serial recurrence 를 병렬화하기 위해 라인 전체가 쓰는 방법이 chunkwise compression, 즉 식 (M4) 다 (유도와 일반론은 9 장 소유; 여기서는 TNT 표기와의 접속만 확인한다). chunk 크기 C , chunk 시작 offset $\xi(t, C) = C \lfloor (t-1)/C \rfloor$ 에 대해:

$$W_t = W_{\xi(t,C)} - \sum_{\tau=\xi(t,C)+1}^t \eta_\tau \nabla_W \ell(W_{\xi(t,C)}; k_\tau, v_\tau), \quad y_t = \mathcal{M}(q_t; W_t) \quad (15-2)$$

[TNT Eq. 3–4] 가 이 형태다.¹ chunk 안의 모든 gradient 가 frozen 된 chunk 시작 상태 $W_{\xi(t,C)}$ 에서 평가되므로, chunk-start 상태가주어지면 per-token gradient 들은 서로 독립이고 하나의 batched forward/backward 로 계산된다. per-token 상태 W_t 는 그 gradient 들의 누적 합 (cumulative sum) 이다. 남은 직렬 의존성은 chunk 경계하나뿐이다: n 번째 chunk 의 마지막 상태 W_{nC} 가 $n+1$ 번째 chunk 를 seed 한다 [TNT § 2.2]. staleness 가 품질을, matmul 크기가 throughput 을 결정하므로 C 가 두 세계의 유일한 교환 변수가 된다. 이근사의 오차 한계 (bound) 는 [TNT] 를 포함해 6 편 어디에도 없다 — 전부실증이다.

15.3.3 Stage 1 — hierarchical memory: global + N local

TNT Stage 1 의 구조를 한 문장으로 요약하면: 큰 chunk 로 도는순차적 global memory 하 나가 long-range context 를 담당하고, 주기적으로초기화되는 N 개의 병렬 local memory 가 fine-grained 정보를 담당한다 [TNT § 4.1.1]. 이것이 이 책이 hierarchical memory 라 부르는구조다.

이 구조 전체를 한 눈에 담은 것이 그림 15-2 다: global 경로는 raw query 를 Retrieval 에 곧장 넣는 반면 local 경로만 Q-K Projection (§ 15.3.4) 을거치며, 아래 블록의 tile 들이 병렬로 쌓인 모습이 뒤에서 설명할 periodic reset 의 context parallelism 을 그대로 시각화한다.

Global memory. 상태 W^g (원문 기호 V ; value 행렬과의 충돌 때문에 개명 — 표 15-1) 는 매우 큰 chunk 크기 C_g (실험에서 2048) 로 식 (15-2) 의 표준 chunkwise recursion 을 돈다:

$$W_{(n+1)C_g}^g = W_{nC_g}^g - \sum_{t=nC_g+1}^{(n+1)C_g} \eta_t \nabla_{W^g} \ell(W_{nC_g}^g; k_t, v_t), \quad n = 0, \dots, L/C_g - 1 \quad (15-3)$$

¹원문의 chunk 시작 함수는 $\xi(i, j) := i - (i \bmod j)$ 로 0-based 관행이다 [TNT § 1]. 이 책은 1-based token 인덱스에 맞춘 $\xi(t, C) = C \lfloor (t-1)/C \rfloor$ (§ 1.2) 를 쓴다. 합의 하한이 원문 $\tau = \xi(t, C)$ 대신 $\tau = \xi(t, C) + 1$ 이 되는 것은 이 인덱스 관행차이일 뿐 알고리즘은 동일하다.

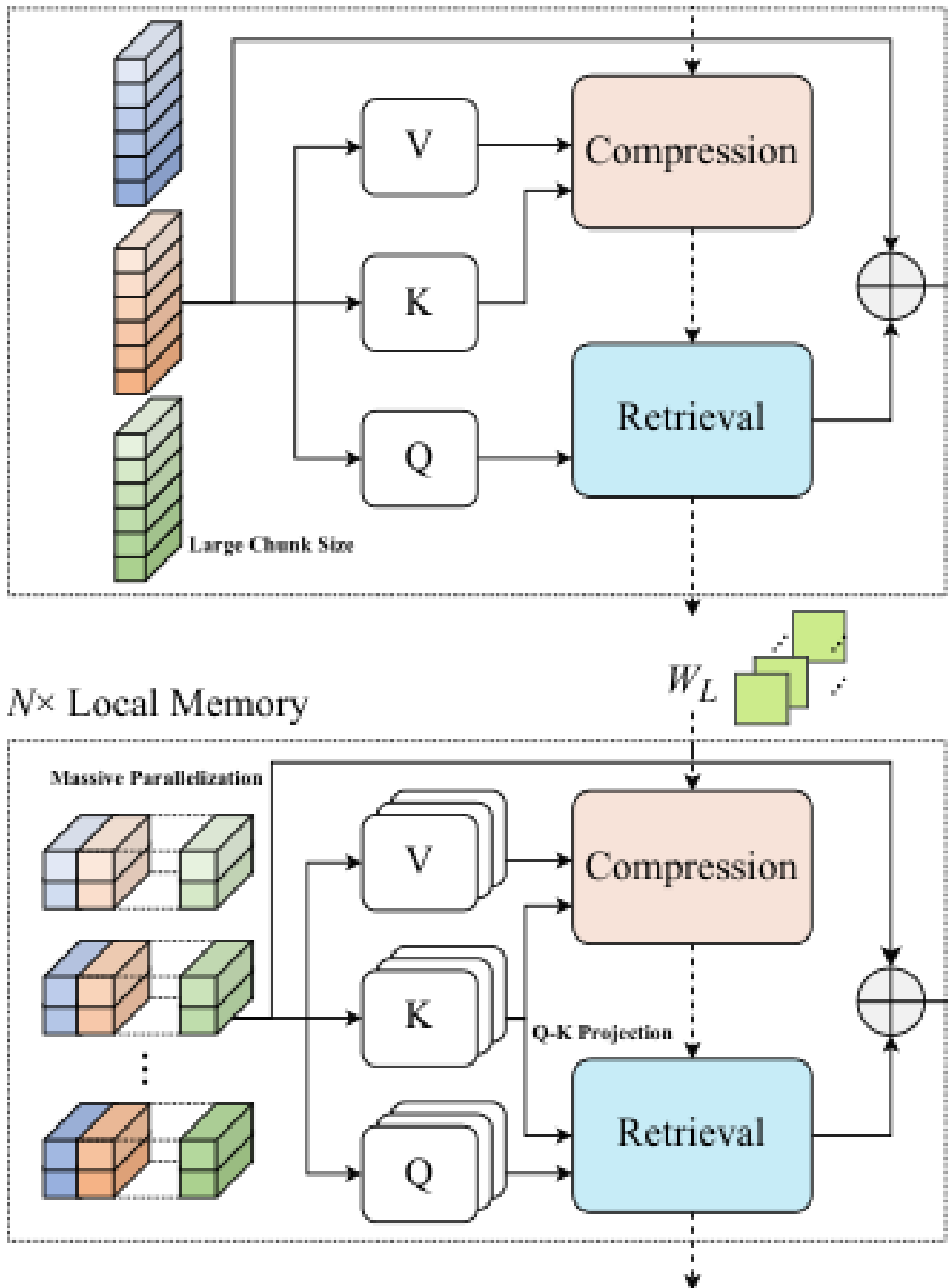


그림 15-2 — TNT Stage 1의 아키텍처 개관. 위 블록: 큰 chunk 크기로 순차적으로 도는 하나의 global memory(long-range 담당). 아래 블록: 학습된 초기 상태 W_L (본서 표기 W_{init})에서 주기적으로재초기화되어 대량 병렬화 (Massive Parallelization) 되는 N 개의 local memory. 두 memory 모두 Compression(write)·Retrieval(read) 두 연산을 갖고, 두 경로의 출력이 합산되어 y_t 가 된다 — 단 Q-K Projection은 local 경로에만 붙는다. 출처: [TNT] (arXiv:2511.07343), 원문 Fig.3 —원저자의 그림 (third-party), 본서의 결과가 아님.

[TNT Eq. 5] 다.² 상태는 sequence 전체를 관통해 순차적으로 이월되므로 long-range 정보가 살아남고, C_g 가 크므로 update 는 드물고 크고 dense 한 matmul 이 된다 — 설계상 compute-bound 다. 16K context 에 $C_g = 2048$ 이면 sequence 당 직렬 handoff 는 8 번뿐이다.

Local memory 와 periodic state reset. 핵심 혁신은 local 쪽에 있다. 기본형 ($N = 1$) 에서 local memory W^1 은 chunk 크기 C_1 , shard 길이 L_s (원문 S_L), 그리고 학습 가능한 초기 상태 W_{init} 을 가지고 다음과 같이 갱신된다:

$$W_t^1 = A_t - \sum_{\tau=\xi(t, C_1)+1}^t \eta_\tau \nabla_W \ell(A_t; k_\tau, v_\tau), \quad A_t = \begin{cases} W_{\text{init}} & \xi(t, C_1) \equiv 0 \pmod{L_s} \text{ (shard 첫 chunk)} \\ W_{\xi(t, C_1)}^1 & \text{그 외} \end{cases} \quad (15-4)$$

[TNT Eq. 6] 이다 ($C_1 \mid L_s$ 가정).³ 이것이 **periodic state reset** 이다: TNT 는 각 segment 의 시작에서 local 상태를 outer loop 가 학습한 W_{init} 으로 되돌린다 [TNT § 4.1.1 “reset ... at the beginning of each segment”]. 즉 shard 첫 chunk 의 anchor A_t 가 직전 shard 의 마지막 상태 대신 W_{init} 이 되어, shard m 의 계산은 그 이전의 무엇에도 의존하지 않는다. (원문 Eq. 6 을 1-based 에서 “ $t \equiv 0$ 일 때 $W_t = W_{\text{init}}$ ” 으로 옮기면 shard 의 마지막 token 이 그 shard 의 memory 대신 W_{init} 을 읽게 되므로 — reset 을 shard 시작의 anchor 로 둔 위 형태가 정합적 독해다.)

이 reset 이 왜 결정적인가. 비선형 recurrence 는 parallel scan 으로 병렬화할 수 없다 — scan 은 결합법칙을 요구하는데 MLP 를 통과하는 상태 전이에는 그것이 없다. reset 은 그 병렬화 불가능한 사슬을 **아예 끊어**, L/L_s 개의 shard 를 완전히 독립인 계산으로 만든다 — 장치에 분산 (**context parallelism**) 하거나 한 accelerator 의 batch 축에 쌓아 kernel 을 fatten 할 수 있다 [TNT § 4.1.1]. 비선형 deep-memory recurrence 를 sequence 방향으로 병렬화하는, TNT 가 제안하는 직접적 · 실용적 수단이다 (일반적 비선형 recurrence 의 exact 병렬화는 largely-unsolved 연구 문제로, 근사 · 반복 기반 시도가 별도로 있다). 대가는 명확하다: local memory 는 shard 경계에서 모든 것을 잊는다. 그 손실의 보전이 global memory 의 존재 이유다 — reset 없는 global 이 long range 를, reset 있는 local 이 병렬성을 든다. ablation 에서 global 을 제거하면 ppl 이 21.04 에서 25.60 으로 붕괴하는 것이 이 역할 분담의 실증이다 [TNT Table 3].

각 local memory 가 shard 경계에서 $t = 0$ 으로 되돌아가 이후 계산이 그 이전의 무엇에도 무관해지는 이 병렬화 구조를 시간축으로 펼친 것이 그림 15-3 이며, 위쪽 global 의 드문 순차 handoff 와 아래쪽 local 의 잦은 reset 이한 그림에서 대비된다.

W_{init} 이 학습된다는 점도 하중을 받는 설계다. 모든 shard 가 0 이 아니라 meta-learn 된 prior 에서 inner loop 를 시작한다. Titans 의 W_{init} (원문 M_0) 은 암묵적 존재였지만 ($\rightarrow$ 12 장), TNT 에서는 reset 을 생존 가능하게 만드는 load-bearing 부품으로 승격된다 — 개념 자체는 4 장의 MAML 류 meta-learned initialization 이다.

local 시스템은 그 자체로 다중 해상도일 수 있다. N 개의 local module $W^{l(i)}$ 가 각자의 chunk 크기 $C_1^{(i)}$, shard 길이 $L_s^{(i)}$, 초기 상태 $W_{\text{init}}^{(i)}$ 을 갖는 일반형은 [TNT App. E Eq. 14] 에 있고, 구성은 chunk 크기 집합으로 표기한다: $C_1 = \{4, 8, 16, 32\}$ 는 서로 다른 시간 스케일을 잡는 4 개의

²원문 수식 두 곳에 표기 슬립이 있다 (원문 자체의 문제로, 이 책의 통일 표기가 그 정정을 겸한다). (a) [TNT Eq. 5] 는 chunk 인덱스로 n 과 k 를 한 식 안에 혼용한다; 정돈된 형태는 [TNT App. E Eq. 13] 이다. (b) [TNT Eq. 6] 은 둘째 경우의 base 상태를 W_{t-1} 로 인쇄했지만 gradient anchor 는 $W_{\xi(t, C_L)}$ 이다 — 문자 그대로 매 t 에 적용하면 chunk 내부 gradient 가 이중 누적되어 [TNT Eq. 3] 과 모순된다. Eq. 3 과 동일한 chunkwise semantics(chunk 시작 상태를 base 로 한 누적 합)로 읽는 것이 유일하게 정합적인 독해이며, 식 (15-4) 는 그렇게 표기했다. (c) 추가로 [TNT Eq. 7] 의 projection 합 하한은 chunk 시작 $\xi(t, C_L)$ 로 인쇄되어 있으나, [TNT App. B] 는 “reset 상태부터의 합” 을, [TNT App. C] 는 shard 경계에서 reset 되는 누적 recurrence 를 명시한다. 이 책은 구현을 기술한 App. C 를 기준으로 삼는다.

³원문 수식 두 곳에 표기 슬립이 있다 (원문 자체의 문제로, 이 책의 통일 표기가 그 정정을 겸한다). (a) [TNT Eq. 5] 는 chunk 인덱스로 n 과 k 를 한 식 안에 혼용한다; 정돈된 형태는 [TNT App. E Eq. 13] 이다. (b) [TNT Eq. 6] 은 둘째 경우의 base 상태를 W_{t-1} 로 인쇄했지만 gradient anchor 는 $W_{\xi(t, C_L)}$ 이다 — 문자 그대로 매 t 에 적용하면 chunk 내부 gradient 가 이중 누적되어 [TNT Eq. 3] 과 모순된다. Eq. 3 과 동일한 chunkwise semantics(chunk 시작 상태를 base 로 한 누적 합)로 읽는 것이 유일하게 정합적인 독해이며, 식 (15-4) 는 그렇게 표기했다. (c) 추가로 [TNT Eq. 7] 의 projection 합 하한은 chunk 시작 $\xi(t, C_L)$ 로 인쇄되어 있으나, [TNT App. B] 는 “reset 상태부터의 합” 을, [TNT App. C] 는 shard 경계에서 reset 되는 누적 recurrence 를 명시한다. 이 책은 구현을 기술한 App. C 를 기준으로 삼는다.

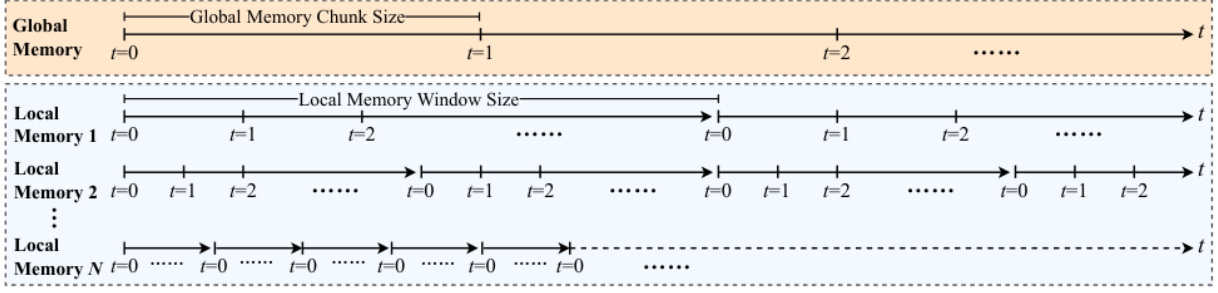


그림 15-3 – TNT memory 계층의 시간축 도해. 같은 행에서 같은 t 값의 갱신은 동시에 (병렬로) 실행되고 $t = 0$ 은 memory 초기화를 뜻한다. 맨 위 global memory 는 큰 chunk 하나가 sequence 전체를 순차적으로 관통하는 반면, 아래 N 개의 local memory 는 각자의 window(shard) 길이마다 $t = 0$ 으로 reset 되어 shard 들이 서로 독립 · 병렬이 된다 – index 가 커질수록 window 가 짧아 더 자주 reset 된다. 출처: [TNT] (arXiv:2511.07343), 원문 Fig.1 – 원저자의 그림 (third-party), 본서의 결과가 아님.

local module 을 뜻한다 [TNT § 5.1]. 실험에서 module 을 하나 더할 때마다 ppl 이 단조 개선된다 (§ 15.6).

15.3.4 Stage 1 – Q-K Projection: read 를 write 의 domain 으로 되돌리기

Challenge 2 의 처방이다. query 를 그대로 memory 에 넣는 대신, 지금까지 관측된 key 들이 스캔하는 부분공간으로 query 를 사영한 뒤 넣는다. 사영을 담당하는 것이 Q-K projection 행렬 $\Pi_t \in \mathbb{R}^{d \times d}$ (원문 기호 $\mathcal{M}_t$; memory 기호와의 충돌 때문에 개명) 이다. key 가 L2 정규화되어 있다는 가정 (이 계열모델의 관행 [TNT § 4.1.2]) 하에서 recurrence 와 chunkwise 분해는:

$$\Pi_t = \begin{cases} k_t k_t^\top & t \equiv 1 \pmod{L_s} \\ \Pi_{t-1} + k_t k_t^\top & \text{otherwise} \end{cases} \quad \Pi_t = \underbrace{\Pi_{\xi(t, C_1)}}_{\text{carry-over}} + \underbrace{\sum_{\tau=\xi(t, C_1)+1}^t k_\tau k_\tau^\top}_{\text{intra-chunk prefix sum}} \quad (15-5)$$

[TNT App. C] 다. 정규화 가정이 없으면 각 항이 $k_\tau k_\tau^\top / \|k_\tau\|^2$ 이 된다 [TNT Eq. 7]. 왼쪽 recurrence 는 local memory 와 같은 reset 규율을 따르고 (shard 시작 $t \equiv 1 \pmod{L_s}$ 에서 $k_t k_t^\top$ 로 재시작), 오른쪽분해는 chunk 내부의 outer product 들의 parallel prefix sum(scan) 으로, chunk 사이를 $d \times d$ 행렬 하나의 carry 로 처리한다. 단 carry $\Pi_{\xi(t, C_1)}$ 는 shard 첫 chunk ($\xi \equiv 0 \pmod{L_s}$) 에서 0 으로 재초기화되어야 직전 shard 의 Π 가 새 나가지 않는다 – TNT 는 이 carry-over state 를 shard 경계에서 re-initialize 한다고 명시한다 [TNT App. C]. 과거 key 를 저장할 필요가 전혀 없다 – 상태는 상수 크기다 [TNT § 4.1.2].

retrieval 은 global 과 local 의 출력 합이다:

$$y_t = \mathcal{M}(q_t; W_{\xi(t, C_g)}^g) + \mathcal{M}(\Pi_t q_t; W_t^l) \quad (15-6)$$

[TNT Eq. 7]. N -local 일반형은 둘째 항을 $\sum_{i=1}^N \mathcal{M}(\Pi_t^{(i)} q_t; W_t^{l(i)})$ 로 바꾼 것이다 [TNT App. E Eq. 15]. 두 가지 비대칭에 주목하라. 첫째, projection 은 **local 에만** 적용된다. fine-grained 한 local memory 가 domain mismatch 에 더 민감하다는 판단이고, global 은 raw query 를 받는다 [TNT § 4.1.2]. 둘째, global 읽기는 현재 상태가 아니라 **chunk 시작에 frozen 된 상태** $W_{\xi(t, C_g)}^g$ 를 읽는다. global chunk 하나 (최대 2048 token) 동안 읽기가 고정된 상태를 보므로 retrieval 도 chunk-병렬이 되지만, 그만큼 (최대 $C_g - 1$ token) 낡은 global 정보를 읽는다는 뜻이기도 하다 – § 15.8 에서 recall 관련 미검증 지점으로 되돌아온다.

[해설] $\Pi_t = \sum_{\tau} k_{\tau} k_{\tau}^{\top}$ 는 rank-1 projector 들의 합이므로 $\Pi_t q_t$ 는 항상 관측된 key 들의 span 안에떨어지고, 반복 관측된 방향일수록 증폭된다. 엄밀한 의미의 직교사영 (idempotent) 은 key 들이 정규직교일 때뿐이므로, “사영” 이라기보다 관측빈도로 가중된 soft projection 이다. inference 어휘로는, Π_t 의 갱신은 KV cache append 의 rank-1 GEMM 대응물이고 $\Pi_t q_t$ 는 mat-vec 한 번이다. attention 이 read 시점에 $q^{\top} K$ 로 하던 “query 를 key 통계와 대면시키는 일” 을, 상수 크기 running sum 으로 미리 압축해 두는셈이다.

[TNT App. B] 는 이 장치의 족보를 밝힌다. $\Pi'_t = \Pi'_{t-1} + k_t k_t^{\top}$ 는 그 자체로 보조 linear memory 이고, $\Pi_t q_t$ 는 그 memory 에 대한 forward pass 다. 따라서 Q-K projection 이붙은 retrieval 은 ABC(Peng, Kasai, et al. 2022)·Gated Slot Attention(Zhang, Yang, et al. 2024)·Trellis(Karami, Behrouz, et al. 2025) 같은 memory-bounded Transformer 의 two-pass read — $W_t = W_{t-1} + \varphi_t v_t^{\top}$, $y_t = W_t \text{softmax}(\sum_{\tau} \varphi_{\tau} \varphi_{\tau}^{\top} q_t)$ — 와 같은 형태다. 차이는 세 가지다: TNT 의 projection 은 linear 뿐 아니라 deep memory 에도 적용되고, 별도의 feature φ 를 학습 하는 대신 k_t 와 묶여 (tied) 있으며, 합산이 $\tau = 1$ 부터가 아니라 마지막 reset 부터다 [TNT App. B].

15.3.5 Stage 2 — 더 fine 한 해상도로의 fine-tuning

Stage 1 이 훈련 효율을 해결했으니 Challenge 3 이 남는다. 큰 chunk 로 pre-train 한 모델을 그냥 작은 chunk 로 평가하면 Fig. 2 의 절벽에서떨어진다. [TNT § 4.2] 의 관찰: **짧은 fine-tuning 으로 이 train-test 불일치가 교정되며, 원래 성능을 회복하는 정도가 아니라넘어선다**. Stage 2 는 효율적으로 pre-train 된 모델을 더 작은 local chunk 크기 $C'_1 < C_1$ 로 계속 훈련하는 것이다. global 의 C_g 는 그대로 둔다. 비용은 pre-training 의 약 5 – 8% — Table 4 기준 Stage 1 이 3.06 – 5.55 시간일때 Stage 2 는 0.15 – 0.46 시간이고, 최고 품질 4-local 구성은 $0.46/5.55 \approx 8.3\%$ 다 [TNT Table 4].

이상적 목표는 $C'_1 = 1$ 이다. 이 지점이 autoregressive serving 의 prefill-and-decode 패턴과 정확히 맞물린다: **global memory 가 큰 chunk 의 dense 연산으로 prompt 를 흡수하고 (prefill), Stage 2 로적응된 local memory 가 생성 중 token 단위로 갱신된다 (decode)** [TNT § 4.2]. 이로써 chunk 크기는 더 이상 하나의 타협값이 아니다 — Stage 1 에서는 훈련 throughput knob, Stage 2 에서는 inference 해상도 knob 이라는서로 독립인 두 개의 knob 이 된다. 이것이 이 책이 **two-stage training**(train-big / serve-small) 이라 부르는 레시피다.

한 가지 원문의 모호함을 기록한다. abstract 는 “only the local memory modules are adapted” 라 쓰고, [TNT § 4.2] 본문은 “continue training the efficiently pre-trained model with a smaller local chunk size” 라 쓴다. 확실한 것은 바뀌는 것이 local chunk-size hyperparameter 뿐이라는점이고, Stage 2 동안 slow weights 의 일부가 동결되는지는 명시되지 않았다. § 4.2 의 자연스러운 독해는 새 해상도에서 end-to-end 로 훈련을 계속한다는것이다.

15.3.6 각 부품이 존재하는 이유 (한 줄정리)

- 큰 C_g 의 global memory: hardware 포화 + long-range context 유지.
- reset-to- W_{init} local memory: 비선형 deep-memory recurrence 를 context-병렬화하는 TNT 의 직접적 수단 (exact 병렬화일반해는 미해결).
- 학습된 W_{init} : 모든 shard 에 주어지는 meta-learn 된 prior — reset 을 정보 전멸이 아니게 만드는 완충재.
- 다중 해상도 $\{C_1^{(i)}\}$: 서로 다른 시간 스케일의 feature 포착.
- Q-K projection: key-write/query-read domain 간극을 $O(d^2)$ 상태로 봉합.
- global 에는 raw query: 거친 granularity 에서는 mismatch 가 덜 아프고 compute 를 아낀다.
- Stage 2: chunk-size over-specialization 을 ~5% 비용으로치료하고 chunk-1 decode 를 해금.

15.3.7 표기 대응표

표 15-1 — [TNT] 원 표기와 이 책의 통일 표기 대응 (§ 1.7.4 기준; 하단 4 행은 장-국소 추가).

TNT 원 표기	통일 표기	의미 / 주의
W_t — fast weights	W_t	동일
$f(W, \cdot)$ — memory net	$\mathcal{M}(\cdot; W)$	
$\mathcal{L}(\cdot, \cdot)$ — inner loss	ℓ	△ 통일체계에서 $\mathcal{L}$ 은 outer 전용
η_t — learned inner lr	η_t	동일
$\xi(i, j)$ — chunk 시작 함수	$\xi(t, C)$	채택 (1-based 정의는 § 1.2)
C, C_G, C_{L_i}, C'_L	$C, C_g, C_1^{(i)}, C'_1$	
S_{L_i} — shard 길이 (reset 주기)	$L_s^{(i)}$	△ momentum S_t 와 충돌 방지
V — global memory 상태	W^g	△ value 행렬 V 와 충돌 방지
$W^{(i)}$ — local memories	$W^{l(i)}$	
W_{init} — 학습된 초기 상태	W_{init}	동일
$\mathcal{M}_t = \sum_{\tau} k_{\tau} k_{\tau}^{\top} / \ k_{\tau}\ ^2$ — Q-K projection	Π_t	△ memory 기호와 충돌 방지
o_t — 출력	y_t	
Eq. 5–6 의 인덱스 표기 슬립 (원문 자체)	(M4) 의 semantics 로 통일	각주로 명시 (이 장 ⁴)
S_L — $N=1$ 기본형의 shard 길이	L_s	장-국소추가
$\mathcal{M}'_t$ — App. B 의 보조 linear memory	Π'_t	장-국소추가
φ_t — App. B 의 ABC feature	φ_t 유지	장-국소 추가 (직접 인용 문맥)
$n = \{0, \dots, L/C_G\}$ — global chunk 범위	$n = 0, \dots, L/C_g - 1$	장-국소 추가; $C_g \mid L$ 가정

15.4 Outer-loop training vs inner-loop test-time learning

이 절이 이 장의 심장이다. TNT 는 “무엇이 학습되는가” 라는 질문에 대해라인에서 가장 계층이 많은 답을 갖고 있기 때문이다.

표 15-2 — TNT 의 구성 요소별 소속 loop 와 갱신 규칙.

구성 요소	loop	갱신 rule	갱신 주기	근거
projection W_K, W_V, W_Q (module 별)	outer (Θ)	AdamW (wd 0.1, cosine, peak lr 10^{-3})	훈련의 매 step; inference 에서 동결	[TNT § 5.1]

⁴원문 수식 두 곳에 표기 슬립이 있다 (원문자체의 문제로, 이 책의 통일 표기가 그 정정을 겸한다). (a) [TNT Eq. 5] 는 chunk 인덱스로 n 과 k 를 한 식 안에 혼용한다; 정돈된형태는 [TNT App. E Eq. 13] 이다. (b) [TNT Eq. 6] 은 둘째 경우의 base 상태를 W_{t-1} 로 인쇄했지만 gradient anchor 는 $W_{\xi(t, C_L)}$ 이다 — 문자 그대로 매 t 에 적용하면 chunk 내부 gradient 가 이중 누적되어 [TNT Eq. 3] 과 모순된다. Eq. 3 과 동일한 chunkwise semantics(chunk 시작 상태를 base 로 한 누적 합) 로 읽는 것이유일하게 정합적인 독해이며, 식 (15-4) 는 그렇게 표기했다. (c) 추가로 [TNT Eq. 7] 의 projection 합 하한은 chunk 시작 $\xi(t, C_L)$ 로 인쇄되어 있으나, [TNT App. B] 는 “reset 상태부터의 합” 을, [TNT App. C] 는 shard 경계에서 reset 되는 누적 recurrence 를 명시한다. 이책은 구현을 기술한 App. C 를 기준으로 삼는다.

구성 요소	loop	갱신 rule	갱신 주기	근거
η_t 산출 map	outer (Θ)	AdamW	훈련의 매 step (값 자체는 매 token 산출)	[TNT § 2.1]
$W_{\text{init}}^{(i)}$	outer (Θ)	AdamW; gradient 가 모든 shard로부터 유입	훈련의 매 step; inference 에서 동결	[TNT § 4.1.1]
backbone (FeedForward, embedding, head)	outer (Θ)	AdamW	훈련의 매 step	[TNT Fig. 3, § 5.1]
global fast weights W_t^g	inner	(M4) 형 chunkwise GD, anchor $W_{nC_g}^g$	$C_g = 2048$ token 마다 1 회	[TNT Eq. 5]
local fast weights $W_t^{l(i)}$	inner	(M4) 형 chunkwise GD + periodic reset	$C_1^{(i)}$ token 마다; $L_s^{(i)}$ 마다 reset	[TNT Eq. 6/14]
Q-K projection $\Pi_t^{(i)}$	inner	rank-1 누적 + $k_t k_t^\top$	매 token; $L_s^{(i)}$ 마다 reset	[TNT App. C]

inner 열에 없는 것도 정보다: momentum buffer S_t 도, retention gate α_t 도 없다. TNT의 inner optimizer 는 학습된 step size η_t 만 가진 plain GD 다 [TNT App. D].

Outer loop 에서 일어나는 일. slow weights Θ 는 next-token cross-entropy $\mathcal{L}$ 에 대해 AdamW 로 훈련된다. training 무경험 독자를 위한 결정적 사실: fast weights $W_t^g, W_t^{l(i)}$ 는 파라미터가 아니라 식 (15-3)·(15-4) 의 기호적 gradient-descent 가 만드는 outer 계산 그래프의 **activation** 이다. 따라서 backprop 은 그 inner GD step 들을 **관통해** 미분하고, $\mathcal{L}$ 의 Θ -gradient 에는 $\partial(\nabla_W \mathcal{L})/\partial\Theta$ 꼴의 2 차 항이 매 inner update 마다 실린다. “ $\eta_t \cdot W_{\text{init}}$ 이 학습된다” 의 정확한 의미가 이것이다: 그 값을 바꾸면 inner 궤적 전체가 바뀌고, 그 파급이 LM loss 에 미치는 효과를 backprop 이 계량해 AdamW 가 그 방향으로 움직인다 (형식화는 → 4 장).

이 그래프를 감당 가능하게 만드는 것이 chunkwise anchoring 이고, TNT 가 바꾸는 것은 그래프의 **모양**이다. (a) local 경로: reset 이 L_s token 마다 그래프를 절단한다. shard 경계를 local 상태를 통해 넘어가는 gradient 경로는 존재하지 않으므로, shard 당 직렬사슬은 L_s/C_1 번의 chunk handoff 뿐이고 L/L_s 개 shard 는 훈련 그래프에서도 완전히 독립이다 — forward 만이 아니라 backward 도 context-병렬이다. 한편 W_{init} 은 모든 shard 의 시작점이므로 모든 shard 로부터 gradient 를 받는다: 조밀하고 잘 평균된 학습 신호다. (b) global 경로: 직렬 사슬은 sequence 당 L/C_g handoff 다 (16K 에서 8 번). 비선형 recurrence 이지만 이 정도 깊이는 싸다. (c) Π_t : 식 (15-5) 의 chunk 내부 prefix scan + chunk 간 $d \times d$ carry + shard 경계 reset 이므로 직렬 병목을 추가하지 않는다.

[해설] 감각으로 옮기면: Titans 의 훈련 그래프가 “sequence 길이만큼 긴 단일 파이프라인” 이었다면, TNT 는 그것을 짧은 파이프하나 (global)·서로 독립인 짧은 파이프 묶음 (local, batch 축에 stack)·scan kernel 하나 (Π) 로 재배관했다. 훈련의 batch 축 역할을 sequence 축이 대신한다는 9 장의 경고를 상기하면, reset 은 sequence 축을 잘라 진짜 **batch** 축으로 되돌리는 연산이다.

Inner loop 에서 일어나는 일 (inference). 동결된 Θ 아래에서, memory layer 당 진화하는 상태는 W^g 하나, $W^{l(i)}$ N 개, $\Pi^{(i)}$ N 개다 — 전부 sequence 길이와 무관한 상수 크기다. update rule 은 훈련 때와 **형태가 동일하다** (그것이 test-time memorization 의 정의다, → 8·12 장). global

은 C_g token 마다 한 번, chunk 시작 상태에서 2048 개 token 의 gradient 를 한 번의 batched forward+backward 로 모아 적용하고, 그 사이 읽기는 frozen 상태를 본다. local 은 Stage 2 가 $C'_1 = 1$ 로 끝났다면 매 token 마다 정확한 online GD $W_t^1 = W_{t-1}^1 - \eta_t \nabla_W \ell(W_{t-1}^1; k_t, v_t)$ 를 수행 하고 — 이때 식 (15-4) 의 stale-snapshot 근사는 소멸한다, chunk 가 1 이므로 — L_s token 마다 W_{init} 으로 reset 된다 (Π 는 $k_t k_t^\top$ 로). decode 한 step 의 비용은, memory sub-network 하 나의 파라미터 수를 P_f 라 할 때 local 당 compression 의 forward+backward $\sim 6P_f$ FLOPs + retrieval forward $\sim 2P_f + \Pi$ 갱신과 적용 각각 $\sim 2d^2$ 이다.

[해설] 위 per-token 비용 분해는 [TNT] 가 보고한 수치가 아니라 이 책의 산정이다 (원 문에는 decode 비용 표가 없다). 요점은 두가지다. 첫째, **backward pass 가 decode 안에 들어온다** — weight update 없음이라는 추론 불변식은 이 라인에서 폐기된다 ($\rightarrow$ 1 장 Rosetta). 둘째, 그 모든 비용이 L 과 무관한 상수라는 것이 KV cache 대비 이계 열의 존재 이유다 (정량 비교는 § 15.7).

15.5 Concept ledger delta

표 15-3 — TNT 가 라인의 개념 원장에 가한 변경.

유형	개념	내용
추가	chunk-size mismatch (train/serve)	pre-training chunk 에 대한 inference 품질의 over-specialization [TNT Fig. 2]. 9 장의 “chunk 크기 = semantic hyperparameter” 명제의 실증적 뿌리
추가	hierarchical memory (global/local)	큰 C_g 의 순차 global + reset 되는 병렬 local 의 역할 분담 [TNT § 4.1.1]
추가	periodic state reset / context parallelism	reset-to- W_{init} 이 비선형 recurrence 의 sequence-방 향병렬화를 최초로 해금 [TNT § 4.1.1]
추가	Q-K projection	write(key) domain 과 read(query) domain 의 정렬; $O(d^2)$ running-sum 상태 [TNT § 4.1.2, App. B - C]
추가	two-stage training (train-big / serve-small)	throughput knob 과 해상도 knob 의 분리; $\sim 5\%$ fine-tune 으로 chunk-1 decode 를 품질 최적점화 [TNT § 4.2]
추가	Challenge 1/2/3 분류법; compression/retrieval 2-연산 정식화; 다중 해상도 local 집합 $C_1 = \{\dots\}$	논문의 어휘 정비 [TNT § 2 - 3]
확장	meta-learned W_{init}	Titans 의 암묵적 $M_0 (\rightarrow 12$ 장) 가 reset 의 생존 조건이라는 하중 부품으로 승격

유형	개념	내용
확장	chunkwise-parallel training	병렬화 트릭 (→ 9 장) 에서 그 자체가연구 대상인 훈련 경제학으로; staleness-throughput 트레이드오프 최소화
확장	multi-timescale memory	Titans 의 persistent/long/short 삼분 (→ 12 장) 이 global/local chunk 계층으로 공학화 — 16 장 CMS 의 전조
확장	retention	reset 은 스케줄된 hard retention 이되 표준 gate 로는정확히 안 담긴다: $\alpha_t = 0$ 은 state 를 0 으로 소거할 뿐 학습된 W_{init} 으로 되돌리지 않으므로, affine 형태 $W_t = \alpha_t W_{t-1} + (1 - \alpha_t) W_{\text{init}} + S_t$ (또는 중심화상태 $W_t - W_{\text{init}}$ 에 대한 $\alpha_t \in \{0, 1\}$) 로 써야한다; 값이 데이터가 아니라 시계로 결정되는 극단형 ([해설] 적재 서술)
폐기 (일시)	momentum β_t , retention gate α_t , Muon/Omega rule	“명료성을 위해” 제거된 단순화 [TNT App. D]; 라인 분류와의 합성은 미검증 채로 유보

15.6 실험과스케일

설정. 품질 실험은 전부 150M 파라미터, 10B tokens, T5 tokenizer(32k vocab), AdamW(weight decay 0.1, cosine schedule, peak lr 10^{-3}), TPUv5 pod(2x2x2 topology, model parallelism 2), custom kernel 없는 순수 JAX 구현이다 [TNT § 5.1]. 효율 벤치마크는 context 2K-32K, batch 0.5M tokens, $L_s = 2048$; 품질 평가는 context 16K, batch 1M tokens, $L_s = 4096$, $C_g = 2048$ 이다 [TNT § 5.1]. TTT와 Titans baseline 은저자들의 재구현이다 [TNT § 5 각주 2]. chunk 민감도 실험 (Fig. 2) 만 550M 모델을 쓴다.

속도 — step 당. token/batch 를 0.5M 으로 고정하고 sequence 길이를 늘리면, TNT 의 step 시간은 선형으로 증가하는 반면 Titans와 JAX attention 은 wall-clock 에서 2 차적으로 증가한다고 논문은보고한다 [TNT § 5.2, Fig. 4]. 32K 에서 같은 memory chunk($C_1 = C = 16$) 의 Titans 보다 $5.1 \times$ 빠르고, $C_1 = \{128\}$ 의 순수-JAX TNT 는 Pallas FlashAttention kernel 보다도 $1.3 \times$ 빠르다 [TNT § 5.2].

속도 — 목표 품질까지. 실전 지표는 같은 training loss(3.20) 에 도달하는 시간이다 [TNT Table 1].

표 15-4 — 목표 loss 3.20 도달 시간 (150M, [TNT Table 1] 발췌).

모델 (구현)	C 또는 C_1	시간 (hrs)	배속
Titans (JAX)	8	19.48	$1.00 \times$
Titans (JAX)	64	4.18	$4.67 \times$
Titans (JAX)	128	3.71	$5.25 \times$
Transformer w/ gating (JAX)	-	1.38	$14.10 \times$
Transformer w/ gating (FlashAttention)	-	0.96	$20.22 \times$
TNT (JAX)	{8}	2.54	$7.68 \times$

모델 (구현)	C 또는 C_1	시간 (hrs)	배속
TNT (JAX)	{64}	1.12	17.37×
TNT (JAX)	{128}	1.16	16.75×

헤드라인 17.37×는 “가장 정확한 Titans” ($C = 8$, ppl 25.07) 대비다. 같은 chunk 크기 8끼리 비교해도 TNT가 7.7× 빠르므로 이득의 원천이 chunk 확대만이 아니라 구조임을 알 수 있다 [TNT § 5.2]. C_1 을 키우면 {64}까지는 단조로 빨라지다가 {128}에서 미세하게 되돌아간다 — step은 더 빠르지만 step 당 품질 이득이 깎이는 지점이다. 그리고 정직하게: kernel 최적화된 Gated Transformer(0.96h)는 아직 이기지 못하며, 논문도 custom kernel 부재를 이유로 들며 future work로 미룬다 [TNT § 5.2].

품질. C4/FineWeb/PG19 perplexity와 4개 common-sense reasoning 과제 (PIQA, HellaSwag, ARC-e, CSQA) 평균 [TNT Table 2]:

표 15-5 — 품질 결과 (150M / 10B tokens, [TNT Table 2] 발췌; ppl은 3개 corpus 평균, acc는 4개 과제 평균).

모델	구성	평균 ppl ↓	평균 acc ↑
Transformer (w/o gating)	-	23.58	38.3
Gated Transformer	-	22.39	39.7
TTT	$C = 256$	27.62	38.1
Titans	$C = 256$	27.13	38.8
Titans	$C = 8$	25.07	39.0
TNT Stage 1	{8}	24.10	40.6
TNT Stage 1	{4,8,16,32}	23.13	40.6
TNT Stage 2	{1}	23.99	40.9
TNT Stage 2	{2,4,8,16}	23.09	40.9

Stage 1만으로 모든 RNN baseline과 vanilla Transformer의 ppl을 이기고, Stage 2가 각 구성에서 ppl을 추가로 내린다 (23.13 → 23.09 등). reasoning acc에서는 Gated Transformer까지 이긴다 (41.0 vs 39.7 [TNT § 5.3]; 표의 40.9는 {2,4,8,16} 행, 41.0은 Stage 1 {8,16} 행이다). 단 ppl에서는 Gated Transformer(22.39)에 진다는 것을 논문 스스로 명시한다 [TNT § 5.3]. 훈련 비용 전액은 Table 4가 준다: Titans $C = 8$ 은 8.44h, TNT Stage 1은 {8} 3.06h에서 {4,8,16,32} 5.55h, Stage 2는 0.15-0.46h — 논문이 “약 5%”라 부르는 근거다 (다만 최고 품질 4-local 구성은 0.46/5.55 ≈ 8.3%이고, 네 구성의 비율은 약 4.9/5.4/5.2/8.3%다) [TNT Table 4, § 5.3].

Ablation [TNT Table 3]: base Titans ppl 23.53/acc 38.8에서 local memory를 1 → 4개 추가하면 ppl 21.04 → 20.74 → 20.47 → 20.15로 단조 개선. global memory 제거는 25.60으로 붕괴 (base보다도 나쁨 — reset만 있고 global 맥락이 없으면 치명적). Q-K projection 제거는 21.04 → 22.01 (projection의 가치 ≈ 1 ppl, acc는 40.6 → 36.4). $N = 1$ 에 Stage 2를 얹으면 20.86/40.9. Table 3의 ppl 열은 열 머리에 corpus 표기가 없으나, 여섯 값 전부가 Table 2의 C4 열과 일치한다 (23.53/21.04/20.74/20.47/20.15/20.86; 다른 corpus 결과는 불일치) — C4 기준으로 읽는다 [TNT Table 2-3].

스케일 정직성. 이 논문의 실증은 150M/10B tokens가 전부이고, chunk-size mismatch 현상 자체도 550M 한 설정의 그림 하나가 근거다. 라인 전체의 실증 상한이 1.3B params / 100B tokens([NL] (Nested Learning, arXiv:2512.24695), → 16장)임을 감안해도 TNT는 그 안에서 가장 작은 축이다. “17×가속이 1B+/100B+에서도 성립하는가”는 열린 질문이며 (§ 15.8), 무엇보다 이 모든 검증이 **momentum·gating·Muon**을 ‘명료성을 위해’ 제거한 단순화 Titans 위에서 이뤄졌다는 것 [TNT App. D] — 즉 라인의 분류 모델과의 합성은 측정된 적이 없다는 것 — 이 이장의 의무 caveat다.

15.7 Systems/serving 합의

병렬화 구조. § 15.4의 그래프 분해 — 직렬 깊이 L/C_g 의 global($16K/C_g=2048$ 이면 8번, 각 handoff가 2048-token batched matmul이라 compute-bound), 완전히 독립인 L/L_s 개의 local shard(장치 간 상태 교환 0의 진짜 context parallelism 이자 단일 accelerator에서는 batch 속에 쌓아 kernel launch를 fatten), $d \times d$ carry 하나의 Π prefix scan — 를 systems 언어로 다시 읽으면 arithmetic intensity의 제조다. Challenge 1의 <5-10% FLOPs utilization의 뿌리는 FLOPs 부족이 아니라 intensity 부족인데, TNT는 병렬 작업 단위를 크게 (global) 만들거나 많고-독립적으로 (local) 만들어 그 intensity를 만들어낸다.

[해설] tiling(bit-exact 재배열)과 달리 TNT의 chunk·reset은 **계산되는 함수 자체를 바꾼다**(chunk는 stale-snapshot 근사, reset은 정보를 실제로 버림 → 1장 Rosetta·9장). 통찰은 그 의미론적 변경을 없애려 하지 않고, 변경분(잃어버린 long range)을 global memory라는 별도 부품으로 회수한 뒤 두 부품에 서로 다른 하드웨어 체질을 부여한 데 있다.

Kernel 관점. 전부 순수 JAX이고 fused kernel은 없으며 저자들이 명시적으로 future work로 남겼다 [TNT § 5.2]. 그런데도 32K에서 TNT($C_l = 128$)가 FlashAttention을 step당 이긴다는 것은, kernel 이전에 **병렬 구조 자체**가 승부를 갈랐다는 뜻이다. GLA/DeltaNet 계열의 SRAM chunk kernel이 deep memory에 이식되지 않는 이유(chunk 간 상태 전파가 비선형을 통과)를 상기하면, TNT의 reset은 현재로서 가장 직접적인 지렛대다. 미래의 fused TNT kernel은 chunk당 batched forward+backward와 누적 합 상태 갱신의 융합이 될 것이다.

State 크기 vs KV cache — 정량 비교. decode 시점에 memory layer당 상태는 $(1+N)P_f + Nd^2$ 개의 값이다(global + N 개 local의 fast weights + N 개 projection 행렬). Transformer는 layer당 $2Ld$ 를 들고 매 step 전량 재독한다.

[해설] 수치를 넣어 보자(이 책의 산정; 원문에 decode 메모리표는 없다). 이 라인의 표준 deep memory인 expansion 4의 2-layer MLP를 가정하면 $P_f \approx 8d^2$. $N = 4$ local 구성이면 상태 $\approx 5 \cdot 8d^2 + 4d^2 = 44d^2$. $d = 1024$ 일 때 약 46M 값/layer로, bf16 기준 ~92MB다 — 결코 가볍지 않다. 그러나 KV cache $2Ld$ 가 이를 넘는 지점은 $L \gtrsim 22d$, 즉 $d = 1024$ 면 **약 22K tokens**이고, 그 뒤로 KV cache는 무한히 자라는 반면 TNT 상태는 그대로다. 32K+ 문맥의 통상적 RNN 논증이지만, TNT는 serving 특화 이득 두 개를 얻는다. (1) Stage 2가 $\{8\} \rightarrow \{1\}$ 같은 작은-chunk 전환을 성공적으로 적응시켜 chunk-1 decode를 품질과 **정렬**한다 (§ 15.2의 절벽과 정반대) — baseline deep memory가 $C = 64$ 훈련 후 $C = 8$ decode 만으로도 ppl이 폭발한 것과 달리 [TNT Fig. 2], TNT는 자연스러운 autoregressive loop과 품질이 어긋나지 않는다. 단 Stage 2 $\{1\}$ (평균 ppl 23.99)이 전역 품질최적은 아니다 — 다중-local $\{2, 4, 8, 16\}$ (23.09)이 더 낫다 [TNT Table 2]. (2) prefill이 하드웨어에 깨끗하게 맵핑된다 — global은 prompt를 C_g 단위 dense batch로 삼키고 local shard들은 서로 병렬로 prefill 되므로, time-to-first-token이 직렬 recurrence가 아니라 chunked linear pass처럼 스케일한다.

Decode의 memory traffic. $C_l' = 1$ decode는 token마다 local fast weights P_f 개 값의 read-modify-write다: 상태를 읽고, forward+backward($\sim 6P_f$ FLOPs)를 돌리고, 다시 쓴다. 값당 FLOP이 한 자릿수이므로 이 갱신 자체는 memory-bound다 — KV cache 재독과 같은 체질이 되, 트래픽이 L 에 비례하지 않고 상수라는 점이다. 한편 갱신 주기의 계층성은 상태의 저장 위치 계층과 자연스럽게 대응한다: 매 token 갱신되는 $W^1 \cdot \Pi$ 는 가능한 한 on-chip에 상주시킬 대상이고, 2048 token에 한 번 큰 batch로 갱신되는 W^8 는 HBM에 두고 드물게 대량으로 만지는 대상이다. update frequency가 배치 (placement)를 결정한다는 이 관찰은 16장의 CMS에서 아키텍처 원리로 승격된다.

Batching과 운영. per-request fast-weight 상태는 shared-weight batching을 깨뜨리므로 (→ 1장 Rosetta, 10장), TNT decode의 local 경로 batching은 request 별 weight를 갖는

grouped-GEMM 형태가 된다. 운영측면에서 reset 은 뜻밖의 선물을 준다: preemption/restore 때재구성해야 할 local 상태 이력이 최대 L_s token 으로 유계다 — shard 시작 상태는 언제나 상수 W_{init} 이기 때문이다. 다중해상도 구성의 비용도 계산에 넣어야 한다: Stage 1 훈련 시간이 {8}의 3.06h 에서 {4,8,16,32}의 5.55h 로 늘고 [TNT Table 4], decode 비용도 N 에 비례하며, 이 스케일에서 품질 대가는 module 당 대략 -0.3 ppl 이다 [TNT Table 3].

마지막으로 의무 caveat: 이 절의 prefill/decode 서사는 **아키텍처 논증이지 벤치마크가 아니다**. chunk-1 모드의 decode throughput/latency/메모리를 KV-cache Transformer 와 맞대 켜 수치는 [TNT] 에 없고, decode wall-clock 수치는 6 편 전체에 부재하다.

15.8 한계와 bridge-out

논문이 스스로 여는 문제와 이 책의 평가.

첫째, **단순화 위의 검증**. TNT 는 Titans 의 momentum 과 forget gating, Comba 류 closed-loop objective, Adam/Muon inner optimizer 를 전부 “명료성을 위해” 뺀 plain-GD memory 로 검증되었고, 이들과의 결합은 명시적으로 future work 다 [TNT App. D]. $17\times$ 가속과 품질 이득이 완전한 Titans 나 Atlas 에 이식되는지는 측정된 바 없다. reset·Q-K projection 이 gated/momentum update 와 합성 가능한지, Stage 2 가 그때도 전이되는지가 라인의 다음 실무 질문이다.

둘째, **Challenge 3 의 증거 폭**. chunk-size mismatch 의 증거는 단일설정 — 550M, gating/momentum 없는 단순화 모델, $C = 64$ 훈련 — 의 그림 하나다 [TNT Fig. 2]. 현상 스케일·아키텍처 변형에 걸쳐 얼마나 보편적인지, 그리고 왜 생기는지에 대한 이론은 없다. chunkwise staleness 의 오차한계가 6 편 어디에도 없다는 9 장의 지적이 여기서도 유효하다: Stage 2 가 mismatch 를 고친다는 것은 알지만, 무엇이 고쳐졌는지는 모른다.

셋째, **reset 아래의 recall**. local memory 는 설계상 L_s token 마다 전부 잊고, global retrieval 은 최대 $C_g - 1$ token 남은 frozen 상태를 읽는다 (식 (15-6)). shard 경계를 넘는 정밀 recall 이 global 경로로 얼마나 생존하는지 — needle-in-a-haystack 류 벤치마크 — 는 평가되지 않았다. Atlas 와 [NL] 이 측정된 in-context retrieval gap ($\rightarrow 14\cdot 16$ 장) 을 생각하면 이공백은 사소하지 않다.

[평가] TNT 의 교환은 명료하다: 병렬성을 사기 위해 local 의 기억을 주기적으로 태우고, 그 보험을 저해상도 global 에 든다. 이 보험의 실효성이 미측정이라는 것이 이 논문의 가장 큰 빈칸이다. 아울러 Stage 2 의 기제 — 무엇이 몇 step 에 걸쳐 재조정되는가, {8} $\rightarrow$ {1}은 23.99 인데 {4,8,16,32} $\rightarrow$ {2,4,8,16}은 23.09 인 이유는 무엇인가 [TNT Table 2] — 도 열려 있다. 하이퍼파라미터기하 (L_s vs C_g vs $\{C_1^{(i)}\}$ 의 동시 선택) 는 실험이 $L_s \in \{2048, 4096\}$, $C_g = 2048$ 만 훑었으므로 사실상 미탐이다.

넷째, **스케일과 kernel**. 150M/10B tokens 라는 실증범위, Gated Transformer 대비 남은 ppl 격차 (22.39 vs 23.09) 와 time-to-loss 격차 (0.96h vs 1.12h), 그리고 custom kernel 의 부재. 논문은 이 셋 모두를 감추지 않고 명시한다 [TNT § 5.2 – 5.3] — 이 라인에서 보기 드물게 systems 논문다운 정직성이다.

Bridge-out: TNT $\rightarrow$ [NL]. TNT 의 global/local 계층은 throughput 으로 정당화된 트릭이다. 그러나 이 트릭이 심어 놓은 관념을 다시 보라: **서로 다른 주기로 갱신되는 부품들이 있고, 느린 시간 스케일에 주차된 지식은 빠른 스케일의 reset 에서 살아남는다**. global 은 2048 token 마다, local 은 매 token 마다, W_{init} 과 slow weights 는 훈련에서만 — TNT 는 이미 3 개 층위의 update frequency 로 돌아가는 시스템이다. 다만 TNT 에게 그것은 공학적 방편이었지, 왜 그것이 모델 전체의 조직 원리여야 하는지는 말하지 않는다.

NL 이 바로 그 선언을 한다: 모델과 훈련 절차 전체가 각자의 update frequency 로 자기 context flow 를 압축하는 중첩된 optimization 문제들의 시스템이라는 존재론이다. TNT 의 global/local 이분은 geometrically spaced 주파수의 연속체 (Continuum Memory System) 로 일반화되고, TNT 의 reset 은 Nested-CMS 의 re-initialization 으로, load-bearing 해진 W_{init} 은 다섯가지 knowledge-transfer 기제 중 하나 (MAML 류 initialization) 로 분류된다. TNT 가 유보했던 것

들 — momentum, gating, 더 강한 inner optimizer — 도 NL 에서 “optimizer 역시 memory 다” 라는 프레임 아래 재입장한다. 한편 Q-K projection 은 후속 논문들이 재채택하지 않은, 이 논문 고유의 열린 실마리로 남는다. 훈련 경제학의 바닥을 깬 것이 TNT 라면, 그 위에 세워질 층위의 존재론이 다음 장이다.

ch16. Nested Learning: deep learning 아키텍처라는 착시, 그리고 Hope

16.1 Bridge-in: TNT가 남긴문제

[TNT] (TNT: Improving Chunkwise Training for Test-Time Memorization, arXiv:2511.07343; → 15 장) 는 이라인의 훈련 경제학 회차로, deep memory 의 chunkwise training(→ 9 장) 이안고 있던 세 문제 (작은 chunk 훈련의 비효율, write/read 도메인 불일치, train/serve chunk-size mismatch) 를 hierarchical memory 와 two-stage training 으로 풀며 하나의 구조를 남겼다: 큰 chunk 로 천천히 갱신되는 global memory W^g 와, meta-learn 된 초기 상태 W_{init} 로 주기적으로 reset 되는 local memory 들의 공존이다. 서로 다른 주기로 갱신되는 구성요소들, 그리고느린 timescale 의 지식이 빠른 timescale 의 reset 을 살아남는 구조 — 이것이 TNT 가 심어 놓고도 정당화하지 않은 아이디어다. TNT 에서 이 계층은 throughput 트릭이었을 뿐, 왜 그것이 모델 전체의 조직 원리여야 하는지, 그위에서 아키텍처와 optimizer 가 어떤 관계인지엔 대답이 없었다.

Part II 의 사슬을 이 책의 기준 수식 (M2) 로 요약하면 공백이 더 선명해진다. [Titans] (Titans: Learning to Memorize at Test Time, arXiv:2501.00663; → 12 장) 는 (M2) 자체 — GD + momentum + weight decay 가 sequence layer 라는 등식 — 를 세웠고, [Miras] (It' s All Connected: A Journey Through Test-Time Memorization, Attentional Bias, Retention, and Online Optimization, arXiv:2504.13173; → 13 장) 는 inner objective ℓ 과 retention 을 설계 축으로 만들었으며, [Atlas] (Atlas: Learning to Optimally Memorize the Context at Test Time, arXiv:2505.23735; → 14 장) 는 ℓ 의범위 (window) 와 inner optimizer(Muon) 를 바꿨고, TNT 는훈련 경제학을 바꿨다. 다섯 논문 모두 “sequence layer 하나” 의 성분을바꿨을 뿐, 그 layer 를 훈련시키는 바깥의 AdamW·momentum·backpropagation 은 여전히 설계공간 바깥의 고정된 배경이었다. 이 장의 논문은 질문의 단위를 바꾼다: layer 가 아니라 **모델과 훈련 절차 전체**가 하나의 설계 대상이고, 그전체는 서로 다른 주기로 자기 문맥을 압축하는 최적화 문제들의 중첩시스템이라는 것이다.

이 장의 논문은 [NL] (Nested Learning: The Illusion of Deep Learning Architecture, arXiv:2512.24695; Ali Behrouz, Meisam Razaviyayn, Peilin Zhong, Vahab Mirrokni, Google Research, v1 2025-12-31; NeurIPS 2025 발표본 존재) 이다. [NL] 은 이라인의 이론 종합 회차다. TNT 가 실용으로 발견한 “다중 timescale + reset 생존” 을 존재론으로 선언하고, 그 위에서 세 가지를 생성한다: 더 표현력있는 optimizer 들 (Delta Momentum, DMGD, DGD, M3), 주파수 스펙트럼 memory(**Continuum Memory System**), 그리고 자기 자신의 update 알고리즘을 학습하는 sequence block(**self-modifying Titans**). 세산물의 결합이 **Hope** 다.

16.2 문제의식과 논문의 핵심주장

논문이 정의한 문제는 LLM 의 정적임 (static) 이다 [NL § 1]. 배포 후 적응가능한 부분은 in-context learning 뿐이라 지식은 (i) 지금 context window(attention 의 KV cache), (ii) “end of pre-

training”에 동결된 MLP weights 두 곳에만 존재한다. 논문은 이를 anterograde amnesia에 비유한다 [NL § 1]: context 정보는 장기 저장소 (feedforward layer)에 도달하지 못하고 window가 밀려나면 사라진다.

신경생리학이 두 처방을 시사한다 [NL § 1.1]. 첫째, 장기 기억 형성은 최소 두 단계의 consolidation을 거친다: 각성 중 online (synaptic) consolidation과 수면 replay의 offline (systems) consolidation(→ 11 장). [NL]은 명시적으로 전자만 다루고 후자를 범위 밖으로 선언하며 [NL § 1.1] — 이 선언이 17 장 [Sleep] (Language Models Need Sleep: Learning to Self-Modify and Consolidate Memories, arXiv:2606.03979)으로 가는 인계선이다. 둘째, 뇌는 다중 timescale로 계산을 조직한다. 반면 현대 모델의 update frequency는 두 극단뿐이다: attention은 매 token 재계산이라 ∞ , MLP는 test time 동결이라 0 [NL § 1.1, § 6].

세 번째 관찰은 구조의 균일성이다 [NL § 1.1]. 뇌 memory는 분산·재사용 요소인 반면 현대 스택은 attention+SSM+convolution+MLP의 이질적 조합으로 보인다. 진단: 이 이질성은 착시다 — 최적화 문제의 **해** (예: attention 닫힌형)만 보고 **문제** (objective·문맥)를 안 봐서 달라 보일 뿐, 모든 구성요소는 GD로 최적화되는 linear 또는 MLP associative memory이고 차이는 level·objective·update frequency 뿐이다 [NL § 5.1]. 제목의 “illusion”이 이것이다.

핵심 주장은 넷이다. (1) 어떤 ML 모델이든 훈련 절차와 함께 하나의 **nested, multi-level, 병렬 최적화 문제 시스템**으로 표현되며, 각 문제는 자기 **context flow**(token, gradient, 상위 신호)를 자기주기로 압축하는 associative memory다 [NL § 3]. (2) 이 렌즈에서 backprop·momentum·Adam·AdaGrad·Muon은 전부 gradient를 압축하는 associative memory이고 특히 Adam은 특정 element-wise ℓ_2 objective의 **최적** memory다 [NL § 4, App B]. (3) 이 프레임은 기술용이 아니라 생성용이다: 새 optimizer·CMS·self-modifying Titans가 도출된다 [NL § 4.4 – § 4.5, § 7, § 8]. (4) 더 많은 layer가 아니라 더 많은 **level**이 computational depth와 continual learning(→ 11 장)을 사준다 — stacking의 새 차원 [NL § 1.2, § 3.2].

부산물 하나가 특히 중요하다: in-context learning은 emergent가 아니라 level이 2개 이상이면 반드시 생기는 구조적 성질이다 — Transformer는 non-parametric 해, recurrent 모델은 하위 level의 parametric 학습, pre-training 조차 “corpus 전체를 context로 하는 ICL”로 재분류된다 [NL § 6].

[해설] 논문 제목은 Merrill et al. The Illusion of State in State-Space Models를 겨냥한다: 그쪽이 “SSM의 state는 얕다”면 [NL]은 “아키텍처 다양성이야말로 착시, 진짜 변수는 level”이라 되받는다 (computational depth 한계는 [NL] § 1의 layer-stacking-불가목록 첫 항목).

16.3 Core mechanism (통일표기)

이 절은 [NL]의 재해석 사슬을 원문 순서로 따라간다: 대수적 엔진 → 훈련 자체의 memory화 → level/nested system 형식화 → level 간 knowledge transfer → optimizer 재해석·새 optimizer → 아키텍처 재해석 → 생성물셋 (CMS·M3·self-modifying Titans) → Hope. 모든 수식은 통일 표기, 원 표기 대응은 § 16.3.12 표 16-2에 있다.

16.3.1 대수적 엔진: associative memory 정의와 proximal/FTRL 등가

출발점은 [Miras]의 associative memory 정의를 그대로 수입한 [NL Def. 1]이다 (정의 소유는 → 13 장): key 집합 $\mathcal{K} \subseteq \mathbb{R}^{d_k}$ 와 value 집합 $\mathcal{V} \subseteq \mathbb{R}^{d_v}$ 가 주어질 때, associative memory는 사상 품질을 재는 objective $\tilde{\ell}$ 에 대해

$$\mathcal{M}^* = \arg \min_{\mathcal{M}} \tilde{\ell}(\mathcal{M}(\mathcal{K}); \mathcal{V})$$

로 얻어지는 연산자다. [NL]이 더하는 것은 둘이다. 첫째, 용어의 신경심리학적 고정: **memory**는 **입력이 일으킨 신경 갱신**, **learning**은 **유효한 memory를 획득하는 과정**이라 [NL § 3.1] 어떤 level의 어떤 GD 갱신도 memory다. 둘째, key/value가 token일 필요가 없다 — gradient,

sub-sequence, 상위 신호 모두 가능하다 [NL § 3.1]. 한 level 이 압축하는 데이터 스트림을 그 문제의 **context flow** 라부른다 (sequence block=token, optimizer=gradient).

재해석의 대수적 엔진은 GD 의 두 등가 형식이다. 한 스텝의 GD 는 linearized objective + proximal 항의 argmin 과 같다 [NL Eq. 2]:

$$\theta_{t+1} = \arg \min_{\Phi} \left\{ \langle \nabla_{\theta} \tilde{\ell}(\theta_t; x_t), \Phi \rangle + \frac{1}{2\eta_t} \|\Phi - \theta_t\|_2^2 \right\},$$

즉 GD 한 스텝은 “1 차 Taylor 근사의 ℓ_2 -proximal 최소화” 다. 누적하면 (상수 η) FTRL 형식이 되고 [NL Eq. 3], 해는 $\theta_{t+1} = \theta_1 - \eta \sum_{s \leq t} \nabla \tilde{\ell}(\theta_s; x_s) - 3$ 장의 FTRL($\rightarrow 3$ 장) 그대로다. 이후 [NL] 의 모든 재해석은 update 식을 “어떤 objective 의 proximal argmin” 으로 되읽고 그 objective 와 context flow 를 묻는 이 치환을 반복한다.

16.3.2 훈련은 surprise 에 대한 memory 다: LSS 와 backprop 의자기참조성

먼저 부호 관행: 전역 기호 ($\rightarrow$ § 1.2) 를 따라 **Local Surprise Signal (LSS)** 을 $u_t = \nabla_{y_t} \mathcal{L}$ 로 고 정한다. [NL] 은 § 3.1 에서 같은 부호로 정의하되 GGD 문맥 ([NL Eq. 58 – 60]) 에서는 $-\nabla_{y_t} \mathcal{L}$ 를 u_t 로도 쓰므로, 이 장은 self-generated value 자리에서 $-u_t$ 를 명시한다.

$y = \theta x$ 인 1-layer linear layer 를 task loss $\mathcal{L}(\text{NTP})$ 로 훈련하는 SGD 한 스텝은 다음처럼 인 수분해된다 [NL Eq. 8]:

$$\theta_{t+1} = \theta_t - \eta_{t+1} \nabla_{\theta} \mathcal{L}(\theta_t; x_{t+1}) = \theta_t - \eta_{t+1} u_{t+1} x_{t+1}^{\top}, \quad u_{t+1} = \nabla_{y_{t+1}} \mathcal{L}(\theta_t; x_{t+1}).$$

여기서 $\nabla_{\theta} \mathcal{L}$ 은 12 장의 surprise 그대로이고, u_{t+1} 은 그 출력 공간 버전 (국소 예측 오차) 이다 [NL § 3.1]. gradient 가 rank-1 outer product $u x^{\top}$ 로 쪼개진다는 사실이재해석의 문을 연다: 이 update 는 proximal 형식으로

$$\theta_{t+1} = \arg \min_{\Phi} \langle \Phi x_{t+1}, u_{t+1} \rangle + \frac{1}{2\eta_{t+1}} \|\Phi - \theta_t\|_2^2 \quad (16-1)$$

과 동치이며 [NL Eq. 9], Def. 1 의 associative memory $-x_{t+1}$ 을 key, 그 LSS u_{t+1} 을 value 로 하는 dot-product objective 의 memory 다. 요약: **backprop 으로 훈련되는 layer 는 “각 입력 $\rightarrow$ 그 예측오차” 의 사상을 weights 에 압축하는 memory 다** [NL § 3.1, § 4.1].

깊은 모델로 가도 형태는 같다. L_{layer} 층 MLP $\{W_{\ell} \cdot + b_{\ell}\}_{\ell=1}^{L_{\text{layer}}}$ 에서 backprop 은 layer 별 gradient 를

$$\frac{\partial \mathcal{L}}{\partial W_{\ell}} = \delta_{\ell} \hat{x}_{\ell-1}^{\top}, \quad \delta_{\ell} = J_{\phi_{\ell}}(z_{\ell})^{\top} W_{\ell+1}^{\top} \delta_{\ell+1}$$

로 계산한다 NL Eq. 29. 따라서 layer ℓ 의 GD update 도 식 (16-1) 꼴의 proximal argmin — “입력 $\hat{x}_{\ell-1}$ 을 국소 오차 δ_{ℓ} 에사상하는 memory” — 이다 [NL Eq. 31].

가장 미묘한 지점: 식 (16-1) 을 보고 “backprop 은 gradient 에 대한 linear attention” 이라 결론지으면 틀린다 [NL § 4.1] — linear attention($\rightarrow 6$ 장) 은 key/value 가 memory 상태와 무 관해 병렬화되지만, backprop 의 value δ_{ℓ} 은 memory 자신의 현재 상태가 만든다:

$$v_t = f_{W_t}(x_t) = -u_t = -\nabla_{y_t} \mathcal{L}(W_t; x_t) \quad [\text{NL Eq. 58}]$$

— 즉 backprop 은 자기 훈련 target 을 스스로 생성하는 **self-referential** memory (Schmidhuber 1993) 이며, 그 성질 때문에 scan 으로 병렬화되지 않는다 [NL § 4.5] — Hope 의 chunk-wise 훈련 (§ 16.4) 은 정확히 이 자기참조성을 chunk 경계 snapshot 으로 절단하는타협이다. inference 어휘로 각 layer 는 KV cache 없이 (입력 $\rightarrow$ 오차) 를 weights 에 압축하는 memory 이고 ($\rightarrow 1$ 장 Rosetta), [NL] 은 2 장이 만든 optimizer-as-object 관점의 나머지 절반 — 그 state 가 곧 한 level 위 memory — 을 더한다.

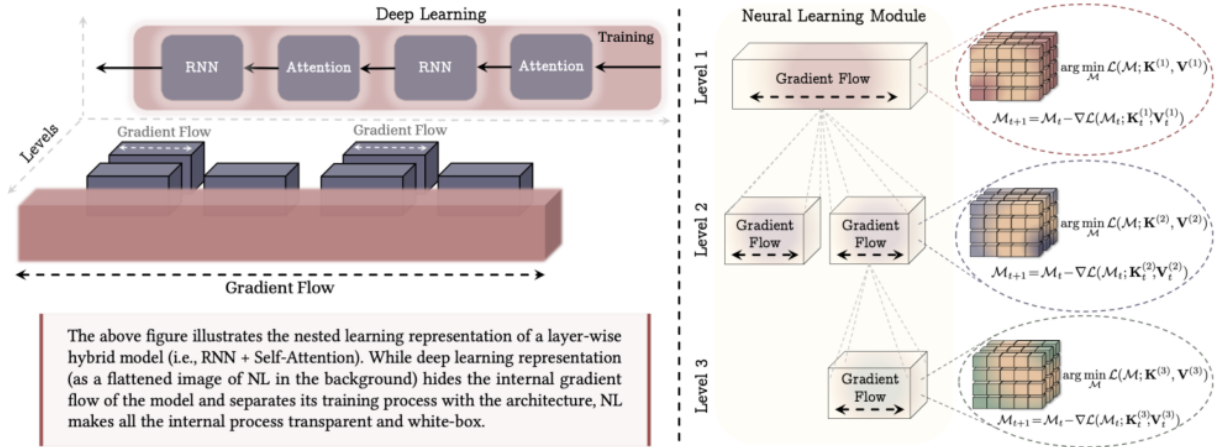


그림 16-1 — Nested Learning 패러다임. (좌) hybrid 아키텍처를 deep learning 관점으로 평탄화하면 내부 gradient flow 가 가려지지만, NL 은이를 서로 다른 level 의 gradient flow 가 겹친 투명한 표현으로 펼친다. (우) Neural Learning Module — 각 level 이 자기 context flow 를 자기 주기로압축하는 중첩 최적화 (associative memory) 문제들. 출처: Behrouz et al., [NL] (arXiv:2512.24695), 원문 Fig. 2 —원저자의 그림 (third-party), 본서의 결과가 아님.

16.3.3 update frequency, level, 그리고 NSAM

여러 최적화 문제로 분해된 모델에 질서를 주는 것이 **update frequency** 다. [NL Def. 2]: 구성 요소 A (parametric 이든 attention 같은 non-parametric 이든) 의 frequency f_A 는 단위 시간 (데이터 포인트하나당 갱신 한 번) 당 갱신 횟수다. attention 은 $f = \infty$, 동결 MLP 는 $f = 0$ — § 16.2 “두 극단” 의 형식화다. 순서는 $A \succ B$: $f_A > f_B$ 이거나, $f_A = f_B$ 이되 B 의 시점 t 계산 이 A 의시점 t 상태를 요구하면 A 가 빠르다. 어느 쪽도 아니면 같은 **level**(같은 주기, 상호 독립) 이다 — Adam 의 1 차/2 차 moment 가 대표적 동물 사례로 한 level 에 병렬로 놓인다 [NL Def. 2, App B]. 정렬에서 **높은 level** 일수록 **낮은 frequency** 다.

이제 시스템 전체가 정의된다. **Nested System** [NL Def. 3]: K 개의 정렬된 level, level k 는 (objective, context flow, feasible parameter) 의 삼중항 집합 $\{(L_i^{(k)}, C_i^{(k)}, \Theta_i^{(k)})\}_{i=1}^{N_k}$ 이고각 문제는 식 (16-1) 꼴 proximal GD 로 최적화되되 일부 “box” 는 non-parametric 해 (예: attention) 가 허용된다 [NL Eq. 19, § 3.2]. **NSAM (Nested System of Associative Memories)** [NL Def. 4] 는모든 context 가 key-value 집합인 특수형이고 [NL Eq. 20], Appendix A 일반형 [NL Def. 6, 7] 은 선형화를 full loss 로 되돌린다. **Neural Learning Module** 은 아키텍처와 그 훈련 과정을 하나의 NSAM 으로 함께 표현한 설계 단위이며, 같은 Transformer 라도 SGD 판과 Adam 판은 **다른 module** 이다 [NL § 3.2]. 원논문은 이 관점을 한장의 그림으로 요약한다 (그림 16-1): hybrid 아키텍처 (RNN+attention) 를 deep learning 식으로 “평탄화” 하면 각 block 내부의 gradient flow 가 가려져아키텍처와 훈련이 분리돼 보이지만, NL 은 같은 모델을 서로 다른 level 의 gradient flow 가 겹쳐 흐르는 white-box 로 펼쳐 각 level 이 자기 objective 로자기 context 를 압축하는 associative memory 임을 드러낸다 [NL § 3.2, Fig. 2].

그림 16-1 — Nested Learning 패러다임. (좌) hybrid 아키텍처를 deep learning 관점으로 평탄화하면 내부 gradient flow 가 가려지지만, NL 은 이를서로 다른 level 의 gradient flow 가 겹친 투명한 표현으로 펼친다. (우) Neural Learning Module — 각 level 이 자기 context flow 를 자기 주기로압축하는 중첩 최적화 (associative memory) 문제들. 출처: Behrouz et al., [NL] (arXiv:2512.24695), 원문 Fig. 2 —원저자의 그림 (third-party), 본서의 결과가 아님.

정의가 실제로 어떻게 작동하는지, 논문의 계단식 예제 네 개가 보여 준다 [NL § 3.1 – § 3.2].

- (a) **1-layer MLP + GD** = 1-level: 데이터를 LSS 에 사상하는 memory 하나 (§ 16.3.2).
- (b) **1-layer MLP + momentum GD** = 2-level [NL Eq. 10–13]: momentum buffer 가 gradient 를 압축하는 inner memory 이고 (전개는 § 16.3.5) outer 가 그 상태를 weights

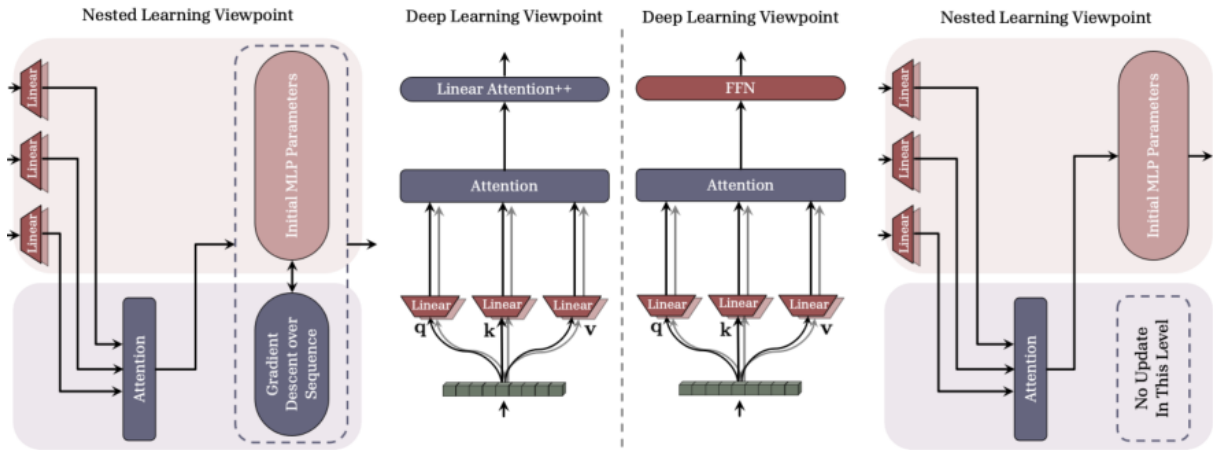


그림 16-2 — AdaTransformer(Linear Attention++). Transformer block 의 FFN 을 초기 상태가 meta-learn 된 linear attention 으로 치환해도출력이 대수적으로 동일하며, 유일한 차이는 두 weight 가 사는 level 이다: 빨강=level 1(frequency 1, context 불변 초기 MLP parameter), 파랑=level 2(sequence 에 대한 in-context 갱신). 좌우는 같은 계산의 Nested Learning 관점과 Deep Learning 관점. 출처: Behrouz et al., [NL] (arXiv:2512.24695), 원문 Fig. 3 — 원저자의 그림 (third-party), 본서의 결과가 아님.

에 적용한다 — 주기는 같지만 weights 계산이 momentum 상태를 요구하므로 momentum 이 더 빠른 level 이다.

- (c) **linear attention + GD** = 2-level [NL Eq. 14–18]: outer 가 projection W_K, W_V, W_Q 를 GD 로, inner 가 recurrence $W_t = W_{t-1} + v_t k_t^\top$ ($\tilde{\ell}(W; k_t, v_t) = -\langle W k_t, v_t \rangle$ 의 GD 1 스텝) 를 돈다 ($\rightarrow$ 6 장). inner 는 projection 을 동결 hyperparameter 로두지만, outer gradient $\partial \mathcal{L} / \partial W_K$ 는 stop-gradient 가 아닌 한 recurrence 를 관통한다 (NL 산문의 “관통하지 않는다” 는 inner 갱신이 projection 을 안 바꾼다는 뜻일 뿐). FWP($\rightarrow$ 6 장) 의 slow/fast 구분 과 겹친다 [NL § 3.1].
- (d) **linear attention + momentum GD** = 3-level: (b) 와 (c) 의 합성 [NL § 3.1].

정점은 **AdaTransformer**(별칭 Linear Attention++) [NL § 3.2, Fig. 3, Eq. 69]: Transformer block 과, MLP block 을 linear attention 으로 치환한 (key/value 는 앞단 attention 공유, 초기 상태 meta-learn) block 의 출력이 대수적으로 동일하다:

$$y_{\text{block}} = y_{\text{attn}} W_{\text{MLP}} \quad \text{VS} \quad y_{\text{block}} = y_{\text{attn}} W_{\text{LinAttn}}.$$

유일한 차이는 두 **weight** 가 사는 **level** 이다: W_{MLP} 은 level 1($f=0$) 이라 context 불변, W_{LinAttn} 은 level 2 에서 $W_t = W_{t-1} + v_t k_t^\top$ 로 in-context 갱신 (초기값 W_{init} 은 level 1 NTP 로 meta-learn) 된다. 즉 **recurrent memory module=level 하나 추가된 MLP block, hybrid 아키텍처=일부 MLP block 에 in-context 적응이 생긴 Transformer** [NL § 3.2, § 6] — “attention + SSM 이중 결합” 통념이 해소된다. 원논문 그림은 이 등가를 색으로 못박는다 (그림 16-2): 빨강은 level 1(frequency 1, context 불변) 의 계산 · weight, 파랑은 level 2(frequency L , sequence 에 대해 in-context 갱신) 의 계산 · weight 로, FFN 판과 Linear Attention++판은 같은 앞단 attention 위에서 대수적으로 동일하되 오직 두 번째 weight 가 사는 level 만 다르다 [NL § 3.2, Fig. 3, Eq. 69].

그림 16-2 — AdaTransformer(Linear Attention++). Transformer block 의 FFN 을 초기 상태가 meta-learn 된 linear attention 으로 치환해도 출력이 대수적으로 동일하며, 유일한 차이는 두 weight 가 사는 level 이다: 빨강=level 1(frequency 1, context 불변 초기 MLP parameter), 파랑=level 2(sequence 에 대한 in-context 갱신). 좌우는 같은 계산의 Nested Learning 관점과 Deep Learning 관점. 출처: Behrouz et al., [NL] (arXiv:2512.24695), 원문 Fig. 3 — 원저자의 그림 (third-party), 본서의 결과가 아님.

따름정리 (12 장 개념 회수): 초기 상태가 meta-learn 되지**않으면** ($W_0 = 0$ 초기 linear Transformer) 그 block 엔 persistent memory($\rightarrow$ 12 장) 가 없고, 출력 gating 의 linear layer 가 이를 대신한다는 것이 [NL § 5] 의 gating 해석 — “gating 이 왜 도움이 되는가” 의 구조적 답이다.

16.3.4 level 간 knowledge transfer: 다섯 가지메커니즘

남는 설계 측은 level 들이 서로 어떻게 지식을 주고받는가다. [NL § 3.3] 은 두 block $B^{(0)}$ (저주파·상위)· $B^{(1)}$ (고주파·하위) 사이의 **knowledge-transfer taxonomy** 다섯 가지를 제시한다.

표 16-1 — 다섯 가지 knowledge transfer 메커니즘 [NL § 3.3]

#	메커니즘	형식	대표 사례
1	direct parametric conditioning	$\mathcal{M}^{(0)}(\cdot) := \mathcal{M}^{(0)}(\cdot; \Theta^{(1)})$ 또는 $\mathcal{M}^{(0)}(\cdot; \mathcal{M}^{(1)}(\cdot))$ [NL Eq. 24–25]	FWP: fast weight 의 출력이 slow net 의 출력을 조건화 ($\rightarrow$ 6 장)
2	direct non-parametric conditioning	$\mathcal{M}^{(0)}(\cdot; \mathcal{C}^{(1)})$ [NL Eq. 27]	softmax attention 이 raw context 에 조건화
3	backpropagation across levels	같은 gradient flow, 다른 갱신 주기	CMS 의 level 연결 (§ 16.3.8)
4	initialization (MAML 형)	$\Theta_0^{(1)} = \arg \min_{\Phi} \mathbb{E}_{\mathcal{C} \sim \mathcal{C}^{(0)}} [\ell(\mathcal{M}^{(1)}(\cdot; \Phi), \mathcal{C})]$ [NL Eq. 28]	meta-learn 된 W_{init} (Titans·TTT/Atlas)
5	generation	weight 생성 (hypernetwork) 또는 context 생성	아키텍처가 optimizer 의 context(gradient) 를 생성

1·2 번의 공통 특징은 **backprop 이 level 을 건너지않는**다는 것 — 각 block 은 상대 상태를 자기 hyperparameter 로 취급한다 [NL § 3.3]. 3 번은 두 상태가 같은 gradient flow 에 있되 주기만 다르다 (§ 16.3.8 CMS 가 이 배선). 4 번은 4 장 MAML($\rightarrow$ 4 장) 이 분류되는자리이자, 라인이 [Titans] 부터 지고 온 meta-learned initial state W_{init} ($\rightarrow$ 12·15 장) 의 공식 좌석이다 — deep memory 모델 (Titans·Atlas·Miras·TTT) 은 4 번을 갖지만 대부분의 linear RNN 엔 level 간 transfer 가 없다 [NL § 6].

5 번이 이 논문 고유의 한 수다: **아키텍처는 optimizer 의 context(gradient) 를 생성한다** [NL § 3.3, § 6] — vanilla GD/Adam 에도 성립한다. 아키텍처가 다르면 gradient 분포가 달라 그것을 압축하는 memory(= optimizer) 의 최적 설계도 달라져야 한다 — 논문이 명명만 하고 납품하지 않는 **architecture-specific optimizer** 의 근거다.

정리: module 설계 = (1) 최적화 문제와 주기 (NSAM 구성) + (2) level 간 knowledge transfer, 두 결정이다 [NL § 3.3]. meta-learning·MAML·hypernetwork·learned optimizer 는 전부 이 두 축의 특수 선택지다.

16.3.5 optimizer 재해석: momentum, Adam, AdaGrad, Muon

momentum 은 gradient 들에 대한 Hebbian memory 다. layer ℓ 의 momentum GD 를 통일 표기로 쓰면 [NL Eq. 33]

$$W_{\ell,t+1} = W_{\ell,t} + m_{t+1}, \quad m_{t+1} = \beta m_t - \eta_{t+1} \delta_{\ell} \hat{x}_{\ell-1}^{\top} \quad (16-2)$$

이다 (원문 decay α 는 통일 retention α_t 와 무관해 β 로 개명, 표 16-2; 부호는 (M2) 관행). $\beta = 1$ 이면 식 (16-2) 의 m 갱신은 $\min_m \langle m \hat{x}_{\ell-1}, \delta_{\ell} \rangle$ 의 GD 1 스텝이고, $\beta \neq 1$ 은 여기에 m 의

ℓ_2 regularization 을 더한 것이다 [NL § 4.2, Eq. 34]. 즉 momentum 은 과거 gradient 를 자기 parameter 에 압축하는 **value-less(Hebbian) associative memory** 이며, 2 장의 (state, update, cost) 객체 m_t 가 “weights 보다 한 level 아래의 memory” 로 재명명된다 — 12 장의 **momentum-as-memory** 가정리적 지위로 승격된다 (구조는 § 16.3.3 (b) 의 2-level 그대로).

memory 라면 capacity 를 물을 수 있다. decay $\beta = 0.9$ 에서 최근 n 개의 정상상태 기여는 $1 - \beta^n$ 이므로 **50%를 넘으려면 7개 · 99%를 넘으려면 44개**가 필요하다 (원문은 6/43 으로 적었으나 $1 - 0.9^6 = 46.9\%$, $1 - 0.9^{43} = 98.9\%$ 로 임계값에 못 미친다) [NL § 4.3] — momentum 은 장기가 아니라 최근 기억이다. 이것이 continual learning 실패로 나타난다 (직교 task gradient 설정 [NL Eq. 45] 에서 momentum 이 새 task 로 이동해 옛 subspace 를 잃음). 진단은 **모델 capacity 가 아니라 optimizer 의 memory management 실패**다 [NL § 4.3]. 따름정리 [NL § 4.5]: “end of pre-training” 에서 optimizer state 를 버리면 momentum 이 저장한 landscape 지식이 통째로 사라진다 — continual learning module 이라면 이 저주파 level 도 보존 대상이다.

Adam 은 특정 objective 의 최적 memory 다 [NL App B]. element-wise memory m 에 대해 objective

$$\tilde{\ell}_t = \sum_{i=1}^t \|m \odot g_{i+1} - P_t\|_2^2 + \lambda \|m\|_F^2, \quad g_{t+1} = -\nabla_W \mathcal{L}(W_t; x_{t+1}) \quad [\text{NL Eq. 101}]$$

을 상정하면 — gradient 를 전역 통계 P_t 에 사상하는 ℓ_2 regression — 닫힌형 최적해의 recurrence 는 $\tilde{m}_{i+1} = \tilde{m}_i + \beta_1 g_{i+1}$ (1 차), $h_{i+1} = h_i + \beta_2 g_{i+1}^2$ (2 차) 이고 [NL Eq. 102], $P_t = \sqrt{\sum_i g_i^2}$ 로 고르면 **Adam 형 정규화 update 를 근사적으로 회수**한다 [NL Eq. 105]. 단 위 두 recurrence 는 EMA 가 아니라 누적 합 (AdaGrad 계열) 이고 bias correction 도 없으므로 표준 Adam 의 EMA 와 정확히 같지는 않다. outer-product 버전은 AdaGrad-with-momentum 을 [NL Eq. 106 – 111], 나머지 (RMSProp·SignSGD·NAdam·AMSGrad·RAdam·Lion; Shampoo·SOAP) 는 Adam·AdaGrad 와의 알려진 관계로따라온다 [NL § 4.2].

[평가] 이 역공학들은 존재 논증이지 유일성 결과가 아니다: Eq. 101 의 objective· P_t 는 Adam 이 나오도록 상정된 것이라 “Adam 은 최적 memory” 는 항상 “그 특정 element-wise ℓ_2 objective 에 대해” 로 읽어야 한다. 프레임의 가치는 새 도출이 아니라 변형을 체계적으로 생성하는 문법에 있다.

preconditioning 과 Muon. preconditioned GD $W_{t+1} = W_t - \eta_{t+1} P_{t+1}^{-1} g_{t+1}$ [NL Eq. 38] 의 P 는 “gradient 를 선택된 좌표계로 사상하는 법” 을 내부 objective $\min_P \tilde{\ell}(P(\hat{g}); g)$ 로 배우는 nested memory 다 [NL Eq. 39 – 41]. Muon(→ 2·14 장) $W_{t+1} = W_t + \text{NS}_\kappa(m_{t+1})$ [NL Eq. 42] 에서 좌표계는 직교 공간이다: orthogonalization objective

$$\tilde{\ell}(P(g); g) = \frac{1}{2} \|P(g) - g\|_F^2 + \frac{1}{2} \|P(g)^\top P(g) - I\|_F^2 \quad [\text{cf. NL Eq. 43}]$$

를 $O_0 = g$ 에서 GD 1 스텝 (내부 step size ζ) 으로 풀면 Newton – Schulz 의 3 차 다항 반복 [NL Eq. 44] 이 그대로 나온다. (주의: [NL Eq. 43] 은 직교화 항만 인쇄하나 그 gradient 엔 $O - g$ 항이 없어, Eq. 44 의 $O_i - g$ 가 나오려면 proximity 항 $\frac{1}{2} \|P(g) - g\|^2$ 이 함께 있어야 한다 — 위 objective 에 포함했다.) 결론: **Muon 의 NS 반복은 momentum 갱신 한 번당 κ 스텝을 도는 내부 최적화 level 이다.** [NL § 6] 은 이를 “neuron 당 더 많은 계산” 이라 부른다 — level 추가가 memory 계층만이 아니라 계산 심도의 증폭기이기도 하다.

16.3.6 프레임의 생성적 사용: 새 learning rule 들 — Delta Momentum, DMGD, DGD, GGD

재해석이 옳다면 optimizer 설계는 memory 설계와 같은 문법을 따른다: 6 장·13 장에서 sequence layer 에 했던 조작 (association·objective·memory 구조·feature map·출력비선형성·learning rule) 이 optimizer 에 이식된다 [NL § 4.4 – § 4.5].

- **preconditioned momentum** [NL Eq. 47]: value 를 preconditioner P_i 로 두면 momentum 이 (preconditioner $\leftrightarrow$ gradient) 사상을 배운다.
- **Delta Momentum** [NL Eq. 48–49]: 내부 objective 를 dot-product 에서 ℓ_2 regression $\|m g_i^\top - P_i\|_2^2$ 로 바꾸면 ($g_i = \nabla_W \mathcal{L}(W_i; x_i)$) update 가 delta rule($\rightarrow$ 5 장) 꼴이 된다:

$$m_{i+1} = \beta_{i+1} m_i - \eta_i (m_i g_i - P_i) g_i^\top.$$

update 가 자기 상태에 의존해 **gradient-dependent decay** 가 생긴다 (제한된 capacity $O(N)$ 관리 개선) — linear attention $\rightarrow$ DeltaNet 전이 ($\rightarrow$ 6 장) 의 optimizer 버전으로, 진동 곡률 toy 에서 표준 momentum 보다 빠르게 수렴한다 [NL Eq. 53, Fig. 4].

- **Deep Momentum GD (DMGD)** [NL Eq. 50]: momentum 을 행렬에서 MLP 로 승격한다 — $W_{i+1} = W_i + m_{i+1}(u_i)$, $m_{i+1} = \beta_{i+1} m_i - \eta_i \nabla \tilde{\ell}^{(2)}(m_i; u_i, 1)(u_i = \nabla_W \mathcal{L})$. deep memory($\rightarrow$ 12 장) 가 token 에 했던 것을 gradient 에 하는 것 — 과거 gradient 의 비선형 사상까지 저장.
- **higher-order feature map** [NL Eq. 51]· **비선형 출력** [NL Eq. 52]: $\phi(\nabla \mathcal{L})$ 로 key 들어 올리기 ($\rightarrow$ 14 장 ϕ_p 문법) 와 $W_{i+1} = W_i + \sigma(m_{i+1}(u_i))$. $\sigma = \text{NS} \cdot m$ 이 linear 면 **Muon** 이 특수 사례로 회수된다 — Muon 은 “비선형 출력을 단 Hebbian momentum” 이었다.

정점은 learning rule 자체의 교체다. **Delta Gradient Descent (DGD)** [NL § 4.5, Eq. 56–57]: 식 (16-1) 의 dot-product objective 는 각 gradient 를 상태와 무관하게 취급한다 — i.i.d. 면 합리적이거나 상관된 token 공간에서는 낭비다. objective 를 ℓ_2 regression 으로 바꾸면 ($u_t = -\nabla_{y_t} \mathcal{L}(W_t; x_t)$)

$$W_{t+1} = \arg \min_{\Phi} \frac{1}{2} \|\Phi x_t - u_t\|_2^2 + \frac{1}{2\eta_t} \|\Phi - W_t\|_2^2 \quad [\text{NL Eq. 56}]$$

이고, 입력이 정규화되어 있으면 ($\|x_t\|_2 = \lambda$ — normalization layer 가 있으면 성립) Sherman – Morrison lemma [NL Eq. 117, App C] 로 닫힌형이 나온다:

$$W_{t+1} = W_t (I - \eta'_t x_t x_t^\top) - \eta'_t \nabla_{y_t} \mathcal{L}(W_t; x_t) x_t^\top, \quad \eta'_t = \frac{\eta_t}{1 + \eta_t \lambda^2} \quad (\lambda = \|x_t\|; \lambda=1 \text{ 이면 } \frac{\eta_t}{1+\eta_t}). \quad (16-3)$$

GD 에 **data-dependent decay** ($I - \eta'_t x_t x_t^\top$) 가 자동으로 붙는다 — 지금 입력과 상관된 방향을 지우고 쓰는 rule 이다. 13 장 어휘로는 attentional bias 를 dot-product $\rightarrow \ell_2$ 로 바꾼 것, 5 장 어휘로는 delta rule 의 재발명이 되 **learning rule** 층위에서다. 일반화: **Generalized Gradient Descent (GGD)** [NL Def. 5, Eq. 59–60] 는 self-generated value $u_t = f_{W_t}(x_t)$ 를 갖는 임의의 self-referential memory

$$W_{t+1} = \arg \min_W \tilde{\ell}(\mathcal{M}(x_t; W), u_t) + \text{Ret}(W, \{W_i\}_{i=t-c+1}^t)$$

의 family 다 (Ret 은 최근 상태 근방에 해를 묶는 retention 항 $\rightarrow$ 13 장). [NL Eq. 59] 는 $\tilde{\ell}(x_t, u_t)$ 로 인쇄하지만 argmin 이 의미를 가지려면 $\tilde{\ell}$ 이 mapping $\mathcal{M}(x_t; W)$ 를 통해 W 에 의존해야 한다. backprop(GD·DGD 포함) 이 이 family 의 원소이고, 같은 형식을 momentum 에 적용한 것이 **Generalized Momentum (GM)** 이다 — 단 momentum 은 key/value 가 하위 level 에서 주어져 self-referential 이 아닌 conventional memory 다 [NL § 4.5].

16.3.7 아키텍처 재해석: 카탈로그의 재확인

[NL § 5] 는 [Miras] 의 결과 ($\rightarrow$ 13 장) 를 NSAM 어휘로 재수록하므로 자리만 확인한다. softmax attention 은 $\sum_i s(k_i, q) \|v_i - \mathcal{M}\|_2^2$ 의 **non-parametric** 해 (Nadaraya – Watson, $\rightarrow$ 8·13 장) [NL Eq. 62·63], Hebbian RNN(linear attention·RetNet·RWKV) [NL Eq. 64], delta rule RNN(DeltaNet·Longhorn·RWKV-7) [NL Eq. 65], OjaNet [NL Eq. 66–67], Omega rule($\rightarrow$ 14 장) [NL Eq. 68], L_p bias($\rightarrow$ 13 장) — 전부 dot-product· ℓ_2 ·Oja·window objective 의 GD 변

형으로 § 1.6 카탈로그와 1:1 이다. [NL] 의 추가 기여는 목록이 아니라 **자리 지정**이다: 이들 전부가 pre-training 보다 한 level 위이고 projection 들은 그 아래 level 에서 최적화된 다 [NL § 5]. 여기서 § 16.2 균일성 명제가 완결된다 (차이는 level·objective·learning rule 뿐) [NL § 5.1].

16.3.8 Continuum Memory System (CMS)

기존 backbone 의 memory 는 이분법이다: sequence 축을 섞는 working memory(attention) 와 pre-training 지식의 persistent 저장소 feedforward layer($f=0$) [NL § 7]. **Continuum Memory System (CMS)** 은 이를 update frequency 스펙트럼으로 일반화한다 [NL § 7.1]. CMS 는 MLP block 사슬 $\text{MLP}^{(f_1)}, \dots, \text{MLP}^{(f_k)}$ 이고 level ℓ 의 chunk 크기는

$$C^{(\ell)} := \frac{\max_i f_i}{f_\ell} C$$

— 즉 level ℓ 은 $C^{(\ell)}$ token 마다 한 번 갱신된다. forward 는 단순 합성 $y_t = \text{MLP}^{(f_k)}(\text{MLP}^{(f_{k-1})}(\dots \text{MLP}^{(f_1)}(x_t)))$ [NL Eq. 70] 이고, 갱신은 이 책의 표준형 (M5) 그대로 [NL Eq. 71]:

$$\theta_{i+1}^{(\ell)} = \theta_i^{(\ell)} - e_{i,\ell}, \quad e_{i,\ell} = \begin{cases} \sum_{t=i-C^{(\ell)}+1}^i \eta_t^{(\ell)} \varepsilon(\theta_t^{(\ell)}; x_t) & i \equiv 0 \pmod{C^{(\ell)}} \\ 0 & \text{otherwise,} \end{cases}$$

여기서 $\varepsilon(\cdot)$ 은 임의 optimizer 의 error 항이다 (GD 면 $\nabla_{\theta} \mathcal{L}$; 실험은 단순 gradient 누적). 두 강조: 첫째, **누적되는 loss 는 inner ℓ 이 아니라 task loss $\mathcal{L}(\text{NTP})$ 다** [NL § 7.1] — CMS 는 inference 중에도 NTP truncated gradient step 을 밟는 문자 그대로의 “느린 훈련” 이다. 둘째, 표준 Transformer MLP block 은 $k = 1 \cdot \text{frequency } 0$ 의 퇴화 사례로, CMS 는 대체물이 아니라 일반화다.

level 배선은 § 16.3.4 transfer 선택에 따라 세 변형이다 [NL § 7.1]. **Nested CMS**: level s 가 level $s+1$ 의 초기 상태를 meta-learn 하고 ($\theta_0^{(s+1)} = \arg \min_{\Phi} \mathbb{E}_{T \sim \mathcal{C}(s)} [\ell(\Theta, T; \Phi)]$ [NL Eq. 72]) 각 block 은 자기 context 가 끝날 때마다 그 초기값으로 **re-initialize** 된다 — higher-order ICL 을 주며, TNT 의 periodic reset($\rightarrow$ 15 장) 이 “throughput 트릭” 에서 “level 구조의 원리” 로 일반화되는 자리다. **Sequential CMS**: block 을 직렬로 놓고 모든 초기 상태를 최저 주파수 level 에서 일괄 meta-learn 한다 [NL Eq. 73]. **Independent (head-wise) CMS**: 서로 다른 context 길이의 block 을 병렬로 두고 $y_t = \text{Agg}(\text{MLP}^{(f_k)}(x_t), \dots, \text{MLP}^{(f_1)}(x_t))$ (예: 학습된 가중합) 로 결합한다 [NL Eq. 74].

CMS 가 continual learning 을 돕는 논리는 명시적이다 [NL § 7.1]: 빠른 block $\text{MLP}^{(f_s)}$ 가 옛 지식을 덮어써도 그 지식은 더 느린 block($s' < s$) 에 살아 있고, 초기 상태의 backprop transfer 가 이를 다시 빠른 block 으로 **순환**시켜 완전한 소실을 어렵게 한다 (12 장 persistent memory·15 장 “느린 곳에 주차된 지식” 이 한 스펙트럼 원리로 합쳐진다).

효율 논거도 논문이 준다 [NL § 7.1]. 갱신은 예정 시각의 block 에만 일어나 스텝당 건드리는 parameter 는 평균 $O(\frac{1}{f} \cdot \frac{L_{\text{layer}}}{5} \cdot d^2)$ (L_{layer} layer, 4-level, 최고 주파수 $\hat{f}$, 차원 d) — forward 전 체의 작은 조각이다. (M5) 구조상 $i \not\equiv 0 \pmod{C^{(\ell)}}$ 위치엔 순차 의존이 없어 전부 병렬 처리된다 — 9 장 chunkwise 스킴 ($\rightarrow$ 9 장) 이 그대로 적용된다.

배포 경로: **ad-hoc level stacking** [NL § 7.3] 은 CMS block 의 초기값을 pre-trained Transformer 의 MLP weights 로 놓는다 ($\text{MLP}_0^{(f_i)} = \text{MLP}_{\text{pre-trained}, i}$). inner learning rate $\eta^{(\ell)}$ 이 보 간 다이얼이다: $\eta^{(\ell)} \rightarrow 0$ 이면 동결 pre-trained 와 동일, 키울수록 in-context 적응이 커진다. § 16.6 의 Llama-3 개조 레시피가 이것이다.

16.3.9 CMS 를 optimizer 에 적용하면: M3

CMS 원리는 token 이 아닌 gradient context flow 에도 적용된다. **M3 (Multi-scale Momentum Muon)** [NL § 7.2, Alg. 1] 는 gradient context flow 에 CMS(independent 변형) 를 적용한 proof of concept 으로, § 16.3.5 의 진단 (momentum 은 43 스텝 너머를 못 기억한다) 에 대한 처방으로 momentum 을 두주기로 나눈다:

$$m_t^{(1)} = m_{t-1}^{(1)} + \beta_1 g_t \quad (\text{매 스텝}), \quad m_t^{(2)} = m_{t-1}^{(2)} + \beta_3 \sum_{i=t-\hat{C}+1}^t g_i \quad (\hat{C} \text{ 스텝마다}),$$

여기에 Adam 형 2 차 moment $h_t = h_{t-1} + \beta_2 g_t^2$ 를 유지하고, 두 momentum 을 각각 NS_κ 로 직교화한 뒤 가중합으로 결합한다:

$$\Theta_t = \Theta_{t-1} - \eta \frac{\text{NS}_\kappa(m_t^{(1)}) + \mu \text{NS}_\kappa(m_t^{(2)})}{\sqrt{h_t} + \epsilon} \quad [\text{NL Alg. 1, Eq. 75}]$$

(μ 는 aggregation 계수 — 원문 α 는 retention gate 와 충돌해개명, 표 16-2). 요약: **M3 = Adam + Muon + CMS**(independent CMS의 Agg가 가중합). 요점: 느린 momentum 이 decay 조절 (AdEMAMix 등) 이 아니라 **갱신을 늦추는 방식**으로 장기 문맥을 얻는다 — CMS chunked accumulation 을 gradient 에 옮긴 것 [NL § 7.2]. 논문은 M3 의 계산 오버헤드 · 대규모 도전을 자인한다 (수치 § 16.6) [NL § 7.2].

16.3.10 self-modifying Titans

Hope 의 sequence block 은 [Titans] 의 마지막 동결 지점을 푼다. Transformer·Titans 의 projection W_K, W_V, W_Q ·gate head 는 in-context 동결이라 token contextualize 가 pre-training 지식으로 상계된다: 1-layer projection 은 token·위치만의 함수라 문맥에 따라 뜻이 바뀌는 단어를 원리적으로 놓치고, short convolution·canon layer 도 국소 mixing 일 뿐이다 [NL § 8]. 처방: **모든 projection 과 gate 를 test-time 에 갱신되는 memory 로 승격시킨다.**

1 단계 — 완전 적응형 memory [NL Eq. 79 – 82]. 다섯 산출 $k_t, v_t, q_t, \eta_t, \alpha_t$ 전부를 각자의 memory 가 만든다:

$$\begin{aligned} k_t &= \mathcal{M}_k(x_t; W_{k,t-1}), & v_t &= \mathcal{M}_v(x_t; W_{v,t-1}), & q_t &= \mathcal{M}_q(x_t; W_{q,t-1}), \\ \eta_t &= \mathcal{M}_\eta(x_t; W_{\eta,t-1}), & \alpha_t &= \mathcal{M}_\alpha(x_t; W_{\alpha,t-1}), \end{aligned}$$

각 $\mathcal{M}_\square$ 와 본체 $\mathcal{M}_{\text{mem}}$ 은 $\min \ell(\mathcal{M}_\square; k_t, v_t)$ 를 최적화하고 출력은 $y_t = \mathcal{M}(q_t; W_{\text{mem}})$ 이다. 요점: **inner learning rate η_t 와 retention gate α_t (→ 13 장) 자체가 memory 의 출력이다** — 12 장에서 slow head 가 산출하던 hyperparameter 가 in-context 학습 객체가 된다. 모든 초기 상태 $\mathcal{M}_{\square,0}$ 은 sequence 전반에 meta-learn 된다 [NL § 8.1].

2 단계 — self-modification [NL Eq. 83 – 85]. 1 단계는 모든 memory 가 같은 v_t 를 공유하는 준최적 설계라 아직 self-modification 이 아니다 [NL § 8.1] — 그래서 각 memory 가 **자기 target** 을 스스로 생성한다:

$$\hat{v}_{\square,t} = \mathcal{M}_\square(v_t; W_{\square,t-1}) \quad [\text{NL Eq. 84}]$$

— 모델이 자기 훈련 데이터를 만들어 자기 update 알고리즘을 학습한다. Schmidhuber 의 self-referential weight matrix(1993)·SRWM(Irie et al. 2022) 계보를 chunk 병렬화가 가능한 형태로 계승한 것이다 (명칭 **self-modifying Titans**). 이 단계에서 $q_t = W_Q x_t$ 만은 비적응 projection 으로 남는다 [NL § 8.1].

3 단계 — learning rule [NL Eq. 86 – 88]. token 이 고도로 상관되므로 § 16.3.6 의 논리에 따라 inner optimizer 로 dot-product GD 대신 **DGD + weight decay** 를 쓴다. inner objective 는 표준 $\ell(W; k, v) = \|\mathcal{M}(k; W) - v\|_2^2$, update 는

$$W_{\square,t} = W_{\square,t-1}(\alpha_t I - \eta_t k_t k_t^\top) - \eta_t \nabla_W \ell(W_{\square,t-1}; k_t, \hat{v}_{\square,t}), \quad \square \in \{k, v, q, \eta, \alpha, \text{mem}\} \quad (16-4)$$

이며 read 는 $y_t = \mathcal{M}(q_t; W_{\text{mem},t-1})$ 이다. 식 (16-4) 를 (M2)·(16-3) 과 겹쳐 읽으면: $\alpha_t I$ 는 retention gate((M2)의 $\alpha_t W_{t-1}$ 과 동일 방향), $-\eta_t k_t k_t^\top$ 는 DGD의 data-dependent decay, 마지막 항은 self-generated target에 대한 gradient다. (단 우측 곱은 memory가 행렬일때만 type이 맞으므로 linear matrix memory 특수 사례로 읽는 것이 정확하다; deep-memory 공백은 각주 2.) 여섯 memory 모두 표준 deep memory(2-layer residual MLP $\mathcal{M}_{\square}(z) = z + W_{\square,1} \sigma(W_{\square,2} z)$) NL Eq. 89.

memory가 linear(행렬)인 특수 사례엔 닫힌 recurrence가 나온다 [NL Eq. 92–93]. ℓ_2 objective면 $W_{\square,t} = W_{\square,t-1}(\alpha_t I - \eta_t k_t k_t^\top) - \eta_t (W_{\square,\xi} k_t - \hat{v}_{\square,t}) k_t^\top$ (ξ =chunk anchor, § 16.4) — Gated DeltaNet(→ 6 장)의 일반화된 delta rule 꼴이다. dot-product objective면 gradient 항이 $\hat{v}_{\square,t} k_t^\top$ 로준다 NL Eq. 92.

두 정직한 각주를 남긴다. 첫째, 식 (16-4)의 $\square$ 집합엔 원문 그대로 q 가 포함되나 [NL Eq. 88, 96], 같은 절 본문은 $q_t = x_t W_q$ 를 유일한 비적응 projection으로 명시하고 [NL § 8.1] ablation에서 “w/o inner-projection q ”가 perplexity를 오히려 개선한다 (12.19 vs 12.24 [NL Table 6]). 최종 설계는 q 를 정적으로 읽는 것이 정합적이며, 이는 “level 추가가 항상 이득은 아니다”라는 논문 자신의 프레임에 대한 내부반례다 (§ 16.8). 둘째, deep memory(2-layer MLP)에서 $(\alpha_t I - \eta_t k_t k_t^\top)$ 항이 어느 행렬에 어떻게 적용되는지는 원문이 명시하지 않는다 (read 시점 표기 $\mathcal{M}_{\text{mem},t}$ vs $\mathcal{M}_{\text{mem},t-1}$ 의 흔들림도 배포형 chunkwise에서는 chunk 경계 snapshot으로 수렴해 소멸한다).

16.3.11 Hope block

Hope [NL § 8.3]는 두 산물을 한 block에 직렬로 잇는다: **self-modifying Titans**(작은 state, 표현력 있는 DGD rule) 뒤에 **CMS**(큰 capacity, 단순한 rule)를 붙인다 — CMS는 지속 저장을 위해 capacity가 커 단순 rule, self-modifying Titans는 state가 작아 표현력 rule로 상보적이다 [NL § 8.3]. forward는 (normalization·convolution 생략)

$$z_t = \mathcal{M}(q_t; W_{\text{mem},t-1}) \quad (\text{식 (16-4)의 갱신과 함께}), \quad y_t = \text{MLP}^{(f_k)}(\dots \text{MLP}^{(f_1)}(z_t)) \quad [\text{NL Eq. 94–97}]$$

이다. 실무 세부 [NL § 8.3]: q, k 는 ℓ_2 normalize(식 (16-3) DGD의 $\|x\| = \lambda$ 전제와 정합), window 4 local convolution. 변형 **Hope-Attention**은 self-modifying Titans를 softmax global attention으로 치환한 통제 변형으로 CMS 기여를 분리측정한다 [NL § 8.3].

12 장 어휘로: Hope는 Titans-MAC/MAG처럼 attention과 memory를 병렬 합성한 것이 아니라, Titans block의 모든 구성요소를 한 level씩 위로 올리고 (projection·gate의 memory화) 뒤따르는 MLP를 CMS 주파수 스펙트럼으로 펼친 것이다. Titans의 persistent/long-term/short-term 삼분류(→ 12 장)는 연속체의 세 점으로 흡수된다. 원논문의 backbone 비교 그림(그림 16-3)이 이 직렬 구조를 Transformer와 나란히 놓는다: self-modifying Titans 뒤에 high/mid/low frequency FFN 사슬이 이어지고, 각 block에 붙은 chunk length·frequency 표기가 Transformer의 두 극단 (attention $f = \infty$, FFN $f = 0$)을 하나의 주파수 스펙트럼으로 대체함을 보여 준다 [NL § 8.3, Fig. 5].

그림 16-3 — Hope의 아키텍처 backbone과 Transformer 비교 (normalization·data-dependent 성분 생략). Hope는 self-modifying Titans(작은 state, 표현력 있는 rule) 뒤에 서로 다른 update frequency의 FFN 사슬 (CMS: high/mid/low frequency)을 직렬로 잇는다. 각 block의 chunk length·frequency 표기가 long/short-term 이분법이 주파수 연속체로 펼쳐짐을 보여 준다. 출처: Behrouz et al., [NL] (arXiv:2512.24695), 원문 Fig. 5 — 원저자의 그림 (third-party), 본서의 결과가 아님.

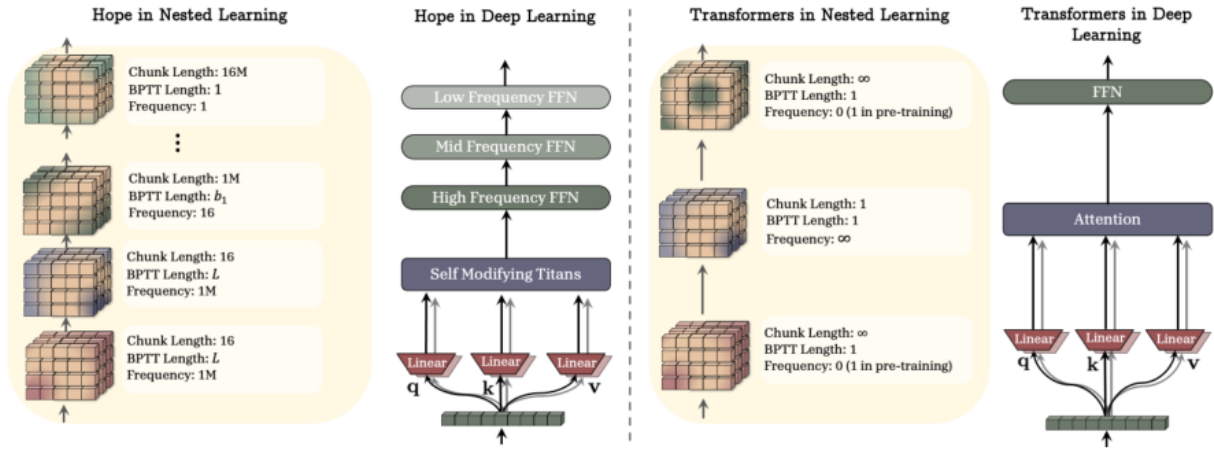


그림 16-3 — Hope 의 아키텍처 backbone 과 Transformer 비교 (normalization·data-dependent 성분 생략). Hope 는 self-modifying Titans(작은 state, 표현력 있는 rule) 뒤에 서로 다른 update frequency 의 FFN 사슬 (CMS: high/mid/low frequency) 을 직렬로 잇는다. 각 block 의 chunk length·frequency 표기가 long/short-term 이분법이 주파수 연속체로 펼쳐짐을 보여 준다. 출처: Behrouz et al., [NL] (arXiv:2512.24695), 원문 Fig. 5 —원저자의 그림 (third-party), 본서의 결과가 아님.

16.3.12 표기대응표

표 16-2 — [NL] 원 표기 ↔ 통일 표기 대응표 (STYLE-NOTATION § 1.7.5 의표를 그대로 복사; “(장-국소 추가)” 로 표시된 행은 이 장에서 더한 것)

NL 원 표기	통일 표기	의미 / 주의
Def. 1 의 $\mathcal{M}^*$	$\mathcal{M}^*$	
W_t (inner), $\theta^{(f_\ell)}$ (level params)	$W_t, \theta^{(\ell)}$	frequency 병기 필요시 f_ℓ 사용
$u_t = \nabla_y L$ — Local Surprise Signal	u_t	동일 (공식용어 LSS)
δ_ℓ — backprop 오차	δ_ℓ	동일
$m_{\ell,t}$ — momentum (outer)	m_t	
$\hat{M}, H$ — Adam 1 차/2 차 moment	m_t, h_t	
f_A , levels, $C^{(\ell)}$	f_ℓ (또는 f_A), $C^{(\ell)}$	동일
$\mathcal{M}_{\square,t}$, $\square \in \{k, v, q, \eta, \alpha, \text{mem}\}$ — self-modifying memories	동일	
$\hat{v}_{\square,t}$ — self-generated target	장-국소 유지	
DGD 의 $\eta'_t = \eta_t / (1 + \eta_t)$	$m^{(1)}, m^{(2)}, h_t; \beta$ 들은 장-국소 상수	$\triangleleft V_t \rightarrow h_t$ (value 와 충돌 방지)
M3 의 $M^{(1)}, M^{(2)}, V_t; \beta_1, \beta_2, \beta_3$	NS $_{\kappa}$	$\otimes$ 금지
NewtonSchulz $_T$	$vk^\top$ 표기	첫 등장 시 정의
$\otimes$ — outer product	유지	
고유명사: NSAM, Neural Learning Module, CMS, Hope, DGD/GGD/GM, Delta Momentum, DMGD, M3, AdaTransformer		

NL 원 표기	통일 표기	의미 / 주의
(장-국소 추가) $\alpha_{\ell,t+1}$ — optimizer 문맥의 momentum decay ([NL Eq. 33, 47 – 52])	β, β_{i+1}	$\triangle$ 통일 α_t (retention) 와 무관 — NL 은 optimizer 문맥에서 α 를 decay 로 쓴다
(장-국소 추가) α_t — self-modifying Titans 의 retention gate ([NL Eq. 88])	α_t	동일 방향 (남기는 비율)
(장-국소 추가) M3 의 aggregation 계수 α ([NL Alg. 1])	μ	retention gate 와 충돌 방지
(장-국소 추가) ζ — NS 유도 의 내부 step size ([NL Eq. 44])	ζ	장-국소 유지
(장-국소 추가) $\mathcal{M}_{\square, C[t/C]}$ — chunk 경계 snapshot	$W_{\square, \xi(t, C_{\square})}$	원문 인덱스는문면상 현재 chunk 끝이나 문맥상 직전 chunk 끝 상태 (§ 16.4 각주)
(장-국소 추가) $\tilde{L}$ — 임의 level 의 내부 objective	$\tilde{\ell}$	통일 체계에서 $\mathcal{L}$ 은 outer(task) 전용
(장-국소 추가) P_t — preconditioner / momentum 의 value	P_t	장-국소
(장-국소 추가) Eq. 92 의 dot-product 사례 부호	(M1) 관행으로고정	부호 관행 차이 (알고리즘 동일)
(장-국소 추가) o_t — Hope 내부 (self-modifying Titans) 의 중간 출력 ([NL Eq. 94 – 97])	z_t	최종 출력 y_t ·pre-activation z_ℓ 과 첨자로 구분 (ch12 의 중간항 개명 패턴)

16.4 Outer-loop training vs inner-loop test-time learning

이 절의 제목 자체가 심문 대상이다. [NL] 의 핵심 주장이 “outer/inner 이분법은 K -level 스펙트럼의 $K = 2$ 특수사례” 이고 극단적으로 “pre-training 도 corpus 를 context 로 하는 ICL 이라 training/test 경계가 없다” 이기 때문이다 [NL § 6]. 그러나 **실제로 실험된 Hope** 는 이 책의 이분법대로 만들어지고 서빙된다: 한번의 outer(pre-training) 뒤 inference 에서 정해진 rule 로 움직이는 상태들이 있다. 이 절은 그 구분을 Hope 에 전개한 뒤 어디까지가 이분법이고 어디부터 스펙트럼인지 표시한다.

표 16-3 — Hope 의 level 지도: 무엇이 어디서 누구에 의해 갱신되는가

구성요소	층위 · 주체	갱신 주기	갱신 rule
embedding, W_Q , convolution, normalization	slow Θ (outer)	pre-training 에서만	AdamW
여섯 memory 의 초기 상태 $\mathcal{M}_{\square, 0}$ (각 2-layer residual MLP)	slow Θ meta-learn (outer, transfer 4)	pre-training 에서만	AdamW — gradient 가 unrolled inner 를 관통

구성요소	층위 · 주체	갱신 주기	갱신 rule
CMS 각 level 의 초 기값 $\theta_0^{(\ell)}$	slow Θ meta-learn (outer, transfer 3+4)	pre-training 에서만	Sequential 이면 최저 주파수 level 에서 일괄 backprop [NL Eq. 73]
gate η_t, α_t	inner 파생값 (값 =inner, 산출 함수 =outer)	매 token (chunk 경계 snapshot)	$\mathcal{M}_\eta, \mathcal{M}_\alpha$ 의 forward
self-target $\hat{v}_{\square,t}$	inner 파생값 (self-generated)	매 token	$\mathcal{M}_{\square}(v_t; W_{\square,\xi})$
fast weights $W_{\square,t}$, $\square \in \{k, v, \eta, \alpha, \text{mem}\}$	fast W (inner)	$C_{\square}$ token 마다 일괄 (훈련 · 서빙 동일)	DGD + weight decay, 식 (16-4) (+ momentum — 각주)
CMS parameters $\theta_t^{(\ell)}$	중간 주파수 (inner, objective=outer $\mathcal{L}$)	$C^{(\ell)}$ token 마다	(M5): task-loss error 누적 [NL Eq. 71]
AdamW 의 m_t, h_t (훈 련에만)	optimizer 자신의 level	훈련 스텝마다	EMA ($\rightarrow$ 2 장); “end of pre-training” 에 서 폐기 [NL § 4.5]

16.4.1 outer loop: 무엇이 meta-learn 되고, gradient 는 어떻게 inner 를관통하는가

pre-training 이 학습하는 것은 표 16-3 의 위 세 행 전부터. outer objective 는 4 장 MAML 형 bilevel($\rightarrow$ 4 장) 그대로 **outer gradient가 inner-loop 갱신 경로 (각 memory 의 per-chunk DGD recurrence (16-4)·CMS 갱신 (M5))** 를 관통해 $\partial\mathcal{L}/\partial\mathcal{M}_{\square,0}$ 이 unrolled 경로를 거슬러 계산된다. training 무경험 독자에게: “optimizer 를 통과하는 backprop” 이라 훈련이 비싸고 서빙에서는 안 보인다.

훈련을 성립시키는 것은 chunkwise 병렬화다 [NL § 8.2]. 길이 L 을 $\lceil L/C \rceil$ 개 chunk 로 자르고 (각 memory 의 update frequency $f_{\square} = L/C_{\square}$) 두 가지를 chunk 경계 snapshot 에 고정한다: chunk anchor $\xi := C_{\square} \lfloor (t-1)/C_{\square} \rfloor$ 에 대해파생값 $k_t, v_t, \eta_t, \alpha_t$ 와 self-target $\hat{v}_{\square,t} = \mathcal{M}_{\square}(v_t; W_{\square,\xi})$ 는 전부 snapshot 상태 $W_{\square,\xi}$ 에서 생성되고, fast weight 는

$$W_{\square,t} = W_{\square,t-1}(\alpha_t I - \eta_t k_t k_t^{\top}) - \eta_t \nabla_W \ell(W_{\square,\xi}; k_t, \hat{v}_{\square,t}) \quad [\text{NL Eq. 90}] \quad (16-5)$$

— (i) 다음 chunk 전체의 key/value/self-target/gate 를 직전 chunk 끝 상태에서 **한 번의 배치 pass** 로 생성하고 (ii) inner gradient 도 그 snapshot 기준으로 평가한다. 그러면 chunk 내 모든 gradient 가 chunk 처리 전에 병렬 계산 가능해져 TTT dual form($\rightarrow$ 8 장)·Titans 병렬 훈련 ($\rightarrow$ 12 장) 이 그대로 적용된다 (chunk 크기는 실무상 둘: $\mathcal{M}_{\text{mem}}$ 용과 나머지 다섯 공용) [NL § 8.2].

각주 (표기 슬립): 원문 [NL Eq. 90] 의 snapshot 인덱스 $C \times \lceil t/C \rceil$ 는 문면상 **현재** chunk 끝 (아직 없는상태) 을 가리키나, 같은 절 산문 (“직전 chunk 의 마지막 상태” [NL § 8.2]) 이 의도를 확정하므로 [TNT Eq. 5–6] 슬립 ($\rightarrow$ 15 장) 과 같은유형으로 $\xi(t, C)$ 로 통일한다.

§ 16.3.2 에서 backprop 이 병렬화되지 않는 이유는 value 의 자기참조성이었다. self-modifying Titans 는 그 자기참조성을 **아키텍처로** 가져온모델인데, 왜 훈련이 병렬화되는가? 답: **chunk 안에서 value 생성기를 stale snapshot 으로 동결했기 때문이다**. 정확한 self-reference(현재 상태가 target 생성) 는 여전히 순차적이고 [NL § 4.5], chunkwise stale-snapshot 근사 ($\rightarrow$ 9 장 (M4), “semantic hyperparameter” 명제) 가 병렬성을 사는 공학적 타협이다. 배포된 Hope 는정확한 self-modifying 모델의 근사물이며 chunk 크기 $C_{\square}$ 가충실도-throughput 다이얼이다. SRWM 이 정확한 self-reference 를 갖고도 병렬훈련이 없어 못 간 스

케일을 Hope 가 가는 이유가 이 근사이고 [NL § 9.5], 그 대가 (충실도 손실의 정량) 는 논문에 없다 (§ 16.8).

16.4.2 inner loop: inference 에서 무엇이 어떤 rule 로움직이는가

decode 중 token 하나가 들어오면 Hope block 의 일은 세 겹이다. (1) **fast memory** 들 ($\mathcal{M}_k, \mathcal{M}_v, \mathcal{M}_\eta, \mathcal{M}_\alpha, \mathcal{M}_{\text{mem}}; \mathcal{M}_q$ 적응 여부는 원문상충이라 — § 16.3.10 각주 1 — 다섯 (정적)~여섯 (적응형), 각 2-layer residual MLP) 가 식 (16-5) 스케줄로 갱신된다. chunkwise 스케줄의 $1/C_{\square}$ 상황은 **훈련 · prefill** 에서만 성립한다 (chunk 전체 token 이 이미 있어 한 batch 로 계산). autoregressive **decode** 는 미래 token 이 아직 없어 다음 chunk 를 미리 batch 할 수 없다: 이상적 $C = 1$ decode 는 매 token 순차 갱신 (상황 없음, → 15 장 TNT 의 목표), $C > 1$ decode 는 C 개를 버퍼링해 경계에서 지연 batch-update(chunk-size mismatch 위험, → 9·15 장) 하는 형태다 — 어느 쪽이든 정확한 decode 비용 · latency 는 공개 구현 · 측정이 없어 미확정이다. gate η_t, α_t 는 별도 학습기가 아니라 이 memory 들의 forward 산출이다 (표 16-3). (2) **CMS 의 각 level** 이 $C^{(\ell)}$ token 마다 (M5) 로 갱신된다 — inference 중에도 NTP truncated gradient step 을 기하급수 간격으로 계속 밟는다. (3) 나머지 ($W_Q \cdot \text{embedding} \cdot \text{conv} \cdot \text{norm}$) 는 움직이지 않는다.

state 는 KV cache 처럼 $O(L)$ 로 자라지 않고 시퀀스 길이에 상수다 (산수는 § 16.7). fast memory 엔 별도 test-time optimizer 가 필요 없다 (DGD rule · gate 가 곧 optimizer). 단, ablation 표의 “w/o Momentum” 행 (13.58 vs 12.24 [NL Table 6]) 과 § 9.6 산문 (“removes the momentum term in the self-modifying Titans”) 은 배포된 inner rule 이 식 (16-4) 에 없는 Titans 식 momentum 항 S_t (→ 12 장) 를 실제로 지님을 말해 준다 — 그 정확한 수식 위치는 원문 어디에도 없다.

마지막으로 이분법/스펙트럼의 경계: outer/inner 이분법이 완전히 성립하는 것은 **한 번의 pre-training** 뒤 **동결서빙** 뿐이고, CMS 를 계속 살려 두는 continual 배포에서는 중간 주파수 level 이 “훈련도 추론도 아닌” 제 3 의 지위 (→ 1 장 불변식의 영구 위반) 를 차지한다 — § 16.7·17 장 lifecycle 으로 이어지는 문이다.

16.5 Concept ledger delta

표 16-4 — [NL] 이 개념 원장에 가한 변경 (기원 표기: 관련 장)

구분	개념	내용 / 전작과의 관계
신규	Nested Learning / Nested System / NSAM [NL Def. 3, 4]	모델+훈련 절차 = 중첩 최적화 시스템; § 1.6 카탈로그 전체를 포섭하는 상위 프레임
신규	context flow	각 level 이 압축하는 데이터 스트림 (token/gradient/상위 신호)
신규	update frequency 와 level [NL Def. 2]	attention $f = \infty$, frozen MLP $f = 0$; 높은 level=낮은 주파수. 17 장 [Sleep] 이 재사용
신규	Neural Learning Module	아키텍처 $\times$ optimizer 결합이 설계단위; Transformer+SGD $\neq$ +Adam
신규	Local Surprise Signal (LSS) u_t	surprise (→ 12 장) 의 출력 공간 대응물; backprop 재해석의 축

구분	개념	내용 / 전작과의 관계
신규	backprop = self-referential associative memory	훈련병렬화 불가의 구조적 이유; Schmidhuber 1993 계보 복권
신규	knowledge-transfer taxonomy (5 기제)	meta-learned W_{init} (→ 12·15 장) 이 기제 4 로 공식 분류
신규	DGD / GGD / GM / Delta Momentum / DMGD	delta rule(→ 5 장)·Miras objective 축 (→ 13 장) 의 optimizer 층위 이식
신규	CMS (+ Nested / Sequential / Independent)	TNT global/local hierarchy(→ 15 장) 의아키텍처화; long/short-term 이분법의 스펙트럼화
신규	self-modifying Titans	Titans(→ 12 장) projection·gate 전부의 memory 화 + self-generated target
신규	Hope / Hope-Attention	라인의 새 기준모델; CMS 분리 측정용 통제 변형
신규	M3	CMS 를 gradient context flow 에 적용; momentum-as-memory 의 생성적 산물
신규	ad-hoc level stacking	pre-trained MLP → CMS 초기값; retrofit 배포 경로
신규	parametric vs non-parametric ICL	Transformer=non-parametric 해, recurrent/TTT=parametric; emergent 아닌 구조적
신규	CTNL benchmark	MTOB(Kalamang)+Manchu 순차 학습으로 in-context CF 측정 (CF 축 → 11 장)
확장	momentum-as-memory	12 장 관찰 (S_t =past surprise) → 정리: 모든 gradient optimizer 가 gradient 의 memory; capacity 산수 (6/43) 까지 정량화
확장	surprise	momentary/past(→ 12 장) → LSS 로 출력 공간확장; backprop 전체가 surprise 기반 memory
확장	retention gate (→ 13 장)	13 장 소유 그대로; DGD 가 같은원리 (data-dependent decay) 를 learning rule 층위에서 재생산
확장	meta-learned W_{init} (→ 4·15 장)	TNT load-bearing 장치 → 5 대 transfer 중 기제 4 로 일반화
확장	periodic reset (→ 15 장)	TNT throughput 트릭 → Nested CMS 의 context-end re-initialization

구분	개념	내용 / 전작과의 관계
확장	persistent memory (→ 12 장)	prefix token → 주파수 0 의 극점; init 미-meta-learn 시 gating 이 대역을 맡는 corollary
확장	Omega rule (→ 14 장)	그대로 수입; GGD 의 Ret 자리 에워치 지정
폐기	아키텍처 vs optimizer 구분	같은 객체 (associative memory) 의 다른 level — 논문 제목의 한 수
폐기	long-term/short-term 이 분법	주파수 연속체로대체
폐기	“test-time” learning/ memorization 명칭	continual 설정에서 오도적 — parametric ICL 로 대체 제안 [NL § 6] (이 책은 명칭유지, 이의만 기록)
폐기	ICL 의 emergent 지위	2 개 이상 level 의 구조적 귀결 (단, 좋은성능엔 잘 훈련된 저 주파 level 필요 [NL § 6])

16.6 실험과스케일

공통 setup. 언어 모델 실험은 FineWeb-Edu+장문 혼합 corpus, 32K vocab, AdamW(모델별 lr 튜닝, 나머지 [Titans] 기본) 로 from scratch 훈련이다 [NL § 9.2, § 9.3]. 두 스케일: **760M / 30B tokens** 와 **1.3B / 100B tokens** [NL § 9.3]; RULER 는 약 50B tokens 훈련 [NL § 9.2]. baseline 은 Transformer++, Samba, RetNet, DeltaNet, RWKV-7, Comba, TTT, Miras(Memora), DLA, Titans 이다.

언어 모델링과 상식 추론 [NL Table 2]. 760M/30B 에서 Hope 는 Wikitext ppl 18.68, LAMBADA ppl 20.07, 8 개 상식 추론 평균 acc 52.28 로 전 baseline 최고다 (차점 Titans 51.68). 1.3B/100B 에서 Wikitext 14.39, LAMBADA 10.08, 평균 58.04(Titans 56.82) 로, attention-free 대비 격차가 스케일과 함께 커진다 [NL § 9.3].

RULER needle-in-a-haystack [NL Table 1]. Hope 는 attention-free 중 전 과제 · 난이도 최고다: S-NIAH-3(UUID) 16K 에서 24.8 vs Titans 21.2·RWKV-7 5.8, MK-NIAH 16K 에서 14.8 vs Titans 8.2 — linear memory 가 붕괴하는 곳에서 deep memory capacity 가 버틴다 [NL § 9.2]. 통제 변형 Hope-Attention 은 S-NIAH-1 4K/8K/16K 모두 100 으로 같은 attention 의 Transformer(88.6/76.4/79.8) 를 상회 — 이 증분은 CMS 의 몫이다.

BABILong [NL Fig. 9]. 원 설정대로 fine-tune 한 소형 모델비교에서 Titans·ARMT·Hope 가 1M token 까지 경합하다 Titans·ARMT 는 급락하고 **Hope 는 10M token 까지 유지된다** (논문은 CMS 설계를 원인으로 지목) [NL § 9.2]. 정직성 항목: 이 10M 은 **fine-tuning 전제다** — zero-shot 에서는 Hope 포함 모든 소형 모델이 크게 떨어지며, 저주파 level 이 과제에 맞게 조정돼야고 주파 memory 관리가 작동한다 [NL § 9.2].

continual learning [NL Fig. 6, Fig. 8]. class-incremental 분류 (CF 측정 프로토콜의 배경은 → 11 장) 세 벤치마크 — CLINC(150 intent), Banking(77 intent), DBpedia(70 클래스) — 에서, Llama3-8B/Llama-3B 를 ad-hoc level stacking (§ 16.3.8) 으로 개조하고 15B tokens continual pre-training 한 Hope 변형이 ICL, EWC(→ 11 장), 외부 학습기 SOTA 인 InCA 를 전부 상회한다 [NL § 9.1]. 신설 benchmark CTNL(MTOB Kalamang+Manchu; ChRF) 에서는: 언어 하나씩은 Hope 변형이 ICL 과 동급 이상, **두 언어를 순차로 배우면 ICL 은 붕괴** (pre-training 으로 회귀) 하는 반면 Hope-1/2/3(추가 level 1/2/3 개) 은 level 수에 단조 개선되어 Hope-3 는 단일 언어 성능을 거의 회복한다 [NL § 9.1]. level ablation [NL Fig. 7]: level 이 많을수록

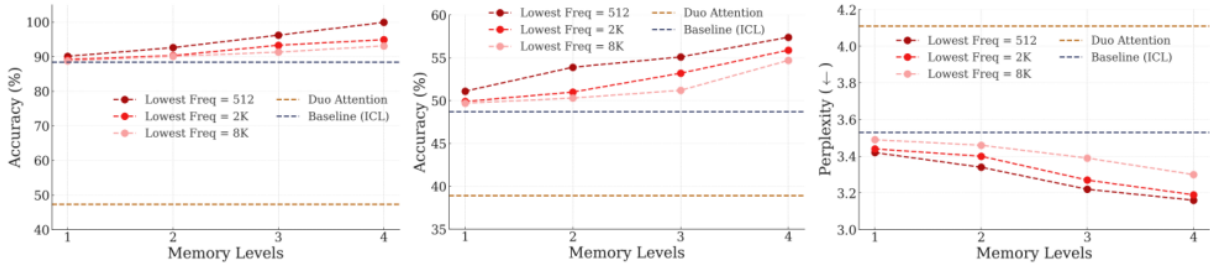


그림 16-4 — memory level 수가 in-context learning 성능에 미치는 효과 (좌: RULER MK-NIAH, 중: LongHealth, 우: QASPER — 우측은 perplexity 라 낮을수록 좋음). level 이 많을수록, 그리고 최저 주파수 (lowest freq) 가 낮을수록 (persistent memory 가 강할수록) 성능이 오르며, ICL·DuoAttention baseline(점선) 을 상회한다. 출처: Behrouz et al., [NL] (arXiv:2512.24695), 원문 Fig. 7 — 원저자의 그림 (third-party), 본서의 결과가 아님.

좋고, 최저 주파수는 512 가 최고이되 2K 가 근접성능에 훨씬 저렴해 효율 sweet spot 이다 [NL § 9.1]. 이 그래프 (그림 16-4) 는 세 long-context 벤치마크 (MK-NIAH·LongHealth·QASPER) 에서 memory level 수를 1 → 4 로 늘리면 성능이 단조 개선되고, 어떤 level 수 · 최저 주파수에서도 ICL·DuoAttention baseline 을 상회함을 보여 논문의 핵심 주장 — “더 많은 layer 가 아니라 더 많은 level” — 을 실증한다 [NL § 9.2, Fig. 7].

그림 16-4 — memory level 수가 in-context learning 성능에 미치는 효과 (좌: RULER MK-NIAH, 중: LongHealth, 우: QASPER — 우측은 perplexity 라 낮을수록 좋음). level 이 많을수록, 그리고 최저 주파수 (lowest freq) 가 낮을수록 (persistent memory 가 강할수록) 성능이 오르며, ICL·DuoAttention baseline(점선) 을 상회한다. 출처: Behrouz et al., [NL] (arXiv:2512.24695), 원문 Fig. 7 — 원저자의 그림 (third-party), 본서의 결과가 아님.

in-context recall 과 **MAD** [NL Table 3, 4]. 의무 서술 caveat 이여기 있다: 짧은 in-context recall 에서는 여전히 **Transformer 가 최고이고 격차가 크다** — FDA 에서 Transformer 67.3 vs Hope 41.9, SWDE 71.4 vs 65.9. Hope 는 attention-free 중 최고 (Titans·RWKV-7·Comba 전 부 상회) 로 격차를 좁혔을 뿐이다 [NL § 9.4]. [Atlas] 의 53.55 vs 43.70 ([Atlas Table 5], → 14 장) 에서 확인된 in-context retrieval gap 이 이 라인의 종합판에서도 **미해소**라는 뜻이다. 반면 합성 벤치마크 MAD 에서는 Hope 가 Transformer 를 포함한 전부를 이긴다 (compression 51.2, fuzzy ICR 52.1, memory 85.2) [NL Table 4].

formal language 인식 [NL Table 5]. Irie et al. 2023 구성의 parity, $(aa)^*$, $(abab)^*$, $a^n b^n$, $a^n b^n c^n$, Shuffle-2 에서 Hope 는 훈련 분포 밖 길이 구간 (Bin1) 을 포함해 **전부 100.0** 이다 (Transformer 는 parity Bin1 0.0 으로 실패, linear attention·DeltaNet 도 Bin1 대부분 0). LSTM·SRWM 도 전 항목 100 이나 병렬 훈련이 불가하다 — Hope 의 차별점은 “비선형 recurrence 급 state tracking + 병렬 훈련” 의 양립이다 [NL § 9.5]. 정직성 항목: 이것은 **실증이지 정리가 아니다** — [NL] 엔 K -level NSAM 의 표현력 정리가 없고 state-tracking 실험이 그 자리를 대신한다 (→ § 16.8; 12 장 [Titans Thm 4.1] 과 같은 계열).

component ablation [NL Table 6] (760M 급, 구성요소를 하나씩 제거).

표 16-5 — Hope 구성요소 ablation ([NL Table 6] 재구성; 언어 모델링 평균 ppl ↓ / 상식 추론 평균 acc ↑)

변형	ppl	acc
Hope (전체)	12.24	58.1
DGD → 단순 GD	13.41	56.5
momentum 제거	13.58	56.9
weight decay 제거	13.71	57.2
CMS 제거	13.04	57.3
inner projection k 동결	13.77	56.9
inner projection v 동결	13.90	55.1

변형	ppl	acc
inner projection q 동결	12.19	57.4

읽는 법: DGD·momentum·weight decay·CMS·inner k ·inner v 는 전부 제거 시 악화 (특히 inner v (self-target 원료) 동결이 acc 를 가장 크게 깎음, 55.1). 유일한 예외가 inner q 동결로 ppl 이 오히려 개선된다 (12.19) — § 16.3.10 각주 · § 16.8 “level 배치는 경험적” 한계로 직결된다.

M3 [NL § 9.7]. ViT-24M/86M 을 ImageNet-21K 에서 optimizer 만 바꿔 훈련하면 M3 가 AdamW·Muon 대비 최저 train/test loss 를 준다 [NL Fig. 11]. 효율은 별도: Transformer LM(140M·1.3B) 훈련에서 momentum 이여럿 ·NS pipeline 추가라 **Muon** 보다 느리고 **AdaMuon** 과 동급이다 [NL Fig. 12] — 품질 이득이 비용과 함께 온다.

스케일의 정직한 상한. from-scratch 실증 상한은 라인 공통 **1.3B / 100B tokens**(retrofit 은 Llama3-8B 까지) 이고, Hope 의 decode throughput·latency·메모리 수치는 논문에 없다 — wall-clock 측정은 M3(훈련 시간) 뿐으로, 6 편 공통의 decode wall-clock 부재가 종합판에서도 이어진다. 7B+ 및 production Transformer 대비 우위는 열린 문제다 (§ 16.8).

16.7 Systems/serving 합의

state 의 산수: KV cache 가 아니라 per-request weights. Hope block 하나의 inference state 는 여섯 개의 2-layer residual MLP memory(memory 당 행렬 2 개, 도합 약 12 개의 $d \times d_h$ 급 행렬)에 inference 중표류하는 CMS level 별 MLP weights 를 더한 것 — KV cache 의 $O(L \cdot d)$ 성장 과 달리 시퀀스 길이에 상수이되 그 상수가 weights 규모다 (crossover 는 단일 matrix memory(→ 6 장) 보다 훨씬 오른쪽). 질적 차이: 이 state 는 read-only cache 가 아니라 **mutable weights** 라 per-request 로 달라 shared-weight batching 이 깨지고 (→ 1 장 Rosetta: grouped-GEMM decode), checkpoint/restore 는 이 텐서들의 영속화다. Titans/TTT/Atlas 와 같은 문제 클래스 이되 fast-weight 텐서가 약 6 배이고 chunk 주기 갱신이 이를 부분 상쇄한다.

decode state 의 RMW 트래픽. 각 fast memory 는 $C_{\square}$ token 마다 자기 parameter 텐서 전체를 read-modify-write 하므로 token 당 상환 write 트래픽은 (state bytes)/ $C_{\square}$ — chunk 크기가 대역폭 다이얼이다. CMS 도 level 별로 같은 구조를 반복하며 상환추정은 § 16.3.8 의 $O(\frac{1}{f} \cdot \frac{L_{\text{layer}}}{5} \cdot d^2)$ 이다 [NL § 7.1]. “최저 주파수 2K 가 sweet spot” [NL § 9.1] 은 이 트래픽상환 이 품질을 크게 잃지 않는다는 보고다.

update frequency ↔ memory 계층 배치. 주파수 스펙트럼 = 배치스펙트럼: 매 chunk 갱신되는 여섯 fast memory 는 decode 중 상시 접근되는 hot set(HBM 상주) 인 반면, $C^{(\ell)} = 2K$ 급 CMS level 은 수천 token 에 한 번만 RMW 돼 더 차가운 계층을 허용한다 — “level 번호가 곧 cache tier 힌트” [NL § 7.1].

병렬화와 kernel 형태. 훈련 ·prefill 은 § 16.4 의 chunkwise dual 스킴이다. memory 가 MLP 이므로 kernel 은 TTT-MLP 형 **batched small-GEMM** 시퀀스이고 (→ 8·9 장), linear 특수 사례 (식 (16-5) 행렬 버전) 는 일반화된 delta rule 이라 DeltaNet 계열 WY/UT-transform chunk kernel(→ 9 장) 이 자연 템플릿이다. 두 chunk 크기 ($\mathcal{M}_{\text{mem}}$ 용 · 나머지용) 는 staleness 를 따로 튜닝하는 kernel 자유도다. decode 의 정확한 update 비용 ($C = 1$ 순차 vs $C > 1$ 경계-지연 batch) 은 미측정이고 (§ 16.4.2), 짧은 recall 에서는 Transformer 가 이긴다는 crossover 가 [NL Table 3] 그대로다.

retrofit 배포 경로. ad-hoc level stacking (§ 16.3.8) 은 이 라인최초의 현실적 이식 스토리다: 기존 Transformer MLP 를 CMS 초기값으로 삼고 15B tokens continual pre-training 으로 개조해 [NL § 7.3, § 9.1] from-scratch 없이 채택 장벽을 우회한다.

[평가] 가장 파괴적인 합의는 수식이 아니라 배포 모델에 있다. continual 배포에서 모델은 **서빙 중 사용자 데이터로 자신의 NTP truncated gradient step** 을 계속 밟는다 — train/serve 경계의 소거는 서빙 불변식의 영구 폐기다. per-tenant weight 발산 · rollback·provenance·안전성, speculative decoding 상호작용 (→ 1 장) — 어느 것

도 논문이 다루지 않고 Hope 의 서빙경제학 (throughput/latency/메모리) 은 수치가 전무하다. 이 라인의 서빙 문제는 종합판에서 풀리기는커녕 6 배로 늘었다.

16.8 한계와 bridge-out

논문이 스스로 그은 한계선부터 정리한다.

catastrophic forgetting(→ 11 장) 은 해결되지 않았다. forgetting 은 압축의 귀결 (유한 capacity 는 새 정보를 위해 잊도록 강제) 이며, Hope·CMS 는 실험된 과제에서 이를 줄였을 뿐일 반적으로 “풀지” 않았다 [NL § 10]. CMS 가 하는 일은 어디서 · 얼마나 빨리 잊는지를 주파수 축에 재배치하는 것 — level 간 capacity 의 원리적 배분은 열려 있다.

level 설계는 경험적이다. level 수 · 주파수 · 구성요소 배치에 대한 이론이 없다. inner q ablation(12.19 vs 12.24 [NL Table 6]) 은 잘못 놓인 level 이 해가 됨을 보이는 내부 반례이고, chunk 크기 · level 수 · 주파수는 전부 손으로 고른 hyperparameter 다 (2K sweet spot 도 실험 적 발견). 주파수 스케줄 학습은 미착수다.

이론은 재해석까지만이다. optimizer 역공학은 존재론증이고 (§ 16.3.5 [평가]), K -level NSAM 이 level 수에 따라 오르는 표현력 계층에 대한 정리는 없다 — formal language 100.0 은 실증이다 (§ 16.6). [Titans Thm 4.1] 의 증명 부재 (→ 12 장) 라는 이론공백이 종합판에서도 이어진다.

self-reference 의 충실도 분석이 없다. chunkwise stale-snapshot 근사가 정확한 self-modifying 모델을 얼마나 훼손하는지 (chunk 크기에 대한 오차 한계) 는 어디에도 없다 (→ 9 장 라인 공통 caveat). 정확한 self-reference 는 여전히 순차적이고, 더 나은 병렬화 존재 여부는 열린 문제다.

서빙 경제학과 스케일. § 16.6 – 16.7 대로: Hope 의 wall-clock 부재, per-request mutable weights 의 미해결 인프라, from-scratch 1.3B/100B 상한, M3 의 자인된 오버헤드 · 대규모 미검 증, 그리고 “architecture-specific optimizer” 는 § 16.3.4 에서 근거까지 두고도 설계는 미래 과제로 남았다 [NL § 6]. in-context retrieval gap(FDA 67.3 vs 41.9) 도 미해소, Cartridges 류비 교도 유보다 [NL § 9.1].

[평가] 이 장의 프레임으로 라인을 되돌아보면 [NL] 의 기여는 두 겹이다. 재해석의 층 (“모든 것이 associative memory 다”) 은 검증불가능한 관점 선언에 가깝지만, 그 관점이 **생성한 물건** — DGD, CMS, self-modifying Titans, M3 — 은 ablation 으로 기여가 측정된 구체물이다 (표 16-5). 관점의 가치는 참 · 거짓이 아니라 생성물성능으로 정산되며, 그 정산서엔 inner q 반례와 미측정 서빙 비용이 함께 적힌다.

Bridge-out: [Sleep] 으로. [NL] 이 다음 논문에 넘기는 것은 스스로 범위 밖으로 선언한 반쪽이다. § 16.2 에서 신경생리학은 두 단계의 consolidation 을 말했다: [NL] 이 구현한 것은 **online consolidation** — 입력이 흐르는 동안, CMS 의 주파수 스펙트럼으로 — 뿐이고, 수면 중 replay 로 기억을 재조직하는 **offline consolidation** 은 명시적으로 다루지 않았다 [NL § 1.1]. 따라서 [NL] 의 세계에서 모든 적응은 입력이 흐르는 동안 일어나고, capacity 는 고정이며, 그래서 forgetting 은 필연으로 남는다. 빠른 level 의 지식이 예정된 갱신으로 덮여써지기 전에, 그것을 **어디로 옮길 것인가?**

Sleep 의 답은 train/test 구분의 잔재를 마저 지우고 **wake/sleep lifecycle** 을 세우는 것이다. wake 는 [NL] 의 CMS online consolidation 그대로다 (update frequency 정의 Def. 2· CMS 갱신식 (M5) 를 [Sleep] 이 문자 그대로 수입). sleep 은 두 offline 프로세스다: (1) Memory Consolidation — 빠른 block 의 예정된 갱신이 지식을 덮어쓰기 직전 그 지식을 다음 느린 block 에 새 **low-rank expert** 로 위쪽 **distillation**(Knowledge Seeding) 하고 빈 block 은 reset(synaptic pruning) — forgetting 을 regularization 이 아니라 **capacity 성장**으로 재정의 한다; (2) Dreaming — 자기 생성 데이터로 자기를 강화하는 RL loop. sleep 이 발화하는 시점은 정확히 CMS 의 chunk 경계다 — TNT 의 reset → Nested CMS re-initialization → [Sleep] 의 “consolidation 후 reset” memory 위생 원리로 이어진다. [NL] 이 “잊히기 전에 옮길 곳이 없다” 는 문제를 만들었다면, 17 장은 옮기는 절차 — 그리고 이 라인 전체의 완성형 — 를 다룬다.

ch17. Sleep: Learning to Self-Modify and Consolidate Memories

17.1 Bridge-in: [NL] 이 남긴 문제 — consolidation 의 절반

Part II 의 마지막 논문은 [Sleep] (Language Models Need Sleep: Learning to Self-Modify and Consolidate Memories, arXiv:2606.03979; Ali Behrouz, Farnoosh Hashemi, Vahab Mirrokni; arXiv v1 2026-06-02) 이다. 논문 1 면의 각주는 이작업의 한 버전이 2025 년 9 월부터 OpenReview 에 공개되어 있었다고 명시하는데 [Sleep p.1 각주], 이 시점 주장은 § 17.8 에서 다룰 2026 년 on-policy self-distillation 물결과의 우선권 문제에서 다시 등장한다.

이 장의 출발점은 16 장이 멈춘 자리다. [NL] (Nested Learning, arXiv:2512.24695) 은 아키텍처와 optimizer 를 update frequency($\rightarrow$ 16 장)의 단일 스펙트럼 위에 재배열했다. attention 은 frequency ∞ 의 memory 이고 frozen MLP 는 frequency 0 의 memory 이며, Continuum Memory System(CMS, $\rightarrow$ 16 장) 은 그 사이의 빈 구간을 chunk 주기 $C^{(\ell)}$ 마다 갱신되는 MLP 사슬로 채웠다. Hope($\rightarrow$ 16 장) 는 self-modifying Titans 와 CMS 를 결합해 이 구도를 실증했다. 그러나 [NL] 이 남긴 것이 네 가지다. 첫째, **catastrophic forgetting($\rightarrow$ 11 장) 은 해결이 아니라 지연되었다**. 다중 frequency 는 덮어쓰기를 미룰뿐이고, 모든 level 의 update 주기가 정렬되는 순간 CF 는 일어난다 — [Sleep § 2.2] 가 [NL] 을 인용해 이 진단을 그대로 재확인한다. 둘째, **offline consolidation — replay-sleep 계열의 기계 — 는 [NL] 스스로 명시적으로 범위 밖에 두었다**: “두 번째 단계가 동등하게, 또는 그 이상으로 중요함에도, 이 작업은 첫 단계 — online 과정으로서의 memory consolidation — 에 집중한다” [NL § 2]. 셋째, **capacity 가 고정이다** — 유한 크기 파라미터로의 압축인 이상 새 지식은 언젠가 옛 지식을 덮어쓴다는 이 진단은 [Sleep § 1] 이 online-only consolidation 에 들이대는 비판이다. 넷째, **모든 적응이 입력이 흐르는 동안 일어난다** — 둘째 항목의 직접적 따름정리로, 모델이 입력을 끊고 자기 내부를 정리하는 시간이 [NL] 의 설계에는 존재하지 않는다.

[Sleep § 1] 은 [NL] 의 anterograde amnesia 비유를 이어받아 이결핍을 다시 조준한다. 배포된 LLM 의 지식은 두 곳에만 있다: 세션이 끝나면 소멸하는 context window, 그리고 pre-training 종료 시점에 동결된 MLP·projection weights. 단기 기억과 장기 기억 사이를 잇는 경로가 없으므로, 모델은 새 장기 기억을 형성하지 못하는 환자처럼 “영원한 현재” 를 산다. [NL] 의 CMS 는 이 경로의 절반 — 깨어 있는 동안 fast block 에서 slow block 으로 지식이 end-to-end 로 흘러가는 **online consolidation**(개념은 $\rightarrow$ 11 장, [NL] 의 기법은 $\rightarrow$ 16 장) — 을 놓았다. [Sleep § 1] 은 online-only consolidation 의 구조적 결함을 셋으로 정리한다. (1) **추상화 수준이 그대로다**: 추가적인 lossy 압축 없이 같은 표현 수준의 지식을 옮기므로 capacity 를 그만큼 다시 소모한다. (2) **선택적이고 retrieval 의존적이다**: 활발히 recall 되는 기억만 강화된다. (3) **context 에 갇혀 있다**: update 가 현재 context 에서만 유도되므로, 새 지식과 기존 지식의 상위 수준 통합이 일어나지 않는다.

[Sleep] 의 답은 스펙트럼에 축을 하나 더 놓는 것이다 — 수식의 축이 아니라 시간의 축. 인간 기억 연구가 말하는 두 번째 consolidation, 즉 수면중의 offline consolidation 을 LLM 의 lifecycle 에 이식한다. 이 장에서 보겠지만, 그 이식은 [TNT] (TNT: Improving Chunkwise Training for Test-Time Memorization, arXiv:2511.07343) 의 reset 장치와 [NL] 의 chunk 스케줄을 재료

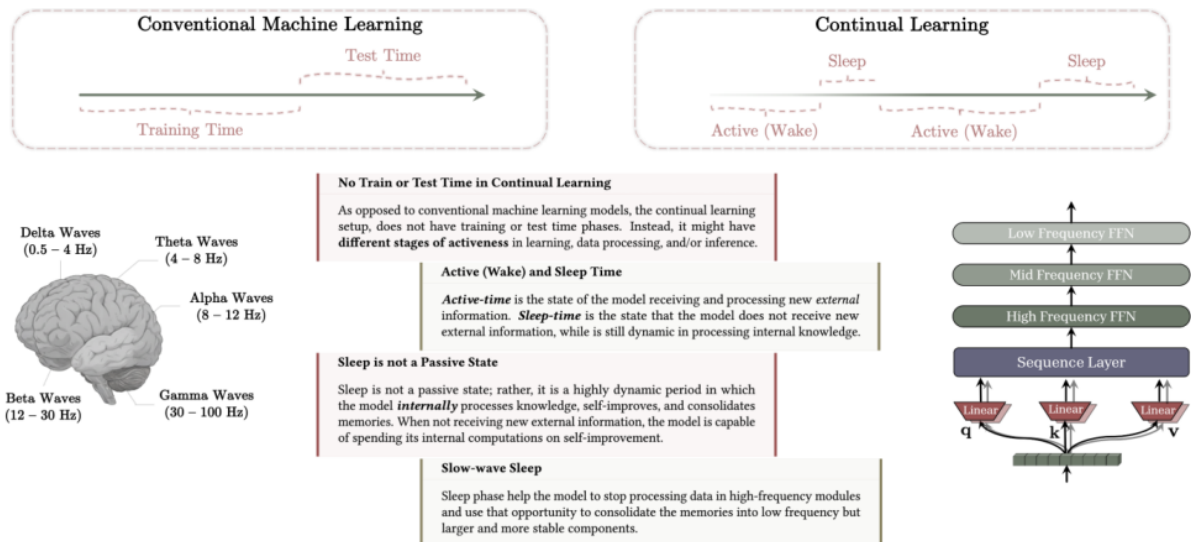


그림 17-1 — [Sleep] 이 그린 lifecycle 재정의. 왼쪽 (Conventional Machine Learning) 은 모델의 수명이 training time 과 test time 으로 갈리지만, 오른쪽 (Continual Learning) 에는 그 구분이 없고 Active(Wake) 와 Sleep 이 주기적으로 교대한다. 원저자는 sleep 을 수동 상태가 아니라, 새 외부 입력을 끊고 fast·고주파 모듈의 기억을 저주파의 더 안정한 (느린·큰) 성분으로 consolidate 하는 내부처리 구간으로 규정한다 (오른쪽 아래는 그 처리를 수행하는 Hope 백본의 Low/Mid/High Frequency FFN 사슬). 출처: Behrouz et al., [Sleep] (arXiv:2606.03979), 원문 Fig.1 — 원저자의 그림 (third-party), 본서의 결과가 아님.

로 쓰되, 라인의 앞 다섯 논문과는 방법론적으로 결이다른 물건이 된다.

17.2 문제의식과 논문의 핵심주장

논문이 스스로 정의한 문제는 배포 후 정적인 (static-after-deployment) LLM 이다 [Sleep § 1]. 지식 갱신의 기존 처방은 딜레마의 두 뿔에 각각 걸린다: re-pretraining 은 효과적이지만 잦은 갱신에는 비용이 불가능한 수준이고, continual fine-tuning 이나 LoRA 류 경량 갱신은 반복 적용 시 catastrophic forgetting 을 부른다 [Sleep § 1]. in-context learning 은 효율적인 continual learning 이지만 context 가 끝나면 지식이 소멸한다. 그래서 질문은 “fragile 한 short-term memory 를 어떻게 stable 한 long-term 지식으로 옮기는가” 가 된다 [Sleep § 1].

핵심 주장은 셋이다. 첫째, **continual learner 에게는 training time 도 test time 도 없다**. 모델의 lifecycle 은 새 입력을 받아 처리하는 **wake(active) phase** 와, 입력을 최소화하거나 끊고 내부 계산으로 기억을 정리하고 자기를 개선하는 **sleep phase** 의 주기적 교대로 재정의되어야 한다 [Sleep § 3.1]; 원논문은 이 재정의를 conventional ML 의 train/test 이분법과 나란히 놓아 대비시킨다 (그림 17-1). 이 책은 이것을 **wake/sleep lifecycle** 이라고 부른다 — 이 라인이 Titans 이후 유지해 온 “test time” 이라는 단어의 마지막 잔재를 지우는 주장이다. 둘째, **CF 는 근본적으로 capacity 문제다**. 파라미터 수가 유한하므로 새 지식을 넣으려면 덮어써야 하고, 그래서 잊는다. 논문은 생물학의 offline consolidation 을 replay (수면 중 최근 pattern 의 재생) 와 neuroplasticity (새 연결의 형성) 의 **결합**으로 읽고, 그 처방으로 replay 기반 Knowledge Seeding 에 점진적 **parameter expansion** 을 함께 쓴다 Sleep § 3.2, § 3.3. 셋째, sleep 은 두 단계다: NREM (slow-wave sleep) 의 hippocampus → neocortex 기억 이전과 synaptic homeostasis 에 대응하는 **Memory Consolidation**, 그리고 REM 의 시냅스 강화·통합·미래 시뮬레이션에 대응하는 **Dreaming** [Sleep § 1, § 3].

[해설] 이 책의 좌표로 옮기면 이렇게 된다. 12–16 장의 논문들은 전부 master update (M) 의 성분을 바꿨다 — [Miras] (It’s All Connected, arXiv:2504.13173; 이 책은

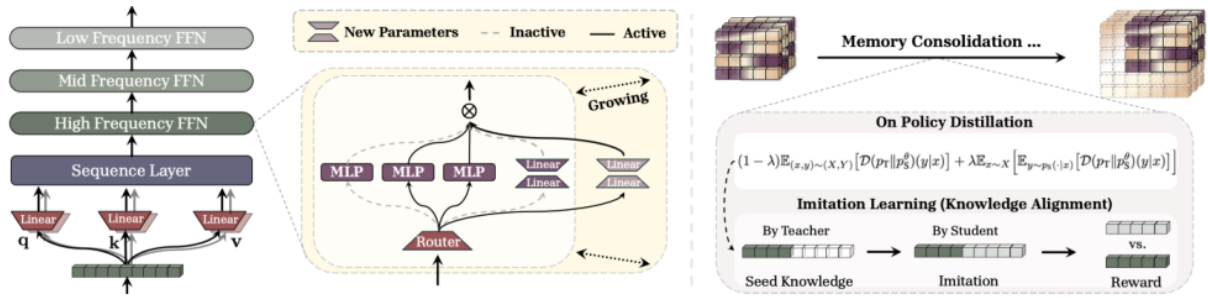


그림 17-2 — Memory Consolidation 개관. 모델은 새 low-rank 파라미터를 활성화해 capacity를 키운 뒤 (왼쪽 · 가운데의 새 MLP/Linear expert), Knowledge Seeding으로 고주파에서 저주파 memory로 지식 추상을 이전한다. 오른쪽은 그 이전을 구현하는 두 항 — teacher가 만든 데이터와 student의 on-policy rollout을 섞는 GKD distillation, 그리고 teacher 행동을 student가 모방하도록 보상을 주는 Imitation Learning — 이다. 출처: Behrouz et al., [Sleep] (arXiv: 2606.03979), 원문 Fig.2 — 원저자의 그림 (third-party), 본서의 결과가 아님.

framework 이름 Miras로 통칭한다)는 objective를, [Atlas] (Atlas: Learning to Optimally Memorize the Context at Test Time, arXiv:2505.23735)는 window와 optimizer를, [TNT]는 훈련 경제학을, [NL]은 층위를 바꿨다. [Sleep]은 (M)을 건드리지 않는다. 바꾸는 것은 그 update들이 살아가는 **lifecycle**이다. inference 어휘로는 1장 Rosetta의 마지막 행이 이장의 전부다: sleep phase는 서버 fleet에 붙는 주기적 백그라운드 job, “weights의 background compaction”이다. 낮에는 요청을 처리하며 fast memory에 쓰고, 밤에는 트래픽을 끊고 compaction·GC를 돌린다 — 단지 그 대상이 로그나 cache가 아니라 모델의 파라미터일 뿐이다.

비슷한 이름의 선행물과의 경계도 논문이 직접 긋는다. sleep-time compute(Lin et al. 2025)는 유휴 시간에 과거 상호작용의 **텍스트요약**을 만들고, Cartridges(Eyuboglu et al. 2025)는 KV 표현을 보조 모델로 압축한다 — 둘 다 token/KV 공간의 압축이다. [Sleep]은 자기생성 데이터를 통한 distillation으로 지식을 **parametric weight 공간**으로 옮긴다는 점에서 다르다고 주장한다 [Sleep § 2.3, App. A.2 – A.3]. 이 구분의 실증은 § 17.6의 Cartridges 비교가 담당한다.

17.3 Core mechanism (통일표기)

이 절은 sleep 한 사이클을 재구현 가능한 수준까지 전개한다. 구조는 기제의 실행 순서를 따른다: wake의 기반 구조 (CMS) → sleep의 발화 시점 → Stage 1 Memory Consolidation(expansion) → Knowledge Seeding → Learning to Imitate → reset) → Stage 2 Dreaming.

Stage 1 한 사이클의 큰 그림이 그림 17-2다: 모델이 먼저 자기 파라미터수를 늘려 capacity를 확보하고 (§ 17.3.3의 expansion), 그다음 Knowledge Seeding으로 지식 추상을 고주파 memory에서 저주파 memory로 옮긴다 — 그 이전 (§ 17.3.4)이 그림 오른쪽의 On-Policy Distillation(식 17-2)과 Imitation Learning(teacher가 seed → student가 imitate → reward로 정렬, § 17.3.5)의 두 항으로 구현된다.

17.3.1 기반 구조: CMS와 식 (M5) — wake가 돌리는것

[Sleep]은 새 sequence layer를 제안하지 않는다. 기반은 [NL]에서 그대로 수입한 CMS다 (정의 소유는 16장; 여기서는 이 장에 필요한 만큼만 환기한다). backbone의 한 블록은 sequence-mixing layer(attention이든 Titans류 memory module이든) 뒤에 MLP블록 사슬 $MLP^{(f_1)}, \dots, MLP^{(f_k)}$ 가 이어지는 구조이고, 각 블록은 자기만의 update frequency f_ℓ 를 가진다 [Sleep § 2.2, Eq. 1]. 블록 ℓ 의 chunk 크기는 $C^{(\ell)} := \max_{\ell'} C^{(\ell')}/f_\ell$ 로 정의되며, 일반성을

않지 않고 $C^{(\ell)}$ 은 $C^{(\ell-1)}$ 로 나누어떨어진다고 가정한다 [Sleep § 2.2]. 블록 ℓ 의 파라미터 $\theta^{(\ell)}$ 은 $C^{(\ell)}$ step 마다 한 번 갱신된다. 통일 표기의 (M5) 그대로다:

$$\theta_{i+1}^{(\ell)} = \theta_i^{(\ell)} - e_{i,\ell}, \quad e_{i,\ell} = \begin{cases} \sum_{t=i-C^{(\ell)}+1}^i \eta_t^{(\ell)} \varepsilon(\theta_t^{(\ell)}; x_t) & i \equiv 0 \pmod{C^{(\ell)}} \\ 0 & \text{otherwise} \end{cases} \quad (17-1)$$

여기서 $\eta_t^{(\ell)}$ 는 level ℓ 의 learning rate 이고, $\varepsilon(\cdot; \cdot)$ 은 임의 optimizer 의 error 항 — gradient descent 라면 task objective 의 gradient $\nabla_{\theta} \mathcal{L}(\theta_t^{(\ell)}; x_t)$ — 이다 [Sleep Eq. 2]. 원문은 이 error 항에 $f(\cdot)$ 기호를 쓰지만 frequency f_{ℓ} 과 충돌하므로 이 책은 ε 으로 개명한다 (표 17-1). 식 (17-1) 이 말하는 것: 각 블록은 token 마다 error 기여분을 누적하고, 자기 chunk 경계에서만 그 누적을 파라미터에 적용한다. chunk 내부에서 $\theta^{(\ell)}$ 이 상수이므로 chunk 전체를 병렬 처리하고 update 한 번을 이루는 chunkwise 구조 ($\rightarrow$ 9 장) 가 level 마다 성립하며, 이것이 [TNT] 가 훈련 경제학으로 다뤘던 바로 그구조다. 해석도 [NL] 의 것을 잇는다: $\theta^{(\ell)}$ 은 자기 chunk 의 context 를 파라미터로 압축한 “그 문맥의 추상적 지식” 이고, 앞쪽 (고주파) 블록은 short-term memory, 뒤쪽 (저주파) 블록은 long-term memory 다 [Sleep § 2.2].

이 구조의 급소가 [Sleep] 의 출발점이다. 다중 frequency 는 CF 를 이루지만, 식 (17-1) 의 갱신 자체가 덮어쓰기다 — 블록의 update 순간, 직전 chunk 가 만든 옛 내용 위에 새 내용이 씌워진다. 그러므로 각 memory 블록의 파라미터가 갱신되기 전에, 그 블록의 지식을 더 안정된 파라미터로 consolidate 하는 기제가 필요하다는 것이 논문의 핵심 관찰이다 [Sleep § 2.2].

17.3.2 sleep 은 언제 오는가: chunk 경계스케줄

sleep 의 발화 시점은 학습되지 않고 chunk 스케줄에 고정된다. chunk 길이목록 $\{C^{(1)}, \dots, C^{(k)}\}$ 가 주어지면, sleep (그리고 memory consolidation) 은 모든 $b \in \mathbb{N}$ 에 대해 step $\{C^{(1)} \times b, \dots, C^{(k)} \times b\}$ 에서만 일어난다 [Sleep § 3.2]. 즉 어떤 블록이든 자기 갱신 경계에 도달하면, 갱신 직전에 그 블록의 지식이 다음 느린 블록으로 먼저 옮겨진다. frequency 가 중첩되어있으므로 consolidation 은 다대일이다: update frequency 1K token 의 블록뒤에 10K token 의 블록이 있으면, 느린 블록이 한 번 갱신되는 동안 빠른블록은 10 번 갱신되고, 따라서 빠른 $\rightarrow$ 느린 consolidation 이 10 번 일어난다 [Sleep § 3.2]. 실험 구성의 chunk/update-period 사다리는 $C= \$1k \rightarrow 5k \rightarrow 10k$ token 이다 [Sleep Fig. 7] — frequency (단위 시간당 갱신 횟수) 는 그 역수라 반대 방향으로 감소한다 (1k 블록이 10k 블록보다 자주 갱신된다). 같은 고정 크기의 느린 memory 에 10 번을 반복해서 써 넣는 것 — 바로 그 지점이 CF 가 일어날 자리이고, 그래서 다음 소절의 expansion 이 필요해진다.

이 다대일 스케줄이 그림 17-3 에 그대로 그려져 있다: High/Mid/Low Frequency FFN 이 각각 갱신 주기 $f_{\text{W}} = \$1k/5k/10k$ token 으로 배열되고, 빠른블록은 Parameter Expansion 을 반복하다가 자기 window 가 만료되는 순간 한단계 느린 FFN 으로 Consolidation 을 흘려보낸다. 빠른 블록이 여러 번 팽창 · 정리되는 동안 느린 블록은 한 번만 consolidation 을 받는 frequency 중첩이 그림에서 눈으로 확인된다.

17.3.3 Stage 1a — parameter expansion: 덮어쓰지 말고 키워라

parameter expansion (periodic parameter (de)activation) 은 consolidation 을 받는 쪽 블록에 새 파라미터를 열어 주는 기제다. 일반성을 잃지 않고 각 $\text{MLP}^{(f_{\ell})}$ 은 router $\mathcal{R}^{(\ell)}$ 을 가진 sparse mixture-of-experts (MoE) 라고 가정한다 (MoE/router $\rightarrow$ 10 장 용어집): 블록 ℓ 은 현재 $s_{\ell} \geq 1$ 개의 expert $\{W^{(f_{\ell}),1}, \dots, W^{(f_{\ell}),s_{\ell}}\}$ 을 가진다 [Sleep § 3.2]. 블록 ℓ^*-1 의 지식을 바로 다음 느린 블록 ℓ^* 로 consolidate 할 때, 전이되는 지식과 $\text{MLP}^{(f_{\ell^*})}$ 에 이미 저장된 지식의 간섭을 피하기 위해 새 low-rank expert 하나를 추가한다: $A^{(\ell^*), s_{\ell^*}+1} \in \mathbb{R}^{d \times d_{\text{low}}}$, $B^{(\ell^*), s_{\ell^*}+1} \in \mathbb{R}^{d_{\text{low}} \times d}$, $d_{\text{low}} \ll d$ 로 파라미터화된 low-rank MLP 다 [Sleep § 3.2]. 전이되는 지식은 오직 이 새 파라미터 ($2 d d_{\text{low}}$ 개) 에만 저장되고, 그 결과 sleep 이 지날 때마다 일부 layer 의 파라미터가 자란다.

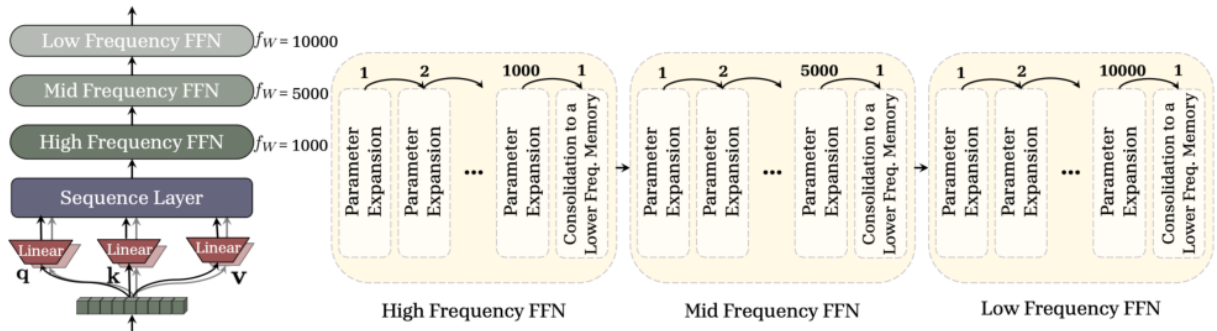


그림 17-3 — Multi-frequency memory hierarchy. 왼쪽은 Sequence Layer 위에 갱신 주기가 다른 FFN 사슬 (High/Mid/Low, $f_W=1k/5k/10k$ token) 이 쌓인 CMS 백본이고, 오른쪽은 각 FFN 이 Parameter Expansion 을 반복하다 window 만료 시 다음저주파 FFN 으로 Consolidation 을 넘기는 스케줄이다 ($1k \rightarrow 5k \rightarrow 10k$). 원문 캡션의 f_W 는 본서 표기의 update 주기 $C^{(\ell)}$ 에 해당한다. 출처: Behrouz et al., [Sleep] (arXiv:2606.03979), 원문 Fig.7 — 원저자의 그림 (third-party), 본서의 결과가 아님.

구현 노트가 infra 독자에게 중요하다 [Sleep § 3.3 “Note on the Implementation”]. tensor 차원을 동적으로 바꾸는 대신, **미래에 열릴 expert 전부를 초기화 시점에 미리 할당해 두고 forward·backward에서 mask** 한다. sleep에서 “expert 를 추가한다”는 것은 mask 를 여는 것이다. shape 가 정적이므로 컴파일된 그래프·kernel autotuning·checkpoint 포맷이 흔들리지 않는다. 논문은 이것을 뇌의 고정 capacity 와 “새 연결 형성에 의한 뉴런 활성화” 에 대응시킨다.

17.3.4 Stage 1b — Knowledge Seeding: 위로 가는 distillation

Knowledge Seeding(KS) 은 이 논문이 이름 붙인 새로운 knowledge transfer 의 방향이다: 하나 이상의 작은 모델 (teacher) 의 지식을 큰 모델 (student) 로 옮기는 distillation — 통상의 KD($\rightarrow$ 11 장) 와 반대 방향이라 논문은 이를 upward distillation 이라고도 부른다 [Sleep § 3.3]. **Self-Knowledge Seeding(SKS)** 은 teacher 와 student 가 같은 모델의 두 버전인 특수 사례다. teacher LM_θ 는 **expansion** 전의 모델 상태이고, student $LM_{\theta_{\text{exp}}}$ 는 (i) parameter expansion 과 (ii) 식 (17-1) 에 따른 $\theta^{(\ell^*-1)}$ 의 갱신, 둘 다 거친 후의 상태다 [Sleep § 3.3]. 이 정의가 기체의 요체이므로 풀어 쓴다: teacher 는 빠른 블록이 아직 덮여지지 **전**의 지식을 들고 있고, student 는 빠른 블록이 이미 새 chunk 로 넘어간 대신 느린 블록에 빈 expert 하나를 새로 받았다. distillation 은 student 의 출력 분포를 teacher 쪽으로 당기는데, student 에서 움직일 수 있는 파라미터는 새 expert 뿐이다. 따라서 최적화가 성공하면, 빠른 블록에서 사라진 지식이 느린 블록의 새 expert 안에 재구성된다 — 이것이 “갱신 전에 consolidate 한다” 의 정확한 수학적 형태다.

이 distillation 에는 통상의 KD 와 다른 두 난점이 있다 [Sleep § 3.3]. (1) student 가 teacher 보다 capacity 가 **크다**. teacher 가 미리 생성해 둔 고정 데이터셋으로 student 를 supervised 학습 (sequence-level KD, Kim & Rush 2016) 시키면 student 파라미터가 과소 활용된다. (2) 모델은 잠들어 있다 — 외부 데이터셋이 없으므로, 보유 corpus 위에서 teacher logits 을 맞추는 고전적 Hinton 식 KD 를 쓸 수 없다. 해법은 Generalized Knowledge Distillation(GKD, Agarwal et al. 2024; $\rightarrow$ 11 장) 이다: teacher 가 생성한 데이터와 student 가 스스로 생성한 on-policy 데이터를 섞는다. 먼저 teacher LM_θ 에서 sampling 해 dataset $\mathcal{D}$ 를 만들고, on-policy distillation objective 를 세운다 [Sleep § 3.3]:

$$\mathcal{L}_{\text{GKD}}(\theta, \theta_{\text{exp}}) = (1 - \lambda_{\text{on}}) \mathbb{E}_{(x,y) \sim \mathcal{D}} [\mathcal{F}(LM_\theta \| LM_{\theta_{\text{exp}}})(y|x)] + \lambda_{\text{on}} \mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim LM_{\theta_{\text{exp}}}(\cdot|x)} [\mathcal{F}(LM_\theta \| LM_{\theta_{\text{exp}}})(y|x)] \quad (17-2)$$

$\mathcal{F}$ 는 teacher 와 student 의 token 출력 분포 사이 divergence 이고 (구체적 선택 — forward/reverse KL 등 — 은 GKD 의 자유도다, $\rightarrow$ 11 장), $\lambda_{\text{on}} \in [0, 1]$ 은 student 가 자기 생성 rollout

위에서 teacher logits 의 token 단위 feedback 을 받는 on-policy 비중을 정한다 (원문 기호 λ ; weight decay 와의 충돌을 피해 개명, 표 17-1). 두 번째 항이 난점 (1) 의 답이다: student 는 자기 가 생성한 sequence 위에서 교정을 받으므로, teacher 의 고정샘플 분포에 갇히지 않는다.

여기에 의도적 제약이 둘 붙는다 [Sleep § 3.3]. 첫째, **student 의 sampling 분포를 통해서 backpropagate 하지 않는다** — 안정성과 속도를 위한 선택이다 (sampling 이라는 이산 연산을 미분하는 REINFORCE 류 추정치의 분산을 피한다; → 11 장). 둘째, **student 의 모든 파라미터를 동결하고 새로 확장된 expert 만 학습한다**. 전이된 지식이 옛 지식을 물리적으로 덮어쓸 수 없으므로, CF 방지가 regularization 이 아니라 구조로 보장된다. 12–16 장의 어휘로 말하면, retention gate(→ 13 장) 가 “얼마나 지울지” 를 배우는 soft 한 장치였다면 이것은 “지울 수 없게 만드는” hard 한 장치다.

17.3.5 Stage 1c — Learning to Imitate: 아는 것과 쓰는 것은 다르다

distillation 만으로는 부족하다는 것이 논문의 경험적 관찰이다: student 가 지식에 접근할 수 있게 되었음에도 그것을 쓰는 법은 배우지 못해서, teacher 의 sampling 행동과 성능을 약하게만 모방한다 [Sleep § 3.3]. **Learning to Imitate(LTI)** 는 이를 교정하는 RL 단계다. 7 장의 LTI(linear time-invariant) 와 무관한 약어다. teacher 가 생성한 데이터 $\mathcal{D}_T = \{d^{(1)}, \dots, d^{(n)}\}$ 에서 각 $d^{(i)}$ 의 random prefix 를 뽑아 student 에게 이어 쓰게 하고, student 의 완성 $\hat{d}^{(i)}$ 에 보상을 준다 [Sleep Eq. 3]:

$$r(\hat{d}^{(i)}; d^{(i)}) = \rho r_{\text{sem}}(\hat{d}^{(i)}; d^{(i)}) + (1 - \rho) r_{\text{abs}}(\hat{d}^{(i)}; d^{(i)}) \quad (17-3)$$

혼합 계수 ρ (원문 γ ; window gate 와의 충돌을 피해 개명) 가 두 보상을 섞는다. semantic 보상 r_{sem} 은 **frozen reward model** 이 $\hat{d}^{(i)}$ 와 $d^{(i)}$ 의 의미가 같으면 1, 다르면 0 을 주는 이진 신호다. absolute 보상 r_{abs} 는 Levenshtein distance $z(\cdot, \cdot)$ 기반의 token 수준 유사도다 [Sleep Eq. 4]:

$$r_{\text{abs}} = \begin{cases} 1 - \frac{z(\hat{d}^{(i)}, d^{(i)})}{\max\{|\hat{d}^{(i)}|, |d^{(i)}|\}} & z(\hat{d}^{(i)}, d^{(i)}) \leq z_0, \\ 0 & \text{otherwise,} \end{cases} \quad (17-4)$$

z_0 는 유사도 threshold 다. LTI 를 식 (17-2) 의 on-policy distillation 과 결합한 최종 **Knowledge Seeding objective** 는 확장 파라미터에 대해 다음을 최대화한다 [Sleep § 3.3]:

$$\mathcal{L}_{\text{KS}}(\theta, \theta_{\text{exp}}) = \mathbb{E}_{x \sim \mathcal{D}} \left[(1 - \lambda_{\text{KD}}) \mathbb{E}_{y \sim \text{LM}_{\theta_{\text{exp}}}(\cdot|x)} [r(y)] - \lambda_{\text{KD}} \mathbb{E}_{y \sim \text{LM}_{\theta_{\text{exp}}}(\cdot|x)} [\mathcal{F}(\text{LM}_{\theta} \parallel \text{LM}_{\theta_{\text{exp}}})(y|x)] \right] \quad (17-5)$$

$\lambda_{\text{KD}} \in [0, 1]$ (원문 α ; retention gate 와의 충돌을 피해 개명) 이 distillation 강도 (divergence 별점) 와 LTI 보상 사이를 조율한다. consolidation 이 끝나면 마지막 동작이 온다: **빠른 블록 MLP(f_{ℓ^*-1}) 에 과거 sleep 들에서 추가되었던 low-rank expert 전부를 reset** 해 그 capacity 를 미래의 wake 를 위해 되돌린다 [Sleep § 3.3]. 이 책은 이를 **synaptic-pruning reset** 이라 부른다 — 뇌가 불필요 · 중복 연결을 쳐내는 synaptic pruning 의 대응물이라는 논문의 해석을 따른 명명이다. 순 효과는 하나의 불변식이다: 빠른 memory 는 작고 plastic 하게 유지되고, 느린 memory 는 (미리 할당된 한도 안에서) 단조 성장하며 오직 신선한 low-rank expert 를 통해서만 쓰인다.

[해설] reset 의 계보를 짚어 둘 가치가 있다. [TNT] 의 periodic reset(→ 15 장) 은 local memory 를 W_{init} 으로 되돌려 sequential chain 을 끊는 **throughput 장치**였다. [Sleep] 의 reset 은 “지식을 위로 옮긴 다음에만 지운다” 는 **memory-hygiene 원리**다. 같은 연산이 훈련 경제학의 도구에서 기억 체계의 규율로 승격되었다. inference 어휘로: eviction 전에 반드시 상위 계층으로 write-back 하는 cache 정책이다.

17.3.6 Stage 2 — Dreaming: 자기 생성 데이터에 의한 self-modification

Dreaming 은 REM 대응 단계로, 모델이 자기 성능을 올려 줄 합성데이터 (dream) 를 스스로 생성하는 법을 배우는 self-modification 과정이다 [Sleep § 3.4]. 자리 자체는 플러그인이다 — 어떤 self-improvement 용 합성 데이터 기법도 이 단계에 꽃을 수 있다. proof of concept 는 SEAL(Zweiger et al. 2025) 위에 세워졌다. SEAL 은 모델이 “self-edit” (자기 갱신용 합성 텍스트) 를 생성하고, 그것으로 자신을 fine-tune 한 결과가 좋으면 그 생성을 RL 로 강화하는 loop 다. 논문은 continual 설정에서 SEAL 의 문제 셋을 지적한다 [Sleep § 3.4]: (1) inner-loop 의 SFT 비용 때문에 self-edit 수가 적다. (2) sleep 주기마다 반복되는 self-improvement 는 CF 를 부른다. (3) sampling 이 모델의 기존 지식 분포 안에서만 이루어진다 — dreaming 의 핵심역할은 기억들의 새로운 합성의 탐색인데도.

절차는 다섯 단계다. task (x_{ctx}, τ) — task 관련 정보를 담은 context x_{ctx} (원문 기호 C ; chunk 크기와의 충돌을피해 개명) 와 downstream 성능 측정 $\tau(\cdot)$ — 가 주어졌을 때:

1. **생성**: x_{ctx} 를 context 로 $m \geq 1$ 개의 dream 을 뽑는다: $\{\text{DREAM}^{(i)}\}_{i=1}^m \sim \text{LM}_\theta(\cdot | x_{\text{ctx}})$. 이 때 sampling 중 모든 **MoE router** 가 정규 **dispatch** 에 더해 **random expert** 하나를 추가로 **활성화**한다 — 무관한 지식을 의도적으로 섞어, 모델의 시야에 숨어 있던 기억 간 패턴을 표면화하는 novelty 주입이다 [Sleep § 3.4]. SEAL 문제 (3) 의 답이 이 randomized routing 이다.
2. **선별**: gradient 기반 data selection(G-DIG·GREATS 계열 문헌에서 차용) 으로 대부분의 dream 을 기각한다. 각 dream 의 중요도점수 $\omega^{(i)}$ 는 language-modeling objective 의 gradient $g_{\text{DR}}^{(i)} = \nabla_\theta \mathcal{L}_{\text{SFT}}(\text{DREAM}^{(i)}, \theta)$ 에서 얻고, 점수 상위 Top- k 에 다양성을 위한 random 샘플 n_{rand} 개 (원문 기호 b ; sleep 스케줄 배수와의 충돌을피해 개명) 를 더해 선별 집합 D 를 만든다 [Sleep § 3.4]. 원문은 점수를 “objective 의 gradient” 라고만 정의하고 스칼라화 방법 (norm 등) 을 명시하지 않는다.
3. **적용**: 각 $\text{DREAM}^{(i)} \in D$ 마다 모델의 **isolated instance** 를 LoRA($\rightarrow$ 11 장) 로 fine-tune 한다: $\theta'^{(i)} \leftarrow \text{SFT}(\theta^{(i)}, \text{DREAM}^{(i)})$.
4. **보상**: dream 의 생성에 보상을 준다 — fine-tune 된 $\text{LM}_{\theta'^{(i)}}$ 가 τ 에서 $\text{LM}_{\theta^{(i)}}$ 보다 개선되면 1, 아니면 0 [Sleep Eq. 5].
5. **강화**: 이 보상으로 dream 생성 policy 를 ReST[^]EM(Singh et al. 2024) 으로 최적화한다 — 생성 $\rightarrow$ 이진 보상으로 필터 $\rightarrow$ 생존 샘플에 SFT $\rightarrow$ 반복하는, value network 가 필요 없는 EM 계열 RL 이다 (SEAL 과 동일한선택; $\rightarrow$ 11 장).

두 stage 의 순서가 설계의 논점이다. consolidation 이 먼저 와서, 갓 습득된 fragile 한 지식을 새로 확장된 저주파 파라미터에 격리해둔다 (전체 파라미터 동결은 KS 최적화에만 적용된다). 그러면 dreaming 의반복적 self-training 이 가하는 weight update(dreaming 은 별도 LoRA SFT 로모델을 바꾼다; SEAL 문제 (2), § 17.8 의 OPSD 계열 collapse-forgetting) 에 의한 그 지식의 forgetting 위험이 줄어든다 — 논문은 침식이 구조적으로 불가능하다고보장하지 않고, 두 단계 순서가 더 robust 하다는 가설 · 실험결과를 제시한다 [Sleep § 3.4, App. A.4].

17.3.7 한 번의 sleep step: 전체절차

블록 ℓ^*-1 의 chunk 경계 $i \equiv 0 \pmod{C^{(\ell^*-1)}}$ 에서, 재구현 관점의 전체 절차는다음과 같다.

1. teacher 스냅샷 확보: 현재 상태를 LM_θ 로 고정한다.
2. expansion: 블록 ℓ^* 에 masked pool 로부터 새 low-rank expert $\{A, B\}$ 를 활성화한다.
3. 빠른 블록 갱신: $\theta^{(\ell^*-1)}$ 에 식 (17-1) 의 누적 update 를적용한다. 이 시점의 모델이 student $\text{LM}_{\theta_{\text{exp}}}$ 다.
4. corpus 생성: teacher 에서 sampling 해 $\mathcal{D}$ (distillation 용) 와 $\mathcal{D}_T$ (LTI 용 dream) 를 만든다.
5. Knowledge Seeding: 새 expert 만 학습 대상으로 식 (17-5) 를 최적화한다 —GKD 항은 식 (17-2), LTI 항은 식 (17-3) – (17-4).

6. synaptic-pruning reset: 블록 ℓ^*-1 에 과거 sleep 들이 추가했던 expert 들을 reset 한다.
7. Dreaming: task (x_{ctx}, τ) 가 주어져 있으면 § 17.3.6 의 5 단계 loop 를 돌린다.
8. wake 재개: 갱신된 모델이 다음 chunk 의 입력을 받는다.

17.3.8 표기 대응표

표 17-1 — [Sleep] 원 표기와 이 책의 통일 표기 대응 (상단은 전역확정분, 하단은 장-국소 추가분).

Sleep 원 표기	통일 표기	의미 / 주의
$\theta^{(f_\ell)}, C^{(\ell)}, f_W$	$\theta^{(\ell)}, C^{(\ell)}, f_\ell$	(M5)
$e_{i,\ell}$ — 누적 error 항	$e_{i,\ell}$	동일
L/T — sequence 길이 혼용	L	
LM_θ (teacher), $\text{LM}_{\theta_{\text{exp}}}$ (student)	동일	teacher = 확장 전 자기 자신
λ — GKD on-policy 비율	λ_{on}	$\triangleq$ weight decay λ 와 구분
γ — reward 혼합 계수 (Eq. 3)	ρ	$\triangleq$ window gate γ 와 충돌 방지
α — distill-vs-reward 계수 ($\mathcal{L}_{KS}$)	λ_{KD}	$\triangleq$ retention gate α_t 와 충돌 방지
$\mathcal{F}, \mathcal{D}$ — divergence	동일	
$r_{\text{sem}}, r_{\text{abs}}; z(\cdot, \cdot), z_0$ — 보상 · Levenshtein	동일	
$\mathbf{A}, \mathbf{B}, d_{\text{low}}$ — low-rank expert	$A^{(\ell),j}, B^{(\ell),j}, d_{\text{low}}$	
s_ℓ — expert 수	동일	
$\mathcal{R}^{(f_\ell)}$ — MoE router	$\mathcal{R}^{(\ell)}$	
$\omega^{(i)}, g_{DR}^{(i)}$ — dream 중요도 · gradient	장-국소 유지	
$\text{DREAM}^{(i)}, \text{KS/SKS/LTI}, \text{ReST}^\wedge \text{EM}, \text{SEAL}$	고유명사 유지	
$f(\cdot; \cdot)$ — Eq. 2 의 optimizer error 항	$\varepsilon(\cdot; \cdot)$	$\triangleq$ frequency f_ℓ 과 충돌 방지 (장-국소 개명, (M5))
C — Dreaming 의 task context ((C, τ) 의 C)	x_{ctx}	$\triangleq$ chunk 크기 C 와 충돌 방지 (장-국소 개명)
$\mathcal{D}(\ \cdot\)$ — $\mathcal{L}_{KS}$ 안의 divergence	$\mathcal{F}(\ \cdot\)$	$\triangleq$ dataset $\mathcal{D}$ 와 충돌방지 (장-국소 개명)
b — Top- k 외 추가 random dream 수	n_{rand}	$\triangleq$ sleep 스케줄 배수 $b \in \mathbb{N}$ [Sleep § 3.2] 와 구분
Eq. 2 합의 하한 $t = i - C^{(\ell)}$	$t = i - C^{(\ell)} + 1$	원전 Eq. 2 의 off-by-one 교정 (같은 상한 $i-1$ -based 에서 두합은 한 항 차이로 수학적으로 동일하지 않다; 원전은 $C^{(\ell)}+1$ 개항을 더해 첫 경계 $i=C^{(\ell)}$ 에서 없는 x_0 를 포함). 이 책은 (M5) 기준 장-국소
$\theta'^{(i)}$ — dream i 로 fine-tune 된 instance	동일	
m — dream 생성 수; Top- k 의 k	동일	장-국소 (k 는 보조인덱스 용법)

17.4 Outer-loop training vs inner-loop test-time learning — 그리고 세 번째 regime

이 절이 이 장에서 가장 주의 깊게 읽어야 할 절이다. [Sleep] 은라인에서 유일하게 outer/inner 이 분법 자체를 거부하는 논문이다 — “continual learner 에게 train/test 는 없다” 가 곧 논문의 제 1 주장이기 때문이다. 그러나 이 책의 좌표계는 여전히 유효하며, 오히려 이 논문에서 좌표계를 대야만 보이는 사실이 있다: [Sleep] 에는 최적화 regime 이 둘이 아니라 셋 있고, 그중 셋째는 앞 다섯 논문 어디에도 없던 종류다.

표 17-2 — [Sleep] 의 세 최적화 regime.

	regime 1: backbone pre-training	regime 2: wake (배 포 중, 입력 흐름)	regime 3: sleep (배 포 중, 입력 차단)
이 책의 좌표	outer loop	inner loop	어느 쪽도 아님 — 배포 중의 offline 훈련 job
움직이는 것	Θ 전체 (projection, backbone, CMS 초 기 상태)	$\theta^{(\ell)}$ (각 level 의 파라 미터), sequence layer 의 state	consolidation: 새 expert $\{A, B\}$ 만. dreaming: LoRA adapter + dream 생성 policy
갱신 규칙	AdamW 류 표준 훈련 (→ 2 장)	식 (17-1): error 누적 + chunk 경계 적용	식 (17-5) 최적화; LoRA SFT; ReST ^{EM}
데이터	pre-training corpus	들어오는 context 그 자체	consolidation corpus 는 자기 생성 (teacher sampling, rollout); dream 은 외 부/wake 보존 task context x_{ctx} 와 평가 함수 τ 에 조건화
cadence	배포 전 1 회	token 마다 누적, $C^{(\ell)}$ 마다 write	chunk 경계 $\{C^{(\ell)} \times b\}$ 마다

regime 1 — 무엇이 meta-learn 되는가. [Sleep] 자체는 backbone 훈련을 새로 유도하지 않고 상속한다. Hope 를 [NL] 레시피로 pre-train 하는 구성이라면 [NL] 의 훈련을 그대로 쓰게 된다: gradient 가식 (17-1) 의 다중 frequency 갱신을 **관통**해 흐르고, 따라서 outer 최적화는 “각 level 의 chunk 단위 자기 갱신이 context 를 유용하게 압축하도록” 블록들을 조형한다 — TTT·Titans 의 meta-learning-through-inner-loop 구조 (→ 4 장, 12 장) 를 frequency 사슬전체로 일반화한 것이다. 병렬화의 고리도 같다: chunk 안에서 $\theta^{(\ell)}$ 이 상수이므로 chunk 전체가 한 번의 지연 update 로 배치처리되고 (→ 9 장의 stale-snapshot 근사), 이 chunkwise 훈련을 경제적으로만 드는 것이 바로 [TNT] 의 주제였다 (→ 15 장). 반면 Llama·Qwen 위의 graft 실험이라면 regime 1 은 그냥 기성 checkpoint 다 — 아래 caveat 참조.

regime 2 — wake 에서 무엇이 어떤 규칙으로 움직이는가. 배포된 모델의 상태는 두 겹이다. sequence layer 는 자기 관행대로 state 를 유지한다 (attention 이면 KV cache, Titans 류 module 이면 고정 크기 W_t). 그 위에서 모든 CMS 블록 ℓ 이 token 마다 error 기여 $\eta_t^{(\ell)} \varepsilon(\theta_t^{(\ell)}; x_t)$ 를 누적하고, $C^{(\ell)}$ token 마다 한 번 자기 weights 에 적용한다. per-token 비용은 블록마다 (파라미터 크기의 누적 1 회/token) + (파라미터 write 1 회/ $C^{(\ell)}$ token) 이고, error 항 계산을 위한 backward 류 연산이 필요하다. state 크기는 “각 블록의 파라미터 + 같은 크기의 accumulator” 다. chunk/update-period 사다리 가 \$C = \\$1k \rightarrow 5k \rightarrow 10k\$ token 이므로 [Sleep Fig. 7] 느린 (주기 큰) 블록일수록 weights 를 건드리는 빈도 (frequency) 는 급감한다.

regime 3 — sleep에서 무엇이 학습되는가. 여기가 신세계다. consolidation은 진짜 gradient 기반 훈련이지만, 배포 전에 한번이 아니라 배포된 삶의 주기적 일부로 돈다. 학습 대상은 새로활성화된 low-rank expert 하나뿐이고, 나머지 전부는 동결이며, 데이터는 전부 자기 생성이다. dreaming은 per-dream LoRA SFT(안쪽)와 ReST[^]EM policy 최적화(바깥쪽)의 이중 구조이고, 선별 단계는 후보 dream 마다 backward pass 한 번 ($g_{DR}^{(i)}$ 계산)을 요구한다. 보고된 공통 설정은 LR 5×10^{-6} , effective batch 32, Sleep 100 step(GRPO baseline은 500; GRPO = group-relative policy optimization — 그룹 내 상대 보상으로 baseline을 대신하는 RLVR 계열 방법), LoRA rank 64/alpha 128 이다 [Sleep Table 5].

그렇다면 “이 gate는 누가 학습하는가?”라는 이 책의 표준 질문에 대한 답은 이 논문에서 이례적이다:

표 17-3 — [Sleep]의 knob 지배 구조: 누가 정하는가.

knob	정체	누가 정하는가
W_K, W_V, W_Q , backbone	slow weights Θ	regime 1 (또는 기성 checkpoint)
$C^{(\ell)}$, f_ℓ , level 수	frequency 사다리	사람이 hand-set (예: 1k/5k/10k)
$\eta_t^{(\ell)}$	level 별 learning rate	Hope 계열이면 regime 1에서 조형; graft에서는 명시 없음
$\lambda_{on}, \lambda_{KD}, \rho, z_0$	KS/LTI의 혼합 계수 · threshold	사람이 hand-set — 알고리즘의 hyperparameter
r_{sem} 의 reward model	외부 동결 모델	학습되지 않음 (frozen)
Top- k , n_{rand} , m , d_{low} dream 생성 policy	dreaming-expansion 예산 모델 자신	사람이 hand-set regime 3에서 ReST [^] EM으로 학습
sleep 시점	chunk 경계 $\{C^{(\ell)} \times b\}$	학습되지 않음 — 스케줄에 고정

이 표가 드러내는 사실이 이 장의 의무 caveat다. **[Sleep]의 sleep 기제** — $\lambda_{on}, \lambda_{KD}, \rho, z_0$, **reward model, ReST[^]EM loop** — 는 어느 것도 **pre-training**에서 **end-to-end**로 **meta-learn**되지 않는다. 실험의 대부분은 pre-trained Llama/Qwen checkpoint 위에 sleep 기제를 **graft**한 것이다 [Sleep § 4, App. B]. 이 라인의 정체성은 지금까지 “update rule 자체를 outer loop가 미분해 학습한다” 였다 — [Titans] (Titans: Learning to Memorize at Test Time, arXiv:2501.00663)의 $\eta_t, \beta_t, \alpha_t$ 는 Θ 가 산출하는 token의 함수였고, [Atlas]의 window gate도, [NL]의 self-modifying 목표 생성도 그랬다. [Sleep]은 라인 최초로 core mechanism이 **미분되어 통과되는 inner loop가 아니라 알고리즘적 wrapper**인 논문이다.

확인된 사실 하나를 못박아 둔다: [Sleep]의 Hope 계열 실험은 NL 레시피로 pre-train한 Hope checkpoint가 아니라 pre-trained Llama-3B/8B에 5개의 dim-64 MLP memory 블록을 얹은 graft 구성이다 [Sleep § 4.1, App. B]. 따라서 regime 1 — 식 (17-1)을 관통하는 sleep-aware pre-training — 은 이 논문의 실험 어디에서도 실행되지 않았다; § 17.4의 regime 1은 논문이 정의한 가능성이지 실증된 경로가 아니다.

[평가] 이것은 결함이라기보다 방법론적 이탈이며, 정직하게 표시되어야 할 라인의 이음새 (seam)다. 이 이탈에는 대칭적인 두 독법이 있다. 하나: lifecycle 수준의 결정 (언제 자고, 얼마나 키우고, 무엇을 꿈꿀지)은 token 수준 gate와 달리 미분 가능한 형태로 만들기 어려우므로, wrapper는 불가피한 첫걸음이다. 둘: [NL]의 논리를 그대로 밀면 sleep의 knob들 역시 “더 느린 level의 학습 대상”이어야 하는데 (frequency 스펙트럼에서 sleep 스케줄보다 느린 것은 없다), 논문은 그 일반화를 시도하지 않았다. 어느 쪽이든, “wake/sleep까지 포함해 전부를 nested optimization으로 훈련한다”는 자리는 비어 있고, 이는 § 17.8의 open problem으로 넘어간다.

training 무경험 독자를 위한 마지막 정지 지점: 식 (17-5)의 기대값이 student의 sampling에 의존하는데 “sampling 분포를 통해 backpropagate하지 않는다”는 것은, y 를 뽑는 행위 자체

는 상수 취급하고 뽑힌 y 위에서의 divergence·보상만 미분한다는 뜻이다. 편향이 생기지만 분산 폭발을 피한다 — GKD에서 상속한 표준 관행이다 (→ 11 장).

17.5 Concept ledger delta

표 17-4 — [Sleep] 이 ledger에 가한 변경 (누적 관점).

구분	개념	내용
신규	wake/sleep lifecycle	train/test 구분의 대체물. wake = 입력수신·처리, sleep = 입력 차단·내부 처리 [Sleep § 3.1]. 이 장이 정의 소유
신규	offline consolidation (기법)	sleep phase에서 자기 생성 데이터로 수행하는 fast → slow 지식 이전. lossy 추상화 + capacity 확장이 online 판과의 차별점 [Sleep § 3.1] (개념 축은 → 11 장, online 판은 → 16 장)
신규	Knowledge Seeding (KS) / Self-KS	작은모델→큰 모델의 upward distillation; SKS는 teacher=확장 전 자기 자신 [Sleep § 3.3]
신규	Learning to Imitate (LTI)	prefix 완성 + $\rho r_{\text{sem}} + (1 - \rho) r_{\text{abs}}$ 보상의 RL로 “저장한 지식을 쓰는 법”을 가르침 [Sleep Eq. 3-4]
신규	parameter expansion (periodic (de)activation)	sleep마다 low-rank expert 활성화; masked pre-allocation으로 정적 shape 유지 [Sleep § 3.2-3.3]
신규	Dreaming + gradient 기반 dream 선별 + random-expert novelty	자기 생성 데이터에 의한 self-modification; g_{DR} 선별; router의 random expert 주입 [Sleep § 3.4]
신규	synaptic-pruning reset	consolidation 완료 후 빠른 블록의 과거 expert 일괄 reset [Sleep § 3.3]
확장	retention 계보의 종점	retention gate(→ 13 장) → window gate(→ 14 장) → periodic reset(→ 15 장) → “덜어쓰지 말고 키워라” : CF를 capacity 문제로 재정의하고 구조적 성장으로 답함 — [Atlas]의 formal capacity(→ 14 장) 논의가 lifecycle 처방으로 전화

구분	개념	내용
확장	reset 의 의미	[TNT] 의 reset-to- W_{init} (throughput 장치) → consolidation 후 reset(memory-hygiene 원리)
확장	self-modification	[NL] 의 weights-by-rule(자 기 생성 target , → 16 장) → weights-by-self-generated- data (dream)
확장	meta-learned 초기 상태의 계보	W_{init} (→ 15 장) 의역할 — “돌 아갈 좋은 상태” — 를 teacher(확장 전 자기 자신) 가 이어받음
재사용	CMS, update frequency f_ℓ , $C^{(\ell)}$	[NL] 에서축자 수입 (정의 소유 → 16 장)
폐기	“test time”	라인 명칭 (test-time memorization) 에 남아 있던 마지막 잔재를 lifecycle 재정 의로 소거
폐기 (암묵)	inner/outer 이분법	세 regime(wake 갱신/ sleep consolidation / sleep dreaming) 으로대체 — 단 이 책은 분석 좌표로 이분법을 유지 (§ 17.4)

17.6 실험과스케일

backbone은 전부 1B–8B의 기성 모델이다: class-incremental에 Llama-3B·Llama3-8B, ARC에 Llama-3.2-1B, 수학 추론에 Qwen3-1.7B·Qwen3-8B, 그리고 Hope 계열 변형 [Sleep § 4]. 추가 파라미터는 dimension 64의 MLP block 5개이고 active 파라미터 수는 base와 같게 유지되며, 공통 훈련 설정은 § 17.4에 적은 대로다 (LR 5×10^{-6} ·batch 32·Sleep 100 step·LoRA $r=64/\alpha 128$) [Sleep App. B, Table 5].

(1) **class-incremental learning**. CLINC150(150 intent/10 domain/23.7K query), Banking77(77 intent, 13,083 example), DBpedia level-2(70 class, 10K train/1K test)에서 memory consolidation을 얹은 Hope가 ICL·EWC·InCA·순정 Hope를 모두상회한다 [Sleep § 4.1, Fig. 3; 수치는 그림으로만]. 논문의 해석: ICL 대비 이득은 prompt 적응을 지속적 parametric memory로 바꾼 데서, 순정 Hope 대비 이득은 명시적 self-distillation이 반복 in-context 갱신보다 나은 추상을 만든 데서 온다.

(2) **sleep level 수의 효과**. MK-NIAH(RULER), LongHealth(환자 기록 20 건, 각 약 5.1K–6.8K 단어, 200 문항), QASPER(논문 약 1.6K 건 위 약 5K 문항)에서 Hope 변형이 ICL·DuoAttention·Cartridges를 상회하고, 두경향이 일관된다 [Sleep § 4.1, Fig. 4]: consolidation 단계 수가 늘수록 성능이 단조 개선되고, 가장 저주파인 memory의 frequency를 올리면(즉 가장 persistent한 기억을 더 adaptive하게 만들면) retention이 약해져 성능이 떨어진다.

(3) **Continual Translation of a Novel Language**. MTOB(Kalamang)+Manchu 조합의 순차 학습에서 ICL은 사전학습 수준으로붕괴하는 반면, Hope-1/2/3(consolidation 단계수 순)은 이득을 유지하고 단계수에 단조 개선되며, Hope-3은 단일 언어성능을 거의 회복한다 [Sleep § 4.1, Fig. 5]. Cartridges와 SFT는 최소한 언어에서 catastrophic forgetting으로 ICL보다도 나빠져 그림 범위밖으로 밀려났다 [Sleep § 4.1].

(4) **BABILong**. Hope(Sleep) 는 10M token 까지 거의 만점을유지한다고 보고된다 [Sleep § 4.1, Fig. 6]. GPT-4·GPT-4o-mini 는 128K – 256K 너머에서 무너지고, Llama-8B+RAG 는 길이에 따라 저하되며, fine-tune 된 Titans·ARMT·RMT 는 약 1M 까지는 Hope 와비등하다가 그 뒤 급락한다 [Sleep § B.2]. 정직성 주의 돌: 소형 모델비교군은 **전부 BABILong 공식 protocol 로 fine-tune** 된 상태이고, fine-tuning 없이는 Hope 를 포함한 모든 소형 모델이 큰 폭으로 하락한다고 논문 스스로 밝힌다 [Sleep § B.2]. 그리고 이 결과 역시 수치 표 없이그림으로만 제시된다.

(5) **수학 추론**. avg@16 기준 전 수치를 표로 옮긴다 (원문 대조 검증완료).

표 17-5 — 수학 추론, avg@16 [Sleep Table 2]. 굵은 글씨는 열 최고.

Method	AIME-24	AIME-25	HMMT-25
Qwen3-1.7B			
Base (Instruct)	49.8	34.5	25.7
SFT	47.3	36.1	22.9
GRPO	51.0	38.6	26.1
OPSD	51.6	40.0	28.1
Sleep	53.2	40.2	29.3
Qwen3-8B			
Base (Instruct)	73.8	68.1	42.4
SFT	75.5	66.4	43.7
GRPO	76.4	68.1	44.9
OPSD	76.6	67.4	45.1
Sleep	79.2	69.0	46.1

Qwen3-8B 에서 Sleep 79.2 vs GRPO 76.4(AIME-24) 가 헤드라인이다. ablation(Qwen3-8B)은 [Sleep Table 1]: 완전체 79.2/69.0/46.1에서 Imitation Learning 제거 시 76.8/67.9/45.0, Semantic Reward 제거 시 78.9/**69.2**/44.5, Expansion 제거 시 78.2/67.9/44.9 이고, OPSD 76.6/67.4/45.1 에 Expansion 만 더해도 77.9/68.2/45.9 로 오른다. 논문은 모든 구성요소가 기여한다고 결론짓지만 [Sleep § 4.2], 표가 실제로 보여 주는것은 더 미세하다: **semantic reward 를 제거하면 AIME-25에서는오히려 69.2 로 완전체 (69.0) 보다 좋다**. frozen reward model 이라는 외부의존이 균일하게 이롭지는 않다는 신호이고, § 17.8 에서 다시 짚는다.

(6) **knowledge incorporation (SQuAD, SEAL protocol)**. no-context 정답률, 전 수치 검증 완료.

표 17-6 — SQuAD knowledge incorporation [Sleep Table 3].

Method	Single Passage (n=1)	Continued Pretraining (n=200)
Base model	31.9	31.9
Fine-tuned, no dreaming	33.4	32.0
SEAL	46.7	43.2
Sleep (Transformer, 2-level)	48.1	44.3
Sleep (Transformer + four-level)	48.9	46.2
– gradient 기반 선별	47.1	45.2
– random expert	48.0	44.7
– Dreaming 전체	35.7	36.2

n=200 설정은 passage 200 개를 한 번의 continued pretraining 으로 흡수하고연관 974 문항 전체로 평가하며, passage 당 dream 5 개를 모아 합성 데이터셋을만든다 [Sleep § 4.1]. 구성요소별로 보면 Dreaming 제거의낙폭 (48.9 → 35.7) 이 압도적이다 — 이 task 의 이득은 대부분 dreaming 에서온다.

(7) **few-shot ARC**. Llama-3.2-1B, train 11 task/held-out 8 task. sleep 훈련 중 task 당 dream 60 개를뽑아 45 개를 기각하고, test 에서는 미지 task 마다 dream 5 개를 생성해**각각 독립**

적으로 적용한 5 개 instance 로 예측하며, 정답을 낸 dream 의 비율을 보고한다 [Sleep § B.3]. 결과: ICL 0%, TTT+synthetic updates 10%, SEAL 72.5%, Sleep 80% [Sleep Table 4].

(8) 효율. step 당 비용은 Sleep 이 SFT 의 $4\times$ 다. 그러나 같은 목표성능에 도달하는 wall-clock 으로 재면 SFT 가 AIME-24/AIME-25/HMMT-25 에서 각각 $4.3\times/3.6\times/4.8\times$ 더 걸린다 [Sleep App. B.5]. 즉 이 패러다임이 사는 것은 싼 step 이 아니라 step·sample 효율이다.

스케일의 정직한 계산. [Sleep] 의 8B 는 이 라인의 from-scratch 사전학습 실증 상한 (여전히 1.3B/100B tokens, [TNT] 는 150M; $\rightarrow$ 15 장) 을 깨는 것이 아니다 — 기성 checkpoint 에 100 step 짜리 LoRA 규모 최적화를 얹은 graft 이다 (§ 17.4). 또한 여러 헤드라인 결과 (Fig. 3–6) 가 수치 표 없이 그림으로만 제시되고, 수백 번의 wake/sleep 사이클을 도는 장기 배포 시뮬레이션도 없다 (decode wall-clock 부재는 라인공통 — § 17.7·§ 17.8).

17.7 Systems/serving 합의

이 논문의 systems 합의는 라인의 어느 논문보다 크다. 앞 논문들은 layer 를 바꿨지만, 이것은 배포 형태를 바꾼다.

첫째, inference 는 더 이상 forward-only 가 아니다. wake phase 의 CMS 는 활성 블록 마다 token 당 optimizer error 항을 계산하고 (backward 류 kernel 필요), 파라미터 크기의 accumulator 를 유지하며, $C^{(\ell)}$ token 마다 weights 에 write 한다. memory 관점의 비용은 두 지점에서정량화된다. (a) **decode state 의 read-modify-write 트래픽**: 블록 ℓ 마다 token 당 accumulator RMW 1 회 (파라미터 크기), 그리고 $C^{(\ell)}$ token 에 1 회로 상각되는 weight write. KV cache 의 append-only 트래픽과 달리 이것은 read-modify-write 이고, 그 대역폭 계산이 10 장의 cost model 에 새 항으로 들어간다. (b) **update frequency 와 memory 계층 배치의 대응**: $\$C=\$1k \rightarrow 5k \rightarrow 10k$ token 사다리 [Sleep Fig. 7] 에서 고주파 (주기 작은) 블록의 accumulator 는 연산 가까이 상주해야하지만, 저주파 블록은 weights 를 수천 token 에 한 번 건드리므로 더 느린 계층에 두고도 write 비용을 상각할 수 있다 — frequency 사다리가 곧 storage-tier 사다리의 설계 힌트다. chunk 안에서 파라미터가 상수라는 성질덕에 chunk 단위 batching·병렬화는 보존된다 ($\rightarrow$ 9 장, 15 장).

둘째, session state 의 범주가 바뀔 수 있다. 순수 Transformer 의 세션 상태는 KV cache, $O(L)$ 이다 — BABILong 의 10M token 에서 이는 매우큰 크기이고, GPT-4 급 모델의 정확도가 128K–256K 너머에서 하락한다 Sleep § B.2. [Sleep] 은 장기 지식을 parametric memory(CMS) 로 옮기지만 sequence-layer state 를 일반적으로 제거하지는 않는다 — CMS 의 sequence model 이 attention 이면 KV cache 의 $O(L)$ 상태가 그대로 남는다 [Sleep § 2.2]. fixed-state sequence module(Titans 류 W_t) 을 택한변형에서만 지속 세션 상태가 파라미터로 대체된다: consolidation 1 회당 low-rank expert 하나, $2d d_{\text{low}}$ 개값 ($d_{\text{low}} \ll d$; 실험 전체가 dim-64 블록 5 개 추가로 수행됨 [Sleep App. B]). 이것은 KV cache 문제가 아니라 **per-user/per-agent weight-delta 서빙 문제**다 — multi-tenant LoRA adapter 서빙과 같은 부류로, adapter 의 버전관리·routing·eviction 이 세션 관리의 어휘가된다. 1 장 Rosetta 의 “paged KV cache $\leftrightarrow$ per-session weight state” 대응이여기서 문자 그대로 실현된다.

셋째, 정적 shape 는 설계로 보장된다. masked pre-allocation (§ 17.3.3) 덕에 tensor 크기 변경·재컴파일없다. 대가는 비활성 expert 의 죽은 자리인데, sparse MoE dispatch 가 masked expert 를 건너뛰면 FLOP 낭비는 자연히 사라진다 — router 수준에서 처리가능한 비용이다.

넷째, sleep 은 서빙 배포에 붙는 스케줄된 훈련 job 이다. 한 번의 sleep 이 요구하는 것을 나열하면 그 자체로 인프라 명세서다: teacher corpus 생성 + student on-policy rollout + LTI 완성 생성 (전부 generation-heavy, 즉 inference 모양의 부하), reward-model 서비스 호출 + 값 싼 Levenshtein 채점, 후보 dream 당 backward pass 1 회 (ARC protocol 이면 task 당 60 회), dream 별로 완전히 독립인 LoRA SFT job 들 (embarrassingly parallel), 그리고 ReST^{EM} 반복. 서빙 fleet 이 fine-tuning capacity 와 주기적으로결합되어야 하고, **sleep 1 회마다 새 모델 버전이 태어난다** — checkpoint 관리, cache 무효화, regression test, rollback 이 릴리스 시점 의일이 아니라 상시 운영의 일이 된다. ARC 의 test-time protocol 은 여기에 한걸음 더한다: 미지 task 하나의 “추론” 이 dream 5 개로 각각 adapt 된 5 개 instance 로의 fan-out 이다 [Sleep § B.3] — gradient update 가 serving loop 안에 들어와 있다.

다섯째, **비용 프로파일**. step 당 $4\times$ 라는 가격과 iso-accuracy $3.6 - 4.8\times$ 이득 [Sleep App. B.5] 은 “고정 capability 목표에는 compute 우위, 대신 복잡도를 선불” 로 요약된다 — RL, reward model, MoE 성장 부기, 버전 관리가 그 선불이다.

[평가] 이 절의 그림은 논문이 그린 것이 아니라 논문이 **비운** 자리다. [Sleep] 에는 sleep 한 사이클의 wall-clock·에너지 분해가 없고, dream 선별의 60-backward-pass 비용 분석이 없으며, 동일 compute 의 replay-buffer baseline 비교도 없다. decode wall-clock 은 라인 6 편 공통으로 부재하다. “sleep 은 서빙에 붙는 훈련 job” 이라는 명제의 경제성은 이 책의 Part III 가검증해야 할 가설이지, 논문이 입증한 사실이 아니다.

17.8 한계와 bridge-out: 라인의 완성형과 남은문제

마지막 장이므로 bridge-out 은 다음 논문이 아니라 라인 전체의 결산으로향한다. 먼저 [Sleep] 자신의 한계를 정리한다.

[Sleep] 의 한계. (1) **graft** 이지 **co-training** 이 아니다 — sleep 기계는 기성 backbone 에 부착되었고, wake/sleep 을처음부터 함께 훈련한 실증이 없다 (§ 17.4 의 의무 caveat). (2) **sleep** 시점이 학습되지 않는다 — chunk 경계에 hard-wire 되어 있고, 불규칙한 실전트래픽 아래의 거동은 미지수다. (3) **dreaming** 이 **task-aware** 다 — (x_{ctx}, τ) , 즉 context 와 평가 가능한 성능 측정이 sleep 중에 주어져야 한다. 완전 비지도 배포에는 τ 가 없으므로 [Sleep Eq. 5] 의 이진 개선 보상을 무엇으로 대체할지가 공백이다. (4) **frozen reward model** 의존 — 계속 변하는 student 에 대한 고정심판의 편향·drift 가 검토되지 않았고, § 17.6 에서 본 대로 semantic reward 제거가 AIME-25 에서는 오히려 이롭다 (69.2 vs 69.0 [Sleep Table 1]). (5) **router 동역학 미명세** — 새로 활성화된 expert 로 dispatch 를 배우고 reset 된 expert 로의 dispatch 를 멈추는 과정이 서술되지 않았다. 실구현의 실질적 공백이다. (6) **장기 안정성** — 수백 번의 sleep 에 걸친 expansion+reset 의 안정성, pre-allocate 된 expert pool 의 고갈시점 모두 미검증. (7) **이론 부재** — CF 를 capacity 문제로재정의했지만, consolidation 당 얼마의 capacity 가 얼마의 forgetting 을막는지, upward distillation 의 lossy “추상화” 가 무엇을 보존하는지에 대한형식적 진술이 없다. (8) **재귀적 self-modification** 의 안전성 — 매일 밤 자기 weights 를 고쳐 쓰고 random expert 로 novelty 를 주입하는모델의 안전·거버넌스 논의가 전혀 없다.

동시대 지형에서의 위치도 기록해 둔다. [Sleep App. A.4] 는 2026 년의 on-policy self-distillation(OPSD) 물결 전체를 지도화하고 네 축으로차별화한다: (i) 고정 teacher 의 reconditioning 이 아니라 새로 키운 capacity 로의 upward distillation, (ii) 평평한 teacher/student 쌍이 아니라 frequency 로 정렬된 memory 사슬, (iii) consolidation 만이 아니라 Dreaming 을 포함한 2 단계 sleep, (iv) 순수 per-token reverse-KL 이 아니라 GKD+imitation learning. 그리고 OPSD 의보고된 실패 모드 — epistemic verbalisation 억제로 인한 최대 40% 의 OOD 하락 (Kim et al. 2026, arXiv:2603.24472), 반복적용 시의 leakage·collapse — 를 2 단계 설계의 동기요인용한다. OpenReview 2025 년 9 월 공개를 근거로 한 우선권 주장도 이 부록의 일부다.

라인의 완성형. 이제 여섯 논문을 겹쳐 놓고 이 책의 결산을적는다.

[평가] 여섯 논문이 스스로 세운 것만 합성하면 하나의완성형이 실제로 그려진다. (1) **state** 는 모든 시간 규모에서 **weights** 다 — per-token fast memory([Titans]·[Atlas]), chunk 주기의 CMS level([NL]), sleep 주기의 grown expert([Sleep]), frequency 0 의 persistent weights 까지; Titans → Sleep 은 $\{\infty, 0\}$ 두 점뿐이던 frequency 스펙트럼을 연속체로 넓혀 온 하나의 긴 운동이다. (2) **모든 블록은 같은 객체** 다 — (architecture, attentional bias, retention, inner optimizer, frequency) 로 정의되는 associative memory 이고, attention 자체가 그 안의 non-parametric·무한 capacity·frequency- ∞ 꼭짓점이다 (→ 13 장, 14 장, 16 장). (3) **inner optimizer** 는 **architecture** 와 대등한 설계표면이다 — GD(TTT, → 8 장) → momentum([Titans]) → objective 동물원 ([Miras]) → window+Muon([Atlas]) → 자기 생

성 target([NL]). (4) **훈련은 어디서나 chunk-anchored 이고, 필요한 곳에서 계층적이다** — stale-snapshot 근사가 여섯 편 전부의 병렬화 기반이고 (→ 9 장), 그 경제학과 reset 은 [TNT] 가 정리했다. (5) **lifecycle 은 wake/sleep 이다** — 모델은 결코 “훈련이 끝나지” 않으며, 서버 fleet 은 설계상 주기적 fine-tuning job 을 돌린다. 제 품으로 읽으면: 세션 상태가 mutable weights 인 continually-learning LLM 에, sleep 서비스가 배포에 붙어 있는 시스템이다. 이 완성형이 “보이는” 이유는 여섯 논문 의 open-questions 절들이 전부 같은 다섯 공백을 가리키기 때문이다 — 남은 일이 텍스트에 의해 과잉 결정되어 있다.

단, 두 가지 유보 없이 이 계산은 정직하지 않다. 첫째, 완성은 개념적 완성이다. from-scratch 실증은 1.3B/100B tokens 에서 멈춰 있고, in-context retrieval 의 격차는 측정된 채 닫히지 않았다 — attention 53.55 vs 43.70 [Atlas], FDA 67.3 vs 41.9 [NL] (→ 14 장, 16 장). capacity 이론이 그 이유까지 말해 주므로 (ϕ^* 의 무한 capacity), 완성형은 attention 과의 hybrid 일 가능성이 열려 있고, 라인 자신의 MAC/MAG 결과들이 이를 조용히 인정한다. 둘째, “여섯 편이 한 이야기” 라는 독법은 부분적으로 소급적이다. 이음새가 실재한다: [TNT] 는 이야기가 본질이라 말하는 momentum-gating 을 “명료성을 위해” 제거한 훈련 논문이고 (→ 15 장), [Sleep] 은 end-to-end meta-learning 을 떠났다 (§ 17.4). 이 책은 그 이음새를 지우지 않고 보여 주는 쪽을 택한다.

남은 open problems. 여섯 편의 자체 목록을 중복 제거하면 다섯으로 수렴한다. (1) **스케일:** 모든 품질 주장이 1.3B/100B 에서 멈춘다 — momentum-deep memory-self-modification 의 우위가 7B+·SFT/RLHF·production 데이터에서 살아남는지가 최대 미지수다. (2) **retrieval 격차:** parametric 하게 닫을 수 있는가, 아니면 완성형은 필연적으로 hybrid 인가. (3) **serving 경제학과 kernel:** decode throughput/latency 수치 전무, fused deep-memory kernel 부재, per-request mutable weights 가 깨뜨리는 shared-weight batching (→ 10 장) — grouped-GEMM decode, state snapshot/rollback, optimizer-trajectory state 의 수치 drift, multi-tenant 격리, weight 로컬화된 context 의 privacy 까지. (4) **학습되는 스케줄:** chunk 크기, window c , CMS frequency, level 수, sleep 시점 전부가 hand-set 이다 — “무엇을 언제 갱신할지” 를 배우는 기제가 없다. (5) **task-free 이며 안전한 self-modification:** τ 없는 dreaming, reward model 의 존의 해소, 재귀적 weight 자기 편집의 안전성 분석, 그리고 미뤄진 이론 전무 — linear 특수 사례 밖의 regret-capacity-expressivity, chunkwise staleness 의 오차 한계는 여섯 편 어디에도 없다 (→ 9 장).

[Titans] 가 “test time 에 외우는 법을 배우자” 로 열었던 질문은, 여섯 편째에 이르러 “모델은 언제 깨어 있고 언제 자야 하는가” 로 바뀌었다. 그 질문의 답이 논문 한 편이 아니라 serving 시스템의 설계 문서처럼 생겼다는 것 — 그것이 이 라인이 inference 엔지니어에게 남긴 초대장이고, 이 책의 Part III 가 그 초대에 응한다.

제 3부 — 원저 기여: Scaling과 대규모 학습·서빙 청사진

Part III — 원저 기여

Part II 는 여섯 편이 하나의 배포 형태로 수렴함을 보였다 — 세션 상태가 mutable weights 인 continually-learning LLM. 그러나 여섯 편 어디에도 그 배포가 실제로 무엇을 요구하는지는 측정되어 있지 않다. decode wall-clock 수치가 라인 전체에 부재하다. Part III 는 그 빈 자리를 채우는 원저 기여다: 완성형을 systems 엔지니어의 도구 — roofline, GEMM shape, memory 계층, batching — 로 계량한다.

Part III 의 척추는 하나의 명제, **D4 workload-split pair thesis** —이 라인의 배포가 질적으로 다른 두 부하 (memory-centric 한 decode/serving-state 관리, accelerator 영역인 training/prefill) 로 갈라진다는 것 — 이며, 8 개 실측 실험이 그 두 절반을 각각 정량화한다. 이 명제의 정식 진술, 완성형의 재구성과 두유보, 8 개 claim 의 실험·figure·장 매핑, 그리고 실측의 정직성 계약 (exploration-grade —비율·crossover·순서는 본문으로, 절대치는 A100 runbook 으로 이월) 은 **18 장이 완결된 형태로 세운다**. 이 도입은 그위에 얹힌 나머지 장들이 어떻게 이어지는지만 가리킨다.

18 장이 프로그램 전체 (수렴·pair thesis·8 claim 지도) 를 세운 뒤, 나머지 일곱 장이 두 절반을 전개한다. **19 장** — scaling 분석: state-bytes 와 capacity 를 params·tokens 와 나란한 일급 scaling 축으로 올리고 여섯 편의 published 점들로 후보 scaling law 를 맞춘다. **20 장** — 하드웨어 병목: 각 아키텍처 클래스의 decode roofline 과, backward-pass-at-decode 라는 새 serving primitive. **21 장** — hardware lottery: attention 이 GEMM density 로 이긴 역사를 읽고, 이 라인이 스스로를 matmul 로 빚은 것이 채택·kernel 에 무엇을 예고하는지. **22 장** — player-strategy: Google(TPU·JAX·long-context 제품), NVIDIA(Gated DeltaNet 계보), open-source kernel 생태계가 각자 합리적으로 두는 다음 수. **23 장** — 대규모 학습·서빙 projection: TNT 경제학을 7-70B 로 외삽하고, per-session weight state 를 새로운 cache class 로 설계한다. **24 장** — 제안: 알고리즘 레벨 (§ 5 의 소규모 실험) 과 하드웨어 레벨 (frequency-tiered memory 배치와 fused chunk kernel·grouped-GEMM decode 의 pair). **25 장** — 결론과 연구 어젠다.

여덟 장을 관통하는 규율은 정직성 계약이다. 본문으로 승격되는 것은 비율·순서·bound 분류이지 silicon 정확 절대치가 아니며, memory-centric 논증은 pair thesis 의 두 지점 밖으로 나가지 않는다.

ch18. 여섯 편의 수렴과 Part III의 기여 프로그램

이 장의 목표 — 독자가 이 장을 마치면 (1) 여섯 편이 합성해 낸 “완성형” 을 텍스트 근거와 두 개의 정직한 유보와 함께 재구성할 수 있고, (2) 그 완성형을 엔지니어의 질문으로 옮긴 이 책의 척추 명제 — **D4 workload-split pair thesis** — 를 정식으로 진술할 수 있으며, (3) Part III 가 그 명제를 검증하려고 돌린 8 개 실험 실험이 어느 장에서 무엇을 뒷받침하는지 지도로 읽을 수 있어야 한다. **왜 필요한가** — Part II 는 여섯 논문을 각각 해부했고, 17 장은 그것들을 겹쳐 하나의 완성형을 그렸다 (§ 17.8 의 [평가]). 그 완성형은 layer 가 아니라 **배포형태**의 명제다. Part III 는 그 배포 형태를 systems 엔지니어의 도구 — roofline, GEMM shape, memory 계층, batching — 로 측정하는 원저 기여다. 이 장은 Part II 의 결산을 이어받아 Part III 의 프로그램 전체를 세운다.

18.1 Bridge-in: 초대장에서 시작한다

17 장은 이렇게 닫혔다. “[Titans] 가 ’ test time 에 외우는 법을 배우자’ 로 열었던 질문은, 여섯 편째에 이르러 ’ 모델은 언제 깨어 있고 언제 자야하는가’ 로 바뀌었다. 그 질문의 답이 논문 한 편이 아니라 serving 시스템의 설계 문서처럼 생겼다는 것 — 그것이 이 라인이 inference 엔지니어에게 남긴 초대장이다.” 이 장은 그 초대장에 응하는 첫 걸음이다.

초대의 성격을 분명히 해 둔다. 여섯 편은 layer 를 여섯 번 바꾸었지만, 그 누적 효과는 layer 교체 아니라 **불변식 하나의 폐기**다: “추론 중 weights 는 변하지 않는다” 는 transformer serving 의 정의적 전제 (→ 1 장 Rosetta 의 “weight update 없음” 행) 가 이 라인에서 사라진다. decode 가 weights 를 고쳐 쓰고, 배포에 훈련 job 이 붙는다. 이 폐기가 무엇을 요구하는지는 여섯 논문 어디에도 측정되어 있지 않다 — decode wall-clock 수치는 6 편 전체에 부재하다 (§ 6.4 의 라인 공통 caveat). Part III 는 그 빈자리를 채운다.

18.2 완성형, 여섯 편의 텍스트만으로

17 장이 정식화한 완성형을 여기서는 systems 함의의 순서로 다시 읽는다. 논문들이 **스스로 세우고 검증한 것만** 합성하면 다음이 도출된다 (dossier 의 convergence 분석을 본문 서사로 옮긴 것이며, 각 성분의 근거는 Part II 의 해당 장에 있다).

1. **state** 는 모든 시간 규모에서 **weights** 다. KV cache 는 update frequency 로 색인된 파라미터 블록의 스펙트럼으로 대체 (hybrid 에서는 sliding window 로 최소화) 된다 — per-token fast memory ([Titans]·[Atlas], → 12 장·14 장), chunk 주기의 CMS level ([NL], → 16 장), sleep 주기의 grown expert ([Sleep], → 17 장), 그리고 frequency 0 의 persistent weights. Titans → Sleep 은 $\{\infty, 0\}$ 두 점뿐이던 frequency 축을 연속체로 넓혀 온 하나의 긴 운동이다.
2. **모든 블록은 같은 객체**다 — (architecture, attentional bias, retention, inner optimizer, frequency) 로 정의되는 associative memory 이고, attention 자체가 그 안의 non-parametric·무한 capacity·frequency- ∞ 꼭짓점이다 (→ 14 장의 ϕ^*).

3. **inner optimizer** 는 **architecture** 와 대등한 설계 표면이다 — GD(→ 8 장) → momentum([Titans]) → objective 동물원 ([Miras]) → window+Muon([Atlas]) → 자기 생성 target([NL]).
4. **훈련**은 어디서나 **chunk-anchored** 이고 필요한 곳에서 계층적이다 — stale-snapshot 근사가 여섯 편 전부의 병렬화 기반이며 (→ 9 장), 그 경제학과 reset 을 [TNT] 가 정리했다 (→ 15 장).
5. **lifecycle** 은 **wake/sleep** 이다 — 모델은 결코 훈련이 끝나지 않고, 서버 fleet 은 설계상 주기적 fine-tuning job 을 돌린다 (→ 17 장).

[평가] 제품으로 읽으면 이것은 세션 상태가 **per-session/per-tenant mutable weights** 인 **continually-learning LLM** 에, **TNT 경제학**으로 학습되고, **sleep** 서비스가 배포에 붙어 있는 시스템이다. 저자의 논지 — “여섯 편을 다 읽으면 Google 의 다음 수가 완성형으로 보인다” — 는 실질적으로 지지된다. 여섯편의 open-questions 절이 전부 같은 다섯 공백 (스케일, retrieval 격차, serving 경제학 · kernel, 학습되는 스케줄, task-free · 안전한 self-modification) 으로 수렴하기 때문이다. 남은 일이 텍스트에 의해 과잉 결정되어 있다는 것이 이 논지의 힘이다.

단, 두 유보 없이는 정직하지 않다. **유보 1 — 완성은 개념적완성이다.** from-scratch 실증은 1.3B params / 100B tokens 에서 멈춰 있고 ([TNT] 는 150M), 이 상한이 7B+ · SFT/RLHF · production 데이터에서 유지되는지는 미지수다. **유보 2 — in-context retrieval 격차가 측정된 채 닫히지 않았다:** attention 53.55 vs 43.70 [Atlas Table 5], FDA 67.3 vs 41.9 [NL] (→ 14 장 · 16 장). capacity 이론이 그 이유 (ϕ^* 의 무한 capacity) 까지 말해 주므로, 완성형은 attention 과의 hybrid 일 가능성이 열려있다. 이 두 유보는 Part III 의 모든 정량 주장이 딛고 서는 바닥이다.

18.3 완성형을 엔지니어의 질문으로: D4 pair thesis

완성형이 옳다면 그것은 하나의 **workload** 를 만든다. 그 workload 를 systems 관점에서 반으로 가르는 것이 이 책 Part III 의 척추 명제다.

[평가] **D4 workload-split pair thesis.** 이 라인의 배포는질적으로 다른 두 부하로 갈라지며, 각각의 최적 하드웨어 전략이 다르다. - **decode/serving-state 관리 = memory-centric 기회.** decode step 은 token 마다 fast-weight state 전체의 **read-modify-write(RMW)** — 읽고, 갱신하고, 되쓴다 — 를 수행한다. 이 트래픽은 sequence 마다 **unshared** 이고, **write-heavy** 이며, content 로 주소 지정되지 않는다. 이것은 append-once/read-many 이고 prefix 로 공유 가능한 KV cache 와 **질적으로 다른** 메모리 부하다. - **training/prefill = accelerator 영역.** 같은 알고리즘도 chunk 크기 C 를 키우면 compute-bound 로 옮겨 간다. chunk C 는 문자 그대로 roofline 의 x 축이며, 이 영역의 승부는 fused chunk kernel 과 grouped-GEMM 에서 난다 (→ 15 장의 TNT 가 이미 plain JAX 로 FlashAttention 을 이긴 지점).

흥미로운 하드웨어 제안은 둘 중 하나가 아니라 **쌍 (pair)** 이다. 어느 한쪽만 옹호하는 것은 부하의 절반을 무시하는 것이다.

이 명제의 두 절반이 이 책이 memory-centric 논증을 펴는 **유일한** 두 지점이다. 그 밖 — “이 라인의 훈련이 새 메모리 소자를 요구한다”, “일반 PIM 이 답이다” — 은 논문들의 자체 증거가 반대 방향을 가리키므로 (여섯 편이 알고리즘을 dense matmul 로 다시 빚어 기존 accelerator 에 맞췄다) 이 책은 주장하지 않는다. PIM 은 오직 write-heavy elementwise epilogue(decay · renorm · AXPY) 라는 소수 FLOP 지점에서만, 그것도 directional 하게 등장한다 (§ 18.6, → 20 장 · 24 장).

decode 절반이 “새 기회” 인 이유는 그림 18-1 의 한 교차점으로 압축된다. KV cache 의 읽기 트래픽은 문맥 길이에 비례해 자라지만 (read $\propto$ 문맥), TTT state 의 RMW 트래픽은 문맥에 **무관하게 일정**하다 (state 크기 = $m d^2 L_{\text{layer}}$ 로 고정; m 은 **state multiplier** — state 가 d^2 의 몇 배인

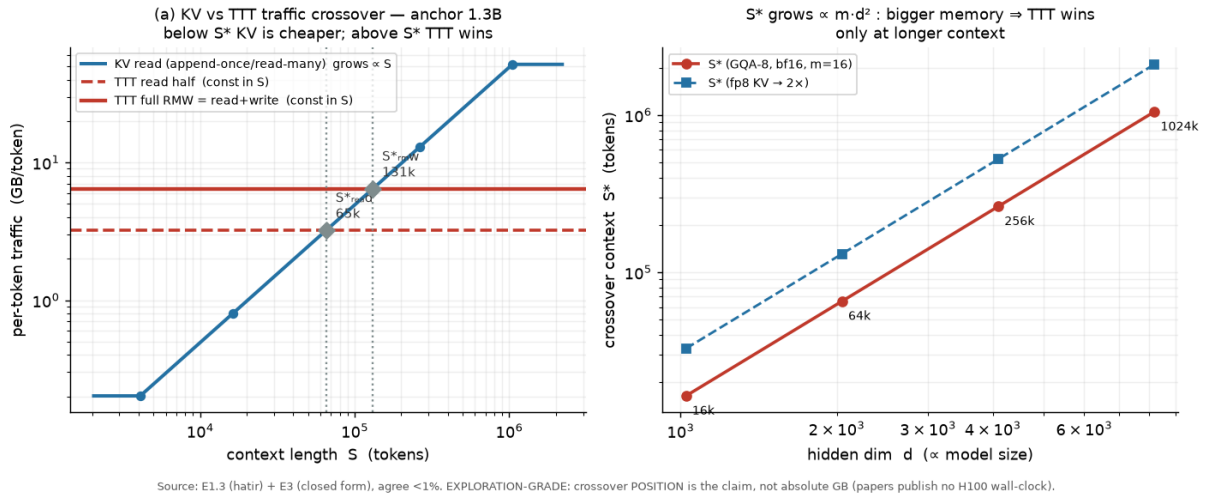


그림 18-1 — KV cache와 TTT state의 트래픽 교차: 문맥 길이에 따른 crossover와 그 스케일링

가로, deep memory $m \approx 8 \cdot \text{momentum}$ 포함 $m \approx 16$ — 이고 L_{layer} 는 layer 수다). 따라서 그 아래에서는 KV가 싼 메모리 시스템이고 그 위에서는 TTT가 싼 **crossover 문맥 길이**가 존재한다.

그림 18-1 — KV cache 읽기 트래픽은 문맥 길이에 비례해 증가하고 TTT state RMW 트래픽은 문맥에 무관하게 일정하므로, 두 곡선이 만나는 crossover 문맥 길이가 존재한다. anchor(neural-mem-1.3B)에서 read-crossover는 약 65k token, 전체 RMW-crossover는 약 131k token이며, crossover는 모델 폭에 따라 $m \cdot d^2$ 로 이동한다 (실험 E1.3/E3 재구성).

18.4 이 책이 돌린 실측 프로그램: 8개 claim

Part III는 위 pair thesis의 두 절반을 각각 정량화하는 8개 실험을 돌렸다. 방법은 세 갈래다: 닫힌 형태 cost model(E3), analytic twin 도구 (hatir + hat-schema, E1.x), CPU micro-benchmark(E2.x). 세 방법이 anchor 설정 neural-mem-1.3B ($d=2048$, $m=16$, $L=24$, GQA-8, bf16)에서 **1% 이내로 일치한다** (RMW 6.44 GB/token, 상태 134 MB/layer, read-crossover 65k token — E3·E1.1·E1.3·plan 4 자합치). 이 교차검증이 비율·crossover·순서를 본문으로 승격할 근거이며, 절대값은 승격하지 않는 이유는 § 18.6에서 명문화한다.

decode 절반 (claim 1 – 6). (1) decode step은 anchor에서 6.44 GB/token을 움직이고 arithmetic intensity가 0.59 FLOP/byte로, H100 twin ridge 295 FLOP/byte보다 두 자릿수 아래다 — **결정적으로 memory-bound**이며, state 트래픽이 GEMV 연산을 약 $394\times$ 압도한다: step 비용이 곧 state 트래픽이다. (2) 그 트래픽의 성격이 KV와 갈라진다 — KV의 write는 read의 0.003%(append-once)인데 TTT는 100%(대칭 RMW). (3) 그 결과 batch를 키우면 HBM 용량보다 **대역폭이 먼저 막힌다** — KV cache가 capacity-bound인 것과 정반대다. (4) TTT state 크기가 문맥에 무관하므로 on-chip 상주 여부가 **설계 가능한 knob**이 되며 (KV와 달리), state가 fit하는 쪽에서는 scratchpad가 HBM 대비 이득이지만 폭이 커지면 spill한다 (residency crossover). (5) NL/Sleep의 update-frequency 연속체가 memory 계층 배치로 **직접** 번역된다 — 상주 사본은 read/write **중빠른 쪽** cadence로 tier가 정해진다. (6) 기존 KV-manager의 semantics(prefix 재사용, append-only placement)는 RMW state에 대해 범주 오류다 (재사용 용이 구조적으로 0).

training/prefill 절반 (claim 7 – 8). (7) chunk C 가 roofline의 x 축이다 — $C=1$ (per-token RMW=decode 영역)은 $AI \approx 1$ 로 memory-bound 평원에 있고, C 를 키우면 crossover를 넘어 compute-bound로 오른다: **한 알고리즘, 두 영역**. (8) on-die/off-die 경계에서 RMW 대역폭이 불연속으로 꺾이며 (cliff), RMW는 read의 절반 throughput만 낸다 — append-once KV가 피하는 **write-back 세 (稅)**를 직접 측정 한 값이다.

표 18-1 — 8개 claim과 뒷받침 실험·figure·장래평.

claim	한 줄 결론	실험	figure	주 담당 장
1	decode = 전체 state 의 memory-bound RMW	E1.1/E3	—	19, 20
2	KV(append-once) vs TTT(RMW) 트래픽 crossover	E1.3/E3	18-1	18, 19
3	batch 는 대역폭이 먼저 막힘 ($\neq$ capacity-bound KV)	E1.3/E3	—	23
4	state 상주는 설계 knob; scratchpad fit $\rightarrow$ spill	E1.2	(b)	20, 24
5	update cadence $\rightarrow$ memory-tier 배치 규칙	E1.4	(d)	20, 23, 24
6	KV-manager 는 RMW state 에 범주 오류	E1.5	—	10, 23
7	chunk $C =$ roofline x 축 (memory $\rightarrow$ compute)	E2.1	(c)	9, 15
8	on/off-die RMW 대역폭 cliff + write-back 세	E2.2	(e)	20

18.5 Part III 로드맵

Part III 는 위 프로그램을 일곱 장으로 편다. pair thesis 의 어느 절반을미는지 괄호로 표시한다.

- **19 장 — Scaling analysis 와 memory-centric scaling law 후보 (decode).** 여섯 편의 ppl-vs-FLOPs/params/context 점을 digitize 하고, state-bytes 와 capacity($O(d_k^p)$, $\rightarrow$ 14 장) 를 params·tokens 와 나란한 일급 scaling 축으로 제안한다. RMW 트래픽이 폭에 따라 $0.45 \rightarrow 6.44 \rightarrow 343$ GB/token 으로, crossover 가 $16k \rightarrow 1.05M$ token 으로 자라는 스케일링이 여기 들어간다.
- **20 장 — Hardware bottleneck analysis(양쪽).** 아키텍처 class 별 decode roofline(state read+write vs KV append-read), decode 에들어온 backward-pass 라는 새 serving primitive, NS-5·deep-memory 의 FLOP density, state placement(그림 b·e). memory-centric 논증의 본진이자 accelerator 반쪽 (fused kernel) 의 본진.
- **21 장 — Transformer/NVIDIA scaling era 의교훈 (accelerator).** hardware-lottery 독법: attention 은 GEMM density 로이겼고, 이 라인은 chunkwise·NS·reset 으로 matmul 안에 설계되어 있다 — 이 가족이 아직 기다리는 FlashAttention-moment 이 무엇을 예고하는지.

- 22 장 — **Player-strategy analysis**. Google Research(TPU pod·JAX·long-context 제품) 가 왜 자연스러운저자인지, NVIDIA 의 위치 (Gated DeltaNet 계보), open-source kernel 생태계 (flash-linear-attention) 가 각각 합리적으로 두는 다음 수.
- 23 장 — 대규모 **training·serving** 투영 (양쪽). TNT 경제학을 7 – 70B 로 외삽; 계층적 memory 의 prefill/decode 분할; **per-session weight state** 를 새 **cache class** 로 (sizing, checkpoint/restore, multi-tenant batching — claim 3 의 $B_{\max}$ 가 여기 근거 — 와 **frequency-tiered placement** — claim 5 의 cadence $\rightarrow$ tier 가 여기 본진, § 23.6); sleep 을 fleet 수준 background job 으로. 오늘의 KV-cache 서빙과의 비용 모델 대조.
- 24 장 — **Proposals**(양쪽, pair 로). 알고리즘 수준의 소규모 실험제안; HW 수준의 정직한 memory-centric 평가 (frequency-tiered placement, RMW-bandwidth decode 소자) 와 accelerator 반쪽 (fused chunk kernel, grouped-GEMM decode engine) 을 쌍으로. 그림 b·d 가 근거.
- 25 장 — **Conclusion** 과 **research agenda**.

18.6 정직성 계약: exploration-grade

Part III 의 모든 수치는 아래 계약 아래에서만 읽어야 한다. 이 계약은 § 6.4 의무 caveat 의 Part III 확장이다.

- **grade = exploration-grade**. 근거는 pre-silicon analytic cost model(hatir + hat-schema twin) 과 CPU micro-benchmark 다. 이들은 **비율 · crossover 위치 · tier 순서 · bound class** 를 산출하도록 자기 선언되어 있으며, silicon-accurate 절대값이 아니다. 따라서 load-bearing 으로 본문에승격되는 것은 이 네 종류의 양뿐이다.
- **절대값은 하한이고, 이월된다**. 6 편의 원 논문이 H100 decode wall-clock 을 하나도 공개하지 않으므로, 이 책이 내는 절대 $\mu\text{s}/\text{token} \cdot \text{mJ}/\text{token}$ (예: anchor 의 1.92 ms/token, 46.5 mJ/token) 은 roofline 하한이며 외부 검증 불가다. 이 값들은 본문 주장의 근거가 아니라, 사내 A100 runbook(Part III-a) 으로 이월되는 검증 대상이다. 본문이 덮는 것은 그 하한이 드러내는 구조 (memory-bound, RMW-지배) 뿐이다.
- **novel twin 은 directional 이다**. scratchpad·PIM twin 은 `simulation_ready=False` 로, verify committee 가 PPA 를 abstain 한다. scratchpad 의 $6.8 \times \text{energy} / 14.9 \times \text{time}$ 이득, PIM 의 $1.6 \times \text{energy}$ 이득은 shipping-device 주장이 아니라 **directional DSE** 로만 인용한다.
- **CPU 실측은 shape 만 이전한다**. E2.x 의 host roofline(ridge ≈ 34 FLOP/byte) 은 H100 twin ridge(295 FLOP/byte) 의 약 1/9 이므로, 측정된 $C^* \approx 32$ 나 cliff 의 GB/s 절대값을 H100 으로 옮기지 않는다. 이전되는 것은 곡선의 모양과 memory $\leftrightarrow$ compute 교차의 존재 뿐이다 (H100 closed-form C^* 는 306 – 430).
- **state multiplier 는 가정값이다**. $m(2\text{-layer MLP } 8d^2, +\text{momentum } 16d^2)$ 과 (d, L) 은 공개된 아키텍처 서술에서 온 canonical shape 이지 공개된 decode trace 가 아니다.

[해설] 이 계약의 실천적 의미는 단순하다. Part III 의 문장이 “TTT decode 는 KV 보다 폭이 클수록, 문맥이 65k token 을 넘을수록 상대적으로 유리해진다” 고 말하면 그것은 load-bearing 주장이다 (비율 · crossover). 반면 “그 decode 가 1.92 ms 걸린다” 고 말하면 그것은 하한의 보고이지 주장이 아니다 — 그 자리에는 항상 “roofline 하한, A100 runbook 으로 이월” 이라는 꼬리표가 붙는다. 이 구분을 흐리는 것이 이 책이 피하려는 유일한 과장이다.

요약

- 17 장의 “초대장” 을 이어받아, 이 장은 여섯 편의 완성형 (state = 모든 시간규모의 weights; 모든 블록이 같은 associative-memory 객체; inner optimizer 가 설계 표면; chunk-anchored · 계층적 훈련; wake/sleep lifecycle) 을 systems 함의 순으로 재구성했다.

- 저자 논지 (“Google 의 next 가 완성형으로 보인다”) 는 open-questions 의수렴을 근거로 실질 지지되나, 두 유보 (1.3B/100B 실증 상한, 미해소 retrieval 격차 53.55 vs 43.70) 를명 시해야 정직하다.
- Part III 의 척추는 **D4 pair thesis** 다: decode/serving-state 관리 = memory-centric 기 회 (unshared·write-heavy·비-content-addressable 한 whole-state RMW), training/ prefill = accelerator 영역 (chunk C = roofline x 축). 흥미로운 HW 제안은 쌍이다.
- 이 책은 pair thesis 를 8 개 실험으로 측정했고, 세 방법이 anchor 에서 1% 이내로 일치한 다. claim 은 표 18-1 로 장에 매핑된다 (19 – 25 장).
- 모든 수치는 exploration-grade 계약 아래 읽는다: 비율 ·crossover·tier·bound 만 load-bearing, 절대 $\mu\text{s}/\text{mJ}$ 은 roofline 하한 (A100 runbook 이월), novel scratchpad/PIM 은 directional.

자가 점검체크리스트

- ☐ 완성형의 다섯 성분을 텍스트 근거와 함께 재구성하고, 두 유보를 짚을 수있다.
- ☐ D4 pair thesis 의 두 절반과 각각의 최적 하드웨어 전략을 진술할 수 있다.
- ☐ memory-centric 논증이 허용되는 두 지점 (decode RMW 트래픽, update-frequency $\leftrightarrow$ tier) 과 금지되는 두 지점 (훈련용 새 소자, 일반 PIM) 을구별할 수 있다.
- ☐ KV(append-once) 와 TTT(RMW) 의 트래픽 crossover 가 왜 생기고 어디에있는지 설명할 수 있다.
- ☐ 어느 claim 이 어느 장을 뒷받침하는지 (표 18-1) 읽을 수 있다.
- ☐ 어떤 수치가 load-bearing 이고 어떤 수치가 이월되는 하한인지 판별할 수있다.
- ☐ pair thesis 를 inference 어휘 (paged KV cache $\leftrightarrow$ per-session weight state; append vs RMW) 로 옮길 수 있다.

다음 장으로

이 장은 완성형을 workload 로, workload 를 pair thesis 로, pair thesis 를 8 개측정으로 내렸다. 남은 것은 각 측정을 그 자체의 깊이로 펴는 일이다. 19 장은 그 첫 축 — state-bytes 와 capacity 를 params·tokens 옆의 일급 scaling 축으로 세우고, RMW 트래픽과 crossover 가 스케일에 따라어떻게 자라는지를 여섯 편의 공개 점 위에 얹는다 — 로 시작한다.

ch19. Scaling 분석과 memory-centric 모델의 후보 scaling law

이 장의 목표 — 독자가 이 장을 마치면 (1) 여섯 편이 공개한 `ppl-vs-{params, tokens, context}` 점을 하나의 표로 digitize 하고 그 비교가 어디까지 정당한지 (tokenizer 이질성·setting_group 경계) 판별할 수 있고, (2) 왜 `params·tokens` 두 축만으로는 이 라인의 scaling 을 기술할 수 없는지 — **state-bytes** 와 **capacity**($O(d_k^p)$) 가 빠진 두 개의 독립 축임을 — 논증할 수 있으며, (3) 그 공개 점에 실제로 후보 법칙을 fit 한 결과 (E4) 를 읽고, capacity 축이 raw state-bytes 대비 예측력을 **어디서 더하고 어디서 더** 하지 못하는지 정량으로 판정할 수 있고, (4) decode 비용·crossover·quality 각각에 대한 후보 scaling law 의 형태를 진술하며 A3 falsification 조건과 실제 검정 결과를 대조할 수 있어야 한다. **왜 필요한가** — 18 장은 완성형을 workload 로, workload 를 D4 pair thesis 로 내렸고, Part III 를 일곱 장으로 폈다. 이 장은 그 첫 축이다. scaling law 는 inference 엔지니어가 fleet 을 sizing 할 때 쓰는 도구인데, 이 라인은 여섯 편에 걸쳐 quality scaling 을 단한 번도 law 로 fit 하지 않았다 (dossier § 3.2 공백 1: “스케일”). 이 장은 그 빈자리에 두 개의 새 축을 세우고, 그 축들이 pair thesis 의 decode 절반과 어떻게 맞물리는지를 실측 claim 으로 뒷받침한다.

19.1 Bridge-in: scaling law 는 왜 두 축이모자란가

18 장의 [평가] 는 이 라인의 배포를 “세션 상태가 per-session mutable weights 인 continually-learning LLM” 으로 읽었다. inference 엔지니어가 그런 시스템의 fleet 을 계획하려면 익숙한 두 질문에 답해야 한다: **주어진 예산에서 어떤 품질이 나오는가** (quality scaling), 그리고 **주어진 품질을 서빙하는 데 자원이 얼마나 드는가** (cost scaling). transformer 세계에서 두 질문의 도구는 각각 Chinchilla 식 법칙 $L(N, D)$ (Hoffmann et al. 2022, arXiv:2203.15556) 과 KV cache sizing 이다 — 둘다 `params` N 과 `tokens` D 라는 두 축 위에 산다.

이 라인은 그 두 축을 깨뜨린다. 정확히는, 두 축을 **불충분하게** 만든다. 이유는 18 장에서 이미 나왔다: state 가 KV cache 가 아니라 weights 이고 (완성형 성분 1), 그 weights 의 크기가 `params` 와 분리된 독립 knob 이며 (§ 18.3 의 decode 절반), 같은 (N, D) 아래서도 architecture class 가 바뀌면 memory **capacity** 가 $O(d_k)$ 에서 무한대까지 움직인다 (→ 14 장의 ϕ^*). 즉 `params·tokens` 는 quality 의 일부만, cost 의 일부만 설명한다. 이 장의 명제는 하나다: **memory-centric 모델의 scaling law 는 `params·tokens` 위에 `state-bytes` 와 `capacity` 라는 두 개의 일급축을 얹어야 완결된다**. 그리고 그 두 축이 정확히 pair thesis 의 decode 절반이 사는 곳이다.

이 장은 그 명제를 세 단계로 세운다. 먼저 여섯 편의 공개 점을 digitize 하고 (§ 19.2), 그 점들을 하나의 fit 위에 올릴 때 넘을 수 없는 방법론적 선 — tokenizer 이질성과 setting_group 경계 — 을 못박는다 (§ 19.3). 그다음 두 개의 빠진 축 (state-bytes § 19.4, capacity § 19.6) 을 세우되, 그 사이에 이 장의 정량 백본을 끼운다: 공개 점에 후보 법칙을 **실제로 fit** 한 E4 의 결과 (§ 19.5) 다. E4 는 A3 의 핵심 질문 — “capacity 는 raw state-bytes 대비 예측력을 더하는가?” — 을 digitize 된 공개 점 위에서 rank-correlation 으로 검정하며, 그 답은 **축마다 다르다**. 마지막으로 후보 법칙

의 형태 (§ 19.7)와 A3 falsification 조건 대실제 판정 (§ 19.8)을 대조한다.

19.2 여섯 편의 공개 scaling 점을 digitize 한다

먼저 재료를 모은다. 여섯 편이 본문·표에 공개한, 서로 다른스케일에서의 품질 점을 하나의 표로 옮긴다(표 19-1). 원문 수치이므로출처를 병기하고, 반올림하지 않는다.

표 19-1 — 여섯 편의 공개 품질 점 (from-scratch LM). ppl 은 Wiki / LAMBADA, acc 는 common-sense 평균. tokenizer 열이 비교가능성의 관문이다.

모델	스케일	tokens	tokenizer	Wiki ppl	avg acc	출처
Titans (LMM)	760M	30B	Llama-2	20.04	51.56	[Titans Table 1]
Titans (LMM)	340M	15B	Llama-2	—	46.17	[Titans Table 1]
Atlas	1.3B	100B	T5-32K	14.97	57.62	[Atlas]
Atlas++	1.3B	100B	T5-32K	14.40	58.03	[Atlas]
Titans (재실행)	1.3B	100B	T5-32K	15.60	56.82	[Atlas]
Gated DeltaNet(이하 GDN)	1.3B	100B	T5-32K	16.42	55.32	[Atlas]
Trans-former++	1.3B	100B	T5-32K	18.53	52.25	[Atlas]
Atlas	760M	30B	T5-32K	19.97	52.77	[Atlas Table 6]
Hope	1.3B	100B	32K	14.39	58.04	[NL]
Titans (재실행)	1.3B	100B	32K	—	56.82	[NL]
Hope	760M	30B	32K	18.68	52.28	[NL]
TNT (best)	150M	10B	T5-32K	—	~41.0	[TNT Table 2]

이 표를 세로로 읽으면 스케일에 따른 단조 개선이 드러난다: Atlas 의 avg acc 는 760M 52.77 → 1.3B 57.62 로, Hope 는 760M 52.28 → 1.3B 58.04 로 오른다. [NL] 은 “gains grow with scale” 을 명시하고, [Atlas Fig. 8] 은 params 와 training context 양쪽에서 baseline 대비 우호적 scaling 을 보고한다 — **이라인이 공개한 유일한 scaling-law 모양의 증거다**. 그러나 이 표를 fit 하려면 먼저 가로 방향으로 어디까지 읽을 수 있는지를 결정해야 한다. 그것이 다음 절의 방법론이고, 이 장의 모든 정량 결과가 그 위에 선다.

19.3 방법론: tokenizer 이질성과 setting_group — fit 의 경계가왜하나인가

표 19-1 은 세로로는 읽히지만 가로로는 함부로 읽히지 않는다. **tokenizer 열이 이유다**. 이 절은 그 이유를 정량으로 못박고, 이장이 어떤 fit 을 legal 하다고 부르는지의 규칙 — **setting_group** — 을 정의한다. 이후 § 19.5 의 모든 fit 은 이 규칙 안에서만 계산된다.

왜 tokenizer 가 ppl 을 비교 불가로 만드는가. perplexity 는 per-token cross-entropy 의 지수 $ppl = \exp(-\frac{1}{T} \sum_t \log p(x_t | x_{<t}))$ 인데, 여기서 token x_t 의 단위 자체가 tokenizer 의 vocabulary 가 정한다. vocab 이 다르면 같은 텍스트가 서로 다른 개수의 token 으로 쪼개지고, 그러면 sum 의 분모 T 와 각 항의 정보량이 함께 달라진다 — 즉 **분모가다른 두 척도의 ppl 은 좌표**

게가 다르다. Titans 원 논문의 점은 Llama-2 tokenizer 로, Atlas·TNT 의 T5, NL/Sleep 의 32K vocab 점과 같은 축의 perplexity 가 아니다. 따라서 “Titans 760M 20.04 가 Atlas 1.3B 14.97 보다 나쁘다” 는 스케일차이와 tokenizer 차이가 뒤섞인 무의미한 비교이며, 여기에 fit 을 없으면 exponent 가 아니라 두 vocab 의 압축률 차이를 재게 된다.

setting_group: legal 한 fit 의 유일한 경계. 그래서 이 장은 fit 을 계산할 수 있는 최소 단위를 하나의 **setting_group** — 같은 논문 + 같은 tokenizer + 같은 data + 같은 metric — 으로 정의한다 (E4 의 fit 경계규칙). 이 경계를 넘는 순간 fit 은 exploration-grade 조차 아니고, 좌표계혼합이다. 이 규칙이 표 19-1 을 세 종류의 비교로 분해한다.

1. **세로 (within-setting) 대조 — legal.** 한 setting_group 안에서 스케일만 바꾼 점들. Miras 의 Moneta·GDN 을 FineWeb-Edu·WikiText ppl 로 340M → 760M → 1.3B 로 켜 계열이 여기다 (§ 19.5 Group A). 이 계열은 fit 이 가능하다 — 단 아래의 confound 를 안고서.
2. **논문 내 재실행 baseline 대조 — legal.** 한 논문이 자기 protocol 로 재실행한 baseline 과의 대조. Atlas 가 재실행한 Titans 1.3B 15.60 vs Atlas 14.97, 그리고 Atlas Table 2 의 고정 1.3B cross-architecture 점 전부 (§ 19.5 Group B).
3. **정렬된 논문 간 앵커 대조 — 제한적으로 legal.** tokenizer·데이터가 우연히 정렬된 논문 간 대조. 여기 한개의 단단한 앵커가 있다: Atlas 와 NL 이 각각 재실행한 **Titans 1.3B 의 avg acc 가 56.82 로 정확히 일치**한다 (FineWeb 계열 + 32K vocab). 이 일치하는 두 논문의 1.3B 점을 같은 축 위에 놓을 수 있게 해주며, 그 위에서 Atlas++ 58.03 과 Hope 58.04 가 사실상 동물로 Titans 56.82 를 넘는다는 **cross-architecture ordering** 은 load-bearing 하다.

N,D confound: 세로 fit 조차 순수 param law 가 아니다. 위 (1) 의 세로 대조에도 함정이 하나 있다. 이 라인의 공개 계열은 params 를 키울 때 tokens 도 함께 키웠다 — Miras 는 340M/15B → 760M/30B → 1.3B/100B 로 N 과 D 가 동시에 자란다. 따라서 ppl-vs-params 만 fit 한 exponent 는 param scaling 과 token scaling 이 뒤엉킨 값이지 순수 param law 가 아니다. 정직한 축은 $\text{compute} \propto 6ND$ 이며 (§ 19.5), params-only 지수는 **descriptive** 로만 보고한다.

왜 rho 인가: tiny-N 에서 정직한 통계. 이 모든 setting_group 은 점이 3–8 개뿐이다. 그런 표본에서 최소제곱 exponent 의 신뢰구간은 사실상의 의미가 없다. 그래서 § 19.5 의 cross-architecture 검정은 continuous exponent 대신 **Spearman rank-correlation ρ** 를 1 급 통계로 쓴다 — 순서 (ordering) 만 읽고 크기는 읽지 않으므로 정직성 제약의 “비율·순서만 load-bearing” 원칙과 정확히 정렬한다. 보고되는 p 값은 방향의 지표일 뿐 유의성 판정이 아니다.

[평가] 이 절의 실질은 “비교 불가” 라는 부정형 결론이 아니라, **정확히 무엇이 legal 한 지를 규정하는 경계**다. 넘어서는 안 되는 선은 표 19-1 의 가로줄을 fitted exponent 로 승격하는 것이다. 지지되는 것은 (i) setting_group 안의 descriptive fit, (ii) 재실행 baseline 대조, (iii) 56.82 앵커 위의 ordering — 이 셋뿐이다. § 19.5 는 이 세 종류 안에서만 움직이고, § 19.8 은 이 경계 자체가 A3 falsifier 1 의 발효임을 보인다.

19.4 빠진 축 하나: state-bytes

params·tokens 가 놓치는 첫 번째 축은 cost 쪽에 있다. transformer 에서 서빙 상태 (KV cache) 는 $2Ld$ 로 **문맥 길이 L 에 비례해** 자라고 params 와는 다른 축이다 — 엔지니어는 이 사실에 익숙하다. memory-centric 모델에서 서빙 상태는 **fast-weight state** 이고, 그 크기는 문맥에 무관하며 다음으로 고정된다:

$$B_{\text{state}} = m d^2 L_{\text{layer}} \cdot s \quad (19-1)$$

여기서 **state multiplier m** 은 표준 deep memory 의 상태배수 (§ 1.6; 2-layer MLP 이면 $8d^2$, momentum 포함이면 $16d^2$), d 는 model 폭, L_{layer} 는 layer 수 (sequence 길이 L 이 아님), s 는 원소당 바이트다. anchor neural-mem-1.3B ($d=2048, m=16, L_{\text{layer}}=24, \text{bf16}$) 에서 layer 당 $16 \cdot 2048^2 \cdot 2 = 134 \text{ MB}$, 전체 $B_{\text{state}} \approx 3.2 \text{ GB}$ 다 (실험 E1.1/E3, 세 방법이 layer 당 134 MB 로 1% 이내 일치).

m 은 **architecture class** 가 정하는 **knob** 이다. (19-1) 에서 d, L_{layer} 가 params 를 따라가는 동안, m 은 **class 에 따라** 독립적으로 움직인다. E4 가 digitize 한 class 별 m (units of d^2/layer , bf16) 은 다음과 같다: matrix memory(GDN) 는 $m=1$, 표준 deep-MLP(Titans/TTT, $W_1: d \times 4d + W_2: 4d \times d = 8d^2$) 는 $m=8$, momentum·2nd-moment buffer 를 더하면 $m=16$, poly feature map 으로 key 차원을 부풀린 Atlas 는 directional 로 $m \approx 24$, 그리고 attention 은 fast-weight state 가 없으므로 $m=0$ (대신 KV cache 가 문맥에 비례해 자라 별도회계된다 — E1.3). 이 m 축을 d, L_{layer} 의 표준 Llama shape 과 곱하면 표 19-2 가 나온다.

표 19-2 — class 별 fast-weight state-bytes(MB/layer, bf16, 표준 Llama shape). d, L_{layer} 는 크기당 표준 config 가정이므로 **directional** 이지만, class 간 배수 m 의 순서 (matrix $\ll$ deep-MLP $\ll$ +momentum $\ll$ +poly) 는 load-bearing 하다 (실험 E4).

class (m)	340M ($d=1024$)	760M ($d=1536$)	1.3B ($d=2048$)
matrix, GDN ($m=1$)	2.10	4.72	8.39
deep-MLP ($m=8$)	16.78	37.75	67.11
+momentum ($m=16$)	33.55	75.50	134.22
+poly, Atlas dir. ($m=24$)	50.33	113.25	201.33
attention ($m=0$)	—	—	— (KV $\propto$ 문맥)

이 B_{state} 가 왜 일급 scaling 축인가. 18 장 claim 1 이 답했다: decode step 은 token 마다 이 상태 전체를 읽고 되쓰므로 (RMW), 이동트래픽이 $2B_{\text{state}}$ 이고, 그것이 decode step 의 GEMV 연산을 약 $394 \times$ 압도한다 — **step 비용이 곧 state 트래픽이다**. 그러므로 (19-1) 은 단순한 memory 회계가 아니라 decode **cost** 의 **scaling** 축이다. 그 축을 따라 트래픽이 어떻게 자라는지가 표 19-3 이다.

표 19-3 — RMW 트래픽 (read + write, per token) 의 폭 scaling. 문맥 길이에 무관하고 모델 폭에 $\propto m d^2 L_{\text{layer}}$ 로 자란다 (실험 E3, tag: REL·M-ASSUMED).

스케일	Titans-170M	Titans-340M	Titans-760M	neural-mem-1.3B	hypo-7B	hypo-70B
RMW GB/token	0.45	1.61	3.62	6.44	34.36	343.6

그림 19-1 — KV cache 읽기 트래픽은 문맥 길이에 비례해 자라고 TTT state RMW 는 문맥에 무관하게 고정이므로 crossover 문맥 S^* 가 존재한다. 오른쪽 panel 의 S^* 스케일링 (모델 폭에 따른 이동) 이 이 장의 load-bearing 결과다 (실험 E1.3/E3 재구성).

[해설] state-bytes 가 memory-centric 논증이 정직하게 서는 두지점 중 첫째 (§ 18.3 의 decode RMW 트래픽) 와 정확히 겹친다는 점이 중요하다. (19-1) 은 “이 모델이 새 메모리 소자를 요구한다” 는 주장이 **아니다** — 그것은 dossier 가 FORCED 로 금지한 지점이다. (19-1) 이 말하는 것은 좁고방어 가능하다: 서빙 상태의 크기가 문맥이 아니라 폭² 에 스케일하고, 그 상태가 매 token RMW 되므로, decode cost 의 scaling 변수는 tokens D 가 아니라 $B_{\text{state}} \propto d^2 L_{\text{layer}}$ 라는 것이다. 이것이 params·tokens 축계에 없던 세 번째 sizing 숫자다.

19.5 후보 법칙을 실제로 fit 한다 — E4 정량백본

지금까지는 축을 제안했다. 이제 그 축들을 § 19.3 의 setting_group 규칙 안에서 공개 점에 **실제로 fit 한다** (실험 E4: digitize 한 data.csv + scipy fit). 여기서 나오는 네 묶음의 결과가 이 장의 정량 백본이며, 그중 하나 (Group B/C) 가 A3 의 핵심 질문에 직접 답한다.

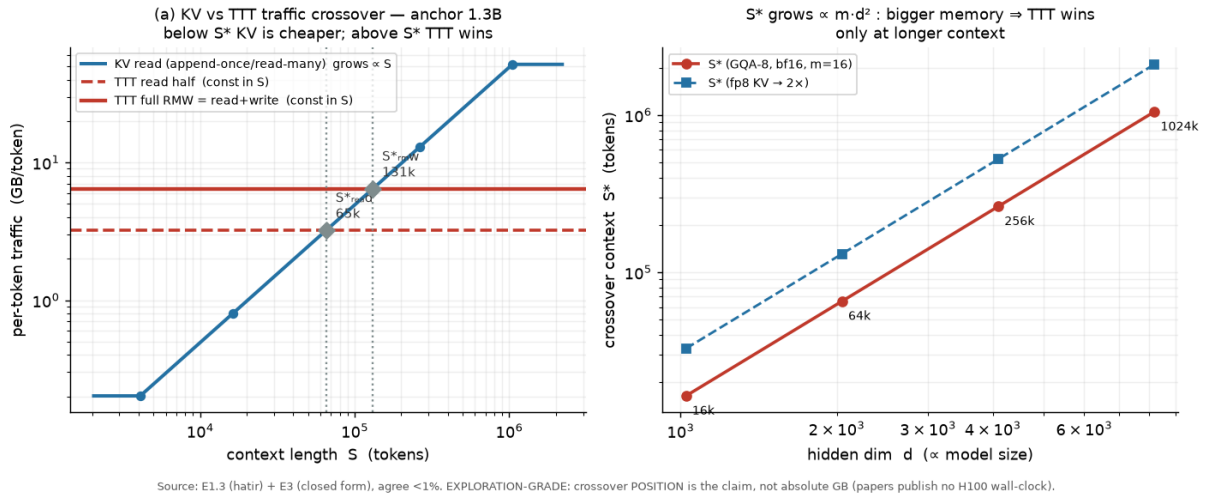


그림 19-1 — KV vs TTT 트래픽 crossover와 그 scaling: 문맥에따라 자라는 KV 읽기 vs 폭에 따라 자라는 TTT RMW, 그리고 crossover 문맥 S^* 의 스케일 이동

(A) **within-line ppl-vs-scale: 세로 fit 과 N,D confound.** Miras 가 FineWeb-Edu·WikiText ppl 로 한 setting_group 안에서 낸 두계열 (Moneta, GDN) 을 fit 했다 (표 19-4). 두 계열 모두 세로로 깔끔하게감소하지만, § 19.3 에서 못박은 대로 params 와 tokens 가 함께 자라므로 params-only exponent 는 **descriptive** 이고, 정직한 축은 compute $C \approx 6ND$ 다.

표 19-4 — within-line(FineWeb-Edu, 동일 tokenizer) ppl-vs-scale fit. exponent 는 3 점 fit 이라 식별 불가에 가깝고, compute 축이 honest axis 다 (실험 E4 Group A).

계열	Wiki ppl (340M / 760M / 1.3B)	params-only 지수 b (r^2)	compute($6ND$) 지수 b (r^2)
Moneta (deep-MLP+poly)	26.19 / 21.18 / 15.52	0.380 (0.951)	0.162 (0.996)
GDN (matrix)	27.01 / 21.18 / 16.42	0.366 (0.984)	0.153 (0.999)

읽는 법: compute 축의 r^2 가 params 축보다 높다는것 (0.996–0.999 vs 0.951–0.984) 은 우연이 아니다 — N 과 D 가 co-scale 하는 계열에서는 둘을 곱한 compute 가 실제 driver 에 더가깝다. 그러나 3 점 fit 의 exponent 절대값 (0.15–0.16) 은 정직성계약상 load-bearing 하지 않다. load-bearing 한 것은 두 축을 모두 fit 해 params-only 지수가 순수 param law 가 아님을 드러낸 방법론적결론이다: 이 라인의 공개 점만으로는 (19-2) 의 α, β 를 분리할수 없다.

(B) **cross-architecture ppl at fixed 1.3B: capacity 는 ppl 축을더하지 못한다.** 이제 A3 의 결정적 검정이다. Atlas Table 2 는 고정 1.3B, 동일 T5 tokenizer·100B tokens 에서 여러 architecture 를 나란히 켜다 — 즉 setting_group 이 하나로 묶이는, cross-architecture 비교가 legal 한 드문 지점이다. 여기서 parametric family(matrix / deep-MLP / deep-MLP+poly) 의 ppl 을 두 축 — raw state-bytes 와 capacity-ordinal(1→2→3) — 에대해 Spearman 으로 검정 하면:

- state-bytes ↔ ppl: $\rho = -0.949$ ($n=4$)
- capacity-ordinal ↔ ppl: $\rho = -0.949$ ($n=4$)

두 ρ 가 정확히 같다. parametric family 안에서 capacity-ordinal 과 state-bytes 는 rank 가 동일하게 정렬되므로, **capacity 는 ppl 예측에 state-bytes 위로 아무것도 더하지 못한다.** 더 결정적인 것은 attention corner(Transformer++ 18.53, Dot 15.28, DeepTransformers 15.67 — 모두 $m=0$, capacity-ordinal 4) 를 넣었을 때다:

- state-bytes ↔ ppl (attention 포함, $m=0$): $\rho = -0.692$ ($n=7$) — 부호 · 방향 유지

- capacity-ordinal $\leftrightarrow$ ppl (attention 포함): $\rho = +0.094$ ($n=7$) — 부호가 뒤집힌다

attention 은 capacity 가 무한 (ordinal 4) 인데 ppl 은 오히려 나쁜 측에속한다. 그래서 capacity 를 ppl 측으로 쓰면 attention 을 “capacity 최고 $\rightarrow$ ppl 최고” 로 잘못 예측한다. **ppl 은 raw retrieval capacity 가 아니라 압축을 보상하기 때문이다.** 결론은 날카롭다: capacity-proxy 는 ppl 측에서 잉여일 뿐 아니라, attention 이 그림에 들어오면 **적극적으로 오도한다.**

(C) **long-context retention vs capacity: capacity 가 유일하게 값을하는 측.** 그러나 같은 capacity 측을 다른 target — BABILong 의 sustained-accuracy 길이 (order-of-magnitude bin) — 에 대면 그림이반전된다. 이 비교는 cross-paper 라 continuous fit 이 아니라 ordinal ρ 로만 읽지만 (§ 19.3), 방향은 뚜렷하다:

- capacity-ordinal $\leftrightarrow$ retention-length: $\rho = +1.000$ ($n=5$, parametric family)
- state-bytes $\leftrightarrow$ retention-length: $\rho = +0.333$ ($n=4$)

가장 선명한 단일 증거는 Titans_MAC $\rightarrow$ Atlas_MAC 의 도약이다: state-bytes 는 $1.5 \times$ (deep-MLP $\rightarrow$ +poly) 밖에 안 늘었는데 retention 길이는 $\sim 6.7 \times$ (1.5M $\rightarrow$ 10M tokens) 뛴다. raw byte 증가가 예측하는 것보다 훨씬 큰 이 도약을, capacity-class 변화 (ordinal 2 $\rightarrow$ 3) 가 설명한다. 즉 **retention 이 바로 capacity 가 raw state-bytes 위로 예측력을 더하는 그 측이다** (attention 은 여기서 별도로 training-context 에 묶여 있다 — [Atlas]/[NL] 이 스스로 인정한 retrieval gap, 200K 근방 포화).

(D) 보조 검정 — state 양의 monotone 효과와, scaling 으로 오독하면 안 되는 U-curve. 두 개의 within-setting 보조 결과가 위 그림을 조인다. 첫째, TNT(150M, 동일 setting) 에서 local memory 수를 0 $\rightarrow$ 4 로 늘리면 avg ppl 이 23.53 $\rightarrow$ 21.04 $\rightarrow$ 20.74 $\rightarrow$ 20.47 $\rightarrow$ 20.15 로 단조감소한다 ($\rho = -1.0$) — state 를 더하면 품질이 오른다는 방향을 within-setting 으로 확인하되, gain 은 포화한다 (+1 에서 +4 까지 21.04 $\rightarrow$ 20.15, diminishing returns). 둘째 **경고**: TNT Fig 2 의 550M Titans(train $C=64$) 에서 inference chunk 를 바꾼 ppl 은 $C=8$ 의 36.45 에서 train-matched $C=64$ 의 13.78 로 내려갔다가 $C=512$ 의 22.40 으로 다시 오르는 **U-curve** 다. 이것은 scaling law 가 **아니라** train/serve chunk resolution mismatch(TNT Challenge 3) 이며, 최소가 스케일이 아니라 train chunk 에 걸린다는 사실이 그 증거다. scaling 표에 이 점을 섞으면 안 된다.

그림 19-2 — E4 정량 백본 (2 $\times$ 2). 세로 fit(좌상) 은 params 측의 within-line 기울기와 N,D confound 를 드러내고, 우상 panel 의 rank test 는 ppl 측에서 capacity 가 state-bytes 와 rank-동치이며 attention 을 넣으면 부호가 뒤집힘을 보인다. 좌하 panel 의 retention 도약이 capacity 측이 예측력을 더하는 유일한 곳이며, 우하 panel 의 U-curve 는 (D) 의 경고 — train/serve chunk mismatch 이지 scaling law 가 아니라는 것 — 을 시각화한다. 모든 절대 exponent 는 directional, ρ 순서 · 비율만 load-bearing.

[평가] E4 는 A3 의 falsifier 3(“capacity 가 state-bytes 대비 예측력을 더하는가”) 을 공개 점 위에서 **부분적으로 실행했고**, 답은 target-의존적이다: **ppl 측에서는 더하지 않으며 (오히려 attention 을 넣으면 오도), long-context retention 측에서는 더한다.** 이 검정은 여전히 이질적 tokenizer 의 digitize 된 점 위에서 rank-correlation 으로만 성립하므로 (§ 19.3), 같은 tokenizer · 같은 state-bytes 에서 continuous exponent 를 내는 controlled run 은 § 19.8 로 이월된다. 그러나 falsifier 의 **방향**은 이미 결정되었다: capacity 를 일급 측으로 세우되 그 자리는 quality-일반이 아니라 **retention** 이다.

19.6 빠진 측 둘: capacity 를 일급 측으로 (그리고 A3 가 그것을 어디에 놓는가)

두 번째 빠진 측은 quality 쪽에 있고, 이론은 이미 14 장에 있다. [Atlas] 의 capacity 정리는 memory 가 정확히 저장할 수 있는 linearly-independent (key, value) 쌍의 최대 수를 architecture class 별로 준다: matrix memory 는 $O(d_k)$ (Prop. 1), deep MLP 는 subquadratic(Thm. 1),

E4 — scaling-law digitization + fits (dossier A3): capacity is a long-context axis, not a perplexity axis

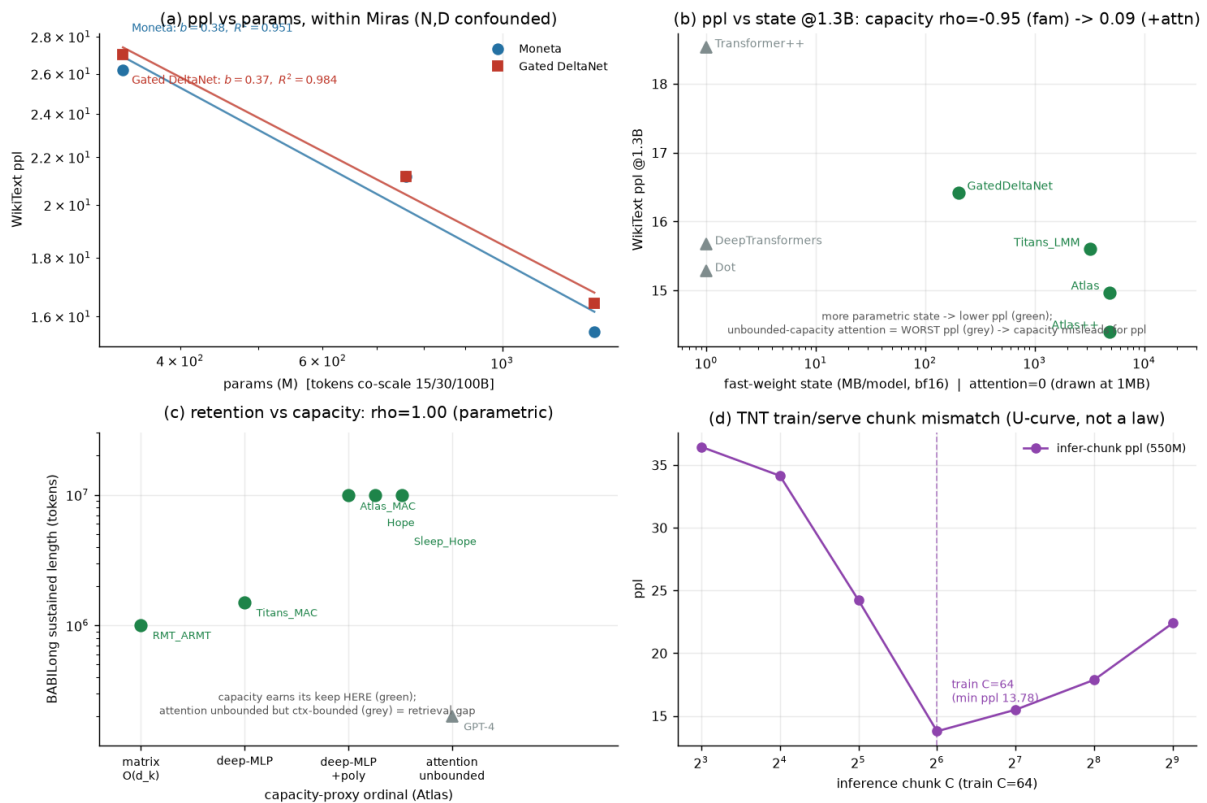


그림 19-2 — E4 scaling fit(2×2 panel): (좌상) within-line ppl-vs-params fit 과 N,D confound, (우상) 고정 1.3B cross-architecture ppl 에 대한 state-bytes·capacity rank test, (좌하) capacity 가 값을 하는 유일한 축인 BABILong retention 과 Titans → Atlas 도약, (우하) TNT train/serve chunk mismatch 의 U-curve(scaling law 가 아니라 resolution mismatch)

degree- p polynomial feature 는 $O(d_k^p)$ (Prop. 2), 그리고 exponential map ϕ^* 의 softmax attention 은 무한 ([Atlas § 3]). 이 사다리는 architecture 를 quality 잠재력 순으로 정렬한다 — params 나 state-bytes 로는 보이지 않는 순서다.

capacity 를 params·tokens·state-bytes 와 나란한 네 번째 축으로 제안한다 — 단, § 19.5 가 정한 자리에서. 근거는 이것이 두 관측을 동시에 설명하기 때문이다. 첫째, 미해소 retrieval 격차 (§ 18.2 유보 2): attention 이 in-context recall 에서 앞선다 (53.55 vs 43.70 [Atlas Table 5]; FDA 67.3 vs 41.9 [NL]). capacity 사다리가 그 이유를 준다 — ϕ^* 의 무한 capacity 대 fixed-state 의 유한 capacity. 둘째, feature map 을 커면 **LM perplexity 가 개선되는 ablation** ([Atlas] 의 “w/o Polynomial Mapping” 22.14 → full 19.97 ppl) 은 capacity 축을 따라 움직인 것이다 — 단 poly mapping 은 capacity 뿐 아니라 표현 · 최적화 조건도 함께 바꾸므로 이 ablation 만으로 capacity 효과를 단독 분리하지 못하고, § 19.5(B) 가 보였듯 그 ppl 개선을 capacity 의 retrieval 이득으로 읽어서는 안 된다 (capacity 가 예측력을 더하는 곳은 ppl 이 아니라 retention 축이다). 그리고 § 19.5(C) 가 이 둘을 하나의 정량으로 묶었다: capacity-class 변화가 Titans → Atlas 의 $6.7\times$ retention 도약을 설명하고, raw state-bytes($1.5\times$) 는 그것을 underpredict 한다.

핵심 질문은 A3 의 질문이었다: **capacity 는 raw state-bytes 대비 예측력을 더하는가?** 둘은 상관만 있지만 동일하지 않다. degree- p feature 는 capacity 를 $O(d_k^p)$ 로 사지만 그 대가를 memory MLP 첫 layer 의 폭 — 즉 state-bytes — 으로 치른다 ([Atlas], key 차원이 $\sim d_k^p$ 로 부풀어 B_{state} 를 키운다). 따라서 **capacity-per-byte** 는 architecture class 마다 다르다. § 19.5(B)/(C) 의 답은 이제 정량으로 갈린다: 만약 두 memory 가 같은 B_{state} 인데 하나가 poly feature 로 capacity 가 높다면, 그 차이는 **ppl 에서는 보이지 않고 retention 에서만 보인다**. capacity 가 state-bytes 를 넘어서는 무언가를 잡는 곳은 — 오직 그곳은 — long-context retention 이며, 그때만 일급 축 자격이 있다.

[평가] 이 제안에는 정직하게 붙일 별표가 하나 있다. [Atlas] 의 capacity 는 **exact interpolation** (linearly independent key 의 zero inner loss, piecewise-affine 가정) 의 이상화된 proxy 이지, 측정된 retrieval 품질이 아니다 ([Atlas assumptions_and_scope caveat 1]). 즉 capacity 축은 **이론적으로** 근거가 있고 (정리), state-bytes 축은 **경험적으로** 근거가 있다 (claim 1 의 측정된 decode cost). § 19.5 는 이 비대칭을 좁혔지만 지우지는 않았다 — capacity 가 retention 을 rank-예측한다는 것은 확인했으나 ($\rho=1.0$, ordinal), 그 예측이 continuous law 로서 얼마나 tight 한지는 controlled run (§ 19.8) 이 남았다.

19.7 후보 scaling law 의 형태

이제 세 축 (quality, decode cost, crossover) 에 대한 후보 법칙을 **형태로** 진술한다. § 19.5 가 보였듯 이 라인의 공개 점으로 quality exponent 를 cross-architecture 로 fit 하는 것은 불가능하므로, 여기서 제안하는 것은 함수 형태와, 그 안에서 실측이 승격한 scaling **모양**이다.

(a) **Quality law (제안, cross-arch 미 fit)**. Chinchilla 식 additive power law 는 perplexity 자체가 아니라 **loss** $L = \log \text{ppl}$ (per-token cross-entropy) 위에서 성립하므로, class 항을 그 loss 위에 더한다 ($\text{ppl} = \exp L$):

$$L(N, D; \text{class}) = \log \text{ppl} \approx E_{\text{class}} + \frac{A}{N^\alpha} + \frac{B}{D^\beta} \quad (19-2)$$

여기서 irreducible 항 E_{class} 는 기본적으로 데이터/protocol 의 엔트로피 항이 되 architecture class (capacity $O(d_k) / O(d_k^p) / \text{무한}$) 가 그 무한-capacity 극한 손실을, 그리고 어쩌면 exponent 를 이동시킨다고 본다 (capacity 를 별도 항으로 분리하는 것은 § 19.5B 의 이유로 미 fit). 표 19-1 의 세로 개선과 § 19.5(A) 의 within-line fit (r^2 0.95–0.99) 은 (19-2) 의 N, D 의 존성과 정합하지만, 이 정합은 **하나의 setting_group 안에서 descriptive** 일 뿐, class 간 E_{class} 를 fit 으로 분리하는 것은 § 19.5(B) 가 보인 이유 (ppl 에서 class 축이 state-bytes 와 rank-동치 · attention 에서 반전) 로 인해 지지되지 않는다.

(b) **Decode cost law(하한)**. state-bytes 축의 직접 귀결이다. decode 는 memory-bound RMW 이므로 (claim 1):

$$t_{\text{dec}} \gtrsim \frac{2 B_{\text{state}}}{\text{BW}} = \frac{2 m d^2 L_{\text{layer}} s}{\text{BW}} \quad (19-3)$$

즉 per-token decode 비용은 tokens D 에 무관하고 폭 2 에스케일한다 — quality law (19-2) 와 다른 축을 따른다. anchor 에서이 하한은 1.92 ms/token(HBM3 3.35 TB/s) 이지만, 이 절대 값은 roofline 하한이고 A100 runbook 으로 이월된다 (§ 18.6). 본문이 덮는 것은 우변의 구조 ($\propto m d^2 L_{\text{layer}}$, memory-bound) 뿐이다.

(c) **Crossover law(load-bearing scaling)**. KV 와 TTT 가 갈리는 문맥 S^* 의 폭 scaling 이다 (claim 2):

$$S^* \propto \frac{m d^2}{d_{kv}} \cdot \frac{s_{\text{ttt}}}{s_{kv}} \quad (19-4)$$

이 scaling 의 위치가 이 장의 가장 단단한 정량 결과다 (표 19-5). 단 S^* 에는 두 문턱이 있다 — KV read 가 TTT **read-half** 와갈리는 read-crossover(anchor 65k) 와, KV read 가 TTT **full RMW(read+write)** 와 갈리는 full-cost crossover(anchor $\approx$ 131k, 약 2 배) 다 (E1.3 의 $S_{\text{read}}^*/S_{\text{rmw}}^*$; $\rightarrow$ § 23.8). read-crossover 아래에서는 KV cache 가 더 싼 메모리 시스템이고, full-cost crossover 위에서는 TTT state 가 더 싸며, 그 사이는 회계에 따라 갈리는 혼합 구간이다 (표 19-5 는 read-crossover 의 폭 scaling 이고 두 문턱은 폭에함께 비례 이동한다).

표 19-5 — read-crossover S^* 의 폭 scaling(GQA-8, bf16 TTT vs fp16 KV; 실험 E3/E1.3, tag: REL·M-ASSUMED). fp8 KV 는 S^* 를 2 배로, MHA 는 선형으로 축소한다.

스케일	Titans-340M	Titans-760M	neural- mem-1.3B	hypo-7B	hypo-70B
S^* (tokens)	16,384	36,864	65,536	262,144	1,048,576

[해설] (19-2) – (19-4) 를 나란히 놓으면 pair thesis 가 **scaling law 안에서** 드러난다. quality (19-2) 는 params·tokens·capacity(단 capacity 는 retention 축에서만, § 19.6) 위에 살고, decode cost (19-3) 는 state-bytes 위에 살며, 둘은 서로 다른 변수를 따른다. 그리고 training/prefill 의 cost 는 세 번째 법칙 — chunk C 가 x 축인 roofline(claim 7) — 을 따른다: 같은 알고리즘이 $C=1$ (decode 영역) 에서 $\text{AI} \approx 1$ 로 memory-bound 이다가 C 를 키우면 crossover 를 넘어 compute-bound 로 오른다. **하나의모델, 두 cost 영역, 각자의 scaling law** — 이것이 D4 pair thesis 의 정량적얼굴이다.

그림 19-3 — chunk C 를 키우면 AI 가 오르며 memory $\rightarrow$ compute 영역을넘는다. host 측정의 crossover $C^* \approx 32$ 는 **모양만**이전되고, H100 twin 의 closed-form C^* 는 306 – 430 이다 (실험 E2.1, tag: CPU-SHAPE).

19.8 정직 평가: A3 falsification 조건 대 실제판정

state-bytes 와 capacity 를 일급 축으로 제안했고, § 19.5 에서 그제안을 공개 점 위에서 **검정**했다. 정직하려면 이 제안이 무엇에의해 반증되는지를 못박고, 그 각각에 대해 E4 가 실제로 무엇을 내놓았는지를대조해야 한다. dossier § 5 의 A3 falsifier 를 이 장의 구체로 옮기고, 옆에판정을 붙인다.

1. 공개 점의 이질성 $\rightarrow$ **cross-architecture quality fit 불가능**. 판정: 발효 (confirmed). § 19.3 이 규정했듯 Titans(Llama-2) 와나머지 (T5/32K) 는 다른 tokenizer 이고, ppl 은 분모가 달라좌표계가 다르다. 따라서 (19-2) 의 exponent α, β 를 여섯 편의공개 점만으로 식별하는 것은 불가능하다 — 지지되는 것은 setting_group 안의 descriptive fit (§ 19.5A)

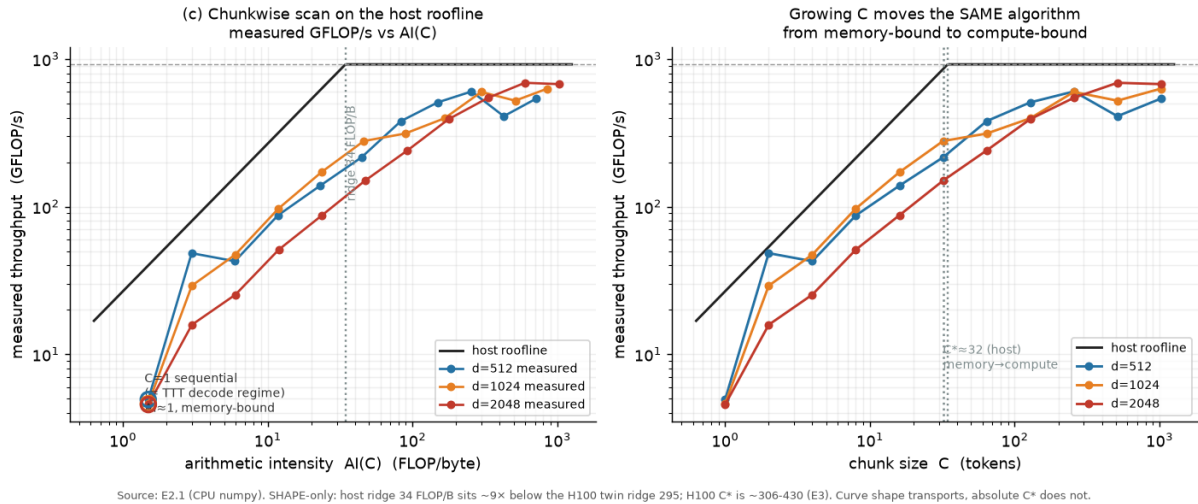


그림 19-3 — chunk C 가 roofline 의 x 축: $C=1$ 의 memory-bound 평원에서 C 를 키우면 compute-bound 로 넘어가는 곡선 (host 측정, 모양만 이전)

과, 정렬된 곳의 ordering(Titans-1.3B 56.82 앵커 위의 Hope/Atlas++ 58.0 대) 뿐이다. 이 falsifier 는 이미 발효되어 있고, 그래서 이 장은 cross-arch quality exponent 를 내지 않는다.

2. 자체 run 이 continuous exponent 를 주기엔 너무 작다. 판정: 부분 이월 (carried). Part III 는 cost model 과 micro-benchmark 이지 from-scratch training 이 아니다 — E4 는 3 점 · 4 점 setting_group 의 digitize 된 공개 점을 fit 했지, 3-4 스케일 $\times$ 3 architecture class 의 controlled scaling run 을 새로 돌리지 않았다. 따라서 (19-2) 의 continuous fit 은 그런 run 이 나올때까지 이월되며, E4 가 낸 것은 exponent 가 아니라 rank-correlation(ρ) 과 within-line descriptive fit 이다.
3. capacity 가 state-bytes 대비 예측력을 못 더하면, capacity 측은 재명명일 뿐이다. 판정: target-분할된 응답 — ppl 에서 반증, retention 에서 지지. 이것이 결정적 falsifier 였고, § 19.5(B)/(C) 가 digitize 된 점 위에서 실행했다. 결과는 이분된다. ppl 측에서는 parametric family 안에서 capacity-ordinal 과 state-bytes 가 rank-동치 ($\rho = -0.949$ 동일) 이고, attention 을 넣으면 capacity 가 부호반전 ($\rho = +0.094$) 하여 오도한다 — 즉 ppl 에 관한 한 falsifier 3 는 발효되어 “capacity 는 state-bytes 의 그림자 (더 나쁘게는 오도하는 그림자)” 다. 그러나 long-context retention 측에서는 capacity 가 raw state-bytes 를 명확히 이긴다 ($\rho = 1.0$ vs 0.333 ; Titans $\rightarrow$ Atlas $6.7\times$ 도약을 $1.5\times$ byte 증가가 underpredict). 완전한 controlled run(같은 state-bytes 에서 capacity-proxy 가 continuous law 로 얼마나 예측하는가) 은 여전히 남지만, 방향은 결정되었다.

[평가] 세 falsifier 의 판정을 합치면 이 장의 A3 입장이 이전보다 날카로워진다. state-bytes 측은 살아 있다 — (19-1) 은 정의이고, 그 위의 decode cost scaling(표 19-3) 과 crossover scaling(표 19-5) 은 세 방법이 anchor 에서 1% 이내로 합치한 측정된 모양이다 (REL-M-ASSUMED tag 아래 load-bearing). capacity 측은 미결에서 부분 지지 · sharply-scoped 로 이동했다 — E4 가 이론 (Atlas 정리) 과 데이터 사이를 좁혀, capacity 가 예측력을 더하는 자리는 quality-일반이 아니라 retention 임을 rank 로 못 박았다. 정직한 최종 판정은 이렇다: state-bytes 는 params·tokens 옆의 일급 cost 측으로 승격되고, capacity 는 일급 long-context/retention 측으로 승격되며 ppl 측으로 쓰는 것은 명시적으로 금지된다 (attention 에서 오도). cross-arch continuous exponent 와 controlled same-state-bytes run 은 이월된다. capacity 를 “quality 일반의 확정 측” 으로 파는 것 — 그리고 ppl-vs-capacity 를 cross-architecture 로 fit 하는 것 — 이 이 장이 피하는 과장이다.

19.9 systems 합의: sizing 숫자가 둘에서셋으로

이 장의 실천적 결론은 fleet sizing 의 산수를 바꾼다. transformer 를서빙하는 엔지니어는 두 숫자로 계획한다: params(weights 상주분, 고정) 와 문맥 길이 (KV cache, 세션마다 자람). memory-centric 모델은 세번째 숫자를 강제한다 — $B_{\text{state}} \propto m d^2 L_{\text{layer}}$ (per-session, 문맥 무관, 매 token RMW). 이 숫자는 KV cache 의 자리를 대신하되 성격이정반대다: 문맥이 아니라 폭으로 자라고, capacity-bound 가 아니라 **bandwidth-bound** 이며 (claim 3: 10 ms/token 에서 B_{max} 가 1.3B 에 5.2 sequence 로 대역폭이 먼저 막힘 — 23 장), on-chip 상주 여부가 설계 knob 이다 (claim 4 의 residency crossover $d^* \approx 2896 - 20$ 장).

세 번째 숫자가 폭² 에 스케일한다는 (19-1) 의 귀결은 배포에직접적이다: 7B 로 가면 RMW 가 34 GB/token, S^* 가 262k token 으로, 70B 에서는 344 GB/token, S^* 가 1.05M token 으로 이동한다 (표 19-3·19-5). 즉모델이 커질수록 (a) decode 가 더 깊이 memory-bound 로 밀리고, (b) TTT state 가 KV cache 를 이기는 문맥 문턱이 더 높아진다 — 큰 모델일수록더 긴 문맥에서만 state 방식이 유리해진다.

그리고 § 19.5 가 sizing 에 하나를 더 얹는다: 어느 축을 어느 목표에써야 하는지다. class 를 고를 때 ppl 만 보면 capacity 사다리느잉여이고 (state-bytes 가 이미 rank 를 정한다), 심지어 attention 을 후보에넣으면 capacity 기준은 잘못된 선택으로 이끈다. 반대로 목표가 **long-context 유지** 라면 — 이 라인이 겨냥하는 바로 그 시장 — class(capacity ordinal) 가 state-bytes 보다 나은 예측자다: 같은 GB/layer 라도 poly feature 로 class 를 올리면 retention 이 byte 증가분보다 크게 오른다. 즉 sizing 결정은 “params + state-bytes” 의 두 cost 숫자에, 목표가 retention 일 때 켜지는 **capacity-class** 라는조건부 quality 축을 덧붙인다. 이 세 (+조건부 한) scaling 이 23 장의 대규모 serving 투영과 24 장의 HW proposal 이 달는 정량 바닥이다.

요약

- 이 라인의 scaling 은 params·tokens 두 축으로 기술되지않는다. cost 쪽에 **state-bytes** ($B_{\text{state}} = m d^2 L_{\text{layer}} s$, 식 19-1), quality 쪽에 **capacity** ($O(d_k)/O(d_k^p)/\text{무한}$, → 14 장) 라는 두 독립 축이 빠져 있다. class 별 m (matrix 1 · deep-MLP 8 · +momentum 16 · +poly ~24 · attention 0) 이 state-bytes 를 params 와 분리한다.
- fit 의 legal 한 경계는 **하나의 setting_group** (같은논문 ·tokenizer·data·metric) 뿐이다. tokenizer 가 다르면 ppl 의 분모가 달라 좌표계가 다르므로, 표 19-1 의가로줄을 exponent 로 승격할 수 없다. 정당한 대조는 setting_group 내세로 fit, 논문 내 재실행 baseline, 그리고 Titans-1.3B avg 56.82 앵커뿐이며, tiny-N 에서는 Spearman ρ 가 honest 통계다.
- E4 가 공개 점에 후보 법칙을 실제로 fit 했다: within-line ppl 은 compute($6ND$) 축에서 r^2 0.996–0.999 로 fit 되나 N, D confound 로 param exponent 는 식별 불가 (descriptive). cross-arch(고정 1.3B) 에서 capacity 는 ppl 에 대해 state-bytes 와 **rank-동치** ($\rho = -0.949$) 이고 attention 을 넣으면**부호 반전** ($\rho = +0.094$) 해 오도한다. 반면 BABILong retention 에서는 capacity 가 state-bytes 를 이긴다 ($\rho = 1.0$ vs 0.333; Titans → Atlas 6.7× 도약을 1.5× byte 가 underpredict).
- 후보 법칙은 셋이다: quality (19-2, 제안 ·cross-arch 미 fit), decode cost 하한 (19-3, 구조 만 load-bearing · 절대값 A100 이월), crossover (19-4, 위치가 load-bearing). RMW 는 폭에 따라 0.45 → 6.44 → 343 GB/token, S^* 는 16k → 1.05M token 으로스케일한다.
- pair thesis 가 scaling law 안에서 드러난다: quality 는 params·tokens·(retention 목표 제한해) capacity 위에, decode cost 는 state-bytes 위에, training/prefill cost 는 chunk C 의 roofline 위에 산다 — 하나의 모델, 두 (세) cost 영역, 각자의 법칙.
- A3 판정은 sharply-scoped 다: **state-bytes** 축은 측정된 모양으로살아 있고, **capacity** 축은 미결에서 부분 지지로 이동해 **long-context/retention** 축으로 승격되되 ppl 축으로 쓰는것은 금지된다 (attention 에서 오도). continuous exponent 와 controlled same-state-bytes run 은 이월된다.

자가 점검체크리스트

- 표 19-1 의 세로 비교와 가로 비교의 정당성을, tokenizer 가 ppl 의 분모를바꾼다는 근거와 setting_group 규칙으로 구별할 수 있다.
- within-line ppl-vs-params fit 이 왜 순수 param law 가 아닌지 (N, D co-scale), 왜 $\text{compute}(6ND)$ 축이 honest axis 이고 tiny-N 에서 왜 ρ 를 쓰는지 설명할 수 있다.
- state-bytes 가 왜 params·tokens 와 독립인 일급 cost 축인지, class 별 m 과 decode RMW 가 어떻게 연결되는지 (claim 1) 설명할 수 있다.
- E4 의 A3 검정 결과를 target 별로 진술할 수 있다: capacity 가 ppl 축에서 state-bytes 와 rank-동치이고 attention 에서 반전하지만, retention 축에서는 state-bytes 를 이긴다는 것을.
- Atlas 의 capacity 사다리 ($O(d_k) \rightarrow O(d_k^p) \rightarrow \text{무한}$) 를 quality 축으로 옮기되, 그것이 왜 ppl 이 아니라 retention 에서만 state-bytes 위로 값을하는지 (capacity-per-byte) 말할 수 있다.
- 세 후보 법칙 (19-2) – (19-4) 에서 무엇이 load-bearing 이고 무엇이 이월되는하한·미 fit 인 지 판별할 수 있다.
- S^* scaling(16k $\rightarrow$ 1.05M) 과 RMW scaling(0.45 $\rightarrow$ 343 GB/token) 이 폭에 따라어떻게 자라는지, 그것이 큰 모델 서빙에 무엇을 뜻하는지 설명할 수 있다.
- A3 의 세 falsification 조건과 그 각각의 실제 판정 (발효 / 이월 / target-분할) 을 대조하고, capacity 축의 남은 자격이 어느 controlled run 에 걸려 있는지 지목할 수 있다.
- 이 장의 결론을 inference 어휘로 옮길 수 있다: sizing 숫자가 {params, 문맥-KV} 둘에서 {params, state-bytes} 둘 + 문맥-무관으로 바뀌며, 세번째 숫자는 폭 $^2 \cdot \text{bandwidth-bound}$ 이고, retention 이목표일 때만 capacity-class 가 네 번째 축으로 켜진다.

다음 장으로

이 장은 state-bytes 를 일급 cost 축으로 세우고 decode cost 가 그 위에서 memory-bound 로 스케일함을 (19-3) 으로 보였다 — 그러나 “memory-bound” 의절대값은 하한으로 남겼다. 20 장은 그 하한 아래로 내려가, 아키텍처 class 별 decode roofline 을 실제로 그린다: state read+**write** 가 KV append-read 와 어떻게 갈리는지, decode 에 들어온 backward-pass 가 왜 새 serving primitive 인지, NS-5·deep-memory 의 FLOP density 가 어디에 쌓이는지, 그리고 state placement(그림 b·e) 가 왜 설계 가능한 knob 인지. scaling 축이 가리킨 병목을 hardware 의 언어로 해부하는 것이 다음 장이다.

ch20. 하드웨어 병목 분석: decode roofline과 새 serving primitive

이 장의 목표 — 독자가 이 장을 마치면 (1) TTT 계열 decode step 을 아키텍처 class 별 roofline 위에 올려, 그것이 왜 KV cache 와 다른 종류의 **memory-bound** 인지 (state read+write vs append-read) 수치로 진술할 수 있고, (2) decode 안으로 들어온 **backward pass** 를 하나의 새 serving primitive 로 이름 붙이고, 그것이 오늘의 forward-only 서빙 kernel 에 무엇을 요구하는지 설명할 수 있으며, (3) NS-5-deep-memory 의 FLOP density, chunk kernel 의 C^* , 그리고 MFU 가 실제로 어디로 새는지를 pair thesis 의 두 절반 —decode(memory-centric) vs prefill/training(accelerator) —으로 나누어 배치할 수 있어야 한다. **왜 필요한가** — 18 장은 완성형을 pair thesis 로 내리고 8 개 실측 claim 을 장에 매핑했다. 이 장은 그중 decode 절반의 뼈대 (claim 1·8) 와 state placement (claim 4), 그리고 training 절반의 진입점 (claim 7) 을 roofline 하나 위에서 만나게한다. 여섯 편의 원 논문은 decode wall-clock 을 **하나도** 공개하지 않았고 (§ 6.4 라인 공통 caveat), fused deep-memory kernel 은 존재하지않는다 (dossier § 3.2). 이 장은 그 빈 자리를 roofline·GEMM shape·memory 계층이라는 독자의 모국어로 처음 측정한다.

20.1 Bridge-in: crossover 에서 roofline 으로

18 장은 KV cache 와 TTT state 의 트래픽이 한 문맥 길이에서 교차한다는 것을보였다 (그림 18-1). 그 교차는 “어느 memory 시스템이 더 싸가” 를 문맥길이의 함수로 답했지만, “왜 decode 가 이토록 느린가, 그리고 그 느림을 하드웨어의 어느 축이 결정하는가” 는 아직 답하지 않았다. 그 답은 roofline 위에 있다.

decode step 을 하나의 객체로 본다 — (traffic, arithmetic intensity, bound class) 를 갖는 객체다 (→ 10 장의 roofline 도구). 이 장의 첫 주장은 단정적이다: **이 라인의 decode step 은 아키텍처 class 와 무관하게 결정적으로 memory-bound 이며, 그 bound 를 만드는 것은 fast-weight state 전체의 per-token read-modify-write(RMW) 다.** KV cache 가 “context 를 읽는” memory 부하라면, TTT state 는 “state 를 고쳐 쓰는” memory 부하다. 두 부하는 roofline 의 같은 축 (대역폭) 에 걸리지만, 걸리는 방식이 질적으로 다르다. 이 절부터 그 차이를 class 별로 편다.

먼저 표기를 환기한다 (→ § 0 요약 카드). fast-weight state 는 W_t , momentum buffer 는 S_t , state multiplier (state 가 d^2 의 몇배인가) 는 m , hidden 차원 d , layer 수 L_{layer} (sequence 길이 L 이 **아님**), chunk 크기 C , inner learning rate η_t , Newton-Schulz κ 회 반복은 NS_κ 다. 이 장의 모든 절대 수치는 § 18.6 정직성 계약 아래에서만 읽는다: **비율·crossover·bound class 만 load-bearing 이고, 절대 $\mu\text{s}/\text{token} \cdot \text{mJ}/\text{token}$ 은 roofline 하한 (A100 runbook 으로 이월) 이며, novel twin(scratchpad·PIM) 은 directional DSE 다.**

20.2 아키텍처 class 별 decode roofline

decode 의 비용은 그 step 이 옮기는 byte 로 결정된다. TTT 계열의 decode 는 token 마다 fast-weight state W_t (그리고 momentum 을 쓰면 S_t 까지) 를 읽고, 갱신하고, 되쓴다. 이 RMW 트래픽은 layer 마다 state 크기 $m d^2$ 의 두 배 (read $m d^2$ + write $m d^2$) 이며, L_{layer} 개 layer 에 걸쳐 누적된다 (원소당 s byte). anchor 설정 neural-mem-1.3B ($d=2048$, $m=16$, $L=24$, GQA-8, bf16) 에서이 값은 **6.44 GB/token** 이고 (실험 E1.1/E3/plan 세 방법이 이 RMW 트래픽에서 1% 이내 합치), arithmetic intensity 는 **0.59 FLOP/byte**(E3의 단순 FLOP 회계 — E1.1의 GEMV-inclusive 회계로는 2.25 이며, 두 값 모두 H100 twin ridge 295 FLOP/byte 보다 두 자릿수 아래) 로 결정적으로 memory-bound 다. 그 결과 state 트래픽이 decode step 의 GEMV 연산을 **약 $394\times$ 압도한다** — 즉 **step 비용이 곧 state 트래픽이다**. bound class 는 memory 로 결정적이다.

state multiplier m 은 아키텍처 class 가 정한다. § 18.6 의 canonical shape(M-ASSUMED) 를 따르면: matrix/linear memory 는 순수형 $m \approx 1(d \times d$ 상태 하나), momentum 을 얹으면 $m \approx 2$, 표준 deep memory(2-layer residual MLP, $\rightarrow 12$ 장) 는 $m \approx 8(8d^2)$, 여기에 momentum 을 더한 Titans-LMM 은 $m \approx 16(16d^2, \text{ anchor})$, Hope 의 6-memory 블록 ($\rightarrow 16$ 장) 은 상주상태 총량이 더 크다. 표 20-1 은 이 m 사다리를 anchor 폭 ($d = 2048$, $L_{\text{layer}} = 24$, bf16) 에서 RMW/token 으로 옮긴것이다.

표 20-1 — 아키텍처 class 별 per-token decode RMW 트래픽 (anchor 폭 $d = 2048$, $L_{\text{layer}} = 24$, bf16; state= $m d^2$ 원소, RMW= $2 m d^2 L_{\text{layer}} s$ byte — s 는 dtype byte, bf16 이면 $s=2$). 절대 GB 는 M-ASSUMED canonical shape 에서 유도한하한이며, load-bearing 한 것은 class 간 순서와 전 class 가 memory-bound 라는 **bound 판정**이다. matrix 행은 momentum 포함형 ($m \approx 2$) 이고, 순수 matrix=GDN 은 $m \approx 1(\approx 8.4 \text{ MB/layer})$ 로 § 19.4· § 20.2.1·E4 와정합한다. Hope 행은 **상주 상태 총량**이 최상단이라는 뜻이며, per-token RMW 트래픽은 가장 빠른 level 이 지배한다 (아래 § 20.2.4).

아키텍처 class	state multiplier m	state/layer	RMW/token	bound
matrix / linear memory (+momentum)	≈ 2	$\approx 17 \text{ MB}$	$\approx 0.8 \text{ GB}$	memory
deep memory (2-layer MLP)	≈ 8	$\approx 67 \text{ MB}$	$\approx 3.2 \text{ GB}$	memory
+ momentum (Titans-LMM, anchor)	≈ 16	134 MB	6.44 GB	memory
Hope (6-memory 블록)	상주 합 최대	상주 최대	fast-level 지배 (나머지저빈도)	memory

핵심은 표의 마지막 열이 **전부 같다**는 것이다. deep-memory MLP 의 forward/backward 는 연산 (GEMV) 과 트래픽 (state RMW) 이 함께 $m d^2$ 로 자라 arithmetic intensity 가 낮은 채 머문다. 단 이 “함께 $m d^2$ ” 기제는 GEMV 계열에 한정된다 — Atlas 의 NS₅ 처럼 $d \times d$ 행렬을 반복 orthogonalize 하는 연산은 그 kernel 만 떼어 보면연산 $O(\kappa d^3)$ · 트래픽 $O(d^2)$ 로 AI 가 $O(d)$ 까지 올라 **고립 kernel** 로는 **compute-bound** 다 (§ 20.4). 그럼에도 $C=1$ decode step 전체는 memory-bound 로 남는데, 그 NS 연산시간이 whole-state RMW 트래픽 시간에 가리기 때문이다 (§ 20.4 마지막 문단). 요컨대 class 를 올려도 decode step 의 bound 판정은 memory 로 유지 되지만 그이유는 class 마다 다르고 — GEMV 계열은 AI 가 낮아서, NS 계열은 트래픽이연산을 가려서 — bound class 는 그대로인 채 총 RMW 트래픽과 latency 만 함께커진다. 이것이 decode 절반이 “새 memory 기회” 인 첫 번째 구조적 이유다: capacity 를 늘리려 memory 를 깊게·무겁게 만들수록 decode 는대역폭 벽에 더 세계 부딪힌다.

이 벽은 폭과 함께 자란다. RMW 트래픽은 문맥 길이 S 에 무관하고 모델 폭에 따라 $m d^2 L_{\text{layer}}$ 로 자란다: Titans-170M의 **0.45 GB/token**에서 anchor의 **6.44 GB/token**을 거쳐 가상의 70B에서 **343 GB/token**까지다 (실험 E3). KV cache가 문맥에 비례해 read 트래픽이 자라는 것과 정반대로, TTT는 폭에 비례해 RMW 트래픽이 자란다. 두스케일링의 교차가 18장의 crossover 문맥 길이 (S^*)이며, 그 상세는 19장이 scaling 축으로 편다.

[해설] inference 관점에서 이 표를 한 문장으로 옮기면이렇다. **decode step 하나가 매 token마다 약 3.22 GB의 fast-weight state를 읽고 다시 써 6.44 GB를 옮긴다** —bf16 1.3B weights(≈ 2.6 GB) 전체를 매 token 약 2.5번 왕복하는 셈의트래픽이다. KV cache append가 token마다 몇 KB를 덧붙이는 것과 비교하면 —KV write는 read의 0.003%(anchor $S \approx 65k$; 비율은 $O(1/S)$)인 append-once인데 (실험 E1.3), TTT는 write가 read와 같은 대칭 RMW(100%)다 — 이것은 같은 “memory-bound”라는 단어로 부르기가 어려운 다른 부하다. 앞으로 이 장이 “decode가 memory-bound”라고 말할 때, 그것은 KV cache의 memory-bound(read-many, shareable)가 아니라 **whole-state RMW의 memory-bound(write-heavy, unshared)**를 뜻한다.

절대값에 대한 정직성. anchor의 roofline 하한은 **1.92 ms/token, 46.5 mJ/token**이다 (실험 E1.1). 이 두 수는 이 장의 주장이 딛는 근거가 **아니다**. 6편의 원논문이 H100 decode wall-clock을 하나도 공개하지 않았으므로 (§ 6.4), 이절대값은 외부 검증 불가능한 roofline 하한이며 사내 A100 runbook(Part III-a)으로 이월되는 검증 대상이다. 본문이 딛는 것은 그 하한이 드러내는 **구조** — memory-bound, RMW-지배, class-무관 — 뿐이다.

20.2.1 matrix memory: rank-1 write가 만드는 roofline 바닥

표 20-1의 사다리를 class 별 공식으로 내려 보면 “왜 전부 memory-bound인가”가 각각 다른 이유로 성립함이 보인다. 먼저 matrix(linear) memory는 상태가 하나의 $d \times d$ 행렬 W 다. 읽기는 GEMV $y_t = W q_t (\approx 2d^2 \text{ FLOP})$, 쓰기는 delta rule의 rank-1 outer-product $W_t = W_{t-1}(I - \eta_t k_t k_t^\top) + \eta_t v_t k_t^\top (\Theta(d^2) \text{ FLOP}, \rightarrow 6 \text{ 장})$ 다. state 트래픽은 read+write로 $2d^2 s$ byte이고 (momentum buffer를 두면 $2 \cdot 2d^2 s$, 곧 표 20-1의 $m \approx 2$), 한 step의 arithmetic intensity는

$$\text{AI}_{\text{matrix}} \approx \frac{2d^2 + \Theta(d^2)}{2 m d^2 s} = \Theta\left(\frac{1}{m s}\right) \text{ FLOP/byte}$$

곧 d 가 상쇄되어 폭과 무관하게 $O(1)$ 이다 — $m=2, s=2$ 에서 약 0.75 FLOP/byte. 이것이 roofline 평원의 **바닥**이다: rank-1 write는 옮긴 byte당 겨우 한번꼴의 MAC 밖에 하지 않으므로, matrix memory는 이 라인에서 가장 낮은 AI를 갖는 class다. Gated DeltaNet(이하 GDN) 같은 순수 matrix 모델은 momentum 없이 $m \approx 1$ (anchor 폭에서 8.4 MB/layer)이고, momentum을 얹으면 $m \approx 2$ (≈ 17 MB/layer)로 오른다 (실험 E4 state-bytes 표).

20.2.2 deep memory vs matrix: 연산과 트래픽이 함께 $m d^2$ 로 자란다

표준 deep memory는 2-layer residual MLP $\mathcal{M}(z; W) = z + W_1 \sigma(W_2 z)$ 이고, 상태는 두 weight 행렬 $\{W_1, W_2\}$ (expansion 4이면 $W_2 \in \mathbb{R}^{4d \times d}, W_1 \in \mathbb{R}^{d \times 4d}$, 합 $\approx 8d^2, m \approx 8$)다. 여기서 결정적인 것은 read와 write가 **둘 다** 이 $8d^2$ 를 통과한다는 점이다: forward는 두 GEMV($\approx 16d^2 \text{ FLOP}$), write를 위한 backward는 activation을 거슬러 같은 두 행렬을 통과하는 두 GEMV($\approx 16d^2 \text{ FLOP}, \rightarrow \S 20.3$)다. 따라서 연산도 $\Theta(m d^2)$, 트래픽도 $\Theta(m d^2 s)$ 로 **함께**자라고 그 비는 다시 상쇄된다:

$$\text{AI}_{\text{deep}} \approx \frac{\Theta(m d^2)}{2 m d^2 s} = \Theta\left(\frac{1}{s}\right) \text{ FLOP/byte}$$

matrix보다 상수가 조금 크다 (forward+backward가 rank-1 write보다 byte당 MAC이 많다) — E1.1의 GEMV-inclusive AI 2.25가 이 위치다 — 그러나 여전히 $O(1)$ 이고 ridge 295의 두 자릿수 아래다. **deep memory와 matrix memory의 roofline 차이는 “얼마나 memory-bound인가”가 아니라 평원 위에서의 미세한 x-위치뿐이다**: 둘 다 memory-bound 평원에 앞뒤

deep 쪽이 조금 오른쪽 (AI 2.25 vs 0.75) 일뿐, 어느 쪽도 ridge 에 다가가지 못한다. capacity 를 위해 memory 를 깊게만들면 AI 는 거의 그대로인 채 트래픽의 절대량만 m 배로 커지므로, 깊은 memory 는 더 빠른 것이 아니라 더 세계 대역폭 벽에눌리는 것이다. 이것이 § 20.2 도입부의 “class 를 올려도 memory 로 유지” 를 공식으로 되짚은 결과다 — GEMV 계열에서 bound 가 memory 인 이유는 언제나 “연산이 트래픽과 같은 $m d^2$ 로 자라 AI 가 상수에 묶이기 때문” 이다.

20.2.3 Atlas 의 세 얼굴: matrix·poly·exp 는 pair thesis 를 한 축에담는다

Atlas(→ 14 장) 는 feature map ϕ 로 key 를 들어 올려 같은 memory 틀에서 세 개의 다른 class 를 만든다. 이 셋을 roofline 위에 나란히 놓으면 pair thesis 전체가 한 축에 보인다.

- **matrix**($\phi = \text{identity}$). § 20.2.1 그대로, AI 바닥, 고정 d^2 상태.
- **polynomial**(ϕ_p). 차수 p feature map 이 key 를 lifted 차원 $D = \Theta(d_k^p)$ 로 올려 capacity 를 $O(d_k^p)$ 로 키운다 (Atlas capacity 정리, → 14 장). 그러나 memory 는 여전히 고정 크기 MLP 가 그 lifted key 를 흡수하므로 RMW 상태는 폭발하지 않고 $m \approx 24$ 수준 (anchor 1.3B 에서 201 MB/layer, E4 directional) 에머문다 — momentum deep 대비 약 $1.5\times$. roofline 위치는 deep 와 사실상같은 평원이다. 여기서 정직한 관찰: E4 의 retention fit 은 Titans → Atlas 에서 상태 byte 가 $1.5\times$ 늘 때 BABILong 유지 길이가 $6.7\times$ 된다고 본다 (capacity ordinal $2 \rightarrow 3$, cross-paper·ordinal). 즉 poly 는 roofline 의 트래픽 축에서는 거의 공짜 ($1.5\times$) 인데 retention 축에서는 class 를 하나 올린다 — decode 를 더 memory-bound 로 만들지않으면서 capacity 를 사는, 이 라인에서 가장 유리한 교환이다 (단 retention 수치는 cross-paper ordinal 이므로 방향만).
- **exponential**(ϕ^*). exponential feature map 은 $\exp(q^\top k) = \phi^*(q)^\top \phi^*(k)$ 의 극한, 곧 softmax attention = capacity 무한의 associative memory 다 (→ 14 장 · § 1.6). 이 corner 는 고정 크기 fast-weight 상태를 갖지 않는다: 상태가 문맥과 함께 자라는 KV cache 그 자체다. 따라서 Atlas 의 세 얼굴은 pair thesis 의 두 절반을 한 모델 족 안에담는다 — matrix·poly 는 고정-상태 RMW(memory-centric decode), exp 는 문맥-증가 append-read(KV 기준선). 18 장 crossover(S^*) 는 바로 이 두 얼굴이 트래픽에서만나는 문맥 길이다. Atlas 를 roofline 에 올린다는 것은 이 crossover 의 양쪽을 동시에 보는 것이다.

20.2.4 Hope 의 6-memory 블록: 한 token 에 여섯 개의 RMW

Hope(→ 16 장) 의 self-modifying Titans 블록은 main memory 하나만 RMW 하지않는다. NL 은 k, v, q, η, α 투영 자신을 test-time 에 갱신되는 memory 로 바꾸므로 (self-modifying, → 16 장), 한 블록이 대략 여섯개의 associative memory(main + 다섯 projection-memory) 를 품는다. 단여섯이 모두 매 token 고쳐 쓰이는 것은 아니다 — E3 회계는 가장빠른 level 만 매 token RMW($m_{\text{fast}} \approx 8$) 하고 나머지 level 은 더 크지만 낮은 빈도로 갱신되는 상주 상태로 본다 (CMS 의 level 별 update cadence, → § 20.6 · § 23.6). roofline bound 판정은 그대로 memory 지만, 이 class 는 두 가지를 더 요구한다. 첫째, per-token RMW 트래픽은 fast level 이 지배하되 상주 상태총량이 여섯의 합이라 표 20-1 의 상주 축에서 최상단에 앉는다 (E4 에서 Hope 의 유효 상태는 momentum-deep 급) — 이 “상주는 크되 대부분 저빈도” 가 § 20.6 · § 23.6 의 cadence-tier 배치를 블록 내부로 부르는이유다. 둘째, fused decode kernel 이 하나의 state stream 이 아니라 여섯 개의 독립 RMW stream 을 한 step 안에 엮어야 한다 — § 20.3 의 fusion 문제가 여섯 겹으로 복잡해지고, 여섯 상태가 서로 다른 update cadence 를 가질 수 있어 (self-modifying η, α 는 main 보다 느리게변할 수 있다) § 20.6 의 cadence-tier 배치가 블록 내부로 들어온다. Hope 는 그래서 “가장 무거운 decode class” 이자 “cadence 계층을 한 블록안에서 처음 강제하는 class” 다.

20.2.5 batch 축을 더한 memory forward: per-request fast-weight 의 batched 실행형태

지금까지 § 20.2 는 한 request 의 한 decode step 을 봤다. 서빙은 여러 request 를 동시에 흘리므로, 이 RMW step 에 batch 축을 더한 실행형태가 어떤 GEMM shape 인지가 claim 1 의 “왜

이 memory-bound 가 batch 로도 풀리지 않는가” 를 shape 수준에서 정한다. 이 소절은 그 형태를 통일 표기로 편다 (경제학은 § 23.4· § 24.4 가 이미 답았고, 여기서는 그 아래의 kernel shape 를 준다).

먼저 read 다. 한 request 의 memory read 는 $y_t = \mathcal{M}(q_t; W_t)$ 였다 (§ 20.2.1). B 개 request 를 쌓으면 입력이 $X \in \mathbb{R}^{B \times C \times d}$ ($C=1$ 이 decode), per-request 상태가 $M \in \mathbb{R}^{B \times d \times d}$ 이고, 읽기는 batch 축마다 독립인

$$Y[b] = X[b] M[b]^\top, \quad b = 1, \dots, B$$

곧 **batched matmul(BMM)** 이다 — decode 의 $C=1$ 에서는 request 마다 GEMV, prefill 의 $C > 1$ 에서는 request 마다 GEMM(batch 축은 병렬이 되 chunk 축의 token 병렬은 미래 token 이 주어진 prefill/training 에서만 살아 있고 autoregressive decode 에는 없다, → § 20.5). deep memory 에서는 이 BMM 이 두 겹이 된다: 표준 2-layer residual MLP $\mathcal{M}(z; W) = z + W_1 \sigma(W_2 z)$ (§ 20.2.2) 의 batched shape 는 $W_2 \in \mathbb{R}^{B \times d_h \times d}$, $W_1 \in \mathbb{R}^{B \times d \times d_h}$ (d_h 는 memory-hidden 폭, expansion 4 면 $d_h=4d$) 이고, forward 는 request 별 두 BMM 사이에 elementwise σ -residual 을 끼운 것이다.

핵심은 이 BMM 이 **일반 MLP layer 처럼 안 퍼진다**는 데 있다. 오늘의 dense MLP decode 는 모든 request 가 **같은 weight** 를 공유하므로 batch·token 축을 접어 $[B \cdot C, d] @ [d, d_h]$ 한 장의 **shared-weight GEMM** 으로 편다 — weight 가 한 번 읽혀 $B \cdot C$ 개 행에 재사용되고, 그래서 arithmetic intensity 가 B 와 함께 오른다. 그런데 fast-weight memory 는 request 마다 W_b 가 **다르다** (never-average: inner update 는 request 별 map 연산이라 batch 방향 평균이 정보 누설이다, → § 20.3·claim 7). 축을 접으면 서로 다른 W_b 가 한 GEMM 에 섞일 수 없으므로, 실행은 request 별 weight 를 갖는 **strided-batched / grouped GEMM** 으로 파편화된다 — 이것이 § 20.5 가 든 MFU 포획 손실 셋째 원인 (grouped-GEMM 파편화) 의 shape 수준 정체이고, ch24 제안 G(grouped-GEMM decode engine) 가 복원하려는 바로 그 대상이다 (→ 24 장).

[평가] 이 shape 가 claim 1 을 구조로 못 박는다. shared-weight GEMM 에서는 weight byte 가 $B \cdot C$ 번 재사용돼 AI 가 batch 로 오르지만, grouped-GEMM 에서는 각 W_b 가 그 request 에만 쓰여 **한 번 읽고 한 번 써진다** — FLOP 도 traffic 도 함께 B 에 비례하므로 그비인 **arithmetic intensity 가 batch B 와 무관하다** (→ ch24 제안 G 가 명시하는 economics 의 shape 근거). 그래서 batch 를 아무리 키워도 decode step 은 § 20.2 의 memory-bound 평원을 벗어나지 못하고, 대역폭이 배치를 $B_{\max}$ 에서 조기에 닫는다 (→ § 23.4). 요컨대 batching 은 **가능하되** (grouped-GEMM 으로 batch 가 형성됨) shared-weight batching 의 amortization free-lunch 는 **없다** — 이 둘의 구분이 이소절의 요점이자 pair thesis 의 두 절반이 만나는 지점이다: batch 를 형성하는 것은 accelerator 절반 (제안 G) 의 일이고, 그 트래픽 하한을 낮추는 것은 memory-centric 절반 (§ 20.6· 제안 D) 의 일이다.

세 가지 정밀 보정이 이 shape 서술에 붙는다 — 원 형태 그대로 옮기면 구현합정을 부르는 지점이다.

- **update form 의 두 층위.** 위 batched read 의 짝인 write 는 선형 dot-product memory 의 극한에서 닫힌형 $M_t = M_{t-1} A_t - \eta_t \hat{v}_t k_t^\top$, $A_t = \alpha_t I - \eta_t k_t k_t^\top$ 로 떨어진다 (→ 6 장의 delta rule 계열의 선형 특수형). 이것은 **선형·dot-product objective 의 특수형**이지 배포된 2-layer MLP memory 의 update 가 아니다 — 후자는 § 20.3 의 일반 $\nabla_W \ell$ (L2 형이면 residual-gradient 항포함) 을 memory MLP 를 거슬러 계산한다. 게다가 A_t 는 **identity-minus-rank-1** 이라, batched 로 $A \in \mathbb{R}^{B \times d \times d}$ 을 dense 로 materialize 하거나 $M A$ 를 dense BMM 으로 계산하면 낭비다: 효율 구현은 $M k$ 와 outer-product 만 쓰는 factored 형태로 유지한다 (위 dense- A 표기는 shape 이해용일 뿐 구현 지침이 아니다).
- **query 는 static.** batch 축으로 tensorize 되는 것은 self-modifying memory 들이지만, 배포된 Hope 에서 **query projection 은 갱신되지 않는다** — $q_t = x_t W_q$ 가 유일한 non-adaptive projection 이다 (→ 16 장; fully-adaptive q 는 중간 variant 한정). 따라서 batched 상태 목록에서 M_q 는 빠지고, request 별 BMM 으로 흐르는 self-modifying 상태는 $\{k, v, \eta, \alpha, \text{mem}\}$ 계열이다.

- MoE 유비는 **dispatch-form only**. request 별 W_b 를 batch 로 흘려보내는 이 형태는 MoE 의 grouped-GEMM 과 **dispatch 형태**만 닮았다. MoE 는 다수 token 이 소수 shared expert weight 를 재사용해 그 weight 를 상각하지만, 여기서는 request 마다 weight 가 달라 decode 에서 group 당 token 이 사실상 1 이라 amortization 이득이 없다 (→ 24 장 제안 G, claim 10) — 유비는 kernel-dispatch shape 에서 성립하고 efficiency 에서 깨진다.

20.3 decode 에 들어온 backward pass: 새 serving primitive

RMW 의 byte 수는 무엇이 옮겨지는지를 말하지만, 무엇이 계산되는지는 말하지 않는다. 그리고 계산되는 것 안에 이 라인이서빙 시스템에 던진 진짜 새로움이 있다.

독자는 decode 의 forward pass 를 안다: $y_t = \mathcal{M}(q_t; W_t)$, memory 를 읽는 한 번의 전방 통과다. 그런데 TTT 계열의 decode 는 그 다음에 **state** 를 갱신해야 하고, 갱신의 핵심은 inner loss 의 gradient 다 (→ 8 장 · 12 장):

$$g_t^{\text{in}} = \nabla_W \ell(W_{t-1}; k_t, v_t), \quad S_t = \beta_t S_{t-1} - \eta_t g_t^{\text{in}}, \quad W_t = \alpha_t W_{t-1} + S_t \quad (20-1)$$

식 (20-1) 의 첫 항 g_t^{in} 을 계산하려면 memory MLP 를 통과하는 **backward pass** 가 필요하다. deep memory $\mathcal{M}(z; W) = z + W_1 \sigma(W_2 z)$ 의 gradient 는 forward activation 을 거슬러 오차를 전파해 얻는다 — 이것은 훈련의 backward pass 와 같은 연산이다. 요컨대:

[평가] 이 라인이 서빙 엔진에 요구하는 진짜 새 primitive 는 whole-state RMW 의 byte 수가 아니라, **backward pass 가 decode 의 critical path 위로 올라왔다**는 사실 자체다. 오늘의 inference kernel(FlashAttention decode, paged attention) 은 정의상 forward-only 다 — 추론 중 weights 는 변하지 않는다는 불변식 (→ 1 장 Rosetta) 위에 세워졌기 때문이다. 그 불변식이 이 라인에서 폐기되면서, “훈련에만 존재하던” forward+backward 의 합성이 token 마다 한 번씩 서빙 loop 안에서 돌아간다. 이것이 pair thesis 의 decode 절반을 “새 기회” 로 만드는 것의 핵심이며, byte 회계는 그 결과일 뿐이다.

이 primitive 가 kernel 에 요구하는 것은 세 가지다. 첫째, **forward+backward fused decode kernel** 이 필요하다. $\text{read}(y_t)$, $\text{backward}(g_t^{\text{in}})$, $\text{momentum} \cdot \text{retention} \cdot \text{write}$ (식 (20-1) 의 나머지) 를 한 번의 state streaming 안에서 융합해야 memory-bound step 에서 state 를 두 번 이상 왕복하지 않는다. 그런 fused deep-memory kernel 은 현재 존재하지 않는다 (dossier § 3.2; A1 각도의 근거). 둘째, forward activation 을 backward 에서 재사용하려면 그 activation 의 **residency** 가 문제가 된다 (§ 20.6). 셋째, Atlas 의 Muon(→ 14 장) 을 쓰면 backward 뒤에 momentum 행렬을 orthogonalize 하는 NS_κ 반복이 κ 회 더붙는다 (backward inner pass 가 κ 번 반복되는 것이 아니라, gradient 이후의 별도 행렬 반복이다; § 20.4).

[해설] GEMM shape 로 읽으면 (20-1) 의 backward 는 낫설지 않다. linear memory 의 극한에서 $g_t^{\text{in}} = (\text{오차}) k_t^\top$ 는 rank-1 outer-product write 이고 (→ 6 장의 delta rule), deep memory 에서는 그것이 2-layer MLP 를 거슬러 오르는 두 번의 GEMV 가 된다. 문제는 shape 가 낫설다는 게 아니라 **위치**가 낫설다는 것이다 — 이 GEMV 들이 batch 축이 없는 (per-request, unshared) 상태에서 token 마다 순차로 돈다. shared-weight batching 이 깨지는 지점 (→ 23 장) 이 바로 여기서 시작된다.

20.3.1 fused backward-decode kernel: 설계 수준의 요건

이 primitive 를 서빙 kernel 의 설계 명세로 내리면 세 가지가 구체적 숫자를 얻는다.

(1) **fusion** 은 최적화가 아니라 하한과 $4 - 6 \times$ 의 차이다. decode step 의 트래픽 하한은 state 를 한 번 읽고 한 번 쓰는 $2 m d^2 L_{\text{layer}} s$ 다 (§ 20.2). 그런데 이 하한은 forward·backward·momentum·retention·write 가 한 번의 **state streaming** 안에서 융합될 때만 달성된다. 만약

오늘의 조립 방식대로 forward kernel $\rightarrow$ backward kernel $\rightarrow$ update kernel 을따로 띄우면 각 kernel 이 state 를 독립적으로 read/write 하므로트래픽이 3 - 4 배로, 여기에 momentum-retention 갱신까지분리하면 4 - 6 배로 부른다. state 트래픽이 이미 step 비용의 전부 ($394 \times$ 지배, § 20.2) 이므로, **fusion** 은 **latency** 를 몇 % 깎는 튜닝이 아니라 **memory-bound step** 을 **한에 앞히느냐 그 4-6 배 위에 앞히느냐**의 문제다. 그런 fused deep-memory decode kernel 은 존재하지 않는다 (dossier § 3.2).

(2) **activation residency** 는 **훈련 backward** 와 **정반대로 쉽다**. 훈련의 backward 가 어려운 이유는 sequence 전체의 forward activation 을쌓아 뒤야 하는 $O(L \cdot md)$ activation 메모리 때문이고, 그래서 gradient checkpointing 이 필요하다. 그런데 decode backward 는 **한 token** 의 backward 다: 유지할 activation 은 그 token 이 memory MLP 를통과하며 만든 $O(md)$ 크기 (2-layer 면 hidden $4d$ 수준) 뿐이고, sequence 축 activation 스택이 없다. 따라서 이 backward 는 “activation 메모리 문제가 없는 backward” 이며, 어려움은 메모리가 아니라 **critical path** 위의 **직렬 forward $\rightarrow$ backward latency** 와 그것을 state 한 왕복 안에 묶는 fusion 으로 옮겨 간다. 이것은 훈련 무경험 독자에게오히려 반가운 소식이다 — 훈련 backward 의 가장 악명 높은 비용 (activation stack) 이 decode 에서는 문맥 길이 축이 없어 구조적으로 사라진다.

(3) **checkpoint/rollback** 과 **numerics drift** 가 **새 상태이벤트를 만든다**. state 가 매 token 변하므로 speculative decoding 의 rollback 은 KV cache 슬롯의 폐기가 아니라 **fast-weight state** 의 **snapshot/restore** 가 된다 ($\rightarrow$ 23 장; claim 6 의 checkpoint/rollback 이벤트). 게다가 되돌리는대상이 optimizer 궤적 상태 (W_t 와 momentum S_t) 이므로, rollback 이부정확하면 numerics 가 표류해 되돌린 뒤의 궤적이 원본과 갈라진다 — KV cache 의 rollback 에는 없던 종류의 정확성 요구다. 이셋 (fusion-single-token backward-snapshot) 이 forward-only KV decode kernel 에는 이름조차 없던 상태 이벤트이며, claim 6 이 KVCacheManager 에 빠졌다고 짚은 API(update_in_place, checkpoint/rollback, free_on_update) 가 정확히 이 primitive 의 서빙-스택 대응물이다 ($\rightarrow$ 23 장).

20.4 NS-5 와 deep-memory 의 FLOP density: PIM 경계선을긋는다

decode 가 memory-bound 라는 것은 FLOP 이 적다는 뜻이 아니라 FLOP 이 트래픽에비해 적다는 뜻이다. 그 FLOP 이 **어떤 모양인지**가 memory-centric 논증의 정직한 경계를 긋는다.

식 (20-1) 한 step 의 FLOP 은 세 덩어리로 나뉜다. (a) memory MLP 를 통과하는 forward+backward — deep memory 에서 layer 당 몇 개의 $d \times d$ GEMV, class 를 올릴수록 커지는 대부분의 FLOP. (b) Atlas 의 경우 momentum S_t 를 semi-orthogonalize 하는 $NS_\kappa - \kappa = 5$ 가표준이고 각 반복이 행렬 곱 몇 번이므로, 이것도 GEMM-shaped FLOP 을 κ 배로 더한다 ($\rightarrow$ 14 장). (c) elementwise epilogue — momentum decay($\beta_t S_{t-1}$), retention($\alpha_t W_{t-1}$), write 의 AXPY, 그리고 Miras 계열의 renorm($\rightarrow$ 13 장). 이 (c) 만이 memory-side, elementwise, 1 - 3 MAC/elem 의 PIM-shaped 연산이다.

이 분해가 정직성 계약의 한 경계를 만든다. dossier 의 판정 (§ 5, “Forced” 4 번) 을 이 장의 수치로 재확인하면:

[평가] (a) 와 (b) — memory MLP 의 forward/backward 와 NS-5 — 은 **GEMV/GEMM-shaped** 이고, 이 라인이 여섯 편에 걸쳐알고리즘을 dense matmul 로 다시 빚어 온 이유가 바로 이 FLOP 을 tensor-core 에 태우기 위해서다 (chunk anchoring, banded mask, batched NS-5 — $\rightarrow$ 9 장 · 14 장). 따라서 “이 라인의 연산이 일반 PIM 을요구한다” 는 주장은 이 FLOP 분포에 의해 **반증된다** — 대부분의 FLOP 은 PIM 이 못 태우는 GEMM 이다. PIM 이 정당한 지점은 오직 (c) elementwise epilogue 뿐이고, 그것은 전체 FLOP 의 소수다. E1.2 는 이 epilogue 에 대해 PIM 이 TTT 의 ~ 3 MAC/elem 에서 **$1.6 \times$ energy** 이득 (단 latency 는 $2.1 \times$ 손해), rank-1 에 가까운 1 MAC/elem 에서는 energy-latency 둘 다이득임을 보인다 — 그러나 simulation_ready=False 인 directional DSE 로만 인용한다 (§ 18.6).

FLOP density 의 실천적 결론은 decode 절반에서 일관된다. class 를 올려 FLOP 을 늘려도 (deep memory, NS-5) decode **step** 전체의 유효 arithmetic intensity 는 ridge 아래에 머문다

(§ 20.2) — NS-5 kernel 자체는 고립하면 compute-bound($O(d)$ AI) 일 수 있으나, $C = 1$ 영역에서는 그 연산조차 whole-state RMW 트래픽에 가려 step 은 memory-bound 를 벗어나지 못한다. FLOP density 가 문제가 되는 것은 decode 가 아니라 chunk 를 키우는 순간, 즉 prefill/training 영역에서다. 그 전환을 다음 절이 roofline 위에서 본다.

20.4.1 NS-5 의 FLOP density: $O(\kappa d^3)$ 와 $C=1$ 에서의 traffic 지배

§ 20.4 의 (b) 덩어리 — Atlas 의 Muon 이 momentum 행렬을 semi-orthogonalize 하는 NS_κ (→ 14 장) — 는 이 라인에서 FLOP density 가 가장 높은 연산이므로 따로 정밀하게 본다. Newton-Schulz 반복은 $d \times d$ 행렬 S_t 에 대해 $X_{j+1} = aX_j + bX_j X_j^\top X_j + \dots$ 꼴의 **행렬-행렬 곱**을 반복한다 (deep memory 면 weight 행렬마다 NS 가 돌아 상수 인자만 달라진다). 한 반복이 상수 개의 $d \times d$ matmul 이므로 반복당 $\Theta(d^3)$ FLOP, κ 회면

$$FLOP_{NS} = \Theta(\kappa d^3), \quad \text{traffic}_{NS} = \Theta(d^2 s)$$

곧 **고립 kernel 의 arithmetic intensity** 는 $\Theta(\kappa d/s) - d$ 에 선형으로 자란다. anchor 쪽에서 이는 ridge 를 훌쩍 넘어, NS kernel 만 떼어 보면 **compute-bound** 이고 tensor-core 에 태워야 하는 GEMM 이다 (§ 20.4 의 판정, 그리고 이 라인이 batched NS-5 를 미는 이유). κ 가 test-time-compute dial 이라는 것 (→ 14 장) 은 이 관점에서 “compute 축을 따라 AI 를 더 밀어 올리는 knob” 이다.

그런데 이 compute-bound kernel 이 $C=1$ **decode step** 전체를 **compute-bound** 로 만들지는 못한다. 이유는 회계로 분명하다: NS 의 FLOP 을 그 step 이 옮기는 whole-state RMW 트래픽에 나눠 상각하면, step 에 기여하는 유효 AI 는

$$\frac{FLOP_{NS}}{\text{whole-state RMW}} \approx \frac{\kappa d^3}{2 m d^2 L_{\text{layer}} s} = \frac{\kappa d}{2 m L_{\text{layer}} s}$$

anchor($\kappa=5$, $d=2048$, $m=16$, $L_{\text{layer}}=24$, $s=2$) 에서 약 **6.7 FLOP/byte** 로, ridge 295 의 한 참아래다 (order-of-magnitude, directional). 즉 NS 를 얹어도 deep-memory decode step 은 memory-bound 를 벗어나지 못한다 — NS 연산 시간이 무거운 상태를 왕복하는 RMW 트래픽 시간에 **가려지기** 때문이다. 이것이 § 20.4 가 말한 “트래픽이 연산을 가려서” 의 정확한 값이다.

정직한 경계선 하나. 이 상각은 상태가 무거운 class(deep, momentum-deep, Hope) 에서 강건하다. 반대로 얇은 matrix memory($m \approx 2$) 에 Muon 을 얹으면 위 식의 분모가 작아져 유효 AI 가 약 $\kappa d / (2 \cdot 2 \cdot L_{\text{layer}} \cdot s) \approx 53$ FLOP/byte 까지 오른다 — 여전히 ridge 아래지만 훨씬 가까워지고, 더 큰 κ 나 얇은 L_{layer} 에서는 NS 가 step 을 compute 쪽으로 끌 수 있다. 따라서 “NS 는 트래픽에 가려진다” 는 진술은 **memory-centric 논증이 겨냥하는 상태-무거운 class 에 대해** load-bearing 이고, 얇은 matrix+Muon 은 그 경계 밖의 회색지대다. 이 경계를 명시하는 것이 § 18.6 계약이 요구하는 정직성이다.

20.5 chunk kernel: 같은 알고리즘이 compute-bound 로 넘어가는 곳

pair thesis 의 두 절반은 서로 다른 두 알고리즘이 아니라 **같은 알고리즘의 두 chunk 영역**이다. 이 절은 그 전환을 roofline 위에서 측정한다.

chunk 크기 C 는 arithmetic intensity 를 결정하는 knob 이다 (→ 9 장). $C = 1$ 은 per-token rank-1 RMW — 곧 § 20.2 의 decode 영역 — 이고 AI ≈ 1 FLOP/byte 로 memory-bound 평원에 앉는다. C 를 키우면 chunk 안의 여러 token 이 같은 chunk-start state $W_{\xi(t,C)}$ 에서 gradient 를 평가하므로 (stale-snapshot 근사, 식 (M4), → 9 장) state 재사용이 늘고 AI 가 roofline 을 타고 오른다. 어느 C^* 에서 memory $\leftrightarrow$ compute 교차가 일어난다.

그림 20-1 — DeltaNet-style chunkwise scan 의 measured AI(C) 곡선. $C=1$ (per-token RMW = TTT decode 영역) 은 host roofline 의 약 12%에 머무는 memory-bound 평원이고, C 를 키우면 host ridge(≈ 34 FLOP/byte) 를 넘어 measured $C^* \approx 32$ 에서 compute-bound 로 오

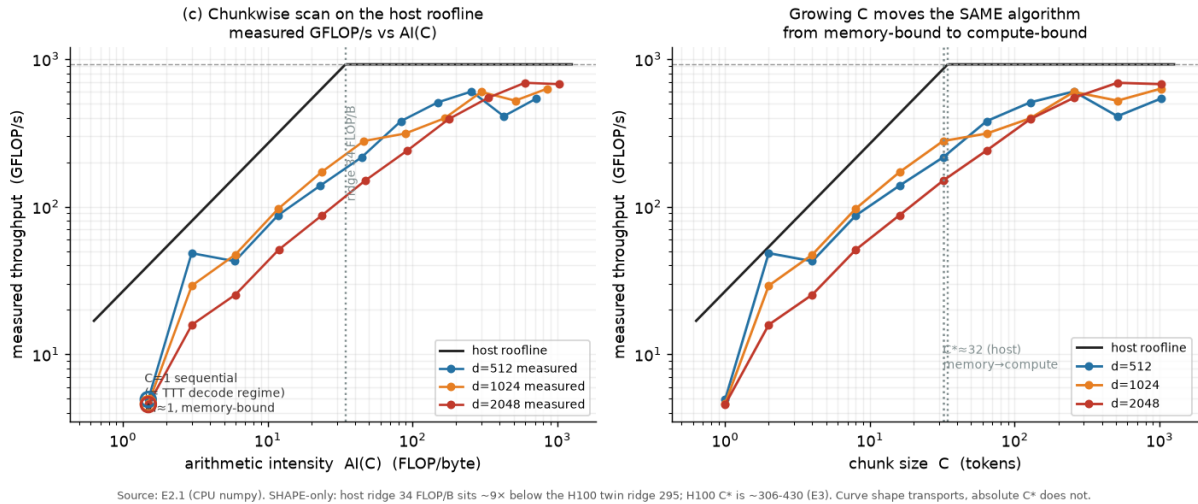


그림 20-1 — chunk 크기 C 에 따른 arithmetic intensity의 host roofline 상승: $C=1$ (decode 영역)의 memory-bound 평원에서 C^* 의 compute-bound 영역으로

른다. $d=2048$ measured throughput은 $C=1$ 의 4.56 GFLOP/s에서 $C=512$ 의 689.5 GFLOP/s까지 오른다. 곡선의 모양과 교차의 존재만이전되고 C^* 의 절대값은 이전되지 않는다: host ridge는 H100 twin ridge(295 FLOP/byte)의 약 1/9이며, H100 closed-form C^* 는 d 에 따라 306 – 430이다 ($d=2048$ 에서 337.1). 실험 E2.1(CPU micro-bench, CPU-SHAPE) / E3 재구성.

이 그림이 pair thesis를 한 장면으로 압축한다: 같은 knob C 가 decode를 memory-bound로, prefill/training을 compute-bound로만든다 — 한 알고리즘, 두 영역. C^* 의 위치는 ridge에의존하므로 (C^* 는 ridge에 비례) H100에서는 300 근처, host CPU에서는 32 근처지만, “낮은 C 는 memory-bound, 높은 C 는 compute-bound”라는곡선의 모양은 두 하드웨어에서 같다. 이것이 CPU 실측에서이전되는 유일한 것이다 (CPU-SHAPE tag, § 18.6).

이제 MFU가 어디로 새는지가 이 곡선 위에서 보인다. Atlas는 품질 최적 C (작은 chunk)에서 deep memory의 FLOPs utilization이 5 – 10% 미만임을 보고한다 ([Atlas], dossier bridge 3). 그 누수의 정체는 그림 20-1의 왼쪽 절벽이다:

[평가] MFU 누수를 roofline 위에서 읽으면 그 큰 몫은 kernel 비효율이 아니라 작동점의 선택으로 드러난다 (단 Atlas의 kernel-level profiling 분해가 없으므로, 작동점의 구조적 몫과 구현 overhead의 몫을 정량으로 가르지는 못한다 — measured host $C=1$ 이 roofline attainable의 약 12%라는 것은 memory-bound 상한 아래에 구현손실도 남아 있음을 함의한다). 품질이 최적인 C 는 작고 ($\rightarrow$ 15장 TNT의 chunk-1 decode 논점), 작은 C 는 그림 20-1의 memory-bound 평원 —roofline의 12% — 위에 있다. 즉 MFU는 “새는” 것이 아니라 품질을위해 지불되는 것이다. 둘째 원인은 stale-snapshot 근사가 chunk마다 chunk-start state를 다시 읽어 재계산하는 트래픽이고, 셋째는 per-request-unshared 상태에서 batch 축이 얇아 GEMM이 grouped-GEMM으로 파편화되는 것이다 ($\rightarrow$ 23장). TNT가 two-stage 훈련으로 chunk-1을 품질 최적점으로 옮긴 것 ($\rightarrow$ 15장)은 이 셋 중 첫째를정면으로 공략한 것이다 — 작동점을 옮겨 MFU 누수의 원인 자체를 없앴다. 단 chunkwise staleness의 오차 한계는 6편 어디에도 없으므로 (§ 6.4), 이 작동점이동의 품질 대가는 정량화되지 않은 채 남아 있다.

20.5.1 MFU 분해: 천장(작동점) × 포획(구현)

Atlas의 5 – 10% MFU를 roofline 위에서 두 곱셈 인자로 가른다. MFU는 achieved FLOP를 peak FLOP로 나눈 값인데, 이를

$$\text{MFU} = \underbrace{\frac{\text{AI}(C)}{\text{ridge}}}_{\text{천장 (작동점)}} \times \underbrace{\frac{\text{achieved}}{\text{roofline attainable}}}_{\text{포획 (구현)}}$$

로 분해한다. 첫 인자 **천장**은 순수하게 작동점 C 의 함수다: memory-bound 영역에서 어떤 kernel도 넘을 수 없는 상한이 $\text{AI}(C) \cdot \text{BW}/\text{peak} = \text{AI}(C)/\text{ridge}$ 이기 때문이다. $C=1$ 에서 $\text{AI} \approx 1$ 이므로 H100 twin(ridge 295)에서는 천장이 **약 0.3%**, host(ridge 34)에서는 **약 3%**다 — 완벽한 kernel조차 $C=1$ 에서는 이 천장에 갇힌다. 둘째 인자 **포획**은 그 천장 중 구현이 실제로 잡는 몫이다: E2.1의 measured host $C=1$ 이 roofline attainable의 약 12%라는 것은 (§ 20.5) 구현이 memory-bound 상한의 12%만 포획한다는 뜻이고, 그 손실은 kernel-launch·decode tail·grouped-GEMM 파편화 (→ 23 장)에서 온다.

이 분해가 MFU 논의를 정직하게 만든다. 품질 최적점이 작은 C (가령 $C \sim 8-16$)라면 천장이 $C=1$ 보다는 오르지만 여전히 한 자릿수%이고, 거기에 포획 손실을 곱하면 Atlas가 보고한 5-10%가 나온다. 곧 **5-10%의 큰 몫은 천장 (작동점 선택)이 정하고, 나머지가 포획 (구현)이다**. 다만 Atlas의 kernel-level profiling 분해가 공개되지 않았으므로 두 인자의 정확한 비를 우리가 가릴 수는 없다 (§ 20.5의 caveat 재확인) — 우리가 단정할 수 있는 것은 곱의 첫 인자가 roofline에서 **강제**된다는 구조뿐이다. TNT의 chunk-1 이동 (→ 15 장)이 공략하는 것은 이 분해의 첫 인자다: 품질 최적점을 큰 C 쪽으로 옮기면 천장자체가 올라가 MFU의 상한이 열린다. 반대로 포획 인자는 fused kernel과 grouped-GEMM의 몫이며, 그 개선은 accelerator 절반의 과제다 (§ 20.7).

이 절반 — chunk를 키워 compute-bound로 넘어가는 prefill/training — 이 accelerator 영역이다. 승부는 fused chunk kernel(banded-mask windowed loss, batched NS-5)과 grouped-GEMM에서나며, TNT가 이미 plain JAX로 FlashAttention을 32K에서 step당 이긴 지점이다 (→ 15 장). 이 영역에서 memory-centric 논증을 펴는 것은 pair thesis가 금지한다: 여섯 편이 알고리즘을 dense matmul로 빚어 기존 accelerator에 맞췄기 때문이다.

20.6 state placement: 상주는 설계 knob이고, 경계에서 대역폭이 꺾인다

decode가 memory-bound이고 그 bound가 whole-state RMW라면, 남은 설계자유도는 **그 state를 어디에 두는가**다. KV cache와 달리 TTT state 크기는 문맥에 무관하게 (d, m, L_{layer}) 로 고정되므로 (§ 20.2), on-chip 상주 여부가 **설계 가능한 knob**이 된다 — 문맥에 따라 자라상주 계획을 세울 수 없는 KV cache와의 결정적 차이다.

먼저 상주의 물리적 한계. on-die L2(50 MB 급)는 anchor에서 **한 sequence의 whole-model state조차** 담지 못한다 (어느 스케일에서도, 실험 E1.3/E3). 따라서 whole-model을 on-die에 pin하는 선택지는 없고, 상주는 **per-layer/streamed**여야 한다. 이제약이 state placement를 layer 단위 DSE로 만든다.

그림 20-2 — 134 MB/layer RMW를 네 device twin에 올린 결과. state가 268 MB scratchpad에 fit하는 동안 scratchpad는 HBM3 대비 **$6.8 \times$ energy / $14.9 \times$ time** 이득을 낸다. 그러나 residency crossover가 있다: 268 MB 버퍼는 anchor($d=2048$, 134 MB/layer)까지 한 layer의 state를 담고 $d=4096$ (7B, 537 MB/layer)에서 **spill**하며, crossover 폭은 $d^* \sim 2896$ 이다. whole-model이 268 MB에 드는 것은 Titans-170M(226 MB total) 뿐이므로, 340M 이상에서는 상주가 per-layer/streamed다. scratchpad·PIM twin은 simulation_ready=False인 **directional DSE**이며, 이 이득 배율은 shipping-device 주장이 아니다 (NOVEL-SIM-FALSE, § 18.6). 실험 E1.2.

그림 20-2의 메시지는 두 겹이다. 겉으로는 “state가 on-chip에 fit하면 scratchpad가 크게 이긴다” (analytic twin이 예측한 fit → spill 이득, directional)이고, 그 아래에는 “fit 여부 자체가 폭의 함수인 crossover”가 있다. 이 fit → spill 불연속은 추상적 경고가 아니라 host silicon에서 **직접 측정되는** 물리적 절벽으로도 확인된다. 그 절벽을 host CPU에서 직접 재면 다음이 나온다.

그림 20-3 — L3 capacity 경계에서 RMW 유효 대역폭이 꺾이는 cliff. in-cache RMW peak는 **~265 GB/s**로 돌지만 capacity를 넘겨 DRAM으로 spill하면 **~68 GB/s로 약 $3.9 \times$ 붕괴**한다

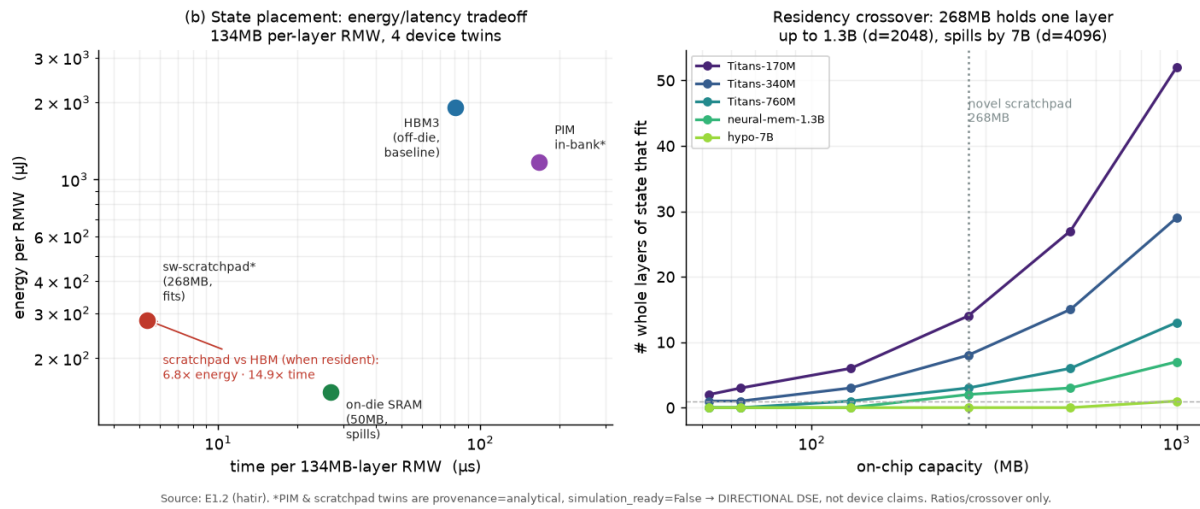


그림 20-2 — per-layer RMW state 의 device 별 energy/latency 와 residency crossover: state 가 fit 하는쪽에서는 scratchpad 가 HBM 을 이기고, 폭이 커지면 spill 한다

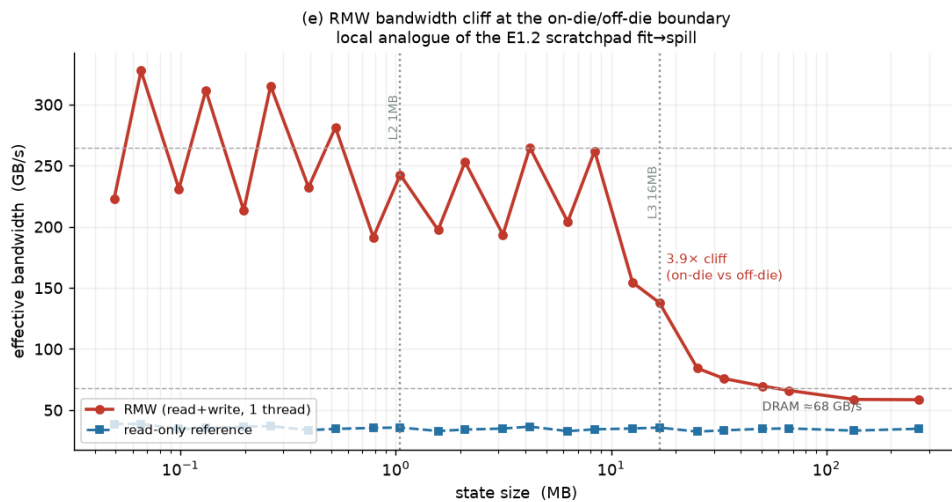


그림 20-3 — on-die/off-die 경계에서의 RMW 대역폭 cliff: in-cache RMW 가 capacity 경계를 넘으면 유효 대역폭이 불연속으로꺾인다

(1-thread; in-cache peak ÷ DRAM 비). 또한 saturated DRAM 에서 RMW 는 element 당 $2 \times \text{byte}(\text{read}+\text{write})$ 를 옮긴다 — 이것이 **append-once KV cache 가피하는 write-back 세 (稅)** 를 직접 측정한 값이고, 같은 byte 대역폭 환산시 유효 element throughput 은 read 의 약 $0.5 \times$ 다. **GB/s 절대값은 H100 으로 이전하지 않는다** (host cache cliff 는 H100 on/off-die ridge 와 $100 \times$ 어긋난다); 이전되는 것은 cliff 의 존재와 $3.9 \times$ 붕괴의 모양뿐이다 (CPU-SHAPE, § 18.6). 실험 E2.2.

그림 20-2 와 20-3 은 같은 현상의 두 얼굴이다 — 20-2 는 device twin 에서 fit → spill 을, 20-3 은 host silicon 에서 그 spill 의 대역폭 대가를 낸다. 둘을 합치면 state placement 의 설계 규칙이 나온다: **RMW state 는 상주경계 (capacity) 를 넘는 순간 대역폭이 불연속으로 붕괴하므로, memory 계층의 경계는 이 부하에서 부드러운 grade 가 아니라 절벽이다.** 이것이 memory-centric 기회의 두 번째 load-bearing 지점 (§ 18.3 의 update-frequency ↔ tier 배치) 과 만난다: state 를 어느 tier 에 두는가가 곧 어느 대역폭 plateau 에 앉는가를 결정한다.

그 tier 배치를 update cadence 가 정한다 (claim 5, 실험 E1.4). 규칙은 단정적이다 — **상주 사본은 read/write 중 빠른 쪽 cadence 로 tier 가 정해진다.** per-token fast-weight 는 read 가 매 token 이므로 HBM3 에 pin 되고, per-token 으로 쓰이는 state 를 CXL 같은 느린 tier 에 두면 매 token 그 대역폭 cliff 의 아래쪽에 앉는다. 반대로 genuinely down-sampled 된 CMS-mid level (4096 token 마다 read+write) 이나 sleep-consolidated expert (offline cadence) 는 sub-per-token 접근이 트래픽을 critical path 아래로 amortize 하므로 CXL 로 **합법적으로** 내려갈 수 있다 (→ 16 장 · 17 장). per-layer fast-weight 블록 (67 MB) 이 scratchpad 에 fit 할 때의 $7.4 \times$ energy 이득 (E1.4) 은 이 배치의 방향 (per-layer streamed 면 on-chip 이득) 만 load-bearing 이고, novel twin 이라 `simulation_ready=False` 인 directional DSE 로만 인용한다.

[해설] inference 엔지니어의 감각으로 옮기면 이렇다. paged KV cache 에서 “page 를 어느 tier 에 둘까” 는 접근빈도 (hot/cold) 로 정한다 — 그것과 정확히 같은 규칙이 여기서도 성립하되, 축이 “얼마나 자주 읽히나” 에서 “얼마나 자주 쓰이나” 로 바뀐다. write-heavy·unshared 라는 성격때문에, cold page 를 evict 하듯 dirty state 를 버릴 수 없다 — evict 는 full writeback 을 빚는다 (claim 6, → 23 장). placement 가 잘못되면 그 writeback 이 매번 대역폭 cliff 의 밑바닥에서 일어난다.

20.7 종합: kernel 수준에서 본 pair

이 장의 측정을 pair thesis 의 두 절반으로 되접는다. 어느 쪽도 소홀히 하지 않는 것이 이 책의 입장이다 (dossier § 5 의 comprehensive stance).

decode 절반 = memory-centric. decode step 은 class 와 무관하게 whole-state RMW 의 memory-bound (§ 20.2, $394 \times$ state-지배) 이고, 그 안에는 backward pass 라는 새 serving primitive 가 있다 (§ 20.3). 이 절반의 하드웨어 기회는 memory-centric 이다 — high-RMW-bandwidth state residency, forward+backward fused decode kernel, cadence-tiered placement, write-heavy elementwise epilogue 를 위한 near-memory 엔진 (§ 20.4-20.6). 이 논증은 논문들 자신의 cost 회계 (whole-state RMW, unshared, 문맥-무관 state) 에 근거하며 부과된 것이 아니다.

training/prefill 절반 = accelerator. 같은 알고리즘도 chunk C 를 키우면 compute-bound 로 넘어가고 (§ 20.5, 그림 20-1), 그 FLOP 은 GEMV/GEMM-shaped 이며 NS-5 조차 matmul 이다 (§ 20.4). 이 절반의 승부는 fused chunk kernel 과 grouped-GEMM, 즉 tensor-core 위에서 난다. 이 영역에 memory-centric 소자를 옹호하는 것은 여섯 편의 중심 엔지니어링 패턴 (알고리즘을 dense matmul 로 빚기) 과 모순되므로 이 책은 하지 않는다.

[평가] 흥미로운 하드웨어 제안은 둘 중 하나가 아니라 **쌍**이다. decode 를 위한 RMW-bandwidth·backward-capable serving 소자와, prefill/training 을 위한 fused chunk-grouped-GEMM accelerator 는 같은 배포의 두 부하를 나눠맡는다. 어느 한쪽만 옹호하는 제안은 이 workload 의 절반을 무시하는 것이다. 그리고 이 쌍을 위한 전용 artifact 는 아직 어느 것도 확인되지 않는다 — deep-memory decode 를 한 번의

state stream 으로 융합하는 fused kernel 도, 그 backward 를 decode 경로에 품는 serving runtime·소자도 (기존 GPU 가 backward·grouped-GEMM 자체를 실행할 수 있다는 것과는 별개의, 전용 서빙 artifact 의 부재다; dossier § 3.2). 이 라인이 기다리는 “FlashAttention-moment” 이 무엇을 예고하는지는 21 장의 hardware-lottery 독법이 있다.

이 장의 모든 절대값은 다시 한번 하한이다. 1.92 ms/token, 46.5 mJ/token, scratchpad 의 $6.8 \times / 14.9 \times$, cliff 의 $265 \rightarrow 68$ GB/s — 이 중 load-bearing 은 $394 \times$ **state-지배 (bound 판정)**, C^* 의 존재와 곡선 모양, $3.9 \times$ cliff 비율, class 간 순서뿐이고, 나머지 절대값은 A100 runbook 으로 이월되거나 directional DSE 로 남는다.

요약

- 이 라인의 decode step 은 아키텍처 class(matrix $\rightarrow$ deep $\rightarrow$ +momentum $\rightarrow$ Hope) 와 무관하게 **결정적으로 memory-bound**이며, state RMW 가 GEMV 연산을 약 $394 \times$ 압도한다 — step 비용이 곧 whole-state 트래픽이다 (anchor 6.44 GB/token, AI 0.59 FLOP/byte). class 를 무겁게 할수록 더 memory-bound 가 될뿐이다.
- batch 축을 더한 memory forward 는 per-request 상태 $M \in [B, d, d] \cdot W_2 \in [B, d_h, d]$ 의 **batched matmul(BMM)** 이다. request 마다 weight 가 달라 (never-average) shared-weight GEMM $[B \cdot C, d] @ [d, d_h]$ 로 안 퍼지고 strided-batched/grouped GEMM 으로 파편화되며, 각 W_b 가 한 번 읽혀 한 번 쓰이므로 **AI 가 batch B 와 무관하다** — batching 은 형성되되 (제안 G) amortization free-lunch 는 없다는 것이 decode 가 memory-bound 로 남는 shape 수준 근거다 ($\rightarrow$ § 23.4·24 장).
- decode 안으로 들어온 **backward pass** 가 이 라인이 서빙 엔진에 던진 진짜 새 primitive 다. forward-only 인 오늘의 decode kernel 과 달리, token 마다 memory MLP 를 거슬러 gradient 를 계산해야 하며, 이를 융합할 fused deep-memory kernel 은 존재하지 않는다.
- FLOP density 가 memory-centric 논증의 경계를 긋는다: memory MLP forward/backward 와 NS-5 는 GEMM-shaped($\rightarrow$ accelerator) 이고, PIM 이 정당한 곳은 element-wise epilogue(decay·renorm·AXPY) 의 소수 FLOP 뿐이며 그것도 directional($1.6 \times$ energy) 이다. NS₅ 는 고립하면 $\Theta(\kappa d^3)$ FLOP· $\Theta(d^2)$ traffic 으로 AI $\Theta(\kappa d)$ 의 compute-bound GEMM 이지만, whole-state RMW 에 상각하면 step 기여 AI 가 anchor 에서 ≈ 6.7 FLOP/byte(directional) 라 $C=1$ step 은 memory-bound 로 남는다 — 단 얇은 matrix+Muon 은 그 경계의 회색지대다.
- Atlas 의 matrix·poly·exp 세 얼굴이 pair thesis 를 한 축에 담는다: matrix·poly 는 고정-상태 RMW(memory-centric decode), $\exp(\phi^*)$ 는 문맥-증가 KV(append-read 기준선) 다. poly 는 상태 트래픽을 $\sim 1.5 \times$ 만 올리며 capacity class 를 하나 높이는 (retention $\sim 6.7 \times$, cross-paper ordinal) 이 라인에서 가장 유리한 교환이다. Hope 는 한 블록에 ~ 6 개 RMW 상태를 품어 decode class 최상단이자 블록-내부 cadence 계층을 강제한다.
- 같은 chunk knob C 가 decode 를 memory-bound($C=1$, roofline 12%) 로, prefill/training 을 compute-bound(C^* 위) 로 만든다 — 한 알고리즘, 두 영역. MFU 가 5–10% 로 새는 것은 kernel 결함이라 아니라 품질 최적 작동점 (작은 C) 이 **memory-bound 평원 위에 있기 때문이다**.
- TTT state 크기는 문맥에 무관하므로 on-chip 상주가 설계 knob 이 되며, state placement 의 memory 계층은 capacity 경계에서 대역폭이 **불연속으로 $3.9 \times$ 꺾이는 절벽**이다 (in-cache peak $\div$ DRAM; RMW 는 element 당 $2 \times$ byte 를 옮기는 write-back 세를 문다 — 동일 대역폭 환산 시 element throughput $0.5 \times$). 상주 tier 는 read/write 중 빠른 cadence 로 정해진다.
- 흥미로운 HW 제안은 **쌍**이다: decode 용 RMW-bandwidth·backward-capable serving 소자 + prefill/training 용 fused chunk·grouped-GEMM accelerator. 모든 절대값은 roofline 하한 (A100 runbook 이월) 이거나 directional 이고, load-bearing 은 비율·crossover·bound·순서뿐이다.

자가 점검체크리스트

- 아키텍처 class 별 decode RMW 트랙픽을 $2md^2L_{\text{layer}}s$ 로 계산하고, 전 class 가 왜 memory-bound 인지 설명할 수 있다.
- decode의 backward pass가 왜 “새 serving primitive” 인지, forward-only KV decode와 무엇이 다른지 진술할 수 있다.
- batch 축을 더한 memory forward가 왜 shared-weight GEMM이 아니라 grouped-GEMM 인지, 그로 인해 AI가 batch B 와 무관해 decode가 memory-bound로 남는 이유를 shape로 설명할 수 있다 (그리고 dot-product 특수형 vs MLP 일반형, static query, MoE dispatch-form-only 세 보정을 구분할 수 있다).
- 한 decode step의 FLOP을 (memory MLP f/b, NS-5, elementwise epilogue)로 나누고, PIM이 정당한 지점을 그중 어디로 한정해야 하는지 말할 수 있다.
- chunk C 가 왜 roofline의 x 축인지, C^* 의 절대값은 이전 안 되지만 곡선 모양은 이전되는 이유를 설명할 수 있다.
- MFU가 5-10%로 “새는” 진짜 원인 (작동점 선택)과 TNT의 chunk-1 이동이 그것을 어떻게 공략하는지 말할 수 있다.
- state placement의 fit $\rightarrow$ spill 절벽과 $3.9\times$ 대역폭 cliff가 왜 부드러운 grade가 아닌지, cadence가 tier를 어떻게 정하는지 설명할 수 있다.
- 어느 수치가 load-bearing(비율 · crossover-bound · 순서)이고 어느 것이 이월되는 하한 · directional 인지 판별할 수 있다.
- Atlas의 matrix · poly · exp 세 얼굴이 왜 pair thesis 전체 (고정-상태 RMW $\leftrightarrow$ 문맥-증가 KV)를 한 축에 담는지, poly가 왜 유리한 capacity 교환인지 설명할 수 있다.
- NS-5의 고립 AI가 $\Theta(\kappa d)$ 로 compute-bound인데도 왜 $C=1$ step은 memory-bound인지 (유효 기여 AI ≈ 6.7 FLOP/byte)를 회계로 보이고, 얇은 matrix+Muon이 왜 경계밖인지 말할 수 있다.
- 이 장의 병목을 inference 어휘 (paged KV placement $\leftrightarrow$ RMW-state placement; forward-only decode $\leftrightarrow$ backward-in-decode)로 옮길 수 있다.

다음 장으로

이 장은 pair thesis의 두 절반을 roofline · kernel · placement의 언어로 측정하고, 어느 것도 아직 존재하지 않는 두 소자 — backward-capable decode 소자와 fused chunk accelerator —를 pair로 남겨 두었다. 그 소자들이 왜 아직 없는지, 그리고 이 라인이 왜 하필 matmul 안으로 스스로를 설계해넣었는지는 하드웨어의 역사가 답한다. 21 장은 hardware-lottery의 독법으로 그 질문을 잇는다 — attention은 GEMM density로 이겼고, 이 가족은 chunkwise · NS · reset으로 그 승리를 모방하도록 설계되어 있다. 그것이 채택과 kernel 생태계에 무엇을 예고하는지가 다음장의 주제다.

ch21. Transformer/NVIDIA scaling 시대의 교훈: hardware lottery

이 장의 목표 — 독자가 이 장을 마치면 (1) attention 이 이긴 진짜이유를 표현력 논쟁이 아니라 **GEMM density** 로 설명하는 hardware-lottery 독법을 진술할 수 있고, (2) 이 neural-memory 라인이본질상 recurrent 임에도 chunkwise·Newton-Schulz-reset 이라는 네 가지수 (手) 로 **matmul 안에 설계되어** 있음을, 그리고 그 순응이 pair thesis 의 training/prefill 절반이 accelerator 영역인 근본이유임을 논증할 수 있으며, (3) 이 가족이 아직 기다리는 **FlashAttention-moment** 이 무엇인지, 그것이채택·kernel-pair thesis 의 경계에 대해 무엇을예고하는지 예측으로 옮길 수 있어야 한다. **왜 필요한가** — 20 장은아키텍처 class 별 decode roofline 과 state placement 로 병목을측정했다. 이 장은 그 병목이 왜 지금 이 모양인지를 한 단계 위 —알고리즘 설계가 하드웨어와 협상해 온 역사 — 에서 읽는다. 이 독법 (dossier § 5 angle A5) 은 falsifiable 한 정량 기여가 아니라 **framing** 이지만, Part III 의 다른 장들이 내놓는 숫자에 “왜 이 라인은 새 소자를 요구하지않는가” 라는 배경을 준다. 그 배경이 없으면 24 장의 pair 제안 —memory-centric decode 소자와 accelerator prefill kernel 을 쌍으로 — 이 왜균형인지가 흐려진다.

21.1 Bridge-in: 20 장이 측정한 병목의 역사적원인

20 장은 이 라인의 decode 를 arithmetic intensity 0.59 FLOP/byte 의 memory-bound RMW 로, prefill 을 chunk C 가 x 축인 roofline 곡선으로 분해했다 (claim 1·7, → 20 장). 그분해는 참이지만 **왜 알고리즘이 그렇게 생겼는가**를 설명하지않는다. 왜 이 라인은 여섯 편 내내 recurrence 를 dense matmul 로 다시빚었는가? 왜 새 하드웨어를 요구하지 않았는가? 이 장의 답은 하나의개념으로 압축된다: **hardware lottery**.

hardware lottery(Hooker 2020, arXiv:2009.06489) 는 어떤 연구 아이디어가본질적으로 더 낫기 때문이 아니라 그 시점에 가용한하드웨어·소프트웨어에 우연히 잘 맞기 때문에 이긴다는관찰이다. 아이디어의 승패는 아이디어의 품질이 아니라 그것이 당대 accelerator 의 연산 primitive 와 얼마나 정렬되는가로 결정된다. 이 렌즈로지난 10 년의 transformer/NVIDIA scaling 시대를 읽으면, 그리고 이 neural-memory 라인이 그 시대에 어떻게 자세를 잡았는지 읽으면, 20 장이 측정한 병목이 **선택된 병목**이었음이 드러난다.

21.2 attention 이 이긴 진짜 이유: GEMM density

softmax attention 의 핵심 연산은 두 개의 큰 행렬곱이다 — score QK^T 와 가중합 PV . 둘 다 sequence 축 전체에 걸쳐 완전히병렬이고, batch·head·hidden 차원을 곱해 큰 arithmetic intensity 를 만들 수 있다. 이것이 GPU/TPU 의 tensor core 가 가장 잘하는 모양, 즉 높은 **GEMM density** 다. 반대로고전 RNN 은 시점 t 의 상태가 $t-1$ 에 의존하는 sequential dependency 때문에 같은 하드웨어에서 죽었다 — 병렬화할 GEMM 이 없고, 매 스텝이 작은 GEMV 로 쪼개져 tensor core 를 놀린다.

여기서 핵심은 이 승부가 **표현력 논쟁과 무관**했다는 점이다. RNN 이원리적으로 무엇을 표현할 수 있는지, attention 이 무엇을 못 하는지는 결과를 바꾸지 않았다. attention 은 자신을 큰 matmul 로 표현할 수 있었기에 scaling 시대의 로또를 이겼고, RNN 은 그러지 못했기에 밀렸다. transformer 가 지배종이 된 것, 그리고 NVIDIA 가 그 지배의 하드웨어지대 (地代) 를 걷은 것은 같은 사건의 두 얼굴이다.

[해설] 독자가 매일 다루는 KV cache 는 이 승리의 부산물이다. attention 은 과거 token 을 **압축하지 않고** 통째로 남겨 두는 대가로 GEMM density 를 얻는다 — softmax attention 은 capacity 가 무한한 (ϕ^*) associative memory, 즉 압축하지 않는 극한이다 (→ 14 장). KV cache 가 문맥에 비례해 자라는 것은 결함이 아니라 이 거래의 명세이다: 메모리를 무한정 쓰는 대신 연산을 완벽한 matmul 로 유지한다. 이라인이 도전하는 것이 바로 그 거래 — 상태를 고정 크기 weights 로 **압축**하되, 압축의 대가로 무엇을 하드웨어에 지불해야 하는가 — 다.

21.2.1 GEMM density 는 왜 하필 승리 조건이었나 — systolic array 에서 tensor core 까지

GEMM density 가 로또의 당첨 번호가 된 것 자체가 하드웨어 역사의 산물이다. Dennard scaling 이 끝나 clock 주파수를 더 올리지 못하게 된 2010 년대, accelerator 가 지수적으로 키운 것은 clock 이 아니라 **병렬 throughput** — 한 사이클에 소화하는 dense-regular 한곱셈-누산의 수 — 였다. 이 throughput 을 값싸게 뽑는 표준 구조가 systolic array 다: Kung - Leiserson(1978) 이 제안한, 데이터가 격자 위를 규칙적으로 흐르며 곱-누산되는 dataflow. TPU 의 MXU 가 정확히 이 systolic array 이고, GPU 의 tensor core 도 같은 원리의 작은 dense-matmul 엔진이다. 이 구조들의 공통 성질은 하나다: **규칙적인 dense GEMM 에 최적이고, irregular-sparse-sequential access 에는 페널티를 매긴다.**

그래서 승부는 표현력 이전에 dataflow 에서 갈렸다. attention 의 $QK^T \cdot PV$ 는 systolic array 가 흡수하도록 태어난 것처럼 규칙적이라, 하드웨어가 넓어질수록 그대로 공짜로 빨라졌다. 반대로 RNN/LSTM 은 — 2014 - 2017 sequence modeling 의 실질 SOTA 였음에도 — 시점 t 가 $t - 1$ 을 기다리는 sequential dependency 때문에 넓어지는 tensor throughput 을 굶겼다. LSTM 이 밀린 것은 무엇을 표현하지 못해서가 아니라, 지수적으로 좋아지는 하드웨어를 먹지 못해서였다. 이것이 hardware lottery 의 교과서적 패배 사례다. 같은 이유로 초기 MoE-sparse attention 류의 conditional-compute 아이디어도 dense GEMM 에 맞지 않아 반복적으로 채택이 지연됐다 (Hooker 자신의 예).

타이밍의 우연도 논제가 예측하는 대로다. attention 을 정의한 transformer(2017) 는 tensor core 를 처음 실은 Volta(2017) 와 같은 해에도 착했고, 그 이전 GPU 딥러닝의 문 (AlexNet, Krizhevsky et al. 2012 → cuDNN) 은 이미 dense-matmul 특화 쪽으로 열려 있었다. 아이디어와 소자가 우연히 같은 창 (窓) 에서 만난 것이다. 이후 둘은 **공진화 (lock-in)** 한다: 지배종 transformer 가 하드웨어 로드맵을 끌고 (더 큰 MXU, 더 낮은 정밀도의 matmul), 그 하드웨어가 transformer 를 더 싸게 만들어 지배를 굳힌다. NVIDIA 가 이 시대의 지대를 걷은 구조가 이것이다.

[평가] 이 배경이 이 장의 전제다. GEMM density 는 알고리즘의 내재적 미덕이 아니라 **특정 시대의 소자가 값싸게 실행할 수 있는 모양**이고, 그 모양을 통과하지 못한 아이디어는 옳고 그름과 무관하게 지연됐다. neural-memory 라인은 본질상 RNN 을 로또에서 떨어뜨린 바로 그 sequential dependency 를 품고 있다. 그러므로 이 라인이 살아남으려면 같은 관문 — dense GEMM 으로 자신을 표현하는 능력 — 을 통과해야 한다. 21.3 은 이라인이 그 관문을 **어떻게** 통과했는지를, 21.4 는 그 통과가 아직 **부분적**임을 읽는다.

21.3 이 라인은 matmul 안에 설계되어있다

neural-memory 라인은 본질적으로 recurrent 다. inner loop 가 token 마다 fast weights W_t 를 W_{t-1} 에서 갱신하는 sequential 과정이고 (→ 8 장 · 9 장), 이는 정확히 RNN 을 로또에서 탈락시

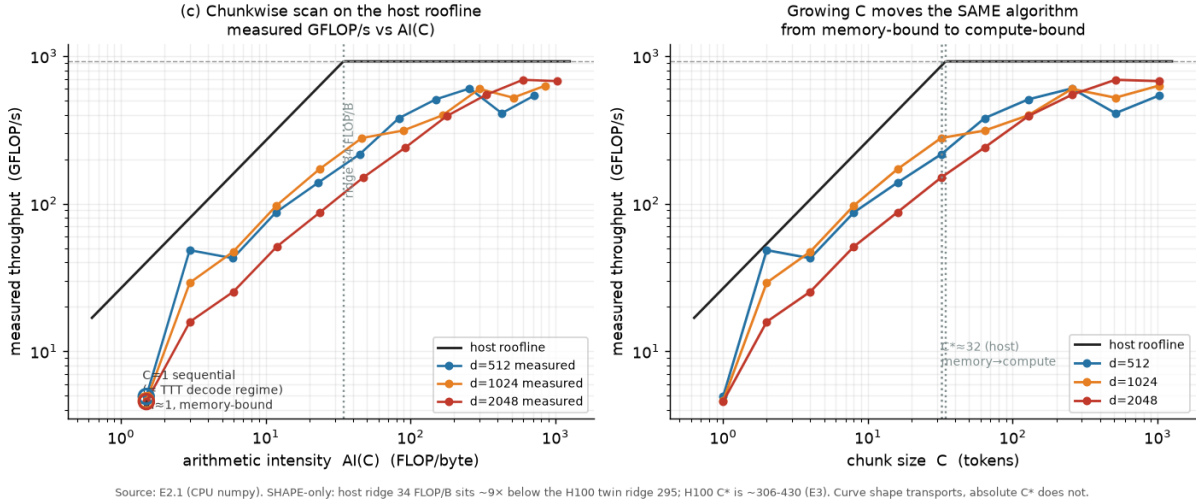


그림 21-1: chunk 크기 C 에 따른 arithmetic intensity $AI(C)$ 곡선 — memory-bound 평원에서 memory $\leftrightarrow$ compute crossover 를 넘어 compute-bound 로 오르는 roofline 이동 (claim 7).

킨 그 dependency 다. 그런데 이 라인은 탈락하지 않았다. 저자들이 hardware lottery 의 세 (稅) 를 선불했기 때문이다 — 알고리즘을 기존 accelerator 에 맞도록 네 번에 걸쳐 다시 빚었다.

1. **chunkwise-parallel training** ($\rightarrow$ 9 장). chunk 안의 모든 gradient 를 chunk 시작 상태 $W_{\xi(t,C)}$ 에서 평가하는 stale-snapshot 근사 (식 (M4)) 는 sequential recurrence 를 chunk 단위 GEMM 으로 바꾼다. 정확성을 내주고 matmul 을 산다 — 이 라인 여섯 편전부의 병렬화 기반이다.
2. **Newton – Schulz orthogonalization** ([Atlas], $\rightarrow$ 14 장). Muon inner optimizer 의 $NS_{\kappa}(\kappa = 5)$ 는 momentum buffer 를 semi-orthogonal 하게 만드는 반복인데, 그 반복 자체가 batched 행렬곱의 연쇄다. [Atlas] 가 자기 방법을 “tensorize and maximize matmuls” 의 원칙으로 서술하는 것은 우연이 아니라 로또를 의식한 설계언어다.
3. **periodic reset** ([TNT], $\rightarrow$ 15 장). local memory 를 학습된 W_{init} 으로 주기적으로 되돌리면 비선형 recurrence 의사술이 끊겨, deep-memory recurrence 를 병렬화하는 유일하게 알려진 길 — context parallelism — 이 열린다. reset 은 품질 장치이기 이전에 **병렬화 장치**다.
4. **featurized keys** ([Atlas], $\rightarrow$ 14 장). polynomial feature map ϕ_p 로 key 를 들어 올려 capacity $O(d_k^p)$ 를 사는 것은, capacity 마저 더 많은 matmul 로 치환하는 수다.

이 네 수의 공통 문법은 하나다: **arithmetic intensity 를 제조한다**. claim 7 이 이 문법을 정량화한다 — chunk $C = 1$ (per-token rank-1 RMW = decode 영역) 은 $AI \approx 1$ FLOP/byte 로 memory-bound 평원에 앉아있지만, C 를 키우면 roofline 을 타고 올라 crossover 를 넘어 compute-bound 로 옮겨 간다. 같은 알고리즘이 knob 하나로 두 영역을 오간다.

(그림 c — chunk C 에 따른 $AI(C)$ 곡선이 memory-bound 평원에서 compute-bound 로 오르는 모양; 주 담당은 9 장 · 15 장, $\rightarrow$ 20 장에서 decode-side 와 함께 배치.)

[해설] claim 7 의 정직한 사용법을 못박아 둔다. **load-bearing** 한 것은 곡선의 모양과 **memory $\leftrightarrow$ compute crossover** 의 존재다: 저자들이 C 를 키워 GEMM density 를 제조할 수 있다는 사실. crossover 의 위치는 ridge 의 존재적이어서, host CPU 에서 측정된 $C^* \approx 32$ 는 곡선의 모양을 확인할 뿐 H100 로 이전되지 않는다 (host ridge ≈ 34 FLOP/byte 는 H100 twin ridge 295 의 약 1/9; H100 closed-form C^* 는 d 에 따라 306 – 430, $\rightarrow$ 20 장 · § 18.6 정직성 제약). 이 장이 덮는 것은 오직 “제조가능하다” 는 구조뿐이다.

GEMM shape 로 읽으면 이 제도가 눈에 보인다. chunkwise gradient $\nabla_W \ell$ 은 chunk 내 C 개 token 의 오차를 key 에 실은 $dW = (\text{오차}) K^\top$ 꼴 — contraction 차원이 정확히 C 인 rank- C GEMM 이다 ($\rightarrow$ Rosetta). 그래서 C 는 arithmetic intensity 를 직접 쥐는 손잡이가 된다. host roofline 위 측정이 이 손잡이의 힘을 보여준다: 같은 커널이 $d = 2048$ 에서 $C=1$ 일 때 4.56 GFLOP/s 로 memory-bound 평원 (host roofline 의 $\sim 12\%$) 에 앉아 있다가, C 를 512 까지 키우면 689.5 GFLOP/s 로 오른다 — 한 손잡이로 약 $150 \times$ throughput 을 제조한것이다 (claim 7, $\rightarrow$ 20 장). 정직성: 이 GFLOP/s 절대치는 host 의것이라 H100 로 이전하지 않는다 (host ridge ≈ 34 FLOP/byte 는 H100 twin ridge 295 의 약 1/9). load-bearing 한 것은 곡선의모양, memory $\leftrightarrow$ compute crossover 의 존재, 그리고 이 한 자릿수 knob 이 만드는 $\sim 150 \times$ swing 이라는 비율뿐이다.

이 제도가 실제로 로또를 이길 만큼 강하다는 증거는 [TNT] 에 있다: plain JAX 로 작성된 TNT 가 32K 문맥에서 step 당 FlashAttention 을 이겼다 ($\rightarrow$ 15 장). recurrent memory 가 — 커스텀 CUDA 없이 — 지배종의 최적화된 커널을 특정 조건에서 앞선 것이다.

[평가] 이 라인 전체는 hardware lottery 에 대한 하나의 긴순응으로 읽어야 한다. 여섯 편 어디에도 “이 알고리즘을 위해 새하드웨어가 필요하다” 는 요구가 없다. 정반대로, 매 편이 알고리즘을 dense matmul 로 다시 빚어 기존 accelerator 에 밀어 넣는다. 이것이 pair thesis 의 training/prefill 절반이 왜 **accelerator 영역**인지의근본 이유다 ($\rightarrow$ 18 장, 24 장): 그 절반에는 chunk 라는 축이 있어 GEMM density 를 제조할 수 있고, 제조할 수 있는 곳에서는 memory-centric 소자를요구하는 것이 아니라 tensor core 를 더 잘 먹이는 것이 답이다. 이 라인의중심 엔지니어링 패턴 자체가 “training 에 새 소자가 필요하다” 는 주장을금지한다 (dossier § 5 의 forced 판정 3).

21.4 그런데 FlashAttention-moment 은 아직 오지않았다

로또의 세를 선불했다고 해서 이 라인이 곧바로 실용화되는 것은 아니다. attention 의 실용화에는 알고리즘 승리 이후 한 번의 **kernel 승리**가더 필요했다. FlashAttention(Dao et al. 2022, arXiv: 2205.14135) 은 attention 의 수학을 한글자도 바꾸지 않고 — bit-exact tiling 으로 — IO 를 재조직해 attention 을메모리 병목에서 풀어냈다. 이후 FlashDecoding 이 같은 아이디어를 decode 까지 확장했다. attention 이 오늘 어디서나 돌아가는 것은 이 kernel-level 돌파 덕이다.

이 neural-memory 가족에는 그런 순간이 **아직 없다**. 그리고 그부재는 추측이 아니라 생태계에서 관측된다. flash-linear-attention(FLA, 5,325 ★; 이하 이 장의 GitHub 별 수는 모두 2026-07 정찰 시점 값, $\rightarrow$ notes/impl-availability.md) 은 이라인의 **matmul-clean** 한 계보 — delta_net, gated_deltanet, gla, rwkv7, mamba2, mesa_net 등 — 을 전부 production-grade Triton 커널로 layer·model 수준까지커버한다. Gated DeltaNet(이하 GDN) 은 NVIDIA 의 공식구현 (NVlabs/GatedDeltaNet, 619 ★, ICLR 2025) 까지 갖췄다. 그런데 이 라인의 정점인 **deep-memory + momentum** Titans 는 FLA 에서 fla/ops/titans 의 **naive PyTorch 참조 구현**에 머물러 있다 — Triton 커널도, layer/model 통합도 없다. 같은 RFC(#107) 에서 TTT 와 Titans 커널이 함께 발의됐으나 TTT 만 Triton 화되고 Titans 는 #214 에서 정체했다.

이 정체의 원인이 이 장의 핵심 증거다: **chunk-level momentum 이 chunkwise closed form 을 깨뜨린다**. momentum buffer $S_t = \beta_t S_{t-1} - \eta_t \nabla_W \ell$ (식 (M2)) 의 재귀 항은 chunk 경계를 넘어 이어지므로, chunk 내부를 하나의 깨끗한 matmul 로 접는 dual form 이 성립하지 않는다. 바로 이 어려움이 [TNT] 의 존재 이유였고 ($\rightarrow$ 9 장 · 15 장), 지금은 오픈소스 커널 생태계에 남긴 **실물흔적** — Titans 만 naive 에 멈춘 자리 — 으로 확인된다.

[해설] 즉 이 라인은 로또의 세를 **부분적으로만** 냈다. matmul-clean 한 부분 (linear attention, delta rule, gating) 은 이미 커널이있고 이미 채택됐다 — Mamba-2 와 GDN 은 production 에 들어갔다. matmul-clean 하지 **않은** 부분 (deep MLP memory, momentum, self-modification) 은 fused 커널이 없어 아직 연구 코드에 머문다. 이라인이 논문에서 그린 완성형 ($\rightarrow$ 18 장) 은 후자에 크게 기댄다. 그러므로완성형과 배포 사이에는 아직 하나의 kernel 승리가 통째로 비어 있다.

21.4.1 왜 momentum·deep memory 는 커널화가 어려운가 (기계적해부)

이 정체가 우연한 엔지니어링 지연이 아니라 알고리즘의 구조적 성질임을 못박아 둔다. chunkwise 병렬화의 문법을 다시 부르자 (→ 9 장): sequential recurrence 를 (i) chunk 내부를 하나의 큰 matmul 로 접는 병렬 파트와 (ii) chunk 요약들만 잇는 inter-chunk sequential scan 으로 분해한다. 이 분해가 성립하려면 per-token update 가 상태의 affine map 이어야 하고, 그 map 들의 chunk-내 합성이 **associatively 결합 가능한 compact object(행렬하나)** 로 접혀야 한다. 어떤 알고리즘이 “matmul-clean” 한가는 정확히 이접힘이 되는가로 갈린다.

되는 쪽 — GDN. GDN 의 update $W_t = \alpha_t W_{t-1} (I - \eta_t k_t k_t^\top) + \eta_t v_t k_t^\top$ 는 W 에 affine 이고 memory 가 linear(행렬) 다. chunk 내 $\prod_i (I - \eta_i k_i k_i^\top)$ 곱은 WY/UT 표현으로 하나의 masked matmul(삼각 역행렬 한 번 — 그 자체가 GEMM) 로 접힌다. 그래서 `delta_net · gated_deltanet · gla · rwkv7 · mamba2` 가 전부 FLA 에 production-grade Triton 으로 존재하고, GDN 은 NVIDIA 공식 커널 (NVlabs/GatedDeltaNet, 619 ★) 까지 갖췄다.

안 되는 쪽 — Titans(deep memory + momentum). 두 겹의 장애가 겹친다.

- **장애 1 — deep memory 의 gradient 는 rank-1 이 아니다.** memory 가 표준 deep memory(2-layer residual MLP, → § 1.3) 이면 ∇_{W^ℓ} 은 outer product $v_t k_t^\top$ 가 아니라 MLP forward·backward 를 통과한 matmul 사슬 (중간 nonlinearity 포함) 이다. linear-in- W closed form 이 존재하지 않는다. stale-snapshot 근사 (식 (M4)) 가 chunk 내 모든 gradient 를 anchor $W_{\xi(t,C)}$ 에서 평가해 chunk 를 하나의 batched forward/backward 로 만들어 **chunk-내** 병렬성은 복원하지만, 이는 chunk 간 결합까지 풀어 주지는 않는다.
- **장애 2 — momentum 이 scan combine 을 companion 행렬로 만든다.** momentum buffer $S_t = \beta_t S_{t-1} - \eta_t g_t^{\text{in}}$ (식 (M2)) 의 재귀는 상태를 pair (W_t, S_t) 로 확장한다. joint map 은 여전히 affine 이지만, chunk-내 합성이 momentum decay 사슬 $\beta_t \beta_{t-1} \dots$ 을 실어야 하고 각 위치의 $\eta_t \cdot \beta_t$ gate 가 얹혀, scan 의 combine 연산이 **위치별 block(companion) 행렬** 곱이 된다. delta rule 의 $(I - \eta_t k_t k_t^\top)$ 처럼 하나의 clean GEMM 으로 접히지 않는다.

이 두 번째 장애가 생태계에 남긴 실물 흔적이 `fla/ops/titans` 다. 이 ops 는 momentum·decay 곱을 log-space 에서 combine 하려고 $L \times L$ (sequence×sequence) 하삼각 가중 행렬을 그대로 materialize 한다 (`naive.py+log_impl.py`). $O(L^2)$ 의 메모리·연산은 C 를 키울수록 chunk 병렬화가 사려던 이득을 도로 반납하므로, Triton 커널이 아니라 **참조 구현**에 머문다. 대비가 결정적이다: RFC #107 이 TTT 와 Titans 커널을 **함께** 발의했으나, 구조적으로 접히는 TTT 만 진짜 Triton 커널 (`chunk_ttt_linear`, `fused_chunk_ttt_linear`; group-norm 융합·varlen 지원) 이 되고 momentum 을 지닌 Titans 는 #214 에서 naive 에 정체했다. 같은 팀·같은 인프라·같은 시점에서 갈린 것은 오직 알고리즘의 matmul-cleanness 다. 사실상의 reference 구현인 `lucidrains/titans-pytorch` (1,966 ★) 조차 dual-form matmul 이 아니라 AssocScan (accelerated associative scan) 기반 병렬화로 우회한다 — scan 은 돌지만 tensor core 를 FlashAttention 만큼 먹이지는 못한다.

같은 메커니즘이 momentum 하나에 국한되지 않는다. Miras 의 새 attentional bias 들도 이 관문에 걸린다: Moneta 의 ℓ_p norm 이나 Yaad 의 Huber loss 는 inner objective 를 nonlinear 하게 만들어 순간의 chunkwise closed form 을 없애고, ℓ_2 계열이 재활용하던 DeltaNet/GLA 커널 대신 naive 순차 루프 (또는 논문의 근사) 로 후퇴하게 한다. 그래서 FLA 에는 Miras 의 세 인스턴스 (Moneta/Yaad/Memora) 가 없다 — 설계공간의 커널 인프라 (`comba`, `mesa_net`, `mom`, `kda`) 는 있는데 nonlinear-loss 지점만 비어 있는 것이다. 즉 이 라인의 matmul-cleanness 는 momentum·deep-memory·nonlinear-loss 라는 세 축에서 동시에 시험받고, fused 커널의 공백은 정확히 이 세 축이 겹치는 곳 — 논문들이 그린 완성형 (Titans/Atlas/Hope) — 에서 가장 크다.

[해설] 문이 원리적으로 닫힌 것은 아니다. MesaNet (Behrouz 라인 인접, arXiv: 2506.05233) 은 locally-optimal test-time regression 을 chunkwise conjugate-gradient solver 로 Triton 화했다 (`chunk_cg_solver`) — Atlas 의 2 차·window 방향에 가장 가까운 커널 증거다. 즉 “2 차 정보를 쓰는 recurrent memory 는 커널화 불가” 가 아니라, **deep-MLP memory + momentum** 이라는 특정 조합에 대한 **fused 커널이 아직 쓰이지 않았을 뿐**이다. 이 구분이 다음 절 예측 2 의 형태를 정한다.

21.5 이 라인의 FlashAttention-moment 이 예고하는것

hardware-lottery 독법은 예측을 낳는다. 네 가지로 정리한다.

예측 1 — 채택은 kernel 가용성이 정한다, 수학적 우수성이 아니라. 커널이 있는 부분만 production 으로 간다. GDN·Mamba-2 는 이미갔고, deep-memory Titans/HOPE 는 fused 커널이 나오기 전까지 못 간다. 이것은 hardware lottery 의 재현이다: 라인 내부에서조차 승자는 더 나은 memory 가 아니라 커널이 있는 memory 다. [Atlas] 자신의 ablation 이 760M 에서 Muon 제거가 오히려 perplexity 를 개선했다고 보고한것 (→ 14 장) 은 이 예측과 나란히 읽힌다 — 가장 matmul-무거운 축의 품질가치조차 미결인데, 그 축은 커널 부담까지 가장 크다.

[해설] 이 재현은 라인의 hybrid 설계 선택에서도 관측된다. Titans 의 세 합성 방식 (MAC/MAG/MAL) 중 MAL (Memory as Layer) 이 throughput 비교에서 유리하게 나오는 국면은, MAL 이 이미 성숙한 FlashAttention 커널 위에 memory 를 얹기 때문이지 memory 자체가 더 우수해서가 아니다 (dossier § 5 A5). 커널 성숙도가 아키텍처 선택의 저울을 기울인 것 — 라인 내부에서 로또가 한 번 더 돈사레다. 채택을 읽을 때 “어느 memory 가 더 나은가” 와 “어느 memory 에 커널이 있는가” 를 분리하지 않으면 이 교란에 걸린다.

예측 2 — moment 의 형태는 fused deep-memory chunk kernel 이다. 21.4.1 의 해부는 그 커널이 구체적으로 무엇을 융합해야 하는지까지 못박는다. 네 가지다.

1. **anchor 에서의 deep-MLP forward+backward 를 batched GEMM 으로.** stale-snapshot(M4) 이 chunk 내 모든 gradient 를 $W_{\xi(t,C)}$ 에서 평가하므로, chunk 전체의 forward·backward 는 중간 activation 을 SRAM 에 유지하는 IO-aware tiling 으로 처리할 수 있다 — FlashAttention 이 softmax 에 쓴 그 수를 MLP 의 backward 로 옮긴 것.
2. **momentum associative scan 을 sequence×sequence materialization 없이 융합.** 장애 2 의 companion-block combine 을 tile 단위로 online 누적한다 — FlashAttention 의 running-softmax 에 대응하는 running-momentum. fla/ops/titans 가 materialize 하는 $L \times L$ 행렬을 tile-local 재귀로 대체하는 것이 커널화의 핵심 난관이다.
3. **periodic reset 경계를 커널 안에서 처리.** local memory 의 W_{init} reset (→ 15 장) 이 커널 tiling 경계와 정렬돼야 context-parallel shard 가 합성된다.
4. **결정적 비대칭 — bit-exact 가 아니라 semantic 이다.** FlashAttention 은 attention 의 수학을 한 글자도 바꾸지 않는 bit-exact tiling 이었다 (→ Rosetta). 그러나 이 커널은 이미 **semantic** 근사위에 올라탄다 — chunk C 가 계산되는 함수 자체를 바꾸는 semantic hyperparameter 이기 때문이다 (M4, → 9 장). 그래서 “무엇을 커널화하느냐” 가 “알고리즘이 무엇을 계산하느냐” 와 분리되지 않는다. 이것이 이 가족의 FlashAttention-moment 이 원본보다 구조적으로 어려운 이유다: 커널 설계와 semantic 선택이 한 문제로 얹혀 있다.

그 커널이 나오기 전까지 quality-optimal 한 작은 chunk 는 deep memory 에서 FLOPs utilization 5 – 10% 미만으로 돈다 ([Atlas] 보고, → 14 장·15 장) — wall-clock 경쟁이 불가능한 영역이다. 누가 그 커널을 쓰든 (Google 내부, FLA 커뮤니티, 혹은 NVIDIA) 그 순간이 이 라인을 잠금 해제한다.

예측 3 — 그 moment 은 pair thesis 의 절반만 푼다 (비대칭). 이것이야 장이 이 책의 척추에 기여하는 지점이다. FlashAttention 은 prefill 과 decode 를 둘 다 도왔다 (FlashDecoding). 그러나 이 가족의 kernel moment 은 **prefill/training** 절반만 도울 수 있다. decode 는 $C = 1$ 에 갇혀 있어 GEMM density 를 제조할 chunk 축이 없다 — arithmetic intensity 0.59 FLOP/byte, 결정적으로 memory-bound 이고, state 트래픽이 GEMV 연산을 약 $394 \times$ 압도한다 (claim 1, → 20 장). 아무리 영리한 GEMM 커널도 움직여야 할 바이트수 자체 ($m d^2 L_{layer}$ 의 RMW) 를 줄이지 못한다.

이 불가능은 기계적으로 airtight 하다. FlashAttention 이 준 것은 **연산 재조직** (tiling·IO 재배치) 이었지 바이트 수감축이 아니었다 — 그럼에도 attention 이 도움받은 것은 그 IO 재조직이 prefill 과 decode 양쪽에서 실효를 냈기 때문이다 (FlashDecoding). 이 가족의 decode 는 다르

다: 매 token 이 전체 state 를 read-modify-write 하고 그 트래픽 $m d^2 L_{\text{layer}}$ 은 문맥 길이 S 에 무관하게 고정이며 (claim 1·2), chunk 축이 $C = 1$ 로 붕괴해 있어 21.3 의 $\sim 150 \times$ 제조 손잡이가 **아예 없다**. 커널이 최적화할 대상 (GEMM 재배치) 이 decode 에는 존재하지 않는 것이다. 병목은 순수하게 memory 계층의 RMW 대역폭이고, 이는 커널 문제가 아니라 소자 · 배치 문제다 ($\rightarrow$ 20 장 · 24 장). KV cache 와의 crossover S^* (anchor 에서 read-half 기준 $\approx 65k$ token, claim 2) 가 정확히 이 경계를 표시한다: S^* 아래에서는 append-once KV 가 싸고 위에서는 고정 RMW 인 이 라인이 싸지지만 — 어느 쪽이든 decode 가 움직이는 바이트 수는 커널이 아니라 memory 시스템이 정한다.

[평가] 따라서 hardware lottery 는 이 라인에서 **비대칭으로만** 작동한다. accelerator 가 이길 수 있는 곳과 memory-centric 접근이 필요한 곳의 경계는 정확히 “chunk 축이 있는가” 의 경계이고, 그 경계는 pair thesis 의 split ($\rightarrow$ 18 장) 과 **정확히 겹친다**. training/prefill 에는 축이 있어 kernel 승리가 가능하고 — 그래서 accelerator 영역 — decode 에는 축이 없어 kernel 이 못풀고 memory 계층 · RMW 대역폭이 병목으로 남는다 — 그래서 memory-centric 영역 ($\rightarrow$ 20 장 · 24 장). FlashAttention 이 attention 에게 준 양면 승리를, 이 가족은 구조적으로 **한 면**밖에 받을 수 없다. 이 비대칭이 24 장이 제안을 **쌍**으로 내는 이유의 하드웨어-역사적 근거다.

예측 4 — 누가 그 커널을 쓸 동기가 있는가는 player 마다 다르다. Google Research 는 자사 long-context 제품과 TPU pod 때문에, NVIDIA 는 GDN 계보의 연장선 때문에, FLA 커뮤니티는 오픈소스 커버리지 완성 때문에 각각 다른 강도의 동기를 갖는다. 이 player 별 유인 구조가 다음 장의 주제다.

21.6 이 독법의 한계 (정직성각주)

[평가] hardware-lottery 독법은 dossier § 5 가 A5 로 분류한대로 **framing** 으로 강하고 **standalone** 기여로는 약하다. 이 장은 예측을 내지만 그 예측들은 아직 검증되지 않았다 — fused deep-memory 커널이 나올지, 나오면 채택이 실제로 따라올지는 미래의 사실이다. 또한 이 독법은 “수학이 중요하지 않다” 는 강한 주장이 **아니다**. in-context retrieval 격차 (attention 53.55 vs 43.70 [Atlas Table 5], FDA 67.3 vs 41.9 [NL], $\rightarrow$ 14 장 · 16 장) 는 어떤 커널도 닫지 못한다 — 커널은 wall-clock 을 풀지 품질을 풀지 않는다. capacity 이론이 그 격차의 이유 (ϕ^* 의 무한 capacity) 까지 말해 주므로, 완성형은 여전히 attention 과의 hybrid 일수 있다. hardware lottery 는 **어느 아이디어가 배포되는가**를 설명하는 렌즈이지, **어느 아이디어가 옳은가**를 판정하는 저울이 아니다. 이 구분을 지키는 한에서 이 장의 예측은 유효하다.

요약

- **hardware lottery** (Hooker 2020): 아이디어는 품질이 아니라 당대 하드웨어와의 정렬로 이긴다. attention 이 scaling 시대를 이긴 것은 표현력이 아니라 두 큰 $\text{matmul}(QK^T, PV)$ 이 만드는 **GEMM density** 덕이고, KV cache 가 문맥에 비례해 자라는 것은 압축을 포기하고 완벽한 matmul 을 유지한 그 거래의 명세서다.
- GEMM density 가 승리 조건이 된 것 자체가 역사적 우연이다: Dennard scaling 이후 지수적으로 는 것은 clock 이 아니라 systolic array (Kung - Leiserson 1978 $\rightarrow$ TPU MXU / tensor core) 가 값싸게 뽑는 dense-matmul throughput 이었고, 이 구조는 sequential-irregular access 에 페널티를 매긴다. RNN/LSTM (2014 - 2017 SOTA) 이 밀린 것은 표현력이 아니라 넓어지는 throughput 을 굶겨서였다 (교과서적 hardware-lottery 패배). attention (2017) 과 tensor core (Volta 2017) 는 같은 창에서 만나 공진화 (lock-in) 했다.
- 이 neural-memory 라인은 본질상 recurrent 인데도 로또의 세를 **선불**했다: chunkwise stale-snapshot ($\rightarrow$ 9 장), Newton - Schulz/Muon (“tensorize and maximize matmuls” , $\rightarrow$ 14 장), periodic reset (context parallelism, $\rightarrow$ 15 장), featurized keys — 네

수 모두 arithmetic intensity 를 제조한다. claim 7 의 load-bearing 결론은 “chunk C 로 GEMM density 를 제조할 수 있다 (곡선 모양과 crossover 의 존재)”이며, C^* 의 절대 위치는 ridge 의존적이라 이전하지 않는다. plain-JAX TNT 가 32K 에서 step 당 FlashAttention 을 이긴 것 ($\rightarrow$ 15 장) 이 그 제조력의 증거다.

- 그러나 알고리즘 승리 뒤에 필요한 **kernel 승리**가 아직 없다. FLA(5,325 ★) 는 matmul-clean 한계보 (DeltaNet/GDN/GLA/RWKV-7/Mamba-2) 를 Triton 으로 전부 커버하지만, deep-memory+momentum Titans 는 fla/ops/titans 의 naive PyTorch 에머문다 (TTT 는 Triton 화, Titans 는 #214 에서 정체). 원인은 기계적이다: GDN 은 chunk 내 $\prod(I - \eta_t k_t k_t^\top)$ 가 WY/UT 표현으로 하나의 matmul 로 접히지만, Titans 는 (i) deep memory 의 gradient 가 rank-1 이 아니고 (ii) momentum 이 scan combine 을 companion 행렬로 만들어 fla/ops/titans 가 $L \times L$ 하삼각 행렬을 materialize 하게 강제한다 — [TNT] 의 존재 이유가 생태계에 남긴 실물 흔적. 단 MesaNet 의 chunk_cg_solver(arXiv: 2506.05233) 가 2 차 방향의 커널화가 원리적으로 가능함을 보인다.
- 예측: (1) 채택은 kernel 가용성이 정한다 (GDN·Mamba-2 는 갔고, deep-memory 는 대기) — 라인 내부의 hardware lottery 재현; (2) 이 가족의 FlashAttention-moment = momentum·deep MLP-reset 을 융합한 fused chunk kernel(그 전까지 deep memory 는 <5 – 10% FLOPs util, [Atlas]); (3) **비대칭** — 그 moment 은 chunk 축이 있는 prefill/training 절반만 풀고, $C = 1 \cdot \text{memory-bound} \cdot 394 \times \text{state-지배인 decode}$ (claim 1) 는 못 푼다; 이 경계가 pair thesis split 과 정확히 겹친다 ($\rightarrow$ 18 장 · 24 장); (4) player 별 유인은 다르다 ($\rightarrow$ 22 장).
- 이 독법은 framing 으로 강하고 falsifiable 한 기여로는 약하다. 커널은 wall-clock 을 풀지 retrieval 격차 (53.55 vs 43.70) 를 풀지 못한다 — hardware lottery 는 무엇이 배포되는가의 렌즈이지 무엇이 옳은가의 저울이 아니다.

자가 점검체크리스트

- attention 의 승리를 표현력이 아니라 GEMM density 로 설명하고, KV cache 를 그 거래의 부산물로 재서술할 수 있다.
- GEMM density 가 왜 승리 조건이 됐는지를 systolic array/tensor core 공진화와 RNN 의 hardware-lottery 패배로 역사화할 수 있다.
- GDN 이 커널화되는 이유 (WY/UT 접힘) 와 Titans 가 안 되는 이유 (rank-1 아님 + companion scan $\rightarrow L \times L$ materialize) 를 기계적으로 대비할 수 있다.
- 이 라인이 matmul 안에 설계된 네 가지수 (chunkwise·NS·reset·featurized keys) 를 각각 어느 장이 소유하는지와 함께 열거할 수 있다.
- claim 7 에서 무엇이 load-bearing 이고 (곡선 모양 · crossover 존재) 무엇이 이전 불가인지 (C^* 절대 위치) 를 구분할 수 있다.
- FlashAttention-moment 이 무엇이었는지, 이 가족에 왜 아직 없는지를 FLA/Titans-naive 증거로 설명할 수 있다.
- 이 가족의 kernel moment 이 pair thesis 의 어느 절반만 푸는지, 그 비대칭이 왜 pair thesis split 과 겹치는지 논증할 수 있다.
- hardware-lottery 독법이 판정하지 못하는 것 (retrieval 격차, 아이디어의 옳음) 을 짚을 수 있다.
- 이 장의 논지를 inference 어휘로 옮길 수 있다: “attention 은 GEMM 로 토를 이겼고 FlashAttention 으로 kernel 로 토까지 이겼다; 이 라인은 GEMM 로 토의 세를 선불했으나 kernel 로 토는 절반만 이길 수 있다.”

다음 장으로

이 장은 kernel 승리가 아직 비어 있다고 진단하고, 그 승리를 쓸 동기가 player 마다 다르다고 예고했다. 22 장은 그 유인 구조를 편다 — 왜 Google Research 가 이 라인의 자연스러운 저자인지

(TPU pod·JAX·long-context 제품), NVIDIA 가 GDN 계보로 어디에 서 있는지, flash-linear-attention 생태계가 오픈소스 커널의 무게 중심을 어떻게 쥐고 있는지, 그리고 각 player 가 이 라인의 FlashAttention-moment 앞에서 합리적으로 두는 다음 수는 무엇인지.

ch22. Player-strategy 분석: 누가 TTT scaling에 어떻게참여하는가

이 장의 목표 — 독자가 이 장을 마치면 (1) 이 라인의 세 player — Google Research, NVIDIA, open-source kernel 생태계 — 가 각각 어떤 자산을 쥐고 있고 그 자산이 어떤 다음 수를 합리적으로 만드는지 설명할 수 있고, (2) 왜 이 연구 프로그램이 Google Research에서 나오는 것이 자산 구조상거의 필연이었는지 논증할 수 있으며, (3) [TNT]가 custom kernel 없이 stock TPU 위 plain JAX로 FlashAttention을 이긴 사실이 player 경쟁에 대해무엇을 예고하는지 — kernel 성숙도가 아직 아키텍처 승부를 결정하지 못했다는 것 — 을 진술할 수 있어야 한다. **왜 필요한가** — 21 장은 hardware-lottery 독법으로 “이 라인이 왜 matmul 안에 설계되어 있는가”를 물었다 (chunkwise-NS-reset이 전부 GEMM density를 사는 수다). 그 독법의 자연스러운 다음 질문은 전략적이다: 그 게임을 실제로 두는 주체는 누구이며, 각자의 자산이 서로 다른 수를 강제하는가. 이 장은 기술 분석이 아니라 그 전략 분석이다. Part III 의척추인 D4 pair thesis(→ 18 장 § 18.3)는 여기서 player 지도로 번역된다 — decode 반쪽 (memory-centric)과 training/prefill 반쪽 (accelerator)에 각 player의 자산이 비대칭적으로 걸려 있기 때문이다.

22.1 Bridge-in: hardware lottery에서 player 지도로

21 장의 결론은 서술적이었다: attention은 GEMM density로 hardware lottery를 이겼고, 이 라인은 그 교훈을 내면화해 자기 알고리즘을 dense matmul로 다시 빚었다 (chunkwise anchoring으로 exactness를 GEMM으로 바꾸고, Newton-Schulz로 orthogonalization을 matmul로 바꾸고, reset으로 non-linear recurrence를 병렬 shard로 바꾼다, → 9 장·14 장·15 장). 이 장은 그 서술을 행위자에게 귀속시킨다. 누가 이 수들을 두었고, 누가 다음 수를 둘 자산을 쥐고 있는가.

이 장의 대부분은 논문이 보고한 사실이 아니라 이 책의 전략적 판단이다. 따라서 § 6.2 규칙에 따라 판단은 **[평가]** 블록으로 명시하고, 사실 — 어느 스택에서 무엇이 측정되었는가 — 만 산문으로 단정한다. 정직하게 밝혀 둔다: player-strategy 분석은 falsifiable한 실험 주장이 아니라 자산 구조에서 다음 수를 읽는 framing이며, 그 자체로는 검증 대상이 아니다. 이 장의 무게는 근거가 되는 사실 (누가 어떤 하드웨어에서 무엇을 냈는가, crossover가 어디인가)의 단단함에서 나오지, 예측의 확실성에서 나오지 않는다.

22.2 Google Research가 자연스러운 저자인 이유

여섯 편은 모두 한 조직에서 나왔다 (Behrouz et al., Google Research, 일부 USC 공저). 이것은 우연이 아니라 자산 구조의 귀결이다. 이 라인을 검증하려면 세 자산이 한 지붕 아래 있어야 하고, 2024-2026 시점에 그 셋을 동시에 전 곳은 사실상 Google Research 뿐이었다.

자산 1 — TPU pod + JAX/XLA 스택. 이 라인의 핵심도박은 “non-linear deep-memory recurrence를 dense matmul로 다시 빚으면 기존 accelerator에서 경쟁력이 난다”이다. 그 도박이 참인지는 large-GEMM에 최적화된 하드웨어와, custom kernel 없이도 그 GEMM을 잘 뽑는 컴파일러가 있어야 검증된다. TPU의 systolic array는 큰 dense GEMM에 특화된 소자이

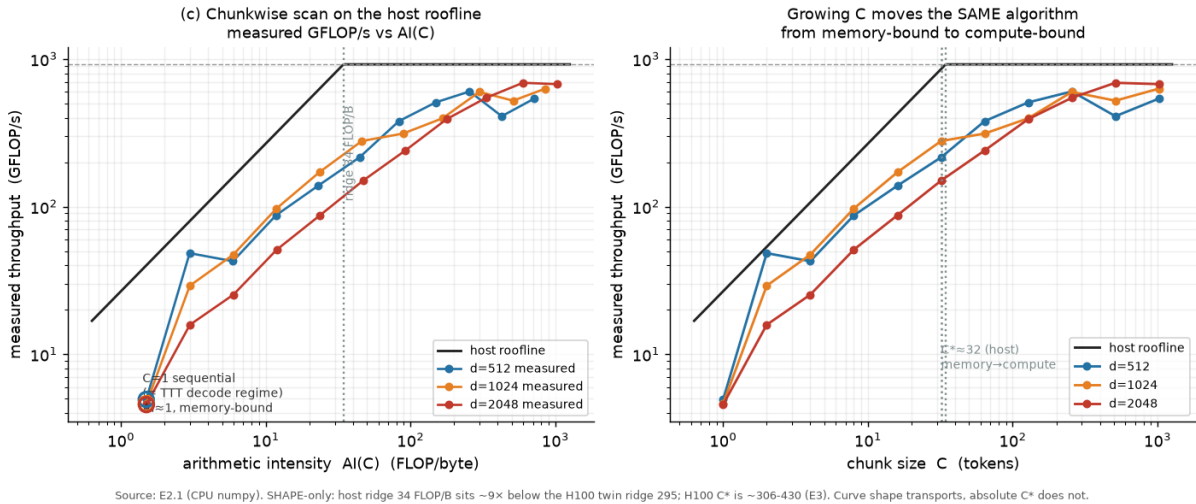


그림 22-1: chunk 크기 C 가 roofline의 x 축 — $C=1$ (decode)은 memory-bound, C 를 키우면 compute-bound로 오른다

고, XLA는 그 위로 JAX 프로그램을 융합·스케줄한다. chunk 크기 C 를 키우면 같은 알고리즘이 memory-bound에서 compute-bound로 오른다는 것 (claim7, → 15 장·9 장)이 이 스택에서 곧바로 throughput으로 환원된다 — C 가 roofline의 x 축이고, TPU+XLA가 그 x 축의 오른쪽 (큰 C)을 값싸게 만든다.

그림 22-1 — chunk 크기 C 가 roofline의 x 축이다: $C=1$ (per-token RMW = decode 영역)은 memory-bound 평원에 있고 C 를 키우면 crossover를 넘어 compute-bound로 오른다. TPU+XLA 스택의 자산은 이 곡선의 오른쪽 구간 (대형 dense GEMM)을 custom kernel 없이 값싸게 만든다는 데 있다 (실험 E2.1 재구성; host ridge는 H100 twin ridge의 약 1/9 이므로 곡선의 모양만 이전, 측정 $C^* \approx 32$ 는 host 값 — H100 closed-form은 306–430).

이 자산의 결정적 증거가 [TNT]다. 150M Titans를 10B tokens로 TPUv5 pod 위 plain JAX(custom kernel 없이)로 훈련한 결과, 가장 정확한 Titans baseline($C=64$) 대비 target loss 까지 $17.4 \times$ 빠르면서 평균 perplexity를 오히려 개선했고 (23.09 vs 25.07), 32K 문맥에서는 $C_L=128$ 의 순수 JAX TNT가 FlashAttention(Pallas kernel)보다 step 당 $1.3 \times$ 빠르다 [TNT § experiments]. 이 사실의 함의는 § 22.4·§ 22.7에서 되짚지만, Google 자산 논증에 국한하면 명료하다: **CUDA/cuDNN moat를 우회한 채로도 이 라인이 성립함을 Google 스택이 자기 하드웨어 위에서 이미 보였다.** 아키텍처를 matmul로 빚어 두면 컴파일러가 kernel 성숙도의 공백을 상당부분 메운다.

이 증거를 더 정밀하게 뜯으면 자산 1은 세 겹이다. 첫째, **pod 경제학**. [TNT]의 검증은 TPUv5 pod($2 \times 2 \times 2$, model parallelism 2) 위에서 돌았고, TNT가 non-linear recurrence를 세 조각으로 분해한 구조 — L/C_g 개의 global handoff (16k 문맥 $\cdot C_g=2048$ 에서 단 8회, 각 handoff가 2048-token 위 거대 batched matmul), device 간 state 교환이 0인 L/L_s 개의 독립 local shard, $d \times d$ carry 하나짜리 prefix-scan — 는 정확히 pod의 자원 배치 (device 격자 + model parallelism)에 얹히도록 설계되었다 [TNT § systems]. deep-memory training의 악명 높은 $<5-10\%$ FLOPs utilization이 FLOP 부족이 아니라 arithmetic intensity 부족이라는 진단, 그리고 그것을 “병렬 work unit을 크게 (global) 혹은 많고 독립적으로 (local) 만들어” 메운다는 처방은, pod라는 자산이 있어야 값이 매겨진다.

둘째, **컴파일러 moat**. NVIDIA의 해자가 CUDA/cuDNN kernel이라면 Google의 대응 자산은 XLA라는 컴파일러 층이다 — hand-written kernel 없이 JAX 프로그램에서 fusion·scheduling을 뽑아낸다. 두 해자는 층위가 다르다: CUDA는 kernel 층에서, XLA는 컴파일러 층에서 GEMM을 값싸게 만든다. 이 라인처럼 알고리즘이 dense matmul로 빚어진 경우, 컴파일러 층의 해자가 kernel 층의 해자를 상당 부분 대체한다.

셋째, **그 대체의 실증**. TNT가 이긴 상대 FlashAttention은 TPU에서 Pallas(TPU의 kernel 언어, Triton의 TPU 대응물)로 손으로 쓴 kernel이다. 즉 32K에서 벌어진 승부는 plain-JAX(컴

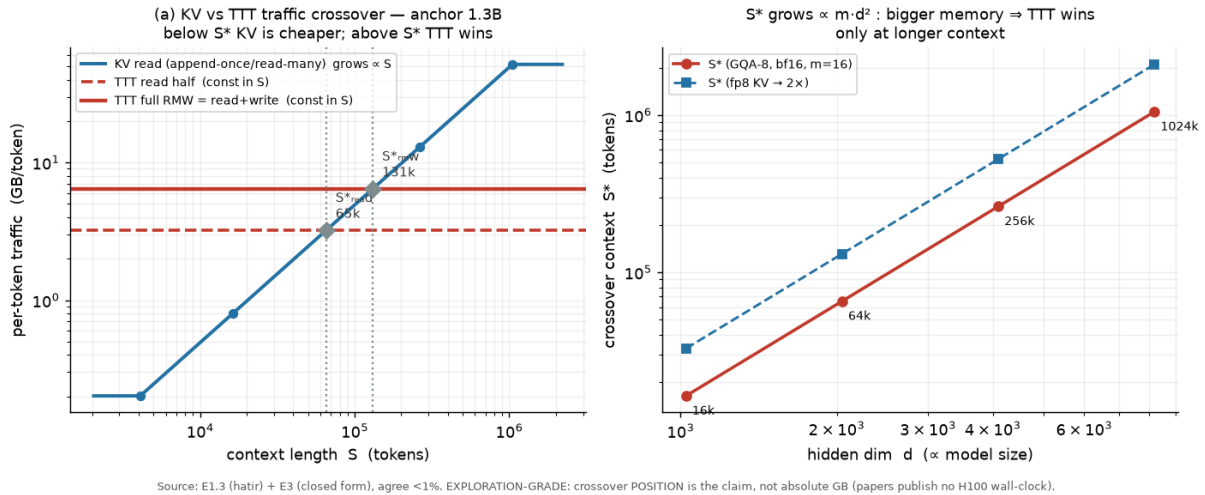


그림 22-2: KV(문맥에 비례해 자라는 read)와 TTT(문맥에 무관한 RMW)의 트래픽 crossover S^*

파일러가 뽑은 코드) 대 hand-Pallas(사람이 쓴 kernel)의 대결이었고, 컴파일러 쪽이 step 당 $1.3\times$ 로이졌다. 이 라인의 검증에 필요한 것은 성숙한 kernel 생태계가 아니라 성숙한 컴파일러이며, 그 컴파일러를 자기 하드웨어 위에 이미 가진 곳이 Google 이라는 것 — 이것이 자산 1 논증의 핵심이다.

자산 2 — long-context 제품. decode 절반의 경제학은 crossover 문맥 길이 S^* 에 걸려 있다. KV cache 읽기 트래픽은 문맥에 비례해 자라지만 TTT state의 RMW 트래픽은 문맥에 무관하게 일정하므로, 그 아래에서는 KV가 싸고 그 위에서는 TTT가 싼 crossover가 존재한다 (claim2, $\rightarrow$ 18장 §18.3). anchor에서 read-crossover는 약 65k token, 전체 RMW-crossover는 약 131k token이며, 폭에 따라 16k $\rightarrow$ 1.05M token으로 이동한다.

그림 22-2 — KV(문맥에 비례해 자라는 read)와 TTT(문맥에 무관한 RMW)의 트래픽 crossover S^* . TTT의 경제적 이점이 나타나는 문맥 길이대 (약 65k – 131k token 이상)가, 정확히 long-context 제품이 판매하는 영역이다 (실험 E1.3/E3 재구성; crossover 위치만 load-bearing, 절대 $\mu\text{s}/\text{token}$ 은 roofline 하한으로 이월).

[평가] crossover가 놓인 자리가 전략적으로 결정적이다. TTT가 KV보다 유리해지는 문맥 길이는 대략 수만~수십만 token 이상인데, 이것은 정확히 Google이 제품으로 파는 regime(장문맥 Gemini 계열)이다. 즉 이 라인의 경제적 이점이 발현되는 지점과 Google 제품 라인의 판매 지점이 겹친다. long-context를 파는 조직만이 이 라인을 자기 제품 곡선 위에서 정당화할 수 있다 — 짧은 문맥만 서빙하는 곳에는 crossover 아래라 KV cache가 여전히 싸고, 이 라인을 도입할 사업적 이유가 약하다.

[평가] 이 겹침은 정적인 관찰이 아니라 동적인 압력이다. claim2가 보이듯 dense-attention KV의 per-token decode latency는 문맥에 따라 오른다 (2k $\rightarrow$ 32k에서 0.79 $\rightarrow$ 2.58 ms/token; 절대치는 roofline 하한, load-bearing은 상승방향뿐) — 문맥을 길게 파는 제품일수록 이 read-many 벽이 단위 token 당 비용으로 직접 나타난다. 반대로 TTT state의 per-token latency는 문맥에 무관하게 일정하다 (anchor 1.9 ms/token, 역시 하한). 초장문맥 (수십만~1M token; 70B에서 crossover가 약 1.05M token으로 이동, claim2 scaling)을 서빙할수록 KV의 문맥-비례 벽이 커지고, crossover 위에서 state가 싸지는 이 라인의 이점이 제품 곡선 위에서 실현된다. 그래서 long-context를 공격적으로 파는 조직일수록 이 압력을 먼저 받고, 이 라인을 자기 제품 위에서 정당화할 동기도 강하다. 이 압력의 방향 (문맥이 길수록 KV가 비싸지고 state가 유리해진다)만이 논증에 실리며, Gemini 제품 규모 같은 6편 밖 정보는 framing으로만 쓰인다.

자산 3 — 연구팀과 저작권. 여섯 편의 개념 소유권 (Titans의 deep memory, Miras의 taxonomy, Atlas의 capacity 이론, TNT의 training 경제학, NL의 nested ontology, Sleep의

lifecycle) 이 한 팀에 누적되어 있다. 후속수의 설계 지식이 조직 내부에 있다는 것 자체가 재현 비용이 큰 자산이다.

[평가] 세 자산은 곱셈적이다. TPU+XLA 만으로는 이 라인을 팔제품이 없고, long-context 제품만으로는 도박을 검증할 스택이 없으며, 연구팀만으로는 검증도 배포도 못한다. 셋이 한 지붕 아래 있어야 “chunkwise-into-matmul 도박 → 자기 하드웨어에서 검증 → 자기 장문맥제품에 배포” 의 페루프가 돈다. 그래서 이 프로그램이 Google Research 에서나온 것은 취향이 아니라 자산 구조의 귀결이며, 저자이자 자연스러운 첫 배포자라는 지위가 여기서 나온다.

22.3 Google 이 합리적으로 두는 다음수

자산이 다음 수를 강제한다. Google 의 합리적 세 수는 모두 자기 자산에 정렬된다.

수 1 — deep-memory fused kernel. [TNT] 는 custom kernel 없이도 이겼지만, 동시에 kernel-optimized Gated Transformer 에는 time-to-loss 에서 아직 진다 (0.96h vs 1.12h) 고 스스로 보고하고, fused kernel 을 명시적으로 미래 과제로 남긴다 [TNT § systems]. 그 kernel 의 본진이 decode 다: claim1 대로 decode step 은 fast-weight state 전체의 memory-bound RMW(anchor 에서 6.44 GB/token, arithmetic intensity 0.59 FLOP/byte 로 결정적 memory-bound, state 트래픽이 GEMV 연산을 약 $394\times$ 압도) 이므로, per-chunk 의 batched forward+backward 와 cumulative-sum state update 를 하나로 융합하는 kernel 이 남은 격차를 달을 자리다. 이 수는 TPU 자산에 정확히 얹힌다.

수 2 — scale up. 라인 전체의 from-scratch 실증 상한은 1.3B params / 100B tokens 이고 ([TNT] 는 150M), chunk-size mismatch 현상조차 550M 단일 그림에서만 관측되었다 (§ 6.4 의 무 caveat, → 15 장). [Titans] 가 2024-12 에 약속한 “larger models” 는 끝내 배달되지 않았다. momentum·deep-memory·self-modification 의 이점이 7B+·SFT/RLHF·production 데이터에서 유지되는지는 이 라인 최대의 미지수다 (→ 18 장 § 18.2 유보 1). 이 규모의 from-scratch 검증은 값싸게 돌릴 수 있는 자산은 소수의 조직에만 있고, Google 은 그중 하나다. 이 방향의 약한 방증은 [TNT] 내부에 있다: local memory 수를 0 → 4 로 늘리면 (=state 를 늘리면) 평균 perplexity 가 단조감소한다 (23.53 → 20.15, E4 support). 더 많은 state 가 품질을 산다는 신호이지만, 이득이 포화하고 (+1 에서 +4 까지 21.04 → 20.15) tokenizer·데이터가 고정된 한 setting 안의 ordinal 관찰이라 scaling law 로 승격되지 않는다 (→ 19 장). scale-up 이 지불할 값과 되돌려줄 값의 곡선을 실제로 긋는 것 자체가 미해결이며, 그 곡선을 값싸게 그릴 위치에 Google 이 있다는 것이 이 수의 요지다.

수 3 — hybrid 로 제품에 삽입. in-context retrieval 격차가 측정된 채 닫히지 않았다 (attention 53.55 vs 43.70 [Atlas Table 5], FDA 67.3 vs 41.9 [NL], → 14 장·16 장). 따라서 합리적 배포는 순수 TTT 가 아니라 attention 과의 MAG/MAC hybrid 이며 (→ 12 장), crossover 위문맥에서만 TTT 경로를 켜는 adaptive 배치다 — crossover 아래에서는 KV 가 짜므로 TTT 를 켤 이유가 없다 (claim2). 이 수는 자산 2(long-context 제품) 에 얹힌다.

[평가] 세 수가 모두 Google 자산에 정렬된다는 사실이 § 22.2 의 “자연스러운 저자” 논지를 완성한다. 저자이자 첫 배포자일 뿐 아니라, 다음 수를 값싸게 둘 수 있는 유일한 위치에 가깝다. 다만 이 정렬은 Google 이 반드시 둔다는 예측이 아니라, 두면 다른 누구보다 저비용이라는 자산 판정이다.

22.4 NVIDIA 의 위치: Gated DeltaNet 계보와 kernel moat

NVIDIA 는 이 라인 (deep memory) 의 저자가 아니지만, 인접 계보인 linear memory 의 주요 player 다. 그 접점이 Gated DeltaNet(이하 GDN, Yang et al. 2024) 이다. GDN 은 delta rule 에 retention 을 더한 **linear-state** 모델로 (§ 1.6 카탈로그: $W_t = \alpha_t W_{t-1}(I - \eta_t k_t k_t^\top) + \eta_t v_t k_t^\top$), 이 라인의 deep memory 가 특수 경우로 흡수한다고 [Titans] 가 자기 Appendix C 에서 보인 바로 그 계보에 있다.

NVIDIA의 자산은 두 가지다: GPU의 CUDA/cuDNN moat, 그리고 linear-attention kernel의 성숙도. 그러나 여기 전략적 함정이 있다. **linear-attention SRAM chunk kernel**(GLA/DeltaNet/GDN 계보)은 **deep memory에 port되지 않는다** — inter-chunk state propagation이 non-linearity(MLP forward/backward, LayerNorm)를 통과하기 때문이며, parallel scan이 적용되지 않는 바로 그 이유다 [TNT § systems]. 즉 NVIDIA가 성숙시킨 kernel 자산은 linear 쪽에서만 통하고, deep-memory 쪽에서는 TNT의 reset trick이 현재 유일한 지렛대다.

[평가] NVIDIA의 합리적 수는 자산 방어에서 나온다. 수 A — “**linear로 충분하다**”를 논증. crossover 아래 (대략 <65k token)에서는 KV/linear가 이기므로 (claim2), 대부분의 제품 문맥이 그 아래에 있는 한 GDN 계보로 충분하다는 방어가 성립한다. NVIDIA의 이익은 아키텍처가 linear에 머무를 때 커진다 — 그쪽에 성숙한 kernel이 이미 있기 때문이다. 수 B — **deep memory가 이기면 kernel로 재진입**. 만약 deep memory가 crossover 위 제품에서 승기를 잡으면, NVIDIA는 아키텍처 승부가 아니라 deep-memory fused kernel을 CUDA로 내는 kernel 승부에서 이기려 할 것이다. 어느 쪽이든 NVIDIA의 승부처는 아키텍처가 아니라 kernel/하드웨어다.

그런데 이 방어를 정확히 겨냥해 흔드는 것이 § 22.2의 TNT 사실이다. TNT가 stock TPU 위 plain JAX로, 즉 **custom kernel 없이** 32K에서 FlashAttention을 이겼다는 것은, 이 라인에서 CUDA moat가 예상보다 약하다는 신호다. 아키텍처가 dense matmul로 빔어져 있으면 XLA 같은 컴파일러가 kernel 우위의 상당 부분을 상쇄한다. NVIDIA에게 이것은 “성숙한 CUDA kernel = 지속 우위”라는 등식이 이 라인에서는 자동으로 성립하지 않는다는 위협이다.

NVIDIA와 kernel 생태계의 incentive 정렬. NVIDIA의 kernel 자산은 순수히 사내에 있지 않다. linear-attention kernel을 실제로 성숙시키는 노동의 상당 부분은 community 프로젝트인 flash-linear-attention(→ § 22.5)에서 일어나고, GDN 자체가 그 계보의 모델이다. 여기 구조적 정렬이 있다: 그 kernel들은 Triton/CUDA로 GPU 위에서 돌고, 따라서 kernel이 성숙할수록 하드웨어 수혜자는 NVIDIA다. 즉 NVIDIA는 linear 계보 kernel 생태계를 반드시 사내에서 소유하지 않고도 그 성숙의 이득을 하드웨어 판매로 회수한다 — 형식적 후원이라기보다 incentive 정렬이다.

[평가] 이 정렬이 왜 전략적으로 중요한가. deep memory 쪽에는 아직 이 정렬이 없다 — deep-memory fused kernel이 존재하지 않으므로 성숙시킬 community 노동도, 그 수혜를 회수할 구조도 비어 있다. 그래서 NVIDIA의 수 A(“linear로 충분”)는 이미 정렬된 생태계를 지키는 저비용 방어이고, 수 B(deep-memory CUDA kernel 재진입)는 정렬을 새로 만들어야 하는 고비용 공세다. 자산이 방어를 싸게, 공세를 비싸게 만든다 — NVIDIA가 아키텍처 승부가 아니라 kernel 성숙의 timing에 베팅하는 것이 합리적인 이유가 여기 있다.

22.5 Open-source kernel 생태계: flash-linear-attention

세 번째 player는 조직이 아니라 생태계다. flash-linear-attention(fla)은 GLA·DeltaNet·GDN·RWKV 등 linear-memory 모델의 Triton kernel을 모은, 이 계보의 사실상 kernel 인프라다. 그러나 fla 역시 NVIDIA 자산과 같은 함정 위에 있다: **linear state를 전제**하므로, deep memory(TTT/Titans/Atlas)의 non-linear recurrence에는 그대로 통하지 않는다. deep-memory fused kernel은 open-source 쪽에서도 아직 공백이며, 이 공백이 21장이 말한 “이 가족이 아직 기다리는 FlashAttention-moment”의 kernel 측면이다.

공백은 kernel에 그치지 않는다. serving 스택도 RMW state를 관리하지 못한다. 기존 KV-cache manager의 semantics — content-addressed prefix 재사용, append-only placement — 는 RMW state에 대해 범주 오류다 (claim6, → 23장): 재사용률이 구조적으로 0이고 (내용이 매 token 바뀌어 block hash가 재발하지 않는다), append-only가 stale 사본을 누적하며, dirty writeback이 값매김되지 않는다. TTT-state manager는 `update_in_place · mark_dirty/writeback · checkpoint/rollback · bind_to_sequence` 같은, KVCacheManager에 없는 event type을 필요로 한다.

[평가] open-source 의 합리적 수는 이 두 공백을 먼저 메우는것이다. 수 A — fla 를 deep memory 로 확장. deep-memory 용 Triton fused forward+backward chunk kernel(per-chunk batched fwd/bwd 와 cumulative-sum state update 의 융합) 을 먼저 내는팀이, 이 라인에서 linear-attention 시대의 FlashAttention 이 그랬던 역할 — 사실상의 kernel 표준 — 을 차지한다. 수 B — serving 인프라에 RMW API 를 추가. vLLM-class 스택에 claim6 의 missing API 를 얻는 팀이 decode 반쪽의 소프트웨어 열쇠를 준다. 두 수 모두 하드웨어 자산 없이 둘 수 있고, 그래서 open-source 가 진입 가능한 지점이다.

fla 의 거버넌스가 왜 병목인가. flash-linear-attention 은 조직이아니라 거버넌스가 느슨한 community repo 다 — kernel 은 개별 기여자가 model 별 (GLA·DeltaNet·GDN·RWKV) 로 Triton 으로 올리고, 표준화 · 유지보수는 소수 maintainer 에 의존한다. 이 구조는 linear 계보에서는 강점이었다: model 이 하나 나올때마다 대응 kernel 이 빠르게 붙어 사실상의 표준이 됐다. 그러나 deep memory 에는 같은 구조가 병목이다. deep-memory fused kernel 은 per-chunk batched forward+backward(non-linear MLP 의 2 차 항 포함) 와 cumulative-sum state update 를 하나로 융합해야 하는, linear kernel 보다 훨씬 큰 단위의공학이다 [TNT § systems]. 느슨한 거버넌스는 작고 독립적인 kernel 을 빠르게 모으는 데는 좋지만, 이런 크고 응집된 kernel 하나를 밀어붙이는데는 약하다. 그래서 이 공백은 “누군가 곧 PR 을 올린다” 로 저절로 메워지지않고, 자원을 집중 투입할 수 있는 주체 (Google·NVIDIA, 혹은자금을 받은 startup) 를 기다린다 — kernel 성숙의 timing 이 거버넌스 구조에걸려 있는 셈이다.

open-source 가 왜 구조적으로 중요한가. Google 이 논문으로 아키텍처를공개해도 TPU kernel 과 내부 serving 스택은 열지 않는다. GPU 세계에서 이라인을 돌리려면 그 공백을 누군가 메워야 하고, 역사적으로 그 역할은 open-source kernel 생태계가 맡아 왔다 (linear-attention 에서 fla 가그랬듯). 즉 Google 이 아키텍처를 열수록 open-source 의 기회는 커진다.

22.6 학계 · 스타트업의진입점

세 대형 player 밖에도 진입점이 있고, 그 위치는 자산 지도에서 정확히읽힌다. 진입 가능한 지점은 하드웨어 자산 없이 둘 수 있는 수 — 소프트웨어 · 측정 · 이론 쪽 — 에 몰린다.

학계의 진입점은 아직 잠긴 이론과 측정이다. 6 편 어디에도 chunkwise staleness 의 error bound 가 없고 (→ 9 장), deep-memory·windowed·self-modifying 변형의 regret/capacity/expressivity 정리도 linear 특수 경우 밖에서는 비어 있다 (dossier § 3.2 가 저자들 스스로 지목한 공백). 이것은 하드웨어 없이 종이와 증명으로 둘 수 있는 수이며, 이 라인의 다음논문이 달아야 할 자리다. 측정 쪽도 마찬가지다: 이 라인은 decode throughput·latency 를 한 줄도 공개하지 않았고 (→ 20 장·23 장), 우리 warrant 조차 절대치는 roofline 하한이다 (정직성 계약). 실제 kernel 로 crossover S^* 를 head-to-head 로재는 일 (claim2 의 A100 이월 항목) 은 GPU 몇 장이면 되는, 소규모 연구실이척 저자로 설 수 있는 측정이다.

스타트업의 진입점은 decode 반쪽의 소프트웨어다. claim6 이 짚은 KV-cache manager 의 범주 오류 — content-addressed 재사용률이 구조적으로 0, stale 사본 $256\times$ 누적, 값 안 매겨진 dirty writeback — 는 곧 시장공백이다. RMW state 를 관리하는 serving 계층 (update_in_place·mark_dirty/writeback·checkpoint/rollback·bind_to_sequence) 은 하드웨어 없이 소프트웨어로 짜는 제품이고, per-session weight state 를 새 cache class 로다루는 인프라 (→ 23 장) 는 vLLM-class 스택 위에서 독립 제품이 될 여지가있다. 하드웨어 자산이 없다는 바로 그 제약이 이들을 소프트웨어 수로몰아넣고, 그 수들이 마침 decode 반쪽에 집중되어 있다.

[평가] 진입점의 공통 문법은 하나다 — 자산의비대칭이 곧 진입로의 지도다. 대형 player 가하드웨어 · 컴파일러 · 제품을 켜 자리 (training 반쪽) 는 진입이 비싸고, 아무도 아직 쥐지 못한 자리 (decode 소프트웨어, 이론, 측정) 는 진입이 싸다. 학계 · 스타트업의 합리적 수는후자에 정확히 정렬되며, 이것은 다음 절의 분업 구조가 대형 3 인에 국한되지 않고 진입자에게까지 확장된다는 뜻이다.

22.7 세 player의 균형과 pair thesis

player 지도를 Part III의 척추인 D4 pair thesis 위에 겹치면 비대칭이 선명해진다. decode 반쪽 (memory-centric: unshared·write-heavy·비-content-addressable 한 whole-state RMW)의 승부는 serving 인프라와 RMW-aware 소프트웨어에서나고 (claim1·claim2·claim6), training/prefill 반쪽 (accelerator: chunk C = roofline x 축)의 승부는 kernel과 대형 GEMM 스택에서 난다 (claim7).

이 비대칭 위에 세 player를 놓으면, 어느 한 player도 pair의 양쪽을 다 쥐지 못한다.

player	강한 자산	pair 상 위치	합리적 다음 수
Google Research	TPU+XLA, long-context 제품, 저작권	training 반쪽 (강)·아키텍처 저자	fused deep-memory kernel; scale up; hybrid 제품 배포
NVIDIA	CUDA moat, linear kernel 성숙도	accelerator 하드웨어 (강)·아키텍처 비저자	“linear로 충분” 방어; deep-memory 이기면 CUDA kernel 재진입
open-source(fla 등)	Triton kernel·serving 커뮤니티	decode 반쪽 소프트웨어 (진입 가능)	fla를 deep memory로 확장; serving에 RMW API 추가

표 22-1 — 세 player의 자산·pair 상 위치·합리적 다음 수.

Google은 training 반쪽과 아키텍처에 강하지만 GPU serving 생태계 밖에 있다. NVIDIA는 하드웨어·kernel에 강하지만 아키텍처의 저자가 아니고 그 자산이 linear 계보에 묶여 있다. open-source는 decode 반쪽의 소프트웨어 공백을 메울 수 있지만 하드웨어가 없다. pair thesis가 예측하는 것은 단일 승자가 아니라 **분업**이다 — 각 player가 자기 자산이 걸린 절반에서 다음 수를 두고, 완성형 (→ 18장)이 배포되려면 세 절반의 수가 모두 놓여야 한다.

이 분업을 game으로 정식화하면 payoff 구조가 드러난다. 각 player의 전략 공간은 {자기 반쪽에서 다음 수를 둔다, 남의 반쪽을 침범한다, 기다린다}이고, 침범 수는 언제나 자기 자산 밖이라 비용이 크다 (Google의 GPU serving, NVIDIA의 아키텍처 저작, open-source의 하드웨어). 따라서 각자에게 dominant strategy는 자기 자산이 걸린 반쪽에서 두는 것이고, 이 dominant strategy들의 조합이 곧 분업 균형이다. 단 이 균형에는 시한이 붙어 있다: deep-memory fused kernel이라는 한 조각은 세 player(그리고 자금을 받은 진입자) 누구든 먼저 둘 수 있고, 먼저 둔 자가 이 라인의 kernel 표준을 정의한다. 그래서 kernel 조각만은 협조적 분업이 아니라 first-mover-takes-standard의 경주다 — 나머지 절반이 각자 자산에 갇힌 분업인 것과 대조적으로, 이 한 칸만은 공유 경합지다. 이 긴장이 아래 마지막 함의로 이어진다.

마지막으로 § 22.2·§ 22.4에서 미룬 TNT-beats-FlashAttention 사실의 가장 넓은 함의를 짚는다. 성숙한 kernel(수년간 최적화된 FlashAttention)이 미성숙 경쟁자(custom kernel 없는 plain-JAX TNT)에게 32K에서 진다는 것은, 이 라인의 승부가 **아직 kernel이 성숙하기 전국면에 있다는 뜻**이다.

[평가] hardware lottery의 통상 독법은 “성숙한 kernel을 가진 연산이 이긴다”이다 (21장). 그러나 TNT의 사실은 그 등식이 이 라인에서 아직 잠겨 있지 않음을 보인다 — kernel 우위가 아직 아키텍처 승부를 결정하지 못했다. 이것이 세 player 모두에게 뜻하는 바는 하나다: **FlashAttention-moment가 오기 전, 지금이 진입 기회**의 창이다. deep-memory fused kernel을 먼저 내는 player(Google·NVIDIA·open-source 누구든)가 이 라인의 kernel 표준을 정의하고, 그 순간 창은 닫히기 시작한다. 이 라인의 player 경쟁이 “누가 그 kernel을 먼저 성숙시키는가”의 경주인 이유가 여기 있다.

요약

- 이 라인이 Google Research 에서 나온 것은 자산 구조의 귀결이다: TPU+XLA 스택, long-context 제품, 여섯 편의 저작권이 곱셈적으로 결합해 “chunkwise-into-matmul 도박 → 자기 하드웨어 검증 → 자기 장문맥 제품 배포” 의 페루프를 돌릴 수 있는 유일한 위치에 있다.
- 자산 1 은 세 겹이다: pod 경제학 (TNT 의 global/local 분해가 pod 자원 배치에 얹힘), 컴파일러 moat (CUDA=kernel 층, XLA=컴파일러 층), 그 대체의 실증 (32K 에서 plain-JAX 가 hand-Pallas FlashAttention 을 이김). 이 라인의 검증에 필요한 것은 성숙한 kernel 이 아니라 성숙한 컴파일러다.
- crossover 와 제품의 겹침은 동적 압력이다: KV 의 per-token latency 는 문맥에 비례해 오르고 state 는 일정하므로, 초장문맥을 팔수록 (70B 에서 crossover $\approx 1.05M$) 이 라인의 이점이 제품 곡선 위에서 실현된다.
- Google 의 합리적 다음 수 셋 (deep-memory fused kernel, scale up, hybrid 제품 배포) 은 모두 자기 자산에 정렬된다 — 저자이자 다음 수를 값싸게 돌 위치.
- NVIDIA 는 deep memory 의 저자가 아니라 인접 linear 계보 (Gated DeltaNet, 이하 GDN) 의 player 이며, 그 kernel 자산은 linear 에만 통한다 (deep memory 의 non-linear recurrence 에 SRAM chunk kernel 이 port 되지 않음). 합리적 수는 “linear 로 충분” 방어이거나, 지면 CUDA deep-memory kernel 로 재진입.
- open-source kernel 생태계 (flash-linear-attention) 도 같은 함정 위에 있어 deep-memory fused kernel 과 RMW-aware serving API 가 공백이다. fla 의 느슨한 거버넌스는 작은 kernel 을 빠르게 모으는 데는 강하지만 크고 응집된 fused kernel 하나를 밀어붙이는 데는 약해, 이 공백은 자원을 집중할 주체를 기다린다. NVIDIA 는 이 생태계를 사내 소유 없이 하드웨어 수혜로 회수하는 incentive 정렬 위에 있다 (linear 계보 한정).
- 학계 · 스타트업의 진입점은 하드웨어 없이 둘 수 있는 수에 몰린다: 학계=잠긴 이론 (staleness bound, 비-linear regret/capacity) 과 측정 (실제 kernel 로 S^*), 스타트업=decode 반쪽의 RMW-aware serving 소프트웨어. 자산 비대칭이 곧 진입로의 지도다.
- pair thesis 위에 겹치면 어느 player 도 양쪽 (decode 소프트웨어 + accelerator kernel) 을 다 쥐지 못하는 분업 구조가 드러나며, 완성형 배포는 세 절반의 수가 모두 놓여야 성립한다. game 으로 보면 각자 dominant strategy 는 자기 반쪽에서 두는 것이지만, deep-memory fused kernel 한 칸만은 first-mover-takes-standard 의 공유 경합지다.
- [TNT] 가 custom kernel 없이 stock TPU 에서 FlashAttention 을 이긴 사실의 함의: kernel 우위가 아직 아키텍처 승부를 결정하지 못했다 — FlashAttention-moment 이전인 지금이 진입 기회인 창이다.

자가 점검 체크리스트

- Google Research 를 자연스러운 저자로 만드는 세 자산과 그것이 곱셈적으로 결합하는 이유를 설명할 수 있다.
- 자산 1 의 세 겹 (pod 경제학, 컴파일러 moat, plain-JAX 가 hand-Pallas 를 이긴 실증) 을 구분해 말할 수 있고, “성숙한 kernel 이 아니라 성숙한 컴파일러” 라는 요지를 진술할 수 있다.
- crossover S^* 의 위치가 왜 long-context 제품 자산과 겹치는지, 그 겹침이 왜 정적 관찰이 아니라 동적 압력인지 설명할 수 있다.
- NVIDIA 의 kernel 자산이 linear 계보에만 통하고 deep memory 에 port 되지 않는 이유 (non-linear recurrence) 를 말할 수 있다.
- open-source 생태계의 두 공백 (deep-memory fused kernel, RMW-aware serving API) 을 짚을 수 있다.
- 학계 · 스타트업의 진입점이 왜 하드웨어 없이 둘 수 있는 수 (이론 · 측정 · decode 소프트웨어) 에 몰리는지, “자산 비대칭이 진입로의 지도” 라는 명제로 설명할 수 있다.
- 세 player 를 pair thesis 위에 겹쳤을 때 왜 단일 승자가 아니라 분업이 예측되는지, 그리고 kernel 한 칸만은 왜 분업이 아니라 경주 (first-mover-takes-standard) 인지 game 으로 논증할 수 있다.

- TNT-beats-FlashAttention 이 왜 “진입 기회의 창” 을 뜻하는지 진술할 수있다.
- player 지도를 inference 어휘 (누가 kernel 을 쥐는가, 누가 serving 스택을쥐는가) 로 옮길 수 있다.

다음 장으로

이 장은 각 player 의 합리적 다음 수를 자산에서 읽었다. 그 수들 — 특히 Google 의 scale up 과 decode 반쪽의 serving 인프라 — 이 실제로 놓이면무엇이 필요한가. 23 장은 그 요구를 정량으로 내린다: TNT 경제학을 7 – 70B 로외삽하고, 계층적 memory 의 prefill/decode 분할을 그리며, per-session weight state 를 새 cache class 로설계하고 (sizing·checkpoint/restore·multi-tenant batching — claim 3 의 $B_{\max}$ 가 근거), sleep 을 fleet 수준 background job 으로 놓아, 오늘의 KV-cache 서빙과 비용 모델로 대조한다.

ch23. 대규모 학습·서빙 projection

이 장의 목표 — 독자가 이 장을 마치면 (1) [TNT]가 남긴 유일한 wall-clock 증거 (학습 처리율)를 7–70B로 외삽할 때 무엇이 근거이고무엇이 directional한지 구분할 수 있고, (2) 하나의 서빙 요청 안에서 prefill과 decode가 pair thesis의 반대편 두 영역 (compute-bound accelerator / memory-bound memory-centric)을 각각 부른다는것을 설명할 수 있으며, (3) per-session weight state를 sizing·checkpoint/restore·multi-tenant batching·frequency-tier 배치·sleep-as-background-job의 다섯 축을 가진 새로운 **cache class**로 설계하고, 이를 오늘의 KV-cache 서빙과 비용모델로 나란히 놓을 수 있어야 한다. **왜 필요한가** — 18장이 세운 D4 pair thesis는 배포 형태의 명제다. 그 배포를 실제로 돌리면 얼마가드는가는 여섯 편 어디에도 없다 — decode wall-clock 수치는 6편 전체에 부재하고 ([TNT]는 학습 wall-clock 만), serving 경제학은 다섯 공백 중하나로 명시적으로 남았다 (→ 18장 § 18.2). 이 장은 그 빈 자리를 로컬 실행프로그램 (claim 1–6)의 비율·crossover·tier 순서로 채우고, 채울 수 없는 절대치는 사내 A100 runbook으로 이월한다.

23.1 Bridge-in: 완성형을 배포하면 청구서가 두 장 나온다

22장은 이 라인의 자연스러운 저자가 왜 Google인지, 각 player가 다음에무엇을 합리적으로 두는지를 분석했다. 이 장은 그 다음 질문을 받는다 — **그 배포를 실제로 서빙하면 얼마가 드는가**. 18장에서 완성형은 “세션 상태가 per-session/per-tenant mutable weights인 continually-learning LLM에, TNT 경제학으로 학습되고, sleep 서비스가 붙어있는 시스템”으로 정식화됐다 (§ 18.2의 [평가]). 이 형태를 청구서로 바꾸면 두 장이 나온다: **학습·prefill 청구서** (accelerator 영역)와 **decode·serving-state 청구서** (memory-centric 영역). 두 청구서는 같은 알고리즘에서 나오지만 지불 통화가 다르다 — 하나는 FLOP, 하나는 byte.

이 장의 정직성 바닥을 먼저 못 박는다. 이 라인의 실증 상한은 from-scratch 1.3B params / 100B tokens 이고 ([TNT]는 150M), decode wall-clock은 6편 전체에 부재하다 (→ 18장 § 18.2 유보 1, § 6.4 라인 공통 caveat). 따라서 이 장의 **load-bearing 주장은비율·crossover 위치·tier 순서·bound class** 뿐이다. 절대 $\mu\text{s}/\text{token}\cdot\text{mJ}/\text{token}\cdot\text{B}_{\text{max}}$ 절대값은 roofline 하한이거나 upper bound이며, 사내 A100 runbook(runbook/hope-reproduction-a100.md)이실 kernel로 닫을 이월 항목이다. novel scratchpad/PIM twin 의수치는 simulation_ready=False인 directional DSE로만 등장한다. 이 태그를 절마다 그대로 붙인다.

23.2 TNT economics를 7–70B로 외삽 (accelerator 절반)

[TNT]는 라인 전체에서 wall-clock을 보고한 유일한 편이다 (→ 15장). 세수치가 앵커다: 정확한 Titans baseline 대비 **target loss까지 최대 $17.4\times$ 빠르고** [TNT], plain-JAX 구현이 **32K 문맥에서 step 당 FlashAttention을 이기며** [TNT], 550M Titans를 $C=64$ 로 학습하면 ppl 13.78인데 $C=8$ 로 서빙하면 36.45로 무너진다 [TNT Fig. 2]. 앞의 둘은 accelerator 영역이 실재함을 보인다 — nonlinear deep-memory recurrence를 reset으로 끊어 context parallelism을 열고, chunk를 키워 arithmetic intensity를 제조하면 이 부하는 tensor core로 간다.

이 처리율을 7-70B 로 외삽할 때 무엇이 자라는가는 chunk 크기 C 가결정한다. claim7(E2.1, → 9 장 ·15 장) 은 C 가 roofline 의 x 축임을 보인다: $C=1$ (per-token rank-1 RMW = decode 영역) 은 $AI \approx 1$ FLOP/byte 로 memory-bound 평원 (host roofline 의 약 12%) 에있고, C 를 키우면 crossover C^* 를 넘어 compute-bound 로오른다. prefill/training 은 큰 C 에서 돌므로 accelerator 영역에 산다. 다만 C^* 의 절대값은 ridge 의존이다 — host CPU 에서 측정된 $C^* \approx 32$ 는 곡선의 모양만 옮기고, H100 twin ridge(295 FLOP/byte) 에서 닫힌 형식 C^* 는 d 에 따라 306-430 이다 [E3]. 즉 “ C 를 키우면 compute-bound 가된다” 는 load-bearing 이고, “얼마나 커야 하나” 는 하드웨어별로 다시 재야한다. [CPU-SHAPE]

[평가] TNT 의 $17.4\times$ 를 7-70B 로 곧이곧대로 옮겨서는 안 된다. [TNT App. D] 는 momentum·gating·Muon 을 “명료성을 위해” 제거한 단순화 Titans 로 경제학을 검증했다 — 라인전체 (self-modifying Titans + CMS 의 HOPE) 와의 합성은 미검증이다 (→ 15 장의의무 caveat). HOPE 의 실제 학습 비용은 outer gradient 가 inner-loop 의 unrolled per-token/per-chunk update 를 통째로 관통하는 데서나 오며 (runbook § 0), 이 그래프 폭발은 TNT 의 단순화 Titans 에는 없다. 따라서 이 절의 외삽은 **accelerator 영역이 존재하고 chunk 가 그 x 축이라는 구조적 주장**까지만 단정하고, 학습 벽시계의 절대 예측은 A100 runbook 의 carry-forward #1(760M/30B 학습 벽시계는 논문부재로 사전 예측 불가, 실측 대기) 로 넘긴다.

두 단계 학습 비용의 외삽. [TNT] 의 학습 경제학은 two-stage(train-big / serve-small, → 15 장) 로 구조화된다: Stage-1 은 큰 chunk C_g 로 대부분의 token 을 pretrain 하고 (compute-bound, MFU 친화), Stage-2 는 전체 token 의 약 5%만작은 chunk 로 fine-tune 해 chunk-1 decode 를 quality-optimal 지점으로옮긴다 [TNT]. 이 구조를 7-70B 로 옮길 때 load-bearing 한 것은**비용의 형태**뿐이다 — 학습 청구서의 지배항은 Stage-1 의 compute($\approx 6ND$ 계열, accelerator 영역) 이고, Stage-2 는 그 위에엮히는 약 5% 부가세다. 즉 작은-chunk 재현이 강요하는 저-MFU 구간이 전체학습 예산의 소수로 억제된다는 **비율 구조**가 두 단계 recipe 의요점이며, 이 비율은 규모와 무관하게 유지된다 (절대 학습 벽시계는 여전히 carry-forward #1, 150M 밖 예측 불가).

단, 서버 청구서는 학습 속도를 물려받지 않는다. E4(→ 19 장) 가 digitize 한 state-byte 스케일은 서버 상태 크기가 **architecture family multiplier** m 에 지배됨을 보인다: matrix($m \approx 1$) → deep-MLP($m=8$) → +momentum($m=16$) → +poly-feature($m \approx 24$) [E4]. 결정적 정직성 지점은 여기다 — [TNT] 의 $17.4\times$ 는 momentum·gating·Muon 을벗긴 **가장 얇은 state family** (단순화 Titans, m 최소) 위에서측정됐다 [TNT App. D]. 그런데 서버 상태 청구서는 $m d^2 L_{\text{layer}}$ 의 m 이 키우고, 그 m 을 키우는성분 (momentum S_t , poly-feature) 이 바로 TNT 가 “명료성을 위해” 제거한것이다. 따라서 학습 가속의 증거 (accelerator 절반) 와 서버 상태비용 (memory-centric 절반) 은 **같은 family 위에서 합성 검증된 적이없다** (→ 15 장 의의무 caveat). 이 미검증 합성이 pair thesis 의 두 청구서가서로 다른 통화로 지불된다는 § 23.1 명제의 정확한 소재지다 — 가속을 재는 축과 상태 비용을 재는 축이 어긋나 있다.

decode 쪽 학습 후 비용의 스케일링은 claim1(E1.1/E3) 이 준다. 전체 state 크기는 폭에 따라 $m d^2 L_{\text{layer}}$ 로 자라고, 그에 비례해 per-token RMW 트래픽이 커진다:

표 23-1 — 모델 폭에 따른 decode-step RMW 트래픽 스케일링 (anchor $m=16$; 문맥 S 에 무관, KV 와의 결정적 대비). 절대 GB/token 은 canonical transformer shape 가정 위의 roofline 값이다. [REL, M-ASSUMED]

규모	RMW GB/token	S^* read-crossover (token)	B_max @10ms (seq)
Titans-170M	0.453	—	—
Titans-340M	1.611	16,384	20.8
Titans-760M	3.624	36,864	9.24
neural-mem-1.3B	6.442	65,536	5.2
hypo-7B	34.36	262,144	0.97
hypo-70B	343.6	1,048,576	0.1

한 줄로 읽으면: **decode** 트래픽은 폭에 따라 $0.45 \rightarrow 6.44 \rightarrow 343 \text{ GB/token}$ 으로자라고 [E3], 이것이 서버 청구서의 지배항이다. 70B 에서 token 당 343 GB 를 움직인다는 것은 — 이 상태는 단일 80 GB HBM 에 담기지 않아 sharding 이전제되며 — 집계 HBM3 대역폭 (3.35 TB/s 기준) state 이동만으로 token 당 하한이 100 ms 규모라는 뜻이다. 절대치는 A100 실측 대기지만, 성장 구조 (per-layer 는 $\propto d^2$, whole-model 은 $\propto d^2 L_{\text{layer}}$ 이며 표 23-1 의 canonical shape 에서는 parameter count 에 거의 선형) 는 load-bearing 이다.

23.3 Hierarchical memory 의 prefill/decode 분리

오늘의 KV-cache 서버에서 prefill 과 decode 는 **같은연산** (attention) 의 두 arithmetic intensity 점이다 — prefill 은 긴 문맥의 GEMM(compute-bound), decode 는 KV 의 read-many(점점 memory-bound). 이 라인에서는 그 구조가 질적으로 달라진다.

[해설] hierarchical memory($\rightarrow$ 15 장의 global W^g + local $W^{l(i)}$) 를 서버에 얹으면, 하나의 요청 안에서 **prefill** 과 **decode** 가 **pair thesis** 의 반대편 두영역을 각각 부른다. - **prefill** = 큰 chunk 의 병렬 write(compression). 입력 문맥을 큰 C_g 로 쪼개 global memory 에 한 번에 압축한다 — claim7 의 큰- C 영역, compute-bound, tensor core. 이것은 독자의 prefill 감각 (큰 GEMM) 과 정확히 유비된다 ($\rightarrow$ 1 장 Rosetta 의 “prefill = 큰 chunk 의 병렬 write” 행). - **decode** = $C=1$ 의 **per-token online RMW + read**. claim1 의 memory-bound 영역, state 트래픽이 GEMV 연산을 약 $394\times$ 압도한다 [E1.1] — step 비용이 곧 state 트래픽이다.

이 분리가 서버 스케줄러에 주는 함의는 직접적이다. 오늘의 엔진은 prefill 과 decode 를 같은 kernel family 로 배치 (batch) 하고 continuous batching 으로 섞는다. 이 라인에서는 두 **phase** 가 다른 병목 자원을 묻는다 — prefill 은 FLOP, decode 는 HBM 대역폭. 따라서 prefill-decode disaggregation(두 phase 를 다른 하드웨어 풀로 분리) 이 KV-cache 서버에서보다 더 강하게 정당화된다: prefill 풀은 compute-dense accelerator 를, decode 풀은 high-RMW-bandwidth memory 시스템을 원한다. 이것이 이 장이 pair thesis 를 서버 토폴로지로 옮기는 첫 결론이다.

batch $\times$ chunk 2D 병렬성: decode 가 prefill 보다 병렬성이 적은 이유. 위 분리가 왜 병렬성 비대칭을 낳는지는 이 라인의 write 가 두 개의 독립 축으로 병렬화된다는 사실에서 나온다. (i) **batch** 축 — 서로다른 sequence 는 각자의 fast-weight 상태 $W_t^{(b)}$ 를 갖고, update 를 결합하는 항이 없다. inner update 는 request 별 **map** 이지 reduction 이 아니어서 (request 방향으로 평균하면 세션 간 정보가 샌다 — 이 never-average 가 § 23.4 의 $B_{\max}$ 벽에 미치 근처 하나를 보탠다), batch 축은 **완전 병렬**이다. batched write 는 request 별 상태를 batch 차원으로 쌓은 grouped-GEMM/BMM 으로 표현된다: matrix memory 는 $W: [B, d_v, d_k]$, 표준 2-layer MLP memory 는 $W_2: [B, d_h, d_l] \cdot W_1: [B, d, d_h]$ 의 batched small-GEMM 이다 ($\rightarrow$ 24 장 제안 G). (ii) **chunk** 축 — 한 sequence 안에서 상태는 chunk 경계를 따라 $W_0 \rightarrow W_1 \rightarrow \dots$ 의 recurrence 를 진다 ($\rightarrow$ 9 장 (M4) stale-snapshot). chunk 내부의 token 은 chunk 시작 상태 $W_{\xi(t,C)}$ 에서 병렬 평가되므로 그 안에서만 token 병렬성이 열린다.

결정적 경계는 이 chunk-내부 token 병렬성이 **token 이 이미 주어진 prefill/training 전용**이라는 것이다. autoregressive **decode** 는 **미래 token 이 없어** chunk 를 채울 수 없으므로 chunk 축이 붕괴하고 — **batch** 축만 남는다. 이것이 § 23.3 에서 decode 를 memory-bound 절반으로 보낸 것의 병렬성 축 근거다: prefill 은 (batch $\times$ chunk) 의 2D 병렬성으로 큰 GEMM 을 제조하지만, decode 는 1D(batch) 로 좁아지고 그나마도 § 23.4 의 $B_{\max}$ 대역폭 벽에서 조기에달린다. 두 각주가 이 그림의 정밀도를 못 박는다.

- **선형 특수형 $\neq$ 배포형.** matrix memory 의 닫힌 write $W_t = W_{t-1}(\alpha_t I - \eta_t k_t k_t^\top) - \eta_t \hat{v}_t k_t^\top$ (unified Gated DeltaNet 형, $\rightarrow$ § 1.6) 은 **linear-dot-product objective** 의 특수형이다; 배포된 표준 2-layer MLP memory 는 일반 $\nabla_W \ell$ (L2 형이면 residual-gradient $-\eta_t (\mathcal{M}(k_t; W) - \hat{v}_t) k_t^\top$ 항) 로 갱신된다 — 둘을 같은 것으로 읽으면 batched-op 비용을 오산한다. 그리고 transition $A_t = \alpha_t I - \eta_t k_t k_t^\top$ 은 **identity-minus-rank-1** 이라, dense

$[B, d_k, d_k]$ 로 materialize 해 $W \cdot A$ 를 BMM 으로 도는 것은 낭비다 $-\alpha_t W_{t-1} - \eta_t (W_{t-1} k_t) k_t^\top$ 의 matvec+outer-product(rank-1) 로 factored 유지한다.

- **MoE 유비는 dispatch form 뿐.** batched grouped-GEMM 은 MoE dispatch 와 **kernel 형태만** 닮았다 — MoE 는 다수 token 이 소수 shared expert weight 를 재사용해 amortize 하지만, 여기선 request 마다 weight 가 달라 group 당 token ≈ 1 이라 **weight-재사용 amortization 이득이 없다** ($\rightarrow$ § 23.4 $B_{\max}$ 가 그 부재의 정량). 즉 batch 는 grouped-GEMM 으로 여전히 **형성**되지만 (feasibility, 제안 G) arithmetic intensity 가 B 와 무관해 트래픽 하한을 낮추지못한다 (economics) — batchability 는 memory-bound 결론을 훼손하지 않고 그 안에 예약된 자리에 앉는다 ($\rightarrow$ § 23.4 [평가], results.json batching_feasibility_gloss).

23.4 Per-session weight state: 새로운 cache class 의 sizing 과 batching

per-request 로 변하는 weights 는 오늘의 서빙 불변식 — “추론 중 weights 는 shared·불변” — 을 깬다 ($\rightarrow$ 18 장 § 18.1). 그 결과 필요한 것이 **per-session weight state** 라는 새 **cache class** 다. KV cache 와 나란히 놓되 성질이 반대인 이 class 를 다섯 축으로 설계한다. 먼저 sizing 과 batching.

Sizing. state 크기는 문맥 S 에 **무관하게** $m d^2 L_{\text{layer}}$ 로 고정된다 (anchor 에서 layer 당 134 MB, [E1.1/E3]). 이것은 KV cache 와의 근본 대비다: KV 는 문맥에 선형으로 자라 capacity-bound 가 되지만, session-weight 는 고정크기라 **on-chip 상주 여부가 설계 가능한 knob** 이 된다 (claim4, $\rightarrow$ 20 장·24 장). 단 그 knob 에는 상한이 있다 — on-die L2(50 MB) 는 어느 규모에서도 **한 sequence 의 whole-model state 조차 담지못한다** [claim3]. 따라서 상주는 whole-model pin 이 아니라 **per-layer/streamed** 여야 하고, 268 MB scratchpad 에 whole-model 이 들어가는 것은 Titans-170M(226 MB) 뿐이다 [E1.2]. 340M 이상은 전부 spill 한다.

Cache-class sizing(7 – 70B 외삽). 이 cache class 의 용량 축은 KV 와 반대로 문맥이 아니라 **폭과 family** 다. E4 의 state-byte 모델 ($m d^2 L_{\text{layer}}$, bf16) 로 한 session 이 pin 하는 whole-model 상태를 family 별로 외삽하면 표 23-2 다. 한 줄로: 서빙 memory 관리자가 세션마다 확보해야 하는 것은 KV 처럼 문맥에 따라 자라는 페이지가 아니라 **입장 시점에 이미 크기가 확정된 고정 블록**이다 — 그래서 admission control 이 KV 의 “이 세션이 앞으로 얼마나 길어질까” 가 아니라 “이 family·이 폭의 고정 블록이 지금 들어갈 자리가 있는가” 로 단순해진다. 대신 그 블록이 크다: momentum family 1.3B 는 세션당 3.22 GB, 7B 는 약 17 GB, 70B 는 약 172 GB 로, **단일세션 상태가 whole-model weights 와 같은 자릿수에 이른다** [E4, directional]. fp8 로 상태를 재양자화하면 이 footprint 와 아래 § 23.4 Batching 의 $B_{\max}$ 대역폭 벽이 함께 2 배로 완화된 다 (fp8 KV 가 S^* 를 두 배로 미는 것과 같은 지렛대, 표 23-1 의 S^* 행과대칭).

표 23-2 — 한 session 이 pin 하는 whole-model session-weight footprint 의 family·폭별 외삽 (bf16, E4 state-byte 모델). load-bearing 은 **family multiplier** 의 순서와 폭에 대한 d^2 성장이고, 절대 GB 는 canonical Llama shape 가정 위의 directional 값이다 (340M·1.3B 는 E4 실산, 7B·70B 는 그 위의 외삽). [REL, M-ASSUMED, directional]

family (m)	340M	1.3B	7B	70B
matrix / GDN ($m \approx 1$)	0.05 GB	0.20 GB	~1.1 GB	~11 GB
deep-MLP ($m=8$)	0.40 GB	1.61 GB	~8.6 GB	~86 GB
+momentum ($m=16$)	0.81 GB	3.22 GB	~17 GB	~172 GB
+poly-feature ($m \approx 24$)	1.21 GB	4.83 GB	~26 GB	~258 GB

읽는 법: family 를 한 칸 올릴 때마다 (matrix $\rightarrow$ deep-MLP $\rightarrow$ +momentum $\rightarrow$ +poly) footprint 가 정수배로 뛰고, 폭을 키우면 d^2 로 자란다. 이 두 축이 § 23.2 의 정직성 지점을 sizing 표로 재확인한다 — TNT 가 가속을 증명한 얇은 family (맨 윗줄) 와 완성형이 요구하는 두꺼운 family (아랫줄) 는 서빙 footprint 에서 한 자릿수 이상 벌어진다.

Batching. 여기서 KV 와 가장 극적으로 갈라진다. 집계 state RMW 는 **HBM 용량보다 대역폭이 먼저 막힌다** — claim3(E1.3/E3) 의 핵심이다. 10 ms/token 목표에서 대역폭 상한 배치 $B_{\max}$ 는 340M/1.3B/7B 에서 **20.8 / 5.2 / 0.97 sequence**(표 23-1), 50 ms 목표에서 104 / 26 / 4.87 이다 [E3]. KV cache 가 capacity-bound 인 것과 정반대로 **bandwidth wall** 이다.

[평가] 이 $B_{\max}$ 절대값은 두 겹의 유보를 진다. 첫째, roofline 하한 latency 에서 나온 값이라 절대치는 A100 실측대기다 [NO-WALLCLOCK]. 둘째, E3 의 $B_{\max}$ 는 weights+activations+KV 가 co-resident 인 full serving stack 을 빼고 계산한 **upper bound** 다 (carry-forward #6). 그러므로 load-bearing 은 “7B 에서 10 ms 목표 시 배치가 한 자리수, 사실상 batch-1 로 몰린다” 는 **class 와 방향**이지 “0.97” 이라는 숫자가 아니다. 함의는 무겁다 — per-request unshared weights 는 shared-weight batching 을 깨고, decode 는 grouped-GEMM 으로 가야 하며 ($\rightarrow$ 24 장), 대역폭 벽이 배치를조기에 닫으므로 memory-centric 논증의 decode 절반이 바로 여기서 실측 근거를 얻는다.

Multi-tenant batching 이 깨지는 **정확한 지점과 완화책**. shared-weight 서빙에서 batch 를 키우는 것은 공짜에 가깝다 — 같은 weights 를 여러 요청이 재사용하므로 GEMM 이 커질 뿐이다. session-weight 에서는 요청마다 상태가 unshared 라, batch B 의 decode step 이 움직이는 상태 트래픽이 B 에 **선형**으로 자란다. 따라서대역폭 예산 $BW \cdot t_{\text{target}}$ 을 상태 트래픽 $2 B m d^2 L_{\text{layer}}$ 가 채우는 지점이 정확한 파탄점이고, 그것이 $B_{\max}$ 다 (표 23-1). 파탄은 급격하다 — 7B/10 ms 에서 $B_{\max} < 1$ 이라는 것은 **단일 세션조차 목표 latency 안에 상태를 한 번 통과시키지 못한다**는 뜻이고, batching 으로 amortise 할상수 오버헤드가 없다 (상태 이동이 곧 step 비용, $394\times$, $\rightarrow$ § 23.3). 완화책은네 갈래이며, 어느 것도 절대치를 단정하지 않고 방향만 load-bearing 이다:

- **상태 제약자화** — fp8/int8 상태는 트래픽을절반/1/4 로 줄여 $B_{\max}$ 를 그만큼민다 (fp8 KV 의 S^* 2 배와 같은 지렛대). numerics-drift 가 optimizer-trajectory 상태 S_t 에 누적되는지가 A100 검증항목이다 (carry-forward #3).
- **per-layer streaming + double-buffering** — 상태가 whole-model 로 on-chip 에 안 들어가므로 (위 Sizing, claim3), layer 별로 HBM 에서 흘리며다음 layer 상태 prefetch 를 GEMV compute 와 overlap 한다. roofline 하한을 achieved 로 좁히는 자리 (carry-forward #2).
- **prefill/decode disaggregation**($\rightarrow$ § 23.3) — decode 풀을 대역폭 기준으로 sizing 해 $B_{\max}$ 벽을 풀 수준에서 관리하고, compute-bound prefill 을 분리한다.
- **hybrid** — 짧은-문맥 성분을 sliding-window attention(shared, batchable) 으로, 긴-문맥 성분만 TTT 상태로 두면 ($\rightarrow$ 18 장 유보 2) batch 가 attention 절반에서 회복된다. crossover S^* 아래 트래픽은 KV 가싸다는 § 23.8 과 정확히 같은 논리다.

Checkpoint/restore. session-weight 는 KV cache 가지지 않는 두 종류의 상태 관리 부채를 진다. (1) **snapshot/rollback** — speculative decoding 의 rollback 은 KV 에서는 pointer 되감기지만, 여기서는 fast weights W_t 뿐아니라 **optimizer-trajectory 상태 (momentum S_t)** 까지 snapshot 해야 한다 ($\rightarrow$ 18 장의 systems 공백; runbook X-계열). (2) **dirty eviction 의 writeback** — session-weight 는 항상 dirty(매 token 갱신) 라 evict 할 때 recompute 가 아니라 **full writeback** 을진다. 이 부채가 오늘의 KV manager 에서 0 원으로 잘못 계상되는 것이 다음절이다.

checkpoint format 과 restore path. 세션이 요청 사이에 suspend/resume 되거나 노드 간 migrate 될 때, 무엇을직렬화하는가가 이 cache class 의 checkpoint format 이다. KV 의 checkpoint 는 append-once 페이지들이라 마지막 상태만 저장하면 되지만, session-weight 의 완전한 checkpoint 는 **layer 별 fast weights W_t + momentum S_t + gate/RNG 상태 + W_{init}**

참조 + **version/provenance** 태그의 튜플이다. 크기는 표 23-2 의 whole-model footprint 그 대로이므로 (문맥 무관, 고정), 세션 하나를 host/CXL 로 suspend 하는 것은 표 23-2 크기의 write 한 번이고 resume 은 그만큼의 read 다 — KV 처럼 “필요한 페이지만” 이 아니라 항상전량이 다. **delta-encoding** 으로 줄일 수 없다는 점이 KV 와 같린다: 상태가 100% dirty(→ § 23.8 write/read 대칭) 라 직전 checkpoint 대비 delta 가 전체와 같은 자릿수다. 그래서 checkpoint 주기는저장 대역폭과 rollback 손실 사이의 tradeoff 로 설계되고 (자주 = 대역폭부담, 드물게 = recompute 창 확대), 그 절대 주기는runbook 이월이다. version 태그는 § 23.7 의 sleep 이 매번 새 weight 버전을 만드는 것과 이어져 per-tenant provenance 의 1 급 필드가 된다. 마지막으로 **multi-tenant isolation**: 상태가 그 세션의 문맥을 weights 로 흡수했으므로 (weight-absorbed context), checkpoint 는 사실상 사용자데이터이고 — 공유 페이지가 없는 만큼 cross-tenant 유출 표면은 KV 보다작지만, 그 전량이 per-tenant 격리 · 암호화 대상이 된다 (→ 18 장 systems 공백).

snapshot/rollback 의 정량 (speculative decoding). speculative decoding 은 draft 가 k token 을 제안하고 verify 가 거부 위치 j 에서 상태를 W_{t+j} 로 되감는다. KV 에서 이 되감기는 block-table pointer 절삭 (공짜) 이다. session-weight 에서는 제안된 k token 각각이이미 full RMW 로 상태를 갱신했으므로, rollback 은 (a) k 개 상태 snapshot 을 들고 있거나 ($k \times$ 상태 메모리 — 이미 $B_{\max}$ 가 한자리인데 k 배는 치명적), (b) 마지막 committed checkpoint 에서 recompute 해야 한다. 게다가 snapshot 대상은 fast weights W_t 만이아니라 **momentum** S_t 까지의 optimizer trajectory 전체다 (→ 18 장 systems 공백; runbook X-계열). 즉 speculative decoding 의메모리 · 정합성 부채가 KV 에서는 0 인데 여기서는 상태 크기 $\times$ speculation 깊이로 곱해진다 — 이 라인에서 speculative decoding 이 KV 만큼값싸지 않다는 것이 load-bearing 이고, $k \cdot$ checkpoint 주기의 절대 tradeoff 는 runbook 이월이다.

Eviction = retention-gate 협력. cache class 의 상태 관리 점점 중하나이자 Rosetta 의 “cache eviction policy = 학습된 eviction” 행 (→ 1 장) 의서빙 구체화다. 두 층위의 eviction 이 협력한다. (i) **상태 내부의학습된 eviction** — retention gate $\alpha_t \in [0, 1]$ (→ 13 장) 는 매 token 상태를 $\alpha_t W_{t-1}$ 로 감쇠시키는 학습된 forget 이다. α_t 가 지속적으로 1 보다 작게 걸린 세션은 상태가 W_{init} 쪽으로 수축해 **정보 보유량이 낮다**. (ii) **서빙 층위의 coarse eviction** — 관리자가 한 세션의 whole-state 를 CXL/host 로 내리거나 drop 하는 결정. 협력 규칙은 직접적이다: retention gate 가 이미 상태를 W_{init} 근처로 몰아넣은세션은 **evict 후 W_{init} 에서 저비용으로 재구성가능한 “cold” 후보이고**, $\alpha_t \rightarrow 1$ 로 정보를 짝 켜 세션은 dirty writeback(→ § 23.5 G3) 을 물어야 하는 “hot” 상태다. 즉 학습된 α_t 가 서빙 eviction 의 **비용 신호**를 공짜로 준다 — KV 의 LRU/recency 가 하던 역할을 학습된 retention 이 대신한다. 이협력은 § 23.6 의 frequency-tier 배치와 결합한다: cold·저-cadence 세션은 CXL tier 로, hot 세션은 HBM 에 pin. (retention gate 로 cold 를 판정하는 규칙 자체는 방향 주장이고, 그임계값 · 정확도는 A100 실측 대기다.)

23.5 KV-manager 의 범주 오류: 왜 새 매니저가필요한가

pair thesis 의 소프트웨어 절반이다. `hat-kv-manager`(오늘의 KV-cache 매니저 축소판) 에 session-weight 를 엮어 보면 세 지점이**범주 오류**로 드러난다 (claim6, E1.5).

- **G1 — content-addressed reuse = 0**. content-addressed(block-hash) prefix 재사용 은 KV 매니저의 1 번가치 (anchor 에서 hit-rate 0.5) 지만, RMW state 에는 그 형태로는 0 이다 — 내용이 매 token 바뀌어 재현되는 block hash 가 없다. 단이 0 은 block-hash 재 사용에 한정된다: 같은 prefix 를 결정론적으로 처리한서로 다른 요청은 분기 전까지 동일한 state snapshot 에 도달하므로 prefix-snapshot 공유 + copy-on-write 는 원리상 가능하며, append-only KV 매니저가 그 mutable-state 재사용을 표현할 API 를 못 가진 것이 바로 새매니저가 필요한 이유다 (→ § 24.5 제안 I).
- **G2 — stale accretion $256 \times$** . in-place update 가 없는 append-only `place()`(no-eviction worst case) 는 256 token 에 걸쳐 $T=256$ 개의 stale 버전을 쌓아 참 1-state working set 의 $256 \times$ 로 부풀린다.
- **G3 — 값 매겨지지 않은 dirty writeback**. `_evict_from()` 은 dirty state 를공짜로 del 한다 (clean/recomputable 가정). 실제 TTT eviction 은 writeback(또는 결정론적 prefix-

replay 재계산) 을 지는데매니저는 **0** 으로 계상한다 — E1.5 의 synthetic eviction set(416 MiB) 에서 그 writeback 은 약 **3.05 mJ / 130 μ s** 이고, whole-session 상태 (anchor 3.22 GB) 전량을내리면 약 22.5 mJ / 962 μ s 로 footprint 에 비례해커진다 [NO-WALLCLOCK for 절대치; 범주 오류 자체는 load-bearing].

결론: session-weight 매니저는 KVCacheManager 에 없는 event type —update_in_place, mark_dirty/writeback, checkpoint/rollback, bind_to_sequence, free_on_update — 을 1 급으로 가져야한다. KV 매니저를 확장하는 것이 아니라 **다른 class 의 매니저**다. 이것이 § 23.4 의 checkpoint/restore 부채를 소프트웨어로 갚는자리이며, hat-kv-manager 의 로드맵 메모로 이월 된다.

Paged Fast-Weight State Manager — 이름과 그 이름의 함정. 이다른-class 매니저를 한 이름으로 부르면 **Paged Fast-Weight State Manager** 다 — § 24.5 제안 I(per-session-weights serving stack) 가 설계표면으로 삼는, 위 다섯 event type 을 1 급으로 갖는 그 매니저다. 단 이름의 “Paged” 는 오해를 부른다. PagedAttention 의 페이지징은 문맥 길이에 따라**가변 길이로** 자라는 KV 를 비연속 block 으로 담아 fragmentation 을 없애는 기제인데 (→ § 23.8), session-weight 상태는 문맥과 무관하게**폭에 의해 크기가 고정** ($m d^2 L_{\text{layer}}$, → § 23.4 Sizing) 이라 페이지징할 가변 길이가 없다. 따라서 이 매니저의 할당기는 PagedAttention 식 page table 이 아니라 **입장 시점에 크기가 확정되는 fixed-size slab** 이고, 그 위에 § 23.6 의 frequency-tier 배치가 없힌다 —관리 측은 “이 세션이 얼마나 길어질까” 가 아니라 “이 고정 블록을 어느 tier 에 pin 할까” 다. 관리 대상도 NL 이 열거한 self-modifying memory $\mathcal{M}_{\square}$ 에 그치지 않는다: 완전한 Hope 는 CMS 의다 **중-frequency mutable block**(level 별 $\theta^{(\ell)}$, → 16 장 (M5)) 과 **optimizer/momentum 상태** S_t (→ § 23.4 snapshot/rollback) 까지 request 별로 관리해야 하므로하한은 오히려 더 크다. 반대로 query 생성기는 배포된 Hope 에서 **static**($q_t = W_Q x_t$ 로 갱신 없음, $\square \in \{k, v, \eta, \alpha, \text{mem}\}$ 만 mutable — → 16 장) 이라이 mutable-state 목록에서 빠진다. 즉 상태 열거는 동시에과소 (CMS·momentum 누락) 이자 과다 (static query 포함) 이며, 매니저 설계는 이 둘을 함께 보정해야 한다.

23.6 Frequency-tiered placement: 서빙 메모리 계층 그자체

새 cache class 의 네 번째 축이자 이 장에서 가장 신선한 memory-architecture 주장이다. [NL]/[Sleep] 이 각 memory level 에 부여한 **update cadence**(→ 16 장·17 장) 는 그대로 **memory 계층의 tier 배치 명세**로번역된다 (claim5, E1.4). pair thesis 가 memory-centric 논증을 펴는 두정당한 지점 중 두 번째다 (첫째는 § 23.4 의 decode RMW 트래픽).

그림 23-1 — 각 memory level 의 update cadence 가 admissible 한 memory tier 를 정한다. 상주 사본은 read/write 중 **빠른 쪽** cadence 에 의해 pin 되며 (gating rule), sub-per-token access 는 트래픽을임계 경로 아래로 amortise 해 write-heavy RMW state 를 legally CXL 로내려보낸다. anchor(neural-mem-1.3B, per-token 임계 경로 예산 $t_{\text{tok}} \approx 962 \mu\text{s}$) 에서의 tier 배정은 표 23-3 과 같다 (실험 E1.4 재구성). [REL, NOVEL-SIM-FALSE for scratchpad]

gating rule: 상주 사본은 read cadence 와 write cadence 중**빠른 쪽**으로 tier 가 정해진다. 이 한 규칙이 latency tolerance 를 update period 에 선형 ($P_{\text{read}} \cdot t_{\text{tok}}$) 으로넓혀 준다 — 자주 접근하는 것은 뜨거운 tier 에 pin 되고, 드물게 접근하는것은 느린 tier 가 허용된다.

표 23-3 — update cadence → memory-tier 배정 (E1.4, anchor). “read cadence 가 pin 한다” 가 핵심 — chunk 마다 쓰더라도 매 token 읽으면 HBM 에남는다. [REL, NOVEL-SIM-FALSE, M-ASSUMED]

memory level	cadence (read / write)	tier	근거
fast-weights	token / token	HBM3	read-gated, 가장뜨거운 admissible tier

memory level	cadence (read / write)	tier	근거
CMS-chunk	token / 64 tokens	HBM3	read cadence 가 pin(chunked write 에도 불구하고)
CMS-mid	4096 / 4096 tokens	CXL	down-sampled cadence → 여전히 admissible 한 최저 tier
sleep-experts	262144 / 262144 tokens	CXL	offline/idle-window cadence
frozen-weights	token / never	HBM3	read-gated, read-heavy(accelerator 친화) 쪽

[해설] 이 표는 오늘의 서버 엔지니어에게 익숙한 것의 재서술이다 — paged KV cache 가 hot page 를 HBM 에, cold page 를 host/CXL 로 내리는 것과 같은 문법이다. 다른 점은 무엇이 **hot** 인지를 문맥 길이가 아니라 **update frequency** 가 정한다는 것이다. per-token fast-weight 는 HBM 에 pin 되고, 진짜로 down-sample 된 CMS-mid level(4096 token 마다 read+write) 은 CXL-class latency 를 감내하며, sleep 으로 consolidate 된 expert 는 offline cadence 라 CXL 로 내려간다. 이매핑은 거의 문자 그대로이고 선행 연구가 진술한 바 없다 (→ 18 장 § 18.3 의 두정당한 memory-centric 지점 중 둘째). 단 이 tier 강등은 **gating rule** 의 “빠른 쪽” 이 실제로 **down-sample** 될 때에만 성립한다는 조건을 진다. NL/Sleep 의 일반 forward 는 모든 CMS block 을 매 token 호출하고 update 만 주기적이므로 ([NL] Eq. 70/[Sleep] Eq. 1), CMS-mid 의 **read** 가 4096 주기로 내려가는 것은 그 block 이 sparse routing·cached activation 으로 forward 에서 genuinely 건너뛰어질 때뿐이다. block 이 매 token forward 체인에 남으면 read cadence 가 per-token 이라 gating rule 은 그것을 HBM 에 pin 한다. E1.4 의 read cadence(표 23-3) 는 공개 decode trace 가 아니라 그 down-sampled-read 가정위의 값이므로, CXL 배치는 그 가정이 성립하는 조건부 배치다 (fabric latency·transfer size 도 함께 들어간다 — → § 24.3.2).

per-layer 로 쪼갠 fast-weight block(67 MB) 은 268 MB scratchpad 에 fit 해 HBM3 대비 **7.4× 에너지** 이득을 낸다 [E1.4] — 단 all-layer 는 fit 하지 않는다. 이 **scratchpad** 수치는 **novel twin** 이라 **directional DSE** 로만 인용한다 [NOVEL-SIM-FALSE]: 배치의 방향 (per-layer streamed 면 on-chip 이득이 실재) 만 load-bearing 이고, 7.4× 는 silicon 전까지 device 주장이 아니다 (→ 20 장·24 장의 상세 DSE, 그림 (b)/(e) 는 → 20 장).

23.7 Sleep 을 fleet-level background job 으로

새 cache class 의 다섯 번째 축이자 서버의 세 번째 regime 이다. Sleep 의 wake/sleep lifecycle 을 배포에 얹으면, decode 요청 (wake) 밖에 주기적 **offline job**(sleep) 이 fleet 에 붙는다.

[해설] sleep phase = **weights** 의 **background compaction job** 이다 (→ 1 장 Rosetta 의 “서빙 fleet 의 background job” 행). CMS 의 update 경계에서 발화해 ([Sleep] 은 sleep 을 chunk 경계에 hard-wire), (i) 빠른 block 의 지식을 느린 block 으로 상향 distill(Knowledge Seeding) 하고, (ii) self-generated data 로 Dreaming 을 돌린다. 서버 관점에서 이것은 GPU 를 점유하는, 중단 가능한, 주기적 fine-tuning job 이다 — 배포가 “훈련이 끝난” 상태로 존재하지 않는다는 § 18.2 완성형의 직접 귀결이다.

(d) Update cadence → memory-tier assignment (★ = recommended)
amortised write energy per token; gated by the FASTER of read/write cadence

fast-weights R+W / token	no cap	★ 11354.82 μJ/tok	19407.88 μJ/tok	24239.72 μJ/tok
CMS-chunk R/tok, W/64	no cap	★ 177.42 μJ/tok	303.25 μJ/tok	378.75 μJ/tok
CMS-mid R+W / 4096	no cap	2.77 μJ/tok	4.74 μJ/tok	★ 5.92 μJ/tok
sleep-experts R+W / 262k	no cap	0.13 μJ/tok	0.22 μJ/tok	★ 0.28 μJ/tok
frozen-weights R/tok, W never	no cap	★ 0.000 μJ/tok	0.000 μJ/tok	0.000 μJ/tok
	SRAM* 60TB/s	HBM3 3.35TB/s	DDR5 0.4TB/s	CXL 0.5TB/s

■ recommended tier
 ■ admissible (not cheapest gated)
 ■ capacity overflow

Source: EL4 (hatir). *SRAM tier = novel scratchpad twin (simulation_ready=False, directional). Sub-per-token cadence lets a write-heavy RMW state legally drop to DDR/CXL without lengthening the critical path.

그림 23-1 — update cadence 에서 memory-tier admissibility 로의 번역: 상주 사본은 read/write 중 빠른 쪽 cadence 로 tier 가정해진다

fleet 관점의 비용은 세 가지다. (1) **cadence → tier**: sleep-consolidated expert 는 read+write 가 262144 token 마다 (offline) 라 표 23-3 대로 CXL 로 내려가고, 임계 경로 밖 데이터 migration 으로 스케줄된다. (2) **versioning**: 매 sleep 이 새 weight 버전을 만든다 — per-tenant divergence·rollback·provenance 가 서빙 스택의 1 급 관심사가 된다 (→ 18 장 systems 공백). (3) **dream fan-out**: [Sleep] 의 Dreaming 은 격리된 instance 에서 randomized routing 으로 m 개 롤아웃을 낸다 (일반 알고리즘 $m \geq 1$; CPT 실험 설정에서 $m=5$) — fleet 용량 계획에 background 훈련 fan-out 을 넣어야 한다.

duty-cycle 경제학. sleep 을 fleet 비용으로 환산하는 축은 **duty cycle** 이다 — sleep job 이 얼마나 자주 발화하고 얼마나 오래 GPU 를 무는가. [Sleep] 은 sleep 을 CMS update 경계에 hardware 하므로 발화 주기는 그 level 의 cadence (표 23-3 의 sleep-experts 는 262144 token 마다) 로 고정된다. 한 세션의 sleep job 은 GKD upward distill + Dreaming 5-롤아웃 fan-out 으로, 한 번의 소규모 fine-tuning 부하다. fleet 관점에서 이것은 **preemptible·저우선순위 background 부하**로 스케줄된다: foreground decode 가 임계 경로를 쥐고, sleep 은 idle-window 나 거부하 시간대로 미뤄진다 (표 23-3 이 sleep-experts 를 CXL·임계 경로 밖 migration 으로 내려보내는 것과 같은 논리). 그러나 숨은 경합이 하나 있다 — sleep 의 distill/dream 도 결국 상태를 읽고 쓰는 memory-bound 부하라, decode 와 같은 **HBM 대역폭자원을 두고 경쟁한다** (→ § 23.3 의 병목). 따라서 sleep 을 “남는 compute” 로 공짜라 계상하면 § 23.5 KV-manager 와 같은 범주 오류를 반복한다: sleep 의 진짜 비용은 FLOP 이 아니라 그것이 잠식하는 decode 대역폭이다. background sleep 이 foreground $B_{\max}$ 를 얼마나 깎는지의 절대 duty-cycle 은 wake-sleep cycle 을 얻은 뒤 측정할 항목이다 (runbook C6/C7).

[평가] 이 절의 모든 서술은 [Sleep] 의 기제가 pre-trained Llama/Qwen 위의 graft 라는 의무 caveat 위에 선다 (→ 17 장) — 라인 최초로 end-to-end meta-learn 되지 않은 편이다. 따라서 sleep-as-fleet-job 은 **배포 아키텍처의 설계 주장**이지 성능 약속이 아니다. 공개 구현도 0 이라 (runbook § 6.5), 이 job 의 실 비용은 HOPE 재현이 성공한 뒤에야 wake-sleep cycle 을 얻어 측정할 수 있다 (runbook C6/C7, 범위 밖으로 이월).

23.8 오늘의 KV-cache 서빙 대비 비용모델

다섯 축을 한 표로 접어 오늘의 KV-cache 서빙과 나란히 놓는다. 이 표가 이장의 결산이다.

표 23-4 — KV-cache 서빙 vs per-session-weight 서빙의 비용 모델 대비. load-bearing 열은 bound class·shareable·문맥의존성·crossover 이고, 절대치는 § 23.9 로 이월. [REL]

측	KV-cache 서빙 (오늘)	per-session-weight 서빙 (이 라인)
상태 성격	append-once / read-many	per-token RMW(read-modify-write)
write / read 비 공유 가능?	0.003%[E1.3] yes(prefix 재사용, hit 0.5)	100%(대칭)[E1.3] block-hash 재사용 0(단결정론적 prefix-snapshot COW는 가능)
문맥 S 의존 배치 병목	read $\propto S$ 로 증가 capacity-bound	일정 ($m d^2 L_{\text{layer}}$ 로 고정) bandwidth-bound (B_{max} 백)[claim3]
prefill	attention GEMM(compute)	큰- C chunk compression(compute)
decode	KV read-many(점점 mem-bound)	whole-state RMW(항상 mem-bound, $394\times$)[E1.1]
eviction 비용	공짜 drop(recompute)	writeback/replay 부채 (E1.5 416 MiB set ≈ 3.05 mJ/130 μ s; whole-session 은 footprint 비례)[E1.5]
tier 배치	문맥 길이 (hot/cold page)	update frequency (cadence $\rightarrow$ tier)[E1.4]

비용 모델의 한 문장 요약은 **crossover 문맥 길이** S^* 다 (claim2, $\rightarrow$ 18 장 그림 18-1). KV read 는 문맥에 비례해 자라고 TTT RMW 는 일정하므로, S^* 아래에서는 KV 가 싼 메모리 시스템이고 위에서는 TTT 가 싸다. anchor 에서 read-crossover $S^* \approx 65\text{k token}$, 전체 RMW-crossover $\approx 131\text{k token}$ 이며 [E1.3/E3], S^* 는 폭에 따라 $m d^2$ 로 **16k $\rightarrow$ 1.05M token** 을 이동한다 (표 23-1). fp8 KV 는 S^* 를 두 배로 밀고, MHA(GQA 대신) 는 선형으로 collapse 시킨다 [E3].

[해설] 이 crossover 가 서빙 배치 결정 규칙을 준다. 짧은-문맥·다중-테넌트(챗봇류, 문맥 $\ll 65\text{k}$)에서는 오늘의 KV cache 가 여전히 싼 메모리 시스템이다. crossover 의 근거는 **트래픽**이다 — KV read 트래픽 vs TTT RMW 트래픽 (claim2). per-token latency 도 문맥에 따라 오르는데 (dense-attention full-serve sanity 로 2k 에서 0.785, 32k 에서 2.581 ms[E1.3]), **단 이 0.785/2.581 은 attention 연산까지포함한 full-serve 값이라 state-only RMW 하한 (1.923 ms) 과 직접 비교하는 척도가 아니다** — 같은 결과에서 순수 KV-read kernel 은 16k 에 0.24 ms·65k 에 0.96 ms 이고, crossover 자체는 이 pure-traffic 측에서 성립한다. 긴-문맥·소수-세션(문서·에이전트류, 문맥 $\gg 131\text{k}$)에서는 TTT state 가 싸다 — RMW latency 가 문맥에 **무관하게** 일정하기 때문이다. 완성형이 hybrid 일 가능성 ($\rightarrow$ 18 장 유보 2) 이 여기서경제적으로도 지지된다: 두 메모리 시스템은 문맥 축에서 서로의 영역을 나눠가진다.

오늘의 서빙 스택 기제와의 정면 대조. 표 23-4 가 비용 모델을 접었다면, 오늘 KV 서빙 스택의 세 핵심 기제 각각이 session-weight 에서 어떻게 뒤집히는지 배포 이식의 실무다. (1) **PagedAttention** — KV 를 비연속 block 으로 페이지징해 fragmentation 을 없애는 기제는, 상태가 위치별로 sparse-growing 이 아니라 **폭에 의해 크기가 고정된 dense** 블록인 session-weight($\rightarrow$ § 23.4 Sizing) 에는 이득이 거의 없다: 페이지징할가면 길이가 없다. 대신 필요한 것은 fixed-size slab allocator 와 § 23.6 의 tier 배치다. (2) **prefix cache / radix-tree 공유** — 공통 prefix 의 KV 를 여러 요청이 재사용하는 기제는 reuse=0($\rightarrow$ § 23.5 G1) 에서 **구조적으로 죽는다**; content-addressed block hash 가 매 token 바뀌어 재현되지 않는다. (3) **continuous batching** — 서

로 다른길이의 요청을 iteration 단위로 섞어 넣는 기제는 shared-weight 를전제하는데, session-weight 는 unshared 라 batch 가 $B_{\max}$ 대역폭 벽 ($\rightarrow$ § 23.4) 에서 조기에 닫힌다; in-flight batching 은 grouped-GEMM decode 로 대체돼야 한다 ($\rightarrow$ 24 장). 세 기제 모두 KV 의 append-once·shareable·position-indexed 성질에 의존하고, session-weight 는 그 셋을 정확히 반대로 뒤집는다 — 이것이 § 23.5 가 “KV 매니저 확장이 아니라 다른 class 의 매니저” 라고 말한 것의 기제수준 근거다.

23.9 정직성 결산과 A100 runbook 이월

이 장이 단정한 것과 유보한 것을 명시적으로 가른다. **단정 (load-bearing)**: (a) 하나의 요청 안에서 prefill=compute-bound / decode=memory-bound 로 갈라진다 (§ 23.3), (b) 배치는 capacity 가 아니라 대역폭이 먼저 막힌다 (class·방향, § 23.4), (c) KV-manager 는 RMW state 에 범주 오류 다 (§ 23.5), (d) update cadence 가 memory-tier 를 정한다는 gating rule 과 tier 순서 (§ 23.6), (e) crossover S^* 가 KV/TTT 영역을 문맥 축에서 나눈다 (§ 23.8). 이 중 세 방법 (E3 닫힌형식·hatir E1.x·plan hand-calc) 이 anchor 에서 1% 이내로 합치한 것은 검증된 anchor scalar 들 — RMW 트래픽/시간, S_{read}^* , state 크기, $B_{\max}$, C^* 의 정성적 shape — 이다 (results.json cross_validation). (c) KV-manager semantics, (d) frequency-tier 순서, § 23.7 의 sleep placement, (a) 의 prefill full-phase bound 는 각각 단일 분석 또는 directional 설계 주장이지 3-way 합치가 아니며 (예: scratchpad 이득은 E1.2 의 $6.8\times$ 와 E1.4 의 $7.4\times$ 로 서로다른 축), 그 지위로만 읽는다.

유보 (사내 A100 runbook 으로 이월): 아래 여섯 항목은 experiments/results.json 의 carry_forward_to_A100_runbook 이며, 이 장의 절대치는 전부 여기에 속한다.

1. 절대 per-token decode wall-clock(ms/token, tok/s) — 논문·로컬 모두 하한만. anchor 의 1.92 ms/token·46.5 mJ/token 은 HBM3 하한이며 A100 실측이 대체한다.
2. achieved-vs-roofline 효율 (kernel-launch·scheduling·decode tail) — E1.1 은 per-kernel only, per-token latency 는 하한.
3. 실 energy/token(r/w-asymmetric DRAM) — twin 은 symmetric 7 pJ/B 가정.
4. S^* 검증 — 실 KV read kernel vs 실 TTT-state kernel head-to-head(로컬은 roofline half). 앵커 $S_{\text{read}}^* \approx 65\text{k}$, $S_{\text{rmw}}^* \approx 131\text{k}$ 는 directional.
5. novel-twin(scratchpad/PIM) — § 23.6 의 $7.4\times$, § 23.4 의 residency 이득은 silicon 전까지 directional DSE. A100 실측은 HBM 실수치만 닫는다.
6. $B_{\max}$ under full serving stack(weights+activations+KV co-resident) — § 23.4 의 $B_{\max}$ 는 upper bound.

[평가] 이 장은 pair thesis 의 양쪽 청구서를 서빙토폴로지로 옮겼다. decode-serving-state 절반은 새 cache class(sizing·checkpoint·batching·frequency-tier·sleep-job) 로 구체화되어 memory-centric 논증의 실측 근거를 얻었고 (claim 1 – 6), training-prefill 절반은 chunk 가 x 축인 accelerator 영역으로 남아 fused kernel·grouped-GEMM 의 본진이 됐다 ($\rightarrow$ 24 장). 어느한쪽만 미는 것은 부하의 절반을 무시하는 것이다 ($\rightarrow$ 18 장 § 18.3). 이 장이 낸절대치는 하나도 device 주장이 아니다 — 전부 A100 runbook 이 실 kernel 로달을 하한·upper bound 이며, 그 이월이 정직성의 조건이다.

다음 장으로. 이 장은 배포를 두 청구서로 갈랐다. 24 장은 그 두 청구서에 각각 제안을 붙인다 — 알고리즘 수준 (§ 5 의 소규모 실험) 과 하드웨어수준 (frequency-tiered placement·RMW-bandwidth decode device 의 memory-centric 반쪽 + fused chunk kernel·grouped-GEMM decode engine 의 accelerator 반쪽). pair thesis 의 흥미로운 하드웨어 제안은 둘 중 하나가 아니라 쌍이라는 § 18.3 의 명제를, 24 장이 구체적 device 제안으로 닫는다.

ch24. 제안: 알고리즘 레벨과 하드웨어 레벨

이 장의 목표 — 독자가 이 장을 마치면 (1) 이 라인을 검증하고 밀어붙이기 위한 **알고리즘 레벨 제안** — 이미 돌린 소규모 실험이 무엇을 확정했고, 그것이 어느 falsifiable 대 형 실험 (train/serve chunk-consistency, state-capacity scaling) 과 HOPE/Sleep 재현 runbook 으로 이어지는지 — 을 진술할 수 있고, (2) **하드웨어 레벨 제안**을 D4 pair thesis 의 두 절반으로 나눠, memory-centric 쪽 (RMW-bandwidth decode 소자, frequency-tiered placement) 과 accelerator 쪽 (fused chunk kernel, grouped-GEMM decode engine, backward-capable serving kernel) 을 **쌍으로** 제시할 수 있으며, (3) 그 사이에 놓이는 소프트웨어 제안 (per-session-weights serving 스택) 과, 이 책이 **명시적으로 배제**하는 두 FORCED 방향 (훈련용 새 메모리 소자, 일반 PIM 옹호) 을 구별할 수 있어야 한다. **왜 필요한가** — 19-23 장은 pair thesis 의 두 절반을 각각 측정하고 투영했다. 이 장은 그 측정을 **설계 제안**으로 바꾼다. 제안은 pair thesis 의 규율을 그대로 물려받는다: memory-centric 논문은 정직 판정이 인정한 두 지점 (decode RMW 트래픽, update-frequency $\leftrightarrow$ tier) 에서만 펴고, 나머지는 accelerator 쪽 제안으로 배치한다. 흥미로운 하드웨어 제안은 어느 한쪽이 아니라 **쌍**이라는 18 장의 명제 (§ 18.3) 가 이 장의 조직 원리다.

24.1 Bridge-in: 측정에서 제안으로

23 장은 TNT 경제학을 7-70B 로 외삽하고 per-session weight state 를 새 cache class 로 세우며 pair thesis 의 serving 절반을 투영으로 닫았다. 남은 것은 그 투영이 가리키는 **구체적 다음 수** — 무엇을 실험하고, 무엇을 만들 것인가 — 를 명제로 내리는 일이다. 이 장은 그것을 세 층으로 편다: 알고리즘 (§ 24.2), 하드웨어 (§ 24.3 - § 24.5), 그리고 배제 (§ 24.6).

제안 전체를 관통하는 한 문장을 먼저 고정한다. **decode/serving-state** 관리는 **memory-centric** 기획이고, **training/prefill** 은 **accelerator** 영역이다 (D4 pair thesis, $\rightarrow$ 18 장 § 18.3). 이 장의 모든 하드웨어 제안은 이 분할의 어느 쪽에 속하는지로 정당성을 얻는다. memory-centric 제안이 정당한 이유는 그것이 논문들 자신의 cost accounting — token 마다 fast-weight state 전체를 read-modify-write (RMW) 하는 트래픽 — 에 근거하기 때문이고 (§ 18.2 의 두 진짜 지점), accelerator 제안이 정당한 이유는 같은 알고리즘이 chunk 크기 C 를 키우면 compute-bound 로 옮겨 가기 때문이다. 두 제안이 하나의 workload 를 나눠 맡는다는 점이 이 장의 유일한 새 주장이다.

정직성 계약 ($\rightarrow$ 18 장 § 18.6) 은 이 장에도 그대로 적용된다. 아래 인용하는 실측 수치 중 **load-bearing** 으로 승격되는 것은 **비율 · crossover 위치 · tier 순서 · bound class** 뿐이다. 절대 $\mu\text{s}/\text{token} \cdot \text{mJ}/\text{token}$ 은 roofline 하한이고 사내 A100 runbook (Part III-a) 으로 이월된다. novel scratchpad · PIM twin 의 이득 배율은 `simulation_ready=False` 인 directional DSE 로만 인용한다. 이 계약이 특히 이 장에서 중요한 이유는, 제안은 미래형이라 과장의 유혹이 가장 크기 때문이다.

24.2 알고리즘 레벨제안

24.2.1 이미 확정한 것: 두 microbench 가 고정한곡선

Part III 는 pair thesis 를 8 개 실험으로 측정했고 (→ 18 장 표 18-1), 그중 두 CPU microbench 가 **알고리즘 레벨의 곡선 모양**을 하드웨어 불문으로고정했다. 이 둘은 제안의 출발점이자 이후 대형 실험의 기대값이다.

첫째, **chunk C 가 roofline 의 x 축이다** (claim7, E2.1). host roofline 에서 $C=1$ 의 per-token rank-1 RMW — 이것이 곧 TTT decode 영역이다 — 은 arithmetic intensity 약 1 FLOP/byte 로 memory-bound 평원 (host roofline 의 약 12%) 에 붙어 있고, C 를 키우면 crossover 를 넘어 compute-bound 로 오른다: **한 알고리즘, 두 영역**. crossover 위치의 절대값 ($C^* \approx 32$ host) 은 ridge 의존적이라 H100 으로 옮기지 않는다 (H100 closed-form $C^* \approx 306 - 430$, → 9 장 · 15 장). 이전되는 것은 곡선의 **모양**과 memory ↔ compute 교차의 **존재**뿐이다. 둘째, **on-die/off-die 경계에서 RMW 대역폭이 계단식으로꺾인다** (claim8, E2.2): in-cache RMW peak 대비 DRAM 으로 spill 하면 유효대역폭이 약 $3.9 \times$ 붕괴하고 (in-cache peak ÷ DRAM), RMW 는 element 당 $2 \times$ **바이트** (read+write) 를 움직인다 — append-once KV 가 피하는 **write-back 세 (稅)** 를 직접 측정한 값이고, 동일 byte 대역폭 환산시 유효 element throughput 은 read 의 약 $0.5 \times$ 다 (GB/s 절대값은 host 좌표, 이전 금지).

[해설] 이 두 곡선은 대칭이다. 같은 rank-1 RMW 가 decode 에서는 memory-bound 의 근원 ($C=1$) 이고 prefill 에서는 C 를키워 벗어나는 대상이다. 제안이 pair 로 갈리는 알고리즘적 뿌리가 여기 있다 — 하드웨어 두 제안 (§ 24.3, § 24.4) 은 이 한 곡선의 두 끝을 각각 겨냥한다.

24.2.2 대형 실험 제안 A: train/serve chunk-consistency

TNT 의 chunk-size mismatch (→ 15 장) 는 라인 전체에서 **단일 설정의 한그림**에만 근거한다: 550M Titans 를 $C=64$ 로 훈련하면 $C=64$ 에서 ppl 13.78 이던 것이 $C=8$ 서빙에서 36.45 로 무너진다는 [TNT Fig. 2]. 이 그림에는 gating 도 momentum 도 없고, mismatch 가 **왜** 생기는지에 대한 이론도 없다. TNT App. D 는 gated/momentum 변형과의 합성을 명시적으로유보했다.

[평가] 제안 A — **chunk-consistency grid**. 100 – 150M 규모에서 (train chunk × inference chunk) 격자를 {plain GD, +momentum S_t , +retention gate α_t } 세 update rule 에 대해 돌린다. 목적은 두가지다: (i) mismatch 가 세 rule 에서 모두 재현되는가, 아니면 momentum/gating 이 그것을 완화하는가; (ii) 왜 over-specialization 이 일어나는지의 loss-landscape probe. 이 실험은 어느방향으로 떨어져도 publishable 하다 — mismatch 가 재현되면 TNT 의 발견을라인 전체로 일반화하는 것이고, gating/momentum 에서 사라지면 TNT 의 일반성 주장에 대한 **정직한 negative result** 다. 비용은 TNT 의 10B-token 예산을 축소 스케일에서 크게 밑돈다.

이 실험이 서 있는 알고리즘적 근거는 § 24.2.1 의 claim7 곡선이다: chunk 크기는 throughput knob 이면서 **계산되는 함수 자체를 바꾸는 semantic hyperparameter** 다 (stale-snapshot 근사, → 9 장). train/serve chunk 가 다르면 함수가 다르다는 것이 mismatch 의 **유력한 후보 기제**이며 (TNT Fig. 2 는 550M · $C_{\text{train}}=64$ 단일 설정이고 원전도인과를 이론으로 규명하지 않았다 — 확정된 원인이 아니라 제안 A 가 loss-landscape probe 로 검증할 가설이다), 이 실험은 그 기제가 gate-momentum 에 얼마나 민감한지를 측정한다.

24.2.3 대형 실험 제안 B: state-capacity 를 scaling 축으로

19 장은 state-bytes 와 capacity ($O(d_k^p)$, → 14 장의 ϕ^*) 를 params · tokens 와 나란한 일급 scaling 축으로 제안했다. 그제안을 실험으로 달는 것이 제안 B 다.

[평가] 제안 B — **capacity scaling fit**. 3-4 개 크기 $\times$ 3 개 architecture class(matrix memory, deep-MLP memory, deep+featurized memory) 를 **동일 토큰**으로 훈련하고, ppl 을 (params, state-bytes, capacity-proxy) 에 대해 fit 한다. 핵심 질문: capacity-proxy 가 raw state-bytes 보다 long-context 벤치마크를 더 잘 예측하는가. falsifier 는정직하게 둘이다 — (i) 공개 점들이 tokenizer·데이터가이질적(라인 전반 Llama-2 vs T5 vocab) 이라 cross-architecture fit 이불안정하고, (ii) 자체 run 이 exponent 를 안정화하기엔 너무 작을 수 있다. 이 실험은 안정적 exponent 가 아니라 **capacity** **측의 예측력** 유무를판정하는 것이 목표다.

[결과] E4 가 이미 좁힌 것. 제안 B 의 대형 run 은 미실행이지만, 그 전초로 여섯 편의 공개점 (params·tokens·context·ppl/score) 을 digitize 해 후보 scaling fit 을 돌린 소규모 실험 (E4) 이 이미 A3 를 **날카롭게 좁혔다**. 결과는 정직하게 둘로 갈린다. (i) **perplexity** **측에서 capacity** 는 **아무것도 더하지 않는다**: parametric family(matrix $\rightarrow$ deep-MLP $\rightarrow$ deep+poly) 안에서 state-bytes 와 capacity-ordinal 은 rank-동치이고 둘 다 ppl 과 완전반상관 ($\rho = -0.95$) 이라, capacity-proxy 가 raw state-bytes 위에 없는예측력이 0 이다. 게다가 attention corner(capacity 무한이되 ppl 은 최악, Transformer++ 18.53) 를 넣으면 capacity $\leftrightarrow$ ppl 상관이되집힌다 ($\rho = 0.09$; raw state-bytes 는 attention=0 으로 $\rho = -0.69$ 유지) — ppl 은 압축을 보상하지 raw retrieval capacity 를보상하지 않으므로, capacity 를 ppl 측으로 fit 하면 **틀린 것을 재는셈**이다. (ii) 반대로 **long-context/retention** **측에서는 capacity** 가 **살아난다**: BABILong 지속 정확도 길이가 parametric family 안에서 capacity-ordinal 과 단조 ($\rho = 1.0$) 이고, 특히 Titans $\rightarrow$ Atlas 의약 $6.7 \times$ retention 도약을 약 $1.5 \times$ 에 불과한 state-byte 증가가 underpredict 하는데 capacity-**class** 변화 (deep-MLP $\rightarrow$ deep+poly, ordinal 2 $\rightarrow$ 3) 가 그 도약을 설명한다. 그러므로 제안 B 의 남은 대형 run 은 ppl 예측이 아니라 **retention** **예측**으로 재조준되고, 19 장은 state-bytes·capacity 를 **long-context** **타깃에 한해**일급 scaling 측으로 제안하되 cross-architecture ppl $\leftrightarrow$ capacity fit 은 명시적으로 경고한다 ($\rightarrow$ 19 장). (모든 점은 tokenizer·데이터가이질적이라 ordinal·directional; 자체 3-8 점은 Spearman ρ 만 load-bearing, 절대 exponent 는 directional 하한.)

24.2.4 이 실험들의 vehicle: HOPE/Sleep 재현 runbook

제안 A·B 와, 라인 전체의 부재한 decode wall-clock(6 편 전부미공개, $\rightarrow$ 18 장 § 18.2) 을 실측하려면 실행 가능한 재현 경로가 있어야 한다. 정찰 결과는 냉정하다: **6 편 전부 공식 구현이 없고**, 사실상의 reference 는 lucidrains/titans-pytorch 하나이며 그마저 논문밖 확장이 섞여 있다. HOPE 는 비공식 2 종 (84 ★/76 ★; 별 수는 2026-07 정찰 시점) 이 소규모·단일 GPU 수준으로만 존재하고, Sleep 의 wake/sleep 파이프라인은 지구상에 구현이 0 개다.

[평가] 제안 C — 하이브리드 4 층 runbook. (1) 베이스라인·백본 = flash-linear-attention + flame(공식급, multi-GPU 검증됨); (2) Titans neural memory = lucidrains 참조를**논문 모드로 flag** 고정하고 fla/ops/titans naive 를 수치 oracle 로 병용; (3) HOPE = 자체 구현 (비공식 2 종은 참조·감사 대상이지신뢰 기반 아님 — 둘 다 surrogate loss 를 섞으므로 fork 하면 “무엇을측정했는지” 가 모호해진다); (4) Sleep/Dreaming = 완전 자체구현. 4 층은 제안 A·B 의 model-training 실험과 A100 runbook 의 wall-clock 실측을 동시에 실어 나른다. Sleep 구현은 이 스터디의 잠재적오픈소스 기여 지점이다.

이 runbook 이 memory-centric 서사에 종속되지 않는다는 점을 분명히 한다. runbook 의 1 차 산출물은 알고리즘 검증 (chunk semantics, capacity 예측력) 과절대 wall-clock 이며, 그 wall-clock 이 나와야 § 24.3 의 하드웨어 제안의절대값 하한이 검증 가능한 실측으로 바뀐다. 즉 알고리즘 제안이 하드웨어제안의 warrant 를 공급한다.

[결과] runbook 이 실측할 것과, 이미 digitize 한 것. 4 층 runbook 의 1 차 산출물은 셋이다: (a) chunk semantics — 제안 A 의 (train chunk $\times$ infer chunk) 격자; (b) capacity 예측력 — 제안 B 의 retention fit; (c) 절대 decode wall-clock — 라인 전체가 미공개한, § 24.3 하드웨어 제안의 절대값 하한을 검증으로 바꾸는 수치. (c) 는 A100 runbook 이 실측하고, (a)·(b) 의 부호는 E4 가 공개 점 digitize 로 이미예열했다. 두 TNT 점이 그것이다: (1) TNT 안에서 local memory 수를 $0 \rightarrow 4$ 로 늘리면 avg ppl 이 $23.53 \rightarrow 20.15$ 로 단조 감소하되 ($\rho = -1.0$) +1 이후 saturate($21.04 \rightarrow 20.15$) 해 state 수의 diminishing return 을 보이고 — 이는 제안 B 가 “state 가 많을수록 좋다” 를 순진하게 fit 하지 못하게 하는 경계다; (2) 550M Titans 를 $C=64$ 로 훈련한 뒤 infer chunk 를 쓸면 ppl 이 훈련 chunk 에서 최소인 U 자 ($C=8:36.45$, $C=64:13.78$, $C=512:22.4$; $\text{argmin } C=64$) 로, scaling law 가 아니라 train/serve resolution mismatch($\rightarrow 15$ 장) 의 digitize 된 재확인이다. 두 점 모두 제안 A·B 의 대형 run 이 어느 쪽으로 떨어질지의 부호를 미리고정한다 — 대형 run 이 이 부호를 뒤집으면 그 자체가 라인의 일반성 주장에 대한 정직한 negative result 다. (모든 수치는 published-data digitize 또는 exploration-grade fit; 절대 ppl 은 원문 좌표.)

24.2.5 알고리즘 제안 J: learned chunk/cadence schedule

제안 A·B·C 가 고정된 schedule 위에서 라인을 측정한다면, 마지막 알고리즘 제안은 그 schedule 자체를 학습대상으로 올린다. 라인 전체에서 chunk 크기 C , window c , CMS 갱신주기 $C^{(\ell)}$, level 수, sleep 발화 시점은 전부 hand-set 이다 ($\rightarrow$ dossier § 3.2 의 다섯 공백 중 네 번째). NL 은 잘못 배치된 level 이 오히려 해가 됨을 자기 ablation(inner- q) 으로 보였고, Sleep 은 sleep 을 CMS chunk 경계에 hard-wire 했다 — 무엇을 언제 갱신할지를 배우는 기제는 아직 없다.

[평가] 제안 J — learned update schedule. claim7 이 고정한 사실 — chunk 크기는 throughput knob 이자 계산되는 함수를 바꾸는 semantic hyperparameter(stale-snapshot 근사, $\rightarrow 9$ 장) — 이 이 제안의 근거다. C 가 semantic 이면 그것을 상수로 두는 것은 손실이다: token·context 에 따라 update cadence 를 내는 작은 controller(예: momentary surprise 크기 $\|g_t^{\text{in}}\|$ 에 조건한 chunk-boundary gate) 를 outer loop 으로 meta-learn 한다. 이것은 § 24.3.2 의 frequency-tiered placement 와 직접 맞물린다 — cadence 가 학습되면 tier admissibility 도 정적 표가 아니라 동적 배치 신호가 되기 때문이다. falsifier 는 둘이다: (i) 학습된 schedule 이 잘 튜닝된 상수 C 를 유의하게 이기지 못하면 (제안 B 의 diminishing-return 경계가 시사하듯 라인 성능이 C ·state 수에 완만하면) 제안은 복잡도만 늘린다; (ii) chunk 경계의 이산성이 gradient 를 끊으므로 straight-through 류 완화가 필요하고, 그 완화가 원래의 stale-snapshot 근사와 겹쳐 “무엇을 측정했는가” 를 흐릴 위험이 있다. 이 제안은 그래서 제안 A(chunk-consistency) 가 mismatch 의 뿌리를 gate·momentum 에 대해 고정한 뒤에야 정직하게 돌릴 수 있다.

24.3 하드웨어 레벨 — memory-centric 절반

이 절의 두 제안은 pair thesis 가 인정한 진짜 두 지점에서만 나온다. 그 밖의 memory-centric 주장은 § 24.6 에서 명시적으로 배제한다.

24.3.1 RMW-bandwidth decode 소자

decode step 은 anchor(neural-mem-1.3B) 에서 token 마다 6.44 GB 를 움직이고 arithmetic intensity 가 0.59 FLOP/byte 로, H100 twin ridge 295 FLOP/byte 보다 두 자릿수 아래다 — 결정적으로 memory-bound 이고, state RMW 트래픽이 decode step 의 GEMV 연산을 약 $394 \times$ 압도한다 (claim1, E1.1/E3). step 비용이 곧 state 트래픽이다. 이 트래픽은 모델 폭에 따라 $m d^2 L_{\text{layer}}$ 로 자라 $0.45 \rightarrow 6.44 \rightarrow 343$ GB/token(Titans-170M $\rightarrow$ 가상 70B) 으로 증가하고,

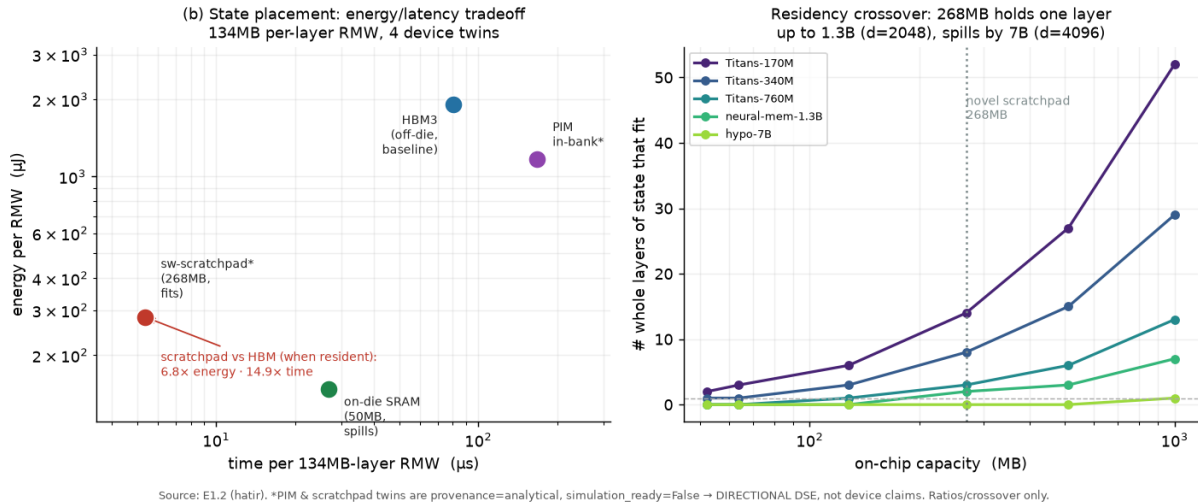


그림 24-1 — state placement 의 device 별 energy/latency 트레이드오프와 residency crossover

KV 와 달리문맥에 무관하게 일정하다. (절대 1.92 ms/token·46.5 mJ/token 은 roofline 하한이며 A100 runbook 으로 이월한다 — 본문이 담는 것은 memory-bound·RMW-지배라는 구조뿐이다.)

이 구조가 요구하는 소자의 성격은 분명하다: **높은 RMW 대역폭, per-tenant state residency**, 그리고 write-heavy elementwise epilogue 를 위한 **near-memory update engine**. 근거는 두 실측이다. 첫째, TTT state 크기는 문맥에 무관하게 (d, m, L) 로 고정되므로 on-chip 상주가 **설계 가능한 knob** 이다 (context 에 따라 자라는 KV 와 결정적으로 다른 지점, claim4/E1.2). state 가 fit 하는쪽에서는 268 MB scratchpad 가 HBM3 대비 $6.8\times$ energy / $14.9\times$ time 이득을 내지만 (directional DSE), 폭이 커지면 spill 한다 — residency crossover 는 hidden dim $d^*\approx 2896$ 이고, 268 MB 버퍼는 한 layer state 를 $d=2048$ (1.3B, 134 MB/layer) 까지 담고 $d=4096$ (7B, 537 MB/layer) 에서 넘친다. whole-model 상주는 Titans-170M(226 MB) 하나만 fit 하므로, 스케일에서 **placement** 는 **whole-model pin** 이 아니라 **per-layer/streamed** 다.

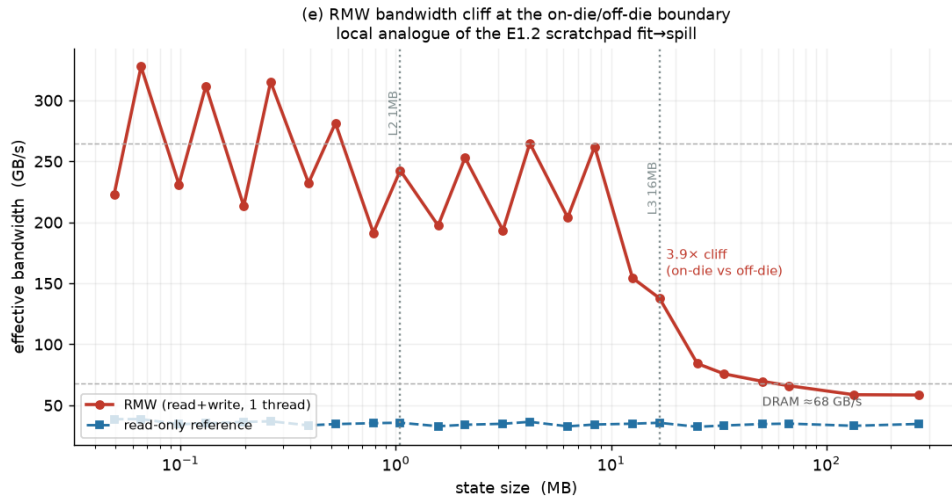
그림 24-1 — 134 MB/layer RMW state 를 device 별로 배치했을때의 energy·time 과, 268 MB scratchpad 가 한 layer state 를담을 수 있는 폭의 상한 (residency crossover $d^*\approx 2896$). state 가 fit 하는 구간에서만 scratchpad 가 HBM3 를 이기며, 이득 배율은 simulation_ready=False 인 directional DSE 다 (실험 E1.2 재구성).

둘째, 이 상주/spill 불연속이 로컬 실측으로도 재현된다: on-die/off-die 경계에서 RMW 대역폭이 약 $3.9\times$ 꺾이는 cliff 가 그것이다 (claim8/E2.2, 그림 24-2). 소자 제안의 요지는 이 cliff 의 **on-die 쪽에 layer state 를 붙잡아 두는 것**이며, per-layer 단위로는 그것이 가능하다는 것이 E1.2/E1.4 가 함께 보이는결론이다.

그림 24-2 — state 가 on-chip cache 에 상주하는 동안 RMW 는 빠르고, 용량경계를 넘겨 DRAM 으로 spill 하면 유효 대역폭이 약 $3.9\times$ 붕괴한다 (in-cache peak ÷ DRAM). RMW 가 element 당 $2\times$ 바이트 (동일 대역폭 환산 시 element throughput $0.5\times$) 를 움직이는 것이 append-once KV 가 피하는 write-back 세다. GB/s 절대값은 host 좌표이며 H100 으로 이전하지 않는다 — 이전되는 것은 cliff 의 존재뿐이다 (실험 E2.2).

near-memory update engine 에 대해서는 PIM 이 오직 **write-heavy elementwise epilogue**(decay·renorm·AXPY) 라는소수 FLOP 지점에서만, 그것도 directional 하게 등장한다. TTT epilogue 의 약 3 MAC/elem 에서 PIM 은 $1.6\times$ energy 이득이나 $2.1\times$ latency 비용을 내고, rank-1 에 가까운 1 MAC/elem 에서는 energy 와 latency 둘 다 이득이다 (claim4/E1.2). latency 손해는 1 과 3 MAC/elem 사이에서 뒤집혀 측정 격자의 3 MAC/elem 에서 이미 물린다 (1 MAC/elem 에서는 $0.7\times$ time 으로 이득).

이 세 요구(높은 RMW 대역폭 · per-tenant residency·write-heavy epilogue) 를 하나의 소자 스케치로모으면 **near-memory update engine** 의 마이크로아키텍처가된다 (directional,



Source: E2.2 (CPU numpy). SHAPE-only: the 3.9× L3→DRAM cliff structure transports; GB/s values do not (H100 on/off-die ridge ~100× elsewhere). RMW ≈ 0.5× read throughput = the write-back tax KV avoids.

그림 24-2 — on-die/off-die RMW 대역폭 cliff (E1.2 residency crossover 의 로컬 아날로그)

silicon 미검증). 데이터패스는 세 스테이지다. (1) **state-tile 상주 버퍼** — 한 layer 의 fast-weight $W(+momentum S_t)$ 를 tile 단위로 on-die/scratchpad 에 붙잡는 SRAM. per-layer 상주가 가능한 폭 (residency crossover $d^* \approx 2896$ 아래) 에서만 tile 이 fit 하고, 넘으면 streamed 다 — whole-model pin 은 Titans-170M 만 가능하므로 (claim4) 상주 단위는 layer 다. (2) **epilogue ALU lane** — read 된 state tile 에 master update $W_t = \alpha_t W_{t-1} + S_t (\rightarrow \text{STYLE (M2)})$ 를 적용하는 write-heavy 경로: decay($\alpha_t \odot$), AXPY($S_t = \beta_t S_{t-1} - \eta_t g_t^{\text{in}}$ 누적), renorm 을한 pass 에 fuse 한다. 이 lane 이 § 24.3.1 이 PIM-적합으로 한정된 minority-FLOP epilogue(약 1–3 MAC/elem) 를 흡수하고, GEMV/GEMM 형태의 gradient 계산 본체는 tensor-core 로되돌린다 — § 24.6 배제 2 와 정합이다. (3) **per-tenant residency slot table** — state 가 sequence 별 unshared 사본이므로 (claim4), engine 은 slot ↔ sequence 바인딩과 slot 별 dirty 비트를 유지하는 작은 directory 를 갖는다. 이 directory 가 곧 § 24.5 소프트웨어 층의 `bind_to_sequence · mark_dirty` 의 하드웨어 짝이다: update 는 append 가 아니라 in-place 로 slot 을 덮어 T 개 stale 버전 누적을 막고 (claim6 G2, 256×), eviction 시 dirty slot 은 반드시 writeback 경로로 흐른다 (무료 drop 금지, G3). 요컨대 소자 · 매니저 · kernel 이 같은 세 measured 사실 (memory-bound whole-state RMW, per-seq residency, 항상-dirty) 을 세 층에서 각각 구현하며, 소자 D 의 근거는 이 세 사실 어느것도 위에 얹은 것이 아니라 논문들의 cost accounting 에서 읽어 낸 것이다.

[평가] 제안 D — **RMW-bandwidth decode** 소자. high-RMW-bandwidth state residency(per-layer 상주를 겨냥한 on-die/scratchpad 계층) + per-tenant state placement + elementwise epilogue 를 위한 near-memory update path. 이 제안은 논문들의 cost accounting 에 근거한 것이지 그 위에 얹은 것이 아니다 — decode 가 memory-bound whole-state RMW 라는 사실은 measured structure 이고, epilogue 의 PIM 적합성은 minority-FLOP·directional 로 한정된다. 절대 이득 배율은 silicon 검증 전까지 directional 로만 인용한다. 한 요구가 더 있다 — **multi-tenant isolation 과 privacy** 다 (dossier § 3.2 의 serving 공백). KV page 는 append-once·공유 가능이라 tenant 간 격리가 논리적 partition 으로 족하지만, RMW state 는 tenant 의 문맥이 weight 에 흡수된 사본이라 slot 이 물리적으로 격리돼야하고 (한 slot 의 잔여가 다음 tenant 에 새면 문맥 누출이다), free 는 반드시 잔여 zeroization 을 동반해야 한다. 이는 slot table 의 bind/free semantics(claim6) 를 소자 수준 격리 요구로 승격한다. 이 요구는 정직 판정의 두 지점에 얹는 새 주장이 아니라 per-tenant residency 의 직접 귀결이다 — falsifier: state 가 사실상 문맥을 복원 불가능하게 압축한다면 (privacy 가 압축으로 자동 확보되면) 물리 격리의 필요성이 약해진다. 그 복원 가능성 자체가 미해결 질문이므로 이 요구는 보수적으로 유지한다.

(d) Update cadence → memory-tier assignment (★ = recommended)
amortised write energy per token; gated by the FASTER of read/write cadence

fast-weights R+W / token	no cap	★ 11354.82 μJ/tok	19407.88 μJ/tok	24239.72 μJ/tok
CMS-chunk R/tok, W/64	no cap	★ 177.42 μJ/tok	303.25 μJ/tok	378.75 μJ/tok
CMS-mid R+W / 4096	no cap	2.77 μJ/tok	4.74 μJ/tok	★ 5.92 μJ/tok
sleep-experts R+W / 262k	no cap	0.13 μJ/tok	0.22 μJ/tok	★ 0.28 μJ/tok
frozen-weights R/tok, W never	no cap	★ 0.000 μJ/tok	0.000 μJ/tok	0.000 μJ/tok
	SRAM* 60TB/s	HBM3 3.35TB/s	DDR5 0.4TB/s	CXL 0.5TB/s

■ recommended tier
 ■ admissible (not cheapest gated)
 ■ capacity overflow

Source: E1.4 (hatir). *SRAM tier = novel scratchpad twin (simulation_ready=False, directional). Sub-per-token cadence lets a write-heavy RMW state legally drop to DDR/CXL without lengthening the critical path.

그림 24-3 — update cadence → memory-tier 허용 표

24.3.2 frequency-tiered memory placement

두 번째 memory-centric 지점은 NL/Sleep 의 update-frequency 정의에서 **직역**된다. Nested Learning(HOPE) 과 Sleep 은 각 memory level 에 update cadence 를 부여한다 (per-token fast weights → chunk 주기 CMS level → sleep 주기 grown expert → frozen weights, → 16 장·17 장). 이 cadence 는 memory 계층의 tier 배치 사양으로거의 문자 그대로 번역된다 (claim5/E1.4).

번역의 규칙은 하나다: **상주 사본은 read/write 중빠른 쪽 cadence 로 tier 가 정해진다** (gating rule). 그 귀결이 placement 표다 — per-token fast weights 는 HBM3 에 pin 되고; per-token read 이되 chunk 단위 write 인 CMS-chunk level 은 read cadence 가 뜨겁게 붙잡아 HBM3 에 남으며; read·write 가 모두 4096 token 주기로 down-sample 된 CMS-mid level 은 CXL 로 legally 내려가고; offline cadence 의 Sleep-consolidated expert 도 CXL 로 간다; frozen weights 는 per-token read 라 HBM3 의 read-heavy(accelerator-friendly) 쪽에 놓인다. 핵심 통찰은 **sub-per-token access cadence 가 트래픽을 critical path 아래로 amortise** 한다는 것이다 — write-heavy RMW state 라도 갱신 주기가 길면 느린 tier 가 허용된다.

정직한 경계 하나 — read cadence 의 출처. CMS-mid 를 CXL 로 내리는근거는 그 level 의 **read** 가 4096 주기로 down-sample 된다는가정이다. 그러나 Nested Learning Eq. 70 과 Sleep Eq. 1 의 일반 forward 는 **모든 CMS block 을 각 token 의 forward chain 에서 호출**하며, 주기가 다른 것은 parameter **update**(Eq. 71/Eq. 2) 뿐이다. 즉 write cadence 가 4096 이라고 read cadence 까지 4096 인 것은 아니다 — block 이때 token forward 에 남으면 read 는 per-token 이고 gating rule 은 그것을 HBM3 에 pin 한다. 따라서 CMS-mid 의 CXL 강등은 그 block 이 **sparse routing·cached activation·비활성 expert offload 로 forward 에서 genuinely 건너뛰어질 때에 한해** admissible 하며 (E1.4 의 read cadence 는 공개 decode trace 가 아닌 그 가정위의 값이다), 그렇지 않으면 활성 block 은 per-token read 때문에 hot tier 에 남는다. Sleep-consolidated expert 도 consolidation cadence(262144)는 offline write 이되 wake 중 read 는 routing 에 의존한다. 이 tier 배치가 조건부 directional 가설인 이유가 여기 있다.

그림 24-3 — CMS level 별 update 주기 (행) 와 memory tier(열) 의 허용 관계. latency tolerance 가 update 주기에 선형으로늘어지므로 ($P_{\text{read}} \cdot t_{\text{tok}}$), down-sampled level 일수록싼 tier 가 legally 허용된다. per-layer fast-weight block(67 MB) 은 268 MB scratchpad 에 fit 해 HBM3 대비 $7.4 \times$ energy 이득을 내지만 all-layer 는 fit 하지 않는다 (실험 E1.4, scratchpad 이득은 directional).

[평가] 제안 E — frequency-tiered placement engine. update-frequency

연속체를 tier admissibility 규칙으로 소비하는 배치계층: hot fast-weights 를 HBM3/on-die 에, down-sampled CMS-mid 를 DDR/CXL-class latency 로, sleep-consolidated expert 를 명시적 data-migration 스케줄로. 이것은 이 스테디에 서 가장신선한 memory-architecture 주장이다 — NL/Sleep의 정의가 **이미** memory-hierarchy 사양이라는 것을 어떤 선행 연구도 진술하지 않았고, 매핑이 거의 문자 그대로 이기 때문이다. 다만 이 규칙은 cadence 를 tier 의 **한 입력 변수**로 삼는 directional 가 설이지 단독 admissibility 법칙이 아니다: 드문 접근이 평균대역폭·energy 를 amortise 하더라도, 그 접근이 실제로 일어나는 token 은 prefetch·비동기 실행이 없으면 여전히 느린 tier 의 **동기 latency** 를 묻다. 따라서 CXL 배치의 실 admissibility 에는 cadence 외에 transfer size, link latency, prefetch window, 동시 상주 sequence 수가 함께 들어가며, 이 tier 판정은 `simulation_ready=False` 인 directional DSE 로 만인용한다.

placement 표를 **정적** 배치로만 읽으면 절반이다. update-frequency 가 tier 를 정한다는 규칙은 곧 **cadence 가 바뀌는순간이 마이그레이션 트리거**라는 뜻이고, wake/sleep lifecycle(→ 17 장) 이 정확히 그 순간을 만든다. 그래서 표 (그림 24-3) 는 다음마이그레이션 스케줄로 퍼진다. (a) **promotion(CXL → HBM3)** — sleep 이 consolidated expert 를 grow 해 그것이 wake 에서 per-token read 대상이 되면, read cadence 가 그 사본을 hot 으로 재분류하므로 다음 wake 진입 전에 HBM3 로올린다. (b) **demotion(HBM3 → CXL)** — sleep 의 synaptic-pruning reset 이 fast block 의 남은 expert 를 무효화하면 그 사본은 더 이상 per-token 접근되지 않으므로 CXL/ idle 로 내린다. (c) **정적 상주** — per-token fast weights 와 frozen weights 는 cadence 가 바뀌 지 않으므로 이동 없이 HBM3 에 남는다 (각각 write-gated·read-gated). 마이그레이션 비용 자체 가 tier 판정의 입력이라는 점이 중요하다 — sub-per-token cadence 가 트래픽을 amortise 하더라도, 승격/강등 transfer 가 임계 window 안에 끝나지 못하면 그 접근 token 은 여전히 느린 tier 의 동기 latency 를 묻다. 따라서 이 스케줄의 admissibility 에는 cadence 외에 transfer size, link latency, prefetch window 가 함께 들어가며, 스케줄 전체는 `simulation_ready=False` 인 directional 가설이지완결된 배치 법칙이 아니다.

24.4 하드웨어 레벨 — accelerator 절반 (pair 의 다른쪽)

memory-centric 제안 (D·E) 은 pair thesis 의 절반일 뿐이다. training/prefill 과 decode 의 dense-matmul 부분은 accelerator 영역이며, 여기서의 승부는 kernel 에서 난다. 세 제안이 § 24.2.1 의 claim7 곡선의 compute-bound 끝을 겨냥한다. E2.1 host 측정에서 같은 chunk-wise scan 이 $C=1$ 의 4.56 GFLOP/s 에서 $C=512$ 의 689.5 GFLOP/s 로 약 $151\times$ 오르는 것이 (측정 wall-clock, shape-only) 이 끝의 존재를 로컬로 확인한다 — 절대 GFLOP/s 는 host 좌표라 이전 금지, 이전되는 것은 상승의 **모양**뿐이다.

제안 F — fused chunk kernel. claim7 이 보이듯 C 를 키우면 chunkwise scan 이 compute-bound 로 오른다. 그런데 정찰 증거는 이 kernel 이 **아직 없다**는 것을 구조적으로 드러낸다: flash-linear-attention 의 `fla/ops/ttt` 는 진짜 Triton chunkwise 커널을 가졌지만, `fla/ops/titans` 는 naive PyTorch 참조 구현에만 머문다. TTT 만 Triton 화되고 Titans 가 naive 에남은 이 사실 자체가 **momentum·retention 을 포함한 deep-memory chunk 의 fused 커널이 아직 없다**는 생태계 측 물증이다 — 그것이 단순한 구현 공백인지 momentum 의 chunkwise 병렬화 자체의 난점인지는 제안 F 가 가를 열린 질문이다 (→ 9 장·15 장; 아래 [리스크] F). 제안은 momentum·retention 을 포함한 deep-memory chunk 를 fuse 하는 커널이며, 이것이 라인이 아직 기다리는 “FlashAttention-moment” 의 후보다 (→ 21 장). TNT 가 plain JAX 만으로 32K 에서 step 당 FlashAttention 을 이미 이겼다는 점 (→ 15 장) 이 이 방향의 상한이 존재함을 시사한다. (단, TNT 는 momentum·gating·Muon 을 “명료성을 위해” 제거한 단순화 Titans 로 검증했으므로 — App. D — 라인 전체와의 합성은 미검증이다.)

[리스크] F. falsifier: momentum·retention 을 포함한 chunk 의 chunkwise closed form 이 존재하지 않아 fuse 해도 sequential 의존이 남으면 — 즉 fla 가 Titans

를 naive PyTorch 에 둔 것이 편의가 아니라 필연이면 — 커널은 부분적 fusion 에 그친다. 또 TNT 의 FlashAttention 우위는 momentum-gating 을 제거한 단순화 Titans 에서만 측정됐으므로 (App. D), 상한의 존재는 시사일 뿐 full-line 합성 커널로 이전된다는 보장이 아니다.

제안 G — grouped-GEMM decode engine. per-request mutable fast weights 는 shared-weight batching 을 깨뜨린다 (→ 23 장, 1 장 Rosetta 의 batching 행). token 마다 request 별로 다른 W 를 RMW 하므로, 오늘의 decode kernel 이 전제하는 “여러 request 가 같은 weight 를 공유한다” 가 성립하지 않는다. 제안은 request 별 state 를 batch 축으로 묶어 처리하는 grouped-GEMM decode path 다. 이것이 accelerator 제안인 이유는, request 별 unshared weight 의 grouped-GEMV 가 근본 **arithmetic intensity** 를 올리지는 못하지만 (각 W_b 가 한 번 읽혀 한 번 쓰이므로 FLOP-traffic 이 함께 B 에 비례해 AI 는 B 와 무관), kernel-launch-occupancy-scheduling 파편화를 견어 memory-bound 평원의 **포획 손실**을 줄이기 때문이다. AI 자체를 올리려면 같은 session W 에 여러 token/vector 를 태우는 chunk-speculative batch 나 구조화-state 공유가 따로 필요하다 (→ § 24.5 bind_to_sequence).

[리스크] G. falsifier: decode 가 결정적으로 memory-bound (AI 0.59 $\ll$ ridge 295, claim1) 이므로 grouped-GEMM 이 arithmetic intensity 를 끌어올려도 state RMW 트래픽이 여전히 지배하면 이득이 상한에 막힌다. G 는 batching 가능성을 복원하는 것이지 트래픽 하한을 낮추지 않는다 — 그 하한은 소자 D 의 몫이며, 그래서 G 와 D 는 같은 decode step 을 나눠 맡는다.

[해설] never-average — $B_{\max}$ 벽의 미시근거. 제안 G 가 batch 를 형성하되 그 batch 가 왜 shared-weight decode 처럼 amortise 되지 않는지의 뿌리는 하나의 불변식이다: serving 에서 request 별 fast weights W_b 는 각자의 독립 trajectory 이고, inner-loop update 는 request 축으로 **절대 평균되어선 안 된다**. request b 의 momentary surprise $g_{b,t}^{\text{in}} = \nabla_W \ell(W_{b,t-1}; k_{b,t}, v_{b,t})$ 를 다른 request 와 reduce 하면 한 tenant 의 문맥이 다른 tenant 의 state 로 새어든다 (정보 오염·leakage — 제안 D 의 per-tenant isolation 요구와 같은 뿌리). 즉 batch 는 request 방향으로 **map** 이지 reduction 이 아니다 — 각 W_b 가 자기 gradient 로만 갱신되는 벡터화된 병렬 실행이다. 이 unshared·비-reduction 성질이 곧 shared-weight 상각이 불가능한 **미시적 이유**이고, arithmetic intensity 가 B 와 무관해 대역폭이 배치를 조기에 닫는 $B_{\max}$ 벽 (bandwidth-bound, → 23 장 표 23-1) 의 근원이다. 대비되는 것은 outer-loop meta-parameter Θ 다 — 이 쪽은 훈련에서 batch 로 평균하는 것이 정상이다 (inner = request 별 개별 map, outer = batch 평균; 두 loop 의 형식구분 → 4 장). 요컨대 $B_{\max}$ 벽은 대역폭 회계의 우연이 아니라 inner-loop 가 request 별로 격리돼야 한다는 정확성 요구의 하드웨어적 그림자다.

[평가] workload-class 라벨 — Self-Modifying Titans decode 는 무엇인가. 이 decode step 을 오늘의 kernel 분류에 얹히면 attention 한 종류가 아니라 “**request 마다 weight 가 다른 recurrent MLP + batched fast-weight optimizer/update kernel**” 에 가깝다 — 세 성분의 결합이다: (i) request 별 unshared 2-layer MLP memory 의 recurrent 상태전이 (TTT layer, → 8 장), (ii) 그 상태를 momentum-retention 으로 갱신하는 batched optimizer step (→ STYLE (M2)), (iii) chunk/recurrent scan (→ 9 장). 세 경계를 지킨다. **첫째**, chunk/scan 축의 token 병렬성은 **prefill/training 전용**이다 — autoregressive decode 는 미래 token 이 없어 chunk 축이 붕괴하고 batch 축만 병렬로 남는다 ($C=1$ 끝, → § 24.2.1 claim7·15 장). 그래서 decode 의 유일한 batchable 차원이 request 축이고, 바로 그 축이 never-average (위 [해설]) 의 지배를 받는다. **둘째**, “MoE 와 닮았다” 는 유비는 **dispatch form** 에서만 성립한다: MoE 는 다수 token 이 소수 shared expert weight 를 재사용해 그 weight 를 amortise 하지만, 여기서 request 마다 weight 가 달라 decode 시 group 당 token 이 사실상 1 이라 expert-weight 상각 이득이 없다 — dispatch 골격은 grouped-GEMM

으로 같되경제학은 정반대다 (→ 리스크 G). 셋째, 상태 전이의 clean matrix form 은 linear·dot-product 특수형이고, 배포되는 2-layer MLP memory 는 일반 $\nabla_{W\ell}$ update(residual-gradient 포함) 를 쓴다 —라벨의 “recurrent MLP” 는 후자다 (→ 8 장). 이 라벨의 실천적 귀결은 앞 절과 같다: 한 decode step 이 두 kernel 로 갈려, state RMW 트래픽 하한은 소자 D 가, batch 축 재조직 (grouped-GEMM) 과 inner-loop backward 는 accelerator kernel G·H 가 맞는다 (제안 G·D 가 같은 step 을 나눠 맡는 분업, → 리스크 G· § 24.4 [평가]).

제안 H — backward-capable serving kernel. 이 라인의 decode 는 inference 인데 **backward pass** 를 품는다 — inner loop 이 $\nabla_{W\ell}$ 를 계산해 state 를 갱신하기 때문이다 (→ 1 장 Rosetta 의 “backward pass 가 decode 에 들어온다” , 8 장). 오늘의 serving kernel 은 forward-only 로 설계되어 있어 이 primitive 를 지원하지 않는다. 제안은 decode 경로에 inner-loop backward 를 일급 연산으로 포함하는 serving kernel 이다. NS-5(Newton – Schulz, → 14 장) 와 deep-memory 의 FLOP density 가여기서 accelerator 자원을 요구하는 지점이며 (→ 20 장), 이것은 memory-centric 소자가 아니라 tensor-core 영역의 문제다.

[리스크] H. falsifier: inner-loop backward 가 forward 대비 지배적 FLOP 이 아니면 (deep memory 가 얇고 NS- κ 반복이 적으면) 전용 primitive 의 이득이 작아 기존 forward-only 커널을 재사용하는 편이 낫다. 즉 H 의 정당성은 deep-memory·NS-5 의 FLOP density 가 실제로 높은 config 에 한정되며 (→ 20 장), 그 density 를 재는 것이 A100 runbook 의 과제 중 하나다.

[평가] 제안 F·G·H 는 D·E 와 경쟁하지 않고 짝을 이룬다. decode 의 state 트래픽은 memory-centric 소자 (D) 가 흡수하고, decode 의 backward 연산과 batching 재조직은 accelerator kernel(G·H) 이 맡으며, prefill/training 의 compute-bound chunk 는 fused kernel(F) 이 처리한다. 어느 한쪽만 옹호하는 제안은 workload 의 절반을 무시한다 —이것이 18 장 pair thesis 의 실천적 귀결이다.

24.5 소프트웨어 제안: per-session-weights serving 스택

하드웨어 제안 사이에 소프트웨어 층이 놓인다. 기존 KV-cache manager 의 semantics 는 RMW state 에 대해 **범주 오류**임이 코드 레벨에서 확인된다 (claim6/E1.5, hat-kv-manager/manager.py 근거). 세 gap 이 결정적이다: (G1) content-addressed prefix reuse — manager 의 1 번 가치, KV 에서 0.5 hit-rate — 가 RMW state 에서는 **구조적으로 0** 이다 (content 가 token 마다 변이해 block hash 가 재현되지 않음). 단 이 0 은 **content-addressed(block-hash) 재사용에 한정**된다 — 같은 prefix 를 결정론적으로 처리한 서로 다른 요청은 분기 전까지 동일한 state snapshot 에 도달하므로 prefix-snapshot 공유 + copy-on-write 는 원리상 가능하며, append-only KVCacheManager 가 그 mutable-state 재사용을 표현할 API 를 갖지 못한 것이 바로 새 manager 가 필요한 이유다; (G2) in-place update 없는 append-only placement 가 T 개의 stale 버전을 쌓아 256 token 에서 참 working set 의 $256\times$ 로 부풀다; (G3) `_evict_from()` 이 dirty state 를 clean/recomputable 로 가정해 **무료로 drop** 하는데, 실제 TTT eviction 은 writeback(또는 결정론적 prefix-replay 재계산) 을 진다 (E1.5 의 416 MiB eviction set 에서 약 3.05 mJ / 130 μ s, whole-session 은 footprint 비례 — NO-WALLCLOCK, A100 runbook 이월).

그 귀결로 TTT-state manager 는 KVCacheManager 에 없는 다섯 event type 을 요구한다 (claim6). 이것을 그대로 per-session-weights serving 설계로 옮긴다.

[평가] 제안 I — per-session-weights serving stack. claim6 의 다섯 누락 API 를 설계 표면으로 삼는다. - `update_in_place(slot,  $\Delta$ )` —append 가 아니라 slot 의 state 를 제자리 RMW($W += \Delta$). 불변식: 한 sequence 당 살아 있는 state 버전은 항상 1 개. G2 의 $256\times$ stale accretion 을 원천 차단하고, 소자 D 의 slot table 위에서 직접 실행된다. 비용: read+write = $2\times$ state_bytes(claim1 의 대칭 RMW). - `mark_`

`dirty(slot) / writeback(slot)` — 모든 TTT state 는 write 직후 항상 dirty 이므로 `mark_dirty` 는 update 의 부수효과로 자동이고, eviction 은 `writeback` 또는 결정론적 prefix-replay 재계산 중 짝 쪽을 **가격한다** (dirty 이므로 uncomputable 인 것은 아니다 — 상태는 초기값 · update rule · prefix 의 결정적 함수다, → § 23.4). G3 의 un-priced drop 을 명시적 비용으로 승격: `writeback` 비용은 상태 footprint 에 비례한다 (E1.5 의 416 MiB eviction set 에서 약 3.05 mJ / 130 μ s; NO-WALLCLOCK, A100 runbook 이월). 불변식: dirty slot 은 clean(writeback 또는 replay-checkpoint) 경유 없이 free 될 수 없다. - `checkpoint(seq) → tok / rollback(seq, tok)` — speculative decoding 의 rollback 이 optimizer-trajectory state(W_t 와 momentum S_t) 까지 되돌려야 한다 (→ 23 장). KV 의 단순 truncate 와 질적으로 다르다: `checkpoint` 는 (W, S) 쌍의 snapshot 을 쓰고 `rollback` 은 그 쌍을 복원한다. 비용: snapshot 당 state_bytes 사본 — `checkpoint` 빈도가 새 메모리 예산 축이다. - `bind_to_sequence(seq) → slot / unbind(slot)` — state 가 sequence 별 unshared 사본이므로 (content-addressed prefix 공유 불가, G1) 배치는 slot $\leftrightarrow$ seq bind 로 표현된다. 단 결정론적 동일 prefix 는 분기 전까지 같은 snapshot 에 도달하므로, bind 는 prefix-snapshot 공유 + 분기 시 copy-on-write 를 옵션으로 허용한다. § 24.4 제안 G 의 grouped-GEMM 이 이 bind 를 batch 축으로 묶는다. - `free_on_update(slot)` — retention gate α_t (→ 13 장) 는 상태를 제자리에서 감쇠시키는 **학습된 decay** 이지 slot 해제 신호가 아니다 (→ 1 장 Rosetta): $\alpha_t \rightarrow 0$ 이어도 활성 sequence 는 다음 token 의 update-retrieval 을 위해 같은 고정 크기 tensor 를 계속 쓴다. 따라서 이 API 가 하는 일은 즉시 dealloc 이 아니라, $\alpha_t \rightarrow 0$ 으로 정보를 비운 slot 을 **저비용 재구성 가능 (cold)** 으로 표시해 서빙 층 eviction 의 **비용 신호**로 삼는 것이다 (→ § 23.4 의 retention-gate 협력). 물리적 free 는 sequence 종료 · offload · checkpoint 정책 이 정하고, gate-triggered zero-page 회수는 별도 검증 대상이다. 불변식: 물리 free 는 sequence lifecycle 이 승인한 slot 에서만 일어난다.

이 스택은 § 24.3 의 memory-centric 소자 (state 가 어디 사는가) 와 § 24.4 의 accelerator kernel (state 를 어떻게 계산하는가) 사이의 접착층이며, claim6 의 semantic gap 을 설계 요구사항으로 직접 번역한 것이다.

24.6 배제: FORCED 방향 두가지

정직 판정 (→ 18 장 § 18.3, dossier § 5 verdict) 이 금지한 두 memory-centric 방향을 이 장은 명시적으로 배제한다. 배제 자체가 pair thesis 의 규율을 지키는 행위다.

배제 1 — “이 라인의 훈련이 새 메모리 소자를 요구한다.” 증거는 반대 방향을 가리킨다. 여섯 편이 연속으로 알고리즘을 **dense matmul** 로 다시 빚어 기존 accelerator 에 맞췄다: chunk anchoring, banded mask, batched NS-5, parallelism 을 위한 reset. TNT 는 plain JAX 로 stock TPU 위에서 FlashAttention 을 이겼다 (→ 15 장). training/prefill 은 tensor-core 영역이며 앞으로도 그렇다 — 여기서 device-level 필요성을 주장하는 것은 이 라인의 중심 엔지니어링 패턴과 모순된다. 훈련 쪽 제안은 그래서 § 24.4 의 accelerator kernel(F) 이지 새 소자가 아니다.

배제 2 — **일반 PIM 옹호**. per-token update 는 MLP 를 통한 gradient 계산으로 GEMV/GEMM 형태이지, 대부분의 PIM 제안을 정당화하는 sparse-irregular access 패턴이 아니다. PIM 형태인 것은 elementwise state-update epilogue (XPY · decay · renorm) 뿐이며, 그것은 FLOP 의 소수다 (§ 24.3.1). 따라서 PIM 은 일반 해법이 아니라 epilogue 국소의 directional 옵션으로만 등장한다.

[평가] 이 두 배제는 memory-centric 논증을 약화하는 것이 아니라 **신뢰 가능하게** 만든다. decode RMW 트래픽 (제안 D) 과 update-frequency $\leftrightarrow$ tier (제안 E) 라는 두 지점은 논문들 자신의 cost accounting 에 뿌리내렸기에 살아남고, 훈련용 새 소자와 일반 PIM 은 논문들 자신의 증거에 반하기에 배제된다. 종합 view 는 이 라인이 memory-centric **이면서** accelerator-centric 이라는 것이다 — 한 workload 의 두절반이기 때문이다.

요약

- 알고리즘 제안은 세 층이다. 두 CPU microbench 가 chunk C -roofline 곡선 (claim7) 과 on/off-die RMW cliff(claim8) 의 **모양**을 하드웨어 불문으로 고정했고, 이것이 제안 A(train/serve chunk-consistency grid) 와 제안 B(state-capacity scaling fit) 의 기대값이며, 두 실험의 vehicle 은하이브리드 4 층 HOPE/Sleep 재현 runbook(제안 C) 이다. E4 가공개 점 digitize 로 두 부호를 이미 예열했다 — capacity 는 ppl 이 아니라 **long-context/retention** 축에서만 예측력을얻고 (A3 를 좁힘), local-memory 수와 chunk mismatch U 자가 제안 A·B 의 낙하 방향을 고정한다. 여기에 제안 J(learned chunk/cadence schedule) 가 고정 schedule 을 학습 대상으로올려 § 24.3.2 의 tier 배치와 맞물린다.
- 하드웨어 memory-centric 절반은 정직 판정의 두 지점에서만 나온다: 제안 D(RMW-bandwidth decode 소자 — per-layer residency, per-tenant placement, near-memory epilogue engine; claim1/4/8) 와 제안 E(frequency-tiered placement — NL/Sleep cadence 의 tier 직역; claim5).
- accelerator 절반이 그 짝이다: 제안 F(fused chunk kernel — momentum 이깨뜨린 chunkwise closed form 의 미해결 커널), 제안 G(grouped-GEMM decode engine — per-request weights 가 깬 batching 복원), 제안 H(backward-capable serving kernel — decode 에 들어온 backward primitive).
- 소프트웨어 접착층은 제안 I(per-session-weights serving stack) 로, claim6 의 다섯 누락 API(update_in_place · mark_dirty/writeback · checkpoint/rollback · bind_to_sequence · free_on_update) 를설계 요구사항으로 번역한다.
- 두 FORCED 방향 — 훈련용 새 소자, 일반 PIM — 은 논문들 자신의증거 (알고리즘을 dense matmul 로 다시 빚음) 에 반하므로 명시적으로배제한다. 흥미로운 하드웨어 제안은 어느 한 쪽이 아니라 **쌍**이다.
- 모든 수치는 exploration-grade 계약 아래 읽는다: 비율 ·crossover·tier·bound 만 load-bearing, 절대 $\mu\text{s}/\text{mJ}$ 은 roofline 하한 (A100 runbook 이월), scratchpad/PIM 이득 배율은 directional.

자가 점검체크리스트

- ☐ 알고리즘 제안 A·B 가 각각 무엇을 falsify 하려 하고, 어느방향으로 떨어져도 왜 publishable 인지 설명할 수 있다.
- ☐ HOPE/Sleep 재현이 왜 하이브리드 4 층으로 강제되는지 (공식구현 0, Sleep 구현 0) 를 진술할 수 있다.
- ☐ memory-centric 제안 D·E 가 정직 판정의 어느 두 지점에근거하는지, 그리고 각 근거 claim 을 짚을 수 있다.
- ☐ accelerator 제안 F·G·H 가 memory-centric 제안과 경쟁이 아니라 pair 를 이루는 이유를 설명할 수 있다.
- ☐ claim6 의 다섯 누락 API 가 per-session-weights serving 설계로 어떻게번역되는지 대응시킬 수 있다.
- ☐ 배제되는 두 FORCED 방향과, 그 배제가 memory-centric 논증을 왜 오히려신뢰 가능하게 만드는지 말할 수 있다.
- ☐ E4 가 capacity 축을 ppl 이 아니라 long-context/retention 축으로 좁힌 근거 (family 안 rank-동치, attention corner 에서의 상관반전, Titans → Atlas 도약을 설명하는 capacity-class 변화) 를 설명할 수있다.
- ☐ near-memory update engine 의 세 스테이지 (state-tile 상주 버퍼, epilogue ALU lane, per-tenant slot table) 가 claim1/4/8 과 claim6 의 API 에 각각 어떻게대응하는지 말할 수 있다.
- ☐ 제안 J(learned schedule) 가 왜 제안 A 의 뒤에야 정직하게 돌릴 수 있는지, 그 두 falsifier 가 무엇인지 진술할 수 있다.
- ☐ never-average 원칙 (inner = request 별 map·no reduction, outer = batch 평균) 이 왜

- shared-weight 상각 불가의 미시근거이자 $B_{\max}$ 벽의 근원인지, 그리고 Self-Modifying Titans decode 의 workload-class 라벨 (request 별 weight recurrent MLP + batched optimizer; chunk 측은 prefill-전용 ·decode 엔 batch 측만; MoE 유비는 dispatch-form only 로 amortization 이득 없음) 이 제안 G·D 의 분업과 어떻게 엮이는지 설명할 수 있다.
- 어떤 제안 수치가 load-bearing 비율 ·crossover 이고 어떤것이 directional·이월 하한인지 판별할 수 있다.

다음 장으로

이 장은 pair thesis 의 두 절반을 각각의 제안 — 알고리즘 실험, memory-centric 소자, accelerator kernel, serving 소프트웨어 — 으로 내렸다. 25 장은 이 제안들을 하나의 research agenda 로 묶고, 이 라인이 열어둔 다섯 공백 (스케일, retrieval 격차, serving 경제학 ·kernel, 학습되는 스케줄, task-free·안전한 self-modification, → 18 장 § 18.2) 에 이 책의 기여가 어디에 놓이는지를 정리하며 스터디를 닫는다.

ch25. 결론과 연구어젠다

이 장의 목표 — 독자가 이 장을 마치면 (1) Part III 가 D4 pair thesis 를 측정으로 어디까지 정착시켰고 어디를 정직하게 열어 두었는지 한문단으로 진술할 수 있고, (2) 이 책이 이 라인에 새로 연 것 (배포형태로서의 완성형 + exploration-grade 실측 warrant + 쌍 (pair) 으로서의 하드웨어 제안) 과 남긴 것을 구별할 수 있으며, (3) 여섯 편의 open-questions 가 수렴한 다섯 공백을 systems 엔지니어가 이어받을 연구어젠다로 옮길 수 있어야 한다. **왜 필요한가** — 18 장이 완성형을 workload 로, workload 를 pair thesis 로, pair thesis 를 8 개 측정으로 내렸고, 19 – 24 장이 그 측정을 각 축의 깊이로 폈다. 이 장은 그 프로그램의 결산이다. 결산의 값은 무엇을 증명했는가만큼 무엇을 증명하지 못한 채 남겼는가에 있다 — Part III 의 정직성 계약 (§ 18.6) 은 결론에서 가장 엄격하게 지켜져야 한다.

25.1 Bridge-in: Part III 가 한일

이 책의 Part I 은 inference 엔지니어에게 훈련의 어휘를 (state, update, cost) 객체로 건네주었고, Part II 는 여섯 편을 하나의 연속된 프로그램으로 해부했으며, 그 프로그램의 완성형이 layer 가 아니라 **배포 형태** — 세션 상태가 mutable weights 인 continually-learning LLM 에 sleep 서비스가 붙은 시스템 (→ 18 장) — 임을 확인했다. Part III 는 그 배포 형태를 systems 엔지니어의 도구로 측정할 원저 기여였다.

측정의 척추는 **D4 workload-split pair thesis** 였다: 이 라인의 배포는 질적으로 다른 두 부하로 갈라지고, 각각의 최적 하드웨어 전략이 다르다. decode/serving-state 관리는 token 마다 fast-weight state 전체를 read-modify-write(RMW) 하는 memory-centric 부하이고, training/prefill 은 chunk 크기 C 가 roofline 의 x 축인 accelerator 부하이다. Part III 는 이 두 절반을 각각 정량화한 8 개 실험을 돌렸고, 세 방법 (단편 형태 cost model E3, analytic twin 도구 E1.x, CPU micro-benchmark E2.x) 이 anchor 설정 neural-mem-1.3B ($d=2048$, $m=16$, $L=24$, GQA-8, bf16) 에서 1% 이내로 일치했다. 이 장은 그 8 개 측정이 무엇을 정착시켰고 무엇을 열어두었는지부터 결산한다.

25.2 pair thesis 재확인: 측정이 정착시킨 것과 열어 둔 것

pair thesis 의 **decode 절반**은 측정으로 견고해졌다. anchor 에서 decode step 은 6.44 GB/token 을 움직이고 arithmetic intensity 가 0.59 FLOP/byte 로 H100 twin ridge(295 FLOP/byte) 보다 두 자릿수 아래에 있어 결정적으로 memory-bound 이며, state 트래픽이 GEMV 연산을 약 $394\times$ 압도한다 — step 비용이 곧 state 트래픽이다. 이 트래픽의 성격이 KV cache 와 갈라지는 지점이 명제의 핵심이다: KV 의 write 는 read 의 0.003%(append-once) 인데 TTT state 는 100%(대칭 RMW) 이고, KV read 가 문맥 길이에 비례해 자라는 반면 TTT RMW 는 문맥에 무관하게 일정 ($m d^2 L_{\text{layer}}$ 로 고정) 하다. 그래서 그 아래에서는 KV 가 싸고 그 위에서는 TTT 가 짙은 **crossover 문맥 길이**가 존재한다 — anchor 에서 read-crossover 약 65k token, 전체 RMW-crossover 약 131k token, 그리고 crossover 는 모델 폭에 따라 $m d^2$ 로 이동한다 (16k → 1.05M token). 이 명제의 세 따름 결과 — 배치를 키우면 용량보다 대역폭이 먼저 막힌다 ($\neq$ capacity-bound KV), state 상주여부가 문맥과 무관하므로 설계 가능한 knob 이 된다 (scratchpad fit →

spill, 폭 crossover $d^* \approx 2896$), update-frequency 연속체가 memory 계층 배치로 직접 번역된다(빠른 쪽 cadence 로 tier 결정) — 도 같은 강도로 측정되었다. 그리고 기존 KV-cache manager의 semantics는 RMW state에 대해 범주 오류다: content-addressed 재사용률이 구조적으로 0이고, append-only placement가 stale 상태를 T배 ($256 \times$)로 누적하며, dirty writeback을 0으로 값매긴다.

pair thesis의 **training/prefill** 절반도 측정으로 확인되었다. chunk $C=1$ (per-token RMW = decode 영역)은 $AI \approx 1$ 로 memory-bound 평원에 있고, C 를 키우면 crossover를 넘어 compute-bound로 오른다 — 한 알고리즘이 knob 하나로 두 영역을 오간다. on-die/off-die 경계에서 RMW 대역폭이 불연속으로 꺾이고 RMW가 read의 절반 throughput만 낸다는 것 — append-once KV가 피하는 **write-back 세 (稅)** — 도 직접 측정되었다.

[평가] 이 결산에서 정착된 것은 **비율 · crossover 위치 · tier 순서 · bound class** 뿐이다. 이것이 정직성 제약 (§ 18.6)의 실천적 의미다. “TTT decode는 폭이 클수록 · 문맥이 65k token을 넘을수록 KV 대비 상대적으로 유리해진다”는 load-bearing 주장이고 (비율 · crossover), “그 decode가 1.92 ms 걸린다”는 하한의 보고이지 주장이 아니다. 6편의 원 논문이 H100 decode wall-clock을 하나도 공개하지 않았으므로, 이 책이 낸 절대 $\mu\text{s}/\text{token} \cdot \text{mJ}/\text{token}$ 은 roofline **하한**이며 외부 검증 불가다 — 그 값들은 본문 주장의 근거가 아니라 사내 A100 runbook(Part III-a)으로 이월되는 검증 대상이다. scratchpad의 $6.8 \times$ energy 이득이나 PIM의 $1.6 \times$ energy 이득은 simulation_ready=False인 novel twin의 **directional DSE**이지 shipping-device 주장이 아니다. Part III가 정착시킨 것은 workload의 **구조**(memory-bound · RMW-지배 · context-무관)이지 그 구조가 실리콘에서 내는 절대 성능이 아니다.

이 유보는 pair thesis를 약화시키지 않는다. 오히려 명제의 형태를 정확히 한다: 이 책은 “TTT decode 소자를 지금 만들어라”고 말하지 않는다. 이 책은 **decode가 append-once KV와 질적으로 다른 memory 부하라는 사실**과, 그 다름이 만드는 crossover · tier · bound의 구조를 측정으로 확립하고, 그 구조가 실리콘에서 어느 절대값으로 나타나는지를 A100 runbook의 과제로 넘긴다. memory-centric 논증은 이 두 지점 — decode state RMW 트래픽 (§ 25.2 앞부분)과 update-frequency $\leftrightarrow$ memory 계층 배치 (claim 5) — 에서만 편다. “이 라인의 훈련이 새 메모리 소자를 요구한다”, “일반 PIM이 답이다”는 논문들의 자체 증거가 반대방향(알고리즘을 dense matmul로 다시 빚어 기존 accelerator에 맞춤)을 가리키므로 이 책은 주장하지 않았고, 결론에서도 주장하지 않는다.

25.3 이 책이 연 것

Part III의 기여는 세 겹이다.

첫째, 재구성 (reframing). 이 책은 여섯 편의 종착점을 layer의 진화가 아니라 하나의 serving workload로 읽었고, 그 workload가 반으로 갈라진다는 것을 명제로 세웠다. 이 재구성이 있기 전까지 이 라인은 inference 엔지니어에게 “또 하나의 sub-quadratic sequence model 후보”였다. 재구성 이후 이것은 **추론 중 weights는 변하지 않는다는 정의적 불변식의 폐기**(→ 1장 Rosetta의 “weight update 없음” 행)이고, 그 폐기가 만드는 새 memory 부하와 새 cache class의 문제다. pair thesis는 그 문제를 systems 관점에서 최초로 분할한 진술이다.

둘째, warrant. 6편이 decode 비용을 하나도 측정하지 않은 자리에, 이 책은 exploration-grade의 실측 warrant를 놓았다. 세 독립 방법이 anchor에서 1% 이내로 일치하는 교차 검증(RMW 6.44 GB/token, 상태 134 MB/layer, read-crossover 65k token, $B_{\max}$ 등)이 비율 · crossover · tier · bound를 본문으로 승격할 근거를 주었다. 이것은 “누구나 할 수 있는 산수”가 아니다 — 어느 crossover가 어디에 있는지, 어느 자원이 먼저 막히는지, 어느 KV-manager가 RMW에서 무너지는지는 계산되기 전까지 명제가 아니었다. 이 warrant는 그것들을 명제로 바꾸었다.

셋째, 쌍 (pair)으로서의 제안. 이 책의 하드웨어 제안은 어느 한쪽이 아니라 쌍이다(→ 24장). decode 쪽에는 RMW-bandwidth 상주소자 · frequency-tiered placement · per-tenant

state 배치를, training/prefill 쪽에는 fused chunk kernel·grouped-GEMM decode engine·backward-capable serving kernel 을 나란히 놓는다. 어느 한쪽만 옹호하는 것은 부하의 절반을 무시하는 것이다. 이 대칭이 이책이 “종합 view” 를 자처하는 근거이며, memory-centric thread 가 정직하게 load-bearing 인 두 지점과 accelerator thread 를 소홀히 하지 않는 균형을동시에 지키는 방식이다.

[해설] 세 기여의 관계는 위계적이다. 재구성이 없으면 warrant 는 측정할 대상이 없고, warrant 가 없으면 제안은 의견이며, 제안이한쪽뿐이면 재구성이 세운 pair thesis 를 배반한다. 이 책은 세 겹을 한 척추위에 세웠다 — 그 척추가 pair thesis 다.

이 세 겹은 저자의 career track 이나 소속과 독립적으로 서는 **학술기여**이며, 그 위치는 세 좌표로 특정된다. **장르**로 보면 이것은 새알고리즘 논문도 새 벤치마크도 아니라, 여섯 편의 1 차 문헌을 하나의 systems 명제로 읽고 그 명제를 exploration-grade 측정으로 검증한**분석적 종합 (analytic synthesis)**이다 — 6 편 중 어느 것도 자기 decode 를 systems 부하로 costed 하지 않았으므로 이 자리는 비어 있었다. **반증 가능성**으로 보면 pair thesis 의 load-bearing 부분 (비율·crossover·tier-bound) 은 A100 runbook 의 real-kernel head-to-head 로 반증 가능한 형태로 진술되어 있다 — 의견이 아니라 명제다. **수명**으로 보면 이 라인은 아직움직인다 ([Titans] 가 약속한 larger-model 논문은 미출간). 이 책의재구성과 warrant 는 7 번째 논문이 나오면 갱신되어야 하지만, “decode 는 append-once KV 와 질적으로 다른 memory 부하” 라는 구조적 진술은 특정모델의 수치가 아니라 update-rule 의 **RMW 대칭성**에 걸려 있으므로세대·스케일에 robust 하다. 이 책이 값매김한 것이 절대 성능이아니라 workload 의 구조인 이유가 여기서 다시 확인된다 — 구조는 소자보다오래 산다.

25.4 남긴 것: 연구어젠다

여섯 편의 open-questions 절이 전부 같은 다섯 공백으로 수렴했다는 것이저자 논지 (“Google 의 next가 완성형으로 보인다”) 의 힘이었다. 그 다섯공백은 이제 Part III 의 측정이 어디까지 갔는지가 확정된 만큼, systems 엔지니어가 이어받을 구체적 어젠다가 된다. 각 항목은 세 박자로 읽는다: 이 책이 **준 것** (어디까지 갔나) → 남은 **열린 질문** (구체 연구질문으로 좁힘) → **접근·falsifier·runbook 과제** (누가어떻게 밝나). 어젠다는 임의의 소망 목록이 아니라, 여섯 편이 스스로의 open-questions 에서 같은 곳을 가리켜 텍스트에 의해 과잉 결정된 자리다.

어젠다 1 — 스케일. 모든 품질 주장이 1.3B params / 100B tokens 에서 멈춰 있고 ([TNT] 는 150M), decode wall-clock 은 6 편전체에 부재하다. 이 책이 준 것은 두 겹이다. (i) RMW 트랙이 폭에따라 0.45 → 6.44 → 343 GB/token 으로, crossover 가 16k → 1.05M token 으로 자라는 스케일링을 twin 의 analytic 외삽으로 제시했고 (→ 19 장), (ii) 여섯 편이 흠어 놓은 (params, tokens, ppl/score) 점을 digitize 해 candidate scaling law 로맞춘 첫 시도를 넣었다 (E4).

그 fit 이 정착시킨 것은 exploration-grade 의 **순서뿐**이다 — rank-correlation·ordering 이 load-bearing 이고 절대 exponent·state-bytes 는 directional 이다. 결과의 핵심은하나의 정직한 역전이다: parametric family 안에서 state-bytes 와 capacity-ordinal 은 rank-등가이고 둘 다 ppl 과 거의 완전히반상관하지만 (Spearman $\rho \approx -0.95$), attention corner 를 더하면 capacity - ppl 상관이 **뒤집힌다** (unbounded capacity 인 attention 이 ppl 은 최악, Transformer++ 18.53). 그래서 state-capacity 는 **long-context/retention** 축이지 (BABILong retention 과 $\rho \approx 1.0$, Titans → Atlas 의 $6.7\times$ retention 도약을 $1.5\times$ state-byte 증가가 underpredict 하는 그 간극을 capacity-class 변화가 설명) **perplexity 축이 아니다** — ppl 은압축을 보상하고 retention 은 raw capacity 를 보상한다. 열린 질문은이제 셋으로 좁혀진다: (Q1) memory-bound RMW 라는 구조가 7B+·SFT/RLHF·production 데이터에서 유지되는가; (Q2) 공개 점들은 tokens 가 params 와 co-scale(15→30→100B) 하므로 3 점에서 순수 param-law 가 식별불가다 — N 과 D 를 분리한 scaling run 이 있어야 state-bytes 가 params·tokens 와 나란한 일급 축인지 판정된다; (Q3) capacity → retention rank-correlation 이 real training pipeline 과스케일에서 살아남는가. 접근: 동일 tokens 위에서 3–4 size $\times$ 3 arch class(matrix·deep-MLP·deep+featurized) 를 돌려 ppl 을 (params, state-bytes, capacity-proxy) 에 회귀하고 capacity

E4 — scaling-law digitization + fits (dossier A3): capacity is a long-context axis, not a perplexity axis

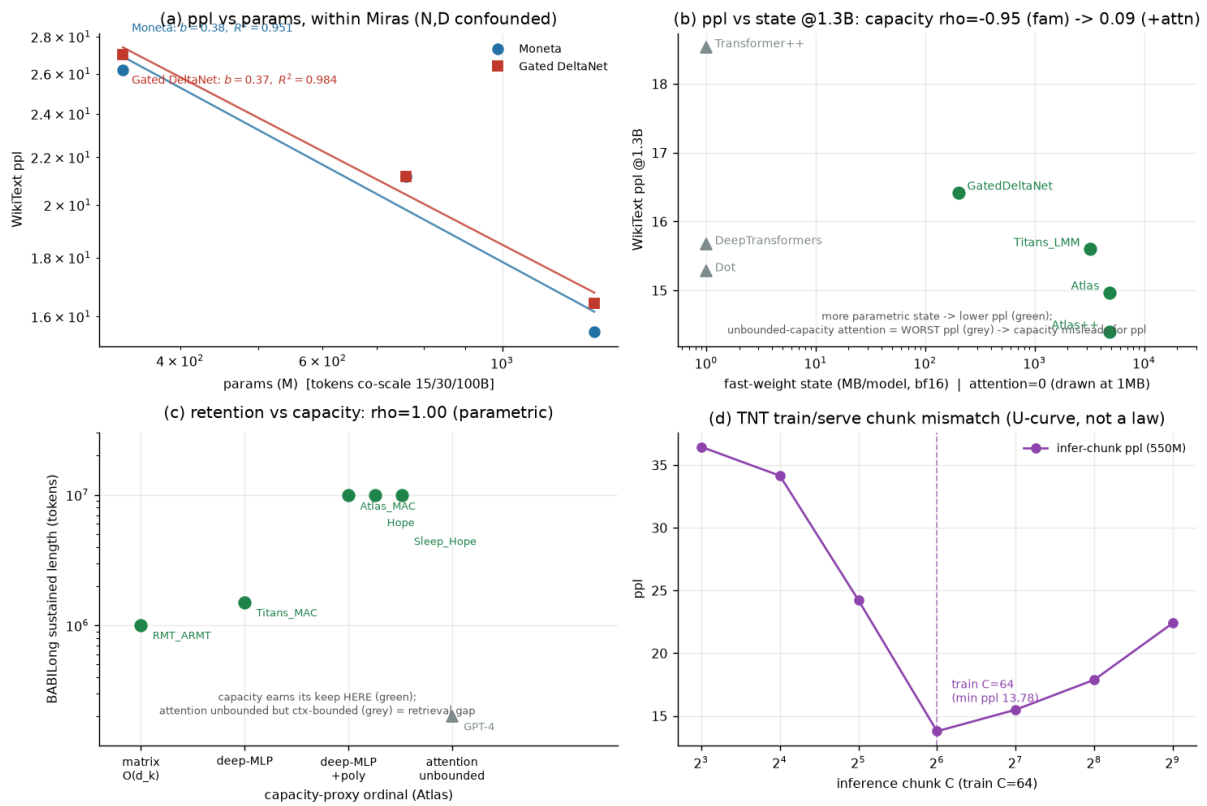


그림 25-1: 여섯 편의 (params, tokens, ppl/score) 점을 digitize 해 맞춘 candidate scaling fit; exploration-grade 의 ordering 만 load-bearing.

가 long-context 를 state-bytes 보다 잘 예측하는지 검증 (→ dossier A3). falsifier: tokenizer·data 가 이질적이면 (6 편이 실제로그렇다) exponent 가 불안정해 continuous law 로 못 세운다. runbook 과제: 각 스케일의 절대 wall-clock — 논문이 하나도 주지 않은 값.

어젠다 2 — retrieval 격차와 hybrid workload. attention 이 in-context recall 에서 여전히 이긴다: 53.55 vs 43.70 [Atlas Table 5], FDA 67.3 vs 41.9 [NL]. capacity 이론이 그 이유 (ϕ^* 의 무한 capacity, → 14 장) 까지 말해준다. 이 책이 준 것: E4 의 역전 (어젠다 1) 이 그 격차의 성격을 좁힌다 — attention 은 ppl 에서 지면서 recall 에서 이기므로, 격차는 일반 품질 측이 아니라 **retrieval/recall 에국소적인 capacity** 측이다. 이것은 문제를 오히려 다루기 쉽게 만든다: 완성형이 recall 만 attention 에서 빌려 오면 되지 압축 품질까지 포기할필요는 없다는 뜻이기 때문이다. 이 격차가 parametric 하게 닫히지 않으면완성형은 attention 과의 **hybrid** 다. 그리고 hybrid 는 pair thesis 를한 소자 위로 접는다: sliding-window KV(append-once·read-many) 와 RMW state(write-heavy·unshared) 가 공존하는 decode 다. 이 책의 $B_{\max}$ ·crossover 는 단일 workload 가정 위에서측정되었다 (carry-forward 6: weights+activations+KV 공존 시 $B_{\max}$ 는 상한). 열린 질문: 두 부하가 한 배포에 공존할 때 (Q1) crossover S^* 가 여전히 의미가 있는가 — window 크기 w 가 KV read 를 상수로 묶으면 crossover 는 w 의 함수로 이동하고 더는 문맥길이의 함수가 아니다; (Q2) hybrid decode 의 비용 모델은 무엇이며 HBM 대역폭 예산을 window-KV read 와 state-RMW 사이에 어떻게 나누는가; (Q3) recall 이 필요한 token 만 window-KV 로 라우팅하는 학습된 게이팅이 격차를닫으면서 RMW 예산을 아끼는가.

어젠다 3 — serving 경제학과 kernel. 이것이 이 라인이 아직기다리는 FlashAttention-moment 이다 (→ 21 장). per-request mutable weights 는 shared-weight batching 을 깨뜨리고 (grouped-GEMM decode), speculative decoding 의 rollback 에 optimizer-trajectory state 의 snapshot 을 요구하며, multi-tenant 격리와 weight 에 흡수된 context 의 privacy 를 새 문제로 만든다. 이 책이 준 것: batch 가 대역폭에 먼저막힌다는 것 (claim 3) 과, 기존 KV-manager 가 RMW state 에 대해 범주 오류라는것 (claim 6: update_in_place, mark_dirty/writeback, checkpoint/rollback, bind_to_sequence, free_on_update 가 KVCacheManager 에없음) 을 보였다. 열린 질문: (Q1) 그 다섯 event type 을 갖춘 **TTT-state manager** 의 실제 구현 — content-addressed 재사용률이구조적으로 0 이고 append-only 가 stale 을 T 배 누적하므로, 재사용이 아니라 in-place RMW 와 writeback 회계를 1 차 값으로 삼는 allocator 가 필요하다; (Q2) **backward-capable fused decode kernel** — decode 경로에 backward pass 가 들어온다는 이 라인의 정의적 신기성을 read·gradient·elementwise-update epilogue 를하나로 묶은 kernel 로 (현재 어떤 fused deep-memory decode kernel 도존재하지 않는다); (Q3) per-request weights 위의 grouped-GEMM decode 와, speculative rollback 을 위한 optimizer-trajectory state 의 numerics-drift 없는 snapshot. runbook 과제: 진짜 KV read kernel 과 진짜 TTT-state kernel 의 head-to-head 로 crossover 를 검증 (현재 S^* 는 roofline 반쪽대조).

어젠다 4 — 학습되는 스케줄. chunk 크기, window c , CMS frequency, level 수, sleep timing 이 전부 손으로 설정되어 있다. [NL] 은 잘못 놓인 level 이 품질을 해칠 수 있음을 보였고 (inner-q ablation), [Sleep] 은 sleep 을 chunk 경계에 하드와이어한다. 이책이 준 것: claim 5 는 cadence 가 주어지면 tier 가 결정적규칙 (resident copy 는 read/write cadence 중 빠른쪽으로 pin 된다) 으로 따라움을 보였다 — 그러나 cadence 자체를 학습하는것은 아무것도 없다. 여기서 두 정직한 memory-centric 지점이 교차한다: 잘못 놓인 level 은 이제 품질 비용만이 아니라 **잘못 놓인 tier** 라는 systems 비용이기도 하다 (HBM 에 있어야 할 것이 CXL 로 내려가면 critical path 를 막는다). 열린 질문: (Q1) memory 계층과 함께 co-design 되는스케줄 — 무엇을 언제 갱신할지 (f_ℓ) 가 무엇을 어디에돌지 (tier) 와 함께 학습되는 것; (Q2) 학습된 cadence 의 목적 함수는무엇인가 — 품질만이 아니라 tier-admissibility(claim 5 의 규칙) 를 제약으로넣은 품질/대역폭 co-objective; (Q3) cadence 가 입력에 따라동적이면 (data-dependent f_ℓ) placement 가 런타임에 바뀌어야하는가, 아니면 학습이 정적 tier 배치로 수렴하는가. 접근: level 별 cadence 를 differentiable relaxation 으로 두고 tier 비용을 penalty 로 주는작은 CMS 모델의 ablation.

어젠다 5 — task-free·안전한 self-modification, 그리고 미룬 이론. [Sleep] 의 Dreaming 은 명시적 (context, metric) 쌍을 요구하고, reward-model 의존은 검토되지 않았으며, 재귀적 weight self-editing 에는 안전성 분석이 없다. 배포 관점에서 이것은 **자기 weights** 를 세션마다 고쳐 쓰는 **serving fleet** 의 안전이다: weight 에 흡수된 context 의 privacy, per-tenant 발

산, rollback 과 provenance, 매 sleep 의 versioning(→ 23 장). 그리고 미룬 이론이 있다 — linear 특수 경우 너머의 regret·capacity·expressivity 결과가 없고, chunkwise staleness 의 오차 한계가 6 편 어디에도 없다(→ 9 장·15 장). 이 책이 준 것: 이 책도 그 이론 공백을 메우지 못했다 — C^* 의 모양은 측정했으나 ($C=1$ 은 AI ≈ 1 의 memory-bound, 큰 C 는 compute-bound, host 에서 $C^*\approx 32$) staleness 가 만드는 근사 오차의 한계는 측정하지 못했다. 열린 질문: (Q1) chunkwise stale-snapshot 근사의 오차를 C 와 memory 비선형성의 함수로 bound 하는 정리 — pair thesis 의 training 절반이실은 이 bound 위 에 서 있다 (큰 C 의 MFU 이득이 어느 오차 까지정당한가); (Q2) self-modifying serving 의 안전 프레임 — per-tenant 발산의 detection, weight-absorbed context 의 privacy 경계, 매 sleep 갱신의 provenance-rollback 을 갖춘 versioning; (Q3) reward-model 없는 task-free consolidation 이 가능한가. 이 어젠다는 유일 하게 measurable 부분조차 이 책이 밝지 못한 곳이다 — 나머지 넷과 달리 exploration-grade warrant 가 아니라 아직 열린 정리와 미설계 안전 계약이 남아 있다.

[평가] 이 다섯 어젠다는 임의로 고른 미래 과제가 아니다. 여섯 편이 스스로의 open-questions 에서 같은 곳을 가리켰기 때문에, 남은일은 텍스트에 의해 과잉 결정되어 있다. Part III 가 한 것은 그 다섯 중 serving 경제학의 절반 (어젠다 3 의 측정 가능한 부분) 을 exploration-grade 로 먼저 밝은 것이다. 나머지 — 스케일의 실증, 격차의 해소, 스케줄의 학습, 안전의 정식화 — 는 이 책이 연 재구성 위에서 다음 연구가 밝을 자리다.

25.5 닫는 말

여섯 편은 “test time 에 외우는 법을 배우자” ([Titans]) 로 열어 “모델은언제 깨어 있고 언제 자야 하는가” ([Sleep]) 로 닫혔고, 그 사이에 추론중 weights 불변이라는 transformer serving 의 정의 적 전제를 지웠다. 그지움이 만든 것이 이 책이 측정한 새 workload 다. 완성형은 개념적으로는 여섯 편의 텍스트만으로 도출되지만, 실증적으로는 1.3B/100B 에서 멈춰 있고 retrieval 격차가 열린 채이며 decode wall-clock 은 부재하다 — 이 세 유보가 이 책의 모든 정량 주장이 딛는바닥이다.

그래서 이 책은 완결이 아니라 on-ramp 이다. pair thesis 는명제이고, 8 개 측정은 그 명제를 비율·crossover·tier·bound 의수준에서 정착시킨 warrant 이며, 다섯 어젠다는 그 명제가 실리 콘과스케일에서 검증될 자리를 표시한 지도다. 이 라인이 attention 의 GEMM density 에 맞선 자기만의 FlashAttention-moment 을 맞을지, 아니면 sliding-window KV 와 RMW state 가 한 소자 위에 공존하는 hybrid 로안착할지는 아직 열려 있다. 어느 쪽이든, 그 결정은 layer 의 우아함 이아니라 decode 가 움직이는 바이트와 그것이 앉는 memory 계층에서난다 — 이 책이 그 자리를 inference 엔지니어의 도구로 처음 측정한 이유가거기에 있다.

요약

- Part III 는 D4 pair thesis 를 8 개 실험으로 측정했고, decode 절반 (memory-bound whole-state RMW, KV 와의 crossover, BW-먼저-막힘, 상주 knob, cadence → tier, KV-manager 범주 오류) 과 training/prefill 절반 (chunk C = roofline x 축, RMW cliff·write-back 세) 을 모두 정착시켰다.
- 정착된 것은비율·crossover·tier·bound 뿐이다. 절대 $\mu\text{s}/\text{token}\cdot\text{mJ}/\text{token}$ 은 roofline 하한이며 A100 runbook(Part III-a) 으로 이월되고, scratchpad·PIM 은 simulation_ready=False 의 directional DSE 다. memory-centric 논증은 decode RMW 트래픽과 update-frequency ↔ tier 두지점에서만 편다.
- 이 책이 연 것은 세 겹이다: 여섯 편의 종착점을 serving workload 로 읽는재구성, 6 편이 비운 자리에 놓은 exploration-grade warrant(세 방법 1% 이내 교차검증), 어느 한쪽이 아니라쌍으로서의 하드웨어 제안.
- 남긴 것은 여섯 편의 open-questions 가 수렴한 다섯 어젠다이며, 각각을 준것→열린 질문 →접근·falsifier 로 좁혔다: 스케일 (구조가 7B+에서 유지되는가; E4 가 digitize 한 점들에서 state-capacity 는 rank-correlation 으로 long-context/retention 축이지 perplexity 축이 아님 — attention corner 가 capacity - ppl 상관을뒤집는다), retrieval 격차와

hybrid workload(격차는 recall-국소 capacity 측; window-KV+RMW 공존의 비용 모델), serving 경제학·kernel(이 가족의 FlashAttention-moment; TTT-state manager·backward-capable fused decode kernel), 학습되는스케줄 (cadence↔tier co-design), task-free·안전한 self-modification 과 미룬 이론 (staleness 오차 한계 — 유일하게 이 책이 measurable 부분조차 밟지 못한 자리).

- 완성형은 개념적으로 도출되나 실증적으로 세 유보 (1.3B/100B 상한, 미해소 retrieval 격차 53.55 vs 43.70, 부재한 decode wall-clock) 위에 있다. 이 책은 완결이 아니라 on-ramp이며, 승부는 layer의 우아함이 아니라 decode가 움직이는 바이트와그것이 앓는 memory 계층에서 난다.

자가 점검체크리스트

- Part III가 pair thesis의 두 절반을 각각 어떤 측정으로 정착시켰는지나열할 수 있다.
- 어떤 양이 load-bearing(비율·crossover·tier·bound)이고어떤 양이 이월되는 하한 (절대 $\mu\text{s}/\text{mJ}$)·directional(novel twin) 인지 판별할 수 있다.
- 이 책이 연 세 겹 (재구성·warrant·pair 제안) 과그 위계 관계를 설명할 수 있다.
- 다섯 연구 어젠다 각각에 대해 이 책이 준 것과 남긴 것 (구체 연구질문·접근·falsifier)을 구별할 수 있다.
- E4가 왜 state-capacity를 long-context/retention 측으로만세우고 perplexity 측에서는 배제하는지 (attention corner의 capacity - ppl 역전) 설명할 수 있다.
- 세 겹의 기여를 career track과 무관한 학술 기여로 위치시키는 세좌표 (장르=분석적 종합, 반증 가능성, 구조가 소자보다 오래 산다는수명)를 진술할 수 있다.
- retrieval 격차가 닫히지 않을 때 완성형이 왜 hybrid가 되고, 그것이 pair thesis를 어떻게 한 소자 위로 접는지 설명할 수 있다.
- 완성형의 세 실증 유보를 짚고, 이 책을 완결이 아닌 on-ramp으로 위치시킬수 있다.

후미

참고문헌 (Bibliography)

범위 — 25 개 장 (Part I ch01–11, Part II ch12–17, Part III ch18–25) 전체를 grep 해 수집·dedupe 한 인용 목록이다. 각 항목끝의 [인용:] 은 그 문헌을 인용하는 장이다. **표기 규약** (STYLE § 2.5) — 6 편 주 논문은 공식 축약 **[Titans]·[Miras]·[Atlas]·[TNT]·[NL]·[Sleep]** 로, 외부 문헌은 저자·연도 + arXiv ID 로 인용한다. arXiv 번호는 본문에 등장한값을 그대로 쓰고, 본문에 저자·연도만 있던 항목은 이 라운드에서 web 으로 확인해 arXiv ID·저자·연도를 채웠다 (확인실패분은 § D 미해결). **검증 상태** — 6 편 메타데이터는 `notes/{id}.json(web-verified)` 에서, 외부 arXiv ID 는 각 장 본문에서, 저자·연도-only 항목 (§ C 일부) 과 P3 이월항목은 2026-07-12 WebSearch 로 교차확인했다. **2026-07-12 마감라운드** — 본문 25 장의 arXiv ID·GitHub 별수·venue·issue #번호를 `notes/impl-availability.md` 및 web 으로 spot-verify(in-text arXiv 토큰 71 종이 모두 § A/B 에 존재함을역방향 확인). 마지막 미해결 “Kim et al. 2026” 해소 (→ § B.9, § D). 그리고본문에 inline 으로만 인용돼 (arXiv ID 없이) grep 수집에서 누락됐던 고전 15 편 — Polyak 1964, Zinkevich 2003, Shalev-Shwartz 2011, McMahan 2011, Anderson 1972, Kohonen 1972, Hopfield 1982, Blleloch 1990, French 1999, McClelland et al. 1995, Williams 1992, Schmidhuber 1987/1992, Flash-Decoding 2023, Kung–Leiserson 1978 — 을역방향 확인해 § B 에 추가했다.

A. 주 논문 — Google Titans/neural-memory 6 부작

이 스터디의 척추. 모두 Ali Behrouz(제 1 저자 라인) + Vahab Mirrokni(Google) 계열. 저자·날짜는 `notes/*.json` 실측값.

- **[Titans]** — Behrouz, A., Zhong, P., & Mirrokni, V. (2025). Titans: Learning to Memorize at Test Time. arXiv:2501.00663 (v1: 2024-12-31). Google Research. [인용: ch01–ch14, ch16, ch17 — 라인 전체의 기준점]
- **[Miras]** — Behrouz, A., Razaviyayn, M., Zhong, P., & Mirrokni, V. (2025). It’s All Connected: A Journey Through Test-Time Memorization, Attentional Bias, Retention, and Online Optimization. arXiv:2504.13173. Google Research. (framework 명칭 **Miras** 로 통칭 — STYLE § 2.5) [인용: ch01–ch09, ch12–ch17]
- **[Atlas]** — Behrouz, A., Li, Z., Kacham, P., Daliri, M., Deng, Y., Zhong, P., Razaviyayn, M., & Mirrokni, V. (2025). Atlas: Learning to Optimally Memorize the Context at Test Time. arXiv:2505.23735. Google. [인용: ch01–ch10, ch13–ch17]
- **[TNT]** — Li, Z., Behrouz, A., Deng, Y., Zhong, P., Kacham, P., Karami, M., Razaviyayn, M., & Mirrokni, V. (2025). TNT: Improving Chunkwise Training for Test-Time Memorization. arXiv:2511.07343. USC / Google Research. [인용: ch01, ch02, ch04–ch10, ch14–ch17]
- **[NL]** — Behrouz, A., Razaviyayn, M., Zhong, P., & Mirrokni, V. (2025). Nested Learning: The Illusion of Deep Learning Architecture. arXiv:2512.24695. Google (Zhong: Columbia 겸직). NeurIPS 2025 버전 존재. [인용: ch01–ch11, ch14–ch17]

- **[Sleep]** — Behrouz, A., Hashemi, F., & Mirrokni, V. (2026). Language Models Need Sleep: Learning to Self-Modify and Consolidate Memories. arXiv:2606.03979 (v1: 2026-06; OpenReview 2025-09 공개). Google / Cornell. [인용: ch01, ch02, ch04, ch08 – ch11, ch14, ch16, ch17] △ 혼동 주의: 별개 계열 논문 arXiv:2605.26099(Lee et al., Do Language Models Need Sleep?, offline recurrence) 와 다름 —notes/impl-availability.md § 2.6.

B. 외부 문헌 —계보별

B.1 Linear attention · Fast Weight Programming (FWP)

- Katharopoulos, A., Vyas, A., Pappas, N., & Fleuret, F. (2020). Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. arXiv:2006.16236. ICML 2020. [인용: ch06]
- Schlag, I., Irie, K., & Schmidhuber, J. (2021). Linear Transformers Are Secretly Fast Weight Programmers. arXiv:2102.11174. ICML 2021. [인용: ch06]
- Irie, K., Schlag, I., Csordás, R., & Schmidhuber, J. (2021). Going Beyond Linear Transformers with Recurrent Fast Weight Programmers. arXiv:2106.06295. NeurIPS 2021. (본문 “Irie & Schmidhuber 2021” (ch06) 도 이 FWP 계보를가리킴) [인용: ch06]
- Irie, K., Schlag, I., Csordás, R., & Schmidhuber, J. (2022). A Modern Self-Referential Weight Matrix That Learns to Modify Itself. arXiv:2202.05780. ICML 2022. (self-modifying Titans 의 SRWM 계보) [인용: ch16]
- Irie, K., et al. (2023). Practical Computational Power of Linear Transformers and Their Recurrent and Self-Referential Extensions. arXiv:2310.16076. EMNLP 2023. (formal-language 인식 구성 — [NL Table 5]) [인용: ch16]
- Hua, W., Dai, Z., Liu, H., & Le, Q. (2022). Transformer Quality in Linear Time (FLASH). arXiv:2202.10447. ICML 2022. (mixed-chunk 기법 — chunkwise training 의 기원) [인용: ch09]

B.2 DeltaNet · Gated DeltaNet · state-tracking

- Yang, S., Wang, B., Zhang, Y., Shen, Y., & Kim, Y. (2024). Parallelizing Linear Transformers with the Delta Rule over Sequence Length. arXiv:2406.06484. NeurIPS 2024. (DeltaNet 병렬화; UT-transform 기반) [인용: ch06, ch09]
- Yang, S., Kautz, J., & Hatamizadeh, A. (2025). Gated Delta Networks: Improving Mamba2 with Delta Rule (Gated DeltaNet, 이하 GDN). arXiv:2412.06464. ICLR 2025. 공식 구현 NVlabs/GatedDeltaNet. **Kautz·Hatamizadeh = NVIDIA**, Yang = MIT. (P3 이월 “GDN NVIDIA affiliation” 확인 완료) [인용: ch06, ch21]
- Siems, J., et al. (2025). DeltaProduct: Improving State-Tracking in Linear RNNs via Householder Products. arXiv:2502.10297. [인용: ch06]
- Grazi, R., et al. (2025). Unlocking State-Tracking in Linear RNNs Through Negative Eigenvalues. arXiv:2411.12537. [인용: ch06]
- Merrill, W., Petty, J., & Sabharwal, A. (2024). The Illusion of State in State-Space Models. arXiv:2404.08819. ICML 2024. [인용: ch06]
- Peng, B., et al. (2025). RWKV-7 “Goose” with Expressive Dynamic State Evolution. arXiv:2503.14456. [인용: ch06]

B.3 SSM · Mamba 계보

- Gu, A., Dao, T., Ermon, S., Rudra, A., & Ré, C. (2020). HiPPO: Recurrent Memory with Optimal Polynomial Projections. arXiv:2008.07669. NeurIPS 2020. [인용: ch07]
- Gu, A., Goel, K., & Ré, C. (2022). Efficiently Modeling Long Sequences with Structured State Spaces (S4). arXiv:2111.00396. ICLR 2022. [인용: ch07]
- Smith, J. T. H., Warrington, A., & Linderman, S. W. (2023). Simplified State Space Layers for Sequence Modeling (S5). arXiv:2208.04933. ICLR 2023. [인용: ch07, ch09]
- Gu, A., & Dao, T. (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv:2312.00752. [인용: ch07]
- Dao, T., & Gu, A. (2024). Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality (Mamba-2, SSD). arXiv:2405.21060. ICML 2024. [인용: ch07]
- Yang, S., Wang, B., Shen, Y., Panda, R., & Kim, Y. (2024). Gated Linear Attention Transformers with Hardware-Efficient Training (GLA). arXiv:2312.06635. ICML 2024. [인용: ch06, ch09]
- Sun, Y., Dong, L., Huang, S., et al. (2023). Retentive Network: A Successor to Transformer for Large Language Models (RetNet). arXiv:2307.08621. [인용: ch06, ch09]
- Martin, E., & Cundy, C. (2018). Parallelizing Linear Recurrent Neural Nets Over Sequence Length. arXiv:1709.04057. ICLR 2018. [인용: ch07]
- Blelloch, G. E. (1990). Prefix Sums and Their Applications. Technical Report CMU-CS-90-190, Carnegie Mellon University. (associative scan 알고리즘의 표준 참조 — S5의 parallel scan) [인용: ch07]

B.4 Test-Time Training (TTT) · online-learning 관점의 sequence model

- Sun, Y., Wang, X., Liu, Z., Miller, J., Efros, A. A., & Hardt, M. (2020). Test-Time Training with Self-Supervision for Generalization under Distribution Shifts. arXiv:1909.13231. ICML 2020. (TTT 명칭의 직계 조상) [인용: ch08]
- Wang, D., Shelhamer, E., Liu, S., Olshausen, B., & Darrell, T. (2021). Tent: Fully Test-Time Adaptation by Entropy Minimization. arXiv:2006.10726. ICLR 2021. [인용: ch08]
- Sun, Y., Li, X., Dalal, K., et al. (2024). Learning to (Learn at Test Time): RNNs with Expressive Hidden States (TTT-Linear / TTT-MLP). arXiv:2407.04620. (dual form 의원형; 라인의 직계 원형) [인용: ch01, ch06, ch08, ch09, ch15]
- Liu, B., Wang, R., Wu, L., et al. (2025). Longhorn: State Space Models are Amortized Online Learners. arXiv:2407.14207. (implicit-GD inner optimizer) [인용: ch03, ch06]
- Dalal, K., et al. (2025). One-Minute Video Generation with Test-Time Training. arXiv:2504.05298. (TTT layer 의대규모 실용성) [인용: ch08]
- Wang, K. A., Shi, J., & Fox, E. B. (2025). Test-time regression: a unifying framework for designing sequence models with associative memory. arXiv:2501.12352. (동시기 통합 framework; 본문 “Wang et al. 2025”) [인용: ch13]
- Zhang, T., Bi, S., Hong, Y., et al. (2025). Test-Time Training Done Right (LaCT, Large-Chunk TTT). arXiv:2505.23884. (본문 “Zhang, Bi et al. 2025”) [인용: ch15]
- von Oswald, J., Scherrer, N., Kobayashi, S., et al. (2025). MesaNet: Sequence Modeling by Locally Optimal Test-Time Training. arXiv:2506.05233. (chunk_cg_solver — 2 차 방향 커널화 가능성의 실물) [인용: ch21]

- Guo, H., Yang, S., Goel, T., Xing, E. P., Dao, T., & Kim, Y. (2025). Log-Linear Attention. arXiv:2506.04761. [인용: ch15]

B.5 병렬화 · optimizer · in-context learning as gradient descent

- Andrychowicz, M., et al. (2016). Learning to learn by gradient descent by gradient descent. arXiv:1606.04474. NeurIPS 2016. (learned optimizer) [인용: ch04]
- Lim, Y. H., et al. (2024). Parallelizing non-linear sequential models over the sequence length. arXiv:2309.12252. ICLR 2024. (Newton 고정점 병렬화) [인용: ch15]
- Gonzalez, X., Warrington, A., Smith, J. T. H., & Linderman, S. W. (2024). Towards Scalable and Stable Parallelization of Nonlinear RNNs. arXiv:2407.19115. NeurIPS 2024. [인용: ch15]
- von Oswald, J., et al. (2023). Transformers learn in-context by gradient descent. arXiv:2212.07677. ICML 2023. [인용: ch04]
- Akyürek, E., Schuurmans, D., Andreas, J., Ma, T., & Zhou, D. (2023). What learning algorithm is in-context learning? Investigations with linear models. arXiv:2211.15661. ICLR 2023. [인용: ch04]
- von Oswald, J., et al. (2023). Uncovering mesa-optimization algorithms in Transformers. arXiv:2309.05858. [인용: ch04]
- Bietti, A., Cabannes, V., Bouchacourt, D., Jégou, H., & Bottou, L. (2023). Birth of a Transformer: A Memory Viewpoint. arXiv:2306.00802. NeurIPS 2023. [인용: ch05]

B.6 Optimizer · 훈련 기초 (Part I 배경)

- Robbins, H., & Monro, S. (1951). A Stochastic Approximation Method. Annals of Mathematical Statistics 22(3). (SGD 의 기원) [인용: ch02]
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. Nature 323. [인용: ch02]
- Polyak, B. T. (1964). Some methods of speeding up the convergence of iteration methods. USSR Computational Mathematics and Mathematical Physics 4(5). (heavy-ball momentum 의 기원) [인용: ch02]
- Sutskever, I., Martens, J., Dahl, G., & Hinton, G. (2013). On the importance of initialization and momentum in deep learning. ICML 2013. [인용: ch02]
- Duchi, J., Hazan, E., & Singer, Y. (2011). Adaptive Subgradient Methods for Online Learning and Stochastic Optimization (AdaGrad). JMLR 12. [인용: ch02]
- Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. arXiv:1412.6980. ICLR 2015. [인용: ch02]
- Loshchilov, I., & Hutter, F. (2019). Decoupled Weight Decay Regularization (AdamW). arXiv:1711.05101. ICLR 2019. [인용: ch02]
- Gupta, V., Koren, T., & Singer, Y. (2018). Shampoo: Preconditioned Stochastic Tensor Optimization. arXiv:1802.09568. ICML 2018. [인용: ch02]
- Jordan, K., et al. (2024). Muon: An optimizer for the hidden layers of neural networks. Blog: kellerjordan.github.io/posts/muon. (NS-5, $(a, b, c) = (3.4445, -4.7750, 2.0315)$) [인용: ch02, ch14]
- Liu, J., et al. (2025). Muon is Scalable for LLM Training. arXiv:2502.16982. [인용: ch02]
- Joffrain, T., Low, T. M., Quintana-Ortí, E. S., van de Geijn, R., & Van Zee, F. G. (2006). Accumulating Householder transformations, revisited (UT transform). ACM TOMS 32(2). [인용: ch09]
- Li, H., Xu, Z., Taylor, G., Studer, C., & Goldstein, T. (2018). Visualizing the Loss Landscape of Neural Nets. arXiv:1712.09913. NeurIPS 2018. [인용: ch03]

B.7 Online learning · meta-learning · bilevel

- Zinkevich, M. (2003). Online Convex Programming and Generalized Infinitesimal Gradient Ascent. ICML 2003. (OGD 와 $O(\sqrt{L})$ regret 정리) [인용: ch03]
- Shalev-Shwartz, S. (2011). Online Learning and Online Convex Optimization. Foundations and Trends in Machine Learning 4(2). (OCO 교과서적 정리; 본문의 “Shalev-Shwartz 2011” 과 “Shalev-Shwartz 2012, § 2.6” [ch03] 은 발행연도 표기만 다른 동일 monograph) [인용: ch03]
- McMahan, H. B. (2011). Follow-the-Regularized-Leader and Mirror Descent: Equivalence Theorems and L1 Regularization. AISTATS 2011. (FTRL $\leftrightarrow$ mirror descent) [인용: ch03]
- Hazan, E. (2019). Introduction to Online Convex Optimization (2nd ed.). arXiv:1909.05207. [인용: ch03]
- Orabona, F. (2019). A Modern Introduction to Online Learning. arXiv:1912.13213. [인용: ch03]
- Finn, C., Abbeel, P., & Levine, S. (2017). Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks (MAML). arXiv:1703.03400. ICML 2017. [인용: ch04]
- Franceschi, L., Frascioni, P., Salzo, S., Grazzi, R., & Pontil, M. (2018). Bilevel Programming for Hyperparameter Optimization and Meta-Learning. arXiv:1806.04910. ICML 2018. [인용: ch04]
- Hospedales, T., Antoniou, A., Micaelli, P., & Storkey, A. (2021). Meta-Learning in Neural Networks: A Survey. arXiv:2004.05439. IEEE TPAMI. [인용: ch04]

B.8 Associative memory · Hopfield

- Widrow, B., & Hoff, M. E. (1960). Adaptive switching circuits (LMS / delta rule). IRE WESCON Convention Record. [인용: ch05, ch06, ch08]
- Anderson, J. A. (1972). A simple neural network generating an interactive memory. Mathematical Biosciences 14(3–4). (correlation matrix memory 의 공동 기원) [인용: ch05]
- Kohonen, T. (1972). Correlation Matrix Memories. IEEE Transactions on Computers C-21(4). (outer-product 저장의 원형) [인용: ch05]
- Hopfield, J. J. (1982). Neural networks and physical systems with emergent collective computational abilities. Proceedings of the National Academy of Sciences (PNAS) 79(8). (energy 기반 auto-associative memory; Atlas/TNT 가 capacity 정식화 의뿌리로 인용) [인용: ch05]
- Krotov, D., & Hopfield, J. J. (2016). Dense Associative Memory for Pattern Recognition. arXiv:1606.01164. NeurIPS 2016. [인용: ch05]
- Ramsauer, H., et al. (2021). Hopfield Networks is All You Need. arXiv:2008.02217. ICLR 2021. [인용: ch05]
- Sukhbaatar, S., Grave, E., Bojanowski, P., & Joulin, A. (2019). Augmenting Self-attention with Persistent Memory. arXiv:1907.01470. [인용: ch12]

B.9 Continual learning · distillation · self-improvement

- McCloskey, M., & Cohen, N. J. (1989). Catastrophic Interference in Connectionist Networks. Psychology of Learning and Motivation 24. [인용: ch11]
- French, R. M. (1999). Catastrophic forgetting in connectionist networks. Trends in Cognitive Sciences 3(4). (CF·pseudo-rehearsal 계보 정리) [인용: ch11]
- McClelland, J. L., McNaughton, B. L., & O’ Reilly, R. C. (1995). Why there are complementary learning systems in the hippocampus and neocortex. Psychological Review 102(3). (CLS 이론 — Sleep 의 wake/sleep 분업의 신경과학 근거) [인용: ch11]

- Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning (REINFORCE). Machine Learning 8(3–4). (policy-gradient 항등식) [인용: ch11]
- Kirkpatrick, J., et al. (2017). Overcoming catastrophic forgetting in neural networks (EWC). arXiv:1612.00796. PNAS 114(13). [인용: ch11]
- Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the Knowledge in a Neural Network. arXiv:1503.02531. [인용: ch11]
- Kim, Y., & Rush, A. M. (2016). Sequence-Level Knowledge Distillation. arXiv:1606.07947. EMNLP 2016. [인용: ch17]
- Agarwal, R., et al. (2024). On-Policy Distillation of Language Models (GKD). arXiv:2306.13649. ICLR 2024. [인용: ch11]
- Kim, J., Luo, X., Kim, M., Lee, S., Kim, D., Jeon, J., Li, D., & Yang, Y. (2026). Why Does Self-Distillation (Sometimes) Degrade the Reasoning Capability of LLMs? arXiv:2603.24472 (2026-03-25). (본문 “Kim et al. 2026” — [Sleep App. A.4] 가 인용한 OPSD 실패 모드: epistemic verbalisation 억제 → 최대 40% OOD 하락) [인용: ch17]
- Singh, A., et al. (2024). Beyond Human Data: Scaling Self-Training for Problem-Solving with Language Models (ReST⁺EM). arXiv:2312.06585. TMLR 2024. [인용: ch11]
- Ouyang, L., et al. (2022). Training language models to follow instructions with human feedback (InstructGPT). arXiv:2203.02155. NeurIPS 2022. [인용: ch11]
- Zweiger, A., et al. (2025). Self-Adapting Language Models (SEAL). arXiv:2506.10943. (Dreaming 의 인접 계보) [인용: ch11]
- Hu, E. J., et al. (2022). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685. ICLR 2022. [인용: ch11]
- Lin, K., Snell, C., Wang, Y., et al. (2025). Sleep-time Compute: Beyond Inference Scaling at Test-time. arXiv:2504.13171. (텍스트-공간 offline compute — Sleep 과 대비) [인용: ch17]
- Eyuboglu, S., Ehrlich, R., Arora, S., et al. (2025). Cartridges: Lightweight and general-purpose long context representations via self-study. arXiv:2506.06266. (KV-공간 offline 압축 — Sleep 과 대비) [인용: ch17]

B.10 Systems · serving · scaling law

- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. arXiv:2205.14135. NeurIPS 2022. [인용: ch10, ch21]
- Dao, T. (2023). FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. arXiv:2307.08691. [인용: ch10]
- Dao, T., Haziza, D., Massa, F., & Sizov, G. (2023). Flash-Decoding for Long-Context Inference. PyTorch / Stanford CRFM 블로그. (FlashAttention 의 IO 재조직을 decode 까지 확장 — ch21 의 prefill/decode 대칭 논거) [인용: ch21]
- Kung, H. T., & Leiserson, C. E. (1978). Systolic Arrays (for VLSI). Sparse Matrix Proceedings 1978, SIAM. (systolic array = TPU MXU / GPU tensor core 의 조상; GEMM density 공진화 서술) [인용: ch21]
- Kwon, W., et al. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention (vLLM). arXiv:2309.06180. SOSP 2023. [인용: ch10]
- Hoffmann, J., et al. (2022). Training Compute-Optimal Large Language Models (Chinchilla). arXiv:2203.15556. NeurIPS 2022. [인용: ch19]
- Hooker, S. (2020). The Hardware Lottery. arXiv:2009.06489. [인용: ch21]
- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks (AlexNet). NeurIPS 2012. [인용: ch21]

B.11 교과서 · 고전참조

- Schmidhuber, J. (1987). Evolutionary Principles in Self-Referential Learning (diploma thesis). TU München. (학습 절차 자체를 학습 대상으로 삼는 self-referential learning) [인용: ch16]
 - Schmidhuber, J. (1992). Learning to control fast-weight memories: An alternative to dynamic recurrent networks. Neural Computation 4(1). (fast weight programming 의 원형) [인용: ch04, ch06, ch16]
 - Schmidhuber, J. (1993). A “self-referential” weight matrix. ICANN 1993. (self-modifying/self-referential 계보의 뿌리) [인용: ch04, ch06, ch16]
 - Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press. (backprop-optimizer 교과서 서술) [인용: ch02]
-

C. 구현 · 저장소 인용 (implementation citations)

Part III(ch21·ch22·ch24) 의 hardware-lottery / player-strategy / proposals 논증은 아래 저장소를 실측 증거로 인용한다. star-issue 번호는 `notes/impl-availability.md`(2026-07-11 GitHub API 실측) 기준.

- **lucidrains/titans-pytorch** (GitHub, 1,966 ★). 사실상의 reference 구현 (MAC variant + 저자 확장). AssocScan 기반병렬화. [인용: ch21, ch24]
 - **fla-org/flash-linear-attention** (FLA, 5,325 ★). DeltaNet/GDN/GLA/RWKV-7/Mamba-2 등의 production-grade Triton 커널. `fla/ops/ttt`(진짜 Triton) vs `fla/ops/titans`(naive PyTorch 만). RFC issue #107, 정제 #214. [인용: ch09, ch21, ch22, ch24]
 - **fla-org/flame** (torchtitan 기반 학습 프레임워크). [인용: ch24]
 - **NVlabs/GatedDeltaNet** (619 ★, ICLR 2025 공식 GDN 구현). [인용: ch21]
 - **obekt/HOPE-nested-learning** (84 ★) · **erikl2/nested-learning** (76 ★). HOPE 비공식 소규모구현 (참조 · 감사 대상, 신뢰 기반 아님; 둘 다 surrogate loss 혼합). [인용: ch24]
-

D. 미해결 인용 (unresolved)

- (없음 — 이 라운드에서 마지막 미해결 항목이던 “Kim et al. 2026” 이해소되어 § B.9 로 편입됨. 아래 “확인 완료” 참조.)

확인 완료 (과거 미해결 → 해소)

- **Kim et al. 2026** (직전 미해결) — 해소. [Sleep App. A.4] 가 OPSD 실패 모드로 인용한 “epistemic verbalisation 억제 → 최대 40% OOD 하락” 의 원전은 Jeonghye **Kim** et al. (2026), Why Does Self-Distillation (Sometimes) Degrade the Reasoning Capability of LLMs?, **arXiv:2603.24472** (2026-03-25; Qwen3-1.7B/8B·DeepSeek-Distill-Qwen-7B·Olmo3-7B-Instruct 에서 최대 40% 하락, “epistemic verbalization 억제” 기재 명시). 2026-07-12 WebSearch+arXiv 초록 대조로 저자 · 제목 · 수치 3 중 확인. § B.9 에 정식 등재. (Sleep 부록은 arXiv ID 없이 저자-연도만 표기했으나 외부 식별 가능했음.)
- **[GDN NVIDIA affiliation]** (P3 이월) — 해소. Gated DeltaNet(arXiv:2412.06464) 의 Jan Kautz·Ali Hatamizadeh 는 NVIDIA, Songlin Yang 은 MIT. 공식구현 NVlabs/GatedDeltaNet, ICLR 2025. (§ B.2)

용어집 색인 (Glossary Index)

구성 — STYLE § 2.4 concept ledger(공식 명칭 확정표, ~57 행) + § 1.2 예약 기호 전역표 + 각 장 첫 정의를 종합했다. 각 항목: 용어 (영어 원어) → 한 줄 정의 + 소유 장 (그개념을 정의하는 단일 소유권, STYLE § 2.3). 영어 원어 유지 원칙 (STYLE § 2.1) 에 따라 표제어는 로마자. 정렬 — § 2 는 알파벳순. § 1 은 inference 엔지니어 어휘와 의 Rosetta 대응 (STYLE § 3). § 3 은 예약 기호. 사용법 — “소유 장” 은 그 개념의 정의를 바꿀 수 있는 유일한장이다. 다른 장은 참조만 한다 (재정의 금지).

1. Rosetta 대응 (inference 어휘 ↔ 이 라인의어휘)

독자 (transformer inference 엔지니어) 의 모국어를 이 라인의 어휘로 옮기는교차 색인. 대응의 성격이 핵심 —동일/유비/차이가 논점을 구분하지 않으면 잘못된직관이 이식된다 (STYLE § 3). ch01 이 이 표를 본문으로 확장한다.

inference 세계 (독자 어휘)	이 라인의 어휘	대응의 성격	소유
KV cache	non-parametric memory state; softmax attention = capacity 무한 (ϕ^*) associative memory	동일 대상의 재서술 — attention 은 압축하지 않는 memory	ch01, ch06
KV cache append	memory write (outer-product Hebbian 이 최근접; delta rule 은 append 아닌 overwrite)	유비 + 차이	ch01, ch05
attention lookup ($q \cdot K$)	memory read $y_t = \mathcal{M}(q_t; W_t)$	동일 추상화 ($k \rightarrow v$ 연상)	ch01
linear-RNN state ($d \times d$)	matrix memory = 고정 크기 lossy 압축 memory	동일	ch06
prefill	큰 chunk 의 병렬 write (compression); TNT 에션 global memory	유비 (정확)	ch09, ch15

inference 세계 (독자 어휘)	이 라인의 어휘	대응의 성격	소유
decode	$C = 1$ 의 per-token online write + read — backward pass 가 decode 에 들어 온다	유비 (정확) + 신세계	ch09
FlashAttention tiling	chunkwise-parallel training 의 chunk	△차이가 논점: tiling 은 bit-exact, chunk 는 함수를 바꾸는 semantic 근사 (M4)	ch09
GEMM shape 감각	∇_{W^ℓ} 의 outer-product 구조: $dW = (\text{오차})k^\top$ 는 rank- C GEMM	동일 (이 책의 교수법)	ch01, ch10
scan / prefix-sum kernel	momentum 의 associative scan (S5 식), Π_t 의 누적 합	동일	ch07, ch09
roofline / arithmetic intensity	chunk 크기 C 가 arithmetic intensity 를 결정; 품질-최적 C (작음) vs MFU-최적 C (큼) 의 긴장	동일 도구, 새 독립 변수	ch10
batching (shared weights 전제)	per-request fast-weight state 가 shared-weight batching 을 깨뜨림 → grouped-GEMM decode	△차이가 논점	ch10, ch23
paged KV cache / session cache	per-session weight state — 새 cache class(sizing·checkpoint·eviction)	유비 → Part III	ch23
cache eviction policy	retention gate = 학습된 eviction	유비 (정확)	ch13
speculative decoding rollback	memory state snapshot/rollback (optimizer-trajectory 포함)	유비 + 미해결	ch23
optimizer state (경험 없음)	훈련의 register file / accumulator: m_t, h_t	도입용 유비	ch02
weight update 없음 (추론 불변식)	폐기되는 불변식 — 이 라인의 정의적 특징	차이 그 자체	ch01, ch08
sequence 축	훈련의 batch 축을 sequence 축이 대신 한다 (inner loop 의 mini-batch = chunk)	△ 최대 혼동 지점	ch02, ch09

inference 세계 (독자 어휘)	이 라인의 어휘	대응의 성격	소유
distributed training (경험 없음)	context parallelism: reset 이 sequential chain 을 끊어 shard 병렬화	도입용 유비	ch15
서빙 fleet 의 백그라운드 job	sleep phase = 주기적 offline consolidation/dreaming (“weights 의 background compaction”)	유비 → ch17, Part III	ch17

2. 용어집 (알파벳순)

A

- **arithmetic intensity** — FLOP/byte 비. 이 라인에선 chunk 크기 C 가 결정하는 새 독립 변수. (Rosetta: 독자의 모국어) — **ch10**
- **associative memory** — key → value 연상을 저장 · 회수하는 memory. 고전 정식화 + Miras 의 attentional-bias Def. (Rosetta: KV cache 의 read 추상화) — **ch05**(고전), **ch13**(Def. 정식화)
- **attentional bias** — inner objective ℓ 가 이루는 family; memory 가 무엇을 “잘 기억하려 하는가” 의 축. — **ch13**
- **Atlas** — windowed inner objective(Omega rule) + polynomial feature map + Muon optimizer 로 memory 를 확장한 논문/모델 (OmegaNet 계열). — **ch14** (arXiv:2505.23735)

B

- **backward pass** — outer loss 의 gradient 를 얻는 역전파. 이 라인에선 inner loop 의 backward 가 **decode 안으로** 들어온다. (Rosetta: 독자에게 없던 개념) — **ch02**
- **bilevel optimization** — inner(memory)·outer(parameter) 두 최적화가 중첩된 구조; meta-learning 의 형식. — **ch04**

C

- **catastrophic forgetting** — 새 데이터 학습이 이전 지식을 덮어쓰는 현상. — **ch11**
- **chunk 크기 C** — 병렬화 knob 이자 (TNT 이후) semantic knob. window 크기 c 와 반드시 구분. — **ch09**(명제화)
- **chunk 크기 = semantic hyperparameter** — C 는 스케줄이 아니라 계산되는 함수 자체를 바꾼다 (FlashAttention tiling 의 bit-exact 와 결정적 차이). — **ch09** (발견: TNT)
- **chunkwise-parallel training** — sequence 를 크기 C chunk 로 잘라 chunk 내부는 병렬 (GEMM-rich), chunk 사이는 순차/scan 으로 재조직하는 훈련 기법의 총칭 (축약 “chunk-wise training”). — **ch09**
- **chunk-size mismatch (train/serve)** — 훈련 chunk 크기와 서빙 chunk 크기가 달라 생기는 품질/효율 괴리. — **ch15**

- **complementary learning systems (CLS)** — 빠른 (해마)· 느린 (신피질) 두 학습계의 상보 이론; consolidation 의 신경과학 근거. — **ch11**
- **context parallelism** — periodic state reset 이 sequential chain 을 끊어 shard 단위 병렬화를 가능케 함 (TNT). — **ch15**
- **contextual memory** — 문맥 (입력 sequence) 에서 test-time 에 채워지는 memory. persistent memory 와 대비. — **ch12**
- **Continuum Memory System (CMS)** — level 별 update 주기 (frequency) 를 가진 FFN chain; 여러 시간척도의 memory 연속체 (식 (M5)). — **ch16**
- **crosstalk** — associative memory 에서 저장 패턴 간 간섭; capacity 한계의 원인. — **ch05**

D

- **deep memory** — memory 함수 $\mathcal{M}$ 가 (linear 가 아니라) 다층 MLP 인 것. 표준형 = 2-layer residual MLP(expansion 4, GELU). — **ch12**
- **delta rule** — $W \leftarrow W - \eta \nabla \ell$ 형태의 오차-기반 갱신 (Widrow-Hoff LMS). 모델 DeltaNet 과 구분 (rule vs 모델). — **ch05**
- **DeltaNet / Gated DeltaNet (GDN)** — delta rule 을 sequence layer 로 구현한 linear-memory 모델; GDN 은 여기에 retention gate 추가. — **ch06**
- **dual form (TTT)** — inner GD 갱신을 chunk 단위 matmul 로 재작성한 병렬 형태 (Sun et al.). 일반화는 ch09. — **ch08**(일반화 **ch09**)
- **Dreaming** — self-generated rollout 로 자기 update 를 개선하는 sleep 2 단계 중 self-improvement loop. — **ch17**
- **DGD / GGD / GM / Delta Momentum / DMGD / M3** — NL 이 도입한 inner-optimizer 변형군 (Deep/Gradient/...Gradient Descent 등). 첫 등장 시 정의. — **ch16**

F

- **fast weights** — inner loop 상태 = memory 상태 W_t . 표기규약상 책 전체에서 W . — **ch06** (Schmidhuber/FWP 계보)
- **fast weight programming (FWP)** — slow net 이 fast net 의 weights 를 만들어내는 (프로그래밍) 관점 (Schlag et al.). — **ch06**
- **FTRL viewpoint** — Follow-the-Regularized-Leader 로 memory 갱신을 보는 Miras 의 관점. FTRL 자체는 ch03 소유. — **ch13**(관점), **ch03**(FTRL)
- **forget gate** → **retention gate** 의 역사적 별칭. 첫 등장시 1 회 병기만 허용. — **ch13**

H

- **hardware lottery** — 알고리즘의 성공이 당대 하드웨어와의 궁합에 좌우된다는 명제 (Hooker 2020). — **ch21**
- **hierarchical memory (global/local)** — 큰 chunk 의 global module + 병렬 local module 로 나눈 TNT 의 memory 구조. — **ch15**
- **Hope / Hope-Attention** — self-modifying Titans + CMS 를 종합한 NL 의 대표 아키텍처. — **ch16**
- **Hopfield network** — energy 최소화 로 패턴을 저장·회상하는 고전 associative memory. — **ch05**

I

- **inner loop** — memory 상태 W_t 를 sequence 를 따라 갱신하는 test-time 최적화 (inner loss ℓ). — **ch01**(비형식), **ch04**(형식화)
- **inner loss ℓ / outer loss $\mathcal{L}$** — 소문자=inner(per-token memorization), 대문자=outer(task, next-token). 두 loop 의 시각적 구분. — **ch01/ch04**

K

- **knowledge distillation / GKD** — teacher 의 출력분포를 student 가 모사하도록 학습; GKD 는 on-policy 판본. — **ch11**
- **Knowledge Seeding (KS) / Self-Knowledge Seeding (SKS)** — 작은 self-memory 에 서 더 큰 network 로의 **upward distillation**(Sleep 의 consolidation). — **ch17**
- **knowledge-transfer taxonomy (5 mechanisms)** — NL 이 정리한 지식이전 5 가지 메커니즘 분류. — **ch16**

L

- **Learning – Retaining viewpoint** — memory 갱신을 “학습 항 + 유지항” 으로 분해하는 Miras 의 관점. — **ch13**
- **Learning to Imitate (LTI)** — Sleep 의 imitation-learning 항. △ ch07 의 LTI(linear time-invariant) 와 무관한 동음 약어 — **ch17 내부 전용**. — **ch17**
- **level / update frequency** — CMS 에서 각 memory 성분이 갱신되는 주기; 시간척도 계층. — **ch16** (Sleep 재사용)
- **linear attention** — softmax 없는 $W_t = W_{t-1} + v_t k_t^\top$ 형태; KV-cache 압축 관점. — **ch06**
- **Local Surprise Signal (LSS)** — $u_t = \nabla_{y_t} \mathcal{L}$, layer 출력에 대한 outer-loss gradient(NL). — **ch16**

M

- **MAC / MAG / MAL** — Memory as Context / Gate / Layer. Titans 의 memory 통합 3 방식. 첫 등장 시 풀네임 1 회 병기. — **ch12**
- **master update (M)** — 이 책의 기준 수식: $S_t = \beta_t S_{t-1} - \eta_t \nabla_W \ell$, $W_t = \alpha_t W_{t-1} + S_t$. 6 편 은 (M) 의 성분 교체로 서술된다. — **ch09/ch12**
- **memory capacity (formal)** — 유한 memory 가저장 · 회상할 수 있는 쌍의 수의 형식적 한계 (Atlas). 고전 (Hopfield) capacity 는 ch05. — **ch14**
- **meta-learned initial state** W_{init} — outer loop 가 학습하는 memory 초기값 (W_0); TNT 에서 load-bearing. — **ch04**(개념), **ch15**(역할)
- **meta-learning / MAML** — 학습 알고리즘 자체를 학습; MAML 은 초기값을 meta-학습. — **ch04**
- **momentary surprise** $-g_t^{\text{in}} = \nabla_W \ell(W_{t-1}; k_t, v_t)$, 현재 token 의 inner gradient. (놀라움 금지) — **ch12**
- **momentum-as-memory** — momentum buffer S_t 자체가 “과거 surprise 를 담은 memory” 라는 관점. — **ch02**(객체로서), **ch16**(정리로서)
- **Muon / Newton – Schulz** — semi-orthogonalization optimizer; NS- κ ($\kappa = 5$) 반복으로 특이값을 1 근방으로. — **ch02**(객체), **ch14**(inner 사용)

N

- **Nested Learning (NL)** — 아키텍처를 중첩된 다-level 최적화로 재해석한 논문; CMS·self-modifying·Hope 를 종합. — **ch16** (arXiv:2512.24695)
- **NSAM / Neural Learning Module** — NL 의 self-modifying memory· 학습 모듈 고유 명사. — **ch16**

O

- **Omega rule** — sliding-window 안의 여러 token 에 대한 inner objective(window gate $\gamma_{t,i}$ 포함); Atlas 의 핵심 (식 (M3)). “ Ω -rule” 표기 금지. — **ch14**

- **online vs offline consolidation** — 스트림 중 (online) vs 별도 sleep phase (offline) 의 memory 통합 구분. — **ch11**(개념), **ch16/ch17**(기법)
- **outer loop** — slow weights Θ 를 task loss $\mathcal{L}$ 로 학습하는 통상적 훈련. — **ch01/ch04**

P

- **parameter expansion (periodic (de)activation)** — sleep 중파라미터를 주기적으로 활성/비활성해 capacity 를 늘리는 Sleep 기법. — **ch17**
- **periodic state reset** — local memory 를 W_{init} 로 주기적으로 되돌려 chain 을 끊는 것 (TNT). — **ch15**
- **persistent memory** — 입력과 무관하게 학습되는 고정 memory token $P(\text{Titans})$. — **ch12**

Q

- **Q-K projection** $\Pi_t - \sum_{\tau} k_{\tau} k_{\tau}^{\top} / \|k_{\tau}\|^2$, TNT 의 누적 projection 행렬 (원문 $\mathcal{M}_t$ 개명). — **ch15**

R

- **regret / FTRL / OGD** — online learning 의 성능 척도 (comparator W^* 대비) 와 대표 알고리즘. — **ch03**
- **retention gate** α_t — memory 에서 남기는비율 ($\alpha_t W_{t-1}$, 1=전부 유지). 공식 용어 (forget gate 는별칭). Titans 와 방향 반대. — **ch13**
- **roofline / MFU** — 성능 상한모델 · model FLOPs utilization. 독자의 모국어 (정의 불필요, 표기만 통일). — **ch10**

S

- **self-modifying Titans** — memory 가 자기 update 규칙을 스스로생성 · 수정하는 구조 (SRWM 계보). — **ch16**
- **slow weights** Θ — outer loop parameter 전체 (projection·gate-producer·backbone). fast weights W 와 대문자 구분. — **ch06/ch16**
- **SSD (state-space duality)** — SSM 과 attention 의이중성 (Mamba-2); chunkwise 계산의 형식적 근거. — **ch07**
- **stale-snapshot 근사 (chunk-start anchor)** — chunk 안 모든 gradient 를 chunk 시작 상태 $W_{\xi(t,C)}$ 에서 평가하는근사 (“gradient anchor”). — **ch09**
- **surprise (momentary / past)** — momentary = g_t^{in} , past = S_t (momentum buffer). Titans 기원. — **ch12**
- **synaptic-pruning reset** — sleep 중 불필요 연결을 잘라내는 reset(Sleep). — **ch17**

T

- **test-time memorization** — Atlas 이후 용어: test-time 에 문맥을학습 (learning) 이 아니라 기억 (memorization) 한다는 구분 자체가논점. — **ch14**
- **test-time training (TTT)** — test 입력에 대해 보조 loss 로 weights 를 갱신하는 계열의 총칭 (“TTT 계열”). — **ch08**
- **Titans** — deep neural memory + momentum + retention 을 test-time 에 갱신하는 라인의 기준점 모델 (LMM). — **ch12** (arXiv:2501.00663)
- **two-stage training (train-big / serve-small)** — 큰 chunk 로 효율 훈련 후 작은 chunk 로 전환하는 TNT 의 훈련 경제학. — **ch15**

W

- **wake/sleep lifecycle** — 스트림 처리 (wake) 와 주기적 offline consolidation/dreaming(sleep) 을 오가는 lifecycle. — **ch17**
- **weight decay = per-token retention** — Titans 문맥에서 weight decay 를 매 token 의 retention(“forgetting mechanism”) 으로 해석. — **ch12**
- **window gate** $\gamma_{t,i}$ — Omega rule 의 in-context pruning gate(어느 과거 token 을 objective 에 넣을지). — **ch14**

3. 예약 기호 색인 (STYLE § 1.2 요약)

기호	의미	소유/정의
W_t	fast weights = memory 상태 (inner loop)	§ 1.2
Θ	slow weights 전체 (outer loop)	§ 1.2
W_{init}	meta-learn 된 초기 memory 상태 (W_0)	ch04/ch15
$\mathcal{M}(\cdot; W)$	memory module (함수). $\mathcal{M}^* = \text{argmin}$ 최적해 (read-only 의미 금지)	§ 1.2
$\ell(W; k, v) / \mathcal{L}$	inner(per-token) / outer(task) loss	§ 1.2
η_t / η	inner learning rate(gate) / outer lr(상수)	§ 1.2
β_t	inner momentum decay (gate)	§ 1.2
$\alpha_t \in [0, 1]$	retention gate (남기는 비율)	ch13
S_t	inner momentum buffer (past surprise)	ch12
g_t^{in}	momentary surprise $= \nabla_W \ell(W_{t-1}; k_t, v_t)$	ch12
u_t	Local Surprise Signal $= \nabla_{y_t} \mathcal{L}$	ch16
C / c	chunk 크기 (병렬화 · semantic knob) / Omega window 길이	ch09 / ch14
$\xi(t, C)$	chunk 시작 offset $= C \lfloor (t - 1) / C \rfloor$	ch15
$W^g, W^{l(i)}$	TNT global / local memory 상태	ch15
Π_t	TNT Q-K projection 행렬	ch15
$\gamma_{t,i}$	Omega rule window gate	ch14
δ_t	Huber threshold (learned) — layer 첨자 δ_ℓ (backprop 오차) 와 구분	ch13
NS_κ	Newton – Schulz κ 회 반복 ($\kappa = 5$ 표준)	ch14

기호	의미	소유/정의
ϕ_p, ϕ^*	polynomial / exponential feature map	ch14
$\theta^{(\ell)}, f_\ell, C^{(\ell)}, e_{i,\ell}$	CMS 의 level parameter· frequency·chunk· 누적 error	ch16/ch17
m_t, h_t	outer 1 차/2 차 moment (Adam; v_t 는 value 와 충돌해 h_t)	ch02
λ	outer weight decay	§ 1.2
$L / L_{\text{layer}} / L_{\mathcal{M}}$	sequence 길이 / 네트워크 깊이 / memory MLP 깊이	§ 1.2 (v1.1)
$L_s^{(i)}$	TNT shard 길이 (local reset 주기; 원문 S_{L_i} 개명)	ch15
$\rho, \lambda_{\text{on}}, \lambda_{\text{KD}}$	Sleep 의 reward 혼합 / GKD on-policy 비율 / distill-vs-reward 계수 (원문 γ, λ, α 개명)	ch17

inference 어휘 Rosetta(재수록): $W_t \approx$ per-session cache state, $\nabla_W \ell$ 의 outer-product $\approx$ rank- C GEMM, S_t associative scan $\approx$ prefix-sum kernel, retention gate $\alpha_t \approx$ 학습된 cache eviction, chunk $C \approx$ arithmetic-intensity knob($\neq$ bit-exact tiling). 상세는 § 1.

부록 E. 실험환경·재현·신뢰성증명

이 부록의 목적 — Part III의 “저자 자체 실험” 8개가무엇으로 어디서 어떻게 나왔는지를 남이 그대로 재현할 수 있게 못박고, 각 숫자가 어느 등급의 증거인지를 숨김없이 등급화한다. 정직성 계약 (§ 18.6, § 25.2)이 무엇을 주장하는지를 말했다면, 이부록은 그 계약을 어떻게 검증하는지를 운영 수준으로 적는다. 기계가읽는 판본은 저장소 experiments/REPRODUCE.md + experiments/results.json(per-number provenance)이다.

먼저 결론. 이 실험들은 **silicon 측정이 아니다**. 실제 GPU wall-clock은 한 줄도 없다 — 실행 host에 NVIDIA GPU가 없다. 신뢰성의근거는 “쫓다”가 아니라 네 가지다: (1) 결정론적 재현, (2) 3중교차검증 (<1%), (3) host CPU 실측으로 **roofline 모양 확인**, (4) 모든 숫자의 정직한 등급화 + 반증 목록. 아래가 그 넷을 각각증명한다.

E.1 무엇을 신뢰하고 무엇을 신뢰하지 말 것인가

신뢰해도 되는 것 (load-bearing, 본문 승격)	신뢰하면 안 되는 것 (이월/directional)
비율 (write/read %, traffic 대 compute 배수)	절대 $\mu\text{s}/\text{token} \cdot \text{mJ}/\text{token}$ (= roofline 하한)
crossover 위치 (S^* , C^* , d^* , B_{max})	novel twin(scratchpad/PIM) 절대치 (simulation_ready=False)
tier 순서 (cadence $\rightarrow$ HBM/CXL/scratchpad 배정)	E1.2 whole-model spill 절대 MB (directional sim)
bound 분류 (memory-bound / compute-bound)	E2.x host GB/s·GFLOPS 절대치 (모양만 이송)

본서 본문은 왼쪽 열만 단정문으로 쓰고, 오른쪽 열은 전부 “하한/방향성/사내 A100 runbook 이월”로 표기한다.

E.2 실행 환경지문

항목	값
repo commit	neural-memory-study @ d709014
Python	3.14.3 (venv ./venv)
numpy / scipy / matplotlib	2.5.1 / 1.18.0 / 3.11.0
hatir (cost model)	editable ~/repos/hatir @ 10aa2a2
hat_schema (device twin SoT)	editable ~/repos/hat-schema @ c8b8913
device twin	hat-schema/twins/dgx_h100_x4.json (sha256 5cb5fe1e...)

항목	값
host	AMD Ryzen 7 255, 16 core, 27 GiB RAM, GPU 없음

device twin 이 무엇인가. `dgx_h100_x4.json` 은 `hat-schema` 가 SoT 로 관리하는 **pre-silicon** 장치 모델이다 — DGX-H100($\times 4$) 의 HBM3 대역폭 · 용량 · 에너지 계수와 compute roofline(ridge 295 FLOP/B) 을 기술한 JSON. E1.x 는 이 twin 위에서 `hatir` 로 roofline 을 계산 하고, E3 는 같은 twin 의상수를 달한 형태 공식에 넣는다. 즉 “H100 에서 잤다” 가 아니라 “H100 twin 의 물리로 roofline 하한을 계산했다” 가 정확한 표현이다.

E.3 재현 절차

```
cd ~/repos/neural-memory-study
PY=.venv/bin/python
export PYTHONHASHSEED=0          # analytical 실험 bit-deterministic 고정 (E.4)

# 8개 실험 실행 (각 experiments/<exp>/run.py)
for d in E1.1-decode-baseline E1.2-state-placement E1.3-kv-vs-ttt \
        E1.4-frequency-tiers E1.5-kvmgr-gap E2.1-chunk-intensity \
        E2.2-rmw-cliff E3-analytical E4-scaling ; do
    ( cd experiments/$d && $PY run.py )
done

# 통합 warrant + 그림 재생성
( cd experiments && $PY consolidate.py )    # -> results.json
$PY figures/render_experiments.py          # -> figures/exp-*.png
```

E.4 결론론 · 재현성 증명 (실측)

8 개를 재실행하고 저장본과 bit-diff 한 결과 — 재현성의 패턴이 정확히 실험의 성격과 일치한다:

실험	종류	재실행 시
E1.1, E1.3, E1.4, E1.5, E3, E4	HATIR / closed-form (analytical)	bit-identical
E1.2 (crossover d^* , energy/time)	HATIR analytical	bit-identical
E1.2 (whole-model spill)	HATIR spill sim	seed 고정으로 bit-identical (아래)
E2.1, E2.2	host CPU 실측	run 마다 변동 (정상)

E2.x 가 흔들리는 건 결함이 아니라 정직성의 증거다 — host CPU 에서 실제시간을 재드로 당연히 jitter 가 있다. 그래서 본서는 이들의 절대 GB/s·GFLOPS 를 이송하지 않고 “곡선모양” (AI(C) 상승, RMW 대역폭 절벽) 만 본문에 올린다.

유일하게 손봐야 했던 비결정성. E1.2 의 whole-model spill sim 은 `hatir.live_set_spill_bytes(policy="largest")` 를쓰는데, 그 함수의 eviction 대상 선택이 set 순회 순서 (문자열 hash 랜덤화 PYTHONHASHSEED 의존) 로 tie-break 되어 run 마다 spill 이 요동친다. PYTHONHASHSEED=0 고정으로 2 회 재실행이 완전히동일해짐을 확인했다. 이 필드는 load-bearing 이 아니며 (본서는 crossover d^* 와 이득 배율만 인용, spill 절대 MB 는 인용 안 함), 그럼에도 재현 절차에 seed 를 못 박아 bit-deterministic 으로 고정했다.

E.5 교차검증 — 신뢰성의 중심 기둥 (3중 <1% 일치)

silicon 측정이 없으므로, 신뢰성은 “서로 독립인 계산 경로가 같은 답에수렴하는가” 로 세운다. Anchor neural-mem-1.3B (d=2048, m=16, L=24, GQA-8, bf16) 에서 세 방법이 1% 이내로 일치:

양	E3 (closed-form)	hatir (E1.1/E1.3)	plan (hand-calc)
RMW GB/token	6.442	6.4425	6.4
RMW ms/token	1.923	1.9231	1.9
S*_read (tokens)	65536	65471	65000
state MB/layer	134.2	134.218	134
B_max @10ms	5.2	5.2	—

이 교차검증이 증명하는 것과 못 하는 것. 증명함 — 세 코드 경로어디에도 구현 버그가 없다 (같은 roofline 물리를 서로 다른 방식으로 계산해같은 값). 못 함 — twin 의 HBM BW·에너지 계수가 실제 H100 silicon 과 얼마나 맞는지는 이 일치로 증명되지 않는다 (E3 와 hatir 는 같은 twin BW 를 입력으로 공유). 그 fidelity gap 이 바로 절대치를 “roofline 하한 → A100 runbook 이월” 로 미루는 이유다. 즉 교차검증은 “우리 계산이 옳다” 를 증명하고, twin fidelity 는 별도로 A100 실측으로 메운다. (C*는 ridge 의존이라 값이 아니라 모양만 이송: host CPU C* $\approx$ 32 vs H100 twin C* $\approx$ 337, ridge $\sim 9\times$ 차이.)

E.6 명시적으로 주장하지 않는 것 — A100 runbook 이월 (=반증목록)

runbook/hope-reproduction-a100.md 로넨긴 6 가지. 이 목록이 있다는 것 자체가 신뢰성의 일부다 — 무엇이 아직증명 안 됐는지를 본서가 먼저 밝힌다.

1. 절대 per-token decode wall-clock(ms/token, tok/s) — 원 논문 6 편도 하나도 안 냄.
2. RMW step 의 achieved-vs-roofline 효율 (kernel-launch/scheduling/tail).
3. 실제 r/w 비대칭 DRAM 에너지/token (twin 은대칭 7 pJ/B 가정).
4. S* 검증 — 진짜 KV 커널 vs 진짜 TTT-state 커널 head-to-head.
5. novel twin(scratchpad/PIM) 절대치 — silicon 전까지 directional.
6. 전체 serving 스택 (weights+activations+KV 공존) 하의 B_max — E3 값은상한.